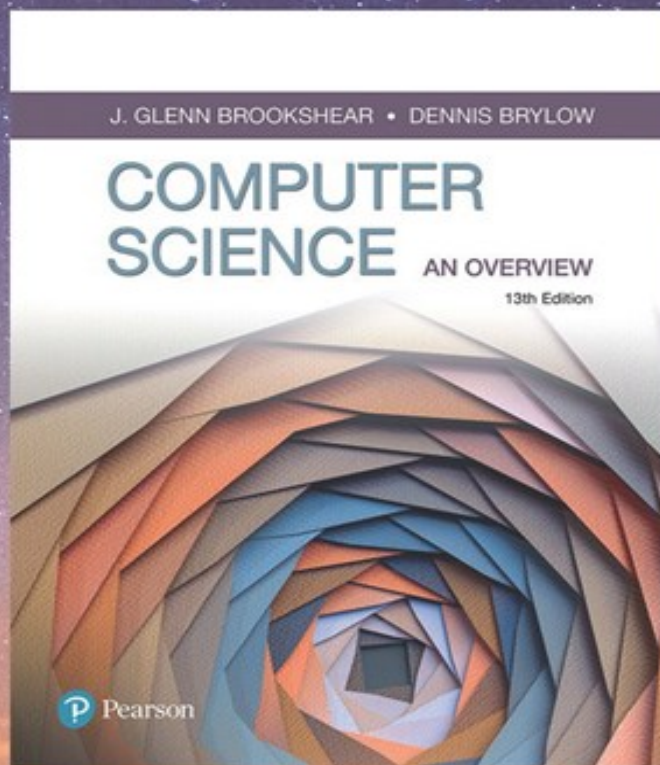


计算机科学概论 (第13版)

[美] J. 格伦·布鲁克希尔 (J. Glenn Brookshear) 丹尼斯·布里罗 (Dennis Brylow) 著

刘艺 吴英 毛倩倩 译



目 录

[版权信息](#)

[版 权](#)

[内容提要](#)

[版权声明](#)

[前 言](#)

[本书与AP®计算机科学原理课程框架之间的关联](#)

[资源与服务](#)

[第0章 绪 论](#)

[持久理解和学习成果](#)

[0.1 算法的作用](#)

[0.2 计算的历史](#)

[0.3 学习大纲](#)

[0.4 计算机科学的首要主题](#)

[0.4.1 算法](#)

[0.4.2 抽象](#)

[0.4.3 创新](#)

[0.4.4 数据](#)

[0.4.5 程序设计](#)

[0.4.6 因特网](#)

[0.4.7 影响](#)

[社会问题](#)

[课外阅读](#)

[第1章 数据存储](#)

[持久理解和学习成果](#)

[1.1 位和位存储](#)

[1.1.1 布尔运算](#)

[1.1.2 门和触发器](#)

[1.1.3 十六进制记数法](#)

[1.2 主存储器](#)

[1.2.1 存储器结构](#)

[1.2.2 存储器容量的度量](#)

[1.3 海量存储器](#)

[1.3.1 磁系统](#)

[1.3.2 光系统](#)

[1.3.3 闪存驱动器](#)

[1.4 用位模式表示信息](#)

[1.4.1 文本的表示](#)

[1.4.2 数值的表示](#)

[1.4.3 图像的表示](#)

[1.4.4 声音的表示](#)

[*1.5 二进制系统](#)

[1.5.1 二进制记数法](#)

[1.5.2 二进制加法](#)

[1.5.3 二进制中的分数](#)

[*1.6 整数的存储](#)

[1.6.1 二进制补码记数法](#)

[1.6.2 二进制补码记数法中的加法](#)

[1.6.3 溢出问题](#)

[1.6.4 余码记数法](#)

[*1.7 分数的存储](#)

[1.7.1 浮点记数法](#)

[1.7.2 截断误差](#)

[*1.8 数据与程序设计](#)

[1.8.1 Python入门](#)

[1.8.2 你好，Python](#)

[1.8.3 变量](#)

[1.8.4 运算符和表达式](#)

[1.8.5 货币转换](#)

[1.8.6 调试](#)

[*1.9 数据压缩](#)

[1.9.1 通用的数据压缩技术](#)

[1.9.2 图像压缩](#)

[1.9.3 音频和视频压缩](#)

[*1.10 通信差错](#)

[1.10.1 奇偶校验位](#)

[1.10.2 纠错码](#)

[复习题](#)

[社会问题](#)

[课外阅读](#)

[第2章 数据操控](#)

[持久理解和学习成果](#)

[2.1 计算机体系结构](#)

[2.1.1 CPU基础知识](#)

[2.1.2 存储程序概念](#)

[2.2 机器语言](#)

[2.2.1 指令系统](#)

[2.2.2 数据传输类](#)

[2.2.3 算术/逻辑类](#)

2.2.4 控制类

2.2.5 Vole：一种演示用的机器语言

2.3 程序执行

2.3.1 程序执行的一个例子

2.3.2 程序与数据

*2.4 算术/逻辑指令

2.4.1 逻辑运算

2.4.2 循环移位运算及移位运算

2.4.3 算术运算

*2.5 与其他设备通信

2.5.1 控制器的作用

2.5.2 直接存储器存取

2.5.3 握手

2.5.4 流行的通信媒介

2.5.5 通信速率

*2.6 数据操控编程

2.6.1 逻辑运算和移位运算

2.6.2 控制结构

2.6.3 函数

2.6.4 输入和输出

2.6.5 马拉松训练助手

*2.7 其他体系结构

2.7.1 流水线

2.7.2 多处理器机器

复习题

社会问题

课外阅读

第3章 操作系统

持久理解和学习成果

3.1 操作系统的历史

3.2 操作系统的体系结构

3.2.1 软件概述

3.2.2 操作系统组件

3.2.3 系统启动

3.3 协调机器的活动

3.3.1 进程的概念

3.3.2 进程管理

*3.4 处理进程间的竞争

3.4.1 信号量

3.4.2 死锁

3.5 安全性

3.5.1 来自外部的攻击

[3.5.2 来自内部的攻击](#)

[复习题](#)

[社会问题](#)

[课外阅读](#)

[第4章 组网及因特网](#)

[持久理解和学习成果](#)

[4.1 网络基础](#)

[4.1.1 网络分类](#)

[4.1.2 协议](#)

[4.1.3 组合网络](#)

[4.1.4 进程间通信的方法](#)

[4.1.5 分布式系统](#)

[4.2 因特网](#)

[4.2.1 因特网体系结构](#)

[4.2.2 因特网编址](#)

[4.2.3 因特网应用](#)

[4.3 万维网](#)

[4.3.1 万维网实现](#)

[4.3.2 HTML](#)

[4.3.3 XML](#)

[4.3.4 客户端活动和服务器端活动](#)

*4.4 因特网协议

4.4.1 因特网软件的分层方法

4.4.2 TCP/IP协议簇

*4.5 简单的客户机服务器

超越无限

4.6 网络安全

4.6.1 攻击的形式

4.6.2 防护和对策

4.6.3 密码学

4.6.4 网络安全的法律途径

复习题

社会问题

课外阅读

第5章 算法

持久理解和学习成果

5.1 算法的概念

5.1.1 非正式的回顾

5.1.2 算法的正式定义

5.1.3 算法的抽象本质

5.2 算法的表示

5.2.1 原语

5.2.2 伪代码

5.3 算法的发现

5.3.1 问题求解的艺术

5.3.2 迈出第一步

5.4 迭代结构

5.4.1 顺序搜索算法

5.4.2 循环控制

5.4.3 插入排序算法

5.5 递归结构

5.5.1 二分搜索算法

5.5.2 递归控制

5.6 效率和正确性

5.6.1 算法效率

5.6.2 软件验证

复习题

社会问题

课外阅读

第6章 程序设计语言

持久理解和学习成果

6.1 历史回顾

6.1.1 早期程序设计语言

[6.1.2 机器无关和超越机器无关](#)

[6.1.3 程序设计范型](#)

[6.2 传统的程序设计概念](#)

[6.2.1 变量和数据类型](#)

[6.2.2 数据结构](#)

[6.2.3 常量和字面量](#)

[6.2.4 赋值语句](#)

[6.2.5 控制语句](#)

[6.2.6 注释](#)

[6.3 过程单元](#)

[6.3.1 函数](#)

[6.3.2 参数](#)

[6.3.3 有返回值的函数](#)

[6.4 语言实现](#)

[6.4.1 翻译过程](#)

[6.4.2 软件开发包](#)

[6.5 面向对象程序设计](#)

[6.5.1 类和对象](#)

[6.5.2 构造器](#)

[6.5.3 附加特性](#)

[*6.6 程序设计并发活动](#)

[*6.7 说明性程序设计](#)

[6.7.1 逻辑推演](#)

[6.7.2 Prolog](#)

[复习题](#)

[社会问题](#)

[课外阅读](#)

[第7章 软件工程](#)

[持久理解和学习成果](#)

[7.1 软件工程学科](#)

[7.2 软件生命周期](#)

[7.2.1 周期是个整体](#)

[7.2.2 传统的开发阶段](#)

[7.3 软件工程方法学](#)

[7.4 模块化](#)

[7.4.1 模块式实现](#)

[7.4.2 耦合](#)

[7.4.3 内聚](#)

[7.4.4 信息隐藏](#)

[7.4.5 组件](#)

[7.5 行业工具](#)

[7.5.1 较老的工具](#)

[7.5.2 统一建模语言](#)

[7.5.3 设计模式](#)

[7.6 质量保证](#)

[7.6.1 质量保证的范围](#)

[7.6.2 软件测试](#)

[7.7 文档](#)

[7.8 人机界面](#)

[7.9 软件所有权和责任](#)

[复习题](#)

[社会问题](#)

[课外阅读](#)

[第8章 数据抽象](#)

[持久理解和学习成果](#)

[8.1 基本数据结构](#)

[8.1.1 数组和聚合](#)

[8.1.2 列表、栈和队列](#)

[8.1.3 树](#)

[8.2 相关概念](#)

[8.2.1 再谈抽象](#)

[8.2.2 静态结构与动态结构](#)

[8.2.3 指针](#)

8.3 数据结构的实现

8.3.1 存储数组

8.3.2 存储聚合

8.3.3 存储列表

8.3.4 存储栈和队列

8.3.5 存储二叉树

8.3.6 操控数据结构

8.4 一个简短的案例研究

8.5 定制的数据类型

8.5.1 用户定义的数据类型

8.5.2 抽象数据类型

8.6 类和对象

*8.7 机器语言中的指针

复习题

社会问题

课外阅读

第9章 数据库系统

持久理解和学习成果

9.1 数据库基础

9.1.1 数据库系统的重要性

9.1.2 模式的作用

[9.1.3 数据库管理系统](#)

[9.1.4 数据库模型](#)

[9.2 关系模型](#)

[9.2.1 关系设计中的问题](#)

[9.2.2 关系运算](#)

[9.2.3 SQL](#)

[*9.3 面向对象数据库](#)

[*9.4 维护数据库的完整性](#)

[9.4.1 提交/回滚协议](#)

[9.4.2 锁定](#)

[*9.5 传统的文件结构](#)

[9.5.1 顺序文件](#)

[9.5.2 索引文件](#)

[9.5.3 散列文件](#)

[9.6 数据挖掘](#)

[9.7 数据库技术的社会影响](#)

[复习题](#)

[社会问题](#)

[课外阅读](#)

[第10章 计算机图形学](#)

[持久理解和学习成果](#)

10.1 计算机图形学的范围

10.2 3D图形学概述

10.3 建模

10.3.1 单个物体的建模

10.3.2 整个场景的建模

10.4 渲染

10.4.1 光 - 表面相互作用

10.4.2 裁剪、扫描转换和隐藏面消除

10.4.3 着色

10.4.4 渲染 - 流水线硬件

*10.5 处理全局光照

10.5.1 光线跟踪

10.5.2 辐射度

10.6 动画

10.6.1 动画基础

10.6.2 运动学和动力学

10.6.3 动画制作过程

复习题

社会问题

课外阅读

第11章 人工智能

持久理解和学习成果

11.1 智能与机器

11.1.1 智能体

11.1.2 研究方法

11.1.3 图灵测试

11.2 感知

11.2.1 理解图像

11.2.2 语言处理

11.3 推理

11.3.1 产生式系统

11.3.2 搜索树

11.3.3 启发式

11.4 其他研究领域

11.4.1 知识的表达和处理

11.4.2 学习

11.4.3 遗传算法

11.5 人工神经网络

11.5.1 基本特性

11.5.2 训练人工神经网络

11.6 机器人学

11.7 后果的思考

[复习题](#)

[社会问题](#)

[课外阅读](#)

[第12章 计算理论](#)

[持久理解和学习成果](#)

[12.1 函数及其计算](#)

[12.2 图灵机](#)

[12.2.1 图灵机的原理](#)

[12.2.2 丘奇-图灵论题](#)

[12.3 通用程序设计语言](#)

[12.3.1 Bare Bones语言](#)

[12.3.2 用Bare Bones语言编程](#)

[12.3.3 Bare Bones的通用性](#)

[12.4 一个不可计算的函数](#)

[12.4.1 停机问题](#)

[12.4.2 停机问题的不可解性](#)

[12.5 问题的复杂性](#)

[12.5.1 问题复杂性的度量](#)

[12.5.2 多项式问题与非多项式问题](#)

[12.5.3 NP问题](#)

[*12.6 公钥密码学](#)

[12.6.1 模表示法](#)

[12.6.2 RSA公钥密码学](#)

[复习题](#)

[社会问题](#)

[课外阅读](#)

[附录A ASCII码](#)

[附录B 用于处理二进制补码表示的电路](#)

[附录C Vole：一种简单的机器语言](#)

[C.1 Vole体系结构\[1\]](#)

[C.2 Vole的机器语言](#)

[附录D 高级程序设计语言](#)

[D.1 Ada](#)

[D.2 C](#)

[D.3 C++](#)

[D.4 C](#)

[D.5 FORTRAN](#)

[D.6 Java](#)

[附录E 迭代结构与递归结构的等价性](#)

[索引](#)

[教师支持申请表](#)

版 权

著 [美] J. 格伦·布鲁克希尔 (J. Glenn Brookshear)

[美] 丹尼斯·布里罗 (Dennis Brylow)

译 刘 艺 吴 英 毛倩倩

责任编辑 杨海玲

人民邮电出版社出版发行 北京市丰台区成寿寺路11号

邮编 100164 **电子邮件** 315@ptpress.com.cn

网址 <http://www.ptpress.com.cn>

读者服务热线：(010)81055410

反盗版热线：(010)81055315

读者服务：

内容提要

本书是计算机科学概论课程的经典教材，全书对计算机科学做了百科全书式的精彩阐述，充分展现了计算机科学的历史背景、发展历程和新的技术趋势。书中首先介绍信息编码及计算机体系结构的基本原理，进而介绍操作系统和组网以及因特网的相关内容，接着探讨算法、程序设计语言及软件工程，然后讨论数据抽象和数据库方面的问题，讲述图形学的主要应用以及人工智能，最后以计算理论的介绍结束全书。本书在内容编排上由具体到抽象逐步推进，便于教学安排，每一个主题自然而然地引导出下一个主题。此外，书中还包含大量的图、表和示例，有助于读者对知识的了解与把握。第13版对前一版进行了全面的修正和更新，还新增了Python相关的内容，并且继续使用第12版引入的Python代码示例和类Python伪代码。

本书非常适合作为高等院校计算机以及相关专业本科生教材，也适合有意在计算机方面发展的非计算机专业读者作为入门参考书。

前言

本书是计算机科学的入门教材。在力求保持学科广度的同时，还兼顾深度，以对所涉及的主题给出中肯的评价。

读者对象

本书面向计算机科学以及其他各个学科的学生。大多数计算机科学专业的学生在最初的学习中都有一个误解，认为计算机科学就是程序设计、网页浏览以及因特网文件共享，因为这基本上就是他们所看到的一切。实际上计算机科学远非如此。在入门阶段，学生们需要了解他们主修的这门学科所涉及的广泛内容，这也正是本书的宗旨。本书力图使学生对计算机科学有一个总体的了解，希望在这个基础上，他们可以领会该领域与日后其他课程的关联性以及相互关系。事实上，本书采用的综述方式也是自然科学入门教程的常用模式。

其他学科的学生如果想融入这个技术化的社会，也需要具备这些宽泛的知识背景。适用于他们的计算机科学课程提供的应该是对整个领域的实用的、现实的理解和剖析，而不仅是培训学生如何上网和使用一些流行的软件。当然，这种培训也有其适用的地方，但本书的目的不在于此，而是用作计算机科学的教科书。

在编写本书前几版时，我们力求保持其对非技术类学生的可读性。因此，本书已经成功地用作教科书，读者囊括了从高中生到研究生的各个教育层级不同专业的学生。这一版会继续保持这一传统。

第13版新增的内容

第13版继续使用第12版引入的Python代码示例和类Python的伪代码。我们做出这种改变有几个原因。首先，本书已经包含了相当多的各种语言的代码，有几章还有详细的伪代码。其次，读者已经吸收了相当多的句法方面的知识，似乎可以将句法重新定位为在后续课程中会实际看到的语言。最后，更重要的是，越来越多的使用本书的教师断定，即使优先介绍计算的广度，对学生来讲，如果缺乏用于探索和实验的编程工具，许多课题也会很难掌握。

那为什么选择Python呢？语言的选择始终是一个有争议的问题，任何一种选择，反对的人都至少和支持的人一样多。Python是一个极好的中间选择，因为Python：

- 句法简洁易学；
- I/O原语简单；
- 数据类型和控制结构与先前版本中使用的伪代码原语很接近；
- 支持多个程序设计范式。

Python是一种成熟的程序设计语言，它拥有充满活力的开发社区和丰富的在线资源，便于进一步的研究。根据某些衡量标准，Python仍然是业界前5种最常用的程序设计语言之一，并且在计算机科学入门课程中的使用急剧增加。对非计算机专业的学生来讲，它是一门极其受欢迎的入门课程，并且已被其他像物理学和生物学这样的STEM^[1]领域广泛接受，作为计算科学应用的首选语言。

[1] STEM是4个英语单词Science（科学）、Technology（技术）、Engineering（工程）和Mathematics（数学）的缩写。——译者注

然而，本书的重点仍然是广义的计算机科学概念；补充Python语言的内容是为了让读者能体会到比先前版本更浓的编程味儿，而不是为了全面地介绍编程。所涵盖的Python主题是由本书现有的结构决定的。因此，第1章涉及Python句法，用于表示数据——整数、浮点数、ASCII字符串或Unicode字符串等。第2章涉及Python运算，详细讨论反映了本章其余部分所讨论的机器原语。条件、循环和函数是在第5章引入的，那里需要使用这些结构来设计一个足够完整的描述算法的伪代码。简而言之，Python结构是用来进一步阐明计算机科学概念而不是劫持话题的。

每一章都能看到对前一版对应章节的修订、更新以及修正。

章节安排

本书主题由具体到抽象逐步推进——这是一种很利于教学的顺序，每一个主题自然而然地引导出下一个主题。首先介绍信息编码、数据存储及计算机体系结构的基本原理（第1章和第2章），进而研究操作系统（第3章）和计算机网络（第4章），探讨算法、程序设计语言及软件开发（第5章至第7章），探索如何更好地访问信息（第8章和第9章），考虑计算机图形学技术的一些重要应用（第10章）及人工智能（第11章），最后介绍抽象的计算理论（第12章）。

虽然本书的编排顺序自然连贯，但各个章节都具有很强的独立性，可以单独阅读，也可以根据不同学习顺序重新排列。事实上，本书通常被用作各类课程的教材，内容选择的顺序是多种多样的。其中一种教法是先介绍第5章和第6章（算法和程序设计语言），然后根据需要返回到前面的相应章节。我还知道有一门课程是从第12章有关可计算性的内容开始的。本书还曾作为深入不同领域项目的基础，用于“高级研讨班”课程的教科书。面对不需要了解太多技术的学生，在教学中可以重点讲述第4章（组网及因特网）、第9章（数据库系统）、第10章（计算机图形学）和第11章（人工智能）。

每章开篇都用星号标出了选学章节。这些选学章节要么是讨论更专业的话题，要么是对传统话题作深入探究。此举仅是为那些想采取不同阅读顺序的读者提供一点建议。当然，还有其他读法。尤其对于那些寻求快速阅读的读者，我们建议采取下面的阅读顺序。

章 节	主 题
-----	-----

章节	主题
1.1 ~ 1.4	数据编码和存储基础
2.1 ~ 2.3	机器体系结构和机器语言
3.1 ~ 3.3	操作系统
4.1 ~ 4.3	组网及因特网
5.1 ~ 5.4	算法和算法设计
6.1 ~ 6.4	程序设计语言
7.1 ~ 7.2	软件工程
8.1 ~ 8.3	数据抽象
9.1 ~ 9.2	数据库系统
10.1 ~ 10.2	计算机图形学
11.1 ~ 11.3 12.1 ~ 12.2	人工智能 计算理论

在本书中有几条贯穿始终的主线。主线之一为计算机科学是不断发展变化的。本书从历史的角度反复呈现各个主题，讨论其当前的状况，并指出研究方向。另一条主线是抽象的作用以及用抽象工具控制复杂性的方式。该主线在第0章引入，然后在操作系统架构、组网、算法的发展、程序设计语言设计、软件工程、数据组织和计算机图形学等内容中反复体现。

致教师

本教材所包含的内容很难在一个学期内讲授完，因此一定要果断砍掉不适合自己的教学目标的那些主题，或者根据需要重新调整讲授顺序。你会发现，尽管本书有它固有的结构体系，但各个主题在很大程度上是相对独立的，完全可以根据需要做出选择。我写本书的目的是把它作为一种课程的资源，而非课程的定义。我们建议鼓励学生自己阅读课堂上没有讲授的内容。如果我们认为所有的东西都一定要在课堂上讲，那就低估学生的能力了。我们应该教会他们独立学习。

关于本书从具体到抽象的组织结构，我们觉得有必要多言几句。作为学者，我们总以为学生会欣赏我们对于学科的观点，这些观点通常是我们在某一领域的多年工作中形成的。但作为教师，我们认为最好从学生的视角呈现教材内容。这就是为什么本书首先介绍数据的表示/存储、计算机体系结构、操作系统以及组网，因为这些都是学生们最容易产生共鸣的主题：他们很可能听说过JPEG、MP3这些术语，可能用DVD和闪存盘刻录过数据，应用过某一操作系统，日常使用过因

特网和智能手机。从这些主题开始讲授这门课程，学生可以为许多让他们困惑多年的问题找到答案，并且把这门课看成实践课程而不是纯理论的课程。由此，就会很自然地过渡到较抽象的算法、算法结构、程序设计语言、软件开发方法、可计算性以及复杂性等内容上，而这些内容就是我们本领域的人认为的计算机科学的主要内容。正如前面所说的，以这种方式呈现全书并不是强求大家都按此顺序讲课，只是鼓励大家如此尝试一下。

我们都知道，学生能学到的东西要远远多于我们直接传授的内容，而且潜移默化传授的知识更容易被吸收。当要“传授”问题的解决方法时，就更是如此。学生不可能通过学习问题求解的方法变成问题的解决者，他们只有通过解决问题，而且不仅仅是解决那些精心设计过的“教科书式的问题”，才能成为问题的解决者。因此，本书加入了大量的问题，并特意让其中一些问题模棱两可——这意味着正确方法或正确答案不一定是唯一的。我们鼓励大家采用并拓展这些问题。

“潜移默化学习”类的其他话题还有职业精神、道德和社会责任感。我认为这种内容不应该独立成章，而是应该在有所涉及时讨论，而这正是本书的编排方法。你会发现，3.5节、4.5节、7.9节、9.7节和11.7节分别在操作系统、组网、软件工程、数据库系统和人工智能的上下文中提及了安全、隐私、责任和社会意识的问题。你还会发现，每一章都包含了一些社会问题，这些问题将鼓励学生思考他们所生活的现实社会与教材中的内容的关系。

感谢你对本书感兴趣。无论你是否选用本书作为教材，我都希望你认同它是一部好的计算机科学教育文献。

教学特色

本书是多年教学经验的结晶，因此在辅助教学方面考虑较多。最主要的是提供了丰富的问题以加强学生的参与——这一版包含1 000多个问题，分为“问题与练习”“复习题”和“社会问题”。

“问题与练习”列在每节末尾（第0章除外），用于复习刚刚讨论过的内容，扩充以前讨论过的知识，或者提示以后会涉及的有关主题。

“复习题”列在每章的末尾（第0章除外）。它们是课后作业，内容覆盖整章，书中没有给出答案。

“社会问题”也列在每章的末尾，供思考讨论。许多问题可以用来开展课外研究，可要求学生提交简短的书面或口头报告。

在每章的末尾还有“课外阅读”，其中列出了与本章主题有关的参考资料。

补充材料

本书的许多补充材料可以从Pearson Higher Education官方网站上找到。以下内容面向所有读者。

- 每章的实践项目帮助读者加深理解本教材的主题，并可以帮助读者了解其他相关主题。

- 每章的“自测题”帮助读者复习本书中的内容。

- 介绍Python基本原理的手册，它在教学顺序上与本书是兼容的。

除此之外，教师还可以登录Pearson Education的教师资源中心或者填写本书后的“教师支持申请表”，申请获得以下教辅资料。

- 包含“复习题”答案的教师指南。

- PowerPoint幻灯片讲稿。

- 测试题库。

致学生

Glenn Brookshear有一点点偏执（他的一些朋友说可远不是一点点），所以写本书时，他很少接受他人的建议，许多人认为其中一些内容对于初学者过于高深。但是，我们相信即使学术界把它们归为“高级论题”，但只要与主题相关就是合适的。读者需要的是一本全面介绍计算机科学的教科书，而不是“缩水”的版本——只包括那些简化了的、被认为适合初学者的主题。因此，我们不回避任何主题，而是力求寻找更好的解释。我们力图在一定深度上向读者展示计算机科学最

真实的一面。就好比对待菜谱里的那些调味品一样，你可以有选择地略过本书的一些主题，但我们全部呈现出来是为了在你想要的时候供你“品尝”，而且我们也鼓励你们去尝试。

我们还要指出的是，在任何与技术有关的课程中，当前学到的详细信息未必就适合以后的需要。这个领域是发展变化的——这正是使人兴奋的方面。本书将从现实及历史的角度展现本学科的内容。有了这些背景知识，你就会和技术一起成长。我们希望你现在就行动起来，不要局限于课本的内容，而要进行大胆探索。要学会学习。

感谢你的信任，选择了我们的这本书。作为作者，我们有责任创作出值得一读的作品。我们希望你觉得我们已经尽到了这份责任。

致谢

首先，我要感谢Glenn Brookshear，他一直照看着这本书——“他的孩子”——包括之前的11个版本，跨越了计算机科学领域快速发展和动荡变化的超过四分之一个世纪的时间。虽然这是他允许合著者来审核所有修订的第二个版本，但这一版收录的仍然基本是Glenn的“声音”，并由他主导，这也是我所希望的。任何新的瑕疵都是我造成的，而优雅的基本框架都是他设计的。

我和Glenn要感谢那些支持本书（阅读并使用本书前几个版本）的人们，我们感到很荣幸。一部计算机科学教科书的13个版本？我们一定是在接近某种记录。

在我们努力确定第13版和CS原理框架之间重叠部分的时候，Andrew Kuemmel（Madison West）一直是我们极重要的参谋。他是我认识的唯一一个成功在高中和大学教授许多CSP课程实例的人，他对我家乡威斯康星州的计算机科学教育工作者的不倦支持一直很鼓舞人心。在David T. Smith（宾夕法尼亚州印第安纳大学）和我共同编写第11版修订内容的过程中，David发挥了很重要的作用。他做的许多修订在第13版中仍然存在。David对之前版本的仔细阅读和对补充材料的仔细关注对本书来讲至关重要。Andrew Kuemmel（Madison West）、George Corliss（Marquette）和Chris Mayfield（James Madison）为这一版和之前版本的初稿提供了有价值的反馈、深刻的见解和鼓励，同时James E. Ames（Virginia Commonwealth）、Stephanie E. August（Loyola）、Yoonsuck Choe（Texas A&M）、Melanie Feinberg（UT-Austin）、Eric D. Hanley（Drake）、Sudharsan R. Iyengar（Winona State）、Ravi Mukkamala（Old Dominion）和Edward Pryor（Wake Forest）都对第12版Python方面的修订提供了宝贵的评价。

其他对这一版和之前版本做出贡献的人包括J. M. Adams、C. M. Allen、D. C. S. Allison、E. Angel、R. Ashmore、B. Auernheimer、P. Bankston、M. Barnard、P. Bender、K. Bowyer、P. W. Brashear、C. M. Brown、H. M. Brown、B. Calloni、J. Carpinelli、M. Clancy、R. T. Close、D. H. Cooley、L. D. Cornell、M. J. Crowley、F. Deek、M. Dickerson、M. J. Duncan、S. Ezekiel、C. Fox、S. Fox、N. E. Gibbs、J. D. Harris、D. Hascom、L. Heath、P. B. Henderson、L. Hunt、M.

Hutchenreuther、 L. A. Jehn、 K. K. Kolberg、 K. Korb、 G. Krenz、 J. Kurose、 J. Liu、 T. J. Long、 C. May、 J. J. McConnell、 W. McCown、 S. J. Merrill、 K. Messersmith、 J. C. Moyer、 M. Murphy、 J. P. Myers, Jr.、 D. S. Noonan、 G. Nutt、 W. W. Oblitey、 S. Olariu、 G. Riccardi、 G. Rice、 N. Rickert、 C. Riedesel、 J. B. Rogers、 G. Saito、 W. Savitch、 R. Schlafly、 J. C. Schlimmer、 S. Sells、 Z. Shen、 G. Sheppard、 J. C. Simms、 M. C. Slattery、 J. Slimick、 J. A. Slomka、 J. Solderitsch、 R. Steigerwald、 L. Steinberg、 C. A. Struble、 C. L. Struble、 W. J. Taffe、 J. Talburt、 P. Tonellato、 P. Tromovitch、 P. H. Winston、 E. D. Winter、 E. Wright、 M. Ziegler，还有一位匿名的朋友。我们向他们中的每一位致以最诚挚的谢意。

Diane Christie为一个之前版本的配套网站设计了Java和C++手册，我们最新的Python资源就是从那里来的。谢谢你，Diane。另外，还要感谢Roger Eastman，他是伴随本书之前版本的每章实践项目幕后的创造动力。

我同时要感谢支持本书出版工作的Pearson出版集团的员工。特别是Tracy Johnson、 Erin Ault、 Carole Snyder和Scott Disanno，他们在整个过程中为本书贡献了他们的智慧并提出了许多改进建议。

最后，我要感谢我的妻子Petra，在我编写本版时，她照顾三个孩子，给我留出了很多个很长的下午和晚上。她是我的后盾。

D. W. B.

马凯特大学

2018年1月1日

本书与AP[®]计算机科学原理课程框架之间的关联

AP[®]是美国大学理事会注册和拥有的商标，它没有参与本产品的生产，也不为本产品背书。

本书各章和各节对应了AP[®]计算机科学原理课程框架（简称AP[®]CSP课程框架）中的“持久理解”（Enduring Understanding，EU）和“学习成果”（Learning Outcome，LO）部分，具体对应关系如下所示。

1.1 创造性开发可能是创建计算工件的必要过程		
1.1.1	在创建计算工件时应用创造性开发过程	5.3
1.2 计算使人们能够使用创造性开发过程来创建用于创造性表达式的计算工件或解决问题		
1.2.1	为创造性表达式创建计算工件	1.0
1.2.3	通过组合或修改现有工件来创建新的计算工件	10.6
1.3 计算可以扩展传统的人类表达和体验的形式		

1.3.1	对创造性表达式使用计算工具和技术	10.1
2.1 根据二进制序列构建的各种抽象可用于表示所有数字数据		
2.1.1	描述用于表示数据的各种抽象种类	1.1, 1.4, 2.1
2.1.2	解释如何用二进制序列来表示数字数据	1.6, 1.7, 2.1, 2.2
2.2 多层次的抽象用于编写程序或创建其他计算工件		
2.2.1	在编写程序或创建其他计算工件时开发抽象	6.3
2.2.2	使用多层次的抽象来编写程序	6.2
2.2.3	确定编写程序时使用的多层次的抽象	1.1, 2.6, 6.1, 6.4
2.3 模型和模拟使用抽象来产生新的理解和知识		
2.3.1	使用模型和模拟来表示现象	10.3, 10.4, 10.6
2.3.2	使用模型和模拟来制订、改进和测试假设	10.4
3.1 人们使用计算机程序来处理信息从而获得深入理解和知识		

3.1.1	找到数字化处理信息的模式和测试假设来获得深入理解和知识	9.1, 9.6
3.2 计算有助于探索 and 发现信息中的连接		
3.2.1	从数据中提取信息来发现和解释连接或趋势	9.1, 11.4
3.2.2	确定大数据集是如何影响使用计算过程发现信息和知识的	9.0, 9.1, 9.4
3.3 在将信息表示为数字数据时要有所权衡		
3.3.1	分析数据的表示、存储、安全和数据传输是如何涉及信息的计算操纵的	1.3, 1.9, 4.4, 4.5, 9.7
4.1 算法是可由计算机执行的过程的精确指令序列，是使用程序设计语言实现的		
4.1.1	开发用于程序实现的算法	5.1, 5.2, 5.4, 6.3
4.1.2	用语言表达算法	5.2, 6.1
4.2 算法可以解决许多但不是所有的计算问题		
4.2.1	解释能在合理时间内运行的算法与不能在合理时间内运行的算法之间的区别	12.5
4.2.2	解释计算机科学中可解问题与不可解问题之间的区别	11.3, 12.5

4.2.3	解释计算机科学中存在的不可决定问题	12.1, 12.4
4.2.4	分析评估和凭经验评估算法的效率、正确性和清晰度	5.6, 12.5
5.1 可以开发程序来进行创造性的表达、满足个人好奇心、创造新知识或解决问题（帮助人员、组织或社会）		
5.1.1	开发程序来进行创造性的表达、满足个人好奇心或创造新知识	10
5.1.2	开发正确的程序来解决问题	6.2, 7.2, 7.3, 7.4, 7.7
5.1.3	协作开发程序	5.1, 7.1, 7.6
5.2 人们编写程序来执行算法		
5.2.1	解释程序是如何实现算法的	2.6, 6.2
5.3 适当的抽象使编程更容易		
5.3.1	使用抽象管理程序复杂性	1.8, 2.6, 6.3, 6.5, 8.1, 8.5
5.4 人们出于不同的目的开发、维护和使用程序		

5.4.1	评估程序的正确性	1.8, 7.5
5.5 程序设计使用数学和逻辑概念		
5.5.1	在程序设计中适当的数学和逻辑概念	1.1, 1.6, 1.7, 1.8, 8.1
6.1 因特网是一个自治系统的网络		
6.1.1	解释因特网中的抽象以及因特网是如何运作的	4.1, 4.2, 4.4
6.2 因特网的特点影响着建立在它上面的系统		
6.2.1	解释因特网的特点和建立在它上面的系统	4.2, 4.4
6.2.2	解释因特网的特点是如何影响建立在它上面的系统的	1.3, 4.1, 4.2, 4.4, 4.6
6.3 网络安全是因特网和建立在它上面的系统的重要关注点		
6.3.1	确定现有的网络安全关注点，以及通过因特网和建立在它上面的系统解决这些问题的潜在选项	4.6
7.1 计算增强了沟通、互动和认知		
7.1.1	解释计算革新是如何影响沟通、互动和认知的	4

7.2 计算几乎能在每个领域实现创新		
7.2.1	解释计算是如何影响其他领域创新的	9.6, 10.6
7.3 计算对人和社会有全球性影响—既有有利的又有有害的		
7.3.1	分析计算的有利影响和有害影响	4.6, 9.7, 11.7
7.4 计算革新既影响着设计和使用它们的经济、社会和文化环境，又受这些环境的影响		
7.4.1	解释计算和现实生活环境（包括经济、社会和文化环境）之间的关联	4.2

第0章 绪 论

在开篇的这一章，我们将探讨计算机科学所涉及的领域，介绍其历史背景，为深入学习奠定基础。

本章内容

0.1 算法的作用

0.2 计算的历史

0.3 学习大纲

0.4 计算机科学的首要主题

持久理解和学习成果

算法研究是计算机科学的核心。

学习成果 解释计算机科学领域中算法的重要性。

计算机科学近代史的特点是计算能力、小型化和连通性的快速进步。

学习成果 确定那些为我们的现代技术社会铺平道路的计算机科学史上的重要里程碑。

计算机科学的进步正在深深地影响着人类文化和社会。

学习成果 讨论计算机科学进步造成的社会、道德或法律难题。

计算机科学是寻求为计算机设计、计算机程序设计、信息处理、问题的算法解决方案和算法过程本身等主题建立科学基础的学科。计算机科学既是当今计算机应用的支柱，又是今后计算基础设施的基础。

在本书中，我们将详细介绍计算机科学，探索广泛的主题，包括构成大学计算机科学课程的大部分主题。我们要领略这个领域的博大精深和动态发展。因此，除了这些主题本身，我们还会关注它们的历史发展、研究现状和未来前景。我们的目标是让人们以学以致用态度理解计算机科学——既帮助那些想在此领域进行深入专业研究的人，又可以使其他领域的人在日益技术化的社会崭露头角。

0.1 算法的作用

首先让我们了解一下计算机科学最基础的概念——“算法”。一般来讲，**算法**（algorithm）是完成一项任务所遵循的一系列步骤。（在第5章，我们将给出比较精确的定义。）例如，有关于烹饪的算法（称为菜谱），有在陌生城市准确寻找道路的算法（通常称为**道路指南**），有使用洗衣机的算法（通常标示在洗衣机盖子的内侧或者贴在自助洗衣店的墙上），有**演奏音乐的算法**（以乐谱的形式表示），还有魔术表演的算法（见图0-1）。

效果：表演者从一副普通的扑克牌中抽取若干张牌，正面朝下放在桌上，充分洗牌后展开。然后，表演者会根据观众的要求相应地翻出红牌或者黑牌。

秘诀和话术：

步骤1 从一副普通扑克牌中抽取10张红牌和10张黑牌。把它们根据颜色分为两摞，正面朝上放在桌面上。

步骤2 告诉观众你已经选取了若干张红牌和黑牌。

步骤3 拿起红牌，装作整理成一摞的样子，将它们正面朝下放在左手里，同时用右手的拇指和食指挤压这摞牌的两端，把牌面向下推，使每张牌呈现略微向下的弧形。然后，把这摞红牌扣在桌子上并宣布：“这是其中的红牌。”

步骤4 拿起黑牌，模仿步骤3的方法，但要使这些牌呈现略微向上的弧形。然后，把牌扣在桌子上并宣布：“这是其中的黑牌。”

步骤5 把黑牌放回桌面后，立即用双手把红牌和黑牌混在一起（仍然正面朝下），平铺在桌面上。告诉大家你已经洗好了牌。

步骤6 只要桌面上还有扣着的牌，就重复下面的步骤。

6.1 请观众要一张红牌或黑牌。

6.2 如果所要的牌为红色，而且桌面上倒扣有凹形的牌，就翻开其中的一张并告诉大家：“这是一张红牌”。

6.3 如果所要的牌为黑色，而且桌面上倒扣有凸形的牌，就翻开其中的一张并告诉大家：“这是一张黑牌”。

6.4 否则，就告诉大家桌面上没有所要求颜色的牌了，然后翻开桌面上所有的牌，以证实你的断言。

图0-1 一个魔术的算法

在机器（如计算机）执行一项任务之前，必须先找到完成这项任务的算法，并用与该机器兼容的形式表示出来。算法的表示称作程序（program）。为了人们读写方便，计算机程序通常打印在纸上或者显示在计算机屏幕上。为了便于机器识别，程序需要以一种与该机器技术兼容的形式进行编码。开发程序、采取与机器兼容的形式进行编码并将其输入机器中的过程，称作程序设计/编程（programming），有时也称作编码（coding）。程序及其所表示的算法统称为软件（software），而机械本身称为硬件（hardware）。

算法的研究起源于数学学科。事实也的确如此，探寻算法是数学家的重要活动，远远早于当今计算机的出现。它的目标是找出一组指令，描述如何解决某一特定类型的所有问题。求解两个多位数商的长除算法是早期研究中一个最著名的例子。另一个例子是古希腊数学家

欧几里得发现的欧几里得算法——求两个正整数的最大公约数（见图0-2）。

描述：本算法假定输入是两个正整数，目的是要计算这两个数的最大公约数。

过程：

步骤1 将这两个输入值中较大的一个和较小的一个分别赋予 M 和 N 。

步骤2 用 M 除以 N ，余数设为 R 。

步骤3 如果 R 不为0，则将 N 的值赋予 M ，将 R 的值赋予 N ，然后回到步骤2；否则最大公约数就是 N 当前被赋予的值。

图0-2 求两个正整数的最大公约数的欧几里得算法

一旦我们找到执行任务的算法，执行该任务时就不再需要了解该算法所依据的原理——任务的执行简化成了遵照指令操作的过程。

（我们可以在不了解算法原理的情况下，根据长除算法求商或者根据欧几里得算法求最大公约数。）在某种意义上，解决这个问题的智能被编码到了算法中。

通过算法来捕获和传达智能（至少是智能行为），我们能够构建出执行有用任务的机器。因此，机器展现出来的智能水平受限于算法所传达的智能。只有存在执行某一项任务的算法时，我们才可以制造出执行这一任务的机器。换言之，如果我们找不到解决某问题的算法，机器就解决不了这个问题。

20世纪30年代，库尔特·哥德尔（Kurt Gödel）发表了不完备性定理的论文，它使确定算法能力的局限性成为数学学科的一个研究课题。这个定理实质上阐述的是，在任何一个包括传统意义的算术系统的数学理论内，总有一些命题的真伪无法通过算法的手段来确定。简

言之，对算术系统的任何全面研究都超越了算法活动的的能力范围。这一认识动摇了数学的基础，于是关于算法能力的研究随之而来，它开创了今天的计算机科学这门学科。的确，正是算法的研究构成了计算机科学的核心。

0.2 计算的历史

今天的计算机有着庞大久远的世系渊源，其中较早的一个计算设备是算盘。历史告诉我们，它可能源于中国古代且曾被用于早期希腊和罗马文明。算盘本身非常简单，一个矩形框里固定着一组细杆，而每个细杆上又各串有一组珠子（见图0-3）。在细杆上，珠子上下移动的位置就表示其所存储的值。这些珠子的位置代表了这台“计算机”表示和存储的数据。这台机器是依靠人的操作来控制算法执行的。因此，算盘自身只算得上一个数据存储系统，它必须在人的配合下才能成为一台完整的计算机。

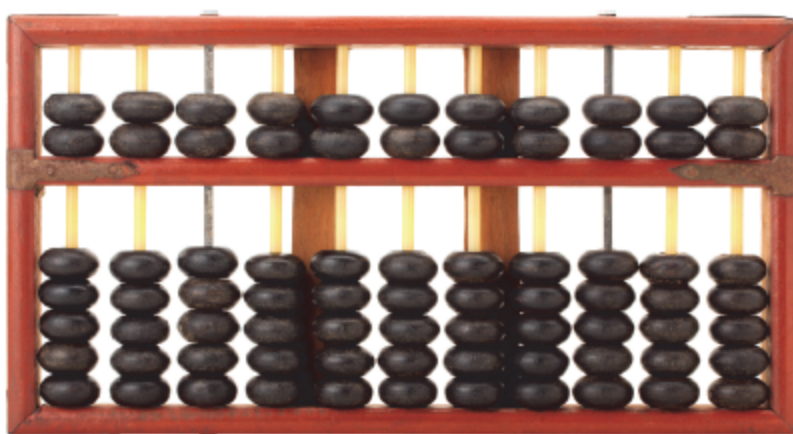


图0-3 中国木制算盘（Ekkapon/Shutterstock）

从中世纪到近代，人们一直在探求更复杂的计算机器。一些发明家开始基于齿轮技术设计计算机器。采用这种技术的发明家有法国的布莱斯·帕斯卡（Blaise Pascal，1623—1662）、德国的戈特弗里德·威廉·莱布尼茨（Gottfried Wilhelm Leibniz，1646—1716）和英国的查尔斯·巴贝奇（Charles Babbage，1792—1871）。这些机器利用齿轮的位置来表示数据，要在规定齿轮初始位置的基础上机械地输入数据。帕斯卡和莱布尼茨的机器所计算的结果是通过观察齿轮的最终位置得到的。但巴贝奇构想的机器，则可以把计算的结果打印在纸上，从而消除可能出现的抄写错误。

就执行算法的能力而言，我们可以看到这些机器在灵活性上的进步。帕斯卡的机器只是为了执行加法而设计的，因此必须将正确的步骤序列嵌入机器结构本身。同样，莱布尼茨的机器也把算法嵌入了其机器结构中，但是操作员可以从它提供的各种算术操作中进行选择。巴贝奇设计的差分机（Difference Engine）（只造了一个演示模型）可以通过修改来执行各种计算，但他设计的分析机（Analytical Engine）（从未得到过构造资助）则能够在打孔纸卡片上读取以洞孔形式表示的指令。因此，巴贝奇的分析机是可编程的。事实上，奥古斯塔·艾达·拜伦（Augusta Ada Byron，通称Ada Lovelace）经常被认为是世界上第一位程序员，她曾发表过一篇论文，阐述巴贝奇的分析机是如何通过编程实现各种计算的。

巴贝奇的差分机

查尔斯·巴贝奇设计的这台机器的确是现代计算机设计的先驱。如果能用比较经济的方式制造出这台机器，如果当时商业和政府的数据处理需求达到今天的规模，那么巴贝奇的想法可能在19世纪就引发了计算机革命。然而结果却是，他在有生之年只造出了差分机的演示模型。该机器通过计算“逐次差分”来确定数字值。我们可以通过考虑整数平方的计算问题，深入了解这一技术。我们先从基础知识开始，0的平方是0，1的平方是1，2的平方是4，3的平方是9。据此，我们可以按照下面的方法得到4的平方（见下图）。现在，我们先计算一下已知的平方之间的差： $1^2 - 0^2 = 1$ ， $2^2 - 1^2 = 3$ ， $3^2 - 2^2 = 5$ 。然后，我们计算这些结果的差： $3 - 1 = 2$ ， $5 - 3 = 2$ 。注意，这些差都是2。假设这个规律能够成立（数学上可以证明它是成立的），那么我们可以得出这样的结论： $(4^2 - 3^2)$ 和 $(3^2 - 2^2)$ 之间的差也一定是2。因为 $(4^2 - 3^2)$ 一定比 $(3^2 - 2^2)$ 大2，所以 $4^2 - 3^2 = 7$ ，因而 $4^2 = 3^2 + 7 = 16$ 。现在，我们已经知道了4的平方，那么就可以依据 1^2 、 2^2 、 3^2 和 4^2 的值继续计算5的平方。（虽然更深入地讨论逐次差分已经超出了现在的学习范围，但是学过微积分的学生可能已观察到，前面的例子是基于“ $y = x^2$ 的二阶导数是一条直线”这个事实得出的。）

x	x^2	一阶差分	二阶差分
0	0		
1	1	1	
2	4	3	2
3	9	5	2
4	16	7	2
5			2

奥古斯塔·艾达·拜伦

奥古斯塔·艾达·拜伦（洛夫莱斯伯爵夫人）是计算界关注的焦点人物。艾达·拜伦的一生近乎悲惨，去世时还不到37岁（1815—1852）。她体弱多病，身处限制妇女从业的社会，是个不墨守成规的人。尽管她对很多科学领域都感兴趣，但最感兴趣的还是数学。1833年，目睹了查尔斯·巴贝奇的差分机样机演示后，她就被这台机器迷住了，从此对“计算科学”产生了兴趣。她把一篇讨论巴贝奇分析机设计的论文从法文翻译为英文，这是她最早对计算机科学做出的贡献。巴贝奇还鼓励她在翻译中增加一个附录，介绍该机器的应用，以及该机器如何编程实现各种任务的示例。巴贝奇对艾达·拜伦的工作十分热情，他希望这篇论文的发表能够帮他得到资金支持，建造他的分析机。（作为拜伦勋爵的女儿，艾达·拜伦具有名人的地位，在生意场上也有一定的关系。）虽然巴贝奇的期望落空了，但是艾达·拜伦的附录保存了下来，人们认为该附录包含了第一批计算机程序的例子。关于巴贝奇对艾达的工作影响有多大，研究计算机历史的历史学家一直争论不休。有些历史学家认为，巴贝奇做出了重大贡献，但另外一些历史学家则认为，巴贝奇并没有帮到艾达，从很大程度上来看反而是一种阻碍。无论如何，奥古斯塔·艾达·拜伦都被认为是世界上第一位程序员，美国国防部为了纪念这位伟大的女性，用她的名字命名了一种杰出的程序设计语言（Ada）。

通过纸卡片上的洞孔来传达算法的想法最初并不是源于巴贝奇。他是从约瑟夫·雅卡尔（Joseph Jacquard，1752—1834）那里得到这个想法的。约瑟夫·雅卡尔于1801年研制出一种织布机，它在织布过程中

执行的步骤，是由宽大厚实的木制（或纸板）卡片上的洞孔样式决定的。因此，织布机的算法很容易进行修改，可以得出不同的编织设计。另一个受益于雅卡尔想法的人是赫尔曼·霍尔瑞斯（Herman Hollerith，1860—1929），他灵活运用这一概念——用纸卡片上洞孔的样式来表示信息，加速了美国1890年人口普查中的表格处理。（霍尔瑞斯的这项改造促成了IBM的诞生。）这种卡片最终被称作穿孔卡片，直到20世纪70年代，它仍是与计算机交互的流行工具。

从成本效益上来讲，19世纪的技术无法让帕斯卡、莱布尼茨和巴贝奇设计的复杂齿轮驱动机器付诸生产。但是，随着20世纪初期电子技术的进步，这个障碍被克服了。这种进步的例子包括乔治·斯蒂比兹（George Stibitz）的电子机械机器和马克一号（Mark I），前者由贝尔实验室于1940年建造，后者由霍华德·艾肯（Howard Aiken）和一组IBM工程师于1944年在哈佛大学建造。这些机器使用了大量机械式继电器，通过电流进行控制。从这个意义上说，这些机器几乎是一造出来就过时了，因为其他研究人员已在应用真空管技术建造完全电子化的计算机。第一台真空管机器显然是阿塔纳索夫-贝瑞（Atanasoff-Berry）机器，由约翰·阿塔纳索夫（John Atanasoff）和他的助手克利福德·贝里（Clifford Berry），于1937—1941年在艾奥瓦州立学院（现在的艾奥瓦州立大学）建造。另一台是称为巨人（Colossus）的机器，在汤米·弗劳尔斯（Tommy Flowers）的指导下建造于英国，该机器在第二次世界大战后期曾被用来破解德国的情报。（实际上，这类机器至少造了十余台，但是由于涉及军事机密和国家安全问题而隐瞒它们的存在，未能列入“计算机家谱”。）不久，更为灵活的机器出现

了，如ENIAC（Electronic Numerical Integrator And Computer，电子数字积分计算器），它是由约翰·莫奇利（John Mauchly）和普雷斯波·埃克特（J. Presper Eckert）在宾夕法尼亚大学莫尔电子工程学院研制的（见图0-4）。

从那时起，计算机的历史就开始和技术进步紧密相连，包括晶体管的发明（物理学家威廉·肖克利、约翰·巴丁和沃尔特·布拉顿因此获得了诺贝尔奖）和后来集成电路的开发（杰克·基尔比因此荣获了诺贝尔物理学奖）。由于这些技术，以往20世纪40年代房间大小的机器在数十年间缩小到了单机柜大小。与此同时，计算机器的处理能力每两年便会翻倍（这一趋势一直持续到了今天）。随着集成电路技术的进步，计算机中的许多元件都可封装在玩具大小的塑料块中做成芯片，在电子市场上很容易就可以买到。

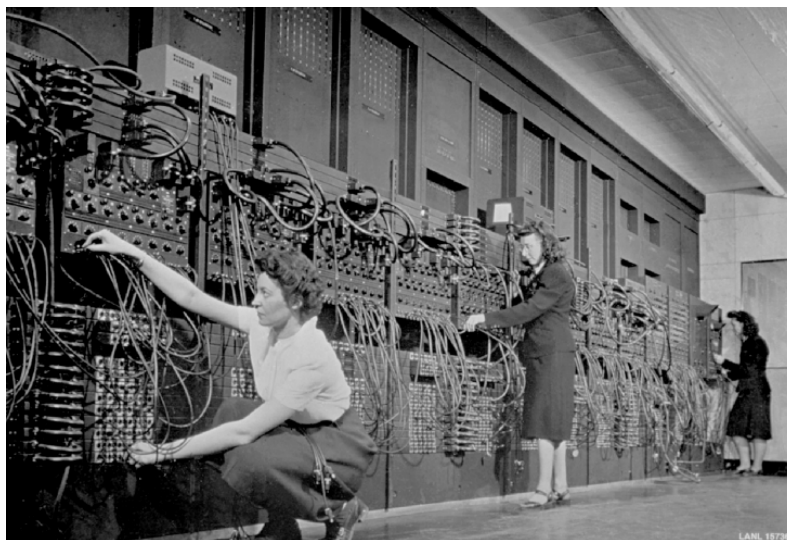


图0-4 3个女人在操作莫尔学院的ENIAC主控制板。
这台机器后来搬到了美国陆军弹道研究实验室
（图片来源于美国陆军）

计算机的普及在很大程度上得益于台式计算机的发展。台式计算机的出现是计算机爱好者的功劳，他们通过芯片组合构建了家用计算机。正是在这些计算机爱好者的“地下”活动中，史蒂夫·乔布斯（Steve Jobs）和斯蒂芬·沃兹尼亚克（Stephen Wozniak）制造出了有商业价值的家用计算机，并于1976年成立了苹果计算机公司（现称苹果公司），专门制造和销售他们的产品。其他经销类似产品的公司有Commodore、Heathkit和Radio Shack。虽然这些产品在计算机爱好者中很畅销，但是并没有被商界普遍接受，商界仍然青睐久负盛名的IBM公司及其大型计算机，这些计算机满足了大多数的商用计算需求。

1981年，IBM公司推出了它的第一款台式计算机，称为**个人计算机**（personal computer），简称PC，其基础软件由一个新成立的称为微软（Microsoft）的公司开发。PC一经推出立即获得了极大的成功，并且奠定了这种台式计算机在商界人士心目中作为日用品的地位。今天，术语PC已被广泛用于指称整个这一类机器（来自各个不同厂商），其设计都是从IBM公司最初的台式计算机演变而来的，而且它们中的大多数还在与微软公司的软件一起销售。不过，有时候，术语PC也能与通用术语**台式机**（desktop）或**笔记本电脑**（laptop）互换使用。

在20世纪后期，**因特网**（Internet）的出现大大改变了人们的沟通方式，这种技术将个人计算机连成了一个全球系统。在这个背景下，英国科学家蒂姆·伯纳斯·李（Tim Berners-Lee）提出了一个系统，这个系统可以通过因特网把计算机上存储的文档链接起来形成错综复杂的

链接信息网，这便是**万维网**（World Wide Web），简称Web。为了能够访问Web信息，人们开发了一种叫作**搜索引擎**（search engine）的软件系统，用于筛选Web上的信息，并对筛选结果进行“归类”，然后利用这个结果帮助用户研究特定的主题。这一领域的主要参与者有谷歌、雅虎和微软。这些公司不断地扩展其与Web相关的活动，而且经常会挑战我们的传统思维方式。

与此同时，台式计算机和笔记本电脑正在被人们所接受，并用于家庭，计算机器的微型化仍在继续。今天，大量的电器和设备中都嵌有微型计算机。现在的汽车可能就包含了几十个小型计算机，用于运行全球定位系统（Global Positioning System，GPS），监控发动机的性能，并提供控制汽车音频和电话通信系统的语音命令服务。

也许，计算机微型化最革命性的应用就是**智能手机**（smartphone）能力的扩展。智能手机是手持通用计算机，通话应用仅是其众多应用之一。从功能上讲，这种钱包大小的设备比几十年前的超级计算机还要强大，它配备有大量的传感器和接口，包括照相机、麦克风、指南针、触摸屏、加速计（用以检测手机的方向和动作），以及一系列无线技术（以便与其他智能手机和计算机通信）。很多人认为智能手机对全球社会的影响大于PC革命。

0.3 学习大纲

本书遵循自底向上的计算机科学研究方法，先从读者有亲身体验的主题开始（如计算机硬件），继而引出比较抽象的主题（如算法复

杂性和可计算性)。结果是，我们的学习遵循这样的模式：随着我们对主题理解的深入，我们构建的抽象工具会越来越大。

我们先学习与设计和构造执行算法的机器有关的主题。在第1章（数据存储）中，我们学习信息是如何编码和存储在现代计算机中的。在第2章（数据操控）中，我们研究简单计算机的内部基本操作。虽然部分学习内容涉及技术，但总体上是独立于具体技术的。也就是说，像数字电路设计、数据编码与压缩系统，以及计算机体系结构这样的主题，与很多技术都相关，并且不管未来技术的发展方向如何，它们的相关性都不会变。

在第3章（操作系统）中，我们学习控制一台计算机总体操作的软件，这种软件称为操作系统。操作系统控制机器与其外部世界之间的接口：保护机器及其内部存储数据不被非授权用户访问；允许计算机用户请求执行各种程序；协调内部活动，以满足用户请求。

在第4章（组网及因特网）中，我们将学习计算机是如何连接成计算机网络的，网络又是如何连接成互联网的。这些知识涉及很多主题，如网络协议、因特网结构与内部操作、万维网以及众多安全问题。

在第5章（算法）中，我们将比较正式地学习算法研究。我们将研究如何发现算法，确定几种基本的算法结构，开发几项用于表示算法的基本技术，并介绍算法效率和正确性的主题。

在第6章（程序设计语言）中，我们讨论的主题是算法表示和程序开发过程。在这一章中，我们会发现，人们在不断改善程序设计技术的过程中，创造出了各种各样的程序设计方法学或范型，每一种都有它自己的一套程序设计语言。我们除了研究这些范型和语言，还会考虑语法和语言翻译的问题。

第7章（软件工程）将介绍计算机科学的一个分支——软件工程。软件工程处理的是开发大型软件系统时遇到的问题。底层主题是，大型软件系统的设计是一项复杂的任务，会遇到传统工程未涉及的许多问题。因此，软件工程这一学科已经成为计算机科学中一个重要的研究领域，它借鉴了诸如工程、项目管理、人事管理、程序设计语言设计，甚至是建筑学等众多领域的研究经验。

在接下来的两章中，我们将学习在计算机系统中组织数据的方法。在第8章（数据抽象）中，我们介绍传统上用于在计算机的主存储器中组织数据的技术，然后探索数据抽象的演变发展，从原语的概念一直到如今的面向对象技术。在第9章（数据库系统）中，我们介绍传统上用于在计算机海量存储器中组织数据的方法，并研究如何实现极其庞大的复杂数据库系统。

在第10章（计算机图形学）中，我们将探索图形学和动画的主题，一个涉及创建并图像化虚拟世界的领域。在计算机科学传统领域（如机器体系结构、算法设计、数据结构和软件工程）发展的基础上，图形学和动画学科取得了重大进展，业已发展成为激动人心、充

满活力的学科。此外，这个领域体现了计算机科学的各个组成部分是如何与物理、艺术和摄影等学科相结合产生显著成果的。

在第11章（人工智能）中，我们将了解到，为了开发更有用的机器，计算机科学已一马当先转向研究人类智能。研究人员希望通过对自己的思维推理和认知的了解，设计出模拟这些过程的算法，从而将类似的能力转移到机器上。结果，计算机科学就有了人工智能这个领域，它非常依赖于心理学、生物学和语言学等领域的研究。

我们的学习到第12章（计算理论）结束，这一章将研究计算机科学的理论基础，这个主题会让我们了解算法（和机器）的局限性。这里我们不但明确了几个算法上无法解决的问题（因而超出了机器的能力），而且认识到许多其他问题的解决方案需要大量的时间或空间，以至于从实践的角度来看它们也是不可解的。因此，通过本章的学习，我们将能够掌握算法系统的范围和局限性。

我们的目标是，每一章主题的探讨都足够深入，使读者真正理解。我们希望所阐述的计算机科学知识对大家的工作能有所帮助——使读者了解自己身处的技术化社会，打好跟随科技进步自我学习的基础。

0.4 计算机科学的首要主题

除了上面列出的每章的主题，我们还希望结合以下几个首要主题来拓宽读者对计算机科学的理解。计算机的微型化及其功能的扩展，

已经把计算机技术推向了当今社会的最前沿。如今，计算机技术已经非常普及，熟练掌握其应用已经成为现代社会成员的基本要求。计算机技术已经改变了政府施加控制的能力，对全球化经济产生了巨大的影响，导致科学研究领域出现了一些令人瞩目的成就，革新了数据收集、存储和应用的作用，为人们提供了新的通信和交互方式，不停地挑战着社会现状。结果是，围绕计算机科学的学科大量涌现，每门学科现在都成了重要的独立研究领域。此外，就像很难区分机械工程和物理一样，我们也很难在这些领域与计算机科学之间画出一条分界线。因此，为了获得正确的视角，我们的研究不仅要涉及以计算机科学为核心的中心主题，还要探索与科学应用和影响相关的各个学科领域。因此，对计算机科学的介绍是一项跨学科的任务。

探索计算领域的广度，能帮助我们记住与计算机科学相结合的主要主题。虽然“计算机科学的七大思想”（Seven Big Ideas of Computer Science）的编纂晚于本书的第10版，但这些思想与本书接下来的各章所要讲述的主题思想有很多相似之处。这“七大思想”简单地说就是算法、抽象、创新、数据、程序设计、因特网和影响。在接下来的章节中，我们将介绍各种主题，在每个主题的介绍中都会涉及这个主题的核心思想、目前的研究领域，以及推动该领域知识进步的一些技术。当我们在后面一遍又一遍地提到这些“大思想”的时候，请多留意。

0.4.1 算法

数据存储容量有限，程序设计过程复杂耗时，这些限制了早期计算机所使用的算法的复杂性。但是，随着这些局限性的消除，机器能完成的任务越来越大、越来越复杂。人们试图用算法表达这些任务，但单凭人类的智力无法做到，于是，越来越多的研究工作转向了算法和程序设计过程的研究。

正是在这种背景下，数学家的理论研究开始有了回报。由于哥德尔不完备性定理，数学家已经在研究有关算法过程的问题了，而这正是先进技术目前面临的问题。由此，孕育出了被称作**计算机科学**的新学科。

如今，计算机科学已经奠定了它算法科学的地位。这门科学范围很广，涉及数学、工程学、心理学、生物学、商业管理和语言学等多个学科。事实上，研究计算机科学不同分支的研究人员对计算机科学的定义也许会截然不同。例如，计算机体系结构领域的研究人员，主要关注微型电路技术，因此他们将计算机科学视为技术的进步和应用，但数据库系统领域的研究人员则认为，计算机科学就是要寻求方法来提升信息系统的有用性，而人工智能领域的研究人员则把计算机科学视为智能和智能行为的研究。

尽管如此，所有这些研究人员的工作还是都涉及了算法科学的方方面面。鉴于算法在计算机科学中扮演的核心角色（见图0-5），找出焦点问题，对学习算法会非常有益。

- 算法过程可以解决哪些问题？

- 怎样才能比较容易地发现算法？
- 如何改进表示和传达算法的技术？
- 如何分析和比较不同算法的特征？
- 如何使用算法来操作信息？
- 如何应用算法来产生智能行为？
- 算法的应用是如何影响社会的？

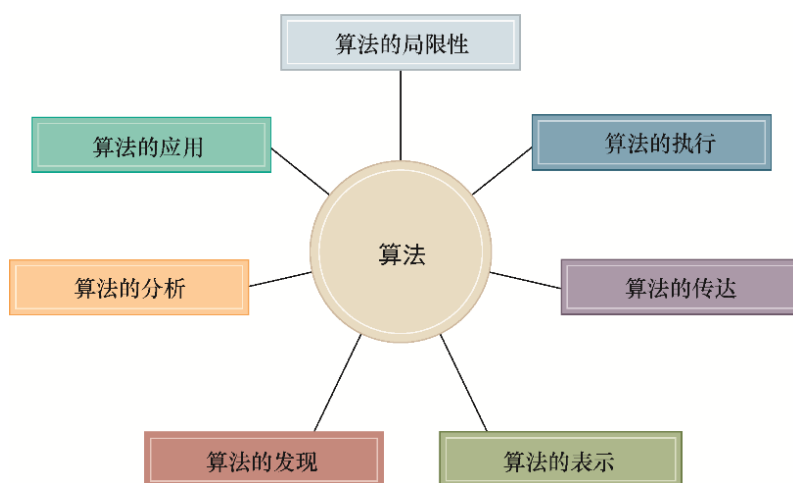


图0-5 算法在计算机科学中的核心地位

0.4.2 抽象

术语**抽象**（abstraction）在本书中是指一个实体的外部特征与其内部构成细节之间的分离。抽象使我们可以忽略复杂设备（如计算机、汽车或微波炉）的一些内部细节，把它们当作一个单个的、可理解的

单元使用。此外，正是通过抽象这种手段，这些复杂的系统才能够被设计和生产出来。计算机、汽车和微波炉都是由若干元件构成的，其中每个元件代表一层抽象，在此层面上，该元件的使用是独立于元件的内部构成细节的。

运用抽象，我们能够构造、分析和管理大型的复杂计算机系统，但如果从细节的层面上看问题，就会不识庐山真面目。在每个抽象层面上，我们都把此系统看成由若干称为**抽象工具**（abstract tool）的构件组成，而忽略这些构件的内部构成。这样我们的精力就可以集中在每个构件如何与同一层面其他构件发生作用，以及这些构件如何作为一个整体构成更高级别的构件。由此，我们就可以理解该系统中与手头任务有关的那部分，而不会迷失在细节的海洋里。

需要强调的是，抽象并不局限于科学和技术领域。它是一门重要的简化技术，我们的社会形成的任何一种生活方式都离不开抽象。很少有人知道，日常生活中各种各样的便利是如何实现的：我们需要吃饭穿衣，但我们自己无法生产；我们使用电气设备和通信系统，但不了解其底层技术；我们享受其他人提供的服务，但不知道其专业细节。对于每一项新的发展，只有一小部分社会成员专职于其实现，其他人则将实现的结果作为抽象工具来使用。这样，社会的抽象工具仓库扩大了，社会进一步发展的能力也增强了。

抽象这一话题在本书中会被反复提及。我们将了解到，计算设备是通过各个层次的抽象工具构建的。我们还会看到，大型软件系统的开发是以模块化的方式完成的，其中每个模块都是较大模块中的一种

抽象工具。此外，在计算机科学本身的发展中，抽象也扮演了很重要的角色，有了它，研究人员可以把精力集中在一个复杂领域中的特定范围上。实际上，本书的编排也反映了计算机科学的这种特征：每一章都围绕着计算机科学的一个特定的方面，而且往往出人意料地独立于其他章，但所有这些章合在一起，又形成了这一巨大研究领域的全面概述。

0.4.3 创新

虽然计算机可能只是复杂的机器，机械地执行着机械式算法指令，但是我们应该看到，计算机科学领域本质上是一个创新性的领域。发现并应用新算法是人类的一项活动，这项活动取决于我们天生的用工具解决我们周围世界中的问题的欲望。计算机科学不仅扩展了表示形式，使其跨越了视觉、语言和音乐艺术，而且还让新的数字表示模式遍及了现代世界。

创建大型软件系统不太像照菜谱做菜，更像是构思一个宏大的新雕塑。构思雕塑的形式和功能需要仔细的规划。制造它的元件需要时间，需要注意细节，还需要熟练的技能。最终的产品体现了设计美学及其创造者的情感。

0.4.4 数据

计算机能表示任何可以被离散化或数字化的信息。算法可以用各种令人眼花缭乱的方式，处理或转换这种数字表示信息。因此，计算机算法不仅能将计算机的一部分数字数据与另一部分混洗，还能让我们搜索模式、创造模拟，通过关联连接产生新知识和新见解。海量存储容量、高速计算机网络以及强大的计算工具，推动着科学、工程和人文领域中许多其他学科的发现。无论是通过模拟复杂蛋白质折叠来预测一种新药的治疗效果，统计分析横跨数个世纪的数字化图书的语言的演化，还是渲染通过非入侵式医学扫描获得的内脏的3D图像，数据都在驱动着现代发现超越人类自身的能力。

在本书中，我们会探讨一些有关数据的问题，包括：

- 计算机是如何存储那些与常见的数字人工制品有关的数据（如数字、文本、图像、声音和视频）的？
- 计算机是如何粗略估计那些现实世界中模拟人工制品的数据的？
- 计算机是如何检测和避免数据中的错误的？
- 我们现在所掌控的这个由日益增长的、互连的数据构成的数字宇宙，最后会变成什么样子？

0.4.5 程序设计

尽管现在涌现的可用语言和工具，与20世纪50年代及20世纪60年代早期的可编程计算机没什么相似之处，但是将人类的意图翻译成可执行的计算机算法的这种行为，现在被广泛称为[程序设计/编程](#)（programming）。虽然计算机科学的组成部分并不只有计算机程序设计，还包括许多其他方面，但是通过设计可执行算法（程序）解决问题的能力依然是所有计算机科学家的一项基本技能。

计算机硬件只能执行相对简单的算法步骤，但有了计算机程序设计语言提供的抽象，人类就能针对复杂得多的问题，进行推理并制定出编码解决方案。下面这几个关键的问题为我们这个主题的讨论提供了框架。

- 如何构建程序？
- 程序中会出现哪些类型的错误？
- 如何发现和修复程序中的错误？
- 现代程序中的错误会产生什么影响？
- 如何对程序进行文档化和评估？

0.4.6 因特网

因特网连接着全世界的计算机和电子设备，这对我们这个技术社会存储、检索和共享信息的方式产生了深远的影响。现在，商业、新

闻、娱乐和通信都越来越依赖这个由较小的计算机网络组成的互联网络。我们的讨论不仅限于把因特网的机制描述为人工制品，还会涉及人类社会业已被全球网络交织在一起的许多方面。

因特网的覆盖对我们的隐私和个人信息的安全也有着深远的影响。网络空间里有很多危险，所以在我们这个互联的世界里，密码学和网络安全正变得越来越重要。

0.4.7 影响

计算机科学不仅对我们用于通信、工作和娱乐的技术有深远的影响，对我们的社会生活也有巨大的影响。计算机科学的进步正在淡化许多差别，而这些差别正是我们过去做出某些决策的基准；计算机科学的进步也向许多长久以来的社会准则提出了挑战。在法律上，它产生了某些疑问——知识产权的度以及伴随这个所有权的权利和义务。在道德上，人们面临着许多挑战传统社会行为准则的抉择。对于政府，又产生了许多争议——计算机技术及其应用应该规范到什么程度？在哲学上，人们开始争论智能行为的存在与智能本身的存在。同时，整个社会也在争论新的计算机应用是代表的是新的自由还是新的控制。

对于那些想涉足计算或者计算机相关领域的人，这种话题是很重要的。科学中的新发现有时会使许多应用产生争议，这使人们对相关

的研究人员产生极大不满。进一步而言，道德上的过错足以摧毁本可以很成功的事业。

计算机技术的发展给人们提出了许多难题，而具备解决这些难题的能力对于非计算机领域的人也十分重要。的确，计算机技术已经在全社会迅速普及，几乎无人不受其影响。

本书提供了一些技术背景，有助于人们以一种理智的思维来处理计算机科学所产生的问题。然而，计算机科学的技术知识本身无法提供全部问题的解决办法。因此，本书的一些章节致力于介绍计算机科学的社会、道德和法律影响，包括安全问题、软件所有权和义务问题、数据库技术的社会影响以及人工智能发展的后果。

此外，一个问题往往没有明确正确答案，许多有效的解决方案都是在两个对立的（也许都是有理的）观点之间进行折中的。在这些情况下寻找解决方案通常需要能够倾听、辨别其他观点、开展理性的讨论，并在获得新见解时改变自己的观点。因此，本书的每一章最后都会在“社会问题”这样一节收集一些问题，研究一系列计算机科学和社会之间的关系。这些问题不一定是需要回答的，而是需要思考的。在许多情况下，当人们发现其他答案时，就不会再满意于那个最先出现的明显答案了。简而言之，给出这些问题的目的并不是让大家找到“正确”答案，而是要提高大家的意识：要意识到一个问题会牵扯多位利益相关者，一个问题会有多个解决方案，那些解决方案都同时具有长短期效应。

哲学家在基础理论的研究中提出了许多伦理学方法，从而产生了指导决策和行为的原则。

性格伦理（有时称为德行伦理）是由柏拉图和亚里士多德提出的，他们认为“好行为”不是应用可识别规则的结果，而是“良好性格”的自然结果。然而其他伦理基础（如结果伦理、职责伦理和合同伦理）认为，一个人在解决伦理难题时，应该考虑的是：“后果是什么？”“我的职责是什么？”或者“我有什么合同？”而性格伦理考虑的是：“我想成为什么样的人？”因此，好行为是建立在好性格基础上的，而这通常得益于良好的教育以及德行习惯。

在向不同领域的专业人士教授伦理知识时，一般以性格伦理为基础。不用教授专门的伦理理论，只要举一些能够暴露该专业领域中各种伦理问题的案例即可。然后，通过讨论这些案例的利弊，让这些专业人士对职业生活中潜在的危险有一个更清醒、更深入和更敏感的认识，并将这种认识融入他们的性格中。这就是每章最后设计社会问题的精神所在。

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1. “如果没有计算机革命，我们现在的社会将有很大不同。”人们已经广泛接受这种观点。与没有计算机革命的社会相比，现在的社会是更好了还是更差了？如果你在社会中的地位不同，你的答案会有所不同吗？

2. 想参与到当今的技术社会中，却又不想努力了解技术的基础知识，这种做法是否可行？例如，要通过表决来决定如何支持和使用某项技术，那么表决者是否有了了解该项技术的义务？你的答案是否与正在考虑的具体技术有关？例如，在考虑使用核技术时和在考虑使用计算机技术时，你的答案一样吗？

3. 传统上，人们使用现金进行交易，因而处理账务时不需要支付服务费用。然而，随着经济生活中自动化程度不断提高，金融机构正在对这些自动化系统的使用收取服务费。那么，“服务收费不公正地限制了人们参与经济活动”这种说法是否正确呢？例如，假设雇主仅用支票支付员工工资，并且所有金融机构都对支票兑现和存款收取服务费，那么员工会因此受到不公正的待遇吗？如果雇主坚持只通过直接存款的方式支付工资呢？

4. 在交互式电视节目中，某一个公司有可能会从孩子那里获取有关其家庭的信息（也许是通过交互式游戏的形式），那应该控制到什么程度呢？例如，应该允许公司通过孩子得知其父母的购物习惯吗？关于孩子自己的信息呢？

5. 政府对计算机技术及其应用应该管制到什么程度？例如，考虑第3题和第4题中提到的问题。什么能证明政府监管是正当的？

6. 对于技术，尤其是计算机技术，我们做出的决策会对我们的后代产生多大的影响？

7. 随着技术的进步，美国的教育系统不断面临挑战，要重新考虑在哪些抽象层次上安排哪些主题。许多问题是类似的，如某项技能是否必要，是否应该允许学生依赖某种抽象工具等。学三角时，不再教学生如何利用函数表求三角函数的值，而是允许学生用计算器作为抽象工具来求函数值。有些人认为，长除也应该让位于抽象。还有哪些主题涉及类似的争论？现代字处理程序是否会使人们不需要练习书法？视频技术的使用是否会在将来的某一天取代阅读？

8. 随着越来越多的信息通过计算机技术存储和传播，是否每一位公民都应该有权使用这项技术呢？答案如果是肯定的，那么公共图书馆是否应该为这种权力提供渠道？

9. 在一个依靠抽象工具的社会里，会产生什么样的伦理问题？是否存在这样的情况，当我们使用某个产品或某项服务时，不了解它的工作原理是有悖伦理的吗？那不知道它是如何生产出来的呢？又或者，不了解其使用的副产品呢？

10. 随着我们的社会变得越来越自动化，政府监督公民的活动变得越来越容易。这是好还是坏呢？

11. 乔治·奥威尔（埃里克·布莱尔）在他的小说《1984》中想象出来的那些技术，其中哪些已经实现了？它们是以奥威尔预想的方式使用的吗？

12. 如果你有一台时间机器，你想活在哪个历史阶段？有你想带去的现代技术吗？你选择带去的技术可以脱离其他技术而被你单独带走吗？一项技术可以在多大程度上独立于其他技术？抗议全球变暖，却又接受现代医疗，这两者一致吗？

13. 假如由于工作关系，你必须生活在另一种文化氛围中。你会按照自己的本土文化的道德观习惯我行我素，还是会选择遵循当地文化的道德观？你的答案会因该问题会因跟穿衣打扮有关还是跟人权有关而不同吗？如果你是在本土文化中，但需要在因特网上与外国文化打交道，你应该遵循哪些道德标准？

14. 在商业、通信或社交互动方面，社会是否过于依赖计算机应用了？例如，如果长期中断因特网或移动电话服务，会有什么后果？

15. 大多数智能手机都能够通过GPS来确定手机的位置。这样一来，相关的应用程序就可以基于手机的当前位置提供与该位置相关的信息（如本地新闻、本地天气或者附近的商业机构）。然而，这些GPS功能可能也允许其他应用将手机的位置广播给其他各方。这样好吗？手机的位置（继而手机用户的位置）信息会被如何滥用？

课外阅读

Goldstine, H. H. *The Computer from Pascal to von Neumann*.
Princeton, NJ: Princeton University Press, 1972.

Haigh, T., M. Priestley, and C. Rope. *ENIAC in Action: Making and Remaking the Modern Computer*. Cambridge, MA: The MIT Press, 2016.

Kizza, J. M. *Ethical and Social Issues in the Information Age*, 5th ed.
London: Springer-Verlag, 2013.

Mollenhoff, C. R. *Atanasoff: Forgotten Father of the Computer*.
Ames, IA: Iowa State University Press, 1988.

Neumann, P. G. *Computer Related Risks*. Boston, MA: Addison-Wesley, 1995.

Quinn, M. J. *Ethics for the Information Age*, 7th ed. Essex, England:
Pearson, 2016.

Randell, B. *The Origins of Digital Computers*, 3rd ed. New York:
SpringerVerlag, 1982.

Spinello, R. A. and H. T. Tavani. *Readings in CyberEthics*, 2nd ed.
Sudbury, MA: Jones and Bartlett, 2004.

Swade, D. *The Difference Engine*. New York: Viking, 2000.

Tavani, H. T. *Ethics and Technology: Controversies, Questions, and Strategies for Ethical Computing*, 5th ed. New York: Wiley, 2016.

Woolley, B. *The Bride of Science: Romance, Reason, and Byron's Daughter*. New York: McGraw-Hill, 1999.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第1章 数据存储

在本章中，我们学习有关计算机中数据表示和数据存储的内容。我们要研究的数据类型包括文本、数值、图像、音频和视频。除了传统计算，本章的很多内容还涉及数字摄影、音频/视频录制和复制，以及远程通信等领域。

本章内容

- 1.1 位和位存储
- 1.2 主存储器
- 1.3 海量存储器
- 1.4 用位模式表示信息
- *1.5 二进制系统
- *1.6 整数的存储
- *1.7 分数的存储
- *1.8 数据与程序设计
- *1.9 数据压缩
- *1.10 通信差错

持久理解和学习成果

计算使人们能够使用创造性开发过程来创建用于创造性表达式的计算制品或解决问题。AP[®]CSP

学习成果 为创新性表达式创建计算制品。

根据二进制序列构建的各种抽象可用于表示所有数字数据。AP[®]CSP

学习成果 描述用于表示数据的各种抽象。

学习成果 解释如何用二进制序列来表示数字数据。

用多层抽象来编写程序或创建其他计算制品。AP[®]CSP

学习成果 确定编写程序时使用的多层抽象。

在将信息表示为数字数据时要有权衡。AP[®]CSP

学习成果 分析数据表示、存储、安全和数据传输是如何涉及信息计算操纵的。

适当的抽象能使程序设计更容易。AP[®]CSP

学习成果 使用抽象来管理程序复杂度。

人为不同目的开发、维护和使用程序。AP[®]CSP

学习成果 评估程序的正确性。

程序设计使用数学和逻辑概念。AP[®]CSP

学习成果 在程序设计中适当的数学和逻辑概念。

人类使用计算机进行创造。软件包、数字媒体、网络内容、数据集、模型和模拟——所有这些都是人们使用计算工具和抽象创建的计算机制品的示例。我们首先要学习的是，信息是如何编码和存储在计算机中的。我们的第一步是讨论计算机数据存储设备的基础知识，然后研究如何对信息进行编码才能将其存储到这些系统中。这些系统的绝对复杂性需要从最简单的电子开关构建的硬件门，到我们用于将命令编码为计算机软件的最高级程序设计语言的多层次的强大抽象。

基本知识点

- 计算机制品是人类用计算机创造出来的东西，可以是但不限于程序、图像、音频、视频、演示或者网页文件。

1.1 位和位存储

抽象允许我们表示许多种数据，但在最底层，在今天的计算机内，所有的信息是以0和1的模式编码的，这些数字称为**位**（bit，是binary digit的简写，意思是**二进制数字**）。尽管你可能倾向于把位与数值联系在一起，但它们实际上只是一些符号，其含义取决于正在处理的应用。位模式有时表示数值，有时表示字母表里的字符和标点符号，有时表示颜色和图像，还有时表示声音。

基本知识点

- 数字数据由不同层次的抽象表示。
- 在最底层，所有数字数据都由位表示。

- 在较高层，位被分组用于表示抽象，包括但不限于数、字符和颜色。
- 在抽象的最底层，数字数据只用数字0和1的组合以二进制（基数2）表示。

1.1.1 布尔运算

为了理解单个的位在计算机中是如何存储和操作的，这里我们假设位0表示假（false），位1表示真（true），这样会很方便。为了纪念数学家乔治·布尔（George Boole，1815—1864），处理真/假值的运算被称为布尔运算（Boolean operation）。乔治·布尔是逻辑数学领域的先驱。3个基本的布尔运算是AND（与）、OR（或）和XOR（异或），图1-1概述了这3种运算。（我们之所以用大写字母来表示这些布尔运算符的名字，是为了与它们对应的英语单词区分开来。）这些运算类似于算术运算的乘法和加法，因为它们都是组合一对值（运算的输入），得出第三个值（运算的输出）。不过，与算术运算不同的是，布尔运算组合的是真/假值，而不是数值。

布尔运算AND可用于计算，连接词AND和两个较小或较简单的语句组成的一条语句的真/假值。这种语句的一般形式如下：

$$P \text{ AND } Q$$

其中， P 代表一条语句， Q 代表另一条语句。例如：

科米是一只青蛙 AND 佩吉小姐是一位演员。

AND运算的输入是复合语句分句的真/假值，输出则是复合语句本身的真/假值。因为 $P \text{ AND } Q$ 形式的语句的值只有在其两个分句都是真时才为真，所以可以得出结论：1 AND 1的输出值应该是1，而其他所有情况的输出值都应该是0，如图1-1所示。

AND运算			
0	0	1	1
AND 0	AND 1	AND 0	AND 1
0	0	0	1
OR运算			
0	0	1	1
OR 0	OR 1	OR 0	OR 1
0	1	1	1
XOR运算			
0	0	1	1
XOR 0	XOR 1	XOR 0	XOR 1
0	1	1	0

图1-1 布尔运算AND、OR和XOR（异或）的可能输入值和输出值

同理，OR运算是建立在如下形式的复合语句的基础之上的：

$$P \text{ OR } Q$$

其中，同样， P 代表一条语句， Q 代表另一条语句。在至少其中一条组成语句为真时，这种语句才为真，这与图1-1中描述的OR运算一致。

英语中没有可以单独表示XOR运算含义的连词。当两个输入中的一个为1（真）、另一个为0（假）时，XOR运算的输出为1（真）。例如， $P \text{ XOR } Q$ 形式的语句的意思是“或者是 P ，或者是 Q ，但不会是两个共存。”（简言之，当两个输入不同时，XOR运算的输出为1。）

NOT（非）运算是另一个布尔运算。它与AND、OR和XOR不同，因为它只有一个输入。它的输出值与输入是相反的；如果NOT运算的输入为真，那么它的输出值就为假，反之亦然。因此，如果NOT运算的输入是下面语句的真/假值：

福齐是一只熊。

那么，其输出就是下面语句的真/假值：

福齐不是一只熊。

1.1.2 门和触发器

门（gate），有时也称**逻辑门**（logic gate），指的是一种设备，给定一种布尔运算的输入值，门可以得出该布尔运算的输出值。门可以通过很多种技术制造出来，如齿轮、继电器和光学设备。现在的计算机中，门通常是由称为晶体管的小型电子电路实现的，其中数字0和1由电压电平表示。不过，我们不需要关注这种细节问题。对我们来说，知道如何用门的符号形式来表示门就足够了，如图1-2所示。注意，与门、或门、异或门和非门，是由不同形状的符号表示的，输入值在一边，输出值在另一边。

基本知识点

- 逻辑门是用布尔函数建模的硬件抽象。

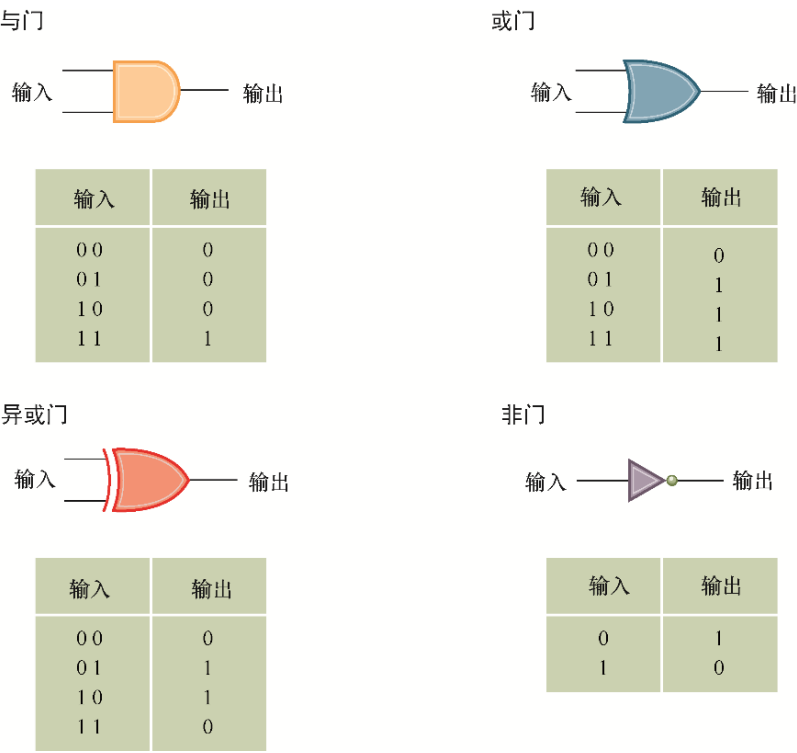


图1-2 与门、或门、异或门和非门的图形表示及其输入值和输出值

门为构造计算机提供了构建块。因此，布尔逻辑和运算符作为基本运算出现在程序设计语言中。图1-3中的电路描述了这个方向中的一个重要步骤。这是称为**触发器**（flip-flop）的元件的一个特例。触发器是计算机存储器的基本单元，它是一个可以产生0或1输出值的电路，它的值会一直保持不变，直到有另一个电路过来的脉冲（临时变成1之后再变为0）使其变换成其他值。换句话说，通过设置，可以让输出在外界刺激的控制下“记住”0或者1。例如，在图1-3中，只要电路的两个输入一直都是0，输出值（无论是0还是1）就不会变。不过，在它的上输入临时放一个1，会强制其输出为1；反之，在它的下输入临时放一个1会强制其输出为0。

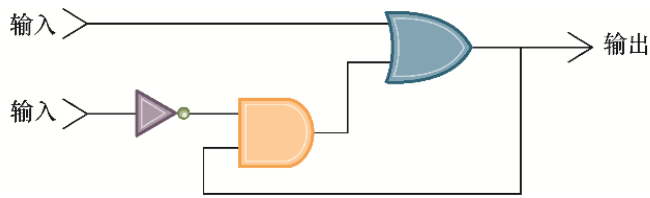


图1-3 一个简单的触发器电路

基本知识点

- 逻辑概念和布尔代数是程序设计的基础。

让我们来仔细研究一下这个问题。在我们不知道图1-3中电路的当前输出的情况下，假设上面的输入变为1，而下输入仍为0（见图1-4a），那么不管或门的另一个输入是什么，它的输出都将为1。接着，因为与门的另一个输入已经为1（触发器的下输入为0时非门产生的输出），所以它的两个输入现在都将为1。于是，与门的输出将变成1，这意味着，现在或门的第二个输入将为1（见图1-4b）。这样就可以确保即使触发器的上输入变回0（见图1-4c），或门的输出也会保持为1。总之，触发器的输出已经变为1，这个输出值将在上输入变回0后仍然保持不变。

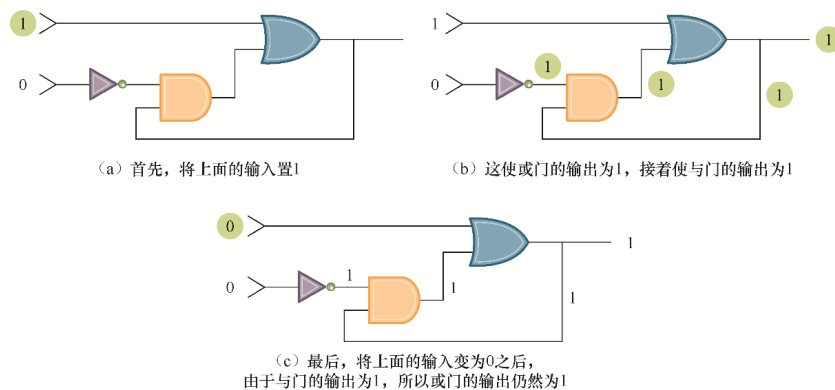


图1-4 将一个触发器的输出值设置为1

同理，在下输入上临时放一个值1会强制触发器的输出为0，并且输出在输入值变回0后保持不变。

我们介绍图1-3和图1-4中的触发器电路的目的有3个。第一，展示设备是如何通过门制造出来的，这是一个数字电路的设计过程，是计算机工程领域的一个重要课题。事实上，在计算机工程中，触发器只是诸多基础工具电路中的一种。

第二，通过触发器的概念为抽象和抽象工具的使用提供一个例子。事实上，还有很多其他的构建触发器的方法。图1-5展示了另一种方法。如果你用这个电路做实验就会发现，尽管它有不同的内部结构，但它的外部特性与图1-3中的是一样的。计算机工程师不必知晓触发器中实际使用的是哪种电路，只需理解触发器的外部特性并将其作为一个抽象工具来使用即可。一个触发器和其他定义良好的电路一起形成了一

个构件的集合，工程师可以直接利用这个构件集合构造更复杂的电路。因此，计算机电路的设计就会呈现一种层次结构，其中每层都将较低层次的构件作为抽象工具使用。

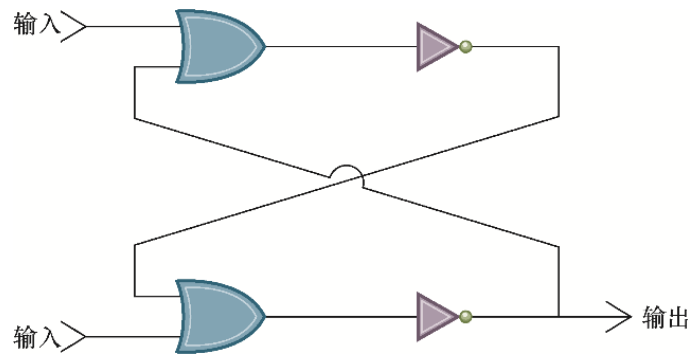


图1-5 构建触发器的另一种方法

第三，触发器是现代计算机中存储二进制位的一种方法。更确切地说，可以将触发器的输出值设置为0或1。其他电路可以通过向触发器的输入端发送脉冲来调整这个值，还有一些电路可以通过将这个触发器的输出用作它们的输入来响应存储的值。因此，许多触发器（被构造成非常小的电子电路）可以用作计算机内部记录信息的一种方法，将信息编码为0和1的模式。实际上，众所周知的[超大规模集成](#)（very large-scale integration, VLSI）技术支持将数百万个电子元件构造在一个称为[芯片](#)（chip）的半导体晶片上，用于创建包含数百万个触发器及其控制电路的微型设备。因此，这些芯片被用作构建计算机系统中较高层构件的抽象工具。事实上，在某些情况下，还可以用超大规模集成技术在单块芯片上创建整个计算机系统。

基本知识点

- 二进制数据由计算硬件（包括门、芯片和元件）的物理层处理。
- 硬件采用多层抽象构建，如晶体管、逻辑门、芯片、存储器、母板、专用卡和存储设备。

1.1.3 十六进制记数法

考虑计算机的内部活动时，我们必须考虑位模式的处理问题，我们将位模式称为位串，有些位串相当长。长位串常被称为[流](#)（stream）。遗憾的是，人脑很难理解流。即便只是抄录位模式101101010011也会令人厌烦，且容易出错。因此，为了简化这种位模式的表示，我们常使用一种称为[十六进制记数法](#)（hexadecimal notation）的简写表示法，它是利用机器内的位模式的长度为4的倍数这样一个事实制定的。具体来说就是，十六进制记数法用一个符号表示位模式的4位。例如，一个12位的串只需要3个十六进制符号就可以表示。数学家一般使用下标来表示非十进制数的基数，因此可能会将 15_{10} 的十六进制值写作 F_{16} 。习惯用不支持下标的基于文本的语言进行程序设计的计算机科学家通常使用前缀来表明非十进制数的基数。为本书简洁起见，我们也将十六进制数的前面使用公共前缀“0x”。

图1-6展示了十六进制编码系统。左边一列是所有可能的长度为4的位模式，右边一列是用于表示其左边位模式的十六进制记数法符号。使用这个系统，可以将位模式10110101表示为0xB5。具体方法是：把这个位模式拆分为长度为4的子串，然后用对应的十六进制符号代替每个子串，即用0xB表示1011，用0x5表示0101。以同样的方式，可以将16位模式1010010011001000简化成更易为人接受的形式0xA4C8。我们将在第2章中大量使用十六进制记数法，到时候你就能体会到它的效率了。

位模式	十六进制表示
0000	0x0
0001	0x1
0010	0x2
0011	0x3
0100	0x4
0101	0x5
0110	0x6
0111	0x7
1000	0x8
1001	0x9
1010	0xA
1011	0xB
1100	0xC
1101	0xD
1110	0xE
1111	0xF

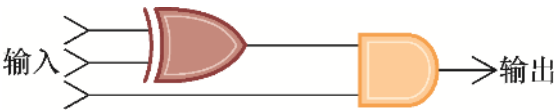
图1-6 十六进制编码系统

基本知识点

- 之所以用十六进制（基数16）表示数字数据是因为十六进制表示使用的数位比二进制的少。

1.1节问题与练习

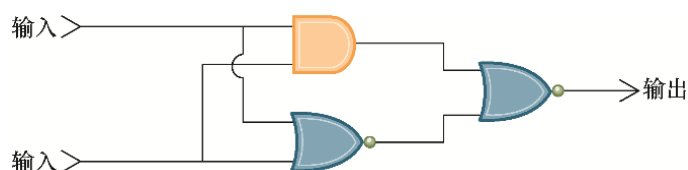
1. 什么样的输入位模式会使下面的电路产生输出1？



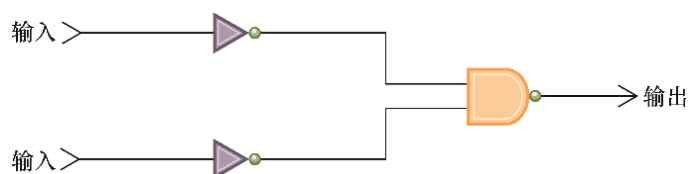
2. 在正文中，我们断言，在图1-3中的触发器的下输入放一个1（同时保持上输入为0）会迫使触发器的输出为0。描述一下这种情况下触发器内部发生的事件序列。

3. 假定图1-5中的触发器的两个输入都以0开始，描述一下当上输入被临时设为1时发生的事件序列。

4. a. 如果一个与门的输出值传递给了一个非门，那么这个组合电路计算的布尔运算称为与非（NAND），当且仅当其两个输入都为1时，其输出为0。与非门的符号和与门的符号类似，只是在输出端有一个圆圈。下面的电路包含一个与非门。这个电路完成的是什么布尔运算？



b. 如果一个或门的输出值传递给了一个非门，那么这个组合电路计算的布尔运算称为或非（NOR），当且仅当其两个输入都为0时，其输出为1。或非门的符号和或门的符号类似，只是在输出端有一个圆圈。下面的电路包含一个与门和两个或非门。这个电路完成的是什么布尔运算？



5. 用十六进制记数法来表示下面的位模式。

a. 0110101011110010 b. 111010000101010100010111 c. 01001000

6. 下面的十六进制模式表示的是什么位模式？

a. 0x5FD97 b. 0x610A c. 0xABCD d. 0x0100

1.2 主存储器

为了存储数据，计算机包含大量的电路（如触发器），每一个电路能够存储一个位。这种位存储器被称为计算机的**主存储器**（main memory）。

1.2.1 存储器结构

计算机的主存储器是由称为**存储单元**（cell）的可管理单元组成的，一个典型存储单元的容量为8位。因为一个8位的串称为一个**字节**（byte），所以一个典型存储单元的容量是一个字节。嵌在家用电器（如恒温器和微波炉）中的小型计算机的主存储器可能仅有几百个存储单元，但台式计算机和智能手机的主存储器可能有数十亿个存储单元。

虽然计算机中没有左或右的概念，但是我们通常假设存储单元的位是排成一行的。该行的左端称为**高位端**（high-order end），右端称为**低位端**（low-order end）。高位端的最左边的一位称作高位或**最高有效位**（most significant bit）。取这个名称是因为，如果把存储单元里的内容解释为数值，那么这一位就是该数的最高有效位。类似地，低位端的最右边的一位称为低位或**最低有效位**（least significant bit）。因此，我们可以像图1-7所示的那样描述字节型存储单元的内容。

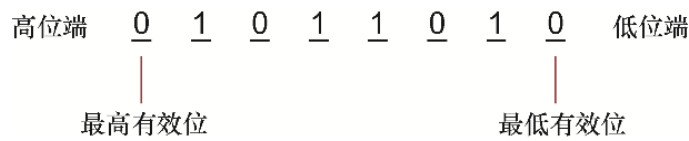


图1-7 字节型存储单元的组织

为了区分计算机的主存储器中的各个存储单元，每个存储单元都被赋予了一个唯一的“名字”，称为**地址**（address）。这类似于通过地址找到城市里的一座座房屋。不过，存储单元中的地址都是用数值表示的。更准确地说，我们把所有的存储单元都看作是排成一行的，并按照这个顺序从0开始编号。这样的编址系统不仅为我们提供了唯一标识每个存储单元的方法，而且给存储单元赋予了顺序的概念（见图1-8），这样就有了诸如“下一个单元”或“上一个单元”的说法。

将主存储器中的存储单元和每个存储单元中的位都进行排序，会产生一个重要的结果：计算机的主存储器中的所有二进制位会实际排成一长行。这个长行上的片段可以存储的位模式因此比单个存储单元要长。具体来说，我们只需要两个连续的存储单元就可以存储16位的串。

为了做成一台计算机的主存储器，实际存放二进制位的电路还组合了其他电路，这个电路使得其他电路可以在存储单元中存入和取出数据。以这种方式，其他电路可以通过电信号请求从存储器中得到指定地址的内容（称为读操作），或者通过请求把某个位模式存放到指定地址的存储单元里（称为写操作）。

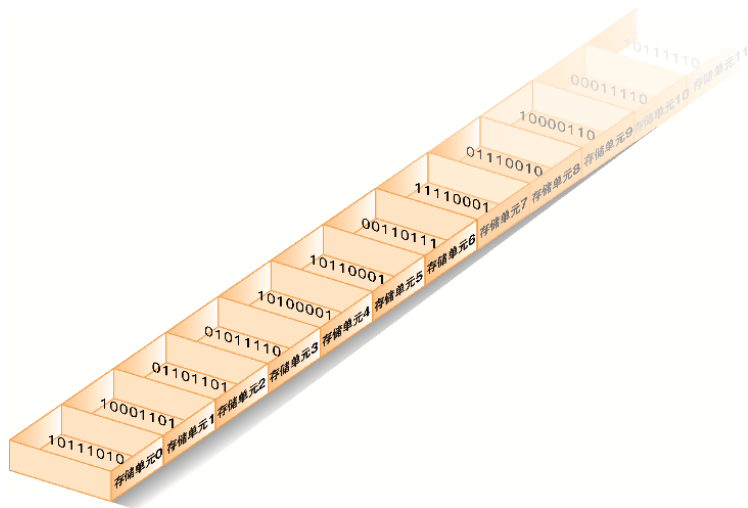


图1-8 按地址排列的存储单元

因为计算机的主存储器是由独立的、可编址的存储单元组成的，所以可以根据需要独立访问这些存储单元。为了体现用任何顺序访问存储单元的能力，计算机的主存储器常被称为**随机存取存储器**（random access memory，RAM）。主存储器的这种随机存取特性与1.3节中要讨论的海量存储系统形成了鲜明的对比，在海量存储系统中，长位串被当作合并块来操控。

尽管我们介绍说，触发器是存储二进制位的一种方法，但是在现代的大多数计算机中，RAM都是用其他类似的更复杂的技术制造的，这些技术可以让RAM更小型化、响应时间更短。其中许多技术将位存储为微小的快速消散的电荷。因此，这些设备需要附加电路（称为刷新电路），在一秒内反复补充电荷很多次。因为它的这种易失性，通过这种技术构造的计算机存储器常被称为**动态存储器**（dynamic memory），于是就产生了术语**DRAM**（读作“DEE-ram”），用来表示动态RAM（dynamic RAM）。有时动态存储器会用术语**SDRAM**（读作“ES-DEE-ram”）来表示同步DRAM（synchronous DRAM），采用这种附加技术可以缩短从存储单元中获取信息所需的时间。

1.2.2 存储器容量的度量

在第2章中我们会了解到，如果主存储器中存储单元的总数是2的幂，那么主存储器设计起来会很方便。因此，早期计算机存储器的大小通常以1024（即 2^{10} ）个存储单元为单位来度量。因为1024接近于值1000，所以计算机界采用前缀**千**（kilo）来表示这个单位。也就是说，术语**千字节**（kilobyte，符号表示为KB）被用于表示1024字节。因此，带有4096个存储单元的机器会被说成有一个4 KB的存储器（ $4096 = 4 \times 1024$ ）。随着存储器容量变得越来越大，这个术语逐渐扩大到了兆字节（megabyte，符号表示为MB，又称百万字节）、吉字节（gigabyte，符号表示为GB，又称十亿字节）和太字节（terabyte，符号表示为TB，又称万亿字节）。遗憾的是，这种前缀**千**（kilo-）、**兆**（mega-）等的用法属于术语的误用，因为这些前缀已经被其他领域用于指称1000的幂。例如，在度量距离时，**千米**（kilometer）指的是1000米，在度量无线电频率时，**兆赫**（megahertz）指的是1 000 000 Hz。在20世纪90年代后期，国际标准组织为2的幂制定了专用术语：**千位字节**（kibi-byte）、**兆位字节**（mebi-byte）、**吉位字节**（gibi-byte）和**太位字节**（tebi-byte），用来表示1024的幂，而不是1000的幂。然而，尽管这种区别在世界上许多地方的当地法律里都有规定，但一直以来，普通大众和许多计算机科学家都不愿意放弃这个已经比较熟悉但会引起歧义的“兆字节”（megabyte）。因此，提醒大家：一般说来，**千**、**兆**等术语在涉及计算机度量时表示2的幂，但在其他语境中表示1000的幂。

1.2节问题与练习

1. 如果地址为5的存储单元包含值8，那么“将值5写入6号存储单元”和“将5号存储单元的内容移到6号存储单元”之间的区别是什么？

2. 假定你想交换存储在2号存储单元和3号存储单元中的值。那么下面的步骤错在哪里？

步骤1：把2号存储单元中的内容移到3号存储单元。

步骤2：把3号存储单元中的内容移到2号存储单元。

3. 请设计能够正确交换这两个存储单元内容的步骤。如有必要,可以使用额外的存储单元。
4. 一台带有4 KB存储器的计算机,其存储器里有多少个二进制位?

1.3 海量存储器

由于计算机的主存储器的易失性和容量的限制,大多数计算机都有称为**海量存储**(mass storage,或者二级存储)系统的附加存储设备,包括磁盘、CD盘、DVD盘、磁带、闪存驱动器和固态驱动器(所有这些我们稍后会讨论)。相对于主存储器,海量存储系统的优点是更稳定、容量大、价格低,并且在许多情况下,能从机器上取下存储介质进行存档。

磁性和光学海量磁存储系统和海量光存储系统的主要不足是,它们一般都需要机械运动。因为主存储器的所有工作都是由电子元件实现的,所以比起计算机的主存储器来,海量存储系统的数据存取需要花费更长的时间。此外,与固态系统相比,带有移动部件的存储系统更容易出现机械故障。虽然闪存驱动器和固态硬盘不需要移动部件,但其他电子方面的考虑会限制其相对于主存储器的速度和寿命。

基本知识点

- 存储介质的选择对操纵它包含的数据的方法和成本都有影响。

1.3.1 磁盘系统

多年来,磁技术已经占据了海量存储领域。最常见的例子便是我们现在使用的**磁盘**(magnetic disk)或者**硬盘驱动器**(hard disk drive, HDD),它里面是可以旋转的薄盘片,表面有用以存储数据的磁介质涂层(见图1-9)。读/写磁头安装在盘片的上面和/或下面,当盘片旋转时,每个磁头都能在盘片上面或下面相对于称为**磁道**(track)的圆圈转动。通过重定位读/写磁头,可以对各个同心的磁道进行存取。在很多情况下,一个磁盘存储系统由安装在公共主轴上的若干盘片组成,两个盘片之间有足够的空间供读/写磁头滑动。在这种情况下,所有读/写磁头是一起移动的。每次读/写磁头重定位时,都可以访问一组新磁道,这组磁道称为**柱面**(cylinder)。

因为一个磁道可以包含的信息通常比我们每次想要处理的多,所以每个磁道又被划分成若干个称为**扇区**(sector)的小弧区,这些扇区上记录的信息是连续的二进制位串。磁盘上所有的扇区都包含相同数目的二进制位(典型的容量是512字节到若干KB),而且在最简单的磁盘存储系统里,每个磁道包含的扇区数都相同。因此,靠近盘片外边缘的磁道扇区上的位的存储密度,要比靠近盘片中心的磁道上的小,因为外磁道比内磁道长。相反,在大容量磁盘存储系统里,靠近外边缘的磁道包含的扇区要远多于靠近中心的磁道,这种存储能力常通过一种称作**区位记录**(zoned-bit recording, ZBR)的技术得以应用。在使用区位记录技术时,一些相邻的磁道被统称为区,一个典型的盘片大约包含10个区。一个区内的所有磁道都有相同数目的扇区,但是靠外的区中每个磁道包含的扇区数,比靠内的区中每个磁道包含的扇区数多。采用这

种方式，能够有效利用整个磁盘的表面。不管细节如何，一个磁盘存储系统都包含许多独立的扇区，每个扇区都可以作为一个独立的位串进行存取。

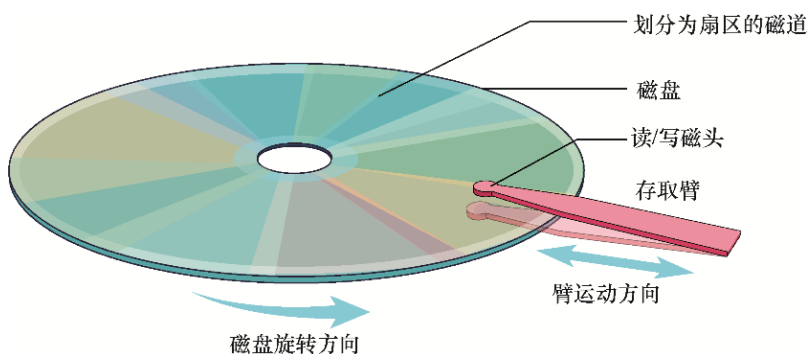


图1-9 磁盘存储系统

一个磁盘存储系统的容量取决于使用的盘片数以及磁道与扇区的划分密度。容量较小的系统可能只有一个盘片，尤其是在硬盘驱动器的物理尺寸必须保持紧凑的情况下。大容量磁盘系统的容量可达数TB，由安装在公共主轴上的3~6个盘片组成。此外，数据有可能存储在每个盘片的上下两面。

有4个标准可以用来评估一个磁盘系统的性能：（1）**寻道时间**（seek time）（读/写磁头从一个磁道移到另一个磁道所需的时间）；（2）**旋转延迟**（rotation delay）（盘片旋转一周所需时间的一半，也就是读/写磁头定位到期望磁道后期望数据旋转到磁头处所需的平均时间）；（3）**存取时间**（access time）（寻道时间和旋转延迟之和）；（4）**传输速率**（transfer rate）（数据传入或传出磁盘的速率）。（注意，在使用区位记录的情况下，盘片旋转一次，外区磁道通过读/写磁头的数据量要多于内区磁道，因此，数据传输速率会随使用的盘片部位的不同而有所变化。）其他用于评估海量存储器和更一般的通信系统的重要性能度量指标有：（1）**带宽**（bandwidth）（单位时间内传输的总位数，如每秒3兆位）；（2）等待时间（latency）（从请求数据传输到数据到达的总时间）。

限制磁盘存取时间和传输速率的一个因素是磁盘系统旋转的速度。为了支持高速旋转，这些系统里的读/写磁头并不接触盘片，而是恰好“悬浮”在盘片表面。磁头与盘片之间的空间很小，以至于一粒小小的灰尘都可能卡在其中，导致磁头和盘片损坏，这一现象便是磁头划伤（head crash）。因此，磁盘系统出厂时都密封在盒子里。凭借这样的构造，磁盘系统能够以每秒几百次的速度旋转，达到每秒数以MB的传输速率。

因为磁盘系统的操作需要物理运动，所以难以与电子电路的速度相比。电子电路的延迟时间是以纳秒（十亿分之一秒）甚至更小的时间单位度量的，而磁盘系统的寻道时间、等待时间和存取时间是以毫秒（千分之一秒）度量的。因此，与电子电路等待结果的时间相比，从磁盘系统检索信息所需要的时间非常长。

磁存储技术现在很少使用了，包括**磁带**（magnetic tape）和**软盘驱动器**（floppy disk drive）。磁带是将信息记录在一条绕在卷轴上的薄塑料带的磁涂层上，软盘驱动器则是将带有磁涂层的单个盘片封在一个

为了便于从驱动器中取出而设计的便携式的盒子里。磁带驱动器的寻道时间极长（高等待时间），它和录音带、录像带一样，都要花费很长的倒带时间和快进时间。不过，低成本和大数据容量使磁带仍然适用于那些数据主要被线性读或写的应用，如存档数据备份。虽然软盘盘片的可移动特性是以比硬盘盘片低得多的数据密度和存取速度为代价的，但是，在更大容量、更耐用的闪存驱动器诞生之前的几十年里，软盘盘片的便携性还是极其有价值的。

基本知识点

- 系统带宽是位速率的度量单位，位速率是一个固定时间内发送的数据量（用位度量）。
- 系统等待时间是传输和收到请求之间所经过的时间。

1.3.2 光系统

另一类海量存储系统应用的是光技术。**光盘**（compact disk，CD）就是其中的一种。光盘的直径为12厘米（大约5英寸），由涂着光洁保护层的反射材料制成。光盘上的信息是通过在反射层上创建偏差的方法记录的，可以通过激光检测CD快速旋转时反射层的不规则反射偏差来读取。

CD技术最初是用于音频记录的，采用的记录格式叫作**数字音频光盘**（compact disk-digital audio，CD-DA），而现在的计算机数据存储所用的CD实质上使用的是相同的格式。具体来说，CD上的信息存储在一条磁道上，呈螺旋形缠绕在CD上，很像老式唱片上的凹槽，但与老式唱片不同的是，CD上的磁道是由内至外的（见图1-10）。这条磁道被划分为称为扇区的单元，每个扇区都有自己的标识，数据存储容量为2 KB，这个容量相当于录制音频情况下的1/75秒的音乐。

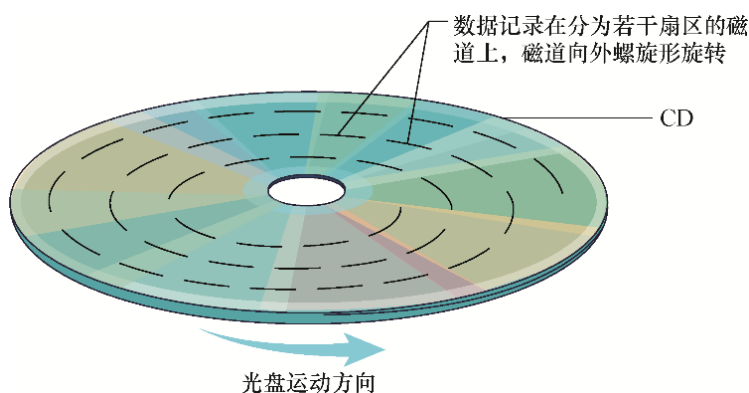


图1-10 CD存储格式

需要注意的是，盘片外边缘螺旋形磁道的距离比内部螺旋形磁道的要长。为了使CD的存储能力达到最大，信息是以均匀的线性密度存储在整个螺旋形磁道上的，这意味着，在螺旋形磁道上，外部边缘环道存放的信息比内部环道的多。所以，如果盘片旋转一整圈，激光在扫描外部螺旋形磁道时，读到的扇区个数要比扫描内部时读到的多。因此，为了获得统一的数据传输速率，CD-DA播放器要能够根据激光在盘

片上的位置来调整盘片的旋转速度。不过，因为用于计算机数据存储的大多数CD系统都以较快的恒定速度旋转，所以必须适应数据传输速率的变化。

这种设计决策的结果是，CD存储系统在处理长且连续的数据串（如音乐复制）时，表现最好。但是，当一个应用需要随机存取数据项时，磁盘存储器所用的方法（单个的同心磁道被划分成独立存取的扇区）就优于CD所用的螺旋形方法。

传统CD的存储容量是600 ~ 700 MB。但是，**数字多功能光碟**（digital versatile disk，DVD）的存储容量能达到几GB，它由多个半透明层构成，这些层在被精确聚焦的激光查看时充当不同的表面。这种盘片能够存储冗长的多媒体信息，包括完整的电影。最后，蓝光技术（blu-ray technology）使用蓝紫色（而非红色）光谱中的激光，能够极为精确地聚焦激光束。因此，单层**蓝光光碟**（blu-ray disk，BD）的容量是DVD的5倍多，较新的多层BD格式容量能够达到100 GB。这个看似巨大的存储容量能满足超高清视频的需要。

1.3.3 闪存驱动器

基于磁技术或光技术的大量存储系统的一个共同特征是，需要通过物理运动（如旋转磁盘、移动读写磁头和瞄准激光束）来存储和读取信息。这就意味着，其数据的存储和读取速度比电子电路的要慢。**闪存**（flash memory）技术有克服这个缺点的潜力。在闪存系统里，二进制位是由电子信号直接发送到存储介质中的，电子信号使得该介质中二氧化硅的微小晶格截获电子，从而改变小的电子电路的性质。因为这些微小晶格能够在没有外力的情况下保持截获的电子很多年，所以闪存技术非常适合存储便携、非易失的数据。

尽管存储在闪存系统里的数据能够像在RAM应用中一样，以小的字节型单元存取，但是当前的技术规定存储的数据应批量擦除。不过，反复的擦除会逐渐损坏二氧化硅晶格，这就意味着，当前的闪存技术不适合一般的主存储器应用，因为主存储器的内容可能一秒就改很多次。然而，在那些改变可以被控制在一个合理水平的应用（如数码相机和智能手机）中，闪存已经成为大量存储技术的一个选择。的确，因为闪存对物理震动不敏感（与磁系统和光系统相比），所以它现在正在代替便携式应用（如笔记本电脑）中的其他大量存储技术。

闪存设备称为**闪存驱动器**（flash drive），容量可达几百GB，可用于一般的大量存储应用。这些闪存设备被封装在越来越小的塑料盒子里，其一端有一个可以取下的帽，当驱动器处于脱机状态时，可以保护这个设备的电子连接器。因为这些便携设备容量大，并且很容易与计算机连接或断开，所以是理想的便携式数据存储器。不过，它们微小存储晶格的易损性决定了它们不像光盘那样可靠，不适合真正的长期应用。

较大的闪存驱动器称为**固态驱动器**（solid-state drive，SSD），是专为替代磁硬盘设计的。与硬盘相比，固态驱动器防震抗摔性能更好，能更安静地运行（因为没有移动部件），存取时间更短。不过，固态

驱动器要比同等大小的硬盘贵，所以在购买计算机时，它仍然被看作高端选择。对于移动应用，如笔记本电脑和智能手机，固态驱动器有明显的优势。虽然固态驱动器扇区也受所有闪存技术所共有的寿命比较有限的影响，但通过**耗损均衡**（wear-leveling）技术频繁地将改变的数据块重定位到驱动器的新位置上，可以减小这个影响。

闪存技术的另一应用是**安全数字**（secure digital，SD）**存储卡**，即**SD存储卡**（简称SD卡）。SD卡的容量高达2 GB，它们被制成塑料封装的晶圆，有邮票大小（还有更小的小型SD卡和微型SD卡），其中，**SD高容量**（SD high capacity，SDHC）**存储卡**，即**SDHC存储卡**（简称SDHC卡）的存储容量可以高达32 GB，新一代的**SD扩展容量**（SD extended capacity，SDXC）**存储卡**，即**SDXC存储卡**（简称SDXC卡）的容量可超过1 TB。由于这些卡物理结构紧凑，可以方便地插入小型电子设备的插槽，因此它们是数码相机、音乐播放器、汽车导航系统，以及其他许多电子产品的理想选择。

1.3 节问题与练习

1. 我们可以从加快磁盘或CD转速中获得什么？
2. 当记录数据到多盘片存储系统时，我们是应该写满一张盘片后再写另一张盘片，还是应该写满一个柱面后再写另一个柱面？
3. 为什么预订系统里的那些需要经常更新的数据应该存储在磁盘里，而不是CD或DVD里？
4. 哪些因素能使同一个驱动器能读包含CD、DVD以及蓝光光碟在内的所有光碟？
5. 相对于本节介绍的其他海量存储系统，闪存驱动器有什么优势？
6. 让磁硬盘驱动器仍具竞争力的优势是什么？

1.4 用位模式表示信息

在研究了位存储的技术后，现在来了解一下如何将信息编码为位模式。我们将集中学习一些流行的文本编码方法、数值数据编码方法、图像编码方法以及声音编码方法。每个编码系统都可能会影响到典型的计算机用户。我们的目标是充分了解这些技术，以便知道应用这些技术的效果。

1.4.1 文本的表示

文本形式的信息通常通过代码方式表示，其中文本中的每个不同的符号（如英文字母和标点符号）均被赋予唯一的位模式。这样，文本就表示为一个长的位串，位串中的连续位模式表示的是原文本中的连续符号。

在20世纪的40年代和50年代，人们设计了许多这样的代码，并结合不同的设备使用，随之增加了不少通信问题。为了缓解这种情况，**美国国家标准化学会**（American National Standards Institute，ANSI，读作“AN-see”）采用了**美国信息交换标准码**（American Standard Code for Information Interchange，

ASCII)。这种代码使用长度为7的位模式来表示大小写英文字母、标点符号、数字0~9以及某些控制信息（如换行、回车和制表符）。后来，ASCII码通过在每个7位位模式的最高端添加一个0，扩展为8位位模式。这个技术不仅产生了代码（其中代码的每个位模式都能方便地存入典型的字节型的存储单元中），还提供了128个附加位模式（通过给附加的位赋予值1），可以用于表示除英语字母和关联的标点符号之外的符号。

美国国家标准化学会

美国国家标准化学会（ANSI）成立于1918年，是由一些工程师协会和政府机构联合创办的非营利性联盟，致力于协调私人部门自发标准的发展。现在，ANSI成员中有1300多个企业、专业组织、行业协会和政府机构。ANSI的总部设在纽约，它是美国在ISO的代表。

其他国家/地区的类似组织包括澳大利亚标准组织（Standards Australia）、加拿大标准委员会（Standards Council of Canada）、中国国家质量技术监督局（China State Bureau of Quality and Technical Supervision）、德国标准化学会（Deutsches Institut für Normung）、日本工业标准调查会（Japanese Industrial Standards Committee）、墨西哥标准指导委员会（Dirección General de Normas）、俄罗斯国家标准计量委员会（State Committee of the Russian Federation for Standardization and Metrology）、瑞士标准化协会（Swiss Association for Standardization）和英国标准学会（British Standards Institution）。

8位位模式的一部分ASCII码在附录A中给出。利用这个附录，我们可以将位模式

01001000	01100101	01101100	01101100	01101111	00101110
----------	----------	----------	----------	----------	----------

解码为消息“Hello.”，如图1-11所示。

国际标准化组织（International Organization for Standardization）简称ISO，这个简称来源于希腊语中的“isos”一词，意思是平等。ISO开发了大量的ASCII扩展，每种扩展都是针对某一主要语种设计的。例如，其中一个标准提供了表达大部分西欧语言文本所需的符号。在其128个附加模式中，有表示英镑和德语元音ä、ö、ü的符号。

01001000	01100101	01101100	01101100	01101111	00101110
<i>H</i>	<i>e</i>	<i>l</i>	<i>l</i>	<i>o</i>	<i>.</i>

图1-11 消息“Hello.”的ASCII或UTF-8编码

ISO——国际标准化组织

国际标准化组织（常称为ISO）建立于1947年，现在它的总部设在瑞士日内瓦，有100多个成员团体和许多通信成员。（通信成员不能直接参与标准的开发，但可以了解ISO的活动。）

ISO扩展的ASCII标准在支持全世界多语言通信方面取得了巨大进展，但是，仍有两个主要障碍。首先，扩展的ASCII中额外可用的位模式数不足以容纳许多亚洲语言和一些东欧语言的字母表。其次，因为一个特定文档只能使用一个选定标准中的符号，所以无法支持包含不同语种的语言文本的文档。实践证

明，这两点严重妨碍了ASCII码的国际化使用。为弥补这一不足，Unicode在一些主要软硬件厂商的合作下诞生了，并迅速赢得了计算界的支持。这种代码采用唯一的21位模式表示每个符号。当Unicode字符集与Unicode转换格式8位（Unicode transformation format 8-bit，UTF-8）编码标准结合在一起时，原来的ASCII字符仍然可以用8位来表示，而像汉语、日语和希伯来语这样的语言所产生的数以千计的其他字符则可以用16位来表示。除了可以表示世界上所有常用语言所需的字符，UTF-8的24位或32位模式还可以表示比较鲜为人知的Unicode符号，为未来的扩展留出了充足的空间。

由一长串根据ASCII或Unicode编码的符号组成的文件常称为文本文件（text file）。区分下面两类文件很重要：一类是由称为文本编辑器（text editor，常简称为编辑器）的实用程序操作的简单文本文件；一类是由字处理程序（word processor），如微软的Word，产生的较复杂的文件。两者都是由文本材料组成的，但是，文本文件只包含文本中各个字符的编码，而由字处理程序产生的文件还包含表示字体变化、对齐信息等的附加结构。

1.4.2 数值的表示

当所记录的信息是纯数值的时，以字符编码的形式存储信息效率就会很低。为了了解其中的原因，让我们来看看值25的存储问题。如果我们坚持用ASCII编码符号来存储，每个符号1字节，那么总共需要16个二进制位。此外，用16个二进制位可以存储的最大数是99。不过，我们马上就可以看到，使用二进制记数法（binary notation），16个二进制位可以存储0~65535范围内的任何一个整数。因此，二进制记数法（或它的变体）被广泛用于计算机存储器中数值数据的编码。

二进制记数法是一种只使用数字0和1来表示数值的方法，与传统的用数字0、1、2、3、4、5、6、7、8和9表示数值的十进制记数系统不同。我们将在1.5节中更详细地学习二进制系统，现在只需要初步了解该系统。为此，我们来考虑一种老式的汽车里程表，它的显示轮只包含数字0和1，而不是传统的十进制数字0~9。里程表以全0读数开始，在汽车行驶的前几公里，最右方的滚动显示轮从0旋转至1；当这个1旋转回0时，会使其左边出现一个1，产生模式10；接着，右边的0旋转至1，产生模式11；现在，最右边的数从1旋转回0，使得它左边的1也旋转回0，这就使另一个1出现在第三位上，产生模式100。简言之，在我们驾驶汽车时，将看到下列顺序的里程表读数：

0000
0001
0010
0011
0100
0101
0110
0111
1000

这个序列包括了整数0~8的二进制表示。尽管这种计数技术有些冗长乏味，但是如果我们扩展它，就会发现16个1组成的位模式表示的是值65 535，这就证实了我们的说法：0~65 535范围内的任何整数都可以用16个二进制位来编码。

由于它的高效性，数字信息通常使用某种形式的二进制记数法来存储，而不用编码符号。我们之所以说“某种形式的二进制记数法”，是因为上面描述的简单二进制系统只是机器里应用的若干数值存储技术的基础。本章后面会讨论一些二进制系统的变体。现在，我们只需要知道，称为**二进制补码**（two's complement）记数法（见1.6节）的系统通常用于存储整个数，因为它提供了一种便利的表示负数和正数的方法。为了表示 $4\frac{1}{2}$ 或 $\frac{9}{2}$ 这样带有分数部分的数，我们还要使用一种称为**浮点记数法**（floating-point notation）的技术（见1.7节）。

最后，无论编写数字时我们是用二进制记数法（基数2）、更熟悉的十进制记数法（基数10），还是用简短的十六进制记数法（基数16），表示的底层数字量都是一样的。也就是说，对于2加2等于4，无论我们是用二进制记数法（ $010 + 010 = 100$ ）、十进制记数法（ $2 + 2 = 4$ ）、十六进制记数法（ $0x2 + 0x2 = 0x4$ ），还是任何其他进制来写，都是对的。对计算机来说，它只是二进制的1和0。

基本知识点

- 数字基数包括2、10和16，被用于表示和研究数字数据。
- 数字可以从任何一种基数转换为任何其他基数。

1.4.3 图像的表达

图像的一种表示方法是：将图像解释为一组点，每个点称为一个**像素**（pixel，是picture element的简写）；然后，对每个像素的显示进行编码，整个图像就表示成了这些编码像素的集合，这个集合称为**位图**（bit map）。这种方法很常用，因为许多显示设备（如打印机和显示屏）都是基于像素概念操作的。因此，位图格式的图像更便于格式化显示。

位图中的像素编码方式随着应用的不同而不同。对于简单的黑白图像，每个像素由一个位表示，位的值取决于相对应像素是黑还是白。这是大多数传真机采用的方法。对于更加精致的黑白照片，每个像素由一组位（通常是8个）表示，这使得许多灰色阴影也可以表示出来。对于彩色图像，每个像素通过更为复杂的系统来编码。表示图像的常见的方法有两种，一种是RGB编码，每个像素表示为3种颜色成分——红、绿、蓝，它们分别对应于光线的三原色。每种颜色成分的强度一般是用1字节来表示。因此，要表示原始图像中的一个单独像素，就需要3字节的存储空间。

另一种替代简单RGB编码的方法是，使用一个“亮度”成分和两个颜色成分。在这种方法中，“亮度”成分被称为像素亮度，基本上就是红、绿、蓝三种颜色成分的总和。（事实上，它是像素中白光的数量，但是我们现在不需要考虑这些细节。）其他两种成分，称为蓝色色度和红色色度，分别由像素中的像素亮度与蓝光或红光数量之间的差来决定。这3个成分合在一起就包含了再现这个像素所需的全部信息。

利用亮度和色度成分对图像进行编码这种方式的普及源自彩色电视广播领域，因为这种方法提供的彩色图像编码方式可以兼容老式黑白电视接收器。事实上，只需要利用编码彩色图像的亮度成分就可以制造出图像的灰度版本。

用位图表示图像的缺点在于，图像不能轻易调整到任意大小。基本上，放大图像的唯一途径就是变大像素，而这会使图像模糊。（这就是应用于数字照相机的“数字变焦”技术，与此相对的“光学变焦”是通过调整照相机镜头实现的。）

图像的另一种表示方法避免了这个缩放问题，将图像描述成了几何结构（如直线和曲线）的集合，这些几何结构可以用解析几何技术来编码。这种描述允许最终显示图像的设备决定几何结构的显示方式，而不是让设备再现特殊像素模式。这种方法被用在了当今的字处理系统中，用于产生可缩放的字体。例如，TrueType（由微软公司和苹果公司开发）是用几何结构描述文本符号的系统，而PostScript（由Adobe系统开发）提供了一种描述字符及更一般的图形数据的方法。这种表示图像的几何方法在[计算机辅助设计](#)（computer-aided design，CAD）系统中也很常见，用于在计算机显示屏上显示和操控三维物体的绘制。

许多绘图软件系统（如微软的画图工具）都允许用户用预先设定的形状（如矩形、椭圆形、基本线条等）画图，对这些用户来说，用几何结构表示图像与用位图表示图像之间的区别是很明显的。用户只需从菜单中选择所需的几何形状，就可以用鼠标绘制出形状。在绘制过程中，软件保存了所画形状的几何描述。当鼠标给出方向后，内部的几何表示就被修改，再转化成位图形式显示出来。这种方法方便图像的缩放和形状的改变。然而，一旦绘制过程完成，系统就会去除基本的几何描述，仅保存位图，这意味着，再做其他修改需要经历冗长的一个像素接一个像素的修改过程。另外，一些绘图系统会将描述作为几何图形保存下来，并允许在之后进行修改。有了这些系统，就可以轻松地调整图形的大小，并可按各种尺寸显示清晰图像。

1.4.4 声音的表示

为了便于计算机存储和操作，对音频信息进行编码的最常用方法是，按有规律的时间间隔对声波的振幅采样，并记录所得到的值序列。例如，序列0、1.5、2.0、1.5、2.0、3.0、4.0、3.0、0可以表示这样一种声波：它的振幅先增大，然后经短暂的减小，再升至更高的幅度，接着又减回至0（见图1-12）。这种技术采用每秒8000次的采样频率，已经在远程语音电话通信中使用了许多年。通信一端的语音被编码为数值，表示每秒8000次的声音振幅。接着，这些数值通过通信线路被传输到接收端，用来重现声音。

尽管每秒8000次的采样频率似乎是很快的速率，但它还是满足不了音乐录制的高保真需求。为了实现现在音乐CD重现声音的质量，我们需要采用每秒44 100次采样的采样频率。每次采样得到的数据要用16位的形式表示（32位用于立体声录制）。因此，录制成立体声音乐，每秒需要100多万个位。

乐器数字化接口（musical instrument digital interface，MIDI，读作“MID-ee”）是另外一种编码系统，被广泛用于电子键盘的音乐合成器、视频游戏声音，以及网站的辅助音效。MIDI是在合成器上编码产生

音乐的指令，而不是对音乐本身进行编码，因此它避免了采样技术那样的大存储容量要求。更精确地说，MIDI是对什么乐器演奏什么音符以及持续时间进行编码。例如，单簧管演奏D音符2秒，可以编码为3字节，而不必按照每秒44 100次的采样频率用两百多万个二进制位来编码。

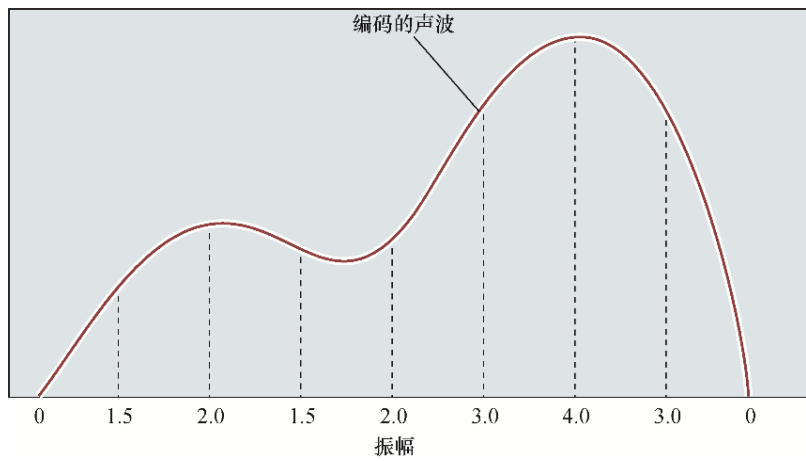


图1-12 序列0、1.5、2.0、1.5、2.0、3.0、4.0、3.0、0所表示的声波

简言之，可以把MIDI看作对演奏者乐谱进行编码的一种方式，而不是对演奏本身编码。因此，MIDI“录制”的音乐在不同合成器上演奏时声音可能是截然不同的。

1.4节问题与练习

1．下面是用ASCII编码的一条消息，每个符号8位。它的含义是什么？（见附录A）

```
01000011 01101111 01101101 01110000 01110101 01110100 01100101
01110010 00100000 01010011 01100011 01101001 01100101 01101110
01100011 01100101
```

2．在ASCII码中，大写字母的ASCII码和这个字母的小写字母的ASCII码之间的关系是什么？（见附录A）

3．用ASCII码对下列语句编码：

- a．“Stop!” Cheryl shouted.
- b．Does 2+3 = 5?

4．描述一种能够呈现两种状态的日常设备，例如，旗杆上的旗帜的升降。给一种状态赋值1，另一种赋值0，请说明当以这样的位来存储时，字母b的ASCII码会怎样表示？

5．将下列二进制表示分别转化为相应的十进制形式。

- a．0101 b．1001 c．1011 d．0110 e．10000 f．10010

6．将下列的十进制表示分别转化为相应的二进制形式。

- a．6 b．13 c．11 d．18 e．27 f．4

7．如果每个数字采用每字节一个ASCII码的模式编码，那么3字节可以表示的最大数值是多少？如果采用二进制记数法呢？

8. 除十六进制记数法以外，另一种表示位模式的方法是**点分十进制记数法**（dotted decimal notation），其中每个字节由相应的十进制数表示，这些字节表示之间用句点分隔。例如，12.5表示模式00001100000000101（12表示字节00001100，5表示字节00000101），而136.16.7表示模式100010000001000000000111。用点分十进制记数法表示下列位模式：

a. 00001111100001111

b. 001100110000000010000000

c. 0000101010100000

9. 相对于位图技术，用几何结构表示图像有哪些优点？位图技术相对于几何结构又有哪些优点？

10. 假如采用文中所讨论的每秒44 100次采样的采样频率，对1小时音乐的立体声录音进行编码。请问这段音乐编码的大小与一张CD的存储容量相比结果如何？

*1.5 二进制系统

在1.4节中我们看到，二进制记数法是表示数值的一种方法，它只使用数字0和1，而不用较常见的十进制记数系统中的10个数字0~9。现在，我们来深入了解一下二进制记数法。

1.5.1 二进制记数法

回顾十进制系统，表示中的每个位置都与一个量值相关联。在375这个表示中，5的位置与量值1相关联，7的位置与量值10相关联，3的位置与量值100相关联（见图1-13a）。每个量是它右边量的10倍。整个表达式表示的值是每个数字的值与与其相关联的位置的量值相乘所得积之和。举例说明：模式375表示 $(3 \times 100) + (7 \times 10) + (5 \times 1)$ ，用更加技术性的表示法表示就是 $(3 \times 10^2) + (7 \times 10^1) + (5 \times 10^0)$ 。

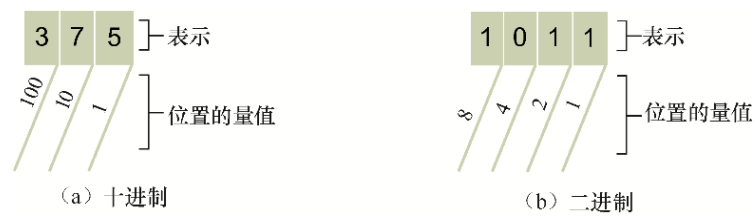


图1-13 十进制和二进制系统

在二进制记数法中，每个数字的位置也与一个量值相关联，只是与每个位置相关联的那个量值是它右边量值的2倍。更精确地说，在二进制表示中，最右边的数字与量值1（即 2^0 ）相关联，其左边的下一个位置与量值2（即 2^1 ）相关联，下一个与量值4（即 2^2 ）相关联，再下一个与量值8（即 2^3 ）相关联，依次类推。例如，在二进制表示1011中，最右边的1的位置与量值1相关联，接下来的1的位置与量值2相关联，0的位置与量值4相关联，最左边的1的位置与量值8相关联（见图1-13b）。

为了求得二进制表示所表示的值，我们可以采取和十进制相同的步骤：先将每个数字的值与与其相关联的位置的量值相乘，再计算各个乘积之和。例如，100101表示的值是37，如图1-14所示。注意，因为二进制记数法仅使用数字0和1，这种求积再求和的步骤就可以简化为求数字的值为1的位置相关联的量值的和。因此，二进制模式1011表示的是十进制值11，因为3个1的位置分别与量值1、2和8相关联。

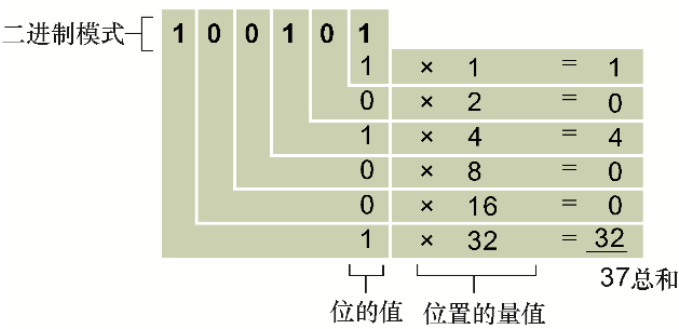


图1-14 二进制表示100101的解码

在1.4节中，我们学习了如何用二进制记数法计数，这使得我们可以对小整数进行编码。为了求得大值的二进制表示，你可能更喜欢图1-15中所描述的算法。让我们利用这个算法来求十进制值13的二进制表示（见图1-16）。首先，将13除以2，得到商6和余数1。因为这个商不是0，步骤2告诉我们还要将商（6）除以2，得到新的商3和余数0。最新的商仍然不为0，所以再除以2，得到商1和余数1。再一次，将最新的商（1）除以2，此时得到商0和余数1。因为现在的商是0，我们进入步骤3，在这一步中，我们从余数列中得到原数（13）的二进制表示1101。

步骤1 将该值除以2，记下余数。

步骤2 只要所得的商不是零，就继续将最新的商除以2，并记下余数。

步骤3 商为0时，将余数按所记录的顺序从右到左依次排列，即得到原数的二进制表示。

图1-15 求正整数二进制表示的算法

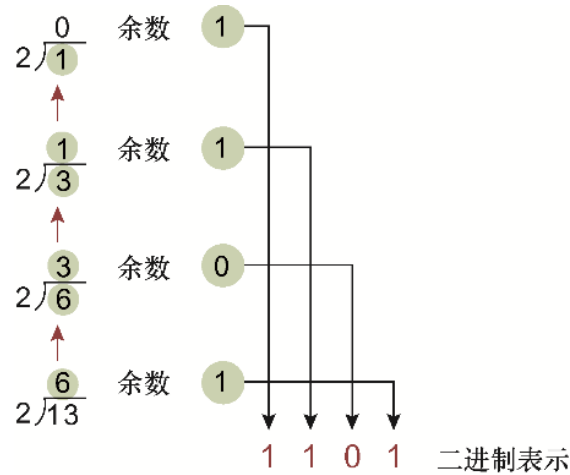


图1-16 利用图1-15中的算法求13的二进制表示

1.5.2 二进制加法

为了理解两个用二进制表示的整数的相加过程，首先让我们回顾一下用传统十进制记数法表示的值的相加过程。例如，考虑下面这道题：

$$\begin{array}{r} 58 \\ + 27 \\ \hline \end{array}$$

我们先把最右列的8和7相加，得到和15，把5记录在这一列的底部，向下一列进1，得到：

$$\begin{array}{r} 1 \\ 58 \\ + 27 \\ \hline 5 \end{array}$$

现在我们把下一列的5和2相加，再加上进到这一列的1，得到和8，把8记录在这一列的底部。结果如下：

$$\begin{array}{r} 58 \\ + 27 \\ \hline 85 \end{array}$$

简而言之，这个过程就是从右到左将每列中的数字相加，把和中的最低有效位写在列的底部，把和中的较高有效位（如果有）进位到下一列。

为了将两个用二进制表示的正整数相加，我们采用相同的步骤，只是所有和的计算都使用图1-17中展示的加法规则，而不是你在小学所学的传统十进制加法法则。

0	1	0	1
+0	+0	+1	+1
0	1	1	10

图1-17 二进制加法法则

例如，为了解下面这道题：

$$\begin{array}{r} 111010 \\ + 11011 \\ \hline \end{array}$$

首先将最右边的0和1相加，得到1，写在该列下方。接着将下一列的1和1相加，得到10。把其中的0写在该列下方，将进位1写在下一列的上方。这时，解答如下：

$$\begin{array}{r}
 1 \\
 111010 \\
 + 11011 \\
 \hline
 01
 \end{array}$$

把下一列的1、0和0相加，得到1，将1写在该列下方。下一列的1和1总和为10，将0写在该列下方，进位1写在下一列。现在，解答如下：

$$\begin{array}{r}
 1 \\
 111010 \\
 + 11011 \\
 \hline
 0101
 \end{array}$$

下一列的1、1和1总和为11（值3的二进制表示），将低位1写在该列下方，将另外一个进位1写在下一列的上方。把进位1与那列原本的1相加，得到10。再一次，将低位0写在该列下方，将进位1写在下一列的上方。现在得到

$$\begin{array}{r}
 1 \\
 111010 \\
 + 11011 \\
 \hline
 010101
 \end{array}$$

下一列的唯一项就是1，是从上一列进位来的，所以我们将其记录在答案里。最终的答案是：

$$\begin{array}{r}
 111010 \\
 + 11011 \\
 \hline
 1010101
 \end{array}$$

1.5.3 二进制中的分数

为了扩展二进制记数法，使其能够表示分数值，我们使用了**基数小数点**（radix point），其功能与十进制记数法中的小数点是相同的，即点左边的数字代表值的整数部分（整个部分），其解释和前面讨论的二进制系统一样。点右边的数字代表值的小数部分，其解释和其他二进制位类似，只是它们的位置被赋予了分数的量值。更确切地说，基数小数点右边第一位的量值是 $1/2$ （即 2^{-1} ），下一位的量值是 $1/4$ （即 2^{-2} ），再下一位是 $1/8$ （即 2^{-3} ），依次类推。注意，这仅仅是前面所述规则的延续：即每位所被赋予的量值是它右边大小的两倍。利用这些赋予二进制位位置的量值，对包含基数小数点和不包含基数小数点的二进

制表示进行解码的步骤基本是相同的。更精确地说，我们是把表示中的每个位的值与其对应位的位置的量值相乘。举例来说，将二进制表示101.101解码为 $5\frac{5}{8}$ ，如图1-18所示。

另外，应用于十进制系统的加法技术同样适用于二进制系统，即两个有基数小数点的二进制表示相加时，我们只需要对齐基数小数点，然后应用前面介绍的加法步骤进行计算即可。例如，10.011加100.11得111.001，如下所示：

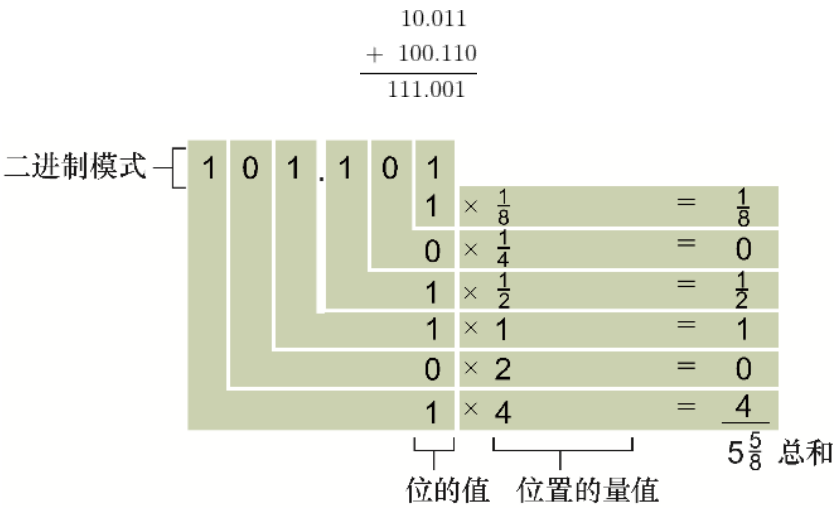


图1-18 二进制表示101.101的解码

1.5节问题与练习

1. 将下列二进制表示转换为相应的十进制形式。

a. 101010 b. 100001 c. 10111 d. 0110 e. 11111

2. 将下列十进制表示转换为相应的二进制形式。

a. 32 b. 64 c. 96 d. 15 e. 27

3. 将下列二进制表示转换为相应的十进制形式。

a. 11.01 b. 101.111 c. 10.1 d. 110.011 e. 0.101

4. 用二进制记数法表示下列值。

a. $4\frac{1}{2}$ b. $2\frac{3}{4}$ c. $1\frac{1}{8}$ d. $\frac{5}{16}$ e. $5\frac{5}{8}$

5. 按照二进制记数法做下列加法。

a. 11011 b. 1010.001 c. 11111 d. 111.11
+ 1100 + 1.101 + 0001 + 00.01

*1.6 整数的存储

数学家长久以来就对数字记数系统很感兴趣，而且他们的许多想法已经被证实与数字电路的设计是相符的。本节，我们将研究其中两种记数系统：二进制补码记数法和余码记数法。它们都是计算设备用于表示整数值的方法。虽然这些系统都是基于二进制系统的，但是它们增加了一些其他的特性，因而与计算机设计更加兼容。尽管它们有这么多的优点，但是它们也有缺点。我们的目标是了解这些特性以及它们是如何影响计算机用法的。

模拟与数字

在21世纪之前，许多研究人员都在讨论数字技术和模拟技术的优缺点。在数字系统中，一个值会被编码成一系列数字，存储在若干设备中，每个设备代表一个数字。在模拟系统中，每个值存储在一个单独的设备中，该设备可以表示一个连续范围内的任何值。

让我们用水桶作为存储设备来比较一下这两种方法。为了模拟数字系统，我们可以约定用空桶表示数字0，用满桶表示数字1。然后我们可以使用浮点记数法（见1.7节）将一个数值存储在一排水桶中。相反，为了模拟一个模拟系统，我们用水桶的水位表示数值。乍一看，模拟系统看起来更精确，因为它不会受到数字系统固有的截断误差（见1.7节）的影响。不过，在模拟系统中，水桶的任何移动都可能导致水位检测出错，而在数字系统中，则必须有剧烈的晃动才能区分出水桶是满的还是空的。因此，数字系统对误差的敏感度低于模拟系统。这种健壮性是许多最初基于模拟技术的应用（如电话通信、音频录制和电视）都转而使用数字技术的主要原因。

1.6.1 二进制补码记数法

现今计算机中最流行的用于表示整数的系统是**二进制补码**（two's complement）记数法。这个系统采用固定数目的二进制位来表示系统中的每个值。现今设备中普遍采用二进制补码系统，其中每个值用一个32位的模式表示。这种大系统能表示很大范围的数字，但不方便演示。因此，在学习二进制补码系统的特性时，我们将着重介绍较小的系统。

图1-19列出了两种完整的二进制补码系统，一种基于长度为3的位模式，另一种基于长度为4的位模式。要构造这种系统，首先要规定一组适当长度的二进制0，接着用二进制计数，直到前面只有一个0、后面都是1的模式形成。这些模式表示值0, 1, 2, 3, ...。要想获得表示负值的模式，首先要规定一组适当长度的二进制1，接着按照二进制反向计数，直到前面只有一个1、后面都是0的模式形成。这些模式表示值-1, -2, -3, ...。（如果你认为利用二进制反向计数有困难，那么可以从表格底部前面只有一个1、后面都是0的模式开始，向上计数到全是1的模式。）

位模式	所表示的值	位模式	所表示的值
011	3	0111	7
010	2	0110	6
001	1	0101	5
000	0	0100	4
111	-1	0011	3
110	-2	0010	2
101	-3	0001	1
100	-4	0000	0
		1111	-1
		1110	-2
		1101	-3
		1100	-4
		1011	-5
		1010	-6
		1001	-7
		1000	-8

(a) 使用长度为3的位模式 (b) 使用长度为4的位模式

图1-19 二进制补码记数系统

注意，在二进制补码系统中，位模式最左边的二进制位指明了所表示的值的符号。因此，最左边的位常称为**符号位**（sign bit）。在二进制补码系统中，符号位为1的模式表示负值，符号位为0的模式表示非负值。

在二进制补码系统中，绝对值相同的正负值的表示模式之间有一种对应关系，从右向左读时，直到第一个二进制1（包括那个1），它们都是相同的。然后，以这个1为分界线，左面的位模式互为补码。（要得到一个模式的**补码**（complement），需要将所有的二进制0转换为1，并将所有的二进制1转换为0。）例如，在图1-19所示的4位系统中，表示2和-2的模式都是以10结尾，但是表示2的模式是以00开始，而表示-2的模式是以11开始。观察到这一点，我们就可以得出在表示绝对值相同的正负值的位模式之间进行来回转换的算法。我们只需要从右到左复制原始模式直到第一个1，然后在将剩余位转换为最终位模式时，对这些剩余位取反（见图1-20）。

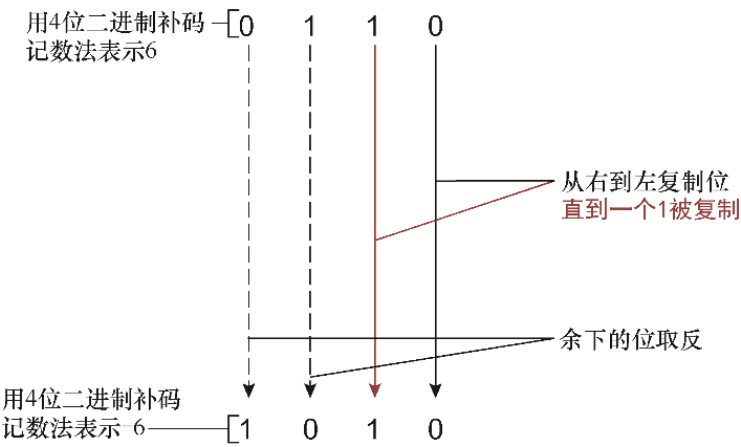


图1-20 用4位二进制补码记数法对值6编码

理解了二进制补码系统的这些基本特性，还可以得出二进制补码表示的一个解码算法。如果要解码的模式有一个符号位0，我们只需要读这个值，就好像这个模式是一个二进制表示。例如，0110表示十进制值6，因为110是6的二进制表示。如果要解码的模式有一个符号位1，我们就知道表示的值是负的，而我们所要做的就是找到其绝对值。为了实现这个目的，我们首先要利用图1-20中的“复制和取反”步骤，然后对获得的模式进行解码，就好像它只是一个简单的二进制表示。例如，为了对模式1010解码，我们首先要认识到，因为这个符号位是1，所以表示的值是负的。因此，我们利用“复制和取反”步骤获得模式0110，认识到这是6的二进制表示之后，我们得出结论：原始模式表示的是-6。

1.6.2 二进制补码记数法中的加法

为了把二进制补码记数法表示的值相加，我们采用二进制加法中使用的算法，只是包括答案在内的所有位模式都是一样长的。这就意味着，在二进制补码系统中做加法时，如果最后一个进位导致答案左边产生了附加位，那么这个附加位一定会被截断。因此，0101和0010“相加”得0111，0111和1011“相加”得0010（ $0111+1011=10010$ ，被截断为0010）。

根据这个理解，我们来分析一下图1-21中的3个加法题。这里的每种情况，我们都把问题转化为二进制补码记数法（采用长度为4的位模式），演示了先前描述过的加法过程，然后对结果进行解码，回到一般的十进制记数法。

十进制问题	二进制补码问题	十进制答案
$\begin{array}{r} 3 \\ + 2 \\ \hline \end{array}$	$\begin{array}{r} 0011 \\ + 0010 \\ \hline 0101 \end{array}$	5
$\begin{array}{r} -3 \\ + -2 \\ \hline \end{array}$	$\begin{array}{r} 1101 \\ + 1110 \\ \hline 1011 \end{array}$	-5
$\begin{array}{r} 7 \\ + -5 \\ \hline \end{array}$	$\begin{array}{r} 0111 \\ + 1011 \\ \hline 0010 \end{array}$	2

图1-21 转换为二进制补码记数法的加法题

注意，图1-21中的第三道题涉及的是一个正数和一个负数的加法，它展示了二进制补码记数法的一个主要优点：有符号数的任意组合的加法，都可以使用相同的算法来完成，因而可以使用相同的电路来实现。这与人类传统上的算术运算方式截然不同。尽管小学生是先学加法，后学减法，但是使用二进制补码记数法的机器只需知道如何相加就可以了。

例如，减法问题 $7-5$ 与加法问题 $7+(-5)$ 是一样的。因此，如果机器被要求计算 7 （存储为 0111 ）减 5 （存储为 0101 ），那么它首先要将 5 转换为 -5 （表示为 1011 ），然后执行 $0111+1011$ 的加法过程，得到 0010 ，它代表值 2 ，如下所示：

$$\begin{array}{r} 7 \\ -5 \\ \hline \end{array} \rightarrow \begin{array}{r} 0111 \\ - 0101 \\ \hline \end{array} \rightarrow \begin{array}{r} 0111 \\ + 1011 \\ \hline 0010 \end{array} \rightarrow 2$$

由此可见，当用二进制补码记数法表示数值时，只需要将一个加法电路和一个取反电路组合在一起，就足以同时解决加法和减法这两个问题了。（这些电路的图示及解释详见附录B。）

1.6.3 溢出问题

我们在前面的例子中避开了这样一个问题：任何一个二进制补码系统所能表示的值大小都有限制。当使用4位模式二进制补码记数法时，可以表示的最大正整数是 7 ，最小负整数是 -8 。具体来说，就是无法表示值 9 ，这意味着我们不能指望得出 $5+4$ 的正确答案。事实上，它的结果会显示为 -7 。这种现象称为**溢出**（overflow）。也就是说，溢出指的是计算得出的值超出了可以表示的值的范围。使用二进制补码记数法时，两个正值相加或两个负值相加都可能会出现这种情况。无论哪种情况，检查答案的符号位就可以发现溢出的条件。如果两个正值相加的结果是负值的模式，或者两个负值相加的结果为正，那么就发生了溢出问题。

当然，因为使用二进制补码系统的大多数计算机的位模式，都比我们上面例子中给出的长，所以在处理较大值时不会产生溢出。如今，人们普遍使用二进制补码记数法的32位模式来存储值，可以得到的最大正值是 $2\,147\,483\,647$ 。如果需要更大的值，我们可以使用更长的位模式，或者改变度量单位。例如，在解答一个问题时用英里代替英寸，所得的数就会变小，而且可以达到所需的精度。

关键问题是计算机制造错误。因此，使用机器的人一定要意识到可能涉及的危险。其中一个问题就是，计算机程序员和使用者会自满而导致忽视一个事实——较小的值可以累加成较大的值。例如，人们过去普遍使用二进制补码记数法的16位模式表示值，这意味着，出现大于或等于 $2^{15}=32\,768$ 的值时就会产生溢出。1989年9月19日，一家医院多年来运行良好的计算机出现了故障。仔细检查后发现，那天距1900年1月1日共 $32\,768$ 天，而计算机程序正是基于那个起始日期开始计算日期的。因此，由于溢出，1989年9月19日产生了负值——设计计算机程序时没有考虑到这种现象。

基本知识点

- 有限表示用于建模数的无限数学概念。
- 在许多程序设计语言中，用于表示字符或整数的固定位数限制了整数值的范围和数学运算；这个限制会导致溢出或其他错误。

- 整数可能会由于存储的局限性被限制在程序能表示的最大值和最小值之间。

1.6.4 余码记数法

表示整数值的另外一种方法是**余码记数法**（excess notation）。与二进制补码记数法相同，余码记数系统中的每个值也都是用相同长度的位模式表示的。为了建立余码系统，我们首先要选择要使用的模式长度，然后根据二进制记数呈现的顺序写下那个长度的所有位模式。接着我们发现，二进制1作为其最高有效位的第一个模式大约就在数列的中间。我们用这个模式表示0，其后的模式分别用于表示1, 2, 3, ...，其前的模式就分别用于表示-1, -2, -3, ...。使用长度为4的模式产生的编码如图1-22所示。我们可以看到，模式1101表示值5，模式0011表示值-5。（注意，余码系统和二进制补码系统的一个区别就是符号位相反。）

图1-22所示的系统称为余8记数法。为了了解其由来，我们先用传统二进制系统的编码解释每个模式，然后将其与余码记数法表示的值进行比较。你会发现，在每种情况下，二进制解释的值都比余码记数法解释的值大8。例如，模式1100在二进制记数法中表示值12，但在余码系统中表示4；0000在二进制记数法中表示值0，但在余码系统中表示-8。与此类似，在基于长度为5的位模式的余码系统中，模式10000表示的是0，而不是通常的值16，该记数法称为余16记数法。同样，你可以证明3位余码系统应该称为余4记数法（见图1-23）。

位模式	所表示的值
1111	7
1110	6
1101	5
1100	4
1011	3
1010	2
1001	1
1000	0
0111	-1
0110	-2
0101	-3
0100	-4
0011	-5
0010	-6
0001	-7
0000	-8

图1-22 余8代码转换表

位模式	所表示的值
111	3
110	2
101	1
100	0
011	-1
010	-2
001	-3
000	-4

图1-23 使用长度为3的位模式的余码记数系统

1.6节问题与练习

1. 将下面每个二进制补码表示转换为相应的十进制形式。

- a . 00011 b . 01111 c . 11100
d . 11010 e . 00000 f . 10000

2. 用8位位模式将下列每个十进制表示转换为相应的二进制补码形式。

- a . 6 b . -6 c . -17
d . 13 e . -1 f . 0

3. 假定下列位模式表示的是用二进制补码记数法存储的值，求出每个值的负值的二进制补码表示。

- a . 00000001 b . 01010101 c . 11111100
d . 11111110 e . 00000000 f . 01111111

4. 假定一台机器用二进制补码记数法存储值，如果机器分别采用下列长度的位模式，那么可以存储的最大数和最小数分别是什么？

- a . 4 b . 6 c . 8

5. 在下列各题中，每个位模式表示一个用二进制补码存储的值。请执行文中所述的加法过程，按照二进制补码记数法求出它们的答案。并将题及答案转换为十进制记数法进行验证。

- a . 0101+0010 b . 0011+0001 c . 0101+1010
d . 1110+0011 e . 1010+1110

6. 计算下列二进制补码记数法表示的题，但这次要观察溢出问题，并指出哪个答案因产生溢出而不正确。

- a . 0100+0011 b . 0101+0110 c . 1010+1010
d . 1010+0111 e . 0111+0001

7. 将下列各题从十进制记数法转换为长度为4的位模式的二进制补码记数法，然后将每道题转换成一个相应的加法题（像机器做的那样），并执行加法。将求得的答案转换为十进制记数法进行验证。

- a . 6-(-1) b . 3-(-2) c . 4-6
d . 2-(-4) e . 1-5

8. 在二进制补码记数法里，一个正数和一个负数相加时会产生溢出吗？请说明理由。
9. 将下面每个余8码表示转换为相应的十进制形式（解题时不要看文中的表格）。
- a. 1110 b. 0111 c. 1000
- d. 0010 e. 0000 f. 1001
10. 将下列的每个十进制表示转换为相应的余8码形式（解题时不要看文中的表格）。
- a. 5 b. -5 c. 3
- d. 0 e. 7 f. -8
11. 值9可以用余8记数法表示吗？用余4记数法表示6呢？请说明理由。

*1.7 分数的存储

不同于整数的存储，对于带有分数部分的值的存储，我们不仅要存储代表其二进制表示的0和1模式，还需要存储其基数小数点的位置。有一种流行的基于科学记数法的存储方法，称为**浮点**（floating-point）记数法。计算机的有限存储空间限制了我们能表示的分数的精度，下一节还会回到这个问题上。

基本知识点

- 实数是通过不一定有无限精度的浮点表示近似得到的。

1.7.1 浮点记数法

为了解释浮点记数法，我们来看一个只用1字节来存储的例子。尽管机器通常使用更长的模式，但是这种8位格式也可以表示实际的系统，而且既可以展示重要的概念，又避免了长位模式的混乱。

首先，我们规定这个字节的高位端为符号位。再次说明，符号位中的二进制0代表存储的值为非负值，1代表值为负值。接着，我们将这个字节剩余的7个位分为两组，或称其为域：**指数域**（exponent field）和**尾数域**（mantissa field）。我们规定符号位右边的3个位为指数域，余下的4个位为尾数域。图1-24描述了如何拆分字节。

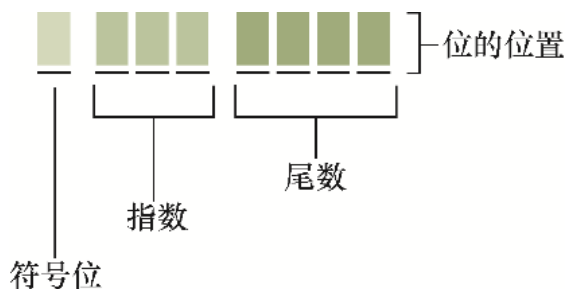


图1-24 浮点记数法成分

我们可以借助下面的例子来解释这些域的含义。假如1字节由位模式01101011组成。利用前面的格式分析这个模式，可以看出符号位是0，指数是110，尾数是1011。为了解码这个字节，我们首先要提取它的尾数，并在它的左边放一个基数小数点，得到

.1011

接着，我们提取指数域（110）的内容，并将其解释为一个用3位余码方法（见图1-23）存储的整数。从而得出，我们所举例子的指数域模式表示正数2。这就要求我们将上面所得结果的基数小数点向右移动2位。（负指数域就意味着向左移动基数小数点。）因此，我们得到

10.11

这就是 $2\frac{3}{4}$ 的二进制表示（二进制中的分数的表示参见图1-18）。接着，我们注意到：示例中的符号位是0，因此所表示的值为非负值。最后我们得出结论：字节01101011表示 $2\frac{3}{4}$ 。如果模式是11101011（除了符号位都与之前相同），表示的值将为 $-2\frac{3}{4}$ 。

再看一个例子，考虑字节00111100。提取尾数后得到

.1100

因为指数域（011）表示值-1，所以将基数小数点向左移动一位，得到

.01100

这表示 $\frac{3}{8}$ 。因为原始模式中的符号位是0，所以存储的值为非负值。最后我们得出结论：模式00111100表示 $\frac{3}{8}$ 。

在用浮点记数法存储值时，要将前面的过程反过来。例如，为了对 $1\frac{1}{8}$ 编码，我们首先要将其用二进制记数法表示，得到1.001。接着，我们从二进制表示中最左边的1开始，从左到右将其位模式复制到尾数域。此时，这个字节看起来像下面这样：

_ _ _ 1 0 0 1

现在我们必须填充指数域。为了达到这个目的，假定尾数域的左边有一个基数小数点，然后规定位的数量以及小数点移动的方向，以此得到原始的二进制数。在这个例子中我们看到，0.1001中的基数小数点要向右移动一位才能得到1.001，指数因此为正1，所以我们将101（这在余4记数法中表示正1，见图1-23）放在指数域中。最后，因为存储的是非负值，我们用0填充符号位。完成的字节看起来像下面这样：

0 1 0 1 1 0 0 1

在填充尾数域时，你可能会漏掉一个微妙的细节。这个规则是从最左边的1开始，从左到右复制以二进制表示的位模式。为阐述清楚，让我们考虑一下存储值 $\frac{3}{8}$ 的过程，它的二进制记数法表示为.011。这时，其尾数将是

_ _ _ _ 1 1 0 0

而不会是

_ _ _ _ 0 1 1 0

这是因为我们是从二进制表示最左边的1开始填充尾数域的。遵循这个规则的表示称为**规范化形式**（normalized form）。

使用规范化形式消除了同一个值有多种表示的可能性。例如，00111100和01000110都可以解码成值 $\frac{3}{8}$ ，但是只有第一个模式是规范化形式。遵循规范化形式也意味着，所有非0值的表示都会有一个以1开始的尾数。不过，值0是一个特例，它的浮点表示是全0的位模式。

1.7.2 截断误差

下面我们来考虑一下，用我们的1字节浮点记数系统存储值，看看 $2\frac{5}{8}$ 会出现什么恼人的问题。我们首先用二进制写 $2\frac{5}{8}$ ，得到10.101。但是，在把这个模式复制到尾数域时，我们就用尽了空间，最右边的1（表示最后的 $\frac{1}{8}$ ）因此丢失了（见图1-25）。如果现在忽视这个问题，继续填充指数域和符号位，那么我们最后得到的位模式将为01101010，它表示的是 $2\frac{1}{2}$ ，而不是 $2\frac{5}{8}$ 。这个现象称为**截断误差**（truncation error）或**舍入误差**（round-off error）。这意味着要存储的值的这部分会因尾数域空间不够大而丢失。

基本知识点

- 在许多程序设计语言中，用于表示实数（像浮点数一样）的固定位数限制了浮点值的范围和数学运算，这个限制会导致舍入误差和其他误差。

通过使用较长的尾数域可以显著减少这种误差。事实上，现在生产的大多数计算机都至少采用32位存储浮点记数法表示的值，而不是我们这里用的8位。这同时使得指数域也更长。不过，即使有这些较长的格式，还是会有需要更高的精度的时候。

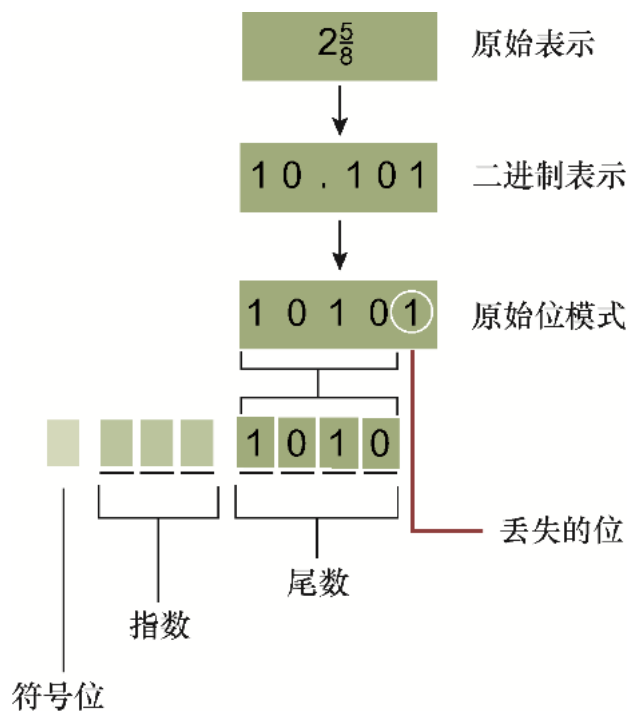


图1-25 值 $2\frac{5}{8}$ 的编码过程

截断误差的另一个来源是十进制记数法中比较常见的一个现象，即无穷展开式问题。例如，我们在用十进制形式表示 $\frac{1}{3}$ 的时候。有些值无论我们用多少位数字都无法精确地表示。传统的十进制记数法与二进制记数法的区别在于，二进制记数法中有无穷展开式的值多于十进制。例如，值 $\frac{1}{10}$ 表示为二进制时为无穷展开式。想象一下，一个粗心的人用浮点记数法存储和处理美元与美分时可能会导致什么问题？尤其是，如果用美元作度量单位，那么一角就不能被精确地存储。这种情况的一种解决方案是，以分为单位处理数据，这样所有的值就都是整数了，都可以用诸如二进制补码这样的方法来精确存储。

单精度浮点数

1.7节介绍的浮点记数法过于简单，不能用于实际的计算机中。毕竟，在全部实数中，这一记数法的8位只能表示其中256个数。虽然我们在讨论中使用了8位模式，但在保持示例简单的情况下，也涵盖了重要的基本概念。

现在的许多计算机都支持32位形式的单精度浮点（single precision floating point）记数法。这一格式用1位表示符号，用8位表示指数（余码记数法中的），用23位表示尾数。因此，单精度浮点数最多有7位十进制有效数字，可以表示极大的数（数量级为 10^{38} ）直至极小的数（数量级为 10^{-37} ）。也就是说，给定一个十进制数，可以非常精确地存储前7位十进制有效数字（但仍有可能存在少量误差），前7位之后的数字一定会因截断误差而丢失（虽然数字的近似值会被保留下来）。

另一种形式是64位的双精度浮点（double precision floating point）记数法，最多有15位十进制有效数字。

截断误差和与之相关的问题是工作在数值分析领域的人们每天都很关注的问题。这个数学分支研究的是执行大规模、高精度有效计算所涉及的问题。

下面的例子可以激起任何一位数值分析家的兴趣。假设我们要应用前面定义的1字节浮点记数法来做这3个值的加法：

$$2\frac{1}{2} + \frac{1}{8} + \frac{1}{8}$$

如果我们按照上述顺序将值相加，先将 $2\frac{1}{2}$ 加到 $\frac{1}{8}$ 上，得到 $2\frac{5}{8}$ ，其二进制表示为10.101，那么，很遗憾，因为这个值不能被精确地存储（如同前面所看到的），第一步的结果最后被存储为 $2\frac{1}{2}$ （与其中一个加数相同）。下一步是把这个结果再加到最后的 $\frac{1}{8}$ 上。截断误差在这里又一次出现，最后的结果是错误的 $2\frac{1}{2}$ 。

现在让我们以相反的顺序来加这些值。先将 $\frac{1}{8}$ 加到 $\frac{1}{8}$ 上，得到 $\frac{1}{4}$ ，其二进制表示为.01。于是，第一步的结果在1字节里被存储为00111000，这是精确的。然后，我们将这个 $\frac{1}{4}$ 加到算式中的下一个值 $2\frac{1}{2}$ 上，得到 $2\frac{3}{4}$ ，就可以在1字节里精确地将其存储为01101011。这次的答案是正确的。

总而言之，在浮点记数法表示的数值加法中，它们相加的顺序很重要。问题是，如果一个很大的数加上一个很小的数，那么小的数就可能被截断。因此，多个值相加的一般规则是先把较小的值加在一起，希望它们被加到一个较大的值上时，能累计成一个显著的值。这就是前面例子中反映的现象。

现在的商用软件包的设计师在这方面做得很好，他们使没有经过培训的使用者也能很好地避免这种问题的发生。在一个典型的电子表格系统中，除非相加的各个值大小差别达到 10^{16} 或更多，否则所得结果都是正确的。因此，如果你认为有必要对值

10 000 000 000 000 000

加1，那么会得到答案

10 000 000 000 000 000

而不是

10 000 000 000 000 001

这样的问题在一些应用（如导航系统）中是很严重的，微小的误差会在加法运算中累加，最终产生严重的后果。但是，对于一般的PC使用者，大多数商用软件提供的精度已经足够了。

1.7节问题与练习

1. 用文中所述的浮点格式对下列位模式进行解码。

a. 01001010 b. 01101101 c. 00111001 d. 11011100 e. 10101011

2. 将下列值编码成文中所述的浮点格式。指出截断误差的出现情况。

a. $2\frac{3}{4}$ b. $5\frac{1}{4}$ c. $\frac{3}{4}$ d. $-3\frac{1}{2}$ e. $-4\frac{3}{8}$

3. 根据文中所述的浮点格式，模式01001001和00111101中哪一个表示的值更大？描述一种确定哪个模式表示的值更大的简单过程。

4. 使用文中所述的浮点格式时，可以表示的最大的值是多少？可以表示的最小的正值是多少？

*1.8 数据与程序设计

虽然人类发明了构成现代计算机的数据表示和基本操作，但很少有人特别擅长于直接在计算机的这个层面上工作。人们喜欢在更高的抽象层次上推理计算问题，他们依赖计算机来处理最底层的细节问题。[程序设计语言](#)（programming language）是人类创造的一个计算机系统，通过它，人们能够使用更高层次的抽象来向计算机精确地表达算法。

在20世纪，对计算机进行程序设计被认为是少数训练有素的专家们才能涉足的领域；当然，仍然有许多计算问题需要有经验的计算机科学家和软件工程师的关注。不过，到了21世纪，随着计算机和计算与我们现代生活中的方方面面交织在一起的程度的日益加深，越来越难以确定哪些职业不需要至少有某种程度的编程技能。事实上，一些人已经把程序设计或[编码](#)确定为现代读写能力中继阅读、写作及算术之后的又一个基础支柱了。

在本节以及后续章节的程序设计补充部分，我们会看到程序设计语言是如何反映本章主要内容，以及如何让人类更容易地解决涉及计算的问题的。

1.8.1 Python入门

Python是一门程序设计语言，由吉多·范罗苏姆（Guido van Rossum）于20世纪80年代后期创立。现在是十大最常用的语言之一，仍然深受网络应用开发领域及科学计算领域人士的喜爱，并被视为学生的入门语言。使用Python的组织范围很广，从谷歌到美国国家航空航天局（NASA），从DropBox公司到工业光魔公司（Industrial Light & Magic），横跨非正式、科学和艺术计算机用户。Python强调可读性，包括命令型程序设计范型、面向对象程序设计范型和函数式程序设计范型等三大要素，这些将在第6章中介绍。

用于编辑和运行用Python编写的程序的软件可以从Python官方网站上免费获得，这里还有很多其他的入门资源。Python语言一直在不断演变，本书的所有示例都将使用Python 3这个版本。较早的Python版本能够运行非常类似的程序，但是自Python 2版本开始有了许多细微的变化，如标点符号。

Python是一种[解释型语言](#)（interpreted language），这意味着初学者可以将Python指令键入交互提示符中，或者将Python指令存储在一个（称为“脚本”的）纯文本文件里以后运行。在下面的示例中，这两种模

式都可以使用，但练习和复习题一般都要求用Python脚本来完成。

1.8.2 你好，Python

许多程序设计语言的介绍长久以来一直有一个传统，描述的第一个程序都是“Hello, World”。这个简单的程序输出一个名义上的问候，演示一种特定语言是如何产生结果以及如何表示文本的。在Python^[1]中，这个程序的写法如下：

[1] 下面这个Python代码是用该语言的第3版写出来的，本书后面会直接把这个版本的Python称作“Python”。较早的Python版本并不总是需要开括号和闭括号。

```
print('Hello, World!')
```

可以将这条语句键入Python的交互解释器里，也可以把它保存为Python脚本然后再执行。不管用哪一种方式，最后的结果都应该是：

```
Hello, World!
```

Python会把两个引号之间的文本返回给用户。

即使是在这种简单的Python脚本里，也有几个需要注意的方面。首先，`print()`是一个内置函数，是Python脚本用来产生输出的一个预定义操作，输出指的是一个能够让用户看到的程序结果。这个打印函数后面有一个开括号和一个闭括号，这两个括号之间的内容就是要打印的值。

其次，Python可以使用单引号来表示文本串。大写字母H前面的引号和感叹号后面的引号，分别表示由字符组成的字符串的开头和结尾，在Python中，这个字符串会被当作值。

程序设计语言能够非常精确地完成它们的指令。即使用户只是稍微修改一下打印语句中开始引号和结束引号之间的消息，最后打印出来的文本也会相应地变化。花一点儿时间，在打印语句中试试不同的大小写、不同的标点符号，甚至不同的单词，读者会看到确实是这样。

1.8.3 变量

Python允许用户给值命名以备日后使用，这是构造简洁、易懂的脚本时的一个重要抽象。这些命名的存储位置被称为变量（variable），类似于代数课程中的数学变量。下面来看一下略有增强的“Hello World”版本：

```
message = 'Hello, world!'
print(message)
```

在这个脚本中，第一行是**赋值语句**（assignment statement）。“=”号的使用可能会让习惯了等号的代数用法的初学者感到迷惑。这个赋值语句应该读作：“变量message被赋予字符串值'Hello, World!'。”通常，赋值语句由等号左边的变量名和等号右边的值构成。

Python是一种**动态类型**（dynamically typed）语言，这意味着，我们不需要在建立脚本时提前创建好一个名为message的变量，或者确定什么类型的值应该存储在message中，而只需要在这个脚本中说明我们的文本串会被赋给message，接着在后面的print语句中引用这个变量message就行了。

变量的命名很大程度上取决于Python用户。Python的简单规则是：变量名必须以字母开头，可以包含任意数量的字母、数字和下划线字符（_）。虽然对一个两行的示例脚本来说，可能把变量命名为m就可以了，但有经验的程序员都会尽量为脚本中的变量起一个有意义的、具有描述性的名字。

Python变量名是**区分大小写的**（case-sensitive），即有大小写问题。名为size的变量，与名为Size或SIZE的变量是截然不同的。有一小部分**关键字**（keyword），即为Python中的一些特殊含义保留的名字，不能用作变量名。在Python的内置帮助系统里能查看到这个关键字清单。

```
help('keywords')
```

变量可用于存储Python能表示的所有类型的值。

```
my_integer = 5
my_floating_point = 26.2
my_Boolean = True
my_string = 'characters'
```

观察上面这些值的类型，它们与本章前面介绍的表示方法是相对应的：布尔值真和假（见1.1节）、文本（见1.4节）、整数（见1.6节）和浮点数（见1.7节）。通过Python的其他代码（超出了本书简单介绍的范围），我们还能够用Python变量存储图像和声音数据（见1.4节）。

Python使用0x前缀来表示十六进制值，如

```
my_integer = 0xFF
print(my_integer)
```

不管程序员在推理过程中使用什么样的记数系统，指定一个十六进制的值都不会改变计算机存储器中该值的表示，存储器都会把整数值存储为许多个1和0。十六进制记数法仍然是人类在用的一种有助于理解脚本的快捷表示。因此，上面的print()语句打印出来的是255，即十六进制0xFF的十进制解释，因为这是print()的默认行为。用更复杂的print()语句可以输出其他表示的值，但本书只讨论我们比较熟悉的十进制表示。

Unicode字符包含着无所不在的ASCII子集中所没有的字符，当文本编辑器支持Unicode字符时，可以直接在字符串里包含Unicode字符：


```
print('₹1000']      # Prints ₹1000, one thousand Indian Rupees
```

也可以用前缀'\u'和4个十六进制数字来指定Unicode字符：

```
print('\u00A31000')  # Prints £1000, one thousand British  
                    # Pounds Sterling
```

字符串的'\u00A3'部分会对英镑符号的Unicode表示进行编码。'1000'紧跟在后面，这样最终输出的货币符号和数量之间就不会有空格了：£1000。

除了Unicode文本串，这些示例语句引入了另一种语言特性。#号表示[注释](#)（comment）的开始，这是一个人类可读的Python代码符号，在计算机执行时会被忽略。有经验的程序员会在他们的代码中使用注释来解释算法难懂的部分，包括历史或来源信息，或者只写些读这段代码的人应该注意的问题。脚本顶端的高级描述向人类读者介绍脚本的总体目标和脚本中使用的方法。#号右边一直到行尾的所有字符都会被Python忽略。

基本知识点

- 程序的解释能帮助人们理解程序的功能和用途。

1.8.4 运算符和表达式

Python的内置运算符允许人们以各种熟悉的方式对数值进行操作和组合。

```
print(3 + 4)      # Prints "7", which is 3 plus 4.  
print(5 - 6)      # Prints "-1", which is 5 minus 6  
print(7 * 8)      # Prints "56", which is 7 times 8  
print(45 / 4)     # Prints "11.25", which is 45 divided by 4  
print(2 ** 10)    # Prints "1024", which is 2 to the 10th power
```

基本知识点

- 在程序中使用整数和浮点数时不需要理解它们是如何实现的。
- 数和数字概念是程序设计的基础。
- 使用算术运算符的数学表达式是大多数程序设计语言的一部分。

当一个运算（如45除以4）产生的是非整数结果（如11.25）时，Python会将表示类型隐式转换为浮点表示。如果希望结果是整数，就要使用另外一组运算符：

```
print(45 // 4)    # Prints "11", which is 45 integer divided by 4  
print(45 % 4)     # Prints "1", because 4 * 11 + 1 = 45
```

双斜线 (//) 表示**整除** (integer floor division) 运算符，百分号 (%) 表示**取模** (modulus) 或取余运算符。将这两个计算综合起来，可读作：“4除45等于11，余数为1。”在前面的示例中，我们用 * 表示幂运算符，这看起来可能有些奇怪，因为在打印文本甚至一些其他程序设计语言中，幂运算符一般都是用插入符号 (^) 表示。在Python中，^运算符是一种**按位布尔运算** (bitwise Boolean operation) 运算符，有关按位布尔运算的内容将在下一章中介绍。

还可以用一些直观的方式对字符串值进行组合和操作。

```
s = 'hello' + 'world'
t = s * 4
print(t)    # Prints "helloworldhelloworldhelloworldhelloworld"
```

其中，加运算符 (+) 用于**拼接** (concatenate) 字符串值，乘运算符 (*) 用于**复制** (replicate) 字符串值。

基本知识点

- 字符串和字符串操作 (包括拼接和某种形式的子字符串) 在许多程序中很常见。

有些内置运算符的多重含义会导致混淆。下面这个脚本会产出一个错误：

```
print('USD$' + 1000)    # TypeError: Can't convert 'int' to str implicitly
```

这个错误指明，字符串拼接运算符不知道在第二个操作数不是字符串的情况下要怎么做。不过幸运的是，Python提供了允许将值从一种表示类型转换为另外一种表示类型的函数。int()函数能把浮点型值转换回整数表示，丢弃分数部分。如果字符串可以正确地拼出一个有效数字，那么int()函数还能将一个文本数字串转换为一个整数表示。同样，str()函数能把数字表示转换成UTF-8编码的文本串。因此，对上述print语句做如下修改就能改正错误。

```
print('USD$' + str(1000))    # Prints "USD$1000"
```

1.8.5 货币转换

下面这个完整的Python脚本示例演示了许多本节要介绍的概念。给定一定数量的美元，脚本会对其进行货币转换，转换成4种其他货币。

```
# A converter for international currency exchange.
USD_to_GBP = 0.76    # Today's rate, US dollars to British Pounds
USD_to_EUR = 0.86    # Today's rate, US dollars to Euros
USD_to_JPY = 114.08  # Today's rate, US dollars to Japanese Yen
USD_to_INR = 63.64   # Today's rate, US dollars to Indian Rupees

GBP_sign   = '\u00A3' # Unicode values for non-ASCII currency symbols.
EUR_sign   = '\u20AC'
JPY_sign   = '\u00A5'
INR_sign   = '\u20B9'
```

```
dollars    = 1000 # The number of dollars to convert

pounds     = dollars * USD_to_GBP    # Conversion calculations
euros      = dollars * USD_to_EUR
yen        = dollars * USD_to_JPY
rupees     = dollars * USD_to_INR

print('Today, $' + str(dollars)) # Printing the results
print('converts to ' + GBP_sign + str(pounds))
print('converts to ' + EUR_sign + str(euros))
print('converts to ' + JPY_sign + str(yen))
print('converts to ' + INR_sign + str(rupees))
```

执行该脚本时，输出如下：

```
Today, $1000
converts to £760.0
converts to €860.0
converts to ¥114080.0
converts to ![PEYPY[0RMB60@R50R$JAR7S{6}]](/api/storage/getbykey/original?key=2208a1b1e5d3774041ef)6364c
```

1.8.6 调试

程序设计语言对初学者不是很宽容，初学者在编写软件时，需要花费大量的时间努力查找代码中的 **bug** 或者错误。定位这些bug并修正它们因此称为 **调试**（debugging）。软件中的bug可分为3大类：**语法错误**（syntax error，键入时产生的符号错误）、**语义错误**（semantic error，程序含义的错误）和**运行时错误**（runtime error，程序运行时发生的错误）。

基本知识点

- 定位和修正程序中的错误被称为**调试**程序。

对新手来说，语法错误是最常见的，包括一些简单的错误，如忘记文本串开头或者结尾的其中一个引号，没有关闭开括号，或者拼错函数名print()。当Python解释器遇到这些错误时，通常会努力指出这些错误，显示问题代码所在行的行号以及问题描述。经过一些练习之后，初学者能很快学会识别和解释常见的错误情况。下面来看几个例子：

```
print(5 + )
SyntaxError: invalid syntax
```

上述表达式在加法运算符和闭括号之间缺少一个值。

```
print(5.e)
SyntaxError: invalid token
```

Python希望基数小数点后面是数字，而不是字母。

```
pront(5)
NameError: name 'pront' is not defined
```

就像叫错一个人的名字一样，拼错已知函数或变量的名字会产生混淆和尴尬。

语义错误是算法中的缺陷，或者某种语言表达算法方式中的缺陷。这样的例子可能包括：在一个计算中使用了错误的变量名，或者弄错了复杂表达式中算术运算符的顺序。Python遵循运算符优先级的标准规则，所以在像`total_pay = 40 + extra_hours * pay_rate`这样的表达式中，乘法会在加法之前执行，错误地计算总工资。（除非你的工资率恰好是1美元/小时。）使用括号正确地指定复杂表达式中的运算顺序，如`total_pay = (40 + extra_hours) * pay_rate`，既可以避免语义错误，又可以避免写出的代码难于理解。

最后，这个层次上的运行时错误可能包括：无意识地除以0，或者使用未定义的变量。Python是从上到下读取语句的，在表达式里使用变量之前，Python必须看到变量的赋值语句。

测试（testing）是高效编写Python脚本（或者任何种类的实际程序）必不可少的一部分。在编写脚本时要频繁运行它，可能每完成一行代码就需要运行一次。这样做能尽早识别和修复语法错误，把程序设计者的注意力集中在脚本每一步应该做什么上。

基本知识点

- 知道程序应该做什么才能找出大多数程序错误。

1.8节问题与练习

1. 是什么使Python成为一门**解释型**程序设计语言？
2. 编写Python语句输出下列内容。
 - a. “Computer Science Rocks”这些词，后面跟一个感叹号。
 - b. 数42。
 - c. π 的近似值，精确到基数小数点后第四位。
3. 编写Python语句按下列描述给变量赋值。
 - a. 将“programmer”这个词赋值给一个名为rockstar的变量。
 - b. 将1小时的秒数赋值给一个名为seconds_per_hour的变量。
 - c. 将人体的平均温度赋值给一个名为bodyTemp的变量。
4. 编写一条Python语句，给定一个已有的名为bodyTemp的变量，单位为华氏度，将与其相等的摄氏度存储到一个名为metricBodyTemp的新变量中。

*1.9 数据压缩

为了存储或传输数据，在保留基础信息的同时，缩小所涉及数据的大小是有益的（有时也是强制的）。用于完成这一过程的技术称为**数据压缩**（data compression）。本节，我们首先学习一些通用的数据压缩方法，然后了解一些为特殊应用设计的方法。

1.9.1 通用的数据压缩技术

数据压缩方案有两类，一类是**无损的**（lossless），另一类是**有损的**（lossy）。无损方案在压缩过程中是不丢失信息的，有损方案在压缩过程中会丢失信息。通常有损技术比无损技术更能压缩，因此在可以容忍小错误的应用中很流行，如图像和音频压缩。

基本知识点

- 在使用有损压缩技术和无损压缩技术来存储和传输数据方面需要权衡。

对于被压缩数据由一长串相同的值组成的情况，流行使用称为**行程长度编码**（run-length encoding）的压缩技术，这是一种无损方法。它的过程是，将一组相同的数据成分替换成一个编码，指出重复的成分以及其在序列中出现的次数。例如，指出一个位模式包括253个1，接着118个0，接着87个1，这样要比实际列出458个位节省空间。

另外一种无损数据压缩技术是**频率相关编码**（frequency-dependent encoding），在这个系统里，用于表示数据项的位模式的长度与这个项的使用频率是反相关的。这种编码是变长编码的例子，意思是项由不同长度的模式表示。戴维·哈夫曼（David Huffman）的功劳是发现了一种常用于开发频率相关编码的算法，人们一般称用这种方法开发的编码为**哈夫曼编码**（Huffman code）。因此，现在使用的大多数频率相关编码都是哈夫曼编码。

让我们看一个频率相关编码的例子，考虑一下编码英文文本的任务。在英文中，字母e、t、a和i的使用频率要大于字母z、q和x。因此，当为英文文本编码时，如果用短位模式表示前面的字母，用长位模式表示后面的字母，就能节省空间。用这种方式编码的英文文本的表示长度要比用统一长度编码的短。

在某些情况下，要压缩的数据流由数据单元组成，每个数据单元与其前面一个的差别很小。动画的连续帧就是一个例子。在这种情况下，使用**相对编码**（relative encoding）——也称为**差分编码**（differential encoding）——技术是很有用的。这些技术记录的是两个连续数据单元之间的区别，而不是整个单元；也就是说，每个单元是根据其与前一个单元的关系被编码的。相对编码可以用无损形式实现，也可以用有损形式实现，具体取决于两个连续数据单元之间的差别是精确编码的还是近似编码的。

还有其他流行的基于**字典编码**（dictionary encoding）技术的压缩系统，这里的术语**字典**（dictionary）指的是一组构建块，压缩的消息通过它们建造起来，而消息本身被编码成字典的一系列参照。我们一般认为字典编码系统是无损系统，不过在学习图像压缩时我们将看到，有时候字典条目仅仅是正确数据成分的近似值，这就使其成了有损压缩系统。

字处理程序可以使用字典编码来压缩文本文档，因为这些字处理程序中已经包含了用于拼写检查的字典，而这些字典都是出色的压缩字典。特别值得一提的是，一个完整的单词可以编码为字典的一个单独参照符，而不是像使用UTF-8这样的系统那样编码成一系列单个字符。在字处理程序中，一个典型的字典大

约有25 000个条目，这意味着，单个的字典条目可以用0到24999范围内的整数来标识。也就是说，字典中的一个特定条目用15位的模式就足可标识。与之相比，如果用到的单词包括6个字母，则它的逐字符编码在使用UTF-8时需要48位。

字典编码的一个变体是[自适应字典编码](#)（adaptive dictionary encoding，也称动态字典编码）。在自适应字典编码系统中，字典在编码过程中是可以改变的。一个流行的例子是[LZW编码](#)（Lempel-Ziv-Welsh encoding），这个编码的名称是根据它的创造者Abraham Lempel、Jacob Ziv和Terry Welsh的姓氏命名的。要用LZW对消息编码，首先要从包含基础构建块的字典开始，用字典中的构建块来构造消息。但是，当人们在消息中发现较大的单元时，会把它们加到字典中——这意味着，这些单元在未来出现时可以被编码为一个（而不是多个）字典参照。例如，当对英文文本编码时，人们首先要从包含单独字符、数字和标点符号的字典开始。但是，当消息中的单词被标识后，会被加到字典中。因此，随着消息的不断编码，字典会不断增大，而随着字典的不断增大，消息中会有更多的单词（或者重复的单词模式）被编码为单个的字典参照。

结果将是，消息用一部相当大的、完全针对本消息的字典进行编码。但是在对这条消息进行解码时，不必用这个大字典，只需用原始的小字典即可。事实上，解码过程可以与编码过程用同一个小字典。接着，随着解码过程的继续，它会遇到编码过程中发现的相同单元，进而将它们加到字典中，作为未来编码过程的参照。

举例说明，考虑对下列消息应用LZW编码：

xyx xyx xyx xyx

首先从一个只有3个条目的字典开始，第一个条目是x，第二个条目是y，第三个条目是一个空格。我们先将xyx编码为121，意思是这条消息的第一个模式依次包括第一个字典条目、第二个字典条目和第一个字典条目。接着，空格被编码，产生结果1213。但是，因为有了—个空格，我们知道前面的字符串已经形成了一个单词，所以我们将模式xyx加到字典中作为第四个条目。继续使用这种编码方式，整条消息将被编码为121343434。

如果我们现在从原始的3条目字典开始对这条消息进行解码，那么我们首先要将起始的1213串解码为xyx后加1个空格。这时我们意识到字符串xyx形成了一个单词，因此将其加到字典中作为第四个条目，同编码过程中所做的一样。我们接着对这条消息解码，发现消息中的4指的是这第四个新条目，将其解码为单词xyx，产生模式

xyx xyx

继续使用这种解码方式，我们最终把字符串121343434解码为

xyx xyx xyx xyx

这就是原始消息。

基本知识点

- 虽然无损数据压缩会减少存储或传输的位数，但允许完整重构原始数据。
- 有损数据压缩可以大大减少存储或传输的位数，但代价是只能重构原始数据的近似值。

1.9.2 图像压缩

在1.4节中，我们已经看到如何用位图技术对图像编码。不过，得到的位图通常是非常大的。因此，人们专门为图像表示开发出了许多压缩方案。

一种称为**GIF**（是graphic interchange format的缩写，即图像交换格式，一些人将其读作“Giff”，还有一些人将其读作“Jiff”）的系统是CompuServe公司开发的一个字典编码系统。它处理压缩问题的方法是，将赋予一个像素的颜色数量减少到只有256个。每种颜色的“红—绿—蓝”组合都用3字节编码，这256个编码被存储在一个称为调色板的表格（字典）里。图像中的每个像素都可以用一个字节表示，该字节的值指明了这个像素的颜色是由256个调色板条目中的哪一个表示的。（回顾一下，一个字节能够包括256个不同位模式中的任意一个。）注意，在将GIF应用于任意图像时，它是一个有损压缩系统，因为调色板中的颜色可能与原始图像中的颜色不一致。

通过使用LZW技术将这个简单的字典系统扩展为自适应字典系统，GIF可以进一步压缩。尤其是，编码过程中遇到的像素模式可以被加到字典中，使这些模式在将来出现时可以被更加高效地编码。因此，最终的字典是由原始调色板和一组像素模式构成的。

GIF调色板中的某一个颜色通常会被赋予值“透明”，这意味着背景色可以透过每个被赋予该“颜色”的区域展现出来。这个选择与GIF系统的相对简便性相结合，使得GIF成为简单动画应用（这种动画应用中的多重图像只能在计算机屏幕上移动）的合理选择。另外，它只能够对256种颜色编码，这使得它不适合精度要求较高的应用，如摄影领域。

另外一种流行的图像压缩系统是**JPEG**（读作“JAY-peg”）。它是由ISO中的**联合图像专家组**（Joint Photographic Experts Group，标准因此得名）研制开发的标准。JPEG已经被证明是压缩彩色照片的一种有效标准，并被广泛用于摄影业，这一点可由大多数数码相机都将JPEG作为它们默认的压缩技术这一事实来印证。

JPEG标准实际上包含多种图像压缩方法，每种都有它自己的目标。在需要尽可能精确的情况下，JPEG可提供无损模式。不过，相对于JPEG的其他模式，JPEG的无损模式不会产生高级别的压缩。此外，JPEG的其他模式已被证明很成功，这意味着人们很少使用JPEG的无损模式。相反，称为JPEG基线标准（也称为JPEG的有损顺序模式）的JPEG模式已经成为许多应用的选择标准。

使用JPEG基线标准的图像压缩需要几个步骤，其中有一些是利用人眼的局限性设计的。尤其是，相对于颜色的变化，人眼对亮度的变化更为敏感。因此，我们首先来看一幅根据亮度成分和色度成分进行编码的图像。第一步是在一个 2×2 的像素方块中求色度的平均值。这能将色度信息的大小减小为 $1/4$ ，同时保留所有的原始亮度信息。结果是，在没有明显损失图像质量的情况下获得了很高的压缩率。

下一步是将图像划分成多个 8×8 的像素块，并将每个块中的信息压缩为一个单元。这可以通过一种称为离散余弦转换的数学技术来实现，这里我们不需要关心这个转换的细节。重要的一点是，这种转换能将原始的 8×8 块变成了另一种块（这种块中的条目反映了原始块中的像素之间是如何相互联系的），而不是实际的像素值。在这个新块里，那些低于预订阈值的值将被替换成0，反映的事实是这些值所表示的变化非常小，人眼无法觉察。例如，如果原始块中包含一个棋盘（checkerboard）模式，那么新块就可能表现为均匀的平均色。（一个典型的 8×8 像素块表示的是图像中一个非常小的方块，因此人眼根本不能够识别棋盘的外观。）

这时候，可以应用更传统的行程长度编码、相对编码以及变长编码技术进行进一步的压缩。总之，JPEG基线标准一般能在没有明显质量损失的情况下，将彩色图像压缩至少10倍，有时甚至能压缩30倍。

另一个与图像相关的数据压缩系统是TIFF（tagged image file format的缩写，即标记图像文件格式）。不过，TIFF最普遍的应用不是数据压缩，而是存储照片及其相关的信息（如日期、时间以及相机设置）的一个标准格式。在存储照片时，图像本身通常会被存储为没有压缩的红、绿、蓝像素成分。

TIFF标准集合里的确有数据压缩技术，其中大多数是为在传真应用中压缩文本文档的图像设计的。为了利用文本文档包含长串的白色像素这一事实，这些标准使用了行程长度编码的变体。TIFF标准中的彩色图像压缩选择是以类似于GIF所使用的技术为基础的，因此并没有被广泛应用于摄影业。

1.9.3 音频和视频压缩

最常用的音频及视频的编码和压缩标准是由ISO领导的运动图像专家组（Motion Picture Experts Group，MPEG）开发的，因此这些标准称为MPEG。

MPEG包含许多不同应用的各种标准。例如，高清电视（high definition television，HDTV）广播的要求与视频会议的要求就不同，广播信号必须找到在各种容量可能很有限的通信路径上传输的方式。另外，这两种应用又都不同于以可回放或跳过片断的方式存储视频的应用。

MPEG使用的技术已经超出了本书的范围，但是一般说来，视频压缩技术是以图片序列构建成的视频为基础的，与将运动图像录制到胶片上的方式基本相同。为了压缩这样的序列，只有一些称为I帧（I-frame）的图片是被整个编码的。两个I帧之间的图片是采用相对编码技术进行编码的，也就是说，不对整个图片进行编码，只记录与前一个图片的差别。I帧本身经常使用类似于JPEG的技术进行压缩。

最著名的音频压缩系统是**MP3**，它是在MPEG标准中开发出来的。事实上，MP3是MPEG layer 3的缩写。与其他压缩技术相比，MP3利用人耳的特性，删除了人耳觉察不到的细节。其中一个特性称为**时域掩蔽**（temporal masking），指的是在一次巨大声响后，短时间内人耳觉察不到本可以听见的轻柔声音。另一个称为**频域掩蔽**（frequency masking），指的是某一频率的声音可能掩盖相近频率的轻柔声音。利用这些特性，可以通过MP3获得显著的音频压缩，而且保持接近CD的音质。

使用MPEG和MP3压缩技术，摄像机用128 MB的存储空间就可以录制长达1小时的视频，便携音乐播放器用1 GB的存储空间就可以存储多达400首流行歌曲。但是，不同于其他压缩目的，音频和视频的压缩目的不一定是节省存储空间。真正重要的是获得编码，使信息能够通过现在的通信系统得到及时的传输。如果每个视频帧需要1 MB的存储空间，而传播帧的通信路径每秒只能中继1 KB，那么根本无法实现成功的视频会议。因此，除了被认可的复制质量，音频和视频压缩系统的选择还要看及时数据通信所需的传输速度。这些速度通常用**位每秒**（bits per second，bit/s）来度量。常见的单位包括Kbit/s（kilo-bps的简写，即千位每秒，等于 10^3 bit/s）、Mbit/s（mega-bps的简写，即兆位每秒，等于 10^6 bit/s）和Gbit/s（giga-bps的简写，即吉位每秒，等于 10^9 bit/s）。使用MPEG技术，视频展示可以在40 Mbit/s传输速率的通信路径上成功中继。MP3录制需要的传输速率一般不超过64 Kbit/s。

1.9节问题与练习

1. 列出4种通用的压缩技术。
2. 如果使用LZW压缩，并且字典最初只包含x、y和一个空格（如文中所述），对下列消息编码，结果是什么？

```
xyx yxxxxy xyx yxxxxy yxxxxy
```

3. 在对彩色卡通编码时，为什么GIF比JPEG要好？
4. 假设你是航天器设计团队的一员，要设计能驶向其他星球并发回照片的航天器。那么，为了减少存储和传输图像所需的资源，使用GIF或JPEG的基准标准压缩照片是否是一个好主意？
5. JPEG的基准标准利用了人眼的什么特性？
6. MP3利用了人耳的什么特性？
7. 说出一种在把数字信息、图像和声音编码为位模式时常见的麻烦现象。

*1.10 通信差错

当信息在一台计算机的各个部分之间来回传输，或者在月球和地球之间来回传输，又或者只是被保存在存储器中时，最终检索到的位模式有可能和原始的位模式不一致。磁记录面上的油脂或灰尘颗粒，或者出了故障的电路，都可能使数据被错误地记录或读取。传输通道上的静电干扰可能会损坏部分数据。在某些技术条件下，普通的背景辐射（background radiation）可以改变存储在机器的主存储器中的模式。

为了解决这样的问题，人们开发了许多编码技术来检测甚至校正差错。现在，由于这些技术被大规模地内置于计算机系统的内部构件中，因此计算机的使用者并不了解它们。不过，它们的存在很重要，对科学研究有重大贡献。因此，我们有必要了解其中一些使现在设备可靠的技术。

1.10.1 奇偶校验位

有一种简单的检测差错的方法，其依据的原则是：如果被操作的每个位模式都有奇数个1，却遇到了有偶数个1的模式，那么一定是出错了。要使用这个原则，需要编码系统中的每个模式都有奇数个1。这是很容易做到的，首先在已经可用的编码系统中的模式（也许在高位端）上添加一个称为**奇偶校验位**（parity bit）的附加位。根据不同情况，我们给这个新的位赋值1或0，使整个结果模式有奇数个1。一旦这样修改了编码系统，出现有偶数个1的模式就表示出现了差错，被操作的模式是不正确的。

图1-26展示了如何将奇偶校验位加到字母A和F的ASCII码上。注意，A的编码变成了101000001（奇偶校验位为1），F的ASCII码变成了001000110（奇偶校验位为0）。尽管A原始的8位模式有偶数个1，F原始的8位模式有奇数个1，但它们的9位模式都有奇数个1。如果将这种技术应用于所有的8位ASCII模式，我们就能够得到一个9位编码系统，其中任何一个9位模式有偶数个1就表示出错了。

上面描述的奇偶校验系统称为**奇校验**（odd parity），因为我们设计的系统中每个正确的模式都有奇数个1。另一种技术称为**偶校验**（even parity）。在偶校验系统中，每个模式都会被设计成包含偶数个1，因此，如果出现奇数个1，就表明出错了。

现在，在计算机的主存储器中使用奇偶校验位已经不是一件稀奇事了。尽管我们假设这些计算机存储单元是8位的，但事实上，它们可能是9位的，其中1位被用作奇偶校验位。每次存储器电路收到要存储的8位模式，都会先给其加上一个奇偶校验位，形成9位模式，然后再存储。稍后检索模式时，存储器电路会检查这个9位模式的奇偶性。如果检查没有发现差错，存储器会移除校验位，然后自信地返回余下的8位模式。否则，存储器会返回那8个数据位，并警告返回的模式可能与原本委托给存储器的模式不同。

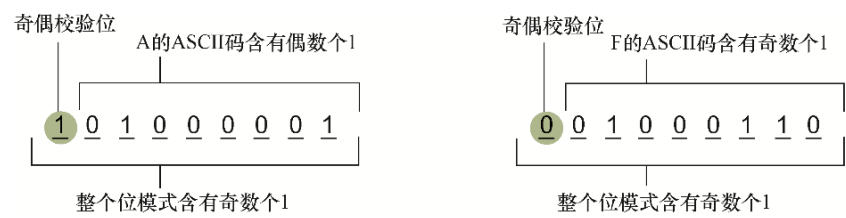


图1-26 为奇校验调整的字母A和F的ASCII码

直接使用奇偶校验位很简单，不过它有局限性。如果一个模式最初有奇数个1，并出现了2次差错，那么它就仍然有奇数个1，这样校验系统就无法检测到这些差错。事实上，直接使用奇偶校验位并不能发现模式中的任何偶数个差错。

有时会对长位模式（如记录在磁盘扇区中的位串）应用一种方法来最大限度地减少这类问题。在这种情况下，模式都有一组奇偶校验位构成的**校验字节**（checkbyte）。校验字节中的每个位都是一个奇偶校验位，与散布在整个模式中的一组特殊位相联系。例如，一个奇偶校验位可能与该模式中从第一个位起的每个第8位相关联，而另一个与该模式中从第二位起的每个第8位相关联。用这种方式，更容易检测到集中在原始模式某个区域中的一些差错，因为这些差错会在一些奇偶校验位的范围内。这个校验字节概念的演变出了称为**校验和**（checksum）及**循环冗余校验**（cyclic redundancy check，CRC）的差错检测方案。

1.10.2 纠错码

虽然使用奇偶校验位可以检测到差错，但是它不能提供纠正那个差错所需的信息。让很多人感到惊讶的是，居然有人设计出了既能够检测出差错又能够纠正差错的**纠错码**（error-correcting code）。毕竟，直觉告诉我们，如果不知道消息的内容就无法纠正接收到的消息中的差错，但图1-27展示了一个具有这种纠错特性的简单编码。

为了明白这个编码是如何运作的，我们先来定义**汉明距离**（Hamming distance），这个术语是根据R. W. Hamming的姓氏命名的，由于对20世纪40年代早期继电器缺乏可靠性的失望，他开始开创性地研究纠错码。两个位模式之间的汉明距离指的是这两个模式中不相同位的个数。例如，在图1-27的编码中，表示A和B的模式之间的汉明距离是4，而B和C之间的汉明距离是3。图1-27中的编码的重要特征是，任何两个模式之间的汉明距离至少是3。

如果图1-27所示的模式中某个位被修改了，差错就能被检测出来，因为结果不会是一个合法的模式。（我们必须至少改变任意一个模式的3个位，这个模式看起来才会像另一个合法模式。）而且，我们还能指出原始模式是什么。毕竟，修改过的模式和其原始形式的汉明距离是1，而和其他任何合法模式的汉明距离至少是2。

因此，对最初用图1-27编码的消息解码，我们只需简单地对比每个接收到的模式和用此编码表示的模式，直到找到一个和接收到的模式之间的汉明距离是1的模式为止。我们将其视为正确的符号进行解码。例如，我们接收到位模式010100，并将其与编码中的模式相比，就会得到图1-28所示的表格。因此，我们得出结论，传输的字符一定是D，因为这是最接近的匹配。

符 号	编 码
A	000000
B	001111
C	010011
D	011100
E	100110
F	101001
G	110101
H	111010

图1-27 一个纠错码

字 符	编 码	接收到的模式	接收到的模式与 编码之间的距离
A	0 0 0 0 0 0	0 1 0 1 0 0	2
B	0 0 1 1 1 1	0 1 0 1 0 0	4
C	0 1 0 0 1 1	0 1 0 1 0 0	3
D	0 1 1 1 0 0	0 1 0 1 0 0	1
E	1 0 0 1 1 0	0 1 0 1 0 0	3
F	1 0 1 0 0 1	0 1 0 1 0 0	5
G	1 1 0 1 0 1	0 1 0 1 0 0	2
H	1 1 1 0 1 0	0 1 0 1 0 0	4

最小
距离

图1-28 用图1-27中的编码对模式010100解码

通过观察可以发现，使用图1-27中的编码方法，每个模式我们最多能检测出2个差错、改正1个差错。如果我们能设计出一种编码，使每个模式和其他任何模式之间的汉明距离都至少是5，那么就能最多检测出4个差错、改正2个差错。当然，设计一种具有长汉明距离的编码并不是一件简单的事。事实上，它构成了数学分支的一部分，称为代数编码理论，它是线性代数和矩阵理论的子领域。

纠错技术被广泛用于提高计算设备的可靠性。例如，它们经常被用于大容量磁盘驱动器，以减少因磁盘表面瑕疵而损坏数据的可能性。此外，最初用于音频盘的CD格式与后来用于计算机数据存储的格式之间的主要差别是纠错的程度。CD-DA格式具有纠错功能，它能把差错率减少到2张CD中只有1个差错。这种格式非常适合于音频录制，但是对于用CD向客户交付软件的公司，若50%的盘有瑕疵是令人无法忍受的。因此，附加纠错功能被用在CD中来存储数据，将出错概率减少到20 000个盘中有1个出错。

1.10节问题与练习

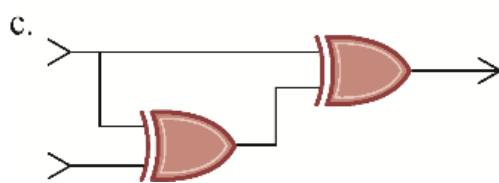
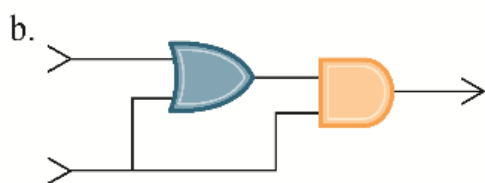
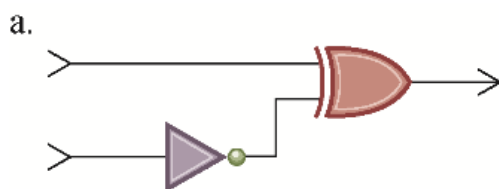
- 下面的字节最初是用奇校验编码的。你知道它们中哪个有错误吗？
a . 100101101 b . 100000001 c . 000000000 d . 111000000 e . 011111111
- 问题1中未发现错误的字节还会有错误吗？请解释你的答案。
- 如果将奇校验换成偶校验，那么问题1和问题2的答案又将如何？
- 通过在每一个字符编码的高位端加一个奇偶校验位，用带奇校验的ASCII码对下列语句编码。
a . “Stop!” Cheryl shouted.
b . Does $2 + 3 = 5$?
- 用图1-27的纠错码解码下面的消息。
a . 001111 100100 001100
b . 010001 000000 001011
c . 011010 110110 100000 011100
- 用长度为5的位模式，为字符A、B、C和D构造编码，使得任何两个模式之间的汉明距离至少是

3。

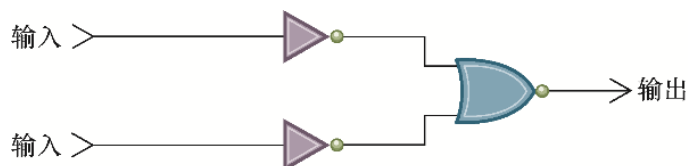
复习题

(带*的题目涉及选学章节的内容。)

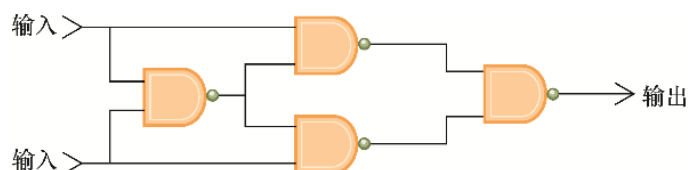
1. 假设上输入为1, 下输入为0, 请确定下面每个电路的输出值。如果上输入为0, 下输入为1呢?



2. a. 下面这个电路做什么布尔运算?

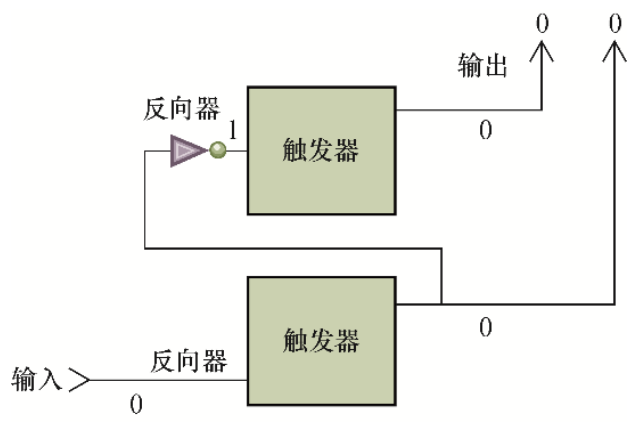


b. 下面这个电路做什么布尔运算?

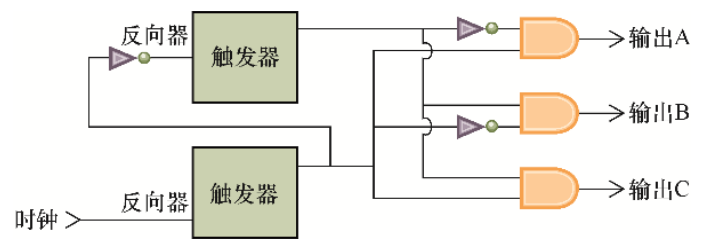


*3. a. 如果要从某个电子元件商店购买触发器电路, 我们会发现它有一个额外的输入, 称为**反向器** (flip)。当反向器的输入从0变成1时, 输出将翻转状态 (如果原来是0, 现在就是1, 反之亦然)。但是, 当反向器的输入从1变为0时, 什么都不会发生。虽然我们可能不知道这个电路完成这一行为的细节, 但我们仍然可以将其用作其他电路的抽象工具。请考虑使用下面两个触发器的电路。如果在电路的输入端发送一个脉冲, 下面的那个触发器将变换状态。但是, 另一个触发器不会改变, 因为其输入 (这一输入是从非门的输出接收到的) 从1变成了0。因此, 这一电路现在的输出为0和1。在电路输入端发送第二个脉冲

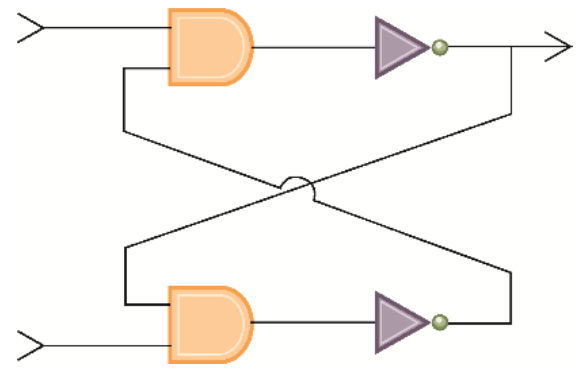
将翻转两个触发器的状态，并产生输出1和0。如果在电路输入端发送第三个脉冲，将会产生什么输出呢？发送第四个脉冲呢？



b. 计算机中不同元件的活动一般是需要协调的。只需将脉冲信号（称为**时钟**）连接到类似a中的电路即可。额外的门（如图所示）以协同的方式发送信号到其他连接的电路。研究这一电路时，你应该能够确认这样一个事实，即在第1个、第5个、第9个.....时钟脉冲下，输出端**A**将发送**1**。哪些时钟脉冲将使输出端**B**发送**1**？哪些时钟脉冲将使输出端**C**发送**1**？在第4个时钟脉冲下，哪个输出端的输出为1？



4. 假设下面电路的两个输入都是1。请描述如果上输出暂时变为0，会发生什么。如果下输入暂时变为0，又会发生什么。用与非门重新绘制这个电路。



5. 下面表格显示的是（采用十六进制记数法表示的）机器的主存储器某些单元的地址和内容。根据这个存储排列，按照下列指令，记录下这些存储单元的最终内容。

地 址	内 容
0x00	0xAB
0x01	0x53
0x02	0xD6
0x03	0x02

步骤1：将地址为0x03的单元的内容移入地址为0x00的单元中。

步骤2：将值0x01移入地址为0x02的单元。

步骤3：将地址0x01中存储的值移入地址为0x03的单元。

6．如果每个单元地址都可以用2个十六进制数字表示，那么一台计算机的主存储器中可以有多少个单元？如果用4个十六进制数字呢？

7．什么位模式可以用下面的十六进制记数法表示？

- a．0xCD b．0x67 c．0x9A
- d．0xFF e．0x10

8．下列用十六进制记数法表示的位模式中，最高有效位的值是什么？

- a．0x8F b．0xFF
- c．0x6F d．0x1F

9．用十六进制记数法表示下面的位模式。

- a．101000001010
- b．110001111011
- c．000010111110

10．假设一个数码相机的存储容量是256 MB。如果一张照片每行每列都有1024像素，而且每个像素需要3字节的存储空间，那么这个数码相机可以存储多少张这样的照片？

11. 假设显示屏上的一张图片是由一个768行×1024列的像素矩阵表示的。如果每个像素的颜色需要用8位进行编码，并且亮度需要用另外8位进行编码，那么要保存整幅图片需要多少字节的存储单元？

12. a. 指出主存储器优于磁盘存储器的两个优点。

b. 指出磁盘存储器优于主存储器的两个优点。

13. 假设你个人计算机上的120 GB的硬盘只剩下50 GB是空闲的，那么用CD备份硬盘上的所有资料是否合理？用DVD呢？

14. 如果磁盘的每个扇区包含1024字节，而且每个字符用2字节的Unicode字符表示，那么存储一页文本（如50行，每行100个字符）需要多少扇区？

15. 如果用ASCII码，那么存储一本400页、每页3500个字符的小说需要多少字节的存储空间？如果用2字节的Unicode字符又需要多少字节？

16. 一个典型硬盘驱动器的转速为每秒3600转，那么它的等待时间是多少？

17. 如果一个硬盘驱动器的转速为每秒360转，寻道时间是10 ms，那么它的平均存取时间是多少？

18. 假设一个打字员每天打字24小时，每分钟打60个字，那么这个打字员要多久才能填满容量为640 MB的CD？假定一个单词由5个字符构成，每个字符需要1字节的存储空间。

19. 下面是用ASCII编码的消息。内容是什么？

```
01010111 01101000 01100001 01110100
00100000 01100100 01101111 01100101
01110011 00100000 01101001 01110100
00100000 01110011 01100001 01111001
00111111
```

20. 下面的消息是使用ASCII编码的，每个字符1字节，并用十六进制记数法表示出来。这条消息的内容是什么？

```
0x686578206E6F746174696F6E
```

21. 用ASCII对下面的句子编码，每个字符1字节。

a. Does $100/5 = 20$?

b . The total cost is \$7.25.

22 . 将前面问题的答案用十六进制记数法表示出来。

23 . 列出整数8到18的二进制表示。

24 . a . 通过用ASCII表示2和3 , 写出数23。

b . 用二进制表示写出数23。

25 . 什么值的二进制表示只有一个位为1 ? 列出具有这个特性的最小的6个值的二进制表示。

*26 . 将下面的每个二进制表示转换成相应的十进制表示。

a . 1111 b . 0001 c . 10101

d . 1000 e . 10011 f . 000000

g . 1001 h . 10001 i . 100001

j . 11001 k . 11010 l . 11011

*27 . 将下面每个十进制表示转换成相应的二进制表示。

a . 7 b . 11 c . 16

d . 17 e . 31

*28 . 将下面的每个余16表示转换成相应的十进制表示。

a . 10001 b . 10101 c . 01101

d . 01111 e . 11111

*29 . 将下面的每个十进制表示转换成相应的余4表示。

a . 0 b . 3 c . -2

d . -1 e . 2

*30 . 将下面的每个二进制补码表示转换成相应的十进制表示。

a . 01111 b . 10100 c . 01100

d . 10000 e . 10110

*31. 将下面的每个十进制表示转换成相应的二进制补码表示，其中每个值用7位表示。

a. 13 b. -13 c. -1

d. 0 e. 16

*32. 假定下面这些位串都是用二进制补码记数法表示的值，执行下面这些加法运算。指出哪一个加法的答案是由于溢出而不正确的。

a. 00101+01000 b. 11111+00001

c. 01111+00001 d. 10111+11010

e. 11111 +11111 f. 00111 +01100

*33. 解答下面的每个问题：将这些值翻译成二进制补码记数法（用5位模式），把减法问题转换成相应的加法问题，并执行那个加法运算。将所得答案转换成十进制记数法进行验证。（观察溢出现象。）

a. 5+1 b. 5-1 c. 12-5

d. 8-7 e. 12+5 f. 5-11

*34. 将下面的每个二进制表示转换成相应的十进制表示。

a. 11.11 b. 100.0101 c. 0.1101

d. 1.0 e. 10.01

*35. 用二进制记数法表示下面每个值。

a. $5\frac{3}{4}$ b. $15\frac{15}{16}$ c. $5\frac{3}{8}$

d. $1\frac{1}{4}$ e. $6\frac{5}{8}$

*36. 用图1-24中描述的浮点格式对下面的位模式解码。

a. 01011001 b. 11001000 c. 10101100

d. 00111001

*37. 用图1-24中描述的8位浮点格式对下面的值编码。指出出现截断误差的每个情形。

a. $-7\frac{1}{2}$ b. $\frac{1}{2}$ c. $-3\frac{3}{4}$

$$d. \frac{7}{32} \quad e. \frac{31}{32}$$

*38. 假定你不受使用标准化格式的限制，用图1-24中描述的浮点格式列出所有可以用于表示值 $\frac{3}{8}$ 的位模式。

*39. 用图1-24中描述的8位浮点格式，求可以表示的最近似于2的平方根的值。如果机器利用这一浮点格式对这个数做平方，实际得到的值是什么？

*40. 用图1-24所示的8位浮点格式可以表示的最近似于 $\frac{1}{10}$ 的值是什么？

*41. 当用浮点记数法记录使用公制系统的度量时，误差是怎么产生的？例如，如果用米制单位记录110 cm会怎样？

*42. 位模式01011和11011表示的是同一个值，一个是用余16记数法存储的，另一个是用二进制补码记数法存储的。

a. 关于这个公共的值可以确定的是什么？

b. 分别用二进制补码记数法和余码记数法存储同一个值，并且这两个系统采用相同的位模式长度，那么表示此值的这两种模式是一种什么关系？

*43. 3个位模式10000010、01101000和00000010表示的是同一个值，采用的是3种不同的表示法：二进制补码记数法、余码记数法和图1-24所示的8位浮点格式。但这3个位模式和这3个表示法的顺序不一定是一一对应的。它们共同表示的那个值是什么？哪个模式对应哪个记数法？

*44. 在下面的值中，哪一个不能用图1-24所示的浮点格式精确地表示？

$$a. 6\frac{1}{2} \quad b. \frac{13}{16} \quad c. 9$$

$$d. \frac{17}{32} \quad e. \frac{15}{16}$$

*45. 如果用于表示二进制整数的位串长度由4变成6，那么能够表示的最大整数的值会发生什么变化？用二进制补码记数法又将如何呢？

*46. 如果一个4 MB的存储器每个单元可以存储1字节，那么其最大存储地址的十六进制表示是什么？

*47. 如果使用LZW压缩，从只包含x、y和一个空格（如1.8节所述）的字典开始，对下列消息编码，结果是什么？

xyx yyx xxy xxy yyx

*48. 下面的消息是使用LZW压缩的，其字典的第一、第二、第三个条目分别是x、y和空格。这条消息的解压缩结果是什么？

22123113431213536

*49. 如果消息

xyy yyx xxy xxyy

是用LZW压缩的，并且最初字典的第一、第二、第三个条目分别是x、y和空格，那么最后的字典里有哪些条目？

*50. 我们将在第2章学到，通过传统电话系统传输位的一种方法：首先将位模式转换成声音，然后通过电话线传输声音，最后再将声音转换回位模式。这种技术的传输速率最高可达57.6 Kbit/s。如果视频采用MPEG压缩，那么这种技术能否满足远程会议的需要？

*51. 使用ASCII对下面的句子编码，使用偶校验，在每个字符编码的高位端添加一个奇偶校验位。

a. Does 100/5 = 20?

b. The total cost is \$7.25.

*52. 下面的消息的每个短位串最初都是用奇校验传输的。哪一个位串肯定会出现差错？

11001 11011 10110 00000 11111

10001 10101 00100 01110

*53. 假如一个24位的编码是这样产生的：将一个符号的ASCII表示连续复制3次，用得到的结果表示该符号（例如，符号A用位串010000010100000101000001表示）。这个新编码有哪些纠错特性？

*54. 用图1-28的纠错码对下面的词语进行解码。

a. 111010 110110

b. 101000 100110 001100

c. 011101 000110 000000 010100

d. 010010 001000 001110 101111

000000 110111 100110

e . 010011 000000 101001 100110

*55 . 国际货币汇率变化很频繁。研究一下货币汇率，并相应地更新1.8节的货币转换器脚本。

*56 . 找出一种1.8节的那个货币转换器里没有的货币。得到它的当前汇率，并在网络上找到它的Unicode货币符号。扩展货币转换器脚本，让它能够转换这种新货币。

*57 . 如果你的网络浏览器和文本编辑器能很好地支持Unicode和UTF-8，那么请把实际的国际货币符号复制/粘贴到1.8节的转换器脚本中，替换掉'\u00A3'这样的复杂代码。（如果你的软件处理Unicode有困难，那么当你尝试这样做时，你的文本编辑器里可能会出现奇怪的符号。）

*58 . 1.8节的货币转换器脚本在执行每个乘法之前，用变量dollars存储了要转换的金额。这使得脚本中每个乘法代码行的长度，比直接键入整数数量1000的长度要长。提前创建这个额外的变量有什么好处？

*59 . 编写并测试一个Python脚本，给定一个字节数，输出与其相等的千字节数、兆字节数、吉字节数和太字节数。再编写并测试一个互补脚本，给定一个太字节数，输出与其相等的GB数、MB数、KB数以及字节数。

*60 . 编写并测试一个Python脚本，给定一个录音的分秒数，计算对这个时长的未压缩的CD品质的立体声音频数据进行编码所用的位数。（复习1.4节的必要参数和公式。）

*61 . 指出下面Python脚本中的错误。

```
days_per_week = 7
weeks_per_year = 52
days_per_year = days_per_week **
weeks_per_year
PRINT(days_per_year)
```

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1 . 某个截断误差出现在一个关键时刻，引起了巨大的损失和人身伤亡。如果需要有人对此负责，那是谁？是硬件设计者？软件设计者？实际编写那段程序的程序员？还是决定在那个特定应用中使用这个软件的人？如果最初开发这个软件的公司已经修正过这个软件，但是用户没有为那个关键应用购买并应用这个升级版，又将如何？如果这个软件是盗版的呢？

2 . 对个人来讲，在开发他自己的应用时忽略截断误差的可能性以及它们的后果，是可以接受的吗？

3. 20世纪70年代开发的软件只用2个数字表示年（如用76表示1976年），忽视了这个软件在即将到来的世纪之交会有缺陷这个事实，这道德吗？若现在只用3个数字表示年（如用982表示1982年，用015表示2015年）又是否道德呢？如果只用4个数字呢？

4. 许多人认为，对信息进行编码经常会削弱或歪曲该信息，因为这实质上迫使信息必须被量化。他们认为，若一份调查问卷要求调查对象只能用给定的5个等级来发表他们的意见，那么这份问卷本身就是有缺陷的。信息可以量化到什么程度？垃圾处理场选址的利弊可以量化吗？关于核能源及核废料的辩论是可量化的吗？将结论建立在平均值和其他统计分析上的做法危险吗？如果新闻通讯社在报道调查结果时不使用问题的准确措辞，这道德吗？能够量化一个人的生命价值吗？一家公司在明知附加投资可以减少产品使用的危险性的情况下，仍然停止产品的改进投资，这是可以接受的吗？

5. 收集和散发数据的权利是否应该根据数据形式的不同而有所差别？也就是说，收集和散发照片、音频或者视频的权利是否应该与收集和散发文本的一样？

6. 无论是有意还是无意的，记者的报道中通常都会反映出记者本人的偏见。一般只要改几个词语，一个故事就可能被赋予正面或负面的含义。（比较“大多数受访者都反对公民投票。”与“受访者中有相当一部分支持公民投票。”）修改一个故事（删掉某些观点或者仔细选词）和修改一张照片有区别吗？

7. 假设数据压缩系统的使用会导致一些微小但重要的信息项的丢失。这会产生什么样的责任问题？应该如何解决？

课外阅读

Drew, M. and Z. Li. *Fundamentals of Multimedia*, 2nd ed. Cham, Switzerland: Springer, 2014.

Halsall, F. *Multimedia Communications*. Boston, MA: Addison-Wesley, 2001.

Hamacher, V. C., Z. G. Vranesic, S. G. Zaky, and N. Manjikian. *Computer Organization and Embedded Systems*, 6th ed. New York, NY: McGraw-Hill, 2011.

Knuth, D. E. *The Art of Computer Programming, Vol. 2*, 3rd ed. Boston, MA: Addison-Wesley, 1998.

Long, B. *Complete Digital Photography*, 8th ed. Boston, MA: Cengage Learning, 2015.

Miano, J. *Compressed Image File Formats*. New York: ACM Press, 1999.

Petzold, C. *CODE: The Hidden Language of Computer Hardware and Software*. Redman, WA: Microsoft Press, 2000.

Saloman, D., and G. Motta. *Handbook of Data Compression*, 5th ed. London, England: Springer, 2010.

Sayood, K. *Introduction to Data Compression*, 5th ed. Cambridge, MA: Morgan Kaufmann, 2017.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第2章 数据操控

本章学习计算机如何操控数据以及如何与外围设备（如打印机和键盘）通信。为此，我们将研究计算机体系结构的基础知识，学习计算机是如何利用称为机器语言指令的编码指令来进行编程工作的。

本章内容

2.1 计算机体系结构

2.2 机器语言

2.3 程序执行

*2.4 算术/逻辑指令

*2.5 与其他设备通信

*2.6 数据操控编程

*2.7 其他体系结构

持久理解和学习成果

建立在二进制序列基础上的各种抽象可用于表示所有数字数据。

AP[®]CSP

学习成果 描述用于表示数据的各种抽象。

学习成果 解释如何用二进制序列来表示数字数据。

多层抽象用于编写程序或创建其他计算工件。AP[®]CSP

学习成果 确定编写程序时使用的多层抽象。

人们编写程序来执行算法。AP[®]CSP

学习成果 解释程序是如何实现算法的。

适当的抽象使编程更容易。AP[®]CSP

学习成果 使用抽象管理程序复杂性。

在第1章中，我们学习了有关计算机数据存储的主题，本章将介绍计算机如何操控这些数据。这里所说的操控包括将数据从一个位置移到另一个位置，以及执行各种操作，如算术计算、文本编辑和图像处理。首先我们要了解解除数据存储系统之外的计算机体系结构。

2.1 计算机体系结构

计算机中控制数据操控的电路称为**中央处理器**（central processing unit），即**CPU**，通常简称为**处理器**。在20世纪中期的机器中，CPU属于大部件，由若干机架中的电子电路组成，这也反映了该部件的重要性。不过，科技进步已经极大地缩小了这些设备。现在，台式计算机和笔记本电脑中的CPU都被封装成了小巧的正方形薄片，可以放在人的手掌中，它们的连接引脚插在机器主电路板〔称为**主板**（motherboard）〕的插座上。在智能手机、平板电脑和其他移动计算设备中，CPU大约是邮票的一半大小。因为这些处理器尺寸小，所以被称作**微处理器**（microprocessor）。

2.1.1 CPU基础知识

CPU由3部分构成（见图2-1）：**算术/逻辑单元**（arithmetic/logic unit），它包含在数据上执行运算（如加法和减法）的电路；**控制单元**（control unit），它包含协调机器活动的电路；**寄存器单元**（register unit），它包含称为**寄存器**（register）的数据存储单元（与主存储器存储单元相似），用于CPU内部的信息的临时存储。

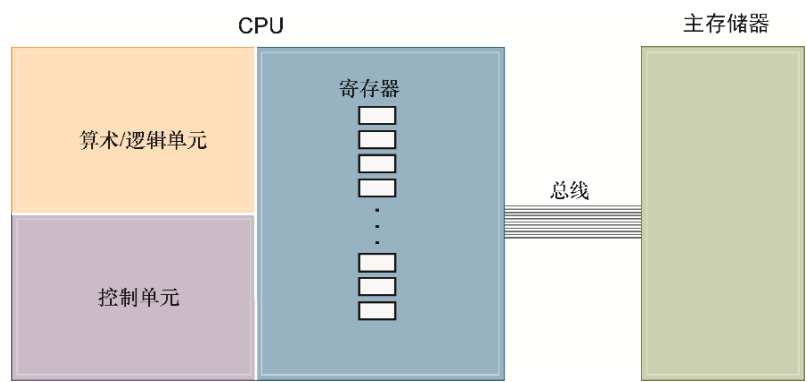


图2-1 通过总线连接的CPU和主存储器

寄存器单元中的一些寄存器被看作**通用寄存器**（general-purpose register），其他寄存器则被看作**专用寄存器**（special-purpose register），我们将在2.3节讨论一些专用寄存器，这里我们只关注通用寄存器。

通用寄存器用于临时存储CPU正在操控的数据。这些寄存器存储算术/逻辑单元电路的输入值以及该单元产生的结果。为了操作存储在主存储器中的数据，控制单元要把存储器里的数据传入通用寄存器，通知算术/逻辑单元哪些寄存器保存了这一数据，激活算术/逻辑单元中的有关电路，并告知算术/逻辑单元哪个寄存器应该接收结果。

为了传输位模式，机器CPU和主存储器通过一组称为**总线**（bus）（见图2-1）的线路进行连接。利用总线，CPU通过提供相关存储单元的地址以及电

信号（告知存储器电路，它应该获取指定单元中的数据），从主存储器提取（读取）数据。同理，CPU可以向主存储器中放入（写入）数据，方法是提供目的单元地址和一起写入的数据以及适当的电信号（告知主存储器，将要存储发送给它的数据）。

基于这个设计，将存储在主存储器中的两个值相加的任务不仅仅涉及执行加法运算。数据必须先从主存储器传输到CPU内的寄存器中，值相加后，先把结果存放在寄存器中，然后再把结果存储到主存储器存储单元中。整个过程被总结成图2-2所示的5个步骤。

- 步骤1 从存储器中取出一个要加的值放入一个寄存器中。
- 步骤2 从存储器中取出另一个要加的值放入另一个寄存器中。
- 步骤3 激活加法电路，以步骤1和步骤2所用的寄存器作为输入，用另一个寄存器存放相加的结果。
- 步骤4 将结果存入存储器。
- 步骤5 停止。

图2-2 主存储器中的值相加

2.1.2 存储程序概念

早期计算机不是很灵活，每个设备执行的步骤都内置于控制单元中，作为计算机的一部分。为了获得更大的灵活性，一些早期电子计算机的设计使CPU可以方便地重新布线。这一灵活性是通过插拔装置体现，这个装置类似于老式的电话交换台上把跳线的端子插到接线孔中。

认识到一个程序可以像数据一样进行编码并存储在主存储器中，这是一个突破性进展（但是将其归功于约翰·冯·诺依曼显然是不正确的）。如果将控制单元设计成从存储器中提取程序、译码指令并执行它们，那么机器要遵照执行的程序就可以被修改，这只需改变计算机存储器的内容而不必对CPU进行重新布线。

基本知识点

- 位序列可表示指令或数据。

高速缓冲存储器

将计算机存储设备与其对应功能进行比较是很有启发性的。寄存器用于存储可立即进行运算的数据，主存储器用于存储即将使用的数据，海量存储器用于存储最近也许不会使用的数据。许多计算机设计都增加了一个附加的存储器层次，称为高速缓冲存储器。高速缓冲存储器（cache memory）是位于CPU内部的高速存储器的一部分（也许有几百KB）。在这个特殊的存储区域中，机器试图保存主存储器中当前最重要的那部分内容的一个副本。这样，通常要在寄存器与主存储器之间进行的数据传输将变成寄存器与高速缓冲存储器之间的数据传输。因此，对高速缓冲存储器的任何更改都会在恰当的时间被一起传输给主存储器。于是，CPU可以更快地执行它的机器周期，因为它不会被与主存储器的通信所延迟。

将计算机程序存储在主存储器中的思想称为**存储程序概念**（stored-program concept），它已经成为今天使用的标准方法——实际上它的标准性很显然。最初的困难源于人们将程序和数据视为不同的实体：数据存储在主存储器中，而程序是CPU的一部分。于是就成了“只见树木，不见森林”的一个极好实例。人们很容易被老一套所束缚，如果今天仍不了解这一点，那么计算机科学的发展也许还会裹足不前。其实，科学中令人兴奋的是，新的思想不断为新的理论和新的应用开启大门。

谁发明了什么

将某项发明的荣誉授予个人总是备受争议。人们将白炽灯的发明归功于托马斯·爱迪生（Thomas Edison），但是其他研究员也曾研制了类似的灯泡，

从某种意义上说，爱迪生只是比较幸运地获得了专利。人们认为是莱特（Wright）兄弟发明的飞机，但他们曾与同时代的其他人竞争并受益于其他人的研究，在某种程度上，他们又被列奥纳多·达·芬奇（Leonardo da Vinci）抢先了，他早在15世纪就有了玩玩飞行机器的想法。甚至达·芬奇的设计看起来也是假借前人的思想。当然，对于这些发明，被认定的发明人还是有权拥有被授予的荣誉的。但对于其他一些情况，历史上的有些荣誉授予似乎是不恰当的，如存储程序的概念。毫无疑问，约翰·冯·诺依曼（John von Neumann）是一位卓越的科学家，理应为自己的许多贡献获得荣誉。但是，历史选择授予他荣誉的贡献是存储程序概念，但这一思想很显然是由宾夕法尼亚大学莫尔工程学院的埃克特（J. P. Eckert）领导的研究人员提出的。约翰·冯·诺依曼不过是第一个在发表的著作中转述这一思想的人，因此计算机界选择他作为发明人。

2.1 节问题与练习

1. 将计算机中一个存储单元的内容移到另一个存储单元，你认为需要哪些事件序列？
2. 要想将一个值写入存储单元中，CPU必须给主存储器电路提供什么信息？
3. 海量存储器、主存储器以及通用寄存器都是存储系统。它们的用途有什么不同？

2.2 机器语言

为了应用存储程序概念，CPU被设计成可以识别位模式编码的指令。这些指令以及编码系统统称为**机器语言**（machine language）。使用此语言表达的指令称为机器级指令，更多人称其为**机器指令**（machine instruction）。

2.2.1 指令系统

典型CPU必须能够译码和执行的机器指令列表非常短。实际上，一旦机器能够完成某些基本而精选的任务，那么添加再多的特性也不会增加该机器理论上的能力。换句话说，超过某个特定点之后，附加的特性也许能够增加一些好处，如便利性，但是机器的基本能力不会有任何改变。

在设计机器时，根据该事实被利用的程度的不同，产生了两种CPU体系结构哲学。一种是CPU只需要执行最小的机器指令集，在这种思想的作用下，产生了**精简指令集计算机**（reduced instruction set computer，RISC）。RISC体系结构的支持者认为，这种计算机效率高、速度快，制造成本低。另一种是另外一些人则认为CPU应该能够执行大量复杂的指令，尽管其中许多在技术上是多余的，在这种思想的作用下，产生了**复杂指令集计算机**（complex instruction set computer，CISC）。CISC的支持者认为，CPU越复杂越容易应对现代软件日益增加的复杂性。通过CISC，程序可以利用一个强大的丰富指令集，其中很多指令都需要RISC设计中的一个多指令序列。

20世纪90年代至21世纪，市场上在售的CISC处理器和RISC处理器都在积极地争夺桌面计算领域的主导地位。用于个人计算机的英特尔处理器是CISC体系结构的代表，而PowerPC处理器（由苹果公司、IBM公司和摩托罗拉公司联合开发）是RISC体系结构的代表，被用于苹果Macintosh中。随着时间的推移，CISC的制造成本大大降低了，因此几乎现在所有的台式计算机和笔记本电脑中都能找到英特尔的处理器（或AMD公司的处理器）（现在连苹果公司生产的计算机都开始使用英特尔的产品了）。

虽然CISC在台式计算机领域站稳了脚跟，但其电功率非常大。与此相反，ARM（Advanced RISC Machine）公司专门针对低电耗设计了一种RISC

体系结构。（ Advanced RISC Machine的前身是Acorn Computers，现在是ARMHoldings。）因此，在游戏控制器、数字电视、导航系统、汽车部件、移动电话、智能手机和其他消费性电子产品中，很容易找到由高通（ Qualcomm ）和德州仪器（ Texas Instruments ）等多个供应商制造的基于ARM的处理器。

不管是选择RISC还是选择CISC，机器指令都可以分为3类：（1）数据传输类；（2）算术/逻辑类；（3）控制类。

2.2.2 数据传输类

数据传输类指令由将数据从一个位置移到另一个位置的请求指令组成，图2-2中的步骤1、步骤2和步骤4都属于这一类。我们应该注意到，使用诸如传输（ transfer ）或移动（ move ）之类的术语来标识这类指令实际上是用词不当。因为传输的数据很少被从原始位置擦除。执行传输指令的过程更像是复制数据而不是移动数据，因此，使用诸如复制（ copy ）或克隆（ clone ）之类的术语能更好地描述这类指令的活动。

关于术语，我们应注意，提到CPU与主存储器之间的数据的传输时，有专门的术语。用存储单元的内容填充通用寄存器的请求通常称为加载（ LOAD ）指令；相反，将寄存器中内容传输给存储单元的请求称为存储（ STORE ）指令。在图2-2中，步骤1和步骤2是LOAD指令，步骤4是STORE指令。

在数据传输类中，有一组与CPU-主存储器环境之外的设备（打印机、键盘、显示屏、磁盘驱动器等）进行通信的重要指令。这些指令负责处理机器的输入/输出（ I/O ）活动，因此被称为I/O指令（ I/O instruction ），有时因为

其特别会被单独归为一类。另外，2.5节将描述如何利用一些相同的指令来处理这些I/O活动，这些指令是请求在CPU及主存储器之间传输数据的。因此，我们应考虑将I/O指令归入数据传输类。

2.2.3 算术/逻辑类

算术/逻辑类指令由告诉控制单元请求在算术/逻辑单元内实现一个活动的指令组成。图2-2中的步骤3属于这一类。顾名思义，算术/逻辑单元还能够执行基本算术运算之外的运算。在这些附加的运算中，有第1章中介绍的布尔运算AND、OR和XOR，本章后面会进一步讲解。

大多数算术/逻辑单元中都还有另外一组运算能让寄存器中的内容在寄存器中左右移动。这些运算称为移位（SHIFT）运算或循环移位（ROTATE）运算，前者丢弃一端“移出的位”，而后者将它们放到另一端留出的空位上。

2.2.4 控制类

控制类指令由指导程序执行而非数据操作的指令组成。图2-2中的步骤5属于此类，但它是一个很初级的例子。这一类指令包括机器指令系统中许多比较有趣的指令，如跳转（JUMP）或分支（BRANCH）系列指令，用于指示CPU执行列表中下一条指令以外的其他指令。这些JUMP指令分两种：**无条件跳转**（unconditional jump）和**条件跳转**（conditional jump）。前者的一个例子是指令“跳转到步骤5”；后者的一个例子是指令“如果所得值为0，就跳转到步骤5”。两者的区别是，只有满足某个条件时，条件跳转才会引起“地点改变”。举例说明，图2-3中的指令序列是表示两个值相除的算法，其中步骤3是条件跳转，用以防止除数为0。

步骤1 把存储器中的一个值加载到一个寄存器中。
 步骤2 把存储器中的另一个值加载到另一个寄存器。
 步骤3 如果第二个值为0，那么转移到步骤6。
 步骤4 用第一个寄存器中的值除以第二个寄存器中的值，结果留在第三个寄存器中。
 步骤5 把第三个寄存器的值存储到存储器中。
 步骤6 停止。

图2-3 对存储器中的值做除法运算

2.2.5 Vole：一种演示用的机器语言

现在让我们来看一下典型计算机的指令是如何编码的。我们把将要讨论的机器称作Vole，附录C中描述了Vole，图2-4展示了它的体系结构。这个假想的Vole处理器有16个通用寄存器和256个主存储器存储单元，每个存储单元容量为8位。为了便于引用，我们将寄存器标号为值0～15，把存储单元的地址编为值0～255。为方便起见，我们将这些标号及地址看作以二进制表示的值，并使用十六进制记数法压缩结果位模式。于是，寄存器标号为0x0～0xF，存储单元的地址为0x00～0xFF。（回顾第1章，我们用“0x”作为前缀来表示十六进制值。）

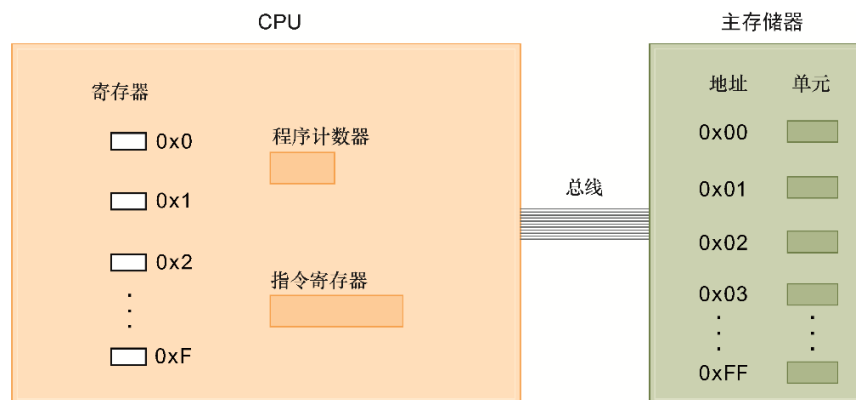


图2-4 Vole的体系结构

机器指令的编码形式包括两部分：**操作码**（operation code，简称为op-code）字段和**操作数**（operand）字段。操作码字段中的位模式指明该指令要

求的是什么基本运算，如STORE、SHIFT、XOR和JUMP。操作数字段中的位模式提供操作码指定运算的更详细信息。以STORE操作为例，其操作数字段中的信息指示哪个寄存器包含将被存储的数据，哪个存储单元用于接收该数据。

我们的Vole机器（见附录C）的整个机器语言只包含12条基本指令。每条指令都用16位编码，由4个十六进制数字表示（见图2-5）。每条指令的操作码由前4位组成，等价于第一个十六进制数字。注意（见附录C），这些操作码是用十六进制数字0x1 ~ 0xC表示的。特别是，附录C中的表说明，以0x3起始的指令表示STORE指令，以0xA起始的指令表示ROTATE指令。

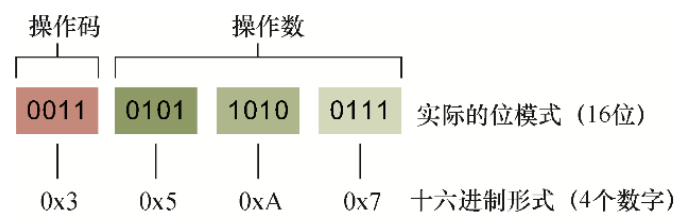


图2-5 Vole指令的组成

变长指令

为了简化文中的解释，本章示例所用的机器语言（在附录C中有描述）对所有指令都使用了固定的大小（2字节）。因此，为取得一个指令，CPU总是需要检索2个连续存储单元的内容，然后给其程序计数器加2。这种一致性简化了取指令的任务，是RISC机器的特征。不过，在CISC机器的机器语言中，指令长度是可变的。例如，现在的英特尔处理器的指令长度有长有短，短的只有1字节，长的有多个字节，具体取决于该指令的确切应用。使用这种机器语言的CPU根据指令操作码来确定所引入的指令的长度。也就是说，CPU首先取指令的操作码，然后基于收到的位模式得知，要得到余下的指令还需要从存储器中取多少字节。

Vole上的每条指令的操作数字段由3个十六进制数字（12位）组成，在每种情况下对操作码给定的通用指令做了进一步澄清（HALT指令除外，因为它不需要进一步的规定）。例如，如果一条指令的第一个十六进制数字为0x3（存储寄存器中内容的操作码），那么该指令的下一个十六进制数字会指出哪个寄存器中的内容需要存储，最后的两个十六进制数字会指出由哪个存储单元接收该数据（见图2-6）。因此，指令0x35A7翻译过来就是“将寄存器5中的位模式存储（STORE）到地址为0xA7的存储单元中”。（注意，十六进制记数法的使用是如何简化我们的讨论的。事实上，指令0x35A7是指位模式0011010110100111。）

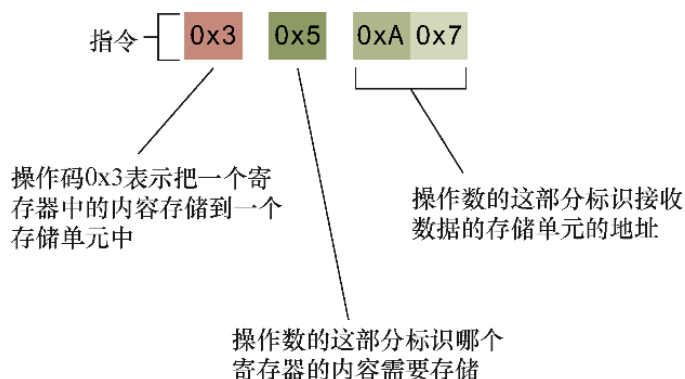


图2-6 指令0x35A7的译码

基本知识点

- 因为十六进制表示使用的数字比二进制少，所以十六进制（基数16）用于表示数字数据。

通过指令0x35A7这个例子，还能清楚地解释为什么主存储器容量是以2的幂为度量单位的。因为该指令中保留了8位用于指定该指令所用的存储单元，所以能够准确地引用 2^8 个不同的存储单元。因此，我们理所当然要用这么多存储单元（地址从0~255）构建1个主存储器。如果主存储器有更多的存

储单元，我们就写不出能区分它们的指令了；如果主存储器的存储单元比较少，我们写出的指令可能会引用不存在的存储单元。

为了说明操作数字段是如何澄清操作码给定的通用指令的，下面再举一个例子。我们来考虑一个操作码为0x7的指令，它请求将两个寄存器的内容进行OR（或）运算。（在2.4节中，我们会看到两个寄存器的OR运算意味着什么。现在，我们感兴趣的只是指令是如何编码的。）在这种情况下，下一个十六进制数字指示的是运算结果应该存储在哪一个寄存器中，而操作数的最后两个十六进制数字指示的是要对哪两个寄存器进行OR运算。因此，指令0x70C5翻译过来就是“对寄存器C和寄存器5的内容进行OR运算，并将结果存入寄存器0中”。

我们所讲的这两条机器的LOAD指令存在细微的差别。这里，操作码0x1标识的指令是将某一存储单元的内容载入寄存器，操作码0x2标识的指令是把一个特定的值加载到寄存器。它们的差别在于，第一种类型的指令的操作数字段包含的是1个地址，而第二种类型的指令的操作数字段包含的是一个要载入的实际位模式。

基本知识点

- 二进制序列的解释取决于如何使用它。

注意，Vole有两条ADD指令：一条用于二进制补码记数法表示的值相加；另一条用于浮点记数法表示的值相加。把它们区分开来的原因是，这两种加法在算术/逻辑单元内部有不同的实现步骤。寄存器和主存储器存储单元没有区分它们存储的数据的类型，二进制序列如何解释完全取决于应用于它的操作。我们用图2-7结束本节，图2-7中显示的是图2-2中指令的编码版本。我们

已经假定，两个相加的值以二进制补码记数法存储在存储地址0x6C和0x6D，相加得到的和存放在地址为0x6E的存储单元里。

基本知识点

- 位序列可以表示不同上下文中不同类型的数据。

编码的指令	翻 译
0x156C	把地址为0x6C的存储单元里的位模式加载到寄存器0x5
0x166D	把地址为0x6D的存储单元里的位模式加载到寄存器0x6
0x5056	把寄存器0x5和0x6的内容按二进制补码表示相加，并把结果存入寄存器0x0
0x306E	把寄存器0x0的内容存储到地址为0x6E的存储单元中
0xC000	停止

图2-7 图2-2中指令的编码版本

2.2节问题与练习

1. 为什么使用术语移动（move）来描述“将数据从机器的一个位置移动到另一个位置的操作”有可能不恰当？
2. 在本书中JUMP指令是通过在指令中给出目的地名称（或步骤号）的方式来明确标识目的地（如“跳转到步骤6”）。这种技术的缺点是，如果一个指令名称（或步骤号）后来改变了，那么必须找到所有跳转到该指令的JUMP指令，并改变这些JUMP指令中的目的地名称。请另外设计一种表达JUMP指令的方式，使得不需要明确给出目的地名称。
3. 指令“如果0等于0，就跳转到步骤7”是条件跳转还是无条件跳转？请解释你的答案。
4. 用实际的位模式编写图2-7中的示例程序。
5. 下面是用Vole机器语言编写的指令。请用自然语言解释这些指令。
a. 0x368A b. 0xBADE c. 0x803C d. 0x40F4
6. 在Vole里，指令0x15AB和0x25AB有什么区别？

7. 下面是用自然语言描述的一些指令。请把它们翻译成Vole机器语言。

a. 将十六进制值0x56加载（LOAD）到编号为0x3的寄存器中。

b. 将编号为0x5的寄存器向右循环移位（ROTATE）3位。

c. 将寄存器0xA中的内容与寄存器0x5中的内容做加（ADD）操作，并将其结果存入寄存器0x0中。

2.3 程序执行

计算机总是按照需要把存储器里的指令复制到CPU中来执行存储器中的程序。每个指令一旦到达CPU，就会被译码并执行。从存储器中取指令的顺序与这些指令存储在存储器中的顺序是相对应的，除非JUMP指令更改了这个顺序。

为了理解整个执行过程是如何进行的，我们有必要仔细了解一下CPU内部的两个专用寄存器：**指令寄存器**（instruction register）和**程序计数器**（program counter）（再见图2-4）。指令寄存器用于存储正在执行的指令。程序计数器中包含的是要执行的下一条指令的地址，因此它用于以机器方式跟踪程序执行到了什么地方。

CPU通过不断重复执行一个算法来完成它的工作，该算法引导它完成一个称为**机器周期**（machine cycle）的3步过程。该机器周期中的3步是取指、译码和执行（见图2-8）。在取指步骤，CPU请求主存储器给它提供程序计数器指定的地址中存储的指令。因为我们机器的每一条指令的长度为2字节，所以取指过程需要从主存储器中读取2个存储单元的内容。CPU将从存储器中读取的指令存放在它的指令寄存器中，然后将程序计数器的值加2，使得程序计

数器包含存储器中存储的下一条指令的地址。这时，程序计数器为下一次取指做好了准备。

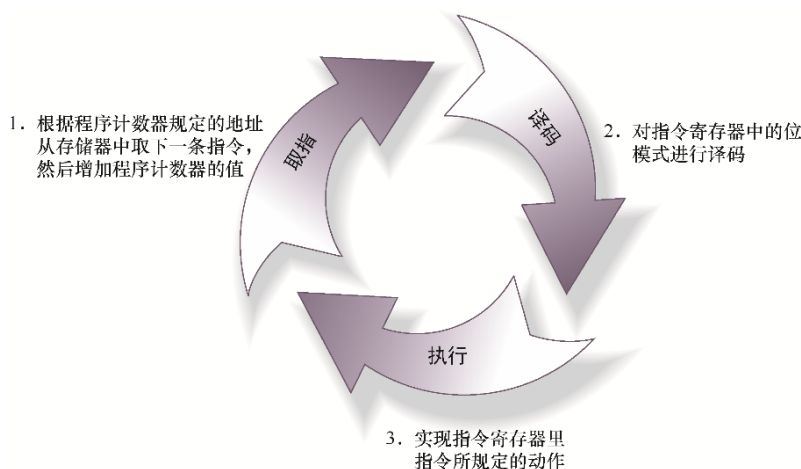


图2-8 机器周期

比较计算机能力

购买个人计算机时，人们通常用时钟速度来比较计算机。计算机的**时钟**（clock）是一个称为振荡器的电路，能生成用于协调机器活动的脉冲——这个振荡器电路生成脉冲的速度越快，机器执行其机器周期的速度就越快。时钟速度以赫兹（hertz，简写为Hz）为单位，1Hz相当于每秒1个周期（或脉冲）。台式计算机的典型时钟周期是几百MHz（较老的型号）到几GHz。（MHz是megahertz的缩写，1 MHz=10⁶ Hz；GHz是gigahertz的简写，1 GHz=1000 MHz。）

遗憾的是，不同的CPU设计在一个时钟周期里完成的工作量是不同的。因此，在比较具有不同CPU的机器时，单单比较时钟速度没有太大的意义。如果你在比较的两台机器，一台基于英特尔处理器，一台基于ARM处理器，那么通过**基准测试**（benchmark）来比较它们的性能才更有意义。基准测试是指，让不同的机器执行同一程序（称为基准）来比较其性能的过程。通过选

择代表不同类型应用的基准，就能够得到对各个细分市场有意义的比较结果。

由于指令现在已经存入了指令寄存器，CPU对该指令译码，其中包括根据该指令的操作码将操作数字段分解为适当的部分。

然后，CPU激活相应电路以执行指令，完成所请求的任务。例如，如果该指令是从存储器中加载的，那么CPU将给主存储器发送相应信号，等待主存储器发送数据，再将数据存入要求的寄存器；如果该指令是算术运算，那么CPU将用正确的寄存器作为输入，激活算术/逻辑单元中相应的电路，计算结果并将结果存入相应的寄存器。

一旦指令寄存器中的指令执行完毕，CPU又将从取指步骤开始进入下一个机器周期。观察可知，因为程序计数器在前一个取指步骤的最后已经增加了值，所以它再次为CPU提供了正确的指令地址。

JUMP指令的执行比较特殊。例如，指令0xB258（见图2-9）的含义是“如果寄存器0x2的内容与寄存器0x0的内容相同，就跳转到地址为0x58的指令”。在这种情况下，该机器周期的执行步骤首先比较寄存器0x2和0x0。如果它们包括不同的位模式，那么执行步骤结束，开始下一个机器周期。不过，如果这两个寄存器的内容相同，那么机器要在执行步骤中将值0x58存入它的程序计数器里。然后，在这种情况下，下一个取指步骤发现程序计数器中的值为0x58，于是该地址的指令将成为下一条要读取和执行的指令。

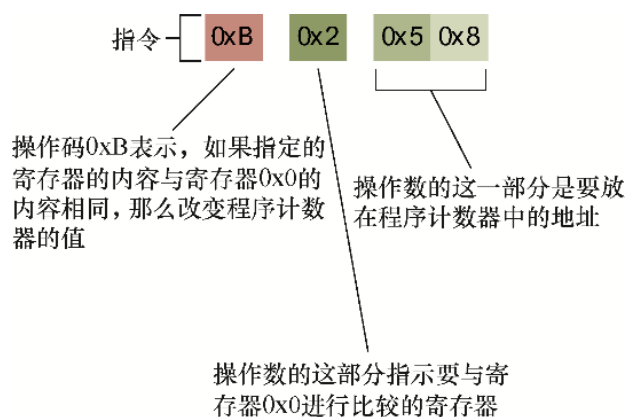


图2-9 指令0xB258的译码

注意，如果该指令是0xB058，那么该程序计数器是否应该更改取决于这个寄存器0x0的内容和那个寄存器0x0的内容是否相同。不过，这两个寄存器是同一个寄存器，因此内容必然是相同的。于是，无论寄存器0x0的内容是什么，任何形式为0xB0XY的指令都将使程序跳转至存储器位置XY执行。

2.3.1 程序执行的一个例子

让我们从机器周期的角度看图2-7的程序是如何执行的。该程序从主存储器中取出两个值，计算它们的和，并将结果存储在一个主存储器存储单元里。首先，我们需要将该程序存放在存储器的某个地方。对于这个例子，假设该程序存放在从地址0xA0开始的连续地址中。在以这种方式存储完程序之后，要让机器执行这个程序只需把该程序的第一条指令的地址（0xA0）存放在程序计数器中并开启机器（见图2-10）。

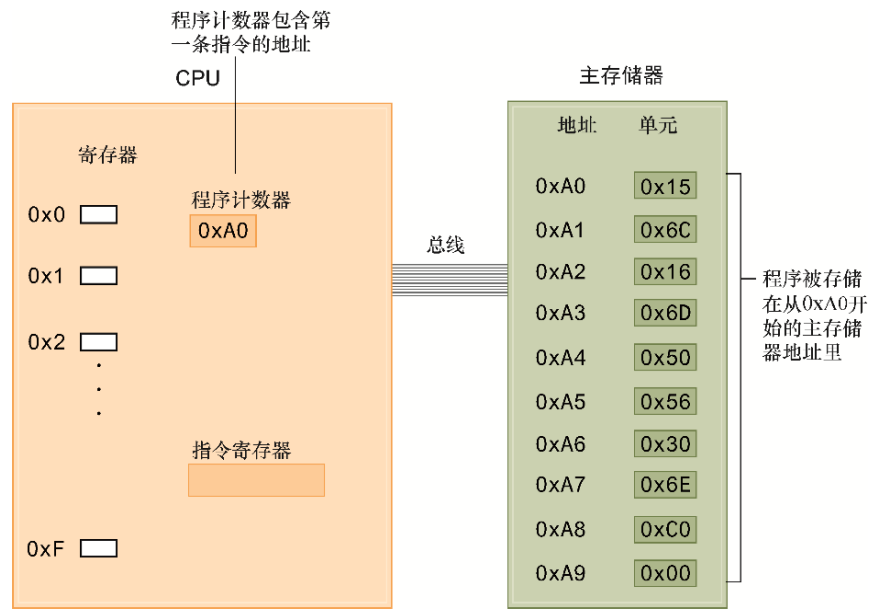


图2-10 图2-7中的程序被存储在主存储器中准备执行

CPU开始机器周期的取指步骤，该步骤从主存储器中取出存储在地址 0xA0的指令，并将该指令（0x156C）放入指令寄存器里（见图2-11a）。注意，在我们的机器里，指令的长度为16位（2字节）。因此，要取的整个指令占用了地址为0xA0和0xA1的两个存储单元。CPU的设计考虑到了这点，它取指时同时读两个存储单元的内容，并将获得的位模式存放在长度为16位的指令寄存器中。接着，CPU给程序计数器加2，使得该寄存器包含下一条指令的地址（见图2-11b）。在第一个机器周期的取指步骤结束时，程序计数器和指令寄存器包含下列数据：

程序计数器：0xA2
指令寄存器：0x156C

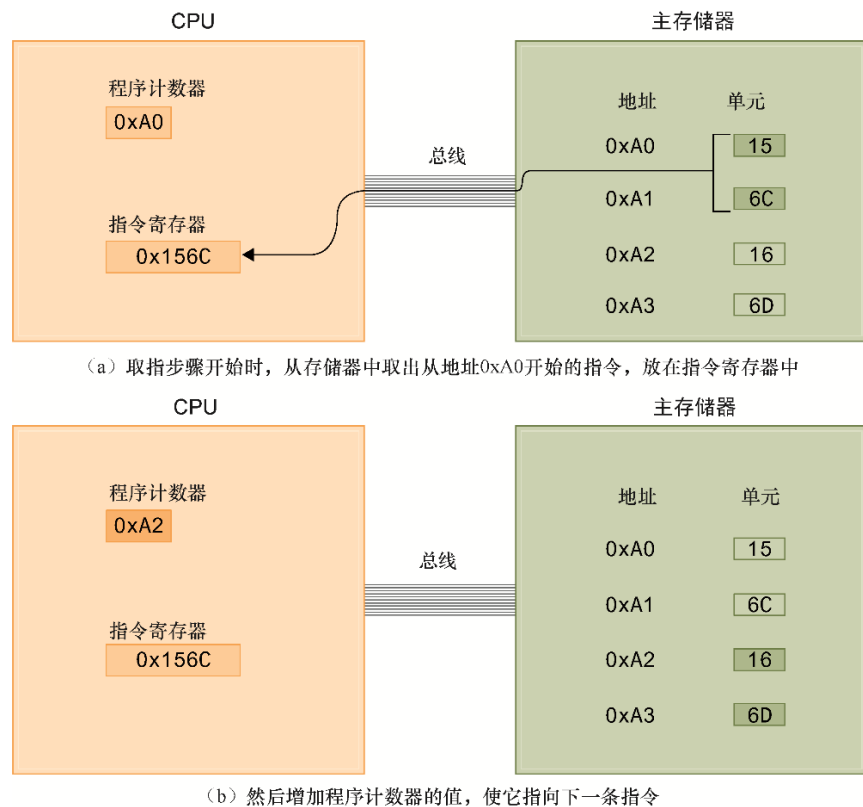


图2-11 执行机器周期的取指步骤

然后，CPU分析其指令寄存器中的指令，并得出结论：它要把地址为0x6C的存储单元的内容加载到寄存器0x5中。该加载工作是在机器周期的执行步骤完成的，接着CPU开始下一个机器周期。

这个周期首先要从以地址0xA2开始的2个存储单元中取指令0x166D。CPU将指令0x166D存入指令寄存器，并将程序计数器增加到0xA4。因此，此时程序计数器和指令寄存器中的值变为：

程序计数器：0xA4 指令寄存器：0x166D

现在，CPU对指令0x166D进行译码，确定它要将地址为0x6D的存储单元的内容加载到寄存器0x6，然后执行该指令。这时，寄存器0x6才真正加载了数据。

因为该程序计数器现在包含的值是0xA4，所以CPU从这个地址开始取下一条指令。于是，指令0x5056被放入指令寄存器，程序计数器被增加到0xA6。现在，CPU对其指令寄存器的内容进行译码，然后通过激活二进制补码加法电路、用寄存器0x5和寄存器0x6作为输入，执行加法运算。

在该执行步骤中，算术/逻辑单元执行所请求的加法运算，将结果存入寄存器0x0（如控制单元所要求的那样），然后向控制单元报告它已经完成了任务。之后CPU开始另一个机器周期，再一次借助程序计数器，从以存储器位置0xA6开始的2个存储单元中取下一条指令（0x306E），然后将程序计数器增加到0xA8，接着译码并执行该指令。此时，和被放在了存储器位置0x6E中。

下一条指令从存储器位置0xA8开始提取，同时将程序计数器增加到0xAA。指令寄存器（0xC000）的内容现在被译码为停止指令。因此，机器在该机器周期的执行步骤停止，程序完成。

综上，我们看到，如果我们需要遵照详细的指令列表执行程序，那么存储在存储器里的程序的执行过程就如同你我都可以做的那样。我们可以通过标记指令来确定执行到的位置，而CPU则利用程序计数器来确定执行到的位置。在确定下一步要执行哪条指令后，我们需要读该指令并分析它的含义，然后实现所请求的任务并返回到指令的列表，为下一条指令做准备。这与CPU执行其指令寄存器中的指令然后从另一个取指步骤开始继续进行的方式是一样的。

2.3.2 程序与数据

许多程序可以同时存储在一个计算机的主存储器里，只要它们的地址不同。机器开启时运行哪个程序可以通过适当地设置程序计数器来确定。

然而，我们必须记住：数据也存储在主存储器中，也是用0和1编码的，所以机器自己无法知道什么是数据，什么是程序。如果程序计数器被赋予了数据的地址而非所希望的程序的地址，那么在没有更好选择的情况下，该CPU会像取指令一样提取此数据的位模式并执行它们。最终的结果取决于所涉及的数据。

然而，我们不应该就此得出结论，以为将程序和数据以相同的形式存入机器的存储器是不好的。事实上，这已经被证明是一个有用的特性，因为它使得某个程序可以操控其他程序（甚至是自己），就像它可以操控数据一样。例如，设想有这样的一个程序，它可以根据其与环境交互自我修正，从而表现出学习能力；或者设想有这样一个程序，它可以编写和执行其他程序，以解决它所遇到的问题。

2.3节问题与练习

1．假设地址从0x00到0x05的Vole存储单元中包含下列位模式。

地 址	内 容
0x0	0x14
0x01	0x02
0x02	0x34

地 址	内 容
0x03	0x17
0x04	0xC0
0x05	0x00

如果我们启动机器时其程序计数器被设为0x00，那么该机器停机时地址为0x17的存储单元中的位模式是什么？

2．假设地址从0xB0到0xB8的Vole存储单元中包含下列位模式。

地 址	内 容
0xB0	0x13
0xB1	0xB8
0xB2	0xA3
0xB3	0x02
0xB4	0x33
0xB5	0xB8
0xB6	0xC0

地 址	内 容
0xB7	0x00
0xB8	0x0F

a．如果程序计数器从0xB0开始，那么第一条指令执行之后，编号为0x3的寄存器中的位模式是什么？

b．在执行停止指令时，存储单元0xB8里的位模式是什么？

3．假设地址从0xA4到0xB1的Vole存储单元中包含下列位模式。

地 址	内 容
0xA4	0x20
0xA5	0x00
0xA6	0x21
0xA7	0x03
0xA8	0x22
0xA9	0x01

地 址	内 容
0xAA	0xB1
0xAB	0xB0
0xAC	0x50
0xAD	0x02
0xAE	0xB0
0xAF	0xAA
0xB0	0xC0
0xB1	0x00

假设Vole启动时其程序计数器包含的是0xA4，请回答下列问题。

- 地址为0xAA的指令第一次执行时，寄存器0x0中是什么？
- 地址为0xAA的指令第二次执行时，寄存器0x0中是什么？
- Vole停止之前，存放在地址0xAA里的指令执行了多少次？
- 假设地址从0xF0 ~ 0xF9的Vole存储单元中包含下列位模式。

地 址	内 容
0xF0	0x20

地 址	内 容
0xF1	0xC0
0xF2	0x30
0xF3	0xF8
0xF4	0x20
0xF5	0x00
0xF6	0x30
0xF7	0xF9
0xF8	0xFF
0xF9	0xFF

如果我们启动Vole时其程序计数器包含的是0xF0，那么当机器到达地址为0xF8的指令时，机器做什么？

*2.4 算术/逻辑指令

如前所述，算术/逻辑指令组由请求算术、逻辑和移位运算的指令组成。在本节中，我们将详细介绍这些运算。

2.4.1 逻辑运算

在第1章中，我们介绍了逻辑运算AND（与）、OR（或）和XOR（异或，一般读作“ex-or”），它们都是组合两个输入的二进制位，得到一个输出的二进制位。这些运算可以扩展成为这样的**按位运算**（bitwise operation）：组合两个二进制位串，在各个列上应用基本运算，产生一个输出串。例如，位模式10011010与11001001进行AND运算所得的结果如下：

$$\begin{array}{r} 10011010 \\ \text{AND } 11001001 \\ \hline 10001000 \end{array}$$

这里，我们只是将每一列的两个二进制位的AND运算结果写在了那一列的底部。同理，对这些位模式进行OR运算及XOR运算时得到的结果是：

$$\begin{array}{r} 10011010 \\ \text{OR } 11001001 \\ \hline 11011011 \end{array} \qquad \begin{array}{r} 10011010 \\ \text{XOR } 11001001 \\ \hline 01010011 \end{array}$$

AND运算的一个主要用途是将位模式的一部分设为0，而不干扰另外一部分。这一用途在实践中有很多应用，如过滤用RGB格式表示的数字图像的某些颜色，如第1章中描述的那样。再看一个例子，考虑如果字节00001111是AND运算的第一个操作数，那么会产生什么结果？即使不知道第二个操作数的内容，我们仍然能够得出这样的结论，即结果的最高有效4位均为0。而且，结果的最低有效4位将与第二个操作数的最低有效4位相同，如下例所示：

```
      00001111
AND 10101010
      00001010
```

AND运算的这个应用是一个称为**屏蔽**（masking）的过程的例子。这里，一个称为**掩码**（mask）的操作数决定另一个操作数的哪个部分会影响结果。在这个AND运算中，屏蔽得出一个结果，其中一部分是一个操作数的复制品，没有复制的部分置0。在这个上下文中，AND运算的一个简单应用就是屏蔽图像像素中与红色部分相关联的所有位，只留下蓝色和绿色部分。这种转换在图像操作软件中是经常使用的。

除了操作图像，AND运算在操作其他类型的**位图**（bit map）方面也很有用，只要所用位串中的每个位表示的都是一个特定对象存在与否，即可使用。再举一个非图形的例子，一个由52位组成的串，其中每个位与一张特定的扑克牌相关联，此位串可以用于表示一手5张牌，我们只需要将与手中牌相关联的5个位赋为1，将所有其他位赋为0。同理，13个位为1的52位位图可以用于表示一手桥牌，32位位图可以用于表示32种口味冰激凌的有无。

接着，假设存储单元中的8位被用作一个位图，我们想知道与从高位端算起的第3位相关联的对象是否存在。我们只需要将整个字节与掩码00100000做AND运算，当且仅当该位图从高位端算起的第3位本身为0时，才会产生一个全0的字节。于是，在该AND运算中安排一个条件跳转指令，程序就可以实现相应的动作。此外，如果该位图从高位端算起的第3位为1，我们想在干扰其他位的情况下将其改为0，那么可以把该位图与掩码11011111做AND运算，然后用结果替代原来的位图。

AND运算可用于复制一个位串的一部分，方法是将不复制的部分置0；OR运算也可用于复制一个串的一部分，但方法是将不复制的部分置1。为此，我

们再次使用掩码，但是这次用0来指示要复制的位的位置，用1指示不复制的位置。例如，一个任意字节和11110000做OR运算，得到的结果是最高有效4位为1，其余位是另一个操作数最低有效4位的副本，如下例所示：

$$\begin{array}{r} 11110000 \\ \text{OR } 10101010 \\ \hline 11111010 \end{array}$$

因此，可以用掩码11011111做AND运算，强制一个字节从高位端算起的第3位变为0，用掩码00100000做OR运算，强制同样的位置变为1。

XOR运算的一个主要用途是形成一个位串的补码。任意一个字节与全部为1的掩码做XOR运算得到的是该字节的补码。例如，注意下面示例中第二个操作数与结果的关系：

$$\begin{array}{r} 11111111 \\ \text{XOR } 10101010 \\ \hline 01010101 \end{array}$$

XOR运算可以用于反转一个RGB位图图像的所有位，结果产生一个“反转的”彩色图像，其中浅色被替换成深色，深色被替换成浅色。

在Vole机器语言（见附录C）中，操作码0x7、0x8和0x9分别用于逻辑运算OR、AND和XOR。每个操作码要求2个指定寄存器的内容之间进行相应的逻辑运算，并将结果存放在另一个指定的寄存器中。例如，指令0x7ABC要求的是：将寄存器0xB和0xC的内容进行OR运算，并将结果存放在寄存器0xA中。

2.4.2 循环移位运算及移位运算

循环移位运算及移位运算提供了移动寄存器内二进制位的方法，常用于解决对齐问题。这些运算根据移动方向（向右或向左）和其过程是否循环来分类。这些分类准则的混合使用产生了许许多多变体。下面我们简单介绍一下所涉及的概念。

我们来考虑包含一个字节二进制位的寄存器。如果我们将其内容向右移一位，我们想象最右端的位被移出边界，最左端出现一个空位。对于这个额外的位及留出的空位如何操作是区别各种移位运算的特征。一种技术是将右端移出的位放置在左端的空位上，这类运算称为**循环移位**（circular shift或rotation）。因此，如果我们对一个字节的位模式进行8次右循环移位，那么我们得到的位模式将与初始位模式相同。

另一种技术是丢弃移出边界的位，并用0填充空位，这类运算称为**逻辑移位**（logical shift）。这种向左的移位可以用于实现2乘以二进制补码的运算。因此，二进制数字左移相当于乘以2，就像十进制数字左移就相当于乘以10。此外，被2除的运算可以通过二进制串右移来完成。无论哪种移位，在使用某些记数法系统时都必须注意保留符号位。因此，我们经常发现，右移时留出的空位（在符号位的位置出现）总是用它原来的值来填。保留符号位不变的移位称为**算术移位**（arithmetic shift）。

在各种可能的移位指令和循环移位指令中，Vole机器语言（见附录C）仅包括一条右循环移位指令，由操作码0xA指定。在这个移位运算中，操作数的第一个十六进制数字指定了要循环移位的寄存器，操作数的其余部分指定了要循环移位的位数。因此，指令0xA501的意思是“将寄存器0x5的内容循环右移1位”。特别地，如果寄存器0x5最初包含位模式0x65，那么执行此指令后将包含0xB2（见图2-12）。（尝试一下如何组合使用Vole机器语言提供的指

令来产生其他移位及循环移位指令。例如，因为一个寄存器的长度为8位，所以循环右移3位与循环左移5位所得结果一样。）

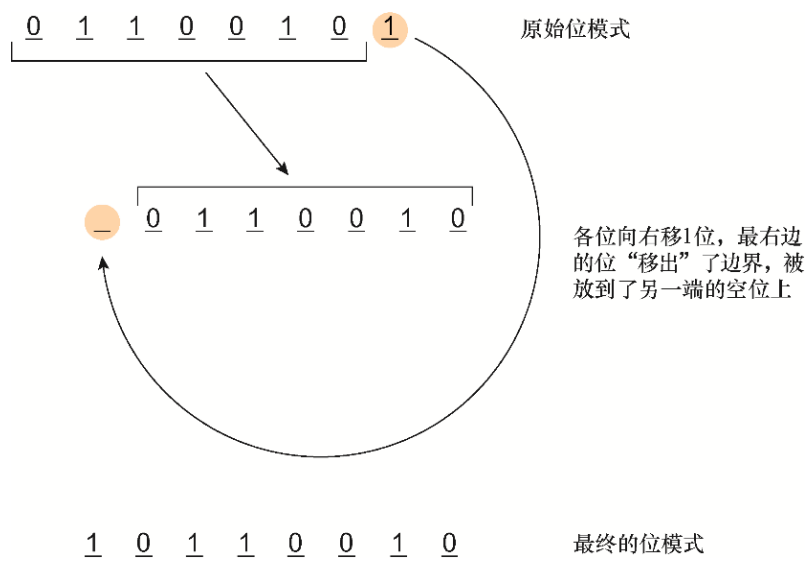


图2-12 将位模式0x65循环右移1位

2.4.3 算术运算

尽管我们已经提到过算术运算——加法、减法、乘法和除法，但一些细枝末节的问题还需要再解释一下。首先，我们已经看到减法运算可以通过加法及取负来模拟。此外，乘法只不过就是反复进行加法运算，除法就是反复进行减法运算。（因为6中可以减去3个2，所以6除以2等于3。）因此，一些小型CPU的设计中只有加法指令，或者只有加法和减法指令。

我们还应该注意到，每种算术运算都有许多变体。在谈及Vole里的加法运算时，我们已经暗示了这一点。例如，对于加法运算，如果相加的值用二进制补码记数法存储，此加法过程的实现就一定是每列数字直接相加。然而，如果操作数用浮点值存储，加法过程就必须提取每个操作数的尾数，根据指数字段将其向左或向右移位，检查符号位，执行加法，最后将其结果翻

译成浮点记数法。因此，尽管这两个运算都被当作加法运算，但是机器的实现过程并不相同。

2.4节问题与练习

1. 完成下列运算。

- | | | | | | |
|----|---------------------|----|---------------------|----|---------------------|
| a. | 01001011 | b. | 100000011 | c. | 11111111 |
| | <u>AND 10101011</u> | | <u>AND 11101100</u> | | <u>AND 00101101</u> |
| d. | 01001011 | e. | 10000011 | f. | 11111111 |
| | <u>OR 10101011</u> | | <u>OR 11101100</u> | | <u>OR 00101101</u> |
| g. | 01001011 | h. | 100000011 | i. | 11111111 |
| | <u>XOR 10101011</u> | | <u>XOR 11101100</u> | | <u>XOR 00101101</u> |

2. 假设你想从一个字节中分离出中间的4位，方法是中间的4位保持不变，将其他4位设为0。那么必须使用什么掩码及什么运算？

3. 假设你想对一个字节的中间4位取反，其他4位保持不变。那么必须使用什么掩码及什么运算？

4. a. 假设你对一个位串的前2位进行XOR运算，然后相继对每一个结果与位串的下一个位进行XOR运算。那么最后的结果与该串中1的个数有什么关系？

b. 这个问题与在对一个信息编码时需要确定使用什么样的奇偶校验位有什么关系？

5. 通常使用逻辑运算代替数值运算是很方便的。例如，逻辑运算AND组合两个位的方法同乘法运算一样。哪一种逻辑运算和两个位的加法几乎相同，这种情况下会导致什么问题？

6. 用什么逻辑运算和什么掩码能将ASCII码中的小写字母变为大写字母？大写变小写时呢？

7. 对下列位串执行循环右移3位会得到什么结果？

a. 01101010 b. 00001111 c. 01111111

8. 对下面用十六进制记数法表示的字节，执行循环左移1位的运算，结果是什么？请用十六进制形式给出答案。

a. 0xAB b. 0x5C c. 0xB7 d. 0x35

9. 一个8位位串循环右移3位等价于循环左移多少位？

10. 如果位模式01101010与11001100是以二进制补码记数法存储的值，那么它们的和的位模式是什么？如果这两个位模式是用第1章讨论的浮点格式存储的呢？

11. 用Vole机器语言（见附录C）编写一个程序，将地址为0xA7的存储单元的最高有效位置为1，存储单元里其他位的值保持不变。

12. 编写一个Vole程序，将存储单元0xE0的中间4位复制到存储单元0xE1中的最低有效4位，并将存储单元0xE1的最高有效4位置为0。

*2.5 与其他设备通信

主存储器和CPU构成了计算机的核心。本节将研究这个核心（将称它为计算机）如何与外围设备通信，如海量存储系统、打印机、键盘、鼠标、显示屏、数码相机甚至其他计算机。

2.5.1 控制器的作用

计算机与其他设备的通信通常是通过称为**控制器**（controller）的中间设备来处理的。对于台式计算机，控制器可能由计算机主板上固定安装的电路组成，有时为了灵活，它还会以扩展板的形式插在主板的插槽中。无论哪种形式，控制器都是通过电缆与计算机机箱里的外围设备，或者是与计算机背

面称为**端口**（port）的连接器的，端口上可以插其他设备。这些控制器有时本身就是小型计算机，每个都有其自己的存储电路和简单的CPU，可以执行引导控制器活动的程序。

控制器将信息和数据在两种形式之间来回翻译：一种是与计算机内部特征相适应的形式，另一种是与所连接的外围设备相适应的形式。最初，每个控制器都是为特定类型的设备设计的，因此，购买一种新的外围设备常常还需要同时购买一种新的控制器。随着计算机越来越普及，可用外围设备的数量增加得越来越快，出现了允许单个控制器处理许多种设备的通信标准。**通用串行总线**（universal serial bus，USB）、**高清晰度多媒体接口**（high definition multimedia interface，HDMI）和**DisplayPort**的后续更快版本都是被广泛采用的将外围设备连接到计算机的标准。例如，一个USB控制器可以用作计算机与其他任何同USB兼容的系列设备的接口。（理论上，最多可同时接 2^7-1 个。）现在，市场上可以与USB控制器通信的设备包括鼠标、打印机、扫描仪、外部磁盘和闪存驱动器、数码相机、扬声器、智能手表、智能手机以及许多其他设备。

每个控制器与其所在计算机的通信，都是通过连接到连接计算机的CPU和主存储器的总线来实现的（见图2-13）。从这个位置，每个控制器都能够监视到CPU与主存储器之间正在发送的信号，还能够将自己的信号传入总线。

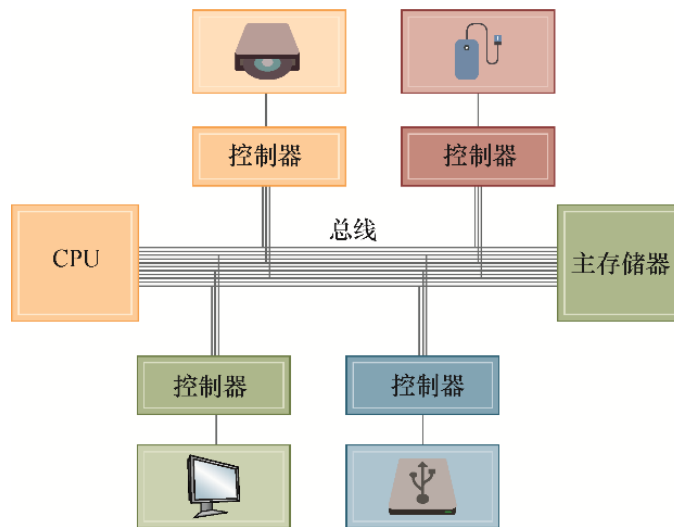


图2-13 连接到机器总线上的控制器

通过这种安排，CPU能够以与主存储器通信的方式与连接到总线上的控制器通信。为了给控制器发送一个位模式，首先要在CPU的其中一个通用寄存器中构建该位模式，然后由CPU执行一个类似STORE指令的指令，将该位模式“存储”到控制器中。同样，从一个控制器接收一个位模式时，要使用一条类似LOAD指令的指令。

在一些计算机的设计中，通过控制器的数据传输（传入与传出）是由LOAD和STORE操作码（这些操作码已经用于同主存储器的通信）指挥的。在这种情况下，每个控制器都被设计为响应唯一一组地址的引用，而主存储器被设计成忽略对这些位置的引用。因此，当CPU在总线上发送一条消息，要把一个位模式存储到一个分配给某个控制器的存储器位置时，这个位模式实际上是“存储”在该控制器中的，而不是主存储器中。同样，如果CPU试图从这样的存储器位置读取数据（如用LOAD指令），那么它收到的位模式将来自控制器而不是存储器。这样的通信系统称为**内存映射输入/输出**（memory-mapped I/O），因为该计算机的输入/输出设备对CPU来讲像是在不同的存储器位置里（见图2-14）。

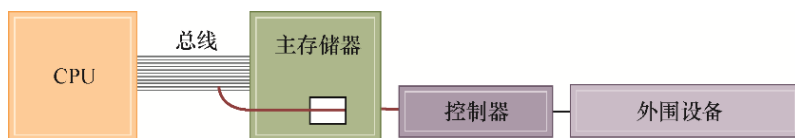


图2-14 内存映射输入/输出的概念表示

另一种内存映射输入/输出是在机器语言中提供特殊的操作码，用以指挥通过控制器的数据传输（传入与传出）。具有这些操作码的指令称为I/O指令。例如，如果Vole遵循这种方法，它可能包括诸如0xF5A3这样的指令，表示“将寄存器0x5的内容存储（STORE）在由位模式0xA3指定的控制器中”。

2.5.2 直接存储器存取

因为控制器是连接到计算机的总线上的，所以它能在CPU不使用总线的几纳秒时间里实现与主存储器的通信。控制器这种存取主存储器的能力称为**直接存储器存取**（direct memory access，DMA），它是计算机性能的重大资本。例如，要从磁盘扇区检索数据，CPU可以将编码为位模式的请求发送给连接这个磁盘的控制器，要求该控制器读取这个扇区并将数据存储指定的主存储器区域中。然后，在该控制器执行此读操作并通过DMA将数据存储在主存储器中时，CPU可以继续执行其他任务。因此，这两个活动将会被同时执行。CPU将执行某个程序，而控制器则监视磁盘与主存储器之间的数据传输。这样，在数据传输相对缓慢时，CPU的计算资源不会被浪费。

使用DMA同样也有不利的影响——使计算机总线上的通信复杂化。位模式必须在CPU与主存储器之间、CPU与每个控制器之间以及每个控制器与主存储器之间进行传送。协调总线上的所有这些活动是一个很大的设计难题。即使设计非常出色，CPU与控制器竞争总线存取时，中央总线也可能成为障碍。此障碍称为**冯·诺依曼瓶颈**（von Neumann bottleneck），因为它源于**冯·诺**

依曼体系结构（ von Neumann architecture ） ，在这种**体系结构**中，CPU是通过中央总线从存储器取指令的。

2.5.3 握手

两个计算机部件之间的数据传输很少是单向进行的。即使我们可以把打印机看作接收数据的设备，但事实上它也向计算机发送数据。毕竟，计算机产生字符并向打印机发送字符的速度要远远快于打印机能够打印的速度。如果计算机盲目地把数据发送给打印机，那么打印机很快就会落在后面，导致数据丢失。因此，诸如打印文档这样的过程都会涉及持续的称为**握手**（ handshaking ） 双向对话，计算机和外围设备通过这种双向对话交换设备的状态信息，协调它们的活动。

握手通常涉及一个**状态字**（ status word ） ，它是由外围设备生成并发送给控制器的一个位模式。该状态字是一个位图，其中的位反映该设备的各种情况。以打印机为例，其状态字的最低有效位的值可以表示该打印机是否缺纸，而下一个位可以表示该打印机是否已经准备好再接收数据，还有一个位可用于指出是否卡纸。控制器是自己响应这个状态信息，还是交由CPU来处理，取决于不同的系统。无论哪种情况，状态字都提供了一种用于协调与外围设备通信的机制。

2.5.4 流行的通信媒介

两个计算设备之间的通信可通过两种途径来处理：并行和串行。这两个术语指的是信号传输的方式。**并行通信**（ parallel communication ） 是指若干信号同时传输，每个信号都在各自的“线路”上。这种技术传输数据的速度快，

但是需要相对复杂的通信路径。例如，计算机的内部总线，其中多条线路被用于允许同时传输大量数据块及其他信号。

与此相反，**串行通信**（serial communication）是指信号在一条线路上一个接一个地传输。因此，相对于并行通信，串行通信只需一条相对简单的数据路径，这也是它很流行的原因。USB和FireWire能在短短几米的距离内提供相对高速的数据传输，是串行通信系统的例子。对于稍长的距离（在家中或者办公楼里），通过以太网连接（见4.1节）的串行通信，无论是通过导线连接的还是通过无线电广播连接的，都很流行。

多年来，传统的语音电话线在远距离通信方面一直主宰着个人计算机领域。这些通信路径都只有一根电线并通过它逐一传输语音信号，本质上属于串行系统。这样的数字数据传输实现过程如下：首先利用**调制解调器**（modulator-demodulator，简称为modem）将位模式转换为听得见的音调，并通过电话系统串行传输，然后在目的地由另一个调制解调器将这些音调重新转换成二进制位。

为了通过传统的电话线实现更加快速的远距离通信，电话公司提供了一种称为**数字用户线**（digital subscriber line，DSL）的服务，它利用了以下事实：现有的电话线实际上能够处理比传统语音通信所用频率范围更宽的频率范围。确切地说，DSL是将高于可听范围的频率用于传输数字数据，而将较低频谱用于语音通信。虽然DSL已非常成功，但电话公司也在迅速将自己的系统升级为光纤线（光纤线比传统电话线更容易支持数字通信）。

线缆调制解调器（cable modem）是一种竞争技术，它可以调制和解调位模式，使其能够在有线电视系统上传输。许多线缆提供商现在都同时使用光纤线和传统的同轴电缆，同时提供高清晰度电视信号和计算机网络访问。

卫星链路通过高频无线电广播甚至能使一些远离高速电话和有线电视网络的偏远地区访问到计算机网络。

2.5.5 通信速率

位从一个计算部件传到另一个计算部件的速率是以**位每秒**（又称比特每秒）（bits per second, bit/s）计量的。常用的单位有**Kbit/s**（kilo-bps的简写，即千位每秒，等于 10^3 bit/s）、**Mbit/s**（mega-bps的简写，即兆位每秒，等于 10^6 bit/s）和**Gbit/s**（giga-bps的简写，即吉位每秒，等于 10^9 bit/s）。注意位与字节之间的区别：8 Kbit/s相当于1 KB/s。使用简写形式时，小写的英文字母b通常表示**位**（bit），而大写的英文字母B通常表示**字节**（byte）。

对于短距离通信，USB 3.0及Thunderbolt（雷电）可以提供多倍Gbit/s的传输速率，对于大多数多媒体应用已经足够了。再加上便利性和相对低价，如今它们已被广泛用于家用计算机与本地外围设备（如打印机、外部磁盘驱动器及相机）之间的通信。

通过结合**多路复用**（multiplexing，数据编码或交织，使得一条通信路径可完成多条通信路径的功能）及数据压缩技术，传统的语音电话系统能够支持57.6 Kbit/s的传输速率。这无法满足当今多媒体和因特网应用（如来自Netflix或YouTube这类网站的高清视频流）的需要。播放MP3音乐录音需要大约64 Kbit/s的传输速率，即使播放低品质的视频也需要以Mbit/s计量的传输速率。这正是能够提供Mbit/s范围的传输速率的DSL、线缆以及卫星链路等已取代传统音频电话系统的原因。

一个特定设置可获得的最大速率，取决于通信路径的种类以及其实现过程中使用的技术。这个最大速率通常大致等同于通信路径的**带宽**

(bandwidth)，但**带宽**这个术语除了传输速率这个含义还有容量的含义。也就是说，说一条通信路径具有高带宽（或提供**宽带**服务）意味着一条通信路径能以高速率传输位，同时意味着该通信路径还能够同时携带大量信息。

2.5节问题与练习

1. 假设Vote使用内存映射输入/输出，地址0xB5是打印机端口内的位置，要打印的数据应该发送给它。
 - a. 如果寄存器0x7包含字母A的ASCII码，那么要通过打印机打印该字母，应该使用哪条Vote机器语言指令？
 - b. 如果该计算机每秒执行100万条指令，那么在一秒内可以向打印机发送这个字符多少次？
 - c. 如果该打印机每分钟可以打印5页传统文本，那么对于(b)中的情况，它能够跟得上发送给它的字符吗？
2. 假设你的个人计算机的硬盘转速为每分钟3000转，每个磁道有16个扇区，每个扇区有1024字节。如果磁盘控制器打算从磁盘驱动器中接收从旋转磁盘中读到的位，那么磁盘驱动器与磁盘控制器之间的通信速率大约需要是多少？
3. 一本用16位Unicode字符编码的300页的小说，按照54 Mbit/s的传输速率需要传输多久？

*2.6 数据操控编程

诸如Python这样的计算机程序设计语言的一个基本特征是，向用户屏蔽机器最底层的乏味的工作细节。刚刚用了多半章的篇幅介绍了计算机处理器中最底层的数据操控，回顾一下Python脚本屏蔽的无须程序员操心的主要细节是有益的。

我们将在第6章更详细地探讨，高级程序设计语言语句为了执行会映射为低级机器指令。一条Python语句可能会映射为一条机器指令，也可能会映射为数十或者数百条机器指令，具体取决于语句的复杂性和机器语言的效率。Python语言解释器的不同实现，和计算机操作系统软件的其他元素是一致的，会为每个特定的计算机处理器处理这个映射过程。因此，Python程序员无须知道自己的Python脚本是在一个RISC处理器上执行，还是在一个CISC处理器上执行。

基本知识点

- 一种程序设计语言的代码经常会被翻译成要在机器上执行的另一种（较低级的）语言的代码。

我们会认识许多与现代计算机或附录C中描述的Vole机器的基本机器指令密切对应的Python操作。Python的整数加法和浮点数加法明显类似于我们的简单机器中的ADD操作码。给变量赋值一定会涉及某个安排中的LOAD、STORE和MOVE操作码。Python不用我们操心都使用了哪些处理器寄存器，它利用机器操作码来执行我们的指令。我们无法看到指令寄存器、程序计数器或者存储单元地址，但是Python脚本是一条语句接着一语句顺序执行的，和简单机器语言程序的执行方式一样。

基本知识点

- 程序指令是顺序执行的。

2.6.1 逻辑运算和移位运算

逻辑运算和移位运算可以在任何类型的数值数据上执行，但因为它们经常处理数据的各个二进制位，所以用二进制值为例来说明这些运算是最容易的。就像Python使用“0x”前缀指定十六进制值一样，“0b”前缀可用于指定二进制值^[1]。

[1] 这个语法是最近加到演变的Python语言中的。请确保至少使用Python 3来复制这些示例。

```
x      = 0b00110011
mask   = 0b00001111
```

注意，实际上，这与给x赋值51（它的二进制值是110011）或者0x33（这是51的十六进制表示）没什么不同，与给mask赋值15（它的二进制值是1111）或者0x0F（15的十六进制表示）也没什么不同。我们在Python赋值语句中用来表达整数值的表示，不会改变这个值在计算机中的表示，而只是为了阅读的人能理解它。

2.4节介绍了每个按位逻辑运算符的内置Python运算符。

```
print(0b000000101 ^ 0b000000100)    # Prints 5 XOR 4, which is 1
print(0b000000101 | 0b000000100)    # Prints 5 OR 4, which is 5
print(0b000000101 & 0b000000100)    # Prints 5 AND 4, which is 4
```

因此，我们可以将2.4节中的每一个示例问题复制为Python代码。

```
print(0b10011010 & 0b11001001)      #      10011010
                                     #  AND 11001001
                                     #      10001000

print(0b10011010 | 0b11001001)      #      10011010
                                     #  OR 11001001
                                     #      11011011

print(0b10011010 ^ 0b11001001)      #      10011010
                                     #  XOR 11001001
                                     #      01010011
```

对于所有这些示例，Python会用它默认的输出表示（即十进制）来打印这个结果。如果用户也喜欢用二进制记数法显示输出结果，那么可以用内置函数将整数值转换成与二进制表示相对应的字符0和1组成的串。

```
print(bin(0b10011010 & 0b11001001))    # Prints "0b10001000"  
print(bin(0b10011010 | 0b11001001))    # Prints "0b11011011"  
print(bin(0b10011010 ^ 0b11001001))    # Prints "0b1010011"
```

因为较新版的Python可以使用任意个数字来表示一个数，不打印开头的0，所以上面的第3行只打印了7个数字，而不是8个。

Python用于执行逻辑移位运算的内置运算符由两个大于号（>>）和两个小于号（<<）组成，直观地显示了移位方向。运算符右边的操作数指示了要移位的位位置的编号：

```
print(0b00111100 >> 2) # Prints  "15", which is 0b00001111  
print(0b00111100 << 2) # Prints  "240", which is 0b11110000
```

将位进行移位除了能屏蔽二进制数的左边或右边，位移运算符还是一种乘以2的幂（左移）或者除以2的幂（右移）的有效方法。

2.6.2 控制结构

本章前面介绍的机器语言的控制类指令为我们提供了一种从程序的一个部分跳转到另一个部分的机制。在Python这样的高级语言中，这实现了所谓的**控制结构**（control structure），使我们能更简洁地表达算法的语法模式。这方面的一个例子是if语句，它允许代码段在脚本中的布尔值不为真的情况下被条件性地跳过。

```
if (water_temp > 140):  
    print('Bath water too hot!')
```

直观来看，这个Python代码段将被映射为多条机器指令，对water_temp变量和整数值140进行比较，这二者很可能是事先载入寄存器的。如果water_temp值不是140或者更大，条件跳转指令会跳过内置print()函数的机器指令。

另一种控制结构是循环结构while，它允许一个代码段被执行多次，一般受某个条件控制。

```
while (n < 10):  
    print(n)  
    n = n + 1
```

假设变量n从小于10的值开始，这个循环会一直打印n并在n上加1，直到它变为大于或等于10的数为止。

我们将在第5章及以后的各章中用更多的时间来研究这些控制结构和其他控制结构。现在，我们重点研究“允许我们从程序的一个部分跳转到另一个部分，在那里执行一个要求的任务之后再返回到之前所在的程序点”的机制。

2.6.3 函数

我们已经看到了3种不遵循算术运算符和逻辑运算符的语法形式的内置Python运算。print()、str()和bin()运算都是使用给定名称而不是符号来调用的，而且它们的操作数外面还包有括号。

这两个都是一个称为**函数**（function）的Python语言特性的例子。在数学中，术语**函数**经常用于描述代数关系，如“ $f(x) = x^2 + 3x + 4$ ”。当看到这种函数定义时，我们就明白，在随后的行中，表达式“f(5)”的意思是值5应该被插入到定义f()的表达式中参数x所在的位置上。因此， $f(5) = 5^2 + 3 \times 5 + 4 = 25 +$

$15 + 4 = 44$ 。程序设计语言函数都是非常相似的，因为它们都允许我们使用一个名称来指代一系列应该在给定的一个参数或多个参数上执行的运算。由于这个语言特性被映射为较低级的机器语言的方式，函数在表达式或语句中的出现被称作**函数调用**（function call），有时称为**调用**（calling）函数。函数调用是程序设计语言降低代码复杂性的重要捷径。

基本知识点

- 函数是程序设计指令的一个命名分组。
- 函数是可复用的程序设计抽象。
- 函数简化了编写和维护程序的复杂性。

上面例子中出现的`print()`和`bin()`就是两个这样的函数调用：它们指明Python解释器要执行的指定函数的定义，然后返回继续它们的工作。语法是在函数名称后面紧接着放一个开括号，然后给出函数**参数值**（argument value），用于在对函数定义进行求值时插入参数，最后跟一个闭括号。开括号与闭括号的匹配很重要——不匹配会导致Python语法错误，这是初学者常犯的错误。每次调用函数都给函数赋不同的参数，允许以不同的参数多次复用函数中的代码，而不需要复制函数代码。

基本知识点

- 在程序中调用过程时，参数提供不同的值作为过程的输入。
- 参数通过允许使用函数而不是复制代码来归纳解决方案。

从现在起，我们将遵从我们讨论`print()`这样的Python函数时提到的括号使用约定，以便清楚地表示它们与变量或其他项不同。

函数有许多种类，比我们看到的多。有些函数有多个参数，如max()函数：

```
x = 1034
y = 1056
z = 2078
biggest = max(x, y, z)
print(biggest)                # Prints "2078"
```

括号内的多个参数是由逗号分隔的。有些函数**返回**（return）值，也就是说，这种函数调用本身可以出现在一个更复杂的表达式中，或者出现在赋值语句的右边。这些函数有时称为**有返回值函数**（fruitful function）。max()（如上）和bin()都是这种函数，它们以整数值为参数，返回相应的由0和1组成的串。其他函数不返回值，通常被用作独立语句，print()就是这种函数。不返回值的函数有时称为**无返回值函数**（void function）或**过程**（procedure），尽管它们在Python的语法规则中没有区别。将无返回值函数的结果赋值给变量是没有意义的，如

```
x = print('hello world!')    # x is assigned None
```

虽然在Python里这不是错误，但它与根本不给x赋值还不太一样。

基本知识点

- 过程有名称，可能还有参数和返回值。

到目前为止，我们所看到的函数都是Python知道的内置函数，这种函数有几十个，但还有一些更高级的脚本可以引用的附加函数的扩展库。Python库模块包含许多可能一般用不到但当需要的时候就可以调用的有用函数。

```
# Calculates the hypotenuse of a right triangle
import math

sideA = 3.0
```

```
sideB = 4.0
# Calculate third side via Pythagorean Theorem
hypotenuse = math.sqrt(sideA**2 + sideB**2)

print(hypotenuse)
```

在这个例子中，import语句预先警告了Python解释器：这个脚本引用的是名为“math”的库，它恰巧是Python配备的一组标准库模块中的一个。math库模块中定义的sqrt()函数提供了参数的平方根，在这个例子里，参数是表达式“sideA的平方加上sideB的平方”。注意，库函数调用包括模块名（math）和函数名（sqrt），中间由一个点连接。

Python的math模块包括几十个有用的数学函数，包括对数函数、三角函数和双曲函数，还有一些熟悉的常数值，如math.pi。

除了内置函数和库模块函数，Python还为脚本提供了定义其自己的函数的语法。我们将在本节末尾定义一些非常简单的例子，并在第3章研究更复杂的变体。

2.6.4 输入和输出

前面的示例代码段和脚本已经使用了Python内置的print()函数来输出结果。许多程序设计语言都提供了类似的实现输入和输出的机制，为程序员提供了一个方便的抽象，用于将数据移入或移出计算机处理器。事实上，这些I/O内置函数都是与硬件控制器及2.5节讨论的外围设备进行通信的。

到目前为止，我们的示例脚本还没有要求用户输入。简单的用户输入可以用Python内置的input()函数来完成。

```
echo = input('Please enter a string to echo: ')
print(echo * 3)
```

`input()`函数把提示字符串当作可选参数，在等待输入时呈现给用户。当这个脚本运行时，在显示完“Please enter a string to echo:”这个提示后，脚本会暂停，等待用户键入内容。当用户按下回车键时，脚本将键入的字符组成的字符串（不包括回车键）赋给变量`echo`。接着，脚本的第二行将这个字符串重复输出3次。（回顾：`*`运算符复制字符串操作数。）

现在有了获取输入的能力，让我们重写前面的斜边脚本来提示用户输入边长，而不是将值硬编码到赋值语句中。

```
# Calculates the hypotenuse of a right triangle
import math

# Inputting the side lengths, first try
sideA = input('Length of side A? ')
sideB = input('Length of side B? ')
# Calculate third side via Pythagorean Theorem
hypotenuse = math.sqrt(sideA**2 + sideB**2)

print(hypotenuse)
```

运行时，这个脚本提示用户“Length of side A? ”，并等待输入。假设用户键入“3”并回车。该脚本继续提示用户“Length of side B? ”，并等待输入。假设用户键入“4”并回车。此时，Python解释器中止脚本，打印输出：

```
hypotenuse = math.sqrt(sideA**2 + sideB**2)
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

像Python这样的动态类型语言很容易产生这种类型的错误。我们的斜边计算在这个脚本的早期版本中能正常工作，现在把值改为由用户输入后就会导致错误。这个问题实际是一个“TypeError”错误，这个错误源于Python不再知道如何获取变量`sideA`的平方这一事实，因为现在这个脚本中的`sideA`是一个字符串，而不是之前那样的整数。这个问题不是在这个脚本中计算斜

边的那一行产生的，而是在前面`input()`返回`sideA`和`sideB`的值时产生的。这个问题也是使用程序设计语言出错时常见的问题。Python解释器试图提供这个程序出错的脚本行，但真正导致出错的行实际上在这行脚本的前面。

在上面的字符串回显代码段中，很明显，赋给`echo`的值应该是用户键入的字符串。虽然程序员的目的现在是输入整数值，但`input()`函数的行为方式还和斜边程序中一样。字符串“4”的ASCII或UTF8编码表示与整数4的二进制补码表示不同，在使用整数继续进行计算之前，该Python脚本必须显式地将一种表示转换为另一种表示。

幸运的是，另一个内置函数提供了这种功能。`int()`函数会尝试将其参数转换为整数表示。如果无法转换，会产生相应的错误信息。

在这个脚本中，至少有3个地方可以使用`int()`函数来修正bug。我们甚至可以在将`input()`函数的结果赋给变量之前就调用它。我们可以只为调用转换函数添加新脚本行，还可以只在对`math.sqrt()`函数调用里的变量进行平方运算之前进行转换。下面这个修订后的脚本使用了第一种方案，第二种方案作为练习留给读者。

```
# Calculates the hypotenuse of a right triangle
import math

# Inputting the side lengths, with integer conversion
sideA = int(input('Length of side A? '))
sideB = int(input('Length of side B? '))
# Calculate third side via Pythagorean Theorem
hypotenuse = math.sqrt(sideA**2 + sideB**2)

print(hypotenuse)
```

修订后的脚本能按预期运行，并且能用于许多直角三角形，而无须编辑脚本，就像预输入版本一样。

最后需要注意的一点是，`int()`函数是通过仔细检查其字符串参数并将其解释为数来执行转换的。如果输入的字符串是一个数，但不是整数，如“3.14”，那么`int()`函数会丢弃小数部分，只返回整数值。这是一个截断操作，不会像人类期望的那样“向上舍入”。

Python中所有的其他标准值类型都有类似的转换函数。

2.6.5 马拉松训练助手

下面这个完整的Python脚本演示了本节介绍的许多概念。随着休闲长跑受欢迎程度的增加，许多参与者发现自己正在追求复杂的身体训练计划，为严酷的马拉松比赛做身体准备。这个脚本能根据跑步者想跑的距离和步速，帮助跑步者计算他需要多久的训练性锻炼。给定步速（pace，跑1英里的分钟数和秒数）和总里程，该脚本计算跑步锻炼的预计跑行时间，以及一个以英里每小时为单位的用户友好的速度。图2-15给出了一些示例数据点，在每行中，前3列是输入，后3列是预期的结果。注意，不同的Python实现输出的速度值的小数点后的位数可能跟图2-15中的不同，图2-15中的值没有四舍五入。

每英里的时间						总跑行时间	
分	秒	英里	速度 (m/h)			分	秒
9	14	5	6.49819494584			46	10
8	0	3	7.5			24	0
7	45	6	7.74193548387			46	30
7	25	1	8.08988764044			7	25

图2-15 马拉松训练数据示例

```
# Marathon training assistant.
import math

# This function converts a number of minutes and seconds into just seconds.
```

```
def total_seconds(min, sec):
    return min * 60 + sec

# This function calculates a speed in miles per hour given
# a time (in seconds) to run a single mile.
def speed(time):
    return 3600 / time

# Prompt user for pace and mileage.
pace_minutes = int(input('Minutes per mile? '))
pace_seconds = int(input('Seconds per mile? '))
miles = int(input('Total miles? '))

# Calculate and print speed.
mph = speed(total_seconds(pace_minutes, pace_seconds))
print('Your speed is')
print(mph)

# Calculate elapsed time for planned workout.
total = miles * total_seconds(pace_minutes, pace_seconds)
elapsed_minutes = total // 60
elapsed_seconds = total % 60

print('Your total elapsed time is')
print(elapsed_minutes)
print(elapsed_seconds)
```

上面的脚本既使用了内置函数`input()`、`int()`和`print()`，又使用了用户定义的函数`speed()`和`total_seconds()`。关键字`def`的后面是用户函数定义，再后面是函数调用时要提供的函数名称和参数列表。接下来的缩进行称为函数体（body），表示的是定义该函数的步骤。在后面的章节中，我们会看到一些函数体内包含多条语句的函数的示例。关键字`return`突出显示了用于计算该函数结果的表达式。

虽然用户定义的函数都是在脚本的顶部定义的，但实际上并没有在顶部被调用，而是在后面的脚本中作为一个较大表达式的一部分被调用的。另外还要注意这个脚本中函数调用堆叠的方式。`input()`函数的调用结果被立即当作参数传递给`int()`函数，然后`int()`函数的结果被赋值给一个变量。同样，`total_seconds()`函数的结果也被立即当作参数传递给`speed()`函数，`speed()`函数的结果然后又被赋值给变量`mph`。在这两种情况下，都允

许一次调用一个函数，将结果赋值给新变量，然后调用下一个依赖头一个结果的函数。然而，这种更紧凑的形式更简洁，不需要增加临时变量来存放计算的中间结果。

给定输入每英里7分45秒，跑6英里，该脚本输出：

```
Your speed is
7.74193548387
Your total elapsed time is
46
30
```

这个输出格式仍然相当原始。它缺乏适当的单位（7.74193548387 mph和46 minutes、30 seconds），对一个简单计算来讲，这个输出值的小数位数不合适，而且整个输出结果有太多断行。更清晰的输出作为练习留给读者。

2.6节问题与练习

1. 斜边示例脚本将两个边长截断为整数，但输出却为浮点数，为什么？改写这个脚本，使其输出整数。
2. 改写斜边脚本，使用浮点数作为输入，不截断它们。前一个问题里的整数版本和这个问题里的浮点数版本，哪个更合适？
3. Python的内置函数`str()`能把一个数值参数转换为一个字符串表示，`'+'`可以用于将字符串连接在一起。使用这些修改马拉松脚本，产生更清晰的输出，例如：

```
Your speed is 7.74193548387 mph
Your total elapsed time is 46 minutes, 30 seconds
```

4. 使用Python内置的`bin()`函数写一个脚本，读取一个十进制整数作为输入，用1和0输出这个整数对应的二进制表示。

5. XOR运算常常用于高效地计算校验和（见1.9节）和加密（见4.5节）。写一个简单的Python脚本，读取一个数，输出这个数与一个1、0模式（如0x55555555）进行XOR运算的结果。同样的脚本将把一个数“加密”为一个看似不相关的数，但当再次运行时，给定这个加密数当作输入将返回原来的数。

6. 通过为本节的示例脚本提供一些意外的输入，创造一些错误，研究这些错误条件。如果你在斜边脚本或马拉松脚本中输入全0会怎么样？负数呢？用字符串代替数值呢？

*2.7 其他体系结构

为了拓宽视角，我们来考虑一些已经讨论过的传统计算机体系结构的替代方案。

2.7.1 流水线

电子脉冲在电线上传播的速度不比光速快。光大约每纳秒（十亿分之一秒）能传播1英尺（约30 cm）的距离，然而CPU从1英尺远的存储单元中取指令至少需要2 ns。（读请求必须发送到存储器，这至少需要1 ns，而指令又必须送回CPU，这至少也需要1 ns。）因此，在这样的机器中取指和执行一条指令需要若干纳秒——这就意味着提高机器执行速度的问题最终变成小型化问题。

然而，提高执行速度并不是改进计算机性能的唯一途径。真正目的是改进机器的吞吐率（throughput）——机器在给定时间内可以完成的工作总量。

在不要求提高执行速度的前提下，增加计算机吞吐量的一个例子涉及**流水线**（pipelining）技术，该技术允许一个机器周期内的各个步骤重叠进行。特别是，当执行一条指令时，可以取下一条指令，这意味着在任何一个时刻可以有不止一条指令在“流水线”上，每条指令处在不同的处理阶段。这样，尽管取和执行每条指令的时间保持不变，机器的总吞吐量却提高了。（当然，当到达JUMP指令时，预取的好处就都没有了，因为“流水线”上的指令不是所需要的。）

现代机器设计中推行的流水线概念已超出了我们所举的简单示例。它们经常能够同时取若干条指令，并且一次可以执行多条彼此互不依赖的指令。

2.7.2 多处理器机器

可以把流水线技术看作迈向**并行处理**（parallel processing）的第一步，并行处理是若干活动在同一时间里实现的性能。然而，真正的并行处理需要多个处理单元，于是产生了称为多处理器或者**多核**（multi-core）机器的计算机。

多核CPU

随着技术的发展在硅芯片上可以放置的电路越来越多，计算机部件之间的物理差异越来越小。例如，一个芯片可能包含一个CPU和一个主存储器。这是“单片系统”（system-on-a-chip，SoC）方法的一个例子，其目标是在单个设备中提供一个完整的装置，该装置可用作更高级设计中的抽象工具。在其他情况下，在单个设备内提供相同电路的多个复制品。后一种策略最初是以芯片的形式出现的，其中包含多个独立的门，可能还有多个触发器。今天的技术发展水平允许将多个完整的CPU放在一个单个的芯片上。这就是称为多核CPU的设备的底层架构，它由位于同一个芯片上的两个或多个CPU和共享

高速缓冲存储器组成。这种设备简化了多指令多数据（MIMD）系统的构造，并已被很容易地用在家用计算机上。

当今大多数计算机的设计都基于这种思想。一个策略是将若干处理单元（每一个都像单处理器机器中的CPU）都连接到同一个主存储器上。在这种配置中，各处理器可以独立工作，并通过把相关的信息放在公共存储单元里来协调各自的工作。例如，当某个处理器遇到一个大任务时，它可以将部分任务的程序存储在这个公共存储器中，然后请求另一个处理器执行它。结果产生这样的计算机：不同的指令序列在不同的数据集上执行，这种体系结构称作**多指令流多数据流**（multiple-instruction stream，multiple-data stream，MIMD）体系结构，与较传统的**单指令流单数据流**（single-instruction stream，single-data stream，SISD）体系结构相对。

多处理器体系结构的一个变体是将多个处理器链接起来，以便它们一起执行同一个指令序列，每个处理器都有各自的数据集。这就产生了**单指令流多数据流**（single-instruction stream，multiple-data stream，SIMD）体系结构。这种机器适用于这样的应用：在一大堆数据中，对于其中每组类似的数据项都要执行同样的任务。并行处理的另一种方法是将大型计算机构建成为小型机器的集合体，每台小型机器都有自己的存储器和CPU。这样的体系结构中，每台小型机器都与其邻居相连接，以便分配给整个系统的任务可以分解到各台小型机器上。因此，如果分配给某台内部机器的一项任务可以分解为若干个独立的子任务，那么这台内部机器就可以请求它的邻居们并发地完成这些子任务。这样，完成任务的时间可以比由一台单处理器机器独立完成任务所需的时间少得多。

2.7节问题与练习

1. 回到2.3节的问题3，如果该机器使用本节讨论的流水线技术，那么在地址为0xAA的指令被执行时，“流水线”里有什么？在什么条件下，该程序不会从流水线技术得到好处？

2. 在一台流水线机器上运行2.3节问题4中的程序时，必须要解决什么冲突？

3. 假设有两个“中央”处理器连接到同一个存储器上，但执行不同的程序；再假设，其中一个处理器需要给一个存储单元的内容加1，几乎同时，另一个处理器需要给同一个存储单元的内容减1。（净效果应该是该存储单元最终保持开始时的值。）

a. 描述一个执行序列，其结果为该单元最终值比开始值少1。

b. 描述一个执行序列，其结果为该单元最终值比开始值大1。

复习题

（带*的题目涉及选读章节的内容。）

1. a. 在什么情况下，通用寄存器和主存储器存储单元类似？

b. 在什么情况下，通用寄存器和主存储器存储单元不同？

2. 根据Vole机器语言（见附录C）回答下列问题。

a. 将指令0x2304写成一个16位的位串。

b. 将指令0xB2A5的操作码写成一个4位的位串。

c. 将指令0xB2A5的操作数字段写成一个12位的位串。

3. 假设一个数据块存储在Vole机器中地址从0x98到0xA2（含）的存储单元中。这一数据块占据了多少存储单元？列出它们的地址。

4. 执行完指令0xB0CD后，Vole中的程序计数器的值是多少？

5. 假设在Vole中地址从0x00到0x05的存储单元中包含下列位模式。

地 址	内 容
0x00	0x22
0x01	0x11
0x02	0x32
0x03	0x02
0x04	0xC0
0x05	0x00

假定该程序计数器初始值为0x00，记录该程序从执行到机器停机这一过程中在每个机器周期取指阶段的末尾，程序计数器、指令寄存器以及地址为0x02的存储单元的内容。

6. 假设3个值 x 、 y 、 z 存储在机器的存储器中。描述在计算 $x + y + z$ 时发生的事件序列（如从存储器载入寄存器，将值保存在存储器中，等等）。计算 $(2x) + y$ 时又如何呢？

7. 下面是用Vole机器语言编写的指令。把它们翻译成为自然语言。

- a . 0x7123 b.0x40E1 c . 0xA304
d . 0xB100 e . 0x2BCD

8. 假设有一机器语言的操作码字段被设计为4位。那么该语言可以有多少种不同的指令？如果操作码字段增加到6位呢？

9. 将下列指令由自然语言翻译为Vole语言。

- a . 将值0x77加载 (LOAD) 到寄存器0x6。
b . 将存储单元0x77的内容加载 (LOAD) 到寄存器0x7。
c . 如果寄存器0x0的内容与寄存器0xA中的值相同，则跳转 (JUMP) 到存储器位置为0x24的指令。
d . 将寄存器0x4循环右移 (ROTATE) 3位。
e . 对寄存器0xE和寄存器0x2的内容进行AND运算，结果留在寄存器0x1中。

10. 重写图2-7中的程序，假定相加的值用浮点记数法编码，而不是二进制补码记数法。

11. 请根据下列Vole指令的执行是否改变位置为0x3C的存储单元的值、是否读取位置为0x3C的存储单元的内容、是否与位置为0x3C的存储单元的内容没有关系，对这些指令进行分类。

- a . 0x353C b.0x253C c . 0x153C
d . 0x3C3C e . 0x403C

12．假设在Vole中地址从0x00到0x03的存储单元中包含下列位模式。

地 址	内 容
0x00	0x26
0x01	0x55
0x02	0xC0
0x03	0x00

a．将第一条指令翻译为自然语言。

b．如果我们启动机器时其程序计数器的值为0x00，那么该机器停机时寄存器0x6中的位模式是什么？

13．假设在Vole中地址从0x00到0x02的存储单元中包含下列位模式。

地 址	内 容
0x00	0x12
0x01	0x21
0x02	0x34

- a . 如果机器在程序计数器值为0x00时启动，那么执行的第一条指令会是什么？
- b . 如果机器在程序计数器值为0x01时启动，那么执行的第一条指令会是什么？

14 . 假设在Vole中地址从0x00到0x05的存储单元中包含下列位模式。

地 址	内 容
0x00	0x12
0x01	0x02
0x02	0x32
0x03	0x42
0x04	0xC0
0x05	0x00

假设机器在其程序计数器的值为0x00时启动，回答下列问题。

- a . 把要执行的指令翻译成自然语言。
- b . 当机器停机时，地址为0x42的存储单元中的位模式是什么？
- c . 当机器停机时，程序计数器中的位模式是什么？

15．假设在Vole中地址从0x00到0x09的存储单元中包含下列位模式。

地 址	内 容
0x00	0x1C
0x01	0x03
0x02	0x2B
0x03	0x03
0x04	0x5A
0x05	0xBC
0x06	0x3A
0x07	0x00
0x08	0xC0
0x09	0x00

假设机器在其程序计数器的值为0x00时启动，回答下列问题。

a．当机器停机时，地址为0x00的存储单元里有什么？

b．当机器停机时，程序计数器中的位模式是什么？

16．假设在Vole中地址从0x00到0x07的存储单元中包含下列位模式。

地 址	内 容
0x00	0x2B
0x01	0x07
0x02	0x3B
0x03	0x06
0x04	0xC0
0x05	0x00
0x06	0x00
0x07	0x23

a．假设机器在其程序计数器的值为0x00时启动，请列出包含待执行程序
的存储单元的地址。

b．列出用于存放数据的存储单元的地址。

17．假设在Vole中地址从0x00到0x0D的存储单元中包含下列位模式。

地 址	内 容
-----	-----

地 址	内 容
0x00	0x20
0x01	0x04
0x02	0x21
0x03	0x01
0x04	0x40
0x05	0x12
0x06	0x51
0x07	0x12
0x08	0xB1
0x09	0x0C
0x0A	0xB0
0x0B	0x06

地 址	内 容
0x0C	0xC0
0x0D	0x00

假设机器在其程序计数器的值为0x00时启动，回答下列问题。

a．当机器停机时，寄存器0x0中的位模式是什么？

b．当机器停机时，寄存器0x1中的位模式是什么？

c．当机器停机时，程序计数器中的位模式是什么？

18．假设在Vole中地址从0xF0到0xFD的存储单元中包含下列（十六进制）位模式。

地 址	内 容
0xF0	0x20
0xF1	0x00
0xF2	0x22
0xF3	0x02
0xF4	0x23

地 址	内 容
0xF5	0x04
0xF6	0xB3
0xF7	0xFC
0xF8	0x50
0xF9	0x02
0xFA	0xB0
0xFB	0xF6
0xFC	0xC0
0xFD	0x00

如果机器在其程序计数器的值为0xF0时启动，那么当机器最终执行到地址0xFC处的停机指令时，寄存器0x0中的值是什么？

19．如果Vole每微秒（百万分之一秒）执行一条指令，那么完成第18题中的程序需要花多长时间？

20．假设在Vole中地址从0x20到0x28的存储单元中包含下列位模式。

地 址	内 容
0x20	0x12
0x21	0x20
0x22	0x32
0x23	0x30
0x24	0xB0
0x25	0x21
0x26	0x24
0x27	0xC0
0x28	0x00

假设机器在其程序计数器的值为0x20时启动。

- a．当机器停机时，寄存器0x0、0x1和0x2中的位模式是什么？
- b．当机器停机时，地址为0x30的存储单元中的位模式是什么？
- c．当机器停机时，地址为0xB0的存储单元中的位模式是什么？

21．假设在Vole中地址从0xAF到0xB1的存储单元中包含下列位模式。

地 址	内 容
0xAF	0xB0
0xB0	0xB0
0xB1	0xAF

如果机器在其程序计数器的值为0xAF时启动，那么会发生什么？

22．假设在Vole中地址从0x00到0x05的存储单元中包含下列（十六进制）位模式。

地 址	内 容
0x00	0x25
0x01	0xB0
0x02	0x35
0x03	0x04
0x04	0xC0
0x05	0x00

如果机器在其程序计数器的值为0x00时启动，那么机器会在什么时候停机？

23．对于下面每种情况，用Vole编写一个小程序来完成以下任务。假定每个程序都放在从地址0x00开始的存储器里。

a．将存储器位置0xD8的值移动到存储器位置0xB3。

b．交换存储在存储器位置0xD8和0xB3的值。

c．如果存储器位置0x44中存储的值是0x00，则将值0x01存放在存储器位置0x46中；否则，将值0xFF存放在存储器位置0x46中。

24．在计算机爱好者中曾经流行一种叫磁芯大战（Core Wars）的游戏——战舰（Battleship）游戏的变体。（术语**磁芯**来源于一种早期的存储技术，它用磁材料的小环的磁场方向表示0和1。这个小环称为磁芯。）这个游戏是在两个对立的程序之间玩的，每个程序分别存储在同一台计算机的存储器的不同位置。假设该计算机轮流执行这两个程序，先执行一个程序的一条指令，再执行另一个程序的一条指令。每个程序的目标都是通过把额外的数据写到另一个程序上来使对方失灵的；不过，哪个程序都不知道对方的位置。

a．用Vole编写一个程序，采用尽可能小的防御方式玩此游戏。

b．用Vole编写一个程序，通过不断变动自己的位置来避免受到对立程序的袭击。更确切地说，从位置0x00开始，编写一个程序，将自身复制到位置0x70，再跳转到位置0x70。

c . 扩展问题b中的程序，继续重定位到新的存储器位置。具体来说，先让程序移至位置0x70，然后移到0xE0（=0x70+0x70），再移到0x60（=0x70+0x70+0x70），等等。

25 . 用Vole编写一个程序，计算存储在存储器位置0xA0、0xA1、0xA2和0xA3中的浮点数的和。程序应该将所得的和存储在存储器位置0xA4中。

26 . 假设在Vole中地址从0x00到0x05的存储单元中包含下列（十六进制）位模式。

地 址	内 容
0x00	0x20
0x01	0xC0
0x02	0x30
0x03	0x04
0x04	0x00
0x05	0x00

如果机器在其程序计数器的值为0x00时启动，会发生什么？

27 . 如果Vole的地址为0x08和0x09的存储单元中分别包含位模式0xB0和0x08，并且机器启动时其程序计数器中的值为0x08，那么会发生什么？

28．假设下列Vole程序存储在从地址0x30（十六进制）开始的主存储器中。当执行该程序时它会完成什么任务？

```
0x2003  
0x2101  
0x2200  
0x2310  
0x1400  
0x3410  
0x5221  
0x5331  
0x3239  
0x333B  
0xB248  
0xB038  
0xC000
```

29．概述当Vole执行一条操作码为0xB的指令时所涉及的步骤。用一组说明来表达你的答案，就像你在告诉CPU要做什么。

*30．概述当Vole执行一条操作码为0x5的指令时所涉及的步骤。用一组说明来表达你的答案，就像你在告诉CPU要做什么。

*31．概述当Vole执行一条操作码为0x6的指令时所涉及的步骤。用一组说明来表达你的答案，就像你在告诉CPU要做什么。

*32．假设Vole中的寄存器0x4和0x5中包含的分别是位模式0x3A和0xC8，在执行下列每条指令后，寄存器0x0中会留下什么位模式？

- a . 0x5045 b . 0x6045 c . 0x7045
d . 0x8045 e . 0x9045

*33．编写Vole程序来执行下面的每个任务。

- a . 将存储器位置0x44中存储的位模式复制到存储器位置0xAA中。

b . 将存储器位置0x34中的最低有效4位变成0 , 并保持其他位不变。

c . 将存储器位置0xA5中的最低有效4位复制到存储器位置0xA6中的最低有效4位 , 同时保持0xA6中的其他位不变。

d . 将存储器位置0xA5中的最低有效4位复制到存储器位置0xA5中的最高有效4位。 (因此 , 0xA5中的前4位将和后4位相同。)

*34 . 完成下列运算。

a . 111001

AND 101001

b . 000101

AND 101010

c . 001110

AND 010101

d . 111011

AND 110111

e . 111001

OR 101001

f. 010100

OR 101010

g . 000100

OR 010101

h. 101010

OR 110101

i . 111001

XOR 101001

j. 000111

XOR 101010

k . 010000

l. 111111

XOR 010101

XOR 110101

*35. 为了完成以下每个目标，确定所需的掩码和逻辑运算。

a. 将一个8位模式的高4位置1，并且不干扰其他位。

b. 对一个8位模式的最高有效位取反，并且不改变其他位。

c. 对一个8位模式取反。

d. 将一个8位模式的最低有效位置0，并且不干扰其他位。

e. 将一个8位模式除最高有效位外的所有位都置1，并且不干扰最高有效位。

f. 过滤掉一个RGB位图图像像素中的所有绿色部分，其中24位模式的中间8位存储的是绿色信息。

g. 反转一个24位RGB位图像素中的所有位。

h. 将一个24位RGB位图像素中的所有位设置为1，表示白色。

*36. 编写并测试几个小Python脚本，实现前一道题中的每个小问题。

*37. 确定一个逻辑运算（以及相应的掩码），使得在其用于一个8位的输入串时，当且仅当输入串为10000001时产生的输出为全0位串。

*38. 编写并测试一个小Python脚本，实现前一道题。

*39. 描述一组逻辑运算（以及它们相应的掩码），使得在其用于一个8位的输入串时，当这个输入串的开始和结尾都为1时产生的输出字节为全0；否则输出中至少应包含一个1。

*40 . 编写并测试一个小Python脚本，实现前一道题。

*41 . 对下列位模式执行循环左移4位后，结果如何？

a . 010101 b . 11110000 c . 001

d . 101000 e . 00001

*42 . 对下列用十六进制记数法表示的字节执行循环右移2位后，结果如何？（用十六进制记数法写出答案。）

a . 0x3F b . 0x0D

c . 0xFF d . 0x77

*43 . a . 在Vole机器语言中，哪条指令可以用于实现寄存器0xB循环右移5位？

b . 在Vole机器语言中，哪条指令可以用于实现寄存器0xB循环左移2位？

*44 . 用Vole编写一个程序，把地址为0x8C的存储单元的内容颠倒过来。（也就是说，从左向右读取时地址0x8C的最终位模式，应与最初从右向左读取时的原始模式一致。）

*45 . 用Vole编写一个程序，将地址0xA2中存储的值减去0xA1中存储的值，并将结果放入地址0xA0中。假定值用二进制补码记数法编码。

*46 . 高清视频可以以30帧每秒（fps）的速率传输，其中每一帧的分辨率为1920像素×1080像素，每像素使用24位。这种格式的无压缩视频流可以通过USB 1.1串行端口发送吗？USB 2.0串行端口呢？USB 3.0串行端口呢？（注

意：USB 1.1、USB 2.0、USB 3.0串行端口的最大速度分别是12 Mbit/s、480 Mbit/s和5 Gbit/s。)

*47．假设某人在键盘上每分钟能打40个单词。（假设一个单词以5个字符计。）如果计算机每微秒（百万分之一秒）执行500条指令，那么该计算机在此人打两个连续的字符之间能执行多少条指令？

*48．对于一个每分钟打40个单词的打字员，键盘每秒传输多少位才能跟得上？（假定每个字符用ASCII编码，每个单词由6个字符组成。）

*49．假设Vole与使用内存映射输入/输出技术的打印机通信，同时假设地址0xFF用于将字符发送给打印机，地址0xFE用于接收打印机的状态信息。特别地，假设地址0xFE的最低有效位用于指示该打印机是否准备好接收下一个字符（0表示“未就绪”，1表示“就绪”）。从地址0x00开始，编写一个机器语言例程，它等到打印机准备好接收下一个字符，然后把由寄存器0x5中的位模式表示的字符发送给打印机。

*50．用Vole编写一个程序，在地址从0xA0到0xC0的所有存储单元中放入0，但是它应该足够小以便能够放入地址从0x00到0x13的存储单元中。

*51．假设某机器的硬盘上有200 GB的可用存储空间，以15 Mbit/s的速率从宽带上接收数据。以这个速率，需要多久能存满可用的存储空间？

*52．假设某卫星系统正以250 Kbit/s的速率接收串行数据流。如果一个突发的大气干扰持续了6.96秒，那么有多少数据位会受到影响？

*53．假设给你32个处理器，每个处理器都能在百万分之一秒内能够完成两个多位数求和运算。描述如何使用并行处理技术实现在六百万分之一秒内完成64个数的求和运算。单独一个处理器完成相同的运算要花多少时间？

*54 . 概述CISC体系结构和RISC体系结构之间的区别。

*55 . 说出两种提高吞吐量的方法。

*56 . 对于计算一组数的平均值，说明用一台多处理器机器是如何算得比用一台单处理器机器快得多的？

*57 . 编写并测试一个Python脚本，读取一个圆的浮点半径，输出这个圆的周长和面积。

*58 . 编写并测试一个Python脚本，读取一个字符串和一个整数，输出将这个字符串重复给定的整数次后得到的结果。

*59 . 编写并测试一个Python脚本，读取一个直角三角形的两个浮点边长，输出斜边长度、周长和面积。

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答这些问题不是唯一的目的，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1 . 假设某计算机生产商开发了一种新的机器体系结构。应该允许该公司在多大程度上拥有该体系结构？什么样的政策对社会最好？

2 . 从某种意义上说，1923年是现今被许多人称为有计划淘汰（planned obsolescence）现象诞生的时间。这一年，由阿尔弗雷德·斯隆（Alfred Sloan）领导的通用汽车公司将汽车工业引向了型号概念的年代。其思想是通过改变风格，而不是必须推出更好的车来提高销售。引用斯隆的一句话：“我们希望

你对自己现在的车不满意，这样你就会购买新车。”如今，计算机行业在多大程度上使用了这种市场策略？

3．我们常常在想，计算机技术是如何改变我们的社会的。不过，许多人辩称，这门技术常常抑制改变的产生，它试图使老系统继续存在，甚至使其地位更加稳固。例如，如果没有计算机技术，中央政府在社会中的角色会继续存在吗？如果没有计算机技术，中央集权在今天能够达到什么程度？如果没有计算机技术，我们在多大程度上会更好或更坏？

4．如果某人认为自己不需要知道机器的任何内部细节，因为有其他人会建造它，维护它，并解决可能发生的问题，这种想法合乎道德吗？你的答案会因这个机器具体是计算机、汽车、核电厂还是烤面包机而不同吗？

5．假设一家厂商生产了一种计算机芯片，但是后来发现它设计上有一个缺陷。再假设该生产商在接下来的生产中修正了这个缺陷，但决定掩盖最初有缺陷这一事实，不回收已经售出的芯片，理由是：已经在用的芯片没有一个用在该缺陷会导致严重后果的应用中。有人会因为该生产商的决定受到伤害吗？如果没有人受到伤害，而且该决定避免了该生产商的资金流失或者可能避免了被迫裁员，那么该生产商的决定正确吗？

6．先进的技术是有助于治愈心脏病，还是会导致久坐的生活习惯进而导致心脏病？

7．很容易想象，由于溢出和截断问题而产生的算术误差可能会导致金融或导航方面的灾难。对于图像存储系统，由于丢失图像细节而产生的错误会有什么后果（也许在勘察或医疗诊断领域）？

8．ARM Holdings是一家为各种消费性电子设备设计处理器的小型公司。它并不制造任何处理器，而是将其设计授权给半导体厂商（如高通、三

星和德州仪器），这些厂商为生产出的每个部件支付特许权使用费。这种商业模式将计算机处理器的高研发成本分散到了整个消费电子市场。现在，95%以上的移动电话（不仅是智能手机）、40%以上的数码相机和25%的数字电视都在使用ARM处理器。此外，ARM处理器还用于平板电脑、音乐播放器、游戏控制器、电子书阅读器、导航系统等设备中。鉴于此，你认为该公司是垄断者吗？为什么是？为什么不是？因为消费设备在当今社会中扮演着越来越重要的角色，依赖这样一个鲜为人知的公司好吗？或者，是否会引起人们的担忧？

课外阅读

Carpinelli, J. D. *Computer Systems Organization and Architecture*. Boston, MA: Addison-Wesley, 2001.

Comer, D. E. *Essentials of Computer Architecture*, 2nd ed. Boca Raton, FL: Taylor & Francis, 2017.

Dandamudi, S P. *Guide to RISC Processors for Programmers and Engineers*. New York: Springer, 2005.

Furber, S. *ARM System-on-Chip Architecture*, 2nd ed. Boston, MA: Addison-Wesley, 2000.

Hamacher, V. C., Z. G. Vranesic, and S. G. Zaky. *Computer Organization*, 5th ed. New York: McGraw-Hill, 2002.

Knuth, D. E. *The Art of Computer Programming*, Vol. 1, 3rd ed. Boston, MA: Addison-Wesley, 1998.

Murdocca, M. J. and V. P. Heuring. *Computer Architecture and Organization: An Integrated Approach*. New York: Wiley, 2007.

Stallings, W. *Computer Organization and Architecture*, 10th ed. Harlow, Essex, England: Pearson, 2016.

Tanenbaum, A. S. *Structured Computer Organization*, 6th ed. Upper Saddle River, NJ: Prentice-Hall, 2012.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第3章 操作系统

本章我们研究操作系统。操作系统是用来协调计算机的内部活动以及检查计算机与外部世界通信的软件包。计算机的操作系统能将计算机硬件转化为有用的工具，我们的目标就是要理解操作系统做哪些工作，以及它们是如何完成这些工作的。要成为有知识的计算机使用者，这样的背景是极为重要的。

本章内容

3.1 操作系统的历史

3.2 操作系统的体系结构

3.3 协调机器的活动

*3.4 处理进程间的竞争

3.5 安全性

持久理解和学习成果

操作系统是控制计算机整体运行的软件。

学习成果 解释操作系统是如何从最早的批处理系统发展为复杂的多任务环境的。

学习成果 识别常用操作系统的主要组件。

操作系统执行用户程序、协调活动并管理系统资源。

学习成果 识别现代计算机上的各种应用和操作系统软件的分类型。

学习成果 解释操作系统的**进程**概念，以及当一个系统包含多个竞争资源的进程时可能出现的问题。

学习成果 确定发生**死锁**需要的条件。

网络安全是操作系统和操作系统软件的一个重要关注点。

学习成果 确定现有的网络安全问题和旨在防范这些问题的现代操作系统的安全功能。

操作系统（operating system）是控制计算机整体运行的软件。它为用户提供可以存储和检索文件的方法、可以请求执行程序的接口，以及执行所请求程序所需的环境。

操作系统最著名的例子可能要数Windows了，微软公司发布了很多版本，并广泛用于PC领域。另一个被广泛认可的例子是UNIX，它是较大的计算机系统和PC的流行选择。事实上，UNIX是其他两个流行操作系统macOS和Solaris的核心。其中，macOS是苹果公司为其一系列Mac机提供的一种操作系统；Solaris是Sun Microsystems（现归

Oracle所有)开发的。另外,还有能够运用于大型机和小型机的Linux操作系统,该系统最初是由一些计算机爱好者以非营利的目的开发的,到目前为止,包括IBM公司在内的许多商业机构都发布了Linux操作系统。

对于非专业的计算机用户,操作系统之间的差异在很大程度上是表面性的;而对于计算机专业人员,不同的操作系统可能意味着使用完全不同的工具或在传播和维护工作中遵循完全不同的理念。然而,所有主流操作系统的核心都是解决计算机专家在半个多世纪之前就已经遇到的那些问题。

3.1 操作系统的历史

今天的操作系统经过长期的演变已经成为大而复杂的软件包。20世纪40~50年代的计算机不是很灵活,效率也不高。一台机器会占据整个房间。程序执行需要大量的设备准备工作,如安装磁带、把穿孔卡片放在卡片读入机上、设置开关等。每个程序的执行称为一个**作业**(job),它是作为一个独立的活动处理的——为执行该程序准备好机器,执行程序,然后在下一个程序的准备工作开始之前,必须重新获取磁带、穿孔卡片等所有一切。当几个用户需要共享一台机器时,操作系统提供签名表,以便各个用户能够预订到一段机器时间。在分配给某个用户的时间段内,机器就完全处于该用户的控制之下。这段时间通常是从程序的准备开始,接下来是短时间的程序执行过程。一个

用户本可以在很短的时间内尽可能多做一件事情（“它只需要一分钟”），但下一个用户已经迫不及待地要使用机器做准备工作了。

在这样的环境下，操作系统开始作为一个系统致力于简化程序的准备工作，提高作业之间的过渡效率。操作系统早期的开发是用户与设备的分离，用以避免人员进出计算机机房，为此雇用了计算机操作员来操作机器。任何人想要运行程序都必须把程序、所需的数据以及有关程序需求的特别说明提交给操作员，稍后再由操作员来取结果。操作员所做的工作就是把这些资料输入机器的海量存储器，然后由称为操作系统的程序从那里一次一个地读入并执行它们。这就是**批处理**（batch processing）的开始——把若干个要执行的作业收集到一个批次中，然后在不需要与用户进行进一步的交互的情况下执行它们。

在批处理系统中，驻留在海量存储器中的作业在**作业队列**（job queue）里等待执行（见图3-1）。**队列**（queue）是一种存储组织，其中对象（这里指作业）按照**先进先出**（first-in, first-out, FIFO，读作“FI-foe”）的方式在队列里排队。也就是说，对象的出列顺序和入列顺序一致。实际上，大多数作业队列不是严格遵循FIFO结构的，主要是因为大多数操作系统都考虑了作业的优先级，结果就造成了在队列中等待的作业可能会被优先级更高的作业挤掉。

在早期的批处理系统中，每个作业都伴随着一组指令，用来说明为这个特定的作业准备机器所需的步骤。这些指令使用一个称为作业控制语言（job control language, JCL）的系统进行编码，与作业一起存储在作业队列里。当一个作业被选中执行时，操作系统在打印机上

打印这些指令以便计算机操作员阅读和遵照执行。现在，操作系统与计算机操作员之间的这种通信仍然存在，如报告“网络不可用”和“打印机无响应”之类的错误的PC操作系统见证了这一点。

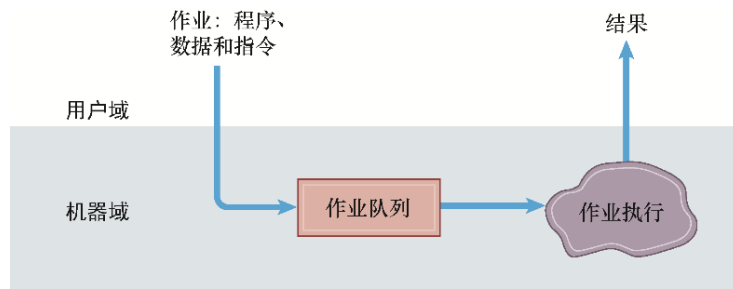


图3-1 批处理

用计算机操作员作为计算机及其用户之间的中介的一个主要缺点是：作业一旦提交给操作员，用户就无法与作业交互了。这种方法对于某些应用是可以接受的，如工资表的处理，因为在这里，数据与所有的处理决策都是事先建立的。然而，当在一个程序的执行期间，用户必须与该程序进行交互时，这种方法就无法让人接受了。例如，在预订系统中，预订和取消操作必须在它们发生时就报告；在字处理系统中，文档是以动态的写入和重写方式开发的；在计算机游戏中，与机器的交互是游戏的核心特征。

为了适应这些需求，人们开发了新的操作系统，它们允许执行一个程序来通过远程终端与用户对话——这种特性称为**交互式处理**（interactive processing）（见图3-2）。（终端只不过是一台电子打字机，用户能够通过它键入输入并读取打印在纸上的计算机响应。虽然今天的PC、笔记本电脑和智能手机比老式的计算机和终端强大得

多，具有讽刺意味的是，我们仍然频繁地使用它们来与更强大的因特网云服务进行交互。）

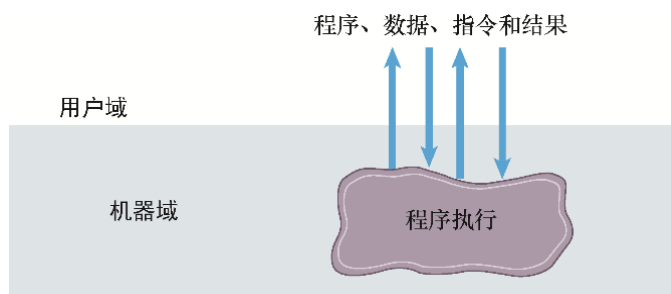


图3-2 交互式处理

成功的交互式处理的最重要之处在于，计算机的动作能够足够快速地协调用户的需求，而不是强迫用户遵守机器的时间表。（在进行工资表的处理时，计算机能够根据所需的时间量调度得很好，但是在使用字处理程序时，如果机器不能及时对键入的字符做出响应，用户会很沮丧。）从某种意义上说，计算机是被强制在一个限期内执行任务的，这一过程就是众所周知的**实时处理**（real-time processing），其中所完成的动作被称为是实时发生的。也就是说，要是说一台计算机实时完成任务的，意思就是，计算机按照其（外部现实世界的）环境的最后期限完成任务。

如果要求交互式系统一次只服务于一个用户，那么实时处理就不存在问题了。但是20世纪六七十年代的计算机比较昂贵，所以每台机器不得不服务于多个用户。因此，工作在远程终端的若干个用户在同一时间寻求一台机器的交互式服务，从而导致实时交互障碍的现象很常见。如果操作系统对于多用户环境仍然坚持一次只执行一个作业，那么将只有一个用户能得到满意的实时服务。

针对这个问题的解决方案就是设计能同时给多个用户提供服务的操作系统，这一特性称为**分时**（time-sharing）。实现分时的一种方式应用称为**多道程序设计**（multiprogramming）的技术，其中时间被分成时间片，然后每个作业的执行被限制为每次仅一个时间片。在每个时间片结束时，当前的作业暂时搁置，让另一个作业在下一个时间片里执行。通过这种方式可以快速地在各个作业之间进行切换，形成若干个作业同时执行的错觉。依据所执行的作业的类型，早期的分时系统能够同时为多达30个用户提供可接受的实时服务。今天，多道程序设计既可用于单用户系统，也可以用于多用户系统，前者通常称为**多任务处理**（multitasking）。也就是说，分时指的是多个用户共享对同一台计算机的访问，而多任务处理指的是一个用户同时执行多个任务。

随着多用户分时操作系统的发展，典型计算机装置被配置为连接到许多工作站的大型中央计算机。通过这些工作站，用户能够从机房外面直接与计算机进行通信，而不用把请求递交给计算机操作员。通常，常用的程序会被存储在机器的海量存储设备上，然后将操作系统设计成按工作站的请求执行这些程序。这样，作为计算机与用户的中间媒介的计算机操作员的作用就不那么明显了。

今天，计算机操作员这个角色已基本不存在了，特别是在个人计算机领域，计算机用户已经能够承担操作计算机的所有职责。即使是最大型的计算机装置，其运行也基本上无须人工参与。事实上，传统的计算机操作员已经让位于系统管理员，系统管理员管理计算机系统

——获得和监督计算机新设备和新软件的安装，实施一些本地的规则（如建立新账号、为不同的用户划分一定的海量存储空间），协调努力解决系统中出现的问题，而不是以手工的方式操作机器。

简而言之，操作系统已经从一次获取和执行一个程序的简单程序，发展为能够协调分时，能够维护机器海量存储设备上的程序和数据文件，也能够直接响应计算机用户请求的复杂系统。

但是，计算机操作系统的发展仍在继续。多处理器机器的发展已经通过把不同的任务分配给不同的处理器进行处理，并且采用分时机制共享单个处理器，让操作系统拥有了分时/多任务处理能力。这些操作系统必须处理**负载均衡**（load balancing，把任务动态地分配给各个处理器，以便所有处理器都得到有效的利用）和**均分**（scaling，把一个任务划分为若干个子任务，其数量与可用的处理器数量相一致）问题。

此外，计算机网络（相距很远的大量机器连接在一起）的出现导致了协调网络活动的软件系统的诞生。计算机网络（我们将在第4章学习）领域在许多方面拓展了操作系统这个学科，其目标是跨多个用户和多个机器（而非单个的、孤立的计算机）管理资源。

操作系统的另一个研究方向聚焦在专用于特定任务的设备，如医疗设备、车载电子设备、家用电器、手机或其他手持计算机。这些设备中的计算机系统称为**嵌入式系统**（embedded system）。嵌入式操作系统经常需要节省电池电量、满足严格的实时期限，或在只有很少或

完全没有人为监管的情况下连续工作。在这项努力中，有代表性的成功系统有Wind River Systems公司开发的曾用于勇气号（ Spirit ）和机遇号（ Opportunity ）火星探测器的VxWORKS、微软公司开发的Windows CE（也就是众所周知的Windows Embedded Compact）、黑莓公司专为手持设备和车辆开发的QNX。

智能手机有什么功能

手机的功能越来越强大，除了能够简单处理语音通话还能提供其他服务。现在，常见的**智能手机**（ smartphone ）可以用于编写文本信息、浏览万维网、提供导航、查看多媒体内容等，简而言之，它可以提供许多传统的个人计算机所能提供的服务。因此，智能手机需要成熟的操作系统来管理有限的智能手机硬件资源，并提供一些支持迅速增加的智能手机应用软件的功能。智能手机操作系统市场主导地位的争夺战将日趋激烈，而争夺的焦点很可能在于哪个操作系统可以以最优的价格提供最有创意的功能。智能手机操作系统领域中的竞争者包括苹果的iOS、黑莓公司的BlackBerry OS、微软的Windows Phone和谷歌的Android。

3.1节问题与练习

1．举出几个队列的例子。对于每一种情况，请指出任何可能破坏FIFO结构的情况。

2．下列活动中哪些需要用到实时处理？

a. 打印邮件标签。

- b. 玩计算机游戏。
- c. 拨号时在智能手机屏幕上显示所拨号码。
- d. 执行一个预报下一年经济状况的程序。
- e. 播放MP3录音。

3. 嵌入式系统与PC的区别是什么？

4. 分时与多任务处理的区别是什么？

3.2 操作系统的体系结构

为了理解典型操作系统的组成，这里我们首先考虑在典型的计算机系统中能找到的所有软件，然后再专门研究操作系统本身。

3.2.1 软件概述

我们通过提出一个软件分类方案来考察在一个典型计算机系统软件。这种分类方案总是把一些相似的软件单元放在不同的类里，其方法如同时区的划分，时区的划分规定相邻时区的时钟必须设置为相差一小时，即使其日出与日落的时间没有明显的差别。此外，在软件分类的情况下，学科的发展变化和某种权威的缺乏，导致出现了一些矛盾的术语。例如，微软公司Windows操作系统的用户会发现，称为“附件”和“管理工具”的程序组既包括应用类软件又包括实用类软

件。因此，下面的分类方法应该被看作在广泛的、动态的学科里占有一席之地的方法，而不是对人们普遍接受的事实的一种表述。

先把机器软件分为两大类：**应用软件**（ application software ）和**系统软件**（ system software ）（见图3-3）。应用软件是由一些完成机器特定任务的程序组成的。一台用来维护某个制造公司库存单的机器所包含的应用软件与电气工程师用的机器里的应用软件是不同的。应用软件的例子有电子表格软件、数据库系统、桌面出版系统、记账系统、程序开发软件以及游戏等。

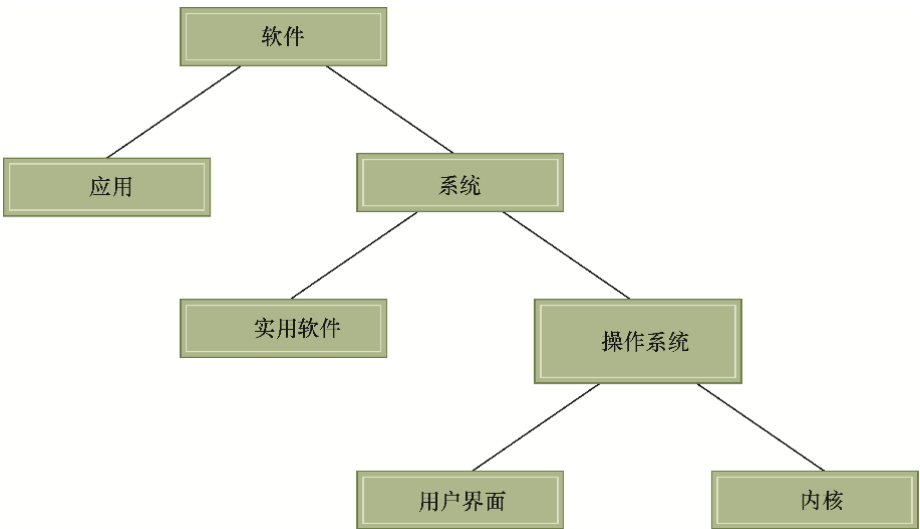


图3-3 软件分类

相对于应用软件而言，系统软件完成的是一般的计算机系统都需要完成的任务。从某种意义上来说，系统软件提供了应用软件所需的基础设施，这和国家基础设施（政府、道路、公共设施、金融机构等）提供公民维系各自生活方式的基础的方式大致相同。

系统软件又可分两类，一类是操作系统本身，另一类是统称为**实用软件**（utility software）的软件单元。安装的大多数实用软件包括这样一些程序，它们实现的活动只是计算机安装的基础，但没有包含在操作系统中。从某种意义上说，实用软件是由一些能够扩充（或定制）操作系统功能的软件单元组成的。例如，格式化磁盘或将文件从磁盘复制到闪存驱动器中去的能力一般都是借助于实用程序来实现的，而不是在操作系统内部实现的。其他的实用软件的例子包括压缩数据和解压缩数据的软件、播放多媒体内容的软件以及处理网络通信的软件。

把某些活动实现为实用软件，比把它们放在操作系统中，更容易按特定安装需求定制系统软件。事实上，公司或个人对机器操作系统原先提供的实用软件进行修改和扩充是很常见的。

遗憾的是，应用软件与实用软件之间的差别已经很模糊了。从我们的观点来看，它们的差别在于其是否是计算机“软件架构”的一部分。因此，如果一个新应用变成一种基础工具，它就很可能成为一种实用软件。当用于因特网通信的软件还在研究阶段时，它被认为是一种应用软件；如今，这类工具对大部分计算机使用是非常基础的，因此被归类为实用软件。

实用软件和操作系统之间的差别同样是模糊的。特别是，美国和欧洲的反垄断诉讼案争论的都是这样的问题：像浏览器和媒体播放器这样的软件单元是微软公司操作系统的一部分，还是微软公司用来压制竞争对手的实用软件。

Linux

对计算机爱好者而言，如果想通过亲手实验来了解一个操作系统，那么就应该选择Linux。Linux操作系统最初是由Linus Torvalds在赫尔辛基大学学习期间设计的。Linux操作系统是一个非专有产品，我们可以免费获得它的源代码（见第6章）和相关文档。因为可以免费获得其源代码，所以该系统在计算机爱好者、学习操作系统的学生和程序员中非常流行。而且，Linux操作系统被认为是当今可用的较可靠的操作系统之一。正因为这个原因，一些公司开始以更实用的形式包装和销售Linux操作系统产品，现在这些产品开始向市场上长期被认可的商用操作系统产品发出了挑战。

3.2.2 操作系统组件

现在，我们把注意力集中在操作系统领域内的组件上。为了完成计算机用户请求的动作，操作系统必须能够与这些用户进行通信。操作系统负责处理这种通信的部分通常称为**用户界面**（user interface）。老式的用户界面称为**外壳**（shell），通过键盘和显示屏用文本信息与用户通信。更现代化的系统利用**图形用户界面**（graphical user interface，GUI）实现与用户的通信，其中操作的对象（如文件和程序）被表示为显示屏上的图标（icon）。这些系统允许用户使用某种常用的输入设备发出命令。例如，有一个或多个按键的计算机鼠标可用来单击或拖曳屏幕上的图标。另外，平面设计师常使用专用的点击设备（pointing device）或手写笔（stylus）代替鼠标来操作图标，或者

在某些类型的手持设备上这样操作图标。最近，高密度触摸屏的进步使得用户可以直接用手指操作图标。而如今的GUI使用二维图像投影系统，三维界面允许人类用户通过3D投影系统、触觉感知设备和环绕声音频再生系统与计算机进行通信，这些都是当前研究的课题。

虽然操作系统的用户界面在实现机器的功能上扮演了重要的角色，但它仅仅是计算机用户与操作系统真实内核之间的一个接口而已（见图3-4）。用户界面与操作系统内部之间的区别主要表现在：一些操作系统允许用户从各种界面中选择最合适的界面为自己服务。例如，UNIX操作系统的用户就可以选择不同的shell，包括Bourne shell、C shell和Korn shell，以及称为X11的GUI。最早版本的Microsoft Windows是一个GUI应用程序，可以通过MS-DOS操作系统的shell命令加载。在最新版的Windows中，人们仍可看见作为实用程序存在的DOS shell cmd.exe，但非专业用户几乎完全不需要使用这一界面。类似地，苹果公司的OS X也保留了一个Terminal实用shell，它承袭了系统的UNIX shell。

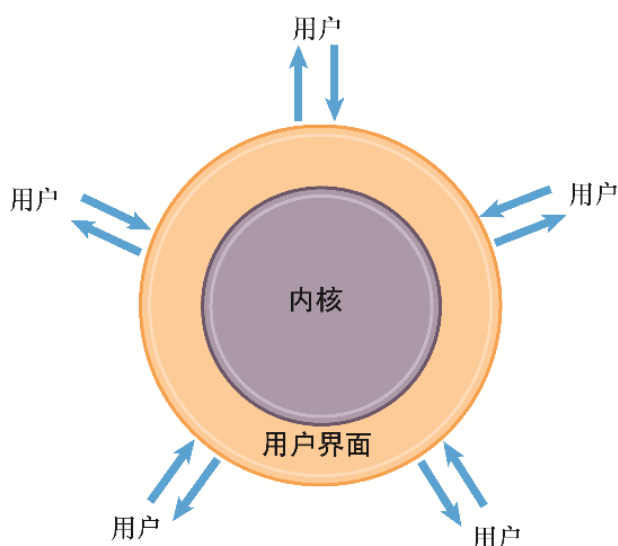


图3-4 用户界面是用户和操作系统内核
之间的中介

现在的GUI shell中的重要组件是**窗口管理程序**（window manager），该程序在屏幕上分配若干个称为窗口的块，跟踪与每个窗口相联系的应用程序。当一个应用程序想在屏幕上显示图像时，它就会通知窗口管理程序，窗口管理程序就会把所需的图像放在分配给该应用程序的窗口里。然后，当鼠标按键被单击时，窗口管理程序计算鼠标在屏幕上的位置，并把这个鼠标动作通知给相应的应用程序。窗口管理程序负责生成GUI“样式”，大多数管理程序会提供一系列配置选项。Linux用户甚至可以选择窗口管理程序，常用的选项包括KDE和Gnome。

与操作系统的用户界面相对，我们把操作系统的内部部分称为**内核**（kernel）。操作系统的内核包含一些完成计算机安装所需的极基本功能的软件组件。其中一个组件是**文件管理程序**（file manager），它的工作是协调机器海量存储器设施的使用。更准确地说，文件管理程序维护着存储在海量存储器上的所有文件的记录，包括每个文件的位置、哪些用户有权访问各种文件以及海量存储器里的哪些部分可以用来建立新文件或扩充现有文件。这些记录被存放在单独的包含相关文件的存储介质中，这样，每次存储介质联机时，文件管理程序就能够检索相关的文件，进而就能知道特定的存储介质中存放的是什么。

为了方便机器用户，大多数文件管理程序都允许把若干个文件组织在一起，放在**目录**（directory）或**文件夹**（folder）里。这种方法允

许用户将自己的文件依据用途划分，把相关的文件放在同一个目录里。一个目录可以包含称为子目录的其他目录，这样就可以构建层次化的目录结构。例如，用户可以创建一个名为MyRecords的目录，它又包含了3个名为FinancialRecords、MedicalRecords和HouseHoldRecords的子目录。每个子目录中都会有属于该类别的文件。（Windows操作系统的用户可通过执行实用程序“Windows资源管理器”，让文件管理程序显示当前的目录结构。）

一条由目录内的目录组成的链称为**目录路径**（directory path）。路径通常是这样表示的：列出沿该路径的目录，然后用斜杠分隔它们。例如，路径animals/prehistoric/dinosaurs表示的是：该路径是从名为animals的目录开始的，经过名为prehistoric的子目录，终止于名为dinosaurs的子目录。（对Windows用户而言，目录路径是用反斜杠表示的，如animals\prehistoric\dinosaurs。）

其他软件单元能否访问某个文件，由文件管理程序来决定。该访问过程先通过一个称为打开文件的过程来请求文件管理程序授权访问该文件。如果文件管理程序批准了该访问请求，那么它就会提供查找和操控该文件所需的信息。

内核的另一个组件是一组**设备驱动程序**（device driver），它们负责与控制器（有时直接与外围设备）通信，以操作连接到机器的外围设备的软件单元。每个设备驱动程序都是专门为特定类型的设备（如

打印机、磁盘驱动器或显示器)设计的,它把通用请求翻译为这种设备(分配给这个驱动程序的设备)所需要的更富技术性的步骤。例如,打印机的设备驱动程序包含的软件能够读取和解码特定打印机的状态字,而且还能够处理其他一些信息交换的细节。这样,其他软件组件就不必为打印一个文件而处理那些技术细节了,相反,只需运用设备驱动程序软件完成打印文件的任务,并把技术细节交由设备驱动程序处理。按照这种方式,其他软件单元的设计就可以独立于具体设备特有的特征。这样做的结果是,我们只要为一个通用操作系统安装合适的设备驱动程序,它就能够使用特定的外围设备。

在操作系统的内核中还有一个组件就是**内存管理程序**(memory manager),它担负着协调机器使用主存储器的任务。在计算机一次仅执行一个任务的环境中,这种职责是最小的。这些情况下,执行当前任务的程序会被放在主存储器中已经定义好的位置上执行,然后被执行下一个任务的程序替换。然而,在要求计算机同时处理多个需求的多用户或多任务处理环境下,内存管理程序的职责就扩大了。在这些情况下,许多程序和数据块必须同时驻留在内存里。因此,内存管理程序必须为这些需求寻找并分配内存空间,并且要保证每个程序只能限制在程序所分配的内存空间内运行。此外,随着不同活动的需求进出内存,内存管理程序必须能跟踪那些不再被占用的内存区域。

当所需的总主存储器空间超过该计算机实际所能提供的可用内存空间时,内存管理程序的任务会更复杂。在这种情况下,内存管理程序会在主存储器与海量存储器之间来回切换程序和数据[这种技术称

为**页面调度**（paging）]，从而造成有额外内存空间的假象。例如，假设需要一个8 GB大小的主存储器，但是计算机所能提供的只有4 GB。为了造成具有更大内存空间的错觉，内存管理程序在磁盘上预留4 GB的存储空间。在这块存储区域里，将记录主存储器有8 GB的实际容量时本应存储在主存储器中的位模式。这个数据被分成大小一致的称为**页面**（page）的存储单元，典型的页面一般是几KB大小。于是，内存管理程序就在主存储器和海量存储器之间来回切换这些页面，以便在任何给定的时间，我们所需的页面都会实际出现在4 GB的主存储器之中，最后的结果是计算机能够像确实拥有8 GB的主存储器一样工作。这块由页面调度所产生的大的“虚构”内存空间被称作**虚拟内存**（virtual memory）。

另外，在操作系统内核中还有**调度程序**（scheduler）和**分派程序**（dispatcher）这两个组件，我们将在下一节介绍。在此，我们只需注意，在多道程序设计系统中调度程序决定哪些活动是要被考虑执行的，而分派程序控制这些活动的时间分配。

3.2.3 系统启动

我们已经看到，操作系统提供了其他软件单元所需的软件基础设施，但是我们还没有细想操作系统本身是如何启动的。启动是通过**引导**（boot strapping，简称为booting）过程实现的，这个过程是由计算机在每次启动的时候完成的。正是这个过程把操作系统从海量存储器（操作系统永久存储的地方）传送到主存储器（在机器首次开机时，

主存储器实际上是空的)中。为了理解启动过程和必须有启动过程的原因,我们先来考察机器的CPU。

CPU的设计使得每次CPU启动时,它的程序计数器都从事先确定的特定地址开始。CPU就期望能在这个地址上找到程序要执行的第一条指令。那么,从概念上讲,只需在这个地址上存储操作系统。然而,从技术上讲,计算机的主存储器通常是采用易失性技术制造的,这意味着,当计算机关闭时,存储在内存上的数据会丢失。因此,在每次重启计算机的时候,必须重新填充主存储器的内容。

简言之,当计算机首次开机时,我们需要将一个程序(最好是操作系统)放在主存储器中,但是每次关机时,计算机的易失性存储器都会被擦除。为了解决这个难题,计算机的主存储器(这是CPU期望找到初始程序的地方)的一小部分是由特殊的非易失性存储单元构成的。由于这种存储器的内容可以读取,但不可以改变,因而被称为**只读存储器**(read-only memory, ROM)。打个比方,虽然所使用的技术是更先进的,但我们可以把在ROM中存储位模式想象成熔断微小的保险丝(熔断的表示1,未熔断的表示0)。更确切地说,如今个人计算机中大多数的ROM是用闪存技术构建的(即不是严格意义上的ROM,因为它可以在特定情况下被改变)。

在通用计算机中,**引导装入程序**(boot loader)被永久存储在机器的ROM中。这样,在计算机开机的时候将最先执行这个程序。引导装入程序中的指令引导CPU把操作系统从海量存储器中预先定义的位置调入主存储器的易失性存储区(见图3-5)。现代的引导装入程序可以

从各种位置将操作系统复制到主存储器中。例如，在嵌入式系统（如智能手机）中，操作系统是从特殊的闪速（非易失性）存储器复制的；在大型公司或大学的小型工作站上，可能要通过网络从远程机器上复制操作系统。一旦操作系统被放入主存储器，引导装入程序就引导CPU执行跳转指令，转到这个存储区。这时，操作系统接管并开始控制机器的活动。执行引导装入程序从而启动操作系统的整个过程称作**引导**（booting）计算机。

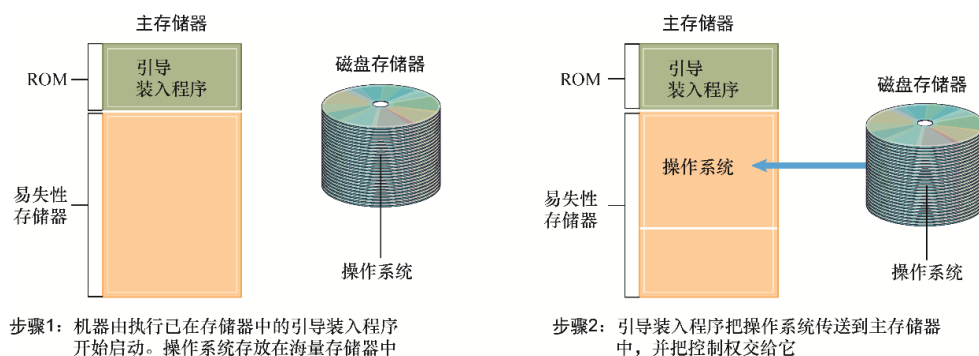


图3-5 引导过程

固 件

除了引导装入程序，PC的只读存储器还包括一组软件例程，用于实现基本的输入/输出活动，如从键盘上接收信息、把信息显示在计算机的屏幕上，以及从海量存储器中读取数据。因为这些例程存储在非易失性存储器（如FlashROM）中，所以它们不会永久地蚀刻到机器的硅片（硬件）中，但也不像海量存储器中的其他程序（软件）那样容易更改。人们创造了**固件**（firmware）这个术语来描述这一“中间地带”。固件例程可以被引导装入程序用来在操作系统开始工作之前完成I/O活动。例如，它们可用于在引导过程真正开始之前，与计算机用户

进行通信，并在引导期间报告错误。得到广泛使用的固件系统包括PC中一直使用的基本输入/输出系统（basic input/output system，BIOS）、较新的可扩展固件接口（extensible firmware interface，EFI）、Sun公司的Open Firmware（现为Oracle的一个产品），以及用于许多嵌入式设备的通用固件环境（common firmware environment，CFE）。

你也许会问，为什么台式计算机不提供足够的ROM来装载整个操作系统呢，这样就不必从海量存储器来引导启动了。虽然对使用小型操作系统的嵌入式系统而言这是可行的，但就当今的技术而言，把通用计算机的大块主存储器专用于非易失性的存储，效率就不高了。此外，计算机操作系统要频繁更新，以确保安全性并与改良了最新硬件的新设备驱动程序同步。虽然也有可能去更新存储在ROM中的操作系统和引导装入程序——通常称为**固件更新**（firmware update），但技术上的限制使海量存储器成了较传统的计算机系统的最常见的选择。

最后，我们要指出，理解引导过程与操作系统、实用软件和应用软件之间的区别，能帮助我们更好地领会大多数通用计算机操作系统运行的总体方法。当这样的机器第一次开机时，引导装入程序会装入并激活操作系统，然后用户向操作系统提出请求，执行实用程序或应用程序。当实用程序或应用程序终止时，用户需要再次与操作系统联系，此时用户可以再提出其他请求。因此，学习使用这样的系统是一个双层过程，除了学习指定的实用程序或期望的应用程序的细节，还

必须学习足够多的关于机器操作系统的知识，这样才能游刃有余地切换应用程序。

3.2节问题与练习

- 1．列出典型操作系统的组件，并分别用一句话概括每个组件的作用。
- 2．应用软件与实用软件之间的区别是什么？
- 3．什么是虚拟存储器？
- 4．概述引导过程。

3.3 协调机器的活动

本节我们讨论操作系统如何协调应用软件、实用软件以及操作系统自身内部软件单元的执行。首先，从进程的概念开始。

3.3.1 进程的概念

现代操作系统的一个最基本的概念是程序与执行程序的活动之间的区别。前者是一组静态的指令，而后者是一个动态的活动，其属性会随着时间的推进而改变。（我们可以把程序想象成插在书架上书里的一页乐谱，而把活动想象成用行动将这个乐谱描述的音乐演奏出来的音乐家。）在操作系统的控制下执行某个程序的活动称为**进程**（process）。与进程联系在一起的是活动的当前状态，称为**进程状态**

（process state）。这个状态包含正在执行的程序的当前位置（程序计数器的值），以及其他CPU寄存器和相关存储单元中的值。大约说来，进程状态就是机器在特定时刻的快照。在程序执行期间的不同时刻（一个进程中的不同时刻），将观察到不同的快照（不同的进程状态）。

与通常一次只尝试演奏一部音乐作品的音乐家不同，典型的分时/多任务处理计算机通常会有许多进程同时运行，并且所有进程都竞争计算机资源。操作系统的任务就是管理这些进程，使每个进程都有其需要的资源（外围设备、主存储器的空间、对文件的访问以及对CPU的访问），确保独立的进程不会相互干扰，确保需要交换信息的进程能够交换信息。

3.3.2 进程管理

与协调进程的执行有关的任务是由操作系统内核中的调度程序和分派程序处理的。调度程序维护一个有关计算机系统中现存进程的记录（也就是进程池），将新进程加到该进程池中，并把已经完成的进程移出进程池。这样，当用户请求执行一个应用时，调度程序就把这个应用加到当前进程池加以执行。

为了跟踪所有的进程，调度程序在主存储器中维护着一个称为**进程表**（process table）的信息块。每当要请求程序执行时，调度程序都在进程表中为该程序创建一个新的表项。这个表项包含分配给该进程

的存储区域（从内存管理程序得到的）、进程的优先级以及该进程是处于就绪状态还是处于等待状态这样的信息。如果进程能够继续执行，那么该进程就处于**就绪**（ready）状态；如果进程因要等待某个外部事件（如完成海量存储操作、按下键盘上的一个键或者等待其他进程传来的消息）的发生而延迟，那么该进程就处于**等待**（waiting）状态。

分派程序是内核的一个组件，它监督被调度的进程能实际执行。在分时/多任务处理系统中，这个任务由**多道程序设计**（multiprogramming）来完成的；也就是说，先将时间划分为小的时间段，每段称为一个**时间片**（time slice），通常用毫秒（ms）或微秒（ μs ）来计量，然后切换CPU的注意力，允许每个进程一次执行一个时间片（见图3-6）。这种从一个进程变换到另一个进程的过程称为**进程切换**（process switch）或**上下文切换**（context switch）。

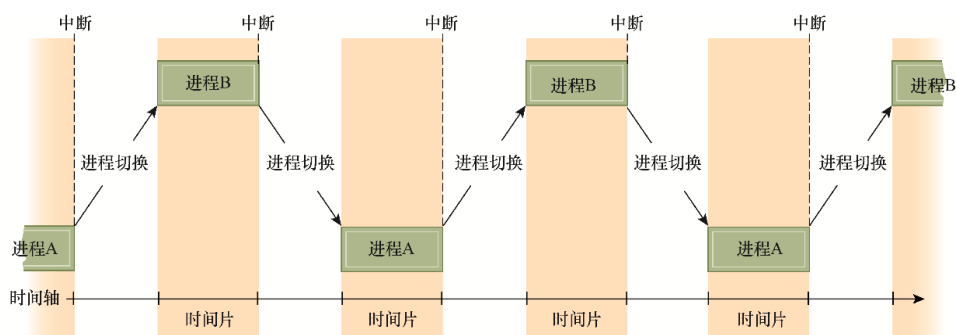


图3-6 进程A与进程B之间的多道程序设计

每次分派程序给进程分配一个时间片时，都会初始化一个计时器电路，通过产生一个称为**中断**（interrupt）的信号来指示时间片的结束。CPU对这个中断信号的反应，与任务被中断时你的反应大致相

同。你被中断时，会停止当时正在做的事情，记录当时任务进展的位置（以便以后能够返回到这个位置），然后处理中断事件。当CPU收到中断信号时，它会完成当前的机器周期，保存它在当前进程中的位置，然后开始执行一个称为**中断处理程序**（interrupt handler）的程序，该程序存储在主存储器中预先定义的位置上。中断处理程序是分派程序的一部分，用来描述分派程序如何响应中断信号。

中 断

中断是用来终止时间片的，正如文中描述的那样，这只是计算机中断系统中众多应用中的一个。有许多可以产生中断信号的环境，每个都有自己的中断例程。事实上，中断为协调计算机的活动与相关环境提供了一个重要的工具。例如，单击鼠标和按下键盘上的一个键都能产生中断信号，使CPU放下正在处理的工作，转而去解决中断。

为了管理识别和响应中断的任务，不同的中断信号被赋予了不同的优先级，以便较重要的任务能够得到优先处理。最高级别的中断通常与电源故障有关，如果计算机电源意外中断就会产生这样的中断信号。相关的中断例程会赶在电压降到不能再进行操作前的几毫秒时间内，引导CPU完成一系列的“内务”琐事。

因此，中断信号的作用就是抢占当前进程，将控制权传回分派程序。此时，分派程序从进程表的就绪进程中选择优先级最高的进程（由调度程序确定），重启计时器电路，让所选的进程开始它的时间片。

多道程序设计系统能够成功的最大关键是可以停止并在稍后重启一个进程。如果你在读书的时候被打断了，那么你能否在稍后继续阅读就取决于你是否记得中断时读到的位置，以及那个位置之前的信息。简而言之，你必须能够重新建立起中断前的环境。

在一个进程的情况下，必须重新建立的环境就是该进程的状态，前面提到过，这个状态包括程序计数器的值以及寄存器和相关存储单元的内容。为多道程序设计系统开发的CPU中，保存这些信息的任务是CPU对中断信号反应的一部分。这类CPU还提供机器语言指令，以重新装入先前保存的状态。这种特性简化了分派程序执行进程切换时的任务，也例证了现代CPU的设计是如何受当今操作系统的需求影响的。

最后，我们应当注意到，多道程序设计的使用提高了机器的总体效率。这有点儿违反直觉，因为多道程序设计对进程的来回切换会产生开销。但是，如果没有多道程序设计，每个进程都要运行到完成才能开始下一个进程，就意味着进程等待外围设备来完成任务的时间，或者等待用户发出下一个请求的时间被浪费了。多道程序设计技术可以把这些浪费的时间给另一个进程。例如，如果一个进程执行一个I/O请求，如从磁盘提出数据的请求，那么调度程序就会更新进程表来反映出这个进程正在等待外部事件。结果是，分派程序将不再给该进程分配时间片。之后（也许是几百毫秒），当这个I/O请求完成时，调度程序将会更新进程表来显示该进程处于就绪状态，这样这个进程就可以重新竞争时间片了。简而言之，当I/O请求正在执行时，其他任务就

会被执行，因此完成一组任务所花的时间要比按照顺序方式执行所花的时间少。

3.3节问题与练习

1. 概述程序和进程的差别。
 2. 概述在中断出现时，CPU要完成哪些步骤。
 3. 在多道程序设计系统中，如何能使高优先级的进程运行得比其他进程快？
 4. 在一个多道程序设计系统里，如果每个时间片是50 ms，每次上下文切换所花费的时间最多是1 μ s，那么机器在1 s内能够服务多少个进程？
 5. 在第4题中，如果每个进程都完全使用了它的时间片，那么实际花费在进程执行上的时间占整个机器时间的比例是多少？如果每个进程在它的时间片后的1 μ s执行I/O请求，那么这个比例又是多少？
-

*3.4 处理进程间的竞争

操作系统的一个重要任务就是将机器的各种资源分配给系统中的各个进程。从广义上讲，我们所用的**资源**（resource）这个术语，不仅包括机器的外围设备，还包括机器本身的特性。文件管理程序分配对文件的访问，并为新文件的建立分配海量存储空间；内存管理程序分配内存空间；调度程序分配进程表中的空间；分派程序分配时间片。正如计算机系统里的许多问题一样，这种分配任务表面上看起来很简

单，但实际上，对于一个设计不佳的操作系统，几个微小的错误就将导致系统失灵。要记住，机器不会自己思考，它只是遵照指令办事。因此，为了构建一个可靠的操作系统，我们必须设计算法克服各种可能出现的意外情况，不管它出现的概率有多小。

3.4.1 信号量

让我们来考虑一个分时/多任务处理操作系统，它控制只有一台打印机的计算机的活动。如果一个进程需要打印它的结果，那么它必须向操作系统提出请求，要求访问打印机的设备驱动程序。这个时候，操作系统必须根据该打印机是否正被另一个进程占用来决定是否批准这个请求。如果没有被占用，那么操作系统应该批准这个请求，并允许该进程继续执行；否则，操作系统应当拒绝这个请求，或者把这个进程归类为等待进程，直到打印机可用为止。毕竟，如果两个进程同时获得对打印机的访问权，那么结果对两者都将毫无用处。

为了控制对打印机的访问，操作系统必须跟踪打印机是否已经被分配。完成这个任务的一种方法是使用一个标志，这里标志指的是存储器中的一个位，其状态通常指的是置位（set）和清零（clear），而不是1和0。清零标志（值为0）表示打印机可用，置位标志（值为1）表示打印机当前已分配出去。表面上看，这种方法似乎可行。每次访问打印机的一个请求到来时，操作系统要做的工作只是检查这个标志。如果是清零标志，操作系统就批准该请求，同时对标志进行置位。如果是置位标志，操作系统就将请求进程放入等待队列中。每当

一个进程用完打印机，操作系统就将打印机分配给一个等待进程，如果没有等待进程，则仅将这个标志清零。

微软的任务管理器

通过执行“任务管理器”这个实用程序（同时按下Ctrl、Alt和Delete键），你可以深入了解微软Windows操作系统的内部活动。特别地，通过选择“任务管理器”窗口的“进程”选项卡，你可以看到进程表。这里你可以做一个试验：在激活任何应用程序之前，看一下进程表。（你也许会惊讶于表中已经有了如此多的进程，它们都是系统基本操作所必需的。）现在激活一个应用，并且确认一个新进程已经进入表中。你还能够看到该进程分配到了多少内存空间。

然而，这个简单的标志系统有一个问题。测试任务和可能的标志置位任务也许需要几条机器指令。（标志的值必须从主存储器检索，在CPU中操控，最终存回主存储器。）因此，在检测到清零标志之后、标志被置位之前，这个任务有可能被中断。具体而言，假设这个打印机当前是可用的，且一个进程请求使用它。从主存储器中检索到标志，而且发现它已清零，表示该打印机可用。但是，在这个时候这个进程被中断了，另一个进程开始了它的时间片，它也请求使用打印机。于是再一次从主存储器检索标志，发现它仍是清零状态，因为前一个进程在操作系统有时间将主存储器中的标志置位之前被中断了。因此，操作系统允许第二个进程开始使用打印机。之后，第一个进程在它被中断的地方恢复执行，那个地方正是操作系统发现标志是清零

的地方。于是，操作系统继续对主存储器中的标志置位并允许第一个进程访问打印机。现在，这两个进程在使用同一台打印机。

这个问题的解决办法是，要坚持让测试任务和可能有的标志置位任务必须在没有中断的条件下完成。一种方法是使用大多数机器语言都提供的禁止中断指令和允许中断指令。在执行时，禁止中断指令能锁定未来的中断，而允许中断指令能使CPU恢复对中断信号的响应。于是，如果操作系统用禁止中断指令开始一个标志测试例程，并以允许中断指令结束，那么该例程一旦开始就不会有其他活动中断它。

另一种方法是使用许多机器语言里都可用的**测试并置位**（test-and-set）指令。这条指令要求CPU检索一个标志的值，记录接收到的值，然后置位该标志——所有这些工作都在一条机器指令内完成。它的优点是，因为CPU在识别中断之前必须完成当前的指令，所以测试和标志置位任务作为一条指令实现时无法被拆分。

正如刚才所述，一个正确实现的标志称为**信号量**（semaphore），它源自用于控制轨道区段的铁路信号。事实上，信号量在软件系统里的用法与其在铁路系统里的用法是一样的。就像一个轨道区段一次只能有一列列车通过，一个指令序列一次也只能由一个进程执行。这样的指令序列称为**临界区**（critical region）。这种一次只允许一个进程执行一个临界区的要求称为**互斥**（mutual exclusion）。概括地说，获得对一个临界区的互斥的常用办法是用一个信号量守护这个临界区。一个进程要进这个临界区，就必须确定这个信号量是清零的，并在进入临界区之前把它置位；然后在出临界区时把这个信号量清

零。如果发现这个信号量在置位状态，那么试图进入临界区的进程必须等待，直到这个信号量被清零。

3.4.2 死锁

在资源分配期间可能发生的另一个问题是**死锁**（deadlock）。在死锁状态下，两个或更多的进程被阻塞，不能继续执行，因为它们中的每一个都在等待已分配给另一个的资源。例如，一个进程可能已有对计算机打印机的访问权，同时它还在等待访问这台计算机的DVD播放器，与此同时另一个进程有DVD播放器的访问权，但它却在等待访问打印机。另一个例子出现在允许进程创建新进程[这种动作在UNIX术语中称为**派生**（forking）]来完成子任务的系统里。如果调度程序因为进程表没有空间而无法创建新进程，同时系统里的每个进程又都必须创建额外的进程才能完成任务，那么没有一个进程可以继续。这种情况和其他环境下的情况一样（见图3-7），会严重降低系统性能。

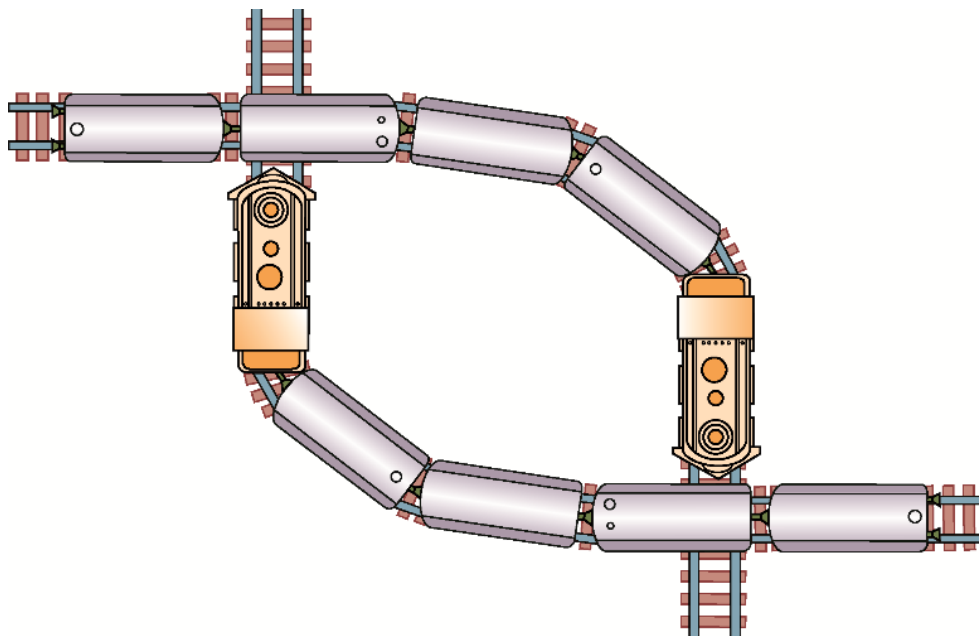


图3-7 由于竞争不可共享的铁路区段造成的死锁

对死锁的分析表明，只有以下3个条件全部满足死锁才会出现。

- (1) 存在对不可共享的资源竞争。
- (2) 这些资源是在不完整的基础上请求的；也就是说，一个进程已经收到了某些资源，稍后还将请求更多资源。
- (3) 一个资源一旦被分配出去，就不能强行收回。

分离出这些条件的意义在于，破坏这3个条件当中的任何一个，就能消除死锁问题。破坏第三个条件的技术被称为死锁检测和改正方案。在这些情况下，死锁被认为不太容易出现，因而不必特别采取办法避免死锁，而只是在死锁出现的时候检测出它，然后通过强制收回某些已经分配的资源来改正它。进程表已满就属于这种情况。如果死锁是由于进程表满产生的，那么操作系统中的例程（或人类管理员利

用其“超级用户”的特权)可以移除[专业术语是**杀死**(kill)]一些进程，释放一些进程表的空间，打破死锁并使得剩下的进程可以继续它们的任务。

着力于抑制前两个条件的技术，一般被称为死锁避免方案。例如，针对上述第二个条件的一个方法是要求每个进程一次性请求它所需要的全部资源。另一个针对第一个条件的方案，不是直接消除竞争，而是把不可共享的资源转变为可共享的资源。例如，假定出问题的资源是打印机，各种进程都请求使用它。每当一个进程请求打印机时，操作系统都会批准这个请求。但是，操作系统不是把这个进程连接到打印机的设备驱动程序上，而是连接到一个设备驱动程序上，该驱动程序把要打印的信息存储在海量存储器上，而不把它发送给打印机。于是，每个进程都认为它访问了打印机，都能正常工作。稍后，当打印机可用时，操作系统会把数据从海量存储器传送到打印机。用这种方式，操作系统通过建立有多个打印机的假象把不可共享的资源变成了好像是可共享的。这种保存数据供以后在合适的时候输出的技术称为**假脱机**(spooling)。

Python和操作系统

当一个用户执行一个Python脚本时，操作系统会启动一个新进程来运行这个脚本。这种脚本通常是应用软件，但如果它们扩展或定制了系统的功能，也可以被视为实用软件。Python脚本通过与操作系统的组件(如读写文件的文件管理器或者提供用户交互的GUI或shell)进行交互来完成它们的工作。Python模块“os”提供了各种预定义的、

系统无关的函数，用于访问操作系统的通用特性，如派生新的Python进程，或者执行其他实用程序或应用程序。

多核操作系统

传统的分时/多任务处理系统通过以快过人类感知的速度快速切换时间片，制造了同时执行多个进程的假象。现代系统继续通过这种方式实现多任务处理，但最新的多核CPU确实能够同时运行2个、4个或更多个进程。与一组协同工作的单核计算机不同，一台多核机器包含多个独立的处理器（称为核），它们共享计算机的外围设备、内存和其他资源。对于一个多核操作系统，这意味着分派程序和调度程序必须考虑在每个核上应该执行哪些进程。随着不同的进程运行在不同的核上，处理进程间的竞争变得更具有挑战性，因为每当一个进程需要进入临界区时，所有核上都会禁用中断，但这种做法效率极低。构建能更好地适应新的多核环境的操作系统机制，是计算机科学中比较活跃的研究方向。

假脱机是一种允许多个进程访问一个公共资源的技术，它有许多变体。例如，文件管理程序可以批准若干个进程访问同一个文件，前提是它们只是从该文件读取数据，但是如果多个进程试图同时更改一个文件就会发生冲突。于是，文件管理程序可以根据进程的需要分配对文件的访问权限，允许若干个进程有读访问权，但在任何给定时刻只能有一个进程有写访问权。其他系统可能会把这种文件分成多个部分，以便不同的进程可以并发地更改文件的不同部分。然而，上述各种技术要得到一个可靠的系统，都有一些必须解决的微妙问题。例

如，当有写访问权的进程更改了这个文件时，如何通知那些只有读访问权的进程呢？

3.4节问题与练习

1．假设进程A和B正在共享同一台机器的时间，并且每个进程都需要短时间使用同一个不可共享的资源。（例如，每个进程可能都打印一系列独立的短报告。）那么每个进程可能都重复地获得这个资源，释放它，稍后再请求它。按照下面的方法控制对该资源的访问有什么缺点？

开始时，给一个标志赋予值0。如果进程A请求这个资源并且该标志为0，那么就批准这个请求；否则让进程A等待。如果进程B请求这个资源并且该标志为1，那么就批准这个请求；否则让进程B等待。每当进程A用完这个资源，就把标志变为1。每当进程B用完这个资源，就把标志变为0。

2．假设一条双车道的道路在过隧道时合并为一个车道。为了协调这个隧道的使用，安装了下述信号系统：

一辆汽车无论从哪个入口进入隧道，隧道入口处上方的红灯都会被打开。当这辆汽车离开隧道时，红灯会被关闭。如果一辆正在接近的汽车发现红灯是亮着的，那么它就要等待，直到红灯关闭才进入隧道。

这个系统有什么缺陷？

3．假设为了消除单车道桥上两辆汽车相遇的死锁问题，提出了下面几个解决方案。指出每个解决方案消除了前文中提到的3个死锁条件

中的哪一个。

- a. 在桥上变空之前不允许汽车上桥。
- b. 如果两辆汽车相遇，让其中一辆倒退。
- c. 为这座桥添加第二个车道。

4. 假设我们用圆点表示多道程序设计系统中的每一个进程，从第一个圆点到第二个圆点的箭头表示第一个（圆点所表示的）进程正在等待第二个（圆点所表示的）进程正在使用的（非共享的）资源。数学家把得到的这种图称为**有向图**（directed graph）。有向图的什么性质等价于系统中的死锁？

3.5 安全性

由于操作系统监督着计算机的活动，很自然，它在维护安全性方面也起了重要的作用。从广义上说，这种责任本身也有多种表现形式，可靠性就是其中一种。如果文件管理程序中的缺陷致使一个文件的一部分丢失了，那么这个文件就是不安全的。如果分派程序中的缺陷致使系统故障（通常称为系统崩溃），使一小时的打字工作白费了，那么我们会说产品是不安全的。因此，计算机系统的安全性需要一个精心设计的可靠的操作系统。

可靠软件的开发不限于操作系统，它渗透到整个软件开发领域，构成了计算机科学领域里的软件工程，我们将在第7章讨论软件工程。在本节中，我们集中讨论与操作系统特性密切相关的安全性问题。

3.5.1 来自外部的攻击

操作系统的一个重要任务就是，保护计算机的资源不被未授权个体的访问。在多人使用计算机的情况下，操作系统一般会通过为不同的授权用户建立“账户”的方法来标记不同权限的用户。账户实际上是操作系统内的一条包含授予该用户的诸如用户名、口令和权限等条目的记录。操作系统在每个**登录**（login）过程（一个事务序列，在这个过程中用户与计算机操作系统建立初步联系）中使用这个信息控制用户对系统的访问。

账户由一个称为**超级用户**（super user）或**管理员**（administrator）的人创建。这个人通过将自己标识为管理员（通常是通过用户名和口令）来获得操作系统的高度访问特权。这个联系一旦建立，管理员就可以更改操作系统内的设置、修改关键的软件包、调整授予其他用户访问系统的权限，进行一般用户不能进行的各种活动。

通过这种“极高的高位”，管理员还能够监视计算机系统活动，检测到恶意的或者偶然的破坏行为。为了协助监视，人们开发了大量称为**审计软件**（auditing software）的软件实用程序，来记录和分析发生在计算机系统内的活动。特别地，审计软件可以揭露许多试图用错误的口令登录系统的活动，指出一个未授权用户可能正在试图获取计算机的访问权。审计软件还可以识别用户账户中与该用户以往行为不一致的活动，这可能表明一个未授权用户访问了这个账户。（以下这样的事情不太可能发生：一个用户，以前仅仅使用字处理软件和

电子表格软件，突然开始访问技术性很强的软件应用，或者试图执行超出其用户权限的实用程序包。）

设计审计软件的另一个目的是检测**嗅探软件**（sniffing software），这种软件在计算机上运行时能够记录活动并在稍后将之报告给潜在的入侵者。一个老的众所周知的例子是一个能够模拟操作系统登录过程的程序。这样的程序可以用来欺骗操作系统的授权用户，使他们认为自己是在和操作系统通信，然而，实际上他们是在把自己的用户名和口令提供给骗子。

在所有与计算机安全相关的技术复杂性上，让很多人感到吃惊的是，计算机系统安全性的其中一个主要障碍居然是用户自己的粗心大意。例如，用户选择的口令相对比较容易猜（如名字和日期）；与朋友共享自己的口令；没有及时更换自己的口令；将自己的离线海量存储设备在机器间来回地转移，这样就潜在地降低了其安全性；在计算机系统中安装未经认可的可能会损坏系统安全性的软件。对于类似这样的问题，大多数使用大型计算机装置的机构都采用并实施一些策略，对用户的需求和职责进行分类。

口令安全

随着计算机存储的个人数据和有价值数据的量越来越大，用安全口令保护计算机账户的需求变得越来越重要。因为许多网络连接系统允许各种类型的远程登录，所以恶意个人可以经常构建和部署快速尝试许多可能口令的软件，这比人类可以猜测或键入口令的速度快得多。因此，安全口令是一个不容易被猜到的口令，是攻击者通过循环

所有可能的组合要花很长时间才能发现的口令。出于这个原因，现在许多账户要求最短口令长度，可能还要求口令包含大写字母、小写字母、数字或各种标点符号的组合。

遗憾的是，人类通常不太擅长记住长的安全口令，特别是如果他们还需要经常更改口令的情况下。

有许多软件系统可以帮助用户管理大量的在线登录和口令，这些登录和口令似乎是日常在线生活的一部分，但这些系统也只能和保护它们的主口令一样安全。

有几个助记符技巧可以帮助普通用户生成并记住一些相当安全的口令。应该避免使用字典中的完整单词以及众所周知的个人数字（如家人的生日），因为这些都是坚定的攻击者最先猜的内容。然而，将几个通常不会一起出现的中等长度的单词组合成一个短语，再混合上不同的大小写和数字或标点符号，就能造出不容易被猜到或被偶然发现的容易记住的心理故事。

举一个例子，作者的一位喜爱动物的朋友曾被要求坐在同事旁边，他的任务是给后院的一群野生浣熊喂狗饼干，并将蛋黄酱涂在家里植物的叶子上让它们有光泽。虽然“Racc00nTreat & May0Sh1n3”很容易记住，但是在本书中讲出这个故事之前很难猜到。

3.5.2 来自内部的攻击

一旦入侵者（也可能是怀有恶意的授权用户）获得了计算机系统的访问权限，他们下一步的工作通常是浏览机器，寻找其感兴趣的信息或者插入破坏性软件的地方。如果一个入侵者获取了系统的管理员

账号，那么上述事情就很容易做到，这也是我们为什么要严格保护好管理员口令的原因。然而，如果是通过普通用户账号进行访问，那么入侵者必然会欺骗操作系统，以获得超越授予该用户的权限。例如，入侵者可能会尝试欺骗内存管理程序，让一个进程访问其被分配的存储区以外的主存储器存储单元；或者欺骗文件管理程序，访问本应该被拒绝访问的文件。

今天的CPU在设计时已经加强了一些功能，能够阻止上面谈到的攻击尝试。举一个例子来说，我们可以考虑这样一个需求：通过内存管理程序，将进程限制在给它分配的主存储器区域内。如果没有这样的限制，一个进程就能够从主存储器中擦除操作系统，从而接管对计算机的控制。为了应对这样的尝试，为多道程序设计系统设计的CPU通常包括若干个专用寄存器，操作系统可以在这些寄存器中存储分配给一个进程的存储区域的上下界。于是，当执行该进程时，CPU把每个存储器引用与这些寄存器中的值进行比较，以确保该引用在指定的界限之内。如果发现这个引用在为该进程指定的区域之外，CPU将自动把控制权交还给操作系统（借助中断处理），这样操作系统就可以采取适当的行动。

这个方案中还有一个小的但很重要的问题。如果没有进一步的安全特性，一个进程只要改变含有存储区界限的专用寄存器的值，就仍然可以访问指定区域以外的存储单元。也就是说，一个进程想要访问更多的内存区域，它只需增加存放存储区域上界的寄存器的值，然后不需要得到操作系统的批准，就可以使用这些额外的内存空间。

为了防止这样的行为，将多道程序设计系统的CPU设计为工作在两种**特权级别**（privilege level）之一的模式下，我们将其中之一称为“有特权模式”，而另一个称为“无特权模式”。当处在有特权模式下时，CPU能够用自己的机器语言处理所有的指令，但处在无特权模式下时，能够接受的指令就是有限的了。这种仅在有特权模式下可用的指令称为**特权指令**（privileged instruction）。（典型的特权指令的例子有：改变内存界限寄存器的内容的指令和改变CPU当前的特权模式的指令。）当CPU处于无特权模式时，任何执行特权指令的企图都将引起中断。这个中断将CPU转变为有特权模式，并将控制权交给操作系统内部的中断处理程序。

多因素认证

对于人类使用弱口令来保护计算机账户存在的普遍问题，一个解决方案是需要一个以上的证据才能获得访问权限。口令是用户知道的一个例子，还有一个常用安全问题的答案，例如“你外祖母的婚前姓是什么？”第二个认证因素可能取决于用户拥有的东西，如ATM卡、专用安全令牌或具有特定电话号码的智能手机。越来越多的在线账户需要双因素认证，其中用户必须提供一个口令，并且在登录过程中还会通过其智能手机短信收到一个特殊的访问码文本。第三组认证因素可能取决于用户自身。嵌入移动设备的生物识别传感器现在能很容易地识别用户的独特指纹、语音或视网膜图案。一些研究人员正在研究其他生物识别机制，例如，可能会使用智能手机加速计数据来识别用户独特的步态模式。

然而，这些机制都没有被证明是绝对安全的，因为聪明的攻击者采用的方案能监听用户口令、愚弄嵌入式传感器，甚至能欺骗人类管理员修改智能手机号码，将手机关联到另一个用户的账户。

刚开机时，CPU处于有特权模式，因此，当操作系统在引导过程结束后开始启动时，所有的指令都是可执行的。然而，每次操作系统允许一个进程开始一个时间片时，它就通过执行“改变特权模式”的指令，将CPU切换到无特权模式。于是，如果一个进程试图执行有特权指令，操作系统就会得到通知，这样操作系统就充当了维护计算机系统完整性的角色。

有特权指令和特权级别的控制是操作系统维护安全性可用的一个主要工具。然而，这些工具的使用对操作系统设计而言是一项复杂的任务，且在当前的操作系统中，错误还在不断出现。因此，在特权级别的控制中，任何一个缺陷都可能给灾难打开大门，不论是恶意程序员引起的，还是无意中的程序设计错误造成的。如果允许一个进程更改控制系统多道程序设计系统的计时器，那么这个进程就能够延长它自己的时间片，甚至控制整个机器。如果允许一个进程直接访问外围设备，那么它就能不受系统文件管理程序的监管而读取文件。如果允许一个进程访问分配给它的区域之外的存储单元，那么它就能读取甚至更改其他进程正在使用的数据。因此，维护计算机的安全性，既是管理员的一个重要任务，也是操作系统和处理器设计的一个目标。

3.5节问题与练习

1. 列举几个口令选取不好的例子，并说明为什么不好。
2. 英特尔奔腾系列处理器提供4个特权级别。为什么CPU的设计人员选择提供4个特权级别，而不是3个或5个？
3. 如果多道程序设计系统里的一个进程可以访问分配给它的区域之外的存储单元，那么它怎样获得该机器的控制权？

复习题

（带*的题目涉及选读章节的内容。）

1. 列出一个典型操作系统的4个活动。
2. 概述批处理和交互式处理的区别。
3. 假设有3个作业R、S、T，按这个顺序排在一个作业队列里。接着，1个作业移出队列，第4个作业X进入队列。然后又有1个作业移出队列，作业Y和作业Z进入队列，最后，按照一次一个作业的顺序移出，使队列变空。请按移出的顺序列出所有的作业。
4. 嵌入式系统与PC的区别是什么？
5. 什么是多任务处理操作系统？
6. 如果你有一台PC或一部智能手机，请列举几个你能用到多任务处理功能的情形。

7．根据你熟悉的计算机系统，列举两个应用软件单元和两个实用软件单元，然后说明为什么这样归类。

8．a. 操作系统的用户界面的作用是什么？

b．操作系统的内核的作用是什么？

9．路径X/Y/Z描述的是什么目录结构？

10．定义操作系统环境下使用的术语“进程”。

11．操作系统的进程表里包含什么信息？

12．就绪进程和等待进程的差别是什么？

13．虚拟内存和主存储器之间的差别是什么？

14．假设某计算机有512 MB（MiB）的主存储器，操作系统要创建主存储器两倍大小的虚拟内存，页面大小为2KB（KiB）。请问需要多少页？

15．在分时/多任务处理系统里，如果两个进程需要同时访问同一个文件，会出现什么样的复杂情况？存在文件管理程序应批准这种请求的情况吗？存在文件管理程序应拒绝这种请求的情况吗？

16．应用软件和系统软件之间的区别是什么？请各举一个例子。

17．定义多处理器体系结构情况下的负载平衡与均分。

18．概述引导过程。

19．为什么说引导过程是必要的？

20．如果你有一台PC，记录开机时你所观察到的一连串活动。然后确定在引导过程实际开始工作之前计算机屏幕上显示了哪些信息？什么软件写下的这些信息？

21．假定多道程序设计操作系统分配的时间片是10 ms，机器每纳秒平均执行5条指令，那么在一个时间片内能执行多少条指令？

22．如果一个打字员每分钟能录入60个单词（在这里假设一个单词含5个字符），那么录入两个字符的间隔是多长？如果多道程序设计操作系统分配的时间片是以10毫秒为单位的，并且我们忽略进程间切换所需的时间，那么录入两个字符之间要分配多少时间片？

23．假定一个多道程序设计操作系统分配的时间片为50毫秒。如果把磁盘的读写磁头定位到所需的磁道上通常要花8毫秒，磁道上所要的数据旋转到读写磁头之下通常要17毫秒，那么等待一个读磁盘操作发生可能要多少个时间片？如果该机器每纳秒能执行10条指令，那么在这个等待时间里可以执行多少条指令？（这就是为什么当一个进程用外围设备完成操作时，多道程序设计系统终止这个进程的时间片，让另一个进程运行而让第一个进程等待外围设备的服务。）

24．列举多任务处理操作系统必须协调访问的5种资源。

25．如果一个进程需要执行大量的I/O操作，我们就说这个进程是I/O密集型的，而如果一个进程由大多数在CPU/存储系统中完成的计算构成，我们就说这个进程是计算密集型的。如果这两种进程都在等待分配时间片，请问如何确定它们的优先级？为什么？

26．在多道程序设计环境中运行两个进程，如果它们两个都是I/O密集型的（参见第25题），或者一个是I/O密集型的，另一个是计算密集型的，那么这个系统能否获得更大的吞吐量？为什么？

27．编写一组指令告诉操作系统的分派程序，在一个进程的时间片用完时该做什么。

28．一个进程的状态中包含哪些信息？

29．列出多道程序设计系统中一个进程不会全部用完分配给它的时间片的情况。

30．按照时间顺序列出一个进程被中断时发生的主要事件。

31．根据你所使用的操作系统回答下列问题。

a. 如何让操作系统把一个文件从一个地方复制到另一个地方？

b. 如何让操作系统显示磁盘上的目录？

c. 如何让操作系统执行一个程序？

32．根据你所使用的操作系统回答下列问题。

- a. 操作系统如何限制仅允许已批准用户访问资源？
- b. 如何让操作系统显示当前在进程表里的有哪些进程？
- c. 如何告诉操作系统你不想该机器的其他用户访问你的文件？

*33 . 解释许多机器语言里都有的“测试并置位”指令的重要用途。为什么将整个测试并置位过程作为单个指令实现很重要？

*34 . 一个银行家只有100 000美元，贷款给两个客户，每位50 000美元。后来这两位客户回了同样的话：他们在能够还贷之前各自还需要10 000美元来完成先前贷款所涉及的商业交易。这个银行家通过从其他地方借来资金（提高贷款利率）贷款给这两个客户，解决了这个死锁问题。在死锁的3个条件中，银行家消除了其中的哪个条件？

*35 . 每个想参加本地大学的铁路修建模型II课程的学生，都要得到教师的允许，并且交纳实验费。这两个要求可以在校园的不同地点办理，可以按照任意顺序独立完成。注册学生限制为20名；这个限制由教师和财务处一起掌握，前者只授权20名学生，后者只收20名学生的费用。假定这个注册系统有19名学生成功注册了这一课程，但是最后这个名额有两名学生竞争——一名仅得到了老师的允许，另一名仅交纳了费用。

下面解决该问题的各个方案中，分别消除了死锁的3个条件中的哪个？

- a. 允许这两名学生都参加该课程。

b. 该班人数降为19人，因此这两名学生都不能注册该课程。

c. 拒绝这两名竞争的学生，让第三名学生成为第20名。

d. 将注册该课程的要求改为一个：交纳费用。于是交了费用的学生注册成功，另一名学生被拒绝。

*36．由于计算机显示器上的每一个区域一次只能被一个进程使用（否则屏幕上的图像将难以认清），这些由窗口管理程序分配的区域是不可共享的资源。为了避免死锁，窗口管理程序应消除死锁的3个必要条件中的哪一个？

*37．假设一个计算机系统里的每个不可共享的资源都可以被归类为1级资源、2级资源或3级资源。此外，假设系统中的每一个进程都要根据这个类别请求它所需要的资源。也就是说，它在请求2级资源之前必须一次请求所有必需的1级资源。一旦它得到了1级资源，就可以申请所有必需的2级资源，依次类推。这个系统会出现死锁吗？为什么？

*38．机器人的两个机械臂是程序控制的，它们从传送带上取零件，测试它们的公差并根据测试结果把它们分别放到两个箱子中。零件一次到达一个，它们之间有足够的间隔。为了防止两个机械臂尝试抓同一个零件，控制机械臂的计算机共享一个公共的存储单元。如果一个零件到来时一个机械臂是可用的，那么控制它的计算机就读公共存储单元的值。如果该值为非零值，机械臂就让那个零件通过；否则，起控制作用的计算机把一个非0的值放到这个存储单元，指挥这个

机械臂抓起该零件，动作完成后再把值0放入该存储单元。什么样的事件序列会导致两个机械臂之间的激烈竞争？

*39．说明队列在假脱机输出到打印机的过程中的使用。

*40．如果一个在等待时间片的进程一直都没有获得时间片，我们就说这个进程在遭受**饥饿**（starvation）。

a. 对竞争通过交叉口的汽车来说，交叉口中间的路面是不可共享的资源。控制这个资源分配的是交通灯，而不是操作系统。如果这个灯能够感知来自每个方向的交通流量，并被编程为给流量较大的方向开绿灯，流量较小的方向就可能遭受饥饿。请问怎么避免“饥饿”？

b. 在每个进程的优先级保持固定的优先级系统中，如果分派程序总是按优先级系统分配时间片，那么在什么时候一个进程会感到“饥饿”？（提示：相对于正在等待的进程，刚执行完时间片的进程的优先级是什么，并且接下来哪个例程会得到下一个时间片？）你能猜到许多操作系统是怎么避免这个问题的吗？

*41．死锁和饥饿（参见第40题）的相似之处是什么？差别又是什么？

*42．下面是“哲学家进餐”问题，它最初是由E. W. Dijkstra提出的，现在已经是计算机科学传说的一部分。5个哲学家围着一张圆桌就座，每个人面前放了一盘意大利面条。桌上有5把叉子，每个盘之间有一把。每个哲学家都在思考和吃面之间轮换。要吃到面，一个哲学

家需要拥有紧挨他盘子的两把叉子。说明“哲学家进餐”问题中出现死锁和饥饿（见第40题）的可能性。

*43．随着多道程序设计系统中时间片的长度越来越短，会出现什么问题？越来越长呢？

*44．随着计算机科学的发展，机器语言已被扩展以提供专门指令。在3.4节中介绍了3种在操作系统中广泛使用的指令。这些指令是什么？

45．列举操作系统管理员能进行而一般用户不能进行的两个活动。

46．操作系统如何防止一个进程访问另一个进程的内存空间？

47．假定一个口令由英文字母表（26个字符）中的9个字符组成。如果测试每个可能的口令需要1 ms，那么测试所有可能的口令需要多长时间？

48．为什么为多任务处理操作系统设计的各个CPU能够在不同特权级别运行？

49．列举两个由有特权的指令请求的典型活动。

50．列举一个进程可能挑战计算机系统（如果未被操作系统防止这样做）安全性的3种方式。

51．什么是多核操作系统？

52．固件更新和操作系统更新之间的区别是什么？

53．窗口管理程序与操作系统有什么关系？

54．Internet Explorer（IE）是微软的Windows操作系统的一部分吗？

55．嵌入式操作系统能够解决哪些特殊问题？

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1．假定你在使用一个多用户操作系统，它允许你查看其他用户的文件的名称，而且如果那些文件没有加保护措施，它还允许查看那些文件的内容。未经允许就查看这些信息是类似于未经允许就闲逛别人未锁门的房间，还是类似于阅读放在公共休息室（如医生的候诊室）的资料？

2．若访问一个多用户计算机系统，你在选择口令时有什么责任？

3．如果一个操作系统有安全缺陷，使得一个恶意的程序员能够在未经批准的情况下访问其敏感数据，那么该操作系统的开发人员应该负多大的责任？

4. 你有责任锁好门防止入侵者入内，还是公众有责任在未受邀请时待在屋外？操作系统有责任防备别人对计算机及其内容的访问，还是黑客有责任不入侵机器？

5. 在《瓦尔登湖》一书中，亨利·戴维·梭罗坚持认为，我们已经变成自己工具的工具；也就是说，我们并非从所拥有的工具中受益，而是要花费时间得到和维护工具。对于计算机，这在多大程度上是真的？如果你有一台个人计算机或者移动设备，那么你要花多少时间去赚钱承担它的费用、学习如何使用它的操作系统、学习如何使用它的实用软件和应用软件、维护它，以及为它的软件下载更新包？你得到的好处的时间量与你花费的时间总量相比又如何？使用它时，你花费的时间值吗？你的人际交往是有个人计算设备的时候更活跃，还是没有的时候更活跃？

课外阅读

Bishop, M. *Introduction to Computer Security*. Boston, MA: Addison-Wesley, 2005.

Craig, B. *CyberLaw, The Law of the Internet and Information Technology*. Upper Saddle River, NJ: Prentice-Hall, 2012.

Davis, W. S. and T. M. Rajkumar. *Operating Systems: A Systematic View*, 6th ed. Boston, MA: Addison-Wesley, 2005.

Deitel, H. M., P. J. Deitel, and D. R. Choffnes. *Operating Systems*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2005.

Silberschatz, A., P. B. Galvin, and G. Gagne. *Operating System Concepts*, 9th ed. New York: Wiley, 2012.

Stallings, W. *Operating Systems: Internals and Design Principles*, 9th ed. Upper Saddle River, NJ: Pearson, 2017.

Tanenbaum, A. S. *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson, 2016.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第4章 组网及因特网

本章讨论计算机科学中被称为组网的领域，包括学习如何将计算机连接起来共享信息和资源。学习内容包括网络的结构与操作、网络的应用以及网络安全问题。学习的重要主题是遍布世界范围的特殊网络——因特网。

本章内容

4.1 网络基础

4.2 因特网

4.3 万维网

*4.4 因特网协议

*4.5 简单的客户机服务器

4.6 网络安全

持久理解和学习成果

因特网是一个自治系统网络。AP[®]CSP

学习成果 解释因特网中的抽象以及因特网是如何运作的。

因特网的特点影响着建立在其上的系统。AP[®]CSP

学习成果 解释因特网的特点和建立在其上的系统。

学习成果 解释因特网的特点是如何影响建立在其上的系统的。

网络安全是因特网和建立在其上的系统的重要关注点。AP[®]CSP

学习成果 确定现有的网络安全问题，以及通过因特网和建立在其上的系统解决这些问题的潜在选项。

计算增强了沟通、互动和认知。AP[®]CSP

学习成果 解释计算革新是如何影响沟通、互动和认知的。

计算对人和社会有全球性影响（既有有益的又有有害的）。

AP[®]CSP

学习成果 分析计算的有益影响和有害影响。

计算革新既影响着设计和使用它们的经济、社会和文化环境，又受这些环境的影响。AP[®]CSP

学习成果 解释计算和现实生活环境（包括经济、社会和文化环境）之间的关系。

人们对不同计算机之间共享信息和资源的需求催生了相互连接的计算机系统，这个系统被称为**网络**（network）。计算机通过网络连接

在一起，以便数据可以从一台计算机传送到另一台计算机。在网络中，计算机用户可以彼此交换信息，并且可以共享分布在全球系统中的资源，如流媒体内容、软件以及数据存储设施。几十年来，支持这类应用所需的基础软件已经从简单的实用程序发展为扩展的网络软件系统，为数十亿计算设备上的数十亿用户提供复杂的网络基础设施。从某种意义上说，网络软件已发展为网络操作系统。在本章中，我们将探讨计算机科学中这个不断发展的领域。

4.1 网络基础

我们从介绍多种基本的组网概念开始学习网络。

4.1.1 网络分类

计算机网络通常分为**个人域网**（personal area network，PAN）、**局域网**（local area network，LAN）、**城域网**（metropolitan area network，MAN）和**广域网**（wide area network，WAN）。个人域网通常用于小范围的通信，一般只有几米远，如无线耳机与智能手机之间或者无线鼠标与PC之间的通信。相比之下，局域网通常由一个建筑物或者一个建筑群中的若干计算机组成。例如，大学校园中的计算机或者制造厂中的计算机都可以用局域网连接。城域网属于中型网络，如横跨一个本地社区的城域网。而广域网能连接距离更远的机器，如相邻的城市或者世界的两端。

网络的另一种分类依据是，网络的内部操作是基于公共领域的设计，还是基于特定实体（如个人或公司）所拥有并控制的创新。前一种类型的网络称为**开放式**（open）网络，后一种类型的网络称为**封闭式**（closed）网络，有时也称为**专有**（proprietary）网络。开放式网络设计允许自由流通，更容易受大众欢迎，这就是它们通常最终战胜专有网络的原因，因为专有网络的应用受到许可费和合约条件的限制。

基本知识点

- 开放式标准促进了因特网的发展。

全球因特网（本章研究的网络中流行的全球网络）属于开放式系统。尤其是，贯穿因特网的通信是由一组称为TCP/IP协议簇的开放标准控制的（这是4.4节的主题）。任何人都可以自由地使用这些标准，而不需要付费或签署许可协议。与此相反，像微软公司这样的公司可能会开发一些专有系统并选择保持其所有权，允许公司通过出售或出租这些产品获利。

还有一种网络分类方法依据的是网络的拓扑，即机器连接的模式。两种比较流行的拓扑是：总线拓扑，即所有机器都通过同一条被称为“总线”的公共通信线路连接起来（见图4-1a）；星形拓扑，即将一台机器作为中心焦点，所有其他机器都与之相连（见图4-1b）。20世纪90年代，总线拓扑得以流行，当时是通过称为以太网的一组标准实现的，但更高速度的以太网版本在行为逻辑上像总线拓扑，在物理连接上像星形拓扑。

星形拓扑的起源可以追溯到20世纪70年代。它由一台服务多个用户的大型中央计算机范例演变而来。随着这些用户使用的简单终端自身发展成为小型计算机，星形拓扑应运而生。今天，星形配置在无线网络中也很流行，无线网络中的通信是通过无线电广播和中央机器实现的，其中，中央机器被称为**接入点**（access point，AP），是协调所有通信的焦点。



图4-1 两种流行的网络拓扑

总线网络和星形网络在设备的物理排列上的区别并不总是很明显。二者的区别在于网络中的机器把自己想象成是通过一条公共总线彼此直接通信的，还是通过中间的中央机器间接通信的。例如，总线网络可能不会出现图4-1中描述的那种各个计算机通过一条短链路连接到一条长总线上的情况。相反，总线网络的总线可能很短，到各个机器的链路却很长，这意味着总线网络看起来会比较像星形网络。确实，有时总线网络会通过链路将每台计算机连接到中央位置，在中央位置再连接到一种叫作**集线器**（hub）的设备上。集线器其实就是一条非常短的总线，其功能在于将接收到的信号（可能会经过一些放大）传回给与之相连的所有机器。尽管使用集线器的结果看起来像星形网络，但在运作上像总线网络。

4.1.2 协议

为了网络运行可靠，建立管理网络活动的规则很重要，这类规则称为**协议**（protocol）。通过开发和采纳协议标准，厂商能够生产出与其他厂商的网络应用产品相兼容的产品。因此，在网络技术的开发中，协议标准的开发是一个必不可少的环节。

为了介绍协议概念，我们考虑这样一个问题：如何协调网络中计算机之间报文的传输。如果没有控制此类通信的规则，所有的计算机就很可能同时抢着传输报文，或者不能协助其他需要协助的机器。

在基于以太网标准的总线网络中，报文传输的许可是通过名为**带冲突检测的载波监听多路访问**（carrier sense, multiple access with collision detection，CSMA/CD）的协议控制的。该协议规定每条报文都要广播给总线上的所有机器（见图4-2）。每台机器都对所有报文进行监听，但是只保存发送给自己的报文。要想传输报文，机器需要等到总线空闲时才能开始传输并继续监听总线。如果另一台机器也开始传输报文，那么两台机器都会检测到这种冲突，并各自暂停一段随机长的时间，然后再次尝试传输。这样的结果类似于一小群人在交谈中使用的系统：如果两个人同时开始讲话，他们两个都会停下来。不同的是，人们可能会有一系列的对话，如“对不起，刚才你想说什么？”“不，不，你先说。”但是在CSMA/CD协议下，每台机器都只会稍后再作尝试。

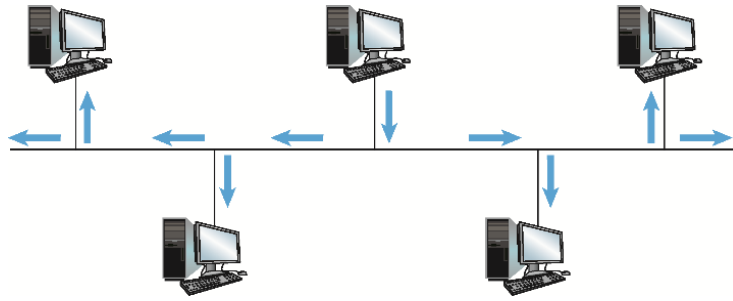


图4-2 通过总线网络通信

注意，CSMA/CD和无线星形网络并不兼容。在无线星形网络中，所有的机器都通过一个中央接入点进行通信，这是因为一台机器可能无法检测到它与其他计算机的传输冲突。例如，一台机器可能监听不到其他机器，因为它自己的信号淹没了其他机器的信号。另一个原因可能是，虽然它们都能与中央接入点进行通信，但来自不同机器的信号被物体或距离阻隔了。这种情况称为**隐藏终端问题**（hidden terminal problem），见图4-3。结果是无线网络采用尝试避免冲突的策略，而不是尝试检测冲突的策略。这种策略被归类为**带冲突避免的载波感应多路访问**（carrier sense, multiple access with collision avoidance，CSMA/CA），其中许多策略是由IEEE（见第7章中提及的“美国电气电子工程师学会”）在IEEE 802.11中定义的协议下进行标准化的，通常称为**无线保真**（WiFi）。需要强调的是，冲突避免协议的设计目的是避免冲突，也许并不能完全消除冲突。当冲突发生时，必须重新传输报文。

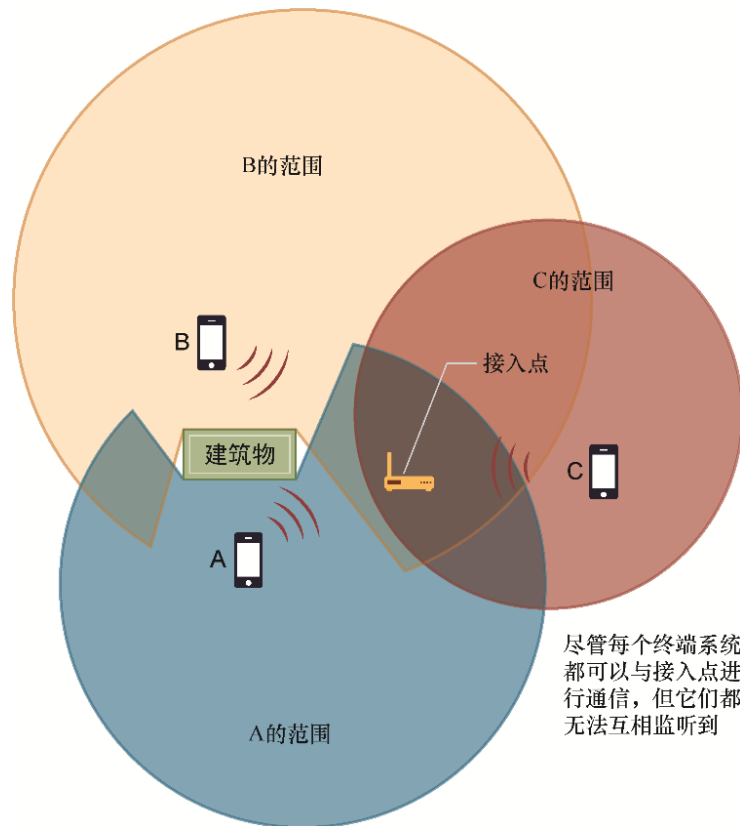


图4-3 隐藏终端问题

最常见的冲突避免方法是将优先权赋予已经在等待传输机会的计算机。这种协议和以太网的CSMA/CD有相似之处。二者的主要区别在于，当一台机器首先需要传输报文，并且发现通信信道处于空闲时，它并不是立即开始传输。相反，它会等一小段时间，只有当信道在这一段时间内都保持空闲时才会开始传输。如果在这个过程中，发现这是一个忙信道，那么机器会等待一段随机决定的时间，然后再重新尝试。一旦这段时间耗尽，这台机器就被允许立即要求占用空闲信道。这就意味着避免了“新来者”和已经处于等待状态的机器之间的冲突，因为“新来者”需要等到一直处于等待状态的所有机器都开始传输之后，才会被允许要求占用空闲信道。

然而，该协议无法解决隐藏终端问题。毕竟，任何基于辨别闲信道或者忙信道的协议都要求每个站点都能够监听其他所有站点。为了解决这一问题，一些WiFi网络要求每台机器都向接入点发送简短的“请求”报文，并等待接入点确认请求，然后再传输完整的报文。如果接入点很忙，正在处理“隐藏终端”，那么它会忽略请求，然后发出请求的机器将获悉并等待。否则，接入点会确认请求，机器将获悉现在进行传输是安全的。注意，尽管机器无法监听正在发生的报文传输，但是网络中所有的机器都能监听接入点发出的所有确认，因此就能知道接入点在任何给定时间是否繁忙。

4.1.3 组合网络

有时候需要连接现有网络以形成一个扩展的通信系统，这可以通过将网络连接成一个相同“类型”的更大的网络来实现。例如，对于基于以太网协议的总线网络，通常可以将总线连接起来形成一根长总线。这可以通过中继器、网桥、交换机等不同的设备来完成，这些设备的区别既微妙信息量又大。它们中最简单的是**中继器**（repeater），它只是在两个原始总线间简单地来回传送信号（通常有某种形式的放大）的设备，而不会考虑信号的含义（见图4-4a）。

网桥（bridge）类似于中继器，但比中继器更复杂。与中继器相似，它也是连接两条总线，但是不是在连接上传输所有的报文。相反，网桥要检查每条报文的目地址，当报文的目的地是另一端的计算机时才在连接上转发这个报文。因此，网桥同一端的两台机器可以

在不干扰另一端发生的通信的情况下交换报文。与中继器形成的系统相比，网桥形成的系统更高效。

交换机（switch）本质上就是具有多个连接的网桥，可以连接若干条总线，不止两条。因此，交换机形成的网络包括若干从交换机延伸出来的总线，它们就类似于车轮上的辐条（见图4-4b）。与网桥一样，交换机也要考虑所有报文的目的地址，并且仅仅转发那些目的地是其他“辐条”的报文。此外，被转发的每一个报文只会被转送至相应的“辐条”，因此最大限度地减少了每根“辐条”的流量。

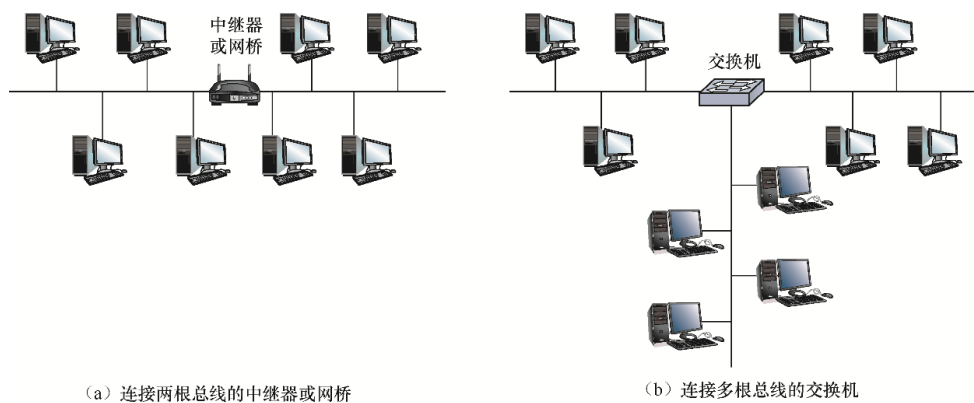


图4-4 将规模较小的总线网络组建成大型总线网络

需要重点注意的是，当计算机通过中继器、网桥和交换机连接时，形成的是一个大型网络。整个系统（使用相同的协议）以相同的方式运作，就像每个规模较小的原始网络一样。

然而，要连接的网络有时会有不兼容的特性。例如，WiFi网络的特性就不易与以太网网络兼容。在这种情况下，网络必须按建立一个网络的网络 [称为 **互联网**（internet）] 的方式连接。在这个网络中，原始网络仍然保持其独立性，并且继续作为独立的网络运行。注意，

通用术语**互联网**（internet）不同于**因特网**（Internet）。因特网的英文Internet的首字母是大写的I，指的是一种独特的、世界范围的互联网，本章后面会介绍。现实中有许多互联网的例子。事实上，在因特网流行之前，传统的电话通信就是由世界范围的互联网系统操控的。

把网络连接起来形成互联网的设备是**路由器**（router），这是一种用来传送报文的专用计算机。注意，路由器的任务与中继器、网桥和交换机的不同，路由器提供的是网络之间的链接，并允许每个网络保持其独特的内部特性。作为一个例子，图4-5描述了通过路由器组连接两个WiFi星形网络和一个以太网总线网络的情形。当某个WiFi网络中的一台机器想要给以太网中的一台机器发送报文时，它会先把报文发送到其网络中的接入点，接入点再把报文发送到与之相连的路由器，该路由器把报文转发给以太网中的路由器。在那里该报文被发送给总线上的一台机器，然后这台机器把报文转发到它在以太网中的最终目的地。

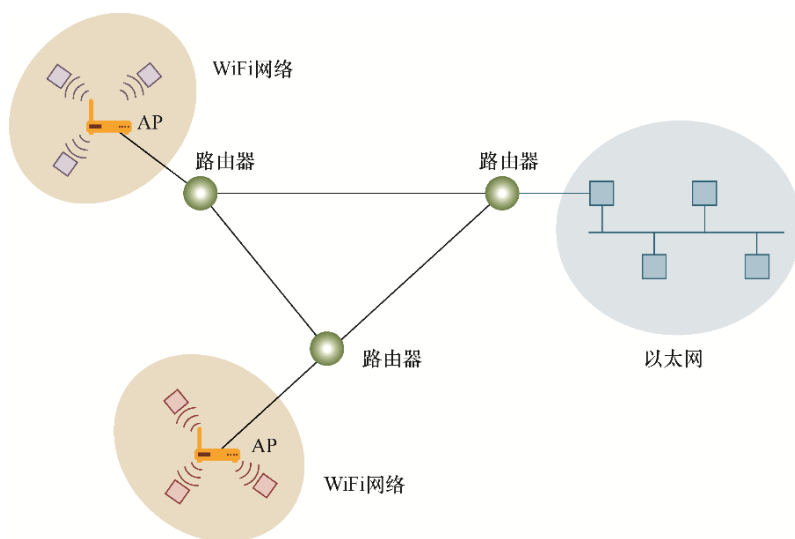


图4-5 路由器将两个WiFi网络和一个以太网连接成了一个互联网

之所以叫路由器是因为它们的用途是向适当的方向转发报文。这个转发过程是基于互联网范围的寻址系统进行的，这个系统为互联网上的所有设备（包括原始网络中的机器和路由器）分配了唯一的地址。（这样，原始网络中的每台机器都有两个地址：自己网络内的“本地”地址和它的互联网地址。）一台机器想给远程网络中的另一台机器发送报文，需要先将报文的目的地互联网地址附在报文上，然后再把报文发送给其本地的路由器，在那里报文将向适当的方向转发。为了转发报文，每个路由器都维护一张**转发表**（forwarding table），表中包含的方向信息能让路由器依据每条信息的目的地址来确定报文应该被发送到哪个方向。

一个网络与互联网链接的“点”通常称为**网关**（gateway），因为它是网络与外部世界之间的通道。网关有多种形式，因而这个术语的使用不太严谨。在许多情况下，网络的网关仅仅是路由器，通过它，网络能与互联网上的其他网络通信。在其他情况下，术语**网关**所指的可能不仅仅是一台路由器。例如，在连接到因特网的多数住宅WiFi网络中，术语**网关**指的是网络的接入点和与接入点相连的路由器，因为这两个设备通常包装在一个单元中。

基本知识点

- 组成因特网的设备和网络使用地址和协议进行连接和通信。

4.1.4 进程间通信的方法

网络中，在不同计算机上执行（甚至在一台计算机上通过分时/多任务处理执行）的各种活动（或进程）必须经常互相通信，以便协调行动并完成指派的任务，这种进程之间的通信称为**进程间通信**（interprocess communication）。

进程间通信通常采用的是**客户/服务器模式**（client/server model）。这种模式规定了进程的基本角色，或者是向其他进程发出请求的**客户机**（client），或者是满足客户机请求的**服务器**（server）。

客户/服务器模式的一种早期应用出现在连接一组办公室的所有计算机的网络中。在这种应用中，只要把一台高质量的打印机连接到网络上，网络中的所有机器就都能使用它。在这种情况下，打印机的角色就是服务器，常称为**打印服务器**（print server），其他机器的角色为客户机，将打印请求发送给打印服务器。

客户/服务器模式的另一种早期应用是用于降低磁盘存储成本，同时消除复制文字记录的需要。在这种情况下，网络中的某一台机器需要配备高容量的海量存储系统（通常是磁盘），存储某一个组织的所有记录；网络中的其他机器在需要这些记录时，可以请求访问这些记录。于是，实际包含记录的机器扮演的就是服务器的角色，称为**文件服务器**（file server），而那些请求访问存储在文件服务器中的文件的其他机器扮演的就是客户机的角色。

今天，客户/服务器模式被广泛用于网络应用，这一点在本章后面能看到。不过，客户/服务器模式不是进程间通信的唯一方式，另一种

模式是**对等计算模式**（peer-to-peer model，通常简称为**P2P模式**）。客户/服务器模式是一个进程（服务器）为许多其他进程（客户机）提供服务，而对等计算模式中的进程既为对方提供服务，也接收对方的服务（见图4-6）。此外，服务器必须持续执行，以便随时准备为其客户机提供服务，对等计算模式中的进程通常都是临时执行的。例如，对等计算模式的应用包括即时消息处理，人们利用即时消息处理通过因特网或者一些竞技性交互游戏进行文字交流。

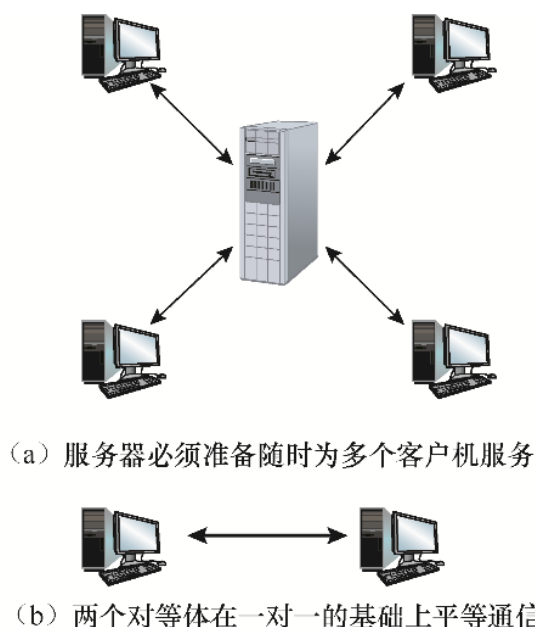


图4-6 客户/服务器模式与对等计算模式的比较

对等计算模式也是通过因特网分发诸如音乐录音和运动图像这类文件的常用方法。在这种情况下，一个对等体可以从另一个对等体接收文件，然后把此文件提供给其他的对等体。参与这种分发的对等体集合有时称为**蜂群**（swarm）。文件分发的蜂群方法与先前使用客户/

服务器模式的方法相反，该模式需要建立中央分发中心（服务器），以使客户机从该中心下载文件（或至少是找到那些文件的源）。

从文件共享的角度来看，P2P模式正在取代客户/服务器模式的一个原因是，它把服务任务分布到许多对等体上，而不是集中在一个服务器上。这种非集中化的操作引出了更高效的系统。遗憾的是，基于P2P模式的文件分发系统流行的另一个原因是，在合法性遭到质疑的情况下，缺乏中心服务器会使版权执法行动变得更为困难。但是，也存在许多案例，在这些案例中一些人发现“困难”并不意味着“不可能”，并且他们发现由于侵犯版权的问题自己面临着重大的责任。

你可能经常读到或者听到[对等网络](#)（peer-to-peer network）这个术语，这个例子正好说明了非科技界采用科技术语时是如何产生误用的。术语[对等](#)指的是两个进程通过网络（或者互联网）进行通信的一种系统，并不是网络（或者互联网）的一种特性。一个进程可能采用对等计算模式与另一个进程进行通信，稍后又采用客户/服务器模式通过同一个网络与另一个进程通信。因此，更准确地说，应该是通过对等计算模式进行通信，而不是通过对等网络进行通信。

4.1.5 分布式系统

因为组网技术的成功，计算机之间通过网络进行的交互已经很普遍，并且涉及方方面面。许多现代软件系统，如全球信息检索系统、公司范围的会计和库存系统、计算机游戏甚至控制网络基础设施本身

的软件，都被设计成**分布式系统**（distributed system）。这意味着，它们由在网络中不同计算机上作为进程执行的软件单元组成。

早期的分布式系统是从零开始独立开发的。不过，现在研究显示，公共的基础设施正运行在包括像通信系统和安全系统这样的系统上。于是，人们做了很多努力生产能够提供这种基本的基础设施的预制系统，因此要构建一个分布式应用，只需要开发系统中应用所特有的部分即可。

现在，有几种类型的分布式计算系统很常见。**集群计算**（cluster computing）指的是一种分布式系统，其中多个独立的计算机密切合作，提供的计算或服务堪比大得多的机器。这些独立的机器加上连接它们的高速网络的总成本，要比一台高价位的超级计算机的成本低，但其可靠性更高，维护成本更低。这样的分布式系统用于提供**高可用性**（high-availability）（因为即使集群中其他成员发生故障或不可用，更有可能保证集群中至少有一个成员能够响应请求）和**负载均衡**（load-balancing）（因为负载可以自动从集群中负载太大的成员转移到负载太小的成员）。**网格计算**（grid computing）是指耦合度没有集群高但是仍能共同完成大型任务的分布式系统。网格计算可能需要专门的软件，以便更容易地分发数据和算法到网格中的机器。威斯康星大学的HTCondor系统和伯克利的开放式网络计算平台（Berkeley's Open Infrastructure for Network Computing，BOINC）都属于这种类型。这两种系统通常安装在用于其他目的的计算机上（如办公用PC或家用PC），当机器空闲时便可为网格自愿提供计算能力。由于因特网

日益增强的连通性，这种自愿的分布式网格计算使数以百万计的家用PC都可以解决无比复杂的数学问题和科学问题。凭借**云计算**（cloud computing），网络上大量的共享计算机得以被按需分配给客户机使用。云计算是分布式系统的最新趋势。就像20世纪初大城市电网的发展使得个体工厂和企业不再需要维护自己的发电机一样，因特网使得实体可以将其数据和计算委托给“云”，这里的“云”指的是网络上可用的大量计算资源。例如，亚马逊的弹性计算云（Elastic Compute Cloud）服务允许客户按小时租用虚拟计算机，而不需要考虑计算机硬件的实际位置。另外，Google Drive和Google Apps允许用户在信息方面进行协作，允许他们在构建Web服务时不需要了解多少台计算机在用于求解同一问题或相关数据存储在何处。云计算服务提供了合理的可靠性和可扩展性保证，但也引发了人们对隐私和安全性的担忧，因为我们也许再也无法知晓谁在拥有和操作着我们所用的计算机。

4.1节问题与练习

- 1．什么是开放式网络？
- 2．概述网桥和交换机之间的区别。
- 3．什么是路由器？
- 4．列举社会中一些符合客户/服务器模式的关系。
- 5．列举社会中使用的一些协议。
- 6．概述集群计算和网格计算之间的区别。

4.2 因特网

互联网中最著名的例子就是**因特网**（Internet，注意首字母是大写的），它起源于20世纪60年代初期的研究项目。它的目标是开发出将许多计算机网络连接起来的能力，以便它们能作为一个连接的系统发挥作用，而这个系统不会受局部灾难的干扰。这项工作的大部分是由美国政府资助并通过美国国防部高级研究计划局（Defense Advanced Research Projects Agency，DARPA，读作“DAR-pa”）发起的。这些年来，因特网的开发已经从政府资助项目转变成了学术研究项目，而且如今它在很大程度上已经是商业项目了，用于连接全世界的个人域网、局域网、城域网及广域网，涉及数百万台计算机。

基本知识点

- 因特网连接着全世界的设备和网络。
- 网络和基础设施得到了商业和政府举措的支持。

4.2.1 因特网体系结构

我们已经提到，因特网是相连网络的集合。总体上，这些网络的构建和维护是由称作**因特网服务提供方**（Internet service provider，ISP）的组织来完成的。就网络本身而言，通常也习惯使用术语ISP来表示。因此，当说到要连接到一个ISP时，我们真正的意思是连接到由ISP所提供的网络。

由ISP运营的网络系统可以按照各网络在整个因特网结构中所起的作用分类成一个层次结构（见图4-7）。整个层次结构的顶部是数量相对较少的**第一层ISP**（tier-1 ISP），这些ISP拥有非常高速的、大容量的国际化广域网。这些网络被看成因特网的主干，它们通常是由通信行业中的大公司来运营的。例如，有一家公司最初是传统的电话或有线电视公司，后来它把业务范围扩展到提供其他通信服务。

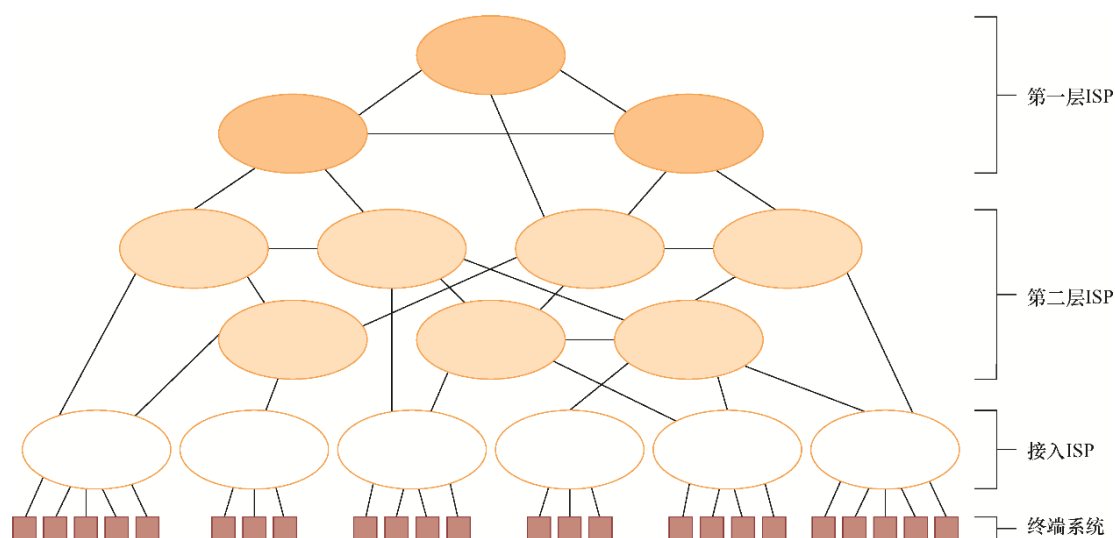


图4-7 因特网的构成

与第一层ISP连接的是**第二层ISP**（tier-2 ISP），第二层ISP往往是区域性的，在能力上更弱一些。（第一层ISP和第二层ISP的区别通常是仁者见仁，智者见智。）此外，这些网络通常也是由通信行业的公司运营的。

第一层ISP和第二层ISP本质上是路由器的网络，集中提供因特网的通信基础设施。路由器之间的许多可能路径被认为是因特网**冗余**（redundancy）的核心，这意味着，即使非常重要的第一层路由器失

效，大多数或全部网络仍然是连通的。通常由称为**接入ISP**（access ISP）或**第三层ISP**（tier-3 ISP）的中间商提供接入这个核心的服务。接入ISP本质上是独立的互联网，有时也称为**内联网**（intranet），由向个人家庭和企业提供因特网接入业务的机构来运营。这些机构包括通过提供服务收取服务费的有线电视和电话公司，另外还包括一些大学或者公司等组织，它们自行为组织内部的个人用户提供因特网接入服务。

将个人用户连接到接入ISP的设备称为**终端系统**（end system）或者**主机**（host）。这些终端系统可能是笔记本电脑或PC，但随着其应用范围的不断扩大，还可能是其他很多种设备，包括电话、摄像机、汽车和家用电器。毕竟，因特网本质上是一个通信系统，因此任何可以从与其他设备通信中获益的设备都是潜在的终端系统。

终端系统与大型网络连接的技术也是多种多样的。或许，发展最快的是基于WiFi技术的无线连接。策略是将接入点连接到接入ISP，从而在接入点的广播范围内通过ISP向终端系统提供因特网接入。接入点或一组接入点范围内的区域通常称为**热点**（hot spot），尤其是当这些区域的网络接入公开可用或者免费的时候。在个人住宅、酒店、写字楼、小型企业、公园里都可以找到热点，并且在某些情况下，热点会遍布整座城市。目前移动电话行业也使用相似的技术，在移动电话行业中热点就是我们所知的服务区，当终端系统从一个服务区移动到另一个服务区时，通过调整产生服务区的“路由器”来提供连续的服务。

连接到接入ISP的其他常见技术使用的是电话线或者电缆/卫星系统。这些技术可以用来提供到单个终端系统的直接连接，或者到多个终端系统被连接到的用户路由器的直接连接。后一种方法在个人住宅中很流行，利用现有的电缆或者电话线，通过连接到接入ISP的路由器/接入点形成本地热点。

因特网工程任务组

因特网工程任务组（Internet Engineering Task Force，IETF）成立于20世纪80年代，负责研发和共享因特网协议标准。最初是美国联邦政府的一个项目，自20世纪90年代初以来，因特网工程任务组一直是由志愿者组成的国际组织。使因特网如此成功的开放标准需要随着网络的成长和发展定期改进。因特网工程任务组监督共识驱动方法，用于标准化和修订处理命名、寻址、路由以及许多流行因特网应用的协议。从因特网工程任务组的官方网站可以了解更多信息。

允许因特网为这么多种设备和技术提供通信服务的因特网设计的方方面面，使因特网成为**端到端体系结构**（end-to-end architecture）。因为终端设备的固定假设特别少，所以因特网能随着新型网络 and 设备的普及而快速支持它们。

基本知识点

- 端到端体系结构方便连接因特网上的新设备和网络。

20世纪的电话、有线电视和卫星广域网都被设计为传送模拟通信信号，如人的语音或预数字（pre-digital）电视信号。现代网络可以被

设计成直接在两台计算机之间传送数字数据，但较旧的模拟网络基础设施仍然是因特网的一个重要组成部分。这些传统模拟链接所产生的问题通常被统称为**最后一公里问题**。通过高速数字技术（如光纤），广域网、城域网和许多局域网的主干线比较容易实现现代化，但替换那些现有的将主干线连接到个人家庭或办公室的铜电话线和同轴电缆的费用就要昂贵得多。因此，源自远离终端系统的因特网大陆的信息，几乎整个旅程可能都在高速数据连接上，就在要到达终端系统的这“最后一公里”时要穿越一根慢速的有百年历史的模拟电话线。第2章中提到过，已经开发了几种巧妙的方案，用于扩展这些传统的模拟链路，以适应数字数据的传输。DSL调制解调器、电缆调制解调器、卫星上行链路，甚至是直接光纤连接入户都被用于将宽带因特网接入终端用户。

4.2.2 因特网编址

如4.1节所介绍的，互联网需要一个互联网范围的编址系统，为该系统中的每台计算机分配一个唯一的标识地址。在因特网中，这些地址称为**IP地址**（IP address）。（术语IP的全称是internet protocol，指的是因特网协议，这个术语在4.4节中会详细介绍。）最初，每一个IP地址都是32位的模式，但是为了提供更大的地址集，人们现在正计划将其转换为128位地址（见4.4节中关于IPv6的介绍）。**因特网名称与数字地址分配机构**（Internet Corporation for Assigned Names and Numbers，ICANN）是一家致力于协调因特网的运营的非营利性的国际组织，它将连续编号的IP地址块授予因特网服务提供方。然后，因

特网服务提供方就可以将其被授权范围内的地址块中的IP地址分配给其管辖范围内的机器，给客户组织的通常是更小的连续编号的地址块。因此，整个因特网上的机器都被分层分配了唯一的IP地址。

基本知识点

- IP地址是分层的。
- 通过分配IP地址可以使设备连接到因特网上。

IP地址通常是采用**点分十进制记数法**（dotted decimal notation）书写的，其中地址的每个字节用圆点分隔，每个字节用传统的十进制记数法所表示的整数表示。例如，使用点分十进制记数法，位模式5.2将可以表示2字节位模式0000010100000010，其中依次包含字节00000101（5的表示）和字节00000010（2的表示）；而位模式17.12.25将可以表示3字节的位模式，其中依次包含字节00010001（用二进制记数法写的17）、字节00001100（用二进制记数法写的12），以及字节00011001（用二进制记数法写的25）。总之，当用点分十进制记数法表示时，32位的IP地址可以表示为192.207.177.133。

用位模式形式表示的地址（即使采用点分十进制记数法压缩）很难为人们所用。基于这个原因，因特网拥有另一种编址系统，利用助记名来标识机器。该编址系统是基于**域**（domain）的概念建立的，域可被看作是单个机构（如大学、俱乐部、公司或者政府机构）运营的因特网“区域”。（此处区域一词加了引号，这是因为它可能不同于因特网的物理区域，稍后我们将看到这一点。）每一个域都必须在

ICANN进行注册，这一过程由称为**注册商**（registrar）的公司处理，注册商由ICANN指定。作为注册流程的一部分，助记**域名（domain name）**会分配给域，域名在整个因特网的所有域名中是唯一的。域名通常是注册域的机构的描述，从而提高它们对人类的实用性。

例如，马凯特大学（Marquette University）的域名是mu.edu。需要注意的是点后面的后缀，它用于反映域的分类，在这个域名中，后缀edu就表明了它是一家教育机构。这些后缀被称为**顶级域名**（Top-Level Domain，TLD）。其他顶级域名包括表示商业机构的.com、表示政府机构的.gov、表示非营利组织的.org、表示军事的.mil以及最初打算用于表示ISP但是现在使用范围更广的.net。在因特网顶级域名的早期，有一个默认假设——被连接的组织位于美国。但是，因特网迅速发展到现在包括全世界，于是就为每个特定的国家或地区创建了新的2个字母的顶级域名，称为**国家或地区代码顶级域名**，如表示澳大利亚的.au和表示加拿大的.ca。21世纪，DNS层次结构中添加了数百个新的顶级域名，如表示企业的.biz或表示博物馆的.museum。

一旦一个域的助记名被注册，注册该域名的组织就可以在域内自由扩展该名称，获得域内各个项的助记标识符。例如，马凯特大学内的某台主机可以标识为eagle.mu.edu。注意，分层域名是向左扩展的，并以点分隔。在一些情况下，被称为**子域**（subdomain）的多个扩展被用作组织域内名称的手段。这些子域通常代表域管辖内的不同网络。例如，如果A公司被分配的域名为A.com，它的某台计算机的域名就可以为overthruster.propulsion.A.com，其含义是计算机

overthruster在顶级域名.com的A域内的propulsion子域中。

（我们应该强调的是，助记地址中使用的点分表示法与位模式形式的地址中使用的点分十进制记数法没有关系。）

基本知识点

- 域名语法是分层的。

虽然助记地址对人类来说比较方便，但是因特网中还是使用IP地址来传送消息。因此，如果某人想要给远程机器发送消息并通过助记地址来标识目的地，那么使用的软件必须能够在发送消息之前将助记地址转换成IP地址。这种转换是在许多称为**名称服务器**（name server）的服务器的帮助下完成的，这些名称服务器本质上是向客户机提供地址转换服务的目录。总体来说，这些名称服务器被用作因特网范围的目录系统，称为**域名系统**（domain name system，DNS）。使用域名系统进行转换的过程称为**域名系统查找**（DNS lookup）。随着域、DNS服务器和因特网主机数量的增长，DNS的分层性质已成为其扩展能力的重要因素。因特网上的主机不需要知道或维护所有域名的完整的世界范围的映射就能执行DNS查找。相反，主机只需进行初始配置就能找到它的本地DNS服务器，该服务器通常由接入ISP提供。DNS协议允许对不熟悉域的DNS查找从涉及顶级域名的根服务器开始进行。尝试定位不熟悉的lectroid.propulsion.A.com服务器的主机可以从已知的.com名称服务器开始，该服务器将查询提交给A.com名称服务器，A.com名称服务器将查询提交给propulsion.A.com名称服务器，它可以将主机名权威地转换为IP地

址。在实践中，高速缓存DNS服务器可以回答许多常见的DNS查找，不必每次都找权威的DNS服务器，既节省了时间又减少了不必要的网络流量。

基本知识点

- 域名系统（DNS）将域名转换成IP地址。
- DNS中的层次结构有助于系统扩展。

因此，如果一台机器可以通过助记域名连接，那么这个域名必须在DNS内的名称服务器中存在。在建立域的实体拥有资源的情况下，可以建立并维护其自己的包含该域内所有名称的名称服务器。事实上，这就是域名系统最初构建所用的模型。每个注册域都代表一个地方当局运营的一个因特网的物理区域（如公司、大学或者政府机构）。这个当局本质上就是一个接入ISP，通过它自己连接到因特网的内联网向其成员提供因特网接入。作为这个系统的一部分，组织维护自己的名称服务器，为其域中使用的所有名称提供转换服务。

目前，这种模式依然很常见。然而，许多个人或者小型组织想要建立展示在因特网上的域，但又不想承担必要的支持资源。例如，一家本地国际象棋俱乐部以Kings***Queens.org的身份出现在因特网上可能是有好处的，但是俱乐部可能不想建立自己的网络、维护从这个网络到因特网的链接、实现自己的名称服务器。在这种情况下，俱乐部可以和接入ISP签订合同，使用因特网服务提供方已建好的资源实现注册域的展示。俱乐部一般可能会在因特网服务提供方的协助下

注册俱乐部选好的域名，并与因特网服务提供方签订合同，将域名放入因特网服务提供方的名称服务器。这就意味着所有关于新域名的域名系统查找都会被指向因特网服务提供方的名称服务器，从而获取正确的域名转换。通过这种方式，许多已注册的域就可以驻留在单个ISP中，每个已注册的域经常只占用一台计算机的一小部分。这整个过程几分钟就能完成，只需要几美元，很多因特网站点都能办理。

4.2.3 因特网应用

在因特网发展的早期，大多数应用都是分开的简单程序，每个应用都遵循一个网络协议。新闻阅读器应用使用**网络新闻传送协议**（network news transfer protocol，NNTP）联系服务器，用于通过网络来列出和复制文件的应用实现了**文件传送协议**（file transfer protocol，FTP），用于访问远距离的另一台计算机的应用使用了**远程登录**（telnet）协议，后来又有了**安全外壳**（secure shell，SSH）协议。随着Web服务器和浏览器越来越成熟，越来越多的传统网络应用开始通过网页和强大的**超文本传送协议**（hyper text transfer protocol，HTTP）来处理。然而，在刚开始研究因特网应用的网络协议时，我们有必要在下一节介绍HTTP之前先介绍几个比较简单的例子。

无线电话的演进

自从20世纪80年代首批简单的手持电子设备推出以来，移动电话技术得到了迅速的发展。自那时起，基本上每10年手机技术就会出现一个全新的浪潮，这一个个浪潮引领我们来到了现在这个复杂的多功

能智能手机时代。第一代无线电话网络通过空气传输模拟语音信号，与传统的电话系统非常相似，只是没有穿墙而过的铜线。我们把这些早期的电话称为“1G”，即第一代。第二代使用数字信号对语音编码，能够更高效地使用无线电波，还能够传输其他种类的数字数据，如文本消息处理。第三代（3G）电话网络提供了更高的数据速率，支持手机视频通话和其他带宽密集型活动。4G网络提供了更高的数据速率和一个全分组交换IP网络，它允许智能手机享受从前只有支持宽带的PC才有的连接性和灵活性。

1. 电子邮件

现在有很多系统可以通过网络交换两个终端用户之间的消息，文本消息处理、基于浏览器的在线聊天、基于Twitter的“tweet”以及Facebook的“wall”就是其中的几个。因特网最古老、最持久的用途之一就是电子邮件系统，简称**电子邮件**（email）。虽然许多用户现在依靠他们的浏览器或者复杂的应用程序（如微软公司的Outlook、苹果公司的Mail或者Mozilla公司的Thunderbird）来阅读和撰写他们的电子邮件，但电子邮件信息在因特网上从一台计算机实际发送到另一台计算机时，使用的仍然是像简单邮件传送协议这样的基本网络协议。

简单邮件传送协议（simple mail transfer protocol，SMTP）定义了一种方式，当从一台主机向另一台主机发送电子邮件信息时，网络上的两台计算机可以通过这种方式交互。考虑**邮件服务器**（mail server）mail.skaro.gov将一封电子邮件从终端用户“dalek”发送给

tardis.edu域中的终端用户“doctor”的例子。首先，mail.skaro.gov上的一个邮件处理进程联系mail.tardis.edu上的邮件服务器进程。为此，它使用域名系统（另一个网络协议）将人类可读的目的域名映射为正确的邮件服务器名，然后再映射到它的IP地址。这与拨号前查找一个熟人电话号码没有什么不同。同样，当另一端的服务器进程应答时，协议规定，它必须向调用者标识自己。它们的SMTP交换的文字记录可能看起来像下面这样：

```
1 220 mail.tardis.edu SMTP Sendmail Gallifrey-1.0; Fri, 23
   Aug 2413 14:34:10
2 HELO mail.skaro.gov
3 250 mail.tardis.edu Hello mail.skaro.gov, pleased to meet you
4 MAIL From: dalek@skaro.gov
5 250 2.1.0 dalek@skaro.gov... Sender ok
6 RCPT To: doctor@tardis.edu
7 250 2.1.5 doctor@tardis.edu... Recipient ok
8 DATA
9 354 Enter mail, end with "." on a line by itself
10 Subject: Extermination.
11
12 EXTERMINATE!
13 Regards, Dalek
14 .
15 250 2.0.0 r7NJYAE1028071 Message accepted for delivery
16 QUIT
17 221 2.0.0 mail.tardis.edu closing connection
```

在第1行，远程邮件服务器进程在应答调用者时，宣布了它的名字、它所使用的协议以及其他选项信息，如协议版本、日期和时间。在第2行，发送邮件服务器进程介绍了它自己。在第3行，远程服务器确认了发送服务器的名字。

大多数因特网协议都不必用ASCII字符发送，所以很容易被人理解。当然，第3行中的那个古怪的礼貌用语“pleased to meet

you”既不会受这个连接两端的软件的欣赏，也不是SMTP的正常功能所必需的。开头的数字响应“250”足以传达前一行已被接受、交换可以继续。但是，这是一个流行的SMTP邮件服务器内置的实际行为，是因特网初期留下来的，那时人工操作员需要频繁地通过审查SMTP记录副本来调试两个邮件服务器之间的不兼容性。这样的交换每天在网络上发生无数次，只有负责在整个因特网上传输电子邮件的软件代理才知道确切的数字。

在这个最简单的案例中，远程邮件服务器将听信于第2行中的mail.skaro.gov，接受发送服务器是来自skaro.gov域的名称为mail的机器。SMTP是建立在信任基础上的典范，但后来被垃圾邮件发送者和其他因特网的不法之徒滥用。现代邮件服务器必须使用SMTP的扩展版本或者与其同等的协议来帮助确保电子邮件被安全发送。在本章的最后一节，我们将会对有关网络安全性的问题进行更详细的讨论。

基本知识点

- 因特网的信任模型需要权衡。

返回到文字记录，在第4行，发送服务器宣布它有一个邮件信息要传递，并标识了这个发送用户。在第5行，远程服务器确认它将接收来自这个域的这个用户的邮件。在第6行，发送服务器宣布了远程服务器上的收件人。在第7行，远程服务器确认它将接收发往那个用户的电子邮件。在第8行，发送服务器免除了介绍，并宣布它准备发送DATA，

即电子邮件信息的实体。在第9行，远程服务器（用代码354，根据SMTP）确认它已准备好接收该信息体，此外这一行还包含了一个有用的关于如何结束该消息传送的人类可读指令。

在第10～14行，发送服务器传送要交付的电子邮件信息文本。远程服务器在第15行确认接受，在第17行确认发送服务器在第16行宣布的QUIT。

描述SMTP的技术文档定义了像上面文字那样的对话中的每个允许的步骤。对于关键字HELO、MAIL、RCPT、DATA和QUIT将被如何发送、可以带哪些选项以及应该被如何解释，都做了精确的定义。同样，远程服务器用于确认的数字响应码也是枚举定义的。软件设计者使用协议描述来开发可正确实现通过网络发送和接收电子邮件的算法。

电子邮件传输的其他方面采用了其他协议。因为SMTP最初是为传送ASCII编码的文本信息而设计的，所以又开发了其他协议，如[多用途因特网邮件扩展](#)（multipurpose Internet mail extensions，MIME），用于将非ASCII编码的数据转换成SMTP兼容的格式。

有两种流行的协议可以用于访问已到达并积累在用户邮件服务器里的电子邮件，它们是[邮局协议第3版](#)（post office protocol version 3，POP3）和[因特网邮件访问协议](#)（Internet mail access protocol，IMAP）。二者中，POP3（读作“pop-THREE”）较为简单。使用POP3，用户可以向其本地计算机传送（下载）信息，在本地计算机

中，这些信息可以被读取、被存储在不同的文件夹中、被编辑，还可以按照用户的需要进行其他操作。这些都是通过使用本地机器的海量存储器在用户的本地机器上完成的。IMAP（读作“EYE-map”）允许用户在与邮件服务器相同的机器上存储和操作信息以及相关的资料。用这种方式，那些必须从不同计算机上访问邮件的用户，就可以将保留记录在邮件服务器上，然后这些记录就可以通过任何一台用户可以访问的远程计算机来访问了。

2 . VoIP

下面来看一个较近的因特网应用的例子——**互联网电话**（voice over Internet protocol，VoIP），它利用因特网基础设施提供与传统电话系统类似的语音通信。最简单的形式下，VoIP由不同机器上的两个进程构成，这些机器通过P2P模式传送音频数据——这一过程本身没有明显的问题。但是，初始化和接收呼叫，把VoIP与传统电话系统对接，以及提供像紧急911通信这样的服务，诸如此类的任务都是超出了传统因特网应用范畴的问题。此外，在美国，拥有国家传统电话公司的政府把VoIP看成一种威胁，要么对其征收重税，要么将其完全取缔。

现在有4种不同形式的VoIP系统在竞争。VoIP**软电话**（soft phone）由P2P软件构成，允许两个或多个PC共享一个电话，所需硬件只是一个扬声器和一个麦克风。VoIP软电话系统的一个例子就是Skype，它还为客户提供与传统电话通信系统的链接。Skype的一个缺点是，它是一个专有系统，因而它的许多操作结构是不对外公开的。

这意味着，Skype用户必须要在没有第三方证明的基础上相信Skype软件的完整性。例如，为了接收呼叫，Skype用户必须把他的PC连接到因特网，供Skype系统使用，这意味着，在PC机主毫无意识的情况下，PC的一些资源可能会被用来支持其他的Skype通信（这一功能已经引起了一些抵触）。

第二种形式的VoIP由**模拟电话适配器**（analog telephone adapter）构成，模拟电话适配器允许用户将其传统电话连接到由某个接入ISP提供的电话服务上。这一选择经常与传统的因特网服务和/或数字电视服务捆绑。

第三种形式的VoIP是以嵌入式VoIP电话的形式出现的，嵌入式VoIP电话把传统电话替换成了直接连接到TCP/IP网络上的等效的手持设备。嵌入式VoIP电话在大型组织中越来越常见，很多大型组织都在将其传统的内部铜线电话系统替换为基于以太网的VoIP，以降低成本和增强功能。

最后，目前这一代智能手机使用的是无线VoIP技术。也就是说，早期的几代无线电话仅使用特定公司的协议与该电话公司的网络通信。通过该公司的网络和因特网间的网关，我们便可以接入因特网，而信号会在那儿被转换为TCP/IP系统。然而，4G电话网络是一个完全基于IP的网络，即4G电话本质上只是全球因特网上接入宽带的另一台主机。

3 . 因特网多媒体流

当前因特网上的音频和视频的实时传输占用了相当大的一部分因特网流量，这种传输被称作**流**（streaming）。仅Netflix一家网站在2013年的前3个月就为终端用户流式传输了40亿小时的节目，到2016年，占全北美因特网流量的三分之一以上。到2017年，音频和视频流服务消耗了70%多的因特网传输容量。

从表面上看，因特网流似乎不需要特殊的考虑。例如，人们可能猜想，一个因特网无线站点仅需建立一台能把节目消息发送给请求它们的每个客户机的服务器即可。这种技术称作**N单播**（N-unicast）。（严格地说，**单播**是指一个发送者向一个接收者发送消息，而**N单播**是指一个发送者参与多个单播。）N单播方法已经投入实际使用，但这种方法有缺陷，它把大量的负担放在了站点服务器和与服务器紧邻的因特网邻居上。实际上，N单播强迫服务器按照实时基准把各条消息发送给它的每个客户机，而所有这些消息必须再由服务器的邻居们转发。

N单播的大多数替代方案都是试图缓解这个问题。其中一种替代方案是使用过去的文件共享系统方法中的P2P模式。也就是说，一旦一个对等体接收到数据，它就开始把数据分发给那些仍在等待的对等体，这意味着许多分发问题从数据源转移到了对等体。

另一种替代方案称为**多播**（multicast），它把分发问题转移给了因特网路由器。使用多播，服务器通过单个地址把一个消息传送给多个客户机，依赖因特网中的路由器来识别这个地址的含义，产生消息的副本并将其转发到合适的目的地。注意，依赖多播的应用需要因特

网路由器具有超过它原来职责范围的功能。多播支持在小型网络中已经实现，但还没有扩展到全球因特网的终端用户。

更重要的是，这一类中的大多数应用现在都是**点播流**（on-demand streaming），点播流中的终端用户希望能在自己选定的任意时间观看或收听媒体。这是一个与因特网无线站点截然不同的问题，因为每个终端用户都希望能够根据自己的节奏来开始、暂停或回放内容。在这种情况下，N单播和多播技术帮不上什么忙。每个点播流就是能有效地从存储着内容的媒体服务器到希望得到它的终端用户的单播。

为了将这种类型的流扩展到几千甚至几百万的并发用户，使每个用户都有自己的个人流，必须将内容复制到许多不同的服务器上。大规模流式服务利用**内容分发网络**（content delivery network，CDN）——战略性地分布在因特网上的服务器组，专门将内容副本流式传输到附近的在服务器组的网络“邻居”中的终端用户。在许多情况下，CDN机器可驻留在一个接入ISP网络中，允许那个因特网接入服务提供商的客户通过流从附近的服务器（在网络中，这台服务器比流服务器的中央服务器距客户的距离近得多）高速下载多媒体内容副本。有一种称为**任播**（anycast）的网络技术，能使终端用户自动连接到一个规定的服务器组之外的最近一台服务器，让CDN更实用。

因特网高清点播视频流的渗透已远超传统PC。一大类嵌入式设备，如各种电视机、DVD/蓝光播放机、智能手机、游戏控制台，现在都可以直接连接到TCP/IP网络，以便从众多的免费服务器和订阅服务器中选择可观看的内容。

4.2节问题与练习

1. 第一层ISP和第二层ISP的作用是什么？因特网接入服务提供商的作用是什么？
2. 什么是DNS？
3. 在点分十进制记数法中，3.6.9代表什么位模式？用点分十进制记数法表示位模式0001010100011100。
4. 计算机在因特网中的助记地址（如 overthruster.propulsion.yoyodyne.com）的结构在哪方面与传统的邮寄地址相似？在IP地址中会出现同样的结构吗？
5. 列出因特网上发现的3种服务器，并说出每种用途。
6. 协议描述了网络通信的哪些方面？
7. 因特网无线广播的P2P和多播方法与N单播在广播方式上有何不同？
8. 当要在4种VoIP中选择一种时，应该考虑什么标准？

4.3 万维网

万维网源于蒂姆·伯纳斯-李（Tim Berners-Lee）的工作，他意识到了将互联网技术与称为**超文本**（hypertext）的链接文档的概念相结合潜力。他于1990年12月发布了第一个实现万维网的软件。虽然这个早期原型还不支持多媒体数据，但它包括了我们现在公认的万维网的关键组成部分：超文本文档格式，用于把**超链接**（hyperlink）嵌入其他文档中；协议，用于通过网络传送超文本；服务器进程，用于根据

请求提供超文本页面。从这个不起眼的软件开始，万维网迅速成长为支持图像、音频和视频的系统，到20世纪90年代中期，已经成为推动因特网发展的主要应用。

4.3.1 万维网实现

允许用户访问因特网上超文本的软件包分为两类：**浏览器**（browser）和**万维网服务器**（webserver）。浏览器安装在用户的计算机上，负责获取用户请求的资料，并将这些资料条理清晰地展示给用户。常见的因特网浏览器包括Chrome、Firefox、Safari和Internet Explorer。万维网服务器驻留在含有待访问的超文本文档的计算机中。它的任务是根据客户机（浏览器）的请求提供对机器里文档的访问权。常见的Web服务器软件包包括Apache、微软IIS和Nginx。超文本文档通常使用超文本传送协议（hypertext transfer protocol，HTTP）在浏览器与万维网服务器之间传输。

为了在万维网上定位及检索文档，每个文档都被赋予了一个唯一的称为**统一资源定位地址**（uniform resource locator，URL）的地址。每个URL都包含浏览器要联系正确服务器和请求所需文档所需要的信息。因此，要浏览网页，人们首先要给浏览器提供所需文档的URL，然后才能要求该浏览器检索和显示这个文档。

图4-8给出了一个典型的URL，它包含以下4段：用于与控制文档访问的服务器进行通信的协议，服务器所在机器的助记地址，服务器

在查找包含该文档的目录时所需的目录路径，以及该文档本身的名字。简言之，图4-8中的URL告知浏览器：使用HTTP协议与称为eagle.mu.edu的主机上的万维网服务器联系，并检索名为Julius_Caesar.html的文档，该文档存放在authors目录的Shakespeare子目录中。

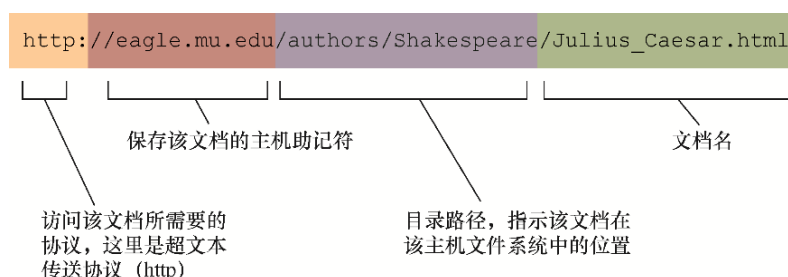


图4-8 一个典型的URL

有时，URL可能不会明确包含图4-8中所示的所有段。例如，如果服务器不需要根据目录路径就能读到那个文档，那么URL中就不会出现目录路径。此外，有时一个URL只包含一个协议以及一台计算机的助记地址。在这些情况下，该计算机的万维网服务器将返回一个预订的文档，该文档一般称为主页，通常描述该网站上可用的信息。这种缩短的URL提供了一种简单的联系机构的方法。

万维网联盟

万维网联盟（World Wide Web Consortium，W3C）创建于1994年，宗旨是通过开发协议标准（称为W3C标准）来促进万维网的发展。W3C总部设在瑞士日内瓦欧洲核子研究组织（CERN）高能粒子物理实验室。CERN是原来的HTML标记语言和用于在因特网上传送

HTML文档的HTTP协议的诞生地。今天的W3C制订了许多标准（包括用于XML和许多多媒体应用的标准），这些标准使得大量因特网产品彼此兼容。

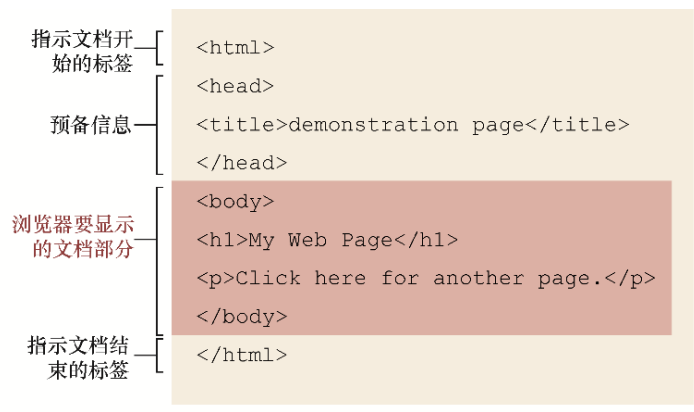
为了进一步简化网站定位，许多浏览器都假定：如果没有明确说明协议，就使用HTTP协议。

4.3.2 HTML

传统的超文本文档类似于文本文件，因为它的正文是使用诸如ASCII或者Unicode这样的系统一个字符接一个字符地编码的。区别是，超文本文档还包含称为**标签**（tag）的特殊符号，用于描述该文档应该如何呈现在显示屏上，该文档还需要什么多媒体资源（如图像），以及该文档中的哪些项被链接到了其他文档上。这个标签系统称为**超文本标记语言**（hypertext markup language，HTML）。

因此，按照HTML的要求，网页的作者要描述浏览器所需的信息，使浏览器能够将该页呈现在用户的屏幕上，并找到当前网页所引用的任何相关文档。该过程类似于向纯输入文本中加入排版说明（也许是用红笔），以便排版人员知道文档最后应该以什么形式出现。对于超文本，HTML标签就是红色记号，浏览器最终扮演了排版人员的角色，读HTML标签就会知道文本应以什么方式呈现在浏览器屏幕上。

图4-9a给出了一个极其简单的网页的HTML编码版本，称为源（source）版本。注意，标签是用符号“<”和“>”来划定的。HTML源文档由两部分组成——头（在<head>和</head>标签之间）和体（在<body>和</body>标签之间）。网页的头和体之间的区别类似于各办公室间备忘录的头和体之间的区别，两者都是头包含文档的预备性信息（如备忘录的日期、主题等），体包含文档的实质内容，对于网页就是显示页面时计算机屏幕上呈现的内容。



(a) 用HTML编码的页面



(b) 显示在计算机屏幕上的页面

图4-9 一个简单的网页

图4-9a给出的网页头只包含了该文档的标题（在两个“title”标签之间）。该标题只用于文档编制目的，不会显示在计算机屏幕上。要显示在浏览器屏幕上的页面部分，但是有一些现代的浏览器会将标题列

在浏览器窗口或标签的顶部。浏览器屏幕上显示的内容包含在该文档体中。

图4-9a所示的文档体的第一个条目是一个包含文本“My Web Page”的一级标题（在<h1>和</h1>标签之间）。一级标题意味着浏览器要将该文本明显地呈现在屏幕上。文档体的下一个条目是一个包含文本“Click here for another page.”的文本段落（在<p>和</p>标签之间）。图4-9b给出的是浏览器展现的页面。

在这种展现形式中，图4-9所示的页面现在是没有实际意义的，因为当用户单击单词here时什么也不会发生，虽然页面上暗示了这么做会使浏览器显示另外一个页面。为了引发相关的动作，我们必须将单词here链接到另一个文档上。

假设单击单词here时，我们打算让浏览器检索并显示URL为http://c****y.com/demo.html的页面。为此，我们必须先用锚标签<a>和在该网页源版本中将单词here括住。在开锚标签（opening anchor tag）里，我们插入参数

href = http:// c****y.com/demo.html

表示与该标签相关的超文本引用（href）是跟在等号后面的URL（http://c****y.com/demo.html）（如图4-10a所示）。加上锚标签后，网页将会像图4-10b所示的那样呈现在浏览器屏幕上。注意，它与图4-9b是一样的，只是单词here用颜色突出显示了，表明

它是到另一个网页的链接。单击这类突出显示的词会使浏览器检索并显示相关的网页。因此，网页是通过锚标签链接到彼此的。

```
<html>
<head>
<title>demonstration page</title>
</head>
<body>
<h1>My Web Page</h1>
<p>Click
  <a href="http://crafty.com/demo.html">
    here
  </a>
  for another page.</p>
</body>
</html>
```

含有参数的锚标签 — [

闭锚标签 — [

(a) 用HTML编码的页面



(b) 显示在计算机屏幕上的页面

图4-10 一个增强的简单网页

最后，我们来说明一下如何将图像加到这个简单的网页里。为此，假设要插入的图像的JPEG编码是存储在I***s.com的Images目录中的文件OurPic.jpg，且对那一位置的万维网服务器可用。在这些条件下，我们可以通过在HTML源文档的<body>标签后插入图像标签<img src =

"http://I****s.com/Images/OurPic.jpg">，告诉浏览器在该网页的顶部显示该图像。这告诉浏览器的是，名为OurPic.jpg的图像应该显示在该文档的开头。（src是“source”的简写，意思是等号后面的信息表示的是要显示的图像的来源。）浏览器发现这个标签就会给位于I****s.com的HTTP服务器发送一条报文，请求名为OurPic.jpg的图像，然后显示该图像。

如果我们将该图像标签移到文档的后面，恰好放在</body>标签的前面，那么浏览器就会将图像显示在网页的底部。当然，在网页上定位图像还有许多更复杂的技巧，但是现在我们不必关心这些问题。

4.3.3 XML

HTML本质上是一个符号系统，文本文档及其外观都可以通过它编码成一个简单的文本文件。同理，我们也可以将非文本内容编码成文本文件，如乐谱。乍一看，传统上表示音乐的“五线谱、小节线和音符”模式并不符合文本文件规定的一个字符接着一个字符的格式。不过，我们可以通过开发另一种符号系统来解决这个问题。更准确地说，我们可以规定用<staff clef = "treble">表示五线谱的开始，用</staff>表示五线谱的结束，用<time> 2/4 </time>这个形式表示拍号，用<measure>和</measure>分别表示小节的开始和结束，用<notes> eighth C </notes>表示八分音符C，等等。那么，下面的文本就可以用于编码图4-11所示的音乐：

```
<staff clef = "treble"> <key>C minor</key>
<time> 2/4 </time>
<measure> <rest> eighth </rest> <notes> eighth G,
eighth G, eighth G </notes></measure>
<measure> <notes> half E </notes></measure>
</staff>
```

使用这种符号，乐谱就可以作为文本文件编码、修改、存储和在因特网上传送。此外，还可以编写软件，将这类文件的内容以传统乐谱的形式表现出来，甚至可以用一个音乐合成器来演奏这个音乐。

注意，我们的乐谱编码系统沿用了HTML使用的风格。对于标识组成部分的标签，我们选择了符号“<”和“>”来划定标签。我们选择用同名标签指示结构（如五线谱、音字符串或者小节）的开始和结束，其中结束标签用斜线指定（以<measure>开始的就以</measure>结束）。我们选择用诸如clef= "treble"这样的表达式来指示标签中的特殊属性。这种风格还可以用于开发表示其他格式（如数学表达式和图形）的系统。

可扩展标记语言（eXtensible Markup Language，XML）是一种标准化的风格（类似于上面乐谱的例子），用于设计将数据表示为文本文件的符号系统。事实上，XML是从一套比较老的称为标准通用标记语言（Standard Generalized Markup Language，SGML）的标准中派生出来的简化版。遵照XML标准，人们已经开发出一批称为**标记语言**（markup language）的符号系统，用于表示数学、多媒体演示和音乐。事实上，HTML是一种基于XML标准开发的用于表示网页的标记语言。（实际上，HTML的原始版本在XML标准巩固之前就已经开发

出来了，因此HTML的一些特征没有严格遵守XML。正是这个原因，我们可能需要参考XHTML，它是严格遵守XML的HTML版本。）

关于如何设计标准以获得广泛应用，XML是个很好的范例。为了编码各种类型的文档，我们不应设计单独的、无关联的各种标记语言，XML所表示的方法就是要开发一种通用标记语言标准。有了这个标准，就可以为各种应用开发不同的标记语言。以这种方式开发出来的标记语言具有一致性，它们可以组合起来获得复杂应用的标记语言，如包含乐谱片段和数学表达式的文本文档。

最后，要说明的是，XML允许开发与HTML不同的新的标记语言，因为新的标记语言强调的是语义而不是外观。例如，使用HTML可以标记菜谱里的配料，使它们以列表形式出现，每一种配料单独占一行。不过，如果我们使用面向语义的标签，菜谱里的配料就可以标记为配料（也许是用标签<ingredient>和</ingredient>）而不只是列表中的各个项。这个区别很细微但很重要。语义方法使[搜索引擎](#)（search engine）（协助用户定位与关注主题相关的资料的网站）能够确定菜谱包含或者不包含某些配料，这将对现有技术的一项重大改进，因为现有技术只能分离出包含或者不包含某些单词的菜谱。更准确地说，如果使用语义标签，搜索引擎就可以找到不包含菠菜的卤汁宽面的菜谱，而只根据单词内容进行的类似搜索就会跳过以“这个卤汁宽面不包含菠菜”开始的菜谱。反过来，如果根据语义而不是词本身使用因特网范围的标准来标记文档，那么创建的就是万维[语义网](#)，而不是现在使用的万维[语法网](#)。

4.3.4 客户端活动和服务器端活动

现在我们来考虑一下，检索图4-10所示的简单网页，并将其显示在浏览器所在的计算机屏幕上，需要哪些步骤。首先，扮演客户机角色的浏览器要使用URL（也许是从使用该浏览器的人那里获得）里的信息，与控制对该网页访问的万维网服务器建立联系，请求它传送该页面的一个副本。服务器通过将图4-10a中显示的文本文档发送给浏览器作为响应。接着，浏览器解释该文档中的HTML标签，以确定如何显示该页面，并将文档呈现在计算机屏幕上。浏览器的用户就将看到类似于图4-10b所描述的图像。如果用户接着用鼠标单击单词here，浏览器会使用相关联的锚标签中的URL联系相应的服务器，以获得并显示另一个网页。总之，整个过程就是浏览器按照用户的要求获取并显示网页。

但是，如果我们想要带有动画的网页，或者允许客户填写订单并提交的网页，该怎么办呢？这些需求就需要浏览器或万维网服务器付出额外的活动。如果这些活动由客户机（如浏览器）完成则称为**客户端**（client-side）活动，如果这些活动由服务器（如万维网服务器）完成则称为**服务器端**（server-side）活动。

举一个例子。假设旅行社想要客户能够确认想去的旅行目的地和旅行日期，这时旅行社会给客户提供一个定制的网页，只包含与该客户需求有关的信息。在这种情况下，旅行社的网站将首先给客户提供一个展示可供选择的旅游目的地的网页。客户根据这个信息，指定感

兴趣的目的地和期望的旅行日期（客户端活动）。然后这些信息会被传回旅行社的服务器，服务器利用这些信息来构建合适的定制网页（服务器端活动），再将这个网页发送给客户的浏览器。

再举一个使用搜索引擎服务的例子。在这种情况下，客户机的用户指定感兴趣的主体（客户端活动），然后这个主体被传送给搜索引擎，搜索引擎会构建一个包含用户可能感兴趣的文档的定制网页（服务器端活动），然后发回给客户机。还有一个例子就是万维网邮件（Web mail）——日益流行的一种方法，通过它计算机用户能通过Web浏览器来访问他们的电子邮件。在这种情况下，万维网服务器是客户机与客户机邮件服务器之间的中间层。从本质上讲，万维网服务器构建包含邮件服务器信息的网页（服务器端活动），把这些网页发送给客户机，由客户机浏览器显示它们（客户端活动）。相反，浏览器允许用户创建消息（客户端活动），并把这一信息发送给万维网服务器，万维网服务器再把这些消息转发给邮件服务器（服务器端活动）进行邮寄。

实现客户端和服务端活动的系统有很多，可谓争奇斗艳，各有千秋。一种早期的控制客户端活动的方法现在仍然很流行，这种方法是在网页的HTML源文档中包含用JavaScript语言（由Netscape通信公司开发）编写的程序。浏览器可以从中提取程序并根据需要执行它们。另一种（由Sun Microsystems公司开发的）方法是，首先将一个网页传送给浏览器，然后根据HTML源文档中的请求将称为小应用程序（applet，用Java语言编写的）的额外程序单元传送给浏览器。还有一

种方法是使用Flash（由Macromedia公司开发），通过它可以实现大量的多媒体客户端演示。

控制服务器端活动的一个早期方法是，使用一组称为公共网关接口（common gateway interface，CGI）的标准，客户机可以用它请求执行存储在服务器中的程序。这种方法的一个变体（由Sun Microsystems公司开发）是允许客户机引发要在服务器端执行的称为小服务程序（servlet）的程序单元。如果所请求的服务器端活动是构造一个定制的网页，如前面旅行社的例子，就适合使用小服务程序方法的简化版本。在这种情况下，称为Java服务器页面（Java Server Page，JSP）的网页模板存储在万维网服务器里，它是利用从客户机收到的信息完成的。微软公司采用了一种类似的方法，其中构建定制网页的模板称为活动服务器页面（Active Server Page，ASP，后来的版本是ASP.NET）。相对于这些专有系统，PHP^[1]是一种实现服务器端功能的开源系统。常见的服务器端脚本语言包括Python、Ruby、Perl和许多其他语言。

[1] 最初代表个人主页（personal home page），但现在被认为是PHP超文本处理程序（PHP hypertext processor）。

最后，如果我们没有认识到，允许客户机和服务器在对方的机器上执行程序会引发一些安全问题和道德问题，我们就严重失职了。万维网服务器会例行公事地给客户机传送要执行的程序这个事实会导致服务器端出现道德问题，客户端出现安全问题。如果客户机盲目地执行万维网服务器发来的任何程序，它就向服务器的恶意活动敞开了大

门。同理，客户机可以让程序在服务器上执行这个事实也会导致客户端带出道德问题，服务器端出现安全问题。如果服务器盲目地执行客户机发来的任何程序，就会导致服务器出现安全漏洞和潜在损害。

4.3节问题与练习

1. 什么是URL？什么是浏览器？
2. 什么是标记语言？
3. HTML和XML的区别是什么？
4. 下列每个HTML标签的用途是什么？

a. <html> b. <head> c. </p> d.

5. 客户端和服务端分别指什么？
-

*4.4 因特网协议

本节研究报文是如何在因特网上传送的。因为传送过程需要系统中所有计算机的合作，所以控制传送过程的软件需要驻留在因特网的每台计算机中。我们首先研究此类软件的总体结构。

4.4.1 因特网软件的分层方法

网络软件的首要任务是提供将报文从一台机器传送到另一台机器所需的基础设施。在因特网上，报文传递活动是通过软件单元的层次

结构完成的，这和你要把一份礼物从美国的西海岸邮寄给东海岸的一位朋友的任务类似（见图4-12）。首先，把礼物打包并在包裹外面写上正确的地址；接着，把包裹拿到运输公司，如美国邮政局。运输公司把这个包裹和其他包裹一同放入一个大型的集装箱，送往与其签有服务合同的航空公司。航空公司将集装箱装入飞机并运往目的城市，也许沿途还要经过中转站。到达最终目的地，航空公司把集装箱从飞机上卸下，然后送往当地的运输公司，最后，运输公司把你的包裹从集装箱里取出送给收件人。

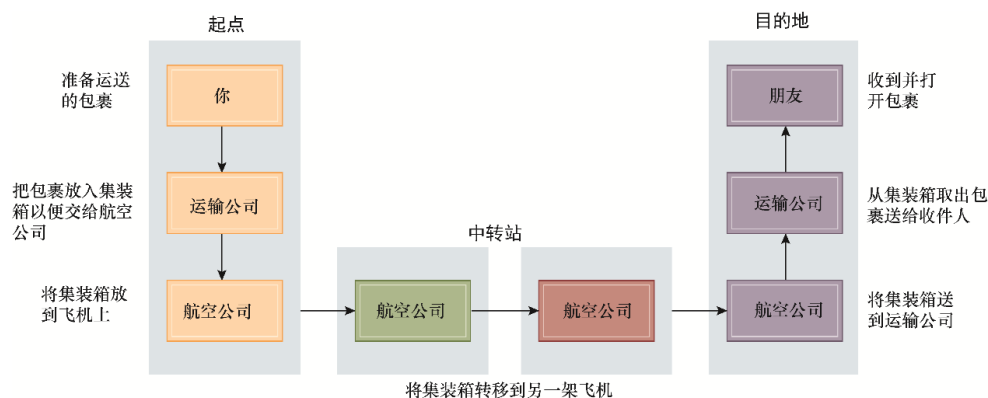


图4-12 包裹运输的例子

简而言之，礼物的运输过程需要3层的层次结构：（1）用户级（包括你和你的朋友）；（2）运输公司；（3）航空公司。每一层都把下一层当作抽象工具来使用。（你不关心运输公司的工作细节，而运输公司也不关心航空公司的内部运作。）层次结构中的每一层在起点和目的地都有代理，目的地的代理所做的常常和起点相应代理所做的相反。

因特网上软件控制通信的过程与运输包裹的情况类似，只是因特网上的软件有4层而不是3层，每层是由软件例程组成的而不是由人和企业组成的。这4层被称为**应用层**（application layer）、**传输层**（transport layer）、**网络层**（network layer）和**链路层**（link layer）（见图4-13）。报文通常始于应用层。当这个报文准备传输时，从应用层向下传递，经由传输层和网络层，最后通过链路层传输。目的地的链路层接收这条报文，沿逆向层次结构向上传递，直到把它交给报文目的地的应用层。

下面通过跟踪一个报文通过网络系统的路径，从总体上研究一下报文的传输过程（见图4-14）。首先从应用层开始。

应用层由那些使用因特网通信来完成任务的软件单元（如客户机和服务器）组成。虽然名称类似，但是这一层不仅包括3.2节介绍的应用软件分类中的软件，还包括许多实用软件包。例如，用于传输文件或者播放媒体的软件已经变得极其常见了，通常被认为是实用软件。

应用层使用传输层在因特网上发送和接收报文，这与我们通过运输公司邮寄和接收包裹非常相似。正像我们有责任提供一个符合运输公司规范的地址一样，应用层负责向因特网基础设施提供兼容的地址。为了满足这个要求，应用层利用因特网中名称服务器的服务把人类使用的助记地址转换成符合因特网规范的IP地址。

传输层的重要任务是从应用层接收报文并确保报文以正确的格式在因特网上传输。为了后一个目标，传输层将长报文分成小的片段作

为独立单位在因特网上一次传输一段。这种分段是必要的，因为一个长报文会阻塞许多其他报文的必经之路——因特网路由器上的其他报文流。实际上，小段的报文可以在这些节点交叉通过，而长报文则会迫使经过这些节点的其他报文等待（很像在铁路交叉道口，许多小汽车等一列长火车通过的情况）。

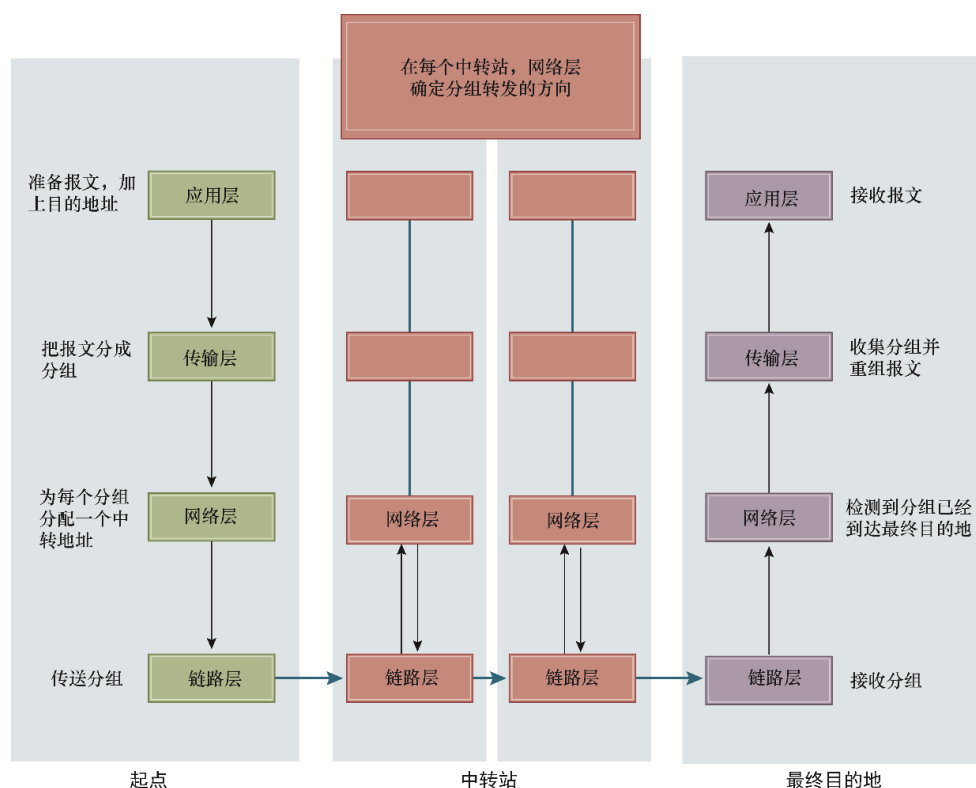


图4-14 因特网上报文的传送过程

传输层给它生成的小段报文增加了序列号，从而使这些片段在报文的目的地可以重组，然后将这些称为**分组**（packet）的片段交给网络层。从这一刻开始，这些分组被认为是彼此独立的不相干的报文，直到它们到达最终目的地的传输层。这些属于同一个长报文的分组很有可能沿着不同的路径通过因特网。

基本知识点

- 因特网是一个发送数字数据的分组交换系统，在发送数据时会将其分解成称为分组的位块，其中包含正在传输的数据和用于路由该数据的控制信息。

网络层的工作是，在分组沿着分组路径通过因特网时，决定每一步分组应该发送到哪个方向。事实上，网络层和它下面的链路层组合在一起构成了驻留在因特网路由器上的软件。网络层负责维护路由器的转发表，并用此表确定分组的转发方向。路由器中的链路层负责接收和传输分组。

这样，当分组的起点的网络层收到来自传输层的分组时，它会使用它的转发表来确定分组应该被发送到哪里，并在那里开始分组的旅程。确定好合适的方向后，网络层把分组交给链路层，进行实际传输。

链路层具有传送分组的职责，因此，它必须处理目的计算机所在的个体网络的特有的通信细节。例如，如果网络是以太网，链路层将使用CSMA/CD协议；如果网络是WiFi网，链路层将使用CSMA/CA协议。

当分组被发送后，连接另一端的链路层会接收它。在那里，链路层把分组向上交给它的网络层，由网络层把分组的最终目的地和网络层的转发表进行比对，确定分组下一步的方向。当做出这个决定后，

网络层把分组返回给链路层，分组沿着它的路径被转发。使用这样的方式，每个分组从一台机器跳到另一台机器，最终到达它的目的地。

注意，在这个旅程中，只涉及中间站的链路层和网络层（再次参见图4-14），因此正如前面提及的，只有这两个层存在于路由器上。此外，为了尽量减少在每个中间“站”上的延迟时间，路由器中的网络层的转发角色是紧密地与链路层集成在一起的。因此，现代路由器转发一个分组所需的时间是用百万分之一秒来衡量的。

在分组的最终目的地，由网络层来确认分组的旅程已经结束。在这种情况下，网络层会把分组交给它的传输层，而不再转发分组。传输层从网络层接收这个分组时，会提取组成报文的基本片段，按照报文起点的传输层所提供的片段序列号重组原始报文。一旦报文重组完成，传输层就把它交给应用层的适当单元——这样便完成了报文的传输过程。

确定应用层内哪个单元来接收到来的报文是传输层的一项重要任务。要完成这项任务，需要为各个单元分配唯一的**端口号**（port number，它与第2章讨论的I/O端口无关），并要求在报文开始传输旅程之前把适当的端口号附加到报文地址上。然后，一旦目的地的传输层收到报文，它只需将报文交给指定端口号上的应用层软件。

因特网用户很少需要关心端口号，因为常见的应用有普遍公认的端口号。例如，如果请求Web浏览器检索URL为 `http://www.z**.org/animals/frog.html` 的文档，那么浏览

器认为应该通过80端口和www.z**.org的HTTP服务器联系。同样，在发送邮件时，SMTP客户机认为应当通过25端口与SMTP邮件服务器通信。

基本知识点

- 因特网上的路由选择是容错的、冗余的。
- 因特网上两点之间路由选择的冗余（即不止有一条通向路由数据的路）能提高因特网的可靠性，并帮它扩大到更多的设备和更多的人。

总之，因特网上的通信涉及4层软件的相互作用。应用层从应用的角度处理报文；传输层把这些报文转换成适合因特网的段，并负责将收到的报文重组好后交给适当的应用程序；网络层负责指导段通过因特网的方向；链路层处理段从一台机器到另一台机器的实际传输。令人惊讶的是，虽然有这么多的工作，因特网的响应时间却是以毫秒计的，所以许多事务看起来像是瞬间发生的。

4.4.2 TCP/IP协议簇

因为需要开放式网络，所以需要颁布一些标准，通过这些标准，不同制造商生产的设备和软件才能和其他厂商的产品一起正常运行。一个已经产生的标准是由国际标准化组织制定的**开放系统互连**（open system interconnection，OSI）参考模型。与我们刚刚描述的4层层次结

构不同，OSI标准是基于7层层次结构的。它具有国际组织的权威性，所以它经常被引用，但是它迟迟未能取代4层层次结构的观点，这主要是因为4层层次结构在OSI模型制定之前已经成为因特网的事实标准。

TCP/IP协议簇是因特网所使用的协议标准的集合，是用来实现4层通信层次结构的。实际上，**传输控制协议**（transmission control protocol，TCP）和**因特网协议**（Internet Protocol，IP）只是这个庞大集合中两个协议的名字——因此，把整个协议集合称为TCP/IP协议簇相当具有误导性。更确切地说，TCP定义了传输层的一个版本，这里说**版本**是因为TCP/IP协议簇不只提供一种传输层实现方式，例如，还可用**用户数据报协议**（User Datagram Protocol，UDP）定义其他版本的传输层。传输层具有多种版本的情况和运输包裹的情况类似，你可以选择不同的运输公司，每家公司都提供相同的基本服务，但又各有特色。因此，根据对特定的服务质量的要求，应用层的软件单元可以选择是通过传输层的TCP版本来发送数据，还是通过传输层的UDP版本来发送数据（见图4-15）。

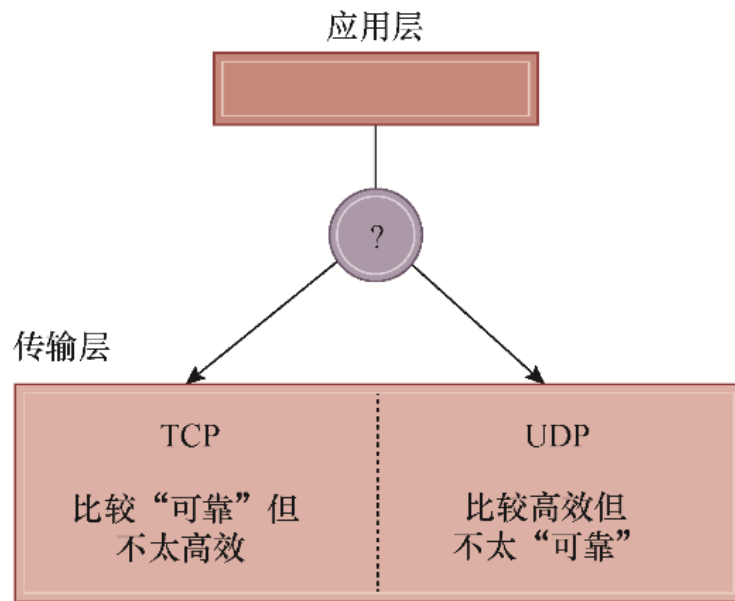


图4-15 在TCP和UDP之间选择

基本知识点

- 分组和路由选择的标准包括传输控制协议/因特网协议（TCP / IP）。

TCP和UDP之间有一些区别。第一个区别是，在发送应用层所请求的报文前，基于TCP的传输层要向目的地的传输层发送它自己的报文，告诉目的地传输层有报文要发送，然后，它要等待目的地确认这个报文后才开始发送应用层报文。采用这种方式时，据说基于TCP的传输层在发送报文前要建立一个连接。而基于UDP的传输层在发送报文前不需要建立这样的连接，它仅按照所给的地址发送报文，然后就忘记这个报文，尽管它知道，目的地的计算机可能根本没运作。由于这个原因，UDP被称为无连接协议。

TCP和UDP之间的第二个区别是，起点和目的地的TCP传输层一起工作，通过确认和分组重发来确保一个报文的所有片段都被成功传输到目的地。因此，TCP被称为可靠协议，而UDP不提供这种重发服务，被称为不可靠协议。

TCP和UDP之间还有一个区别，那就是TCP提供了**流量控制**（flow control）和**拥塞控制**（congestion control），前者是指报文起点的TCP传输层能降低它发送数据段的速率，防止目的地的对应方应接不暇，后者是指报文起点的TCP传输层能调整它的发送速率，减轻它与报文目的地间的拥塞。

所有这些并不意味着UDP是一个不好的协议，毕竟，基于UDP的传输层比基于TCP的更精简，因此，如果一个应用有能力处理UDP的潜在影响，那么基于UDP的传输层会是更好的选择。例如，UDP的高效使得它成为DNS查找和许多其他流选择的协议。但是，因为电子邮件在时间上不太敏感，所以邮件服务器使用TCP传送电子邮件。

IP是用于实现分配给网络层任务的因特网标准。我们已经注意到这个任务包括**转发**（forwarding）和路由选择（routing），前者涉及通过因特网来中继分组，后者涉及更新层的转发表，以反映出条件的改变。例如，一个路由器可能会出现故障（意味着流量应该不会再被转发到它这个方向上来）或因特网的一个区域可能会变得拥塞（意味着流量应该绕过这个障碍进行路由）。当相邻网络层交换路由信息时，有许多与路由选择有关的IP标准可用于处理相邻网络层间通信的协议。

与转发有关的一个有趣的特性是，每次报文起点的IP网络层在准备一个分组时，都会把一个称为**跳数**（hop count）或存活时间（time to live）的值附加到那个分组上。这个值限制了分组在尝试找到通过因特网的出路时应该被转发的次数。IP网络层每次转发一个分组，都要把这个分组的跳数减1。通过这个信息，网络层能保护因特网，以免分组在系统内无休止地循环。虽然因特网的规模每天都在增长，但初始的64跳数仍然足以让分组在当今ISP的路由器迷宫中找到它自己的出路。

这些用于传输、路由和转发分组的功能累加在一起就形成了因特网，它能容忍许多种错误，能在发生拥塞或故障时回退到其他冗余路径进行传输。

多年以来，称为IPv4（IP版本4）的IP版本一直被用于在因特网内实现网络层。然而，因特网迅速超出了IPv4所规定的32位互联网编址系统。为了解决这一问题并实现其他改进（如多播），人们建立了一个称为IPv6的新IP版本，它使用的互联网地址由128位组成。从IPv4到IPv6的转换目前正在进行（这个转换在4.2节介绍因特网地址时提到过），预计到2025年因特网中32位地址的使用将绝迹。

基本知识点

- 可以使用IP地址的设备数量增长得特别快，以至于已经建立了新协议（IPv6）来处理更多设备的路由选择。
-

4.4节问题与练习

1. 因特网软件层次结构的哪些层不需要用在路由器上？
2. 基于TCP协议的传输层和基于UDP协议的传输层之间有哪些区别？
3. 传输层如何确定应用层的哪个单元应当接收即将到来的报文？
4. 是什么阻止了因特网上的计算机记录所有途径它的报文的副本？

*4.5 简单的客户机服务器

在本节中，我们将看到一对儿非常简单的Python脚本，它们一起组成了一个最小限度的组网客户机和服务器。

超越无限

为了让我们的示例保持简单，我打算仔细观察一对儿本身可能不是非常有用的Python脚本。在执行客户机脚本时，脚本将建立到预定汇聚点的网络连接，并将字符串“To Infinity ...”发送给正在另一端监听的设备。然后，它会等待并打印返回的响应。下面给出这个完整的Python客户机脚本。

```
import socket

# Create a TCP/IP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# Connect to the server listening at localhost:1313
server_address = ('localhost', 1313)
```

```
print('Connecting to %s:%s' % server_address)
sock.connect(server_address)
# Send request message
message = b'To Infinity...'
print('Client sending "%s"' % message.decode())
sock.sendall(message)
# Receive reply from server
reply = sock.recv(11)
print('Client received "%s"' % reply.decode())

# Close the server connection
sock.close()
```

`import`语句预先警告Python解释器，该脚本引用的是名为“socket”的库中定义的函数。在组网软件的语境下，**套接字**（socket）是位于网络栈应用层的进程的抽象，用于通过下面的传输层与网络上的其他进程连接。

接下来的赋值语句使用库创建一个新套接字，并给它分配一个名字“sock”。传入socket库的`socket()`函数的参数表明sock应该使用来自因特网地址家族（“AF_INET”）的IPv4网络协议，并且应该使用传输协议TCP（“SOCK_STREAM”）而不是其他选择，如UDP。

接下来的3条语句建立套接字连接的目的地，帮助用户打印那个信息，并尝试连接到预定网络位置的服务器进程。为了让示例保持简单，我们使用了服务器名称“localhost”，因特网连接的主机通常将localhost解释为自己。因此，这个简单的客户机实际上并不会将它的消息通过网络发出去。我们这样设置示例是为了让客户机进程和服务进程位于同一台本地主机上。这样做的原因很简单，因为许多读者可能无法使用两个独立的因特网连接的主机来运行这个示例代码。此外，很多主机都应该就地包含默认的安全措施（见4.6节的**防火墙**概

念)，以防止普通用户在没有被管理员授予特殊访问权限的情况下连接自己的客户机和服务器。扩展这个简单的示例代码来查找远程主机的IP地址（并获取系统管理员权限来访问它）就留给读者做练习。

我们为这个演示选择的端口号1313没有什么特别的，它只是首次出现这个演示的书的版本号。大多数主机都可以使用1023到65 535之间的任何端口号，只要这个端口号没被其他进程使用。

语句`sock.connect(server_address)`导致底层传输层和操作系统尝试到监听localhost主机1313端口的进程的连接。因为这是可靠的TCP套接字连接，所以如果客户机脚本执行时没有监听那个连接的服务器进程，这条语句就会导致错误。

再接下来的3条语句组成我们客户机的简单消息“To Infinity ...”，为用户打印它，并尝试将其全部发送到套接字连接另一端的进程。消息字符串前面的那个字母“b”和另外那个用于打印这个消息的`decode()`函数，突出了通过网络发送消息的一个微妙难题。Python的socket库的`sendall()`函数期望字符串由单个字节组成，但`print()`函数期望字符串使用多字节编码，如UTF-8。Python提供了用于在这些不同形式之间转换字符串表示的机制，但我们不会在关于网络的这个示例中详述这些细节。

接下来的两条语句等待接收来自服务器的应答，我们将这个应答称为“reply”，然后用`print()`函数打印该响应的UTF-8表示。等待

行为内置在socket库的recv()函数中；如果我们的简单客户端无法连接到应答客户机消息的进程，这段脚本将无限期地等待这条语句。

基本知识点

- 数据可以存储为许多种格式，具体取决于数据的特点（如大小和预期用途）。

演示客户机脚本中的最后一条语句关闭到服务器的套接字连接，将那个套接字控制的传输层资源返回给操作系统，以备后用。

下面给出了与客户机示例配套的简单服务器脚本：

```
import socket

# Create a TCP/IP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# Bind the socket to the port localhost:1313
server_address = ('localhost', 1313)
sock.bind(server_address)
# Listen for incoming client connections
sock.listen(1)

print('Waiting for a client to connect')
client, client_port = sock.accept()
print('Connection from client %s:%s' % client_port)

# Receive request from the client
request = client.recv(14)
print('Client sent "%s"' % request.decode())
reply = b'And Beyond!'
print('Server replies "%s"' % reply.decode())
client.sendall(reply)

# Close the client connection
client.close()
```

这个示例网络服务器脚本的细节与相应的客户机脚本非常类似。第一处不同在客户机脚本为套接字调用connect ()函数的地方，服务器改成为套接字调用bind ()函数和listen ()函数。第一个调用请求使用底层操作系统的1313端口，如果该端口可用就将其绑定到这个服务器进程。如果所请求的端口不可用，bind ()语句就产生错误消息，服务器脚本将中止。listen ()函数将这个套接字标记为准备好接受来自客户端的传入连接。

accept ()函数一直等到来自网络的客户机进程连接到该主机和该端口与该服务器进程交谈。返回的两个值包含连接客户机的主机地址和端口。

在这种情况下，知道消息“To Infinity...”将是14字节长会使来自客户端的recv ()的工作变得容易得多。我们的服务器构造传统的应答“And Beyond!”将其全部发送给客户机，并关闭套接字连接。如果我们的服务器脚本在达到close ()语句之前由于某种原因停止了，那么我们正在监听的端口号在操作系统回收资源之前可能会有几分钟不可用。

为了尝试这个简单的客户机/服务器演示，先在一个Python窗口中执行服务器脚本。服务器应该说：

Waiting for a client to connect

然后等待。

在另一个Python窗口中执行客户机脚本。客户机应该说类似下面这样的话：

```
Connecting to localhost:1313
Client sending "To Infinity..."
Client received "And Beyond!"
```

同时，服务器脚本将继续：

```
Connection from client 127.0.0.1:53233
Client sent "To Infinity..."
Server replies "And Beyond!"
```

然后，这两个脚本都应该利索地退出。这个演示需要一些时间和运气才能有效工作。如果哪一端都难以到达1313端口，那么只要客户机和服务器脚本在同一地址上达成一致，可以尝试65 536以下的其他端口号。

这个简单的示例演示了通过因特网上客户机和服务器软件特别常用的传输层套接字系统交换信息的客户机进程和服务器进程。因为这个示例的重点不是高级Python语法和库功能，所以忽略了实际客户机/服务器系统的许多重要细节。真正的服务器进程通常会循环，一次又一次地接受客户端连接。这个服务器不检查连接客户机发送的请求，并且无论客户机说什么都用“`And Beyond!`”应答。当用于做这些事情的Python构造出现在后面的章节中来做这些事情时，会考虑返回来扩展这个客户机/服务器示例。另外的十几行代码可以实际产生一个简单的可以处理多种请求的服务器脚本，并用有用的信息应答客户机。

4.5节问题与练习

1. 什么是套接字？
2. 创建新套接字需要什么信息？
3. 建立客户机连接和服务端连接之间的主要区别是什么？
4. 我们可以用哪种表示来通过网络连接发送字符数据？

4.6 网络安全

当计算机连接到网络上时，它会成为未授权访问和恶意破坏的对象，未授权访问和恶意破坏是**网络犯罪**（cybercrime）的两种形式。在本节中，我们将讨论与这些问题相关的话题。**网络安全**（cybersecurity）是对保护计算机、网络和数据免受这种攻击的研究。

4.6.1 攻击的形式

通过网络连接对计算机系统及其内容进行攻击的方式有很多种。攻击往往集中在计算机系统上的软件上，但计算机系统上的硬件和人类用户本身也可能是系统中的重要弱点。保护这种系统的安全可能需要介入所有这3个方面。

基本知识点

- 实现网络安全有软件、硬件和人3个部分。

许多攻击都会用到怀有恶意的软件——统称为**恶意软件**（malware）。这种软件可能会被传送到某台计算机内部执行，也可能远距攻击计算机。需要传送到受攻击计算机中执行的软件的示例包括病毒、蠕虫、特洛伊木马和间谍软件，它们的名称反映了软件的主要特征。

病毒（virus）是一种软件，它先通过将自身嵌入计算机已有的程序中来感染计算机，然后，当“宿主”程序被执行时，病毒也会被执行。在执行时，许多病毒都不过是尝试把自身传送到计算机中的其他程序上。但是，有些病毒会执行破坏性的动作，例如，使操作系统各个部分的性能下降，擦除海量存储器的大块数据，或者毁坏数据和其他程序。

蠕虫（worm）是一个自主程序，它可以通过网络传送自己，驻留在计算机中并将其自己的副本转发到其他计算机。和病毒的情况一样，蠕虫可以被设计成只是复制自己，也可以设计成实施更严重的故意破坏行为。蠕虫的一个典型后果是，蠕虫副本的激增会使合法应用的性能下降，最终使整个网络或互联网过载。

计算机应急响应小组

1988年11月，发布到因特网上的一个蠕虫病毒造成了网络服务的严重瘫痪。随后，美国国防高级研究计划局（Defense Advanced Research Projects Agency，DARPA，读作“DAR-pa”）成立了计算机应急响应小组（Computer Emergency Response Team，CERT，读作“SERT”），并在卡内基—梅隆大学内设立了CERT协调中心。CERT

是因特网安全的“监督者”，其职责是调查安全问题、发布安全警告和发起运动提高公众的因特网安全意识。

特洛伊木马（Trojan horse）是一种伪装成吸引人的程序（如游戏或有用的实用程序包）进入计算机系统的程序，它们被受害者自愿引入。然而，一旦特洛伊木马程序进入计算机，它就会实施可能有不良影响的额外活动。有时，这些额外活动会立即开始。在其他情况下，特洛伊木马可能会暂时休眠，直到被一个特殊的事件触发，如一个预选日期的出现。特洛伊木马常常以有诱惑力的电子邮件附件的形式出现，当这类附件被打开时（即收件人请求浏览附件时），就会激活特洛伊木马的不义之举。因此，决不要打开来源不明的电子邮件附件。

恶意软件的另一种形式是**间谍软件**（spyware），有时称为**嗅探**（sniffing）软件，这类软件收集它所在计算机的活动信息，并把这些信息报告给攻击的发起者。有的公司使用间谍软件来建立客户档案，这种做法本身的道德价值值得质疑。在其他情况下，间谍软件被用于公然的恶意行为，如通过记录计算机键盘键入的符号序列，搜索口令或信用卡卡号。

间谍软件通过嗅探方式秘密地获取信息，与此相反，**网络仿冒**^[2]（phishing）技术是通过简单地索要信息显式获取信息。由于网络仿冒的过程就是撒下大量的“线”，等人“上钩”，所以术语**网络仿冒**就是**钓鱼**（fishing）的意思。网络仿冒通常用电子邮件来实施，与老式电话骗局差不多。骗子伪装成金融机构、政府机关或者执法机构发送电子

邮件信息，向潜在受害者索要假装用于合法目的的信息，而实际上这些信息被作恶者恶意使用。

[2] 又称网络钓鱼或网络欺诈。——译者注

与遭受病毒和间谍软件等内部感染不同，网络中的计算机还能够被系统中其他计算机上执行的软件攻击。**拒绝服务**（denial of service，DoS）攻击就是其中的一个例子，这种攻击会使计算机的信息过载。拒绝服务攻击已经向因特网上的大型商业万维网服务器发起攻击，从而破坏公司的业务，有时甚至曾导致一些公司的商业活动停止。

拒绝服务攻击需要在短时间内产生大量的信息以便淹没目标计算机。为做到这一点，攻击者通常会在大量未设防的计算机上植入一种能在收到信号时产生信息的软件，然后，只要发出信号，所有这些计算机，有时称为**僵尸网络**（botnet），就会用信息淹没目标计算机。因而，当产生攻击的工作分布在许多主机上时，这就是所谓的**分布式拒绝服务**（distributed denial of service，DDoS）攻击。因此，分布式拒绝服务攻击中内在的是用作帮凶的毫不知情的计算机的可用性。这就是鼓励所有计算机用户安装操作系统供应商提供的所有安全更新、在任何连接因特网的主机上使用适当保护的原因。据估计，一旦计算机连接到因特网，20分钟内就至少有一个入侵者会尝试利用它。反过来，保护不当的主机会对因特网的完整性构成重大威胁。

基本知识点

- 分布式拒绝服务（DDoS）攻击通过使用来自多个系统的请求淹没目标来危害目标。

另一个和大量无用信息有关的问题是散布称为**垃圾邮件**（spam）的无用邮件。然而，和拒绝服务攻击不同的是，垃圾邮件的数量不足以压垮计算机系统。相反，垃圾邮件的作用是压垮接收垃圾邮件的人。正如我们所看到的，这一问题已经被复杂化了，垃圾邮件这一媒介被大量用于网络仿冒和煽动可能传播病毒及其他恶意软件的特洛伊木马。

基本知识点

- 网络仿冒、病毒和其他攻击都有人和软件两部分。

4.6.2 防护和对策

老话“一分预防胜过十分治疗”对控制网络连接上的故意破坏情况来说无疑是正确的。一个主要的防护技术是过滤穿过网络某个点的流量，通常使用称为**防火墙**（firewall）的程序。例如，防火墙可能安装在组织内联网的网关处，用来过滤进出这个区域的信息。这种防火墙的设计目的可能是阻止向某些特定目的地址发送信息，或者阻止接收已知的有问题的来源所发送的信息。后一种功能可以用来终止拒绝服务攻击，因为它提供了阻止来自具有攻击性计算机的流量的方法。安装在网关处的防火墙的另一个常见的作用是，阻止所有起点地址位于通过网关接入该区域的传入信息，因为这样的信息表明有外部计算机

正在假装区域内的成员。把自己伪装成其他成员的行为称为**欺骗**（spoofing）。

基本知识点

- 防病毒软件和防火墙可以帮助避免对私人数据进行未授权访问。

防火墙不但能用于保护整个网络或域，还能用于保护个人计算机。例如，如果一台计算机没有用作万维网服务器、名称服务器或电子邮件服务器，那么安装在这台计算机上的防火墙应当阻止所有针对这些应用的传入流量。实际上，入侵者可能获得计算机入口的一个途径就是通过一个已经不存在的服务器所留下的“漏洞”来建立联系。尤其是，利用间谍软件获取信息的一个方法是在感染的计算机上建立一个秘密的服务器，通过这个服务器，恶意客户机就可以获取间谍软件的嗅探结果。正确安装防火墙可以阻止来自这类恶意客户机的报文。

还有些防火墙的变体是为一些特殊用途设计的，**垃圾邮件过滤器**（spam filter）就是其中一例，设计这种防火墙是为了阻止一些无用的电子邮件。许多垃圾邮件过滤器在区分正常邮件和垃圾邮件时采用了相当复杂的技术。一些过滤器通过一个训练过程来学习区分：先由用户确定哪些属于垃圾邮件，过滤器获得足够多的例子后可以自行做出判断。这些过滤器是“将各种不同的学科领域（如概率论、人工智能等）联合起来可以推动其他领域发展”的例子。

另一种有过滤功能的防护工具是**代理服务器**（proxy server）。代理服务器是一个软件单元，充当客户机和服务器之间的中介，目标是保护客户机免受服务器不利动作的影响。如果没有代理服务器，客户机将直接与服务器通信，这意味着服务器有机会了解客户机的一些信息。由于同一个组织的内联网内的许多客户机都与远程服务器通信，长此以往，远程服务器就能收集到大量关于内联网内部结构的信息，而这些信息在以后可被用于恶意活动。为了防范这一点，组织可以为特定类型的服务（如FTP、HTTP等）建立代理服务器，这样，每次内联网内的客户机试图连接某个类型的服务器时，实际上连接的都是代理服务器。于是，代理服务器扮演了客户机的角色与实际的服务器联系。此后，代理服务器就一直扮演实际客户机与实际服务器之间的中介，来回中继报文。这种安排的第一个好处是，实际服务器无法知道代理服务器是不是真的客户机，事实上，它永远不会意识到实际客户机的存在。这样一来，实际服务器就无法了解内联网的内部特性。第二个好处是，代理服务器能够过滤服务器发送到客户机的所有报文。例如，FTP代理服务器能够检查所有传入文件，看是否感染了已知的病毒，然后阻止所有感染了病毒的文件传入。

还有一种用于防止网络环境中的问题的工具是审计软件，它类似于我们在操作系统安全性的讨论中提到的审计软件（见3.5节）。通过网络审计软件，系统管理员能够察觉到管辖范围内不同位置突然激增的报文流量，监控系统防火墙的活动状态，并且可以对个别计算机的请求模式进行分析，以探测非正常行为。审计软件是管理员用于在问题失控之前及早发现问题的主要工具。

另一种防御通过网络连接进行入侵的方法就是采用**防病毒软件**（antivirus software），这种软件用来探测和移除被已知病毒或其他方式感染的文件。（实际上，防病毒软件代表了一大类软件产品，这类软件产品中每一种都被设计成探测和移除某一特定类型的感染。例如，许多产品专门研究病毒控制，另外一些产品则专门研究间谍软件防护。）重要的是，这些软件包的用户要理解，正如生物系统的情况一样，新的计算机病毒感染会不断地出现，所以需要不断地更新疫苗。因此，防病毒软件必须从软件提供商那里定期下载更新来进行定期维护。然而，即使如此，也不能保证计算机的绝对安全。毕竟，新病毒必须先感染一些计算机，才能被发现并生产疫苗。因此，明智的计算机用户从不打开一个不熟悉来源的电子邮件中的附件，从不在未事先确认软件可靠性的情况下下载软件，从不轻易响应弹出广告，从不在计算机没有必要连接在因特网上时还将其连接在因特网上。

4.6.3 密码学

有时网络破坏行为的目的是干扰系统（如拒绝服务攻击），但有时其最终目标是获取信息的访问权。保护信息的传统方法是通过口令来控制对信息的访问。然而，当数据通过网络和互联网传送时，报文会被一些未知的实体中继，因此口令可能会被泄密，从而没有多大的价值。**密码学**（cryptography）研究的是在对手面前安全发送和接收信息。注入**加密**（encryption）之类的密码工具可用于将报文转换为编码版本，使得即使加密的数据落入不怀好意的人的手中，编码后的信息

依然能保持其机密性。现在，许多传统的因特网应用已经进行了改变，加入了加密技术，产生了所谓的应用的“安全版本”。

基本知识点

- 密码学是许多网络安全模型必不可少的。

一个最好的例子就是HTTP的安全版本——**HTTPS**，大多数金融机构都用它来为客户提供对其账户的安全因特网访问。HTTPS的骨干是称为**传输层安全**（transport layer security，TLS）的协议系统，其前身是**安全套接字层**（secure sockets layer，SSL），它最初是由Netscape公司开发的，用来为万维网中的客户机和服务器提供安全的通信链路。大多数浏览器通过在计算机屏幕上显示一个很小的锁图标来表明SSL已启用。（有的会用图标的出现与否来表示是否正在使用TLS；其他的则通过显示锁是闭合还是打开的来表示是否正在使用TLS。）

基本知识点

- 万维网上的浏览器和服务器之间共享信息和进行通信的标准包括HTTP和安全套接层/传输层安全（SSL/TLS）。

几个世纪以来，密码学家使用的加密方案都依靠编码或解码算法的保密性和/或口令〔也称**密钥**（key）〕的保密性来保护报文。随着对密码学数学的理解越来越深入，计算机已经被证明是破解许多传统加密方案的强大工具，有时就是凭借能够比人更快地试验所有可能的密钥。**对称密钥加密**（symmetric-key encryption）系统使用同一个密钥

来编码和解码报文，它提供了越来越高的安全性，因为计算机能够使用大型的看似随机的密钥。在理想条件下，加密方案的密钥长度每增加一位（bit），密码学家对手必须尝试的可能次数就会增加一倍。HTTPS和SSL使用对称密钥加密技术来保护整个因特网上的网络交易。这些系统的基础是设计的数学运算，使得即使知道如何加密报文的知识（加密算法）也无法知道如何解密报文。这个特征有点儿违反直觉。毕竟，直觉表明，如果一个人知道如何加密报文，那么这个人就应该能将加密过程中的数学运算反过来，从而解密信息。但对称密钥加密系统没遵循这个直觉逻辑。问题仍然是：为了对称密钥系统中的安全通信，仍然必须在接收者和发送者之间共享密钥，不能向任何对手透露秘密。

基本知识点

- 密码学有数学基础。
- 对称加密是一种涉及一个密钥的加密和解密的加密方法。

在密码学领域里一个更令人着迷的话题就是**公钥加密**（public-key encryption），它涉及加密和解密算法已知、密钥已知但编码报文仍然是安全的这样一项技术。

公钥加密系统涉及两个密钥的使用。一个密钥称为**公钥**（public key），用来对报文进行加密；另一个密钥称为**私钥**（private key），用来对报文进行解密。要使用这个系统，首先要将公钥分发给那些可能需要向特定目的地发送报文的一方，将私钥秘密地保存在目的地。

然后，报文发起方可以用公钥对报文进行加密，将报文发送到它的目的地，即使在这期间被其他知道公钥的中间人截获，也仍能保证它的内容是安全的。事实上，唯一能对报文进行解密的是在报文的目的地持有私钥的那一方。因此，如果Bob创建了一个公钥加密系统，并把公钥给了Alice和Carol这两个人，那么Alice和Carol这两个人都能对发给Bob的报文进行加密，但是他们无法窥探对方的通信。事实上，如果Carol截获了来自Alice的报文，即使她知道Alice是怎样对报文进行加密的，也无法对报文进行解密（见图4-16）。虽然直觉看似秘密加密方案可能是对Bob的报文的最佳保护，但是因特网上使用的最可信加密方案是开放标准，每个人都可以知道加密算法，但只要私钥是保密的，报文就是安全的。

基本知识点

- 开放标准有助于确保密码学是安全的。
- 公钥加密不是对称的，因为它能增强安全性，所以它是一种广泛使用的加密方法。

当然，公钥系统内潜藏着一些微妙的问题。一个问题就是，要保证所用的公钥事实上对目的地的那一方而言是一个正确的密钥。举例来说，如果你正在和银行通信，你要确保你用来加密的公钥是针对银行的，而不是针对冒充者的。如果一个冒充者让自己以银行的身份出现（一个欺骗的例子），并把它公钥给你，那么你加密并发送给“银

行”的报文，对这位冒充者就是有意义的，但对银行是没意义的。因此，将公钥关联到正确的另一方的任务很重要。

解决这个问题一个办法是建立一些可信任的称为**认证机构**（certificate authority）的因特网站点，其任务是维护各方及其公钥的准确列表。然后，这些起着服务器作用的机构，以称为**证书**（certificate）的软件包的形式为他们的客户机提供了可靠的公钥信息。证书是一个它包含一方名称和该方公钥的软件包。现在在因特网上有许多商业认证机构，但为了保持对组织通信安全性的更严格控制，由组织来维护他们自己的认证机构也很常见。

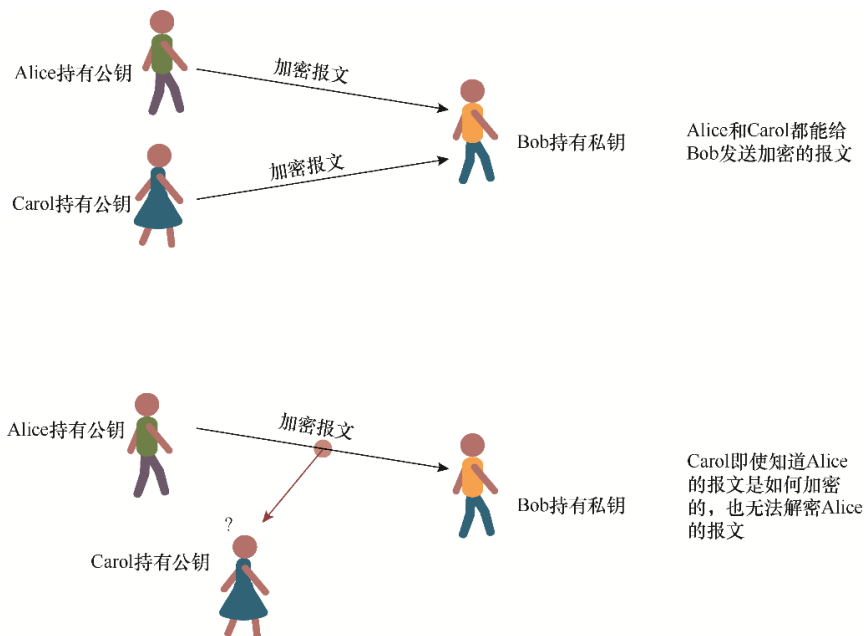


图4-16 公钥加密

基本知识点

- 认证机构（certificate authority，CA）颁发用于验证安全通信中使用的加密密钥的所有权的、基于信任模型的数字证书。

公钥系统的第二个问题是计算成本高，完成加密所需的时间比类似的对称密钥系统的长。因特网上频繁使用的解决方案是使用所有这些密码技术的组合，利用每种密码技术的独特优势。Web浏览器软件通常附带一个可信认证机构列表。使用证书，可以确保客户端浏览器具有远程服务器的真实公钥。然后，浏览器使用相对较慢的公钥密码学与远程服务器协商全新的秘密会话密钥。在交换了没有透露给任何可能的窃听者的新密钥之后，浏览器和服务端会使用快速对称密钥密码来加密安全HTTPS报文，同时通过因特网处理信用卡购买。

最后，我们应该在解决**鉴别**（authentication）问题方面对公钥加密系统进行一下说明，鉴别就是要确保报文的作者实际上确实是他们声称的那一方。这里关键的问题就在于，在有些公钥加密系统中，加密密钥和解密密钥的作用可以颠倒。也就是说，原文可以用私钥来加密，并且因为只有一方可以访问这个密钥，所以这样加密的任何原文都必须是来自那一方的。用这种方式，私钥的持有者就能产生一个位模式，称为**数字签名**（digital signature），只有那一方才知道应怎么生成。通过对报文附加这样的签名，发送者就能对报文做可以信任的标记。数字签名可以和报文本身的加密版本一样简单。所有发送者必须做的就是用自己的私钥（这个密钥通常用于解密）对要发送的报文进行加密。当接收者收到报文时，利用发送者的公钥对这个签名进行

解密。这样得出的报文就能保证是可信的，因为只有私钥的持有者才能生成该加密版本。

在第12章末尾，我们还会回到公钥加密的话题上来，将其作为一个复杂计算机算法的例子。

4.6.4 网络安全的法律途径

另一种增强计算机网络系统安全性的方法是应用法律补救措施。然而，这种方法有两个障碍。第一个障碍在于，认定一个行为不合法，并不意味着会排除该行为，而只是提供了一个法律依靠而已。第二个障碍在于，网络的国际特性意味着要获得依靠通常是很困难的。在一个国家中不合法，但在另一个国家中却可能是合法的。最终，通过法律手段来增强网络安全性是一个国际性的问题，所以必须由国际法律机构（一个可能的机构将是位于海牙的国际法庭）来处理。

尽管这么说，我们必须承认，虽然法律措施并不完美，但还是有很大影响力的，所以对我们而言，在网络领域里，研究用来解决冲突的一些法律步骤还是有必要的。为此，我们用美国联邦法律中的例子来进行说明。另外，还可以从其他一些政体（如欧盟）找到类似的例子。

首先讨论恶意软件的扩散问题。在美国，这个问题是由《计算机欺诈和滥用法》（Computer Fraud and Abuse Act）提出的，该法案于1984年首次通过，其后做了几次修改。通过这个法案，涉及蠕虫和病

毒制造的大多数案件都已经被起诉。简而言之，这个法案需要证据证明被告故意导致造成破坏的程序或数据的传播。《计算机欺诈和滥用法》还涵盖了涉及信息盗窃的案例。具体来说，这个法案规定，通过未经授权的方式访问计算机并获取任何有价值信息的行为均不合法。法院已经对“任何有价值的”赋予了广泛的解释，因此，《计算机欺诈和滥用法》已经不仅仅适用于信息盗窃的情况。例如，法院规定，仅仅是使用了计算机就可以算作是“任何有价值的”。

在法律界，隐私权是另一个，也许是最富争议的，网络问题。这样的问题包括雇主是否有权监控员工的通信，以及因特网服务提供方在多大程度上有权访问其客户正在交流的信息，这些问题已经得到了相当多的关注。在美国，这些问题有许多已经在1986年的《电子通信隐私法》（Electronic Communication Privacy Act，ECPA）中提到，这个法案起初是为控制搭线监听设立的。虽然法案很长，但是仍能从几段简短的摘录中捕捉到它的意图。例如，它指出：

除本章中特别提到的之外，任何人有意截取、力图截取或者唆使他人截取或力图截取任何有线、口头或电子通信……都应按照子条款（4）的规定受到处罚，或者按照子条款（5）的规定受到起诉。^[3]

[3] 资料来源：Peter Toren, *Intellectual Property and Computer Crimes*, 2003。

还有

.....任何向公众提供电子通信服务的个人或实体，不得在服务时故意将任何与那个服务有关的通信的内容.....泄露给除这个通信的收件人或预期收件人，或者这些收件人或预期收件人的代理人之外的任何个人或实体。

基本知识点

- 隐私和安全问题出现在计算系统和制品的开发和使用中。

简而言之，ECPA确认了个人秘密通信的权利——因特网服务提供方泄露其客户的通信信息是非法的，并且未授权人员窃听他人的通信也是非法的。但是，ECPA留下了一些有争论的地方。例如，关于雇主监控雇员的通信的权利问题变成了一个授权问题，在这个问题上，当雇员用雇主的设备实施通信时，法院倾向于给予雇主这项权力。

此外，这个法案在某些条件限制下，会给某些政府机构监控电子通信的权利。这些规定已经引发了很多争论。例如，在2000年，美国联邦调查局（FBI）披露了其Carnivore系统的存在，该系统能报告一个因特网服务提供方的所有订户的通信，而不仅仅是法庭指定的目标。在2001年，为了回应针对世界贸易中心的恐怖袭击，美国国会通过了富有争议的《美国爱国者法案》（USA PATRIOT Act），该法案修改了政府机构所受的限制。后来在2013年，有消息透露，这些法律已经被解释为授权无差别地收集大量普通美国民众电话和因特网使用数据。

提供这种监控权利除了引起了法律和道德上的争论，还引起了与我们的研究更相关的一些技术问题。一个问题是，为提供这些能力，必须构建和编写通信系统，使其可以监控通信。建立这样的能力是《通信协助执法法案》（Communication Assistance for Law Enforcement Act, CALEA）的目标。它要求电信运营商修改它们的设备以适应法律强制窃听，而这个需求一直比较复杂，实现起来也非常昂贵。

另一个富有争议的问题涉及政府监控通信的权利与公众使用加密的权利之间的冲突。如果正被监控的报文加密得很好，那么窃听通信对执法机构来说就没有多大价值。美国、加拿大和欧洲各国政府正在考虑要求注册加密密钥的系统，但是这样的要求受到了企业界的反对。毕竟，由于商业间谍的存在，要求注册加密密钥会让许多守法公司和公民感到不舒服是很容易理解的。注册系统的安全性有多高？随着软件提供商将更强的加密作为消费产品中的基准功能，一些执法机构的反应是强烈呼吁给予例外。2016年，美国联邦调查局要求苹果公司构建新软件，允许他们解锁前一年圣贝纳迪诺枪击事件中一名攻击者的iPhone。苹果公司拒绝了，理由是这会开创一个危险的先例，这种漏洞可能会破坏全球苹果设备的安全性。在法院裁决这起有争议的案件之前，美国联邦调查局找到了一条进入锁定电话的替代途径。

最后，作为识别因特网相关法律问题的范畴的一种手段，我们引用1999年美国的《反网络域名抢注消费者保护法》（Anticybersquatting Consumer Protection Act），设计这个法案是为了

防止冒名顶替者建立一个看上去相似的域名（这个阴谋就称为域名抢注）来欺骗组织。这个法案禁止使用与其他商标或“普通法商标”一样的或相似的容易引起混淆的域名。一个作用是，尽管该法案没有将域名的投机买卖（就是注册一个可能有需求的域名，以后再将该域名的所有权卖出的过程）视为不合法，但是它把这种行为限制定在了通用域名上。因此，域名的投机买卖者可能可以合法地注册一个通用域名，如GreatUsedCars.com，但是如果Big Al公司已经在从事二手车业务，他就不能注册域名BigAlUsedCars.com。这种区别经常会在与《反网络域名抢注消费者保护法》相关的法律诉讼案中成为争论的主题。

4.6节问题与练习

1. 什么是网络仿冒？如何保护计算机来抵御网络仿冒？
2. 能放在域网关的防火墙的类型和能放在域内单个主机上的防火墙的类型之间有什么区别？
3. 从技术上说，术语数据指信息的表示，而信息指基本的含义。口令保护的是数据还是信息？加密保护的是数据还是信息？
4. 和较传统的加密技术相比，公钥加密技术的优势是什么？
5. 与防备网络安全问题的法律尝试相关的问题有哪些？

复习题

（带*的题目涉及选读章节的内容。）

- 1．什么是协议？说出本章介绍的3个协议，并描述每个协议的用途。
- 2．描述客户/服务器模式。
- 3．描述对等计算模式。
- 4．说出3种分布式计算系统。
- 5．开放式网络和封闭式网络之间的区别是什么？
- 6．为什么CSMA/CD协议不能应用于无线网络？
- 7．描述在使用CSMA/CD协议的网络内，一台机器想要发送报文要遵循的步骤。
- 8．什么是隐藏终端问题？描述解决它的技术。
- 9．怎样区分集线器和中继器？
- 10．怎样区分路由器和诸如中继器、网桥及交换机这样的设备？
- 11．网络和互联网的区别是什么？
- 12．说出网络中用来控制报文发送权的两个协议。
- 13．使用32位因特网地址原先被认为是提供了足够大的扩展空间，但这个推测被证实并不准确。IPv6使用128位编址，能证明是足够

的吗？证明你的答案。（例如，你可以把可能的地址数量与世界的人口数量进行比较。）

14．用点分十进制记数法为下列位模式编码。

a．000001010001001000100011

b．1000000000100000

c．0011000000011000

15．下列点分十进制记数法表示的位模式分别是什么？

a．0.0

b．26.19.1

c．8.12.20.13

16．假设因特网上一个终端系统的地址是134.48.4.122，那么这个32位地址的十六进制记数法表示是什么？

17．什么是DNS查找？

18．如果一台计算机的助记因特网地址是batman.batcave.metropolis.gov，你对这台机器所在域的结构有什么猜测。

19．解释一下电子邮件地址kermit@an*****.com的组成。

20．在VoIP语境下，模拟电话适配器和嵌入式电话的区别是什么？

21．邮件服务器的作用是什么？

22．N单播与多播的区别是什么？

23．给出下列术语的定义。

a．名称服务器

b．接入ISP

c．网关

d．终端系统

24．给出下列名词的定义。

a．超文本

b．HTML

c．浏览器

25．因特网的许多“外行用户”经常混用[因特网](#)和[万维网](#)这两个术语。这两个术语的正确含义是指什么？

26．在浏览一个简单的网页时，让浏览器显示该文档的源版本，然后说出该文档的基本结构。特别是，说出该文档的头和体，并列出你在头和体中发现的一些语句。

27．列出5种HTML标签，并说出它们的含义。

28．修改下面的HTML文档，使单词“Rover”链接到URL为 http://an*****.org/pets/dogs.html 的文档。

```
<html>
<head>
<title>Example</title>
</head>
<body>
<h1>My Pet Dog</h1>
<p>My dog's name is Rover.</P>
</body>
</html>
```

29．画一个草图来说明下面的HTML文档在计算机屏幕上显示的信息。

```
<html>
<head>
<title>Example</title>
</head>
<body>
<h1> My Pet Dog</h1>
<img src = "Rover.jpg">
</body>
</html>
```

30．用本章中介绍的非正规XML风格设计一种标记语言，把简单的代数表达式表示为文本文件。

31．用本章中介绍的非正规XML风格设计一组标签，供字处理程序用来标记基础文本。例如，一个字处理程序将如何指示什么文本应该是粗体、斜体、带下划线的等？

32．用本章中介绍的非正规XML风格设计一组标签，用于根据文本项应该在打印页上出现的方式来标记运动图像评论。然后再设计一组标签，用于根据文本中这些项的含义来标记这些评论。

33．用本章中介绍的非正规XML样式设计一组标签，用于根据文本项应该在打印页上出现的方式来标记有关体育赛事的文章。然后再设计一组标签，用于根据文本中这些项的含义来标记这些文章。

34．说出下面URL的组成，并描述各项的含义。

```
http://life****.com/animals/moviestars/kermit.html
```

35．说出下列简写URL的组成。

a . http://www.far****.org/windmills.html

b . http://cas****.org/

c . www.cool****.com

36．如果要浏览器在下列两个URL“找文档”，浏览器的动作有什么不同？

<pre>http://star*****.univ***.org https://star*****.univ***.org</pre>

37．给出万维网上两个客户端活动的例子和两个服务器端活动的例子。

*38．什么是OSI参考模型？

*39．在一个基于总线拓扑结构的网络里，总线是机器为传输报文必须竞争的不可共享的资源。在这种情况下如何控制死锁（见选读的3.4节）？

*40．列出因特网软件层次结构的4层，并说明各层所完成的任务。

*41．传输层为什么把大的报文划分为小的分组？

*42．当某个应用程序要求传输层使用TCP来发送报文时，为了满足应用层的要求，传输层需要附加什么样的报文？

*43．在实现传输层时，什么情况下TCP优于UDP？什么情况下UDP优于TCP？

*44．说UDP是无连接协议的含义是什么？

*45．在TCP/IP协议层次结构里，为了用下列方法过滤进来的通信流，防火墙应该放在哪一层？

a．报文内容

b . 源地址

c . 应用类型

46 . 假定你想建立一个可以过滤掉包含某些术语和短语的电子邮件消息的防火墙。这个防火墙应该放在域的网关上，还是放在域的邮件服务器上？说明理由。

47 . 什么是代理服务器？使用代理服务器有什么好处？

48 . 总结公钥加密的原理。

49 . 一台空闲但未设防的PC是如何威胁因特网的？

50 . 因特网的全球性是如何限制因特网问题的法律解决方案的？

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

基本知识点

- 企业和政府的数字信息审查制度引起了法律和道德关注。
- 电子邮件、短信息服务（SMS）和聊天培育了新的沟通和协作方式。

- 因特网和万维网或积极或消极地影响了许多领域的生产率。
- 视频会议和视频聊天培育了新的沟通和协作方式。
- 社交媒体继续发展和培育了新的沟通方式。
- 云计算培育了新的沟通和协作方式。
- 因特网和万维网增加了沟通和协作的方法和机会。
- 因特网和万维网改变了很多领域，包括电子商务、医疗保健、信息和娱乐的获取，以及在线学习。
- 因特网和基于它的系统促进了协作。

1．将计算机和智能手机连接到因特网的能力使人们更易于以新奇的方式一起工作，使在家办公的观念流行起来。电子邮件、视频会议、文本处理和云中的软件协作工具可以取代许多传统的工作场所的面对面活动。这种变化有哪些利弊？它会影响自然资源的消耗吗？它会使家庭巩固吗？它会减少“办公室政治”吗？在家里办公的人和在现场办公的人会有同样的职务晋升机会吗？社会联系会被削弱吗？减少和同行之间的个人接触是会有正面的影响，还是会有负面的影响？

2．在线商品订购很快就超过了传统的“实体”店中“亲身”购物。这种购物习惯的转变对社会有什么影响？对大型购物中心呢？对你通常只逛不买的小店（如书店和服装店）呢？以尽可能最低的价格购买，到什么程度称之为好，到什么程度称之为不好？你是否有这样的道义

上的责任，多花一点钱购买一个商品来支持本地的商业？比较本地商店里的商品，然后通过因特网以较低的价格订购，这道德吗？这种行为的长期后果是什么？

3．政府对其公民访问因特网（或任何国际性网络）应该审查到什么程度？对于涉及国家安全的问题呢？可能发生哪些安全问题？

4．社交媒体站点允许因特网用户发布消息（常以匿名方式）和阅读其他用户发布的消息。这种服务的管理者应该对站点的内容负责吗？电话公司应该对电话的通话内容负责吗？食品杂货店的管理者要对店内的社团公告牌内容负责吗？

5．因特网的使用应该被监视吗？应该被管制吗？如果应该，由谁来管？管到什么程度？

6．你花费多少时间来使用因特网？那些时间花得值吗？上网改变你的社会活动了吗？你认为通过社交媒体和因特网与人交谈比面对面对人交谈更容易吗？

7．当你为计算机购买软件时，开发者通常要求你向开发者注册，以便你可以得到未来升级的通知。这种注册过程越来越多地通过因特网处理，经常要求你提供诸如姓名、地址以及你如何得知该产品等信息，然后开发者的软件自动把这些数据传输给开发者。如果开发者设计的注册软件在注册过程中还把额外的信息发送给了开发者，那么会出现什么道德问题？例如，注册软件可能会扫描你的系统的内容并报告找到的其他软件。

8．当你访问一个网站时，该网站有在你计算机内记录数据的能力，称为cookie，表明你曾经访问过该站点。然后这些cookie就可被用来识别回访的访问者并记录他们以前的活动，以便网站可以更高效地处理访问者对网站的未来访问。计算机上的cookie还提供了你访问网站的记录。网站应该有在你的计算机上记录cookie的功能吗？应该允许网站在你不知情的情况下在你的计算机上记录cookie吗？cookie的好处可能是什么？使用cookie可能会引发什么问题？

9．如果政府机构要求公司注册它们的加密密钥，公司还是安全的吗？

10．一般来说，出于礼貌我们不会为了安排周末外出之类的个人或社团的事情而给在工作场所的朋友打电话。类似地，我们大多数人也不愿意打电话到客户的家里介绍新产品。按照类似的习俗，我们会把婚礼请柬寄到客人的住所，而把商务会议的通知寄到出席者的工作地址。把给朋友的私人电子邮件通过他工作地点的邮件服务器发送合适吗？

11．假定一个PC的所有者让这台PC接入因特网，但最终这台PC被其他人用来实施拒绝服务攻击。这个PC的所有者该负多大的责任？你的回答和该所有者是否安装了正确的防火墙有关吗？

12．一个生产糖果或玩具的公司，在他们的公司网站上提供游戏，在推荐公司产品同时让孩子们娱乐，这种做法道德吗？如果游

戏是用来收集小孩信息的，那又如何？娱乐、广告和利用之间的界限是什么？

课外阅读

Antoniou, G., P. Groth, F. van Harmelem, and R. Hoekstra. *A Semantic Web Primer*, 3rd ed. Cambridge, MA: MIT Press, 2012.

Bishop, M. *Introduction to Computer Security*. Boston, MA: Addison-Wesley, 2005.

Comer, D. E. *Computer Networks and Internets*, 6th ed. Upper Saddle River, NJ: Prentice-Hall, 2014.

Comer, D. E. *Internetworking with TCP/IP*, Vol. 1, 6th ed. Upper Saddle River, NJ: Prentice-Hall, 2013.

Goldfarb, C. F. and P. Prescod. *The XML Handbook*, 5th ed. Upper Saddle River, NJ: Prentice-Hall, 2004.

Halsall, F. *Computer Networking and the Internet*, 5th ed. Boston, MA: Addison-Wesley, 2005.

Harrington, J. L. *Network Security: A Practical Approach*. San Francisco: Morgan Kaufmann, 2005.

Kurose, J. F. and K. W. Ross. *Computer Networking: A Top Down Approach Featuring the Internet*, 7th ed. Boston, MA: Pearson, 2017.

Peterson, L. L. and B. S. Davie. *Computer Networks: A Systems Approach*, 5th ed. San Francisco: Morgan Kaufmann, 2011.

Rosenoer, J. *CyberLaw: The Law of the Internet*. New York: Springer, 1997.

Spinello, R. A. *Cyberethics: Morality and Law in Cyberspace*, 6th ed. Sudbury, MA: Jones and Bartlett, 2016.

Stallings, W. *Cryptography and Network Security*, 7th ed. Upper Saddle River, NJ: Pearson, 2016.

Stevens, W. R. *TCP/IP Illustrated, Vol 1*, 2nd ed. Boston, MA: Addison-Wesley, 2011.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第5章 算 法

本第0章中我们得知，计算机科学的核心主题是对算法的研究。现在是我们关注这个核心主题的时候了。我们的目标是探索足够的基础内容，以便我们能够真正理解和认识计算科学。

本章内容

5.1 算法的概念

5.2 算法的表示

5.3 算法的发现

5.4 迭代结构

5.5 递归结构

5.6 效率和正确性

持久理解和学习成果

创造性开发可能是创建计算制品的必不可少的过程。AP[®]CSP

学习成果 在创建计算制品时应用创造性开发过程。

算法是可由计算机执行的过程的精确指令序列，是使用程序设计语言实现的。AP[®]CSP

学习成果 开发用程序中实现的算法。

学习成果 用语言表达算法。

算法能解决许多计算问题，但不能解决所有计算问题。AP[®]CSP

学习成果 以分析的方式和凭经验的方式评估算法的效率、正确性和清晰度。

我们已经知道，在计算机能够执行一个任务之前，必须给它一个算法，准确地告诉它做什么。因此，算法的研究是计算机科学的基石。在本章中，我们将介绍算法研究的许多基本概念，包括算法的发现和表示问题，以及迭代和递归这两个主要控制概念。同时，我们还会介绍几个有关搜索和排序的著名算法。下面先回顾一下算法的概念。

5.1 算法的概念

在第0章中，我们把算法非正式地定义为一组步骤，这些步骤定义了完成任务的方法。在本节中，我们将进一步讨论算法的基本概念。

5.1.1 非正式的回顾

在前面的学习中，我们已经遇到过许多算法。我们发现了用来进行数制转换的算法、检测和纠正数据错误的算法、压缩和解压缩数据文件的算法、在多任务处理环境中控制多道程序设计的算法以及很多其他算法。此外，我们已经看到，CPU所遵循的机器周期只不过是下面这个简单算法。

只要未执行停止指令就执行下面的步骤：

- a. 取一条指令；
- b. 译码该指令；
- c. 执行该指令。

就像图0-1中描述魔术的算法所展示的那样，算法并不局限于技术活动，实际上，它甚至可以用来描述剥豌豆壳这样的普通活动。

获得一篮子未剥壳的豌豆和一只空碗。只要篮子里还有未剥壳的豌豆就执行下面的步骤：

- a. 从篮子里拿出一颗豌豆；
- b. 剥开豌豆的豆荚；
- c. 把从豆荚上剥落的豌豆放到碗里；
- d. 扔掉空豆荚。

实际上，许多研究人员相信，人脑中的每个活动，包括幻想、创造和决策，实际上都是算法执行的结果——这个猜想我们将在学习人工智能（第11章）时介绍。

但是，在继续深入研究之前，我们先来考虑一下算法的正式定义。

5.1.2 算法的正式定义

在日常生活中，我们可以使用非正式的、未经严格定义的概念，而且这种概念也很常见，但是科学必须建立在定义明确的术语之上。因此，我们来考虑一下图5-1给出的算法的正式定义。

算法是定义一个可终止过程的一组无歧义的、可执行的步骤的有序集合。

图5-1 算法的定义

注意，该定义要求算法中的一组步骤是有序的。这意味着，算法中的各个步骤必须有一个就执行顺序而言非常明确的结构。在最简单的情况下，算法的步骤是**有次序的**（sequenced），这意味着它们是按照给定的顺序执行的。不过，这并不意味着这些步骤必须按照“第1步到第2步，再到下一步”这样的线性顺序执行。有些算法，称为并行算法，包含的步骤序列不止一个，每个序列都被设计成由多处理器机器中的不同处理器执行。在这种情况下，整个算法并不只包含一个遵照第1步、第2步这样的顺序的执行流，其结构是一种多执行流结构，这些执行流在整个任务的不同部分被不同的处理器执行时不断分支和再接合。（我们会在第6章中再次讨论这个概念。）其他例子包括第1章中讲的触发器电路执行的算法，这个电路中每个门电路都完成整个算法的一步。这里，这些步骤是按照因果关系排序的，因为每个门电路的动作都是通过电路传播的。

基本知识点

- 定序是按给定语句的顺序应用每个算法步骤。

接下来，我们考虑算法必须由可执行的步骤组成这一要求。为了满足这个条件，我们考虑下面这条指令：

给出一个所有正整数的列表

因为正整数有无穷多个，所以这条指令是不可能完成的。因此，任何包括这条指令的指令集都不能称作一个算法。计算机科学家使用**有效的**（effective）这个术语来表示可执行的概念。也就是说，说算法中的一个步骤是有效的就意味着它是可行的。

图5-1中的算法定义的另一要求是算法中的步骤必须是无歧义的。这意味着，在算法的执行过程中，这个过程状态中的信息必须足以唯一且完全地确定每个步骤所需的动作。换句话说，算法中的每个步骤的执行都不需要创造性的技能，只要求遵照指令执行。（在第12章中，我们要学习的“算法”称为非确定性算法，那些算法不受这里的限制，属于另外一个重要的研究主题。）

图5-1给出的定义还要求，算法定义的是一个可终止的过程，也就是说，一个算法的执行必须能够最终结束。这个要求源自理论计算机科学，其目标是要回答诸如“算法和机器的最终限制是什么？”之类的问题。这里，计算机科学试图寻找下面两种问题的区别：一种是通过算法获得答案的问题；另一种是其答案超出算法系统能力的问题。在这个语境下，在以答案告终的过程与那些只能向前执行而不能得到结果的过程之间存在着一条分割线。

可是，还是有一些不可终止的过程是非常有意义的，包括监视病人的生命特征和维持飞行器的飞行高度等。有些人可能辩称这些问题只是算法的重复，其中的每个算法都会在到达结束状态之后自动重复执行。另外一些反对这一论点的人可能认为，这种说法只不过是一种对正式定义限制的过度坚持。不管是哪种情况，结果都是**算法**这个术语经常被用在一些不必非要定义可终止的过程的实用的或者非正式设置的几组步骤中。例如，长除法“算法”没有为1除以3定义一个可终止的过程。从技术上讲，这些例子都表示对该术语的误用。

5.1.3 算法的抽象本质

强调算法与其表示之间的区别是非常重要的，这就好像一个故事和一本书的区别。故事在本质上是抽象的，或者说是概念上的；而书是故事的物理表示。如果一本书被翻译成其他语言或者以另外一种样式出版，那么只是这个故事的表示形式改变了，而故事本身并没有变化。

同样，算法是抽象的，与它的表示是有区别的。一个算法可以用多种方式来表示。例如，将温度读数从摄氏转换为华氏的算法传统上可以用下面的代数公式表示：

$$F = \left(\frac{9}{5} \right) C + 32$$

但也可以用下面的指令表示：

将摄氏读数乘以 $\frac{9}{5}$ ，然后在乘积上加32

甚至可以用电路的形式表示。无论哪种情况，底层算法都是一样的，只是表示方式不同罢了。

算法和它的表示之间的区别提出了我们在传达一个算法的时候存在的问题。一个常见的例子是，一个算法必须描述到什么样的细致程度。对气象学家来说，指令“将摄氏读数转换为相应的华氏读数”就足够了，但是，对一个需要更详细描述的外行来说，这个指令可能就是模糊的、有歧义的。然而，问题并不在于底层算法，而是算法的表示对外行来说不够详细。在下一节中，我们会看到原语的概念是如何用于消除算法表示中的这种歧义性问题的。

最后，在算法及其表示这个主题上，我们应该明确另外两个相关概念——程序和进程——的区别。程序是算法的表示。（这里，我们没有在正式意义上使用术语**算法**，因为许多程序是不可终止的“算法”的表示。）实际上，在计算界，术语**程序**（program）通常是指被设计成计算机应用的算法的正式表示。在第3章中，我们把**进程**（process）定义为执行程序的活动。然而，需要注意的是，执行程序就是执行由该程序表示的算法，所以一个进程可以等价地定义为执行一个算法的活动。我们可以得到这种结论：程序、算法和进程既是不同的又是有关联的实体。程序是算法的表示，而进程又是执行算法的活动。

5.1 节问题与练习

1. 简述进程、算法和程序之间的区别。
2. 给出一些你熟悉的算法的例子。它们真的是精确意义上的算法吗？
3. 指出0.1节中给出的算法的非正式定义中存在的一些含糊的地方。
4. 从什么意义上说，由下列指令列表描述的步骤不能构成算法？
步骤1：从你的口袋里取出一枚硬币并且把它放在桌子上。
步骤2：返回步骤1。

5.2 算法的表示

在本节中，我们考虑与算法表示有关的问题。我们的目标是引入原语和伪代码的基本概念，并且建立一种为我们所用的算法表示系统。

5.2.1 原语

一个算法的表示需要使用某种形式的语言。对人类来讲，这可能是一种传统的自然语言（英语、西班牙语、俄语、日语），也可能是一种图形语言，如图5-2所示，这张图描述了用一张正方形的纸折出一只鸟的算法。然而，这种自然的沟通方式常常会引起误解，有些时候

是因为算法描述中使用的术语可能拥有多种含义。（句子“Visiting grandchildren can be nerve-racking”可能表示孙子孙女们来访时会惹出事情，也可能表示去看孙子孙女们是一件很费周折的事情。）所需的详细程度引发的误解也可能导致问题。很少有读者能够按照图5-2给出的步骤成功地折出一只鸟来，但是对于一个专门学过折纸的学生，按照图中给出的步骤折出一只鸟可能就没什么困难。简而言之，当用来描述算法表示的语言并没有被准确定义或者并没有给出足够详细的信息的时候，就会产生交流问题。

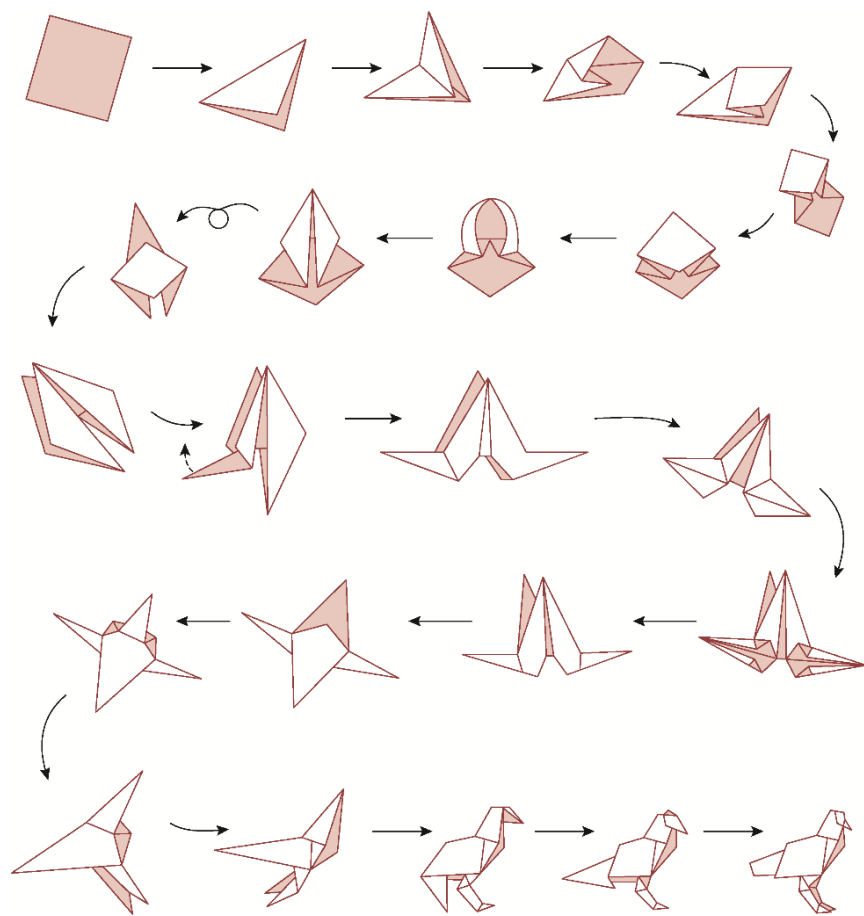


图5-2 将一张正方形的纸折成一只鸟

计算机科学解决这些问题的途径是建立一组定义明确的构建块，利用它们来构造和执行算法的表示。这种构建块称作**原语**（primitive）。赋予原语准确的定义消除了很多由歧义造成的问题，并且要求用这些原语来描述算法建立了统一的细致程度。原语的集合以及说明如何组合这些原语来表示比较复杂的想法的规则集合就构成了**程序设计语言**（programming language）。

基本知识点

- 用程序设计语言描述的算法可以在计算机上执行。

每个原语都有其自己的语法和语义。语法是指原语的符号表示，语义是指该原语的含义。*air*一词的语法由3个符号组成，而其语义是一种充满整个世界的气体物质。作为一个例子，图5-3描述了折纸术中使用的一些原语。

为了获得用于表示计算机执行的算法的原语集合，我们需要借助于为机器执行而设计的单个指令。如果算法在这个细致程度上表达，我们肯定会有一个适合机器执行的程序。然而，在这个细致程度上表达算法是非常单调的，所以我们一般使用一些“更高级的”原语，其中每个原语都是由机器语言提供的较低级的原语组成的一个抽象工具。因此，我们可以得到一种正式的程序设计语言，在这种语言中，算法可以在比机器语言更高的概念层次上表达。我们将在下一章中讨论这种程序设计语言。

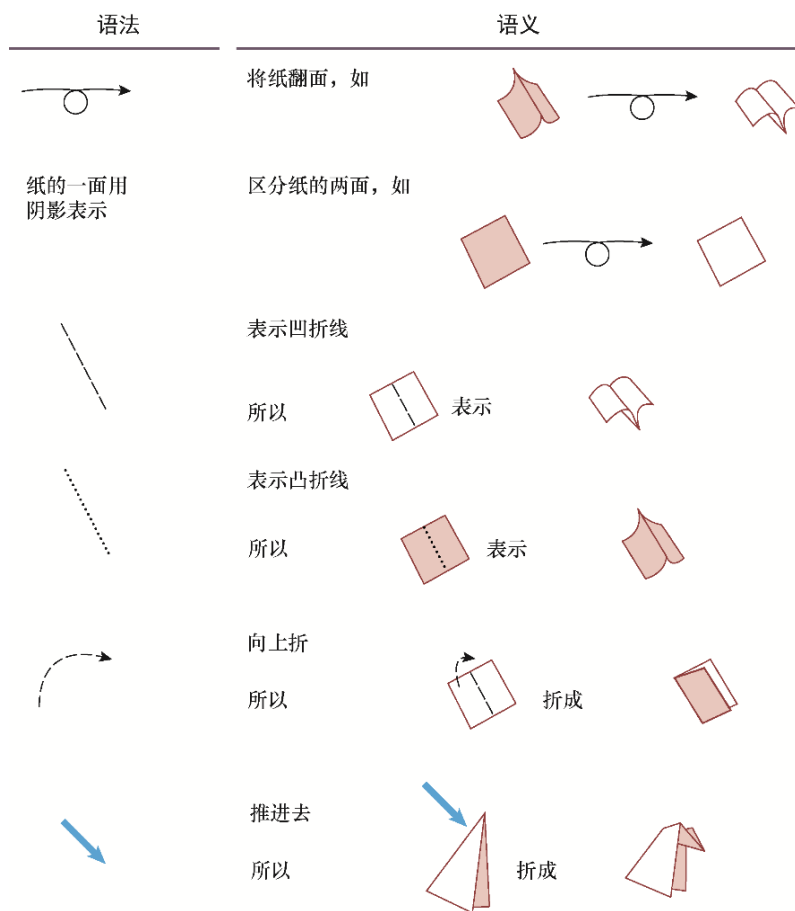


图5-3 折纸术的原语

5.2.2 伪代码

现在，我们暂时不介绍正式的程序设计语言，而是介绍一种不太正式但更直观的符号系统，这个系统称作伪代码。一般而言，**伪代码**（pseudocode）是一种用于在算法开发过程中非正式地表达想法的符号系统。伪代码的优点是比程序设计语言更容易让人理解，且比自然语言更准确。

基本知识点

- 用自然语言和伪代码描述算法是为了让人可以理解它们。

一种获得伪代码的方法是，简单地放宽正式程序设计语言的规则，借用语言的语法—语义结构，与不太正式的结构混在一起。因为现在有很多种程序设计语言，所以这样的伪代码变体有很多。两种特别流行的伪代码是Algol语言和Pascal语言的宽松版本，主要是因为在过去的几十年里，它们被广泛用于教材和学术论文。最近，Java语言和C语言的伪代码怀旧者激增，同样是因为大多数程序员都至少具备阅读这些语言的知识。无论伪代码借用哪种语言的语法形式，其本质目的都是为表示算法服务：伪代码必须有一种一致且简洁的用来表示反复出现的语义结构的表示法。为了我们的目的，我们将使用一种类似Python的语法来表示贯穿本书其余部分的伪代码。我们的一些伪代码语义结构将借用前面章节中的语言构造，其他结构将在本书的后续几章中正式介绍。

这样一种反复出现的语义结构就是保存计算的值。例如，我们计算了支票账户和储蓄账户上的余额总和，并打算命名和保存这个结果以便以后引用。在这种情况下，我们用

```
name = expression
```

的形式来表达，其中，name（名字）是我们要引用结果的名字，expression（表达式）描述的是其结果要被保存的计算。这个伪代码结构直接照搬了Python的**赋值语句**（assignment statement），该语句在第1章介绍将值存入Python变量的时候介绍过。例如，语句

$\text{RemainingFunds} = \text{CheckingBalance} + \text{SavingsBalance}$
--

是一条赋值语句，它把CheckingBalance与SavingsBalance的和赋给了名字RemainingFunds。因此，RemainingFunds可以用在将来引用该和的语句中。

算法设计中的算法表示

在设计一个复杂的算法时，设计者要了解众多不相关的概念，这种要求可能超过了人脑的能力。因此，复杂算法的设计者需要一种方法来记录和回忆不断发展的算法的各个部分，以备不时之需。

在20世纪50年代和60年代，流程图（算法可以由箭头相互连接的几何图形表示）是最顶尖的设计工具。然而，流程图经常变成一个由相互交叉的箭头组成的复杂的网，这样就给理解底层算法的结构带来了难度。因此，流程图这种设计工具只能让位于其他表示方法。一个例子就是本书中使用的伪代码，通过这种伪代码，算法可以表示成定义明确的文本结构。当我们的目标是表示算法而非设计的时候，流程图仍旧是有益的。例如，图5-8和图5-9用流程图展现了由流行的控制语句表示的算法结构。

寻找更好的设计表示法是一个持续性的过程。在第7章中，我们将看到使用图形技术来辅助大型软件系统全局设计的趋势，但是伪代码在设计系统中一些较小的过程化组件中仍然很流行。

另一个反复出现的语义结构是，根据某个条件的真与假从两个可能的活动中**选择**一个。这样的例子有：

如果国内生产总值增长了，就买进普通股，否则卖出普通股。

若国内生产总值增长了，则买进普通股，否则卖出它。

买进或卖出普通股取决于国内生产总值是增长还是降低。

基本知识点

- 选择使用布尔条件来确定使用算法的两个部分中的哪一个。

这些语句中的每一句都可以被重写为符合如下结构的形式：

```
if (条件):  
    活动  
else:  
    活动
```

这里，我们使用关键字if（如果）和else（否则）来指示这个主结构中的不同子结构，使用冒号和缩进限定这些子结构的边界。条件和else后面总是紧跟着一个冒号。相应的活动会被缩进。如果活动由多个步骤组成，则对这些步骤应该采用同样的缩进。通过对我们的伪代码采用这种语法结构，我们便得到了可以表达这种常用语义结构的统一方法。因此，尽管语句

根据年份是否是闰年，分别将总数除以366或365。

可能会产生一种更富有创造性的文字风格，但是我们坚持选择下面这种简单的形式：


```
if (年份是闰年):  
    每日总计 = 总数 / 366  
else:  
    每日总计 = 总数 / 365
```

对于不涉及else活动的情况，我们也可以采用下面这种较短的语法：

```
if (条件):  
    活动
```

使用这种表示形式，语句

```
如果处于销售下降的情况下，价格降低5%。
```

将可以简化为

```
if (销售下降):  
    价格降低5%
```

还有一种常见的语义结构是，只要某个条件仍然为真就重复执行一条语句或一组语句，即**迭代**（iteration）。

基本知识点

- 迭代是算法一部分的重复，一直重复到满足条件或者达到指定次数为止。

非正式的例子有：

只要有票可卖，就继续卖票。

和

当有票可卖时，就继续卖票。

对于这两种情况，我们统一采用下列伪代码：

```
while (条件):  
    活动
```

简而言之，这个语句表示检查条件，如果条件为真，则执行活动，并返回再次检查条件。但是，如果发现条件为假，则转移到while结构后的下一个指令。于是，前面的两个语句可以简化为

```
while (仍有票可卖):  
    卖票
```

迭代通过定序和选择来完成我们构建算法所需的伪代码构建块的集合。然而，还有几种额外的快捷方法可以提升我们用于编写简洁算法的伪代码的可用性。

基本知识点

- 定序、选择和迭代是算法的构建块。
- 每个算法都可以只使用定序、选择和迭代来构造。

在许多程序设计语言中，缩进通常可以提高程序的可读性。在Python以及由Python衍生的伪代码中，缩进也是必不可少的符号。例如，在下面的语句中：

```
if (未下雨):  
    if (温度 == 热):  
        去游泳  
    else:  
        打高尔夫  
else:  
    看电视
```

缩进告诉我们，除非条件为“未下雨”，否则根本不会问“温度是否等于热”这个问题。有关温度的问题是**嵌套**（nested）在if语句内部的，实际上是外部if语句条件为真时才会执行的活动。同样，缩进还告诉我们，“else: 打高尔夫”属于内部if语句，不属于外部if语句。因此，我们将在伪代码中使用缩进。

我们用伪代码来描述那些可在其他应用中用作抽象工具的活动。对于这样的程序单元，计算机科学有许多术语，如子程序、子例程、过程、模块和函数，其中每个在含义上都有自己的变化。我们将按照Python的约定，在我们的伪代码中使用术语**函数**（function），并用Python关键字def来声明用于标识伪代码单元的标题。更准确地说，我们将以下列形式的语句开始一个伪代码单元：

```
def name():
```

其中，name是该单元特有的名字。在这个引导性语句的后面是一系列定义该单元动作的语句。例如，图5-4是名为Greetings的函数的伪代码表示，该函数打印3次“Hello”。

```
def Greetings():  
    Count = 3  
    while (Count > 0):  
        print the message "Hello"  
        Count = Count - 1
```

图5-4 Greetings函数的伪代码

当一个函数完成的任务在伪代码的其他地方需要时，只需通过名字来请求它。例如，如果有取名为ProcessLoan和RejectApplication的两个函数，那么我们可以通过编写下面的语句来在if-else结构内请求它们的服务：

```
if (...):  
    ProcessLoan()  
else:  
    RejectApplication()
```

当测试条件为真时，执行函数ProcessLoan；当条件为假时，执行函数RejectApplication。注意，我们也可以将这个看作算法的组合，因为每个函数本身就是一个算法。也就是说，我们组合“ProcessLoan()”算法和“RejectApplication()”算法，再加上一些额外的算法构建块，就能创建一个审查贷款申请的新算法，然后将其命名为一个函数并复用为更大算法的一部分。

基本知识点

- 算法可以组合形成新算法。

如果函数要在不同的情况下使用，那么应该将它们设计得尽可能通用。一个给名字列表排序的函数应该设计成能够对任何列表（而不

是特定的列表) 排序, 因此应该按照这样的方式来编写该函数: 不在函数中指定要排序的列表。事实上, 该列表应在这个函数的伪代码里以类属名来指称。

在伪代码里, 我们将采用这样的约定: 将这些称为**参数** (parameter) 的类属名列在括号里, 放在标识函数名的那一行上。例如, 一个名为Sort的函数 (被设计成用来对任何名字列表进行排序) 以下列语句开始:

```
def Sort (List):
```

在后面的伪代码里, 在需要引用要排序的列表的地方, 就可以使用类属名List。同样, 当需要Sort服务时, 我们就会确定哪个列表要替代函数Sort中的参数List。因此, 根据需要, 我们可以写成

```
Sort(组织的成员列表)
```

和

```
Sort(婚礼宾客列表)
```

命名程序中的项

在自然语言中, 项 (item) 通常使用多单词的名称, 如“cost of producing a widget”或“estimated arrival time”。经验表明, 在算法的表示中使用这种多单词的名称会使算法的描述复杂化。最好让每个项使用一个连续的文本块来标识。多年来, 有许多技巧用于将多个单词紧缩成一个词法单元, 以便为程序中的项提供描述性的名称。一种技巧

是使用下划线来连接单词，产生像`estimated_arrival_time`这样的名称。另一种技巧是用大写字母帮助读者理解紧缩的多单词名称。例如，我们可以对每个单词的首字母使用大写，获得像`EstimatedArrivalTime`这样的名称。这种技术称为Pascal大小写（Pascal casing），因为该样式在Pascal程序设计语言的用户中很流行。一种Pascal大小写的变形称为驼峰式大小写（camel casing），该样式除首字母小写外，其他都和Pascal大小写相同，如`estimatedArrivalTime`。本书中倾向于使用Pascal大小写，但这种选择很大程度上是个人偏好。

5.2节问题与练习

1. 一种语境下的原语可能是另一种语境下的原语的组合。例如，我们的`while`语句是我们伪代码中的一个原语，但是它最终可以实现为机器语言指令的组合。给出两个非计算机环境下的这种现象的例子。

2. 从何种意义上讲，函数结构就是原语结构？

3. 欧几里得算法通过下列过程求两个正整数 X 和 Y 的最大公约数：

只要 X 和 Y 的值都不为0，就用较大的数除以较小的数，并将得到的余数赋给较大的数。最大公约数（如果存在）将是剩下非零值。

用我们的伪代码表示这个算法。

4. 描述一个在计算机程序设计以外的学科里使用的原语集合。

5.3 算法的发现

程序开发由两个活动组成：发现潜在的算法和以程序的方式表示算法。到现在为止，我们一直在关注算法表示的问题，而没有考虑算法起初是如何被发现的。然而算法的发现在软件开发过程中往往是更具有挑战性的步骤。毕竟，发现一个解决问题的算法需要找到一个解决该问题的方法。因此，要理解算法是如何发现的就要理解问题求解的过程。

5.3.1 问题求解的艺术

问题求解的技巧和了解更多相关知识的需求并不是计算机科学独有的，而是几乎与所有领域都相关的主题。算法发现的过程和一般问题的求解过程之间的密切联系，使计算机科学家们和其他学科的科学学家们一起寻找更好的问题求解的技巧。最终，人们希望把问题的求解过程简化为一个算法，但是这已经被证明是不可能的。（这是第12章的内容的结果，我们将在第12章中说明，有些问题是没有算法解的。）因此，与其说解决问题的能力是需要学习的精确科学，不如说它是有待开发的技艺。

基本知识点

- 创建计算制品用迭代和经常性的探索过程将想法变成有形的形式。

作为问题求解难以琢磨、颇具艺术性的本质的证据，数学家波利亚（G. Polya）在1945年提出的以下非严格定义的问题求解阶段，其蕴涵的基本原理直至今日仍然是很多人教授问题求解技巧的基础。

第1阶段 理解问题。

第2阶段 设计一个解决问题的计划。

第3阶段 完成计划。

第4阶段 评估解决方案的准确度以及它作为解决其他问题的工具的潜力。

我们把上述阶段移植到程序开发的语境中，这些阶段变成：

第1阶段 理解问题。

第2阶段 得到一个算法函数如何求解这个问题的思路。

第3阶段 系统地阐述这个算法并用程序将其表达出来。

第4阶段 评估程序的准确度以及它作为解决其他问题的工具的潜力。

在描述完波利亚的观点后，我们要强调的是，这些阶段并不是在尝试解决问题时要遵循的步骤，而是在求解过程中有时需要完成的阶段。这里的关键词是遵循。仅遵循这些步骤是不能解决问题的，要解决问题必须有主动开创的精神。如果在解决问题的时候总是抱有“现在

我完成了第1阶段，该开始第2阶段了”之类的想法，那么你不太可能成功。然而，如果你仔细考虑问题并且最终解决了它，那么当你回过头来看你都做了些什么的时候，你很可能会发现，你确实经历了波利亚所描述的各个阶段。

另一个重要观点是，波利亚所描述的各个阶段并不一定是按顺序完成的。成功的问题求解者通常是在完全理解问题本身（第1阶段）之前就开始构思解决问题的策略（第2阶段）。然后，如果他们的策略失败了（在第3阶段或者第4阶段），他们会对这个问题的复杂程度有更深入的理解。基于这些比较深入的理解，他们会回过头去构思其他更有希望成功的策略。

必须记住的是，我们正在讨论问题如何解决，而不是我们希望问题如何解决。在理想情况下，我们希望消除前面描述的试错过程中固有的浪费。在开发大型软件系统的情况下，如果在第4阶段这么晚才发现理解错误，会导致极大的资源浪费。避免这样的灾难是软件工程师的主要目标（第7章），他们习惯于在寻求解决方案之前，坚持先对该问题有一个全面透彻的理解。当然，有人可能会说，在一个问题被解决之前是不可能被真正理解的。起码事实上，问题没有解决表明缺乏对问题的理解。因此，坚持在提出任何解决方案之前完全理解这个问题的想法看起来有些过于理想化。

作为一个例子，我们考察下列问题：

人物A承担了确认人物B的3个孩子的年龄的任务。B告诉A说3个孩子的年龄之积是36。在考虑了这个线索以后，A要求B给出其他线索，于是B将3个孩子的年龄之和告诉了A。A再次要求B给出其他线索，于是B又告诉A最大的孩子弹钢琴。在得到这个线索之后，A将3个孩子的年龄告诉了B。

这3个孩子的年龄分别是多少？

乍一看，最后一条线索似乎与该问题完全没有关系，但是显然，正是因为这条线索，A最后确定了3个孩子的年龄。这是为什么呢？让我们制订一个解决计划并且遵循这个计划，尽管我们对这个问题还有很多疑问。我们的计划是跟踪问题陈述所描述的步骤，同时在这个进程中记录对A有用的信息。

给A的第一条线索是，3个孩子的年龄之积是36。这意味着表示3个年龄的三元组肯定是图5-5a中列出的三元组之一。第二条线索是期望的三元组内3个数字的和。我们没有被告知这个和到底是多少，但是我们知道这个信息不足以让A得到正确的三元组；因此，期望的三元组肯定是其和在图5-5b中的表里至少出现两次的那个。但是，图5-5b中出现的具有相同和的三元组只有(1, 6, 6)和(2, 2, 9)，这两组数字的和均是13。这是在给出最后一条线索时A可用的信息，此时，我们终于明白了最后一条线索的意义。这个信息与弹钢琴本身没有什么关系，而是说明了只有一个孩子年龄最大这一事实。这条线索将三元组(1, 6, 6)排除，因此我们可以得出结论，3个孩子的年龄分别是2、2和9。

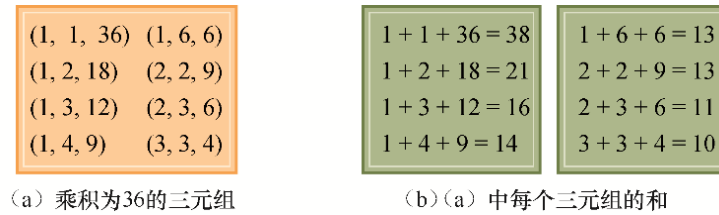


图5-5 分析可能性

在这个例子中，直到我们尝试实施解决问题的计划（第3阶段）的时候，才完全理解了这个问题（第1阶段）。如果我们坚持要先完成第1阶段，那么可能根本得不到孩子们的年龄。这种问题求解过程中的不规则性是开发问题求解的系统方法的根本困难。

另一个不规则性在于，那些对一个问题还没有取得明显成功的潜在的问题求解者，可能会在完成其他任务的时候，突然发现原来那个问题的解决方案。赫尔曼·冯·亥姆霍兹（H. von Helmholtz）早在1896年就发现了这种现象，数学家亨利·庞加莱（Henri Poincaré）在巴黎心理学会的一次演讲中讨论了这种现象。在这个演讲中，庞加莱描述了他获得问题解决方案的经历：他将原来的问题放在一边儿，在开始做其他工作之后，突然意识到原来的问题的一个解决方案。这种现象反映了这样一个过程：大脑的潜意识部分好像一直在思考问题，如果成功，便会把解决方案推给显意识。今天，我们把有意识地解决问题与突然的灵感之间的这个时期称作孵化期（incubation period），对这个时期的理解仍旧是当前研究的目标。

5.3.2 迈出第一步

前面，我们一直从有点儿哲学的角度讨论问题求解，回避了直接面对我们应该如何解决问题的问题。当然，有很多问题求解的方法，每种方法都可以在某些场合获得成功。我们稍后将简要地介绍其中一些方法。目前，我们注意到，这些技术中似乎有一条共同的主线，简单地说就是要“迈出第一步”。作为一个例子，让我们考虑下面这个简单问题。

在A、B、C和D进行赛跑之前，他们做了下列预测：

A预测B将会获胜。

B预测D将是最后一名。

C预测A将是第三名。

D预测A的预测将是正确的。

这几个预测只有一个是正确的，并且是最后的获胜者做出的预测。请据此给出比赛结束时A、B、C、D的名次。

在读了这个问题并且对数据进行分析之后，我们应该很快就可以意识到，因为A和D的预测是相同的，并且只有一个预测是正确的，所以这两个预测都是错误的。因此A和D都不是获胜者。此时，我们已经为解决这个问题迈出了第一步，并且发现获得问题的完整的解决方法的过程只是以此为基础进一步扩展我们的知识。如果A的预测是错误的，那么B也不是获胜者。这样就只剩下获胜者是C这一个选择了。因此，C赢得了比赛，并且C的预测是正确的。从而，我们知道A是第三

名。这就意味着最后的比赛名次是CBAD或者CDAB，但是，因为B的预测肯定是错误的，所以前者被排除了。因此，最后的比赛名次是CDAB。

当然，知道怎样迈出第一步并不等于知道如何去做这件事。得到立足点，并认识到如何把对于问题的初始介入扩展为完整的问题解决方案，需要问题求解者的创造性投入。然而，对于如何迈出第一步，波利亚和其他人已经提出了很多通用的方法，其中一种是反向解决问题。例如，如果问题是找到一种从一个给定输入产生一个特定输出的方法，我们可以从输出开始，尝试反向推导出给定输入。这种方法就是本节前面介绍的人们发现折纸鸟算法的基本思路。他们往往是拆开一只折好的纸鸟，看看它是怎么折的。

另一种通用的解决问题的方法是寻找一个更容易解决或者以前已经得到解决的相关问题，然后尝试把这个问题的解决方案用到当前问题中。这种技术在程序开发中特别有价值。通常，程序开发的过程并不是解决一个问题中的一个特定实例，而是要寻找一种适用于求解一个问题的所有实例的通用算法。更准确地讲，如果我们面对的任务是开发一个把名字列表按照字母顺序排序的程序，那么我们的任务就不是对一个特定列表进行排序，而是找出一个可以用来对任何名字列表进行排序的通用算法。因此，尽管指令

交换名字David和Alice。 将名字Carol移到Alice和David之间。 将名字Bob移到Alice和Carol之间。
--

可以把由David、Alice、Carol和Bob组成的列表正确排序，但这并不是我们期望的通用算法。我们需要的算法应当既可以对这个列表进行排序，又可以对我们可能遇到的其他列表进行排序。这并不是说对特定列表进行排序的解决方案对我们研究通用算法完全没有意义。例如，我们可能会通过考虑这种特殊情况来迈出第一步，寻求能够反过来用于开发所需通用算法的一般原则。于是，在这种情况下，我们可以通过解决一系列相关问题的技术来得到我们的解决方案。

另一种迈出第一步的方法是**逐步精化**（stepwise refinement），这种方法本质上不是试图立即解决整个任务（所有细节）的技术。相反，逐步精化建议，首先把一个手头的问题看作多个子问题。思路是，将原问题分解为多个子问题，通过逐步解决各个子问题来得出原问题的整体解决方案，其中每一步都要比解决完整的原问题更容易。根据逐步精化的建议，还可以把这些步骤分解成更小的步骤，这些更小的步骤还可以继续分解，就这样一直分解，直到整个问题被简化为一组容易解决的子问题为止。

就此而论，逐步精化是一种**自顶向下方法**（top-down methodology），这种方法从一般发展到特殊。相反，**自底向上方法**（bottom-up methodology）是从特殊发展到一般。尽管理论上相反，但是实际上这两种方法经常在实际应用中互为补充。对逐步精化的自顶向下方法提出的问题的分解，通常要以那些可能以自底向上方式工作的问题求解者的直觉为指导。

逐步精化的自顶向下方法从本质上讲是一种组织工具，这种工具的问题求解属性是这个组织方式的结果。逐步精化早已成为计算界的一个重要的设计方法，其中大型软件系统的开发都拥有一个重要的组织组件。但就像我们将要在第7章中学习的，大型软件系统正越来越多地通过组合预制组件的方式来构造（本质上是一种自底向上的方法）。因此，自顶向下方法和自底向上方法仍然是计算机科学中的重要工具。

维持这种宽泛的观点的重要性是通过以下事实例证的：将事先形成的观念和预选的工具带入问题求解任务有时会掩盖问题的简单性。本节前面讨论的求解3个孩子年龄的问题就是这种现象的一个很好的例子。学代数的学生会不约而同地用联立方程来求解这个问题，这种方法可能会将问题带入死胡同，而且时常会使想要成为问题求解者的人陷入相信所给信息不足以解决这个问题的圈套。

下面是另外一个例子：

当你从码头走上船的时候，你的帽子掉进了水里，但是你不知道。河水的流速是4公里每小时，所以你的帽子开始向下游漂去。同时，你开始在船上以相对于水流7.5公里每小时的速度向上游前进。10分钟后，你发现帽子不见了，然后调转船头，开始顺流而下追你的帽子。需要多长时间能赶上你的帽子？

大多数学习代数的学生还有那些热衷使用计算器的人在解决这个问题的时候，都会先确定船在10分钟后向上游行进了多远以及在这个

时间内帽子向下游漂了多远。然后，他们确定船向下游行驶到那个位置需要多长时间。但是，当船到达这个位置的时候，帽子又向下游漂了一段距离。因此，问题求解者要么用微积分技术重新解题，要么陷入一个计算循环——每次船到达帽子的上一个位置时都要计算帽子所处的位置。

然而，这个问题的求解实际要比这简单得多。诀窍是抵制这种一开始就列公式进行计算的冲动。其实，我们需要先将这些技巧放在一边，调整我们的思维方式。整个问题发生在河中。事实是水相对于河岸的流动与解题是不相关的。想象一个相同的问题发生在传送带上而非水中。首先，在传送带停止的情况下解决问题。如果你站到传送带上，然后把帽子放在脚边，之后反向行走10分钟，那么返回到帽子所在处需10分钟。现在启动传送带，这意味着旁边的场景将相对于传送带反向移动，但是，因为你站在传送带上，所以这不会改变你和传送带或者帽子之间的相对关系，因此你返回到帽子所在处仍然是需要10分钟。

我们得出结论：算法的发现仍旧是一种富有挑战的艺术，必须经过一段时间才能完成，而不能像一门由明确定义的方法组成的学科那样学到。因此，要训练潜在的问题求解者，使之遵循一定的方法，就要压制那些原本应该培养的创造性技能。

5.3节问题与练习

1. a. 找出求解下列问题的算法：已知一个正整数 n ，找出一个正整数列表，该列表中所有正整数的乘积是其正整数和为 n 的所有正整数列表中最大的。例如，如果 n 为4，则所求的列表由两个2组成，因为 2×2 大于 $1 \times 1 \times 1 \times 1$ 、 $1 \times 1 \times 2$ 和 1×3 。如果 n 为5，则所求的列表由2和3组成。

b. 如果 $n=2001$ ，那么所求的列表由哪些正整数组成？

c. 说明你是如何迈出第一步的。

2. a. 假设已知方格棋盘是由 2^n 行 2^n 列的正方形组成的，给定正整数 n ，以及一盒L型棋子，每个都恰好可以覆盖棋盘的3个正方形格子。如果从棋盘上切掉任意一个格子，我们是否还可以用这些棋子在既不互相重叠又不跨越棋盘边缘的条件下把剩余的棋盘填满？

b. 说明怎样用问题a的解来证明：对于所有的正整数 n ， $2^{2n}-1$ 是可以被3除尽的。

c. 说明问题a和问题b与波利亚的问题求解阶段的关联性。

3. 解码下面的消息，然后说明你是如何迈出第一步的。

Pdeo eo pda yknnayp wjosan.

4. 如果你打算解决拼图游戏问题，即把图片抛撒在桌面上然后试着将它们拼在一起，你会采用自顶向下方法吗？如果是看着拼图盒上的完整图案来拼，你的答案会改变吗？

5.4 迭代结构

我们现在要学习一些在描述算法过程中使用的重复结构。在本节中，我们讨论**迭代结构**（iterative structure），在这种结构中，一组指令以循环方式重复执行。在下一节中，我们将介绍递归技术。我们将附带介绍一些流行的算法：顺序搜索算法、二分搜索算法和插入排序算法。我们先介绍一下顺序搜索算法。

5.4.1 顺序搜索算法

考虑一下某个特定目标值是否存在于一个列表中的搜索问题。我们希望开发一个算法来确定这个值是否在列表中。如果它在列表中，我们认为搜索成功；否则认为搜索失败。我们假设该列表已依照某种规则对列表条目做了排序。例如，如果这是一个名字列表，我们就假设列表中的名字是按照字母顺序排列的；如果这是一个数字列表，我们就假设它里面的条目是按照从小到大排列的。

为了迈出第一步，我们想象如何在一个大概有20条记录的来宾列表中寻找一个特定的名字。在这种情况下，我们可以从头开始扫描整个列表，将每个条目与目标名字进行比较。如果找到了目标名字，则搜索以成功告终。但是，如果搜索到列表的末尾也没有找到目标值，则搜索以失败告终。实际上，如果到达（按字母顺序）大于目标名字的名字但还没有找到目标名字，我们的搜索就以失败告终。（记住，列表是按照字母顺序排列的，所以到达一个大于目标名字的名字就意味着目标名字没有出现在列表中。）概括来说，我们大致的想法就

是，只要还有名字没检查而且目标名字大于正在考虑的名字，就继续搜索。

在我们的伪代码中，这个过程可以表达为：

```
选择List中的第一个条目作为TestEntry
while (TargetValue大于TestEntry且还有条目):
    选择List中的下一个条目作为TestEntry
```

一旦这个while结构终止，以下两个条件中就有一个为真：找到目标值或者目标值不在列表中。无论哪种情况，我们都可以通过比较测试条目（TestEntry）和目标值（TargetValue）来检测搜索是否成功。如果这两个值相等，则搜索成功。因此，我们把下列语句添加到上述伪代码例程的下面：

```
if (TargetValue == TestEntry):
    声明搜索成功
else:
    声明搜索失败
```

最后，我们观察这个例程的第一条语句，这条语句把列表中的第一个条目选作测试条目，这种选择的前提是假设表中至少有一个条目。我们可能会认为这是一种安全的猜测，但是为了保险起见，我们将前面的例程作为下面语句中的else部分：

```
if (List为空):
    声明搜索失败
else:
    . . .
```

由此产生的函数的伪代码如图5-6所示。注意，可以在其他函数内部使用这个函数，如利用语句

用函数Search()和目标值Darrel Baker搜索乘客列表

来查明Darrel Baker是否是一名乘客，用语句

用函数Search()和目标值nutmeg搜索原料列表

来查明nutmeg（肉豆蔻）是否在原料列表中。

```
def Search(List, TargetValue):  
    if (List为空):  
        声明搜索失败  
    else:  
        选择List中的第一个条目作为TestEntry  
        while (TargetValue大于TestEntry and 还有条目):  
            选择List中的下一个条目作为TestEntry  
        if (TargetValue = TestEntry):  
            声明搜索成功  
        else:  
            声明搜索失败
```

图5-6 用伪代码表示的顺序搜索算法

概括来讲，图5-6所示的算法是按照条目在列表中出现的顺序进行查找的。由于这个原因，这个算法称作**顺序搜索**（sequential search）算法，有时称作**线性搜索**（linear search）。因为这种算法简单，所以经常用于短列表或者出于其他考虑需要使用它的场合。然而，对于长列表，顺序搜索就没有其他技术高效了（我们很快就会看到）。

5.4.2 循环控制

一条指令或者一系列指令的重复使用是一个重要的算法概念。实现这种重复的一种方法是称为**循环**（loop）的迭代结构，这种结构中，一组称为循环**体**（body）的指令在某个控制过程的指引下重复执行。一个典型的例子就是图5-6所示的顺序搜索算法。这里我们使用while语句来控制下面这条语句的重复：

```
选择List中的下一个条目作为TestEntry
```

实际上，while语句

```
while (条件):  
    循环体
```

就是循环结构概念的一个例证，因为它的执行是跟踪循环模式

```
检查条件  
执行循环体  
检查条件  
执行循环体  
⋮  
检查条件
```

直到条件为假。

作为一般规则，循环结构的使用使程序得到了比仅仅将循环体重写多次更高的灵活度。例如，执行3次下面的语句：

```
加一滴硫酸
```

我们可以写：

```
加一滴硫酸  
加一滴硫酸  
加一滴硫酸
```

但是，我们写不出与下面的循环结构等价的相似序列：

```
while (pH值大于4):  
    加一滴硫酸
```

因为我们事先不知道需要加入多少滴硫酸才合适。

现在让我们仔细看看循环控制的组成。你可能认为循环结构的这一部分并不重要，毕竟，通常都是循环体在实际执行手头的任务（如加几滴硫酸）——因为我们选择了以重复的方式执行循环体，所以控制活动仅表现为相关的开销。但是，经验表明，循环控制是循环结构中一个非常容易出错的部分，所以很值得我们注意。

循环控制由初始化、测试和修改3个活动（见图5-7）组成，其中每个活动都决定了循环控制是否能够成功。测试活动有义务通过观察指示终止应该发生的条件来终止循环过程。这个条件就是**终止条件**（termination condition）。为了这个测试活动，我们在我们伪代码的每个while语句中都提供了一个条件。然而，在while语句中，规定的条件是循环体应该被执行的条件——终止条件就是while结构中出现的条件的对立条件。因此，在下面的语句中，终止条件是“pH值**不**大于4”：

```
while (pH值大于4):  
    加一滴硫酸
```

在图5-6所示的while语句中，终止条件可规定为：

(TargetValue <= TestEntry) or (不再有要检查的条目)

初始化：建立一个初始状态，这一状态会朝着终止条件修改
测试：比较当前状态和终止条件，如果相等则终止重复
修改：改变状态使之移向终止条件

图5-7 重复控制的组成

循环控制中的另外两个活动确保了终止条件最终可以出现。初始化步骤建立了一个初始条件，修改步骤将这个条件移向终止条件。例如，在图5-6中，初始化发生在while语句之前的语句中，该处当前的测试条目被设定为列表的第一个条目。在这个例子中，修改步骤实际上是在循环体内完成的，在循环体中，我们关注的位置（由测试条目标识）被移向了列表的末尾。因此，在执行完初始化步骤后，修改步骤的重复应用导致程序可以到达终止条件。（或者到达一个大于或等于目标值的测试条目，或者最终到达列表的末尾。）

我们应该强调的是，初始化步骤和修改步骤必须导致合适的终止条件。这个特性对正确的循环控制至关重要，因此在设计循环结构的时候必须多次确认它是否存在。如果没有进行这样的评估，即使在最简单的情况下也会导致错误。一个典型的例子就是

```
Number = 1
while (Number != 6):
    Number = Number + 2
```

注意Python运算符“!=”的使用，它读作“不等于”。这里，终止条件是“Number == 6”。但是Number的值被初始化为1，然后在修改步骤递增2。因此，在循环过程中，Number的值将是1、3、5、7、9等，但是永远不会是6，因此，循环将无法终止。

循环控制部件的执行顺序可能会产生微妙的结果。事实上，有两种常用的循环结构，它们仅仅在循环控制部件的执行顺序上有所区别。第一种例子如下面的伪代码语句所示：

```
while (条件):  
    活动
```

图5-8给出了这个结构的语义的流程图（flowchart）。（流程图用各种形状来表示单个步骤，用箭头表示步骤的顺序。线框形状之间的不同表示相关步骤涉及的动作类型不同，例如菱形表示判断，矩形表示任意语句或语句序列。）注意，while结构的终止测试出现在循环体执行之前。

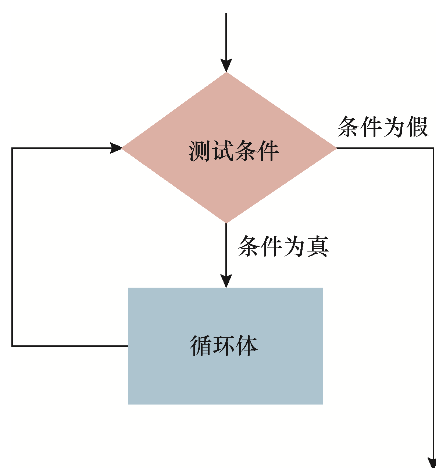


图5-8 while循环结构

相反，图5-9中所示的结构要求循环体在终止测试完成之前执行。在这种情况下，循环体总是至少执行一次，然而在while结构中，如果终止条件在第一次测试时就满足，则循环体一次都不用执行。

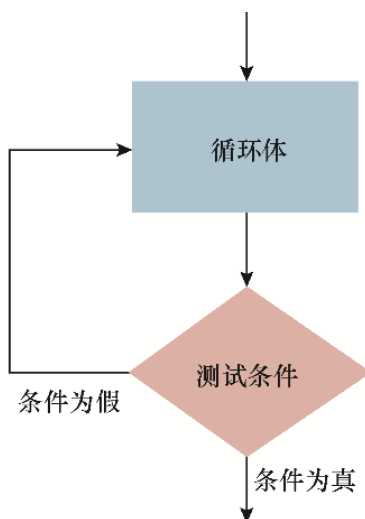


图5-9 repeat循环结构

Python没有这第二种循环的内置结构，但使用现有的while结构，再加一个if语句和循环体末尾的一个break，就能很容易地构建一个等效的结构。对于我们的伪代码，我们将借用几种其他语言中已有的关键字，使用语法形式

```
repeat:  
    活动  
until (条件)
```

来表示图5-9所示的结构。因此，语句

```
repeat:  
    从你的口袋里取出一枚硬币  
until (你的口袋里没有硬币)
```

假设开始时你的口袋里至少有一枚硬币，但是语句

```
while (你的口袋里有硬币):  
    从你的口袋里取出一枚硬币
```

就没有做这个假设。

依照我们伪代码的术语，我们通常会把这些循环称作while循环结构或者repeat循环结构。在更一般的情况下，while循环结构可能会被称作**前测试循环**（pretest loop）（因为终止测试是在循环体执行之前进行的），而repeat循环结构会被称作**后测试循环**（posttest loop）（因为终止测试是在循环体执行之后进行的）。

虽然许多算法要求在控制循环时慎重考虑初始化、测试和修改活动，但其他一些算法则遵从一些非常常见的模式。特别是在处理数据列表时，最常见的模式是从列表中的第一个元素开始，考虑列表中的每个元素，直到到达列表末尾。暂时回到我们的顺序搜索示例上，我们看到这个模式类似于：

```
选择List中的第一个条目  
while (还有条目):  
    . . .  
    选择List中的下一个条目
```

因为这个结构在算法中出现得如此频繁，以致我们将使用下面的语法形式来描述一个迭代遍历列表中每个元素的循环：

```
for Item in List:  
    . . .
```

注意，这个伪代码原语实际上是比较while结构更高级的抽象，因为我们可以用单独的初始化、修改和测试结构来实现同样的效果，但是这个版本去除了不必要的细节，更简洁地传达了这个循环的意思。

每次通过这个for循环结构体时，值Item都会变为List中的下一个元素。这个循环的终止条件在到达List末尾的时候是隐含的。作为一个例子，要想得到列表中数字的总和，可以使用：

```
Sum = 0
for Number in List:
    Sum = Sum + Number
```

在Python以外的语言中，这种类型的循环通常称作for-each循环，是前测试循环的一种特例。for结构最适合的情况是，算法对列表中的每个元素都执行同样的步骤，不必单独跟踪循环计数变量。

5.4.3 插入排序算法

作为另一个使用迭代结构的例子，下面考虑将一个名字列表按照字母顺序进行排序的问题。但是在继续介绍之前，我们应该先了解一下排序的限制。简单来讲，我们的目标是在“列表自身的内部”对列表进行排序。换句话说，我们想通过把条目移来移去来对列表进行排序，而不是把列表移到其他位置。我们的情况类似于下面这种列表的排序问题：列表的每个条目记录在一张索引卡片上，所有卡片分散在桌面上，把桌面挤得满满的；我们已经清理出足够的空间来放这些卡

片，但是不允许挪开其他东西来挤出更多的空间。这种限制在计算机应用里是很典型的，其原因当然不是机器里的工作空间一定像桌面那样拥挤，而仅仅是因为我们想更有效地利用存储空间。

下面我们来迈出第一步，考虑如何在桌面上对名字进行排序。考虑一个名字列表：

Fred

Alex

Diana

Byron

Carol

对这个列表进行排序的一种方法是，注意由最前面的一个名字Fred组成的子表已排序，而由最前面的两个名字Fred和Alex组成的子表还没有排序。因此，我们应该捡起含有名字Alex的卡片，将名字Fred向下滑动放入Alex原来的位置，然后把名字Alex放入Fred原来的位置，如图5-10中第一行所示。此时，名字列表就变成了：

Alex

Fred

Diana

Byron

Carol

现在，前两个名字构成了一个已排序的子表，但是前三个名字还没构成。因此，我们应该捡起第三个名字Diana，将名字Fred向下滑动放入Diana原来的位置，然后将Diana插到Fred 原来的位置，如图5-10中第二行所示。现在列表的前三个条目都已排序。继续以这种方式进行操作，通过捡起第四个名字Byron，将名字Fred和Diana向下滑动，然后将Byron插入空缺处，就能够获得前四个条目已排序的列表了（见图5-10中的第三行）。最后，我们可以通过捡起Carol，把Fred和Diana向下滑动，然后把Carol插到剩下的空缺来完成排序过程（见图5-10中的第四行）。

在分析了一个特定列表的排序过程之后，现在的任务是将这个过程一般化，得到一个对一般列表进行排序的算法。为此，我们观察图5-10，发现其每一行都代表着一个同样的一般过程：捡起列表中还未排序的部分中的第一个名字，将已排序的部分中大于此名字的条目向下滑动，然后将这个取出的名字插入这一部分的空缺处。如果把取出的名字称为主元，那么这个过程可以用下面的伪代码表示：

```
把主元移到一个临时位置使该List留出一个空缺
while (在空缺位置的上面有名字 and 这个名字比主元大):
    将空缺上面的名字下移到空缺上使该名字上面留出一个空缺
将主元移到List中的空缺上
```

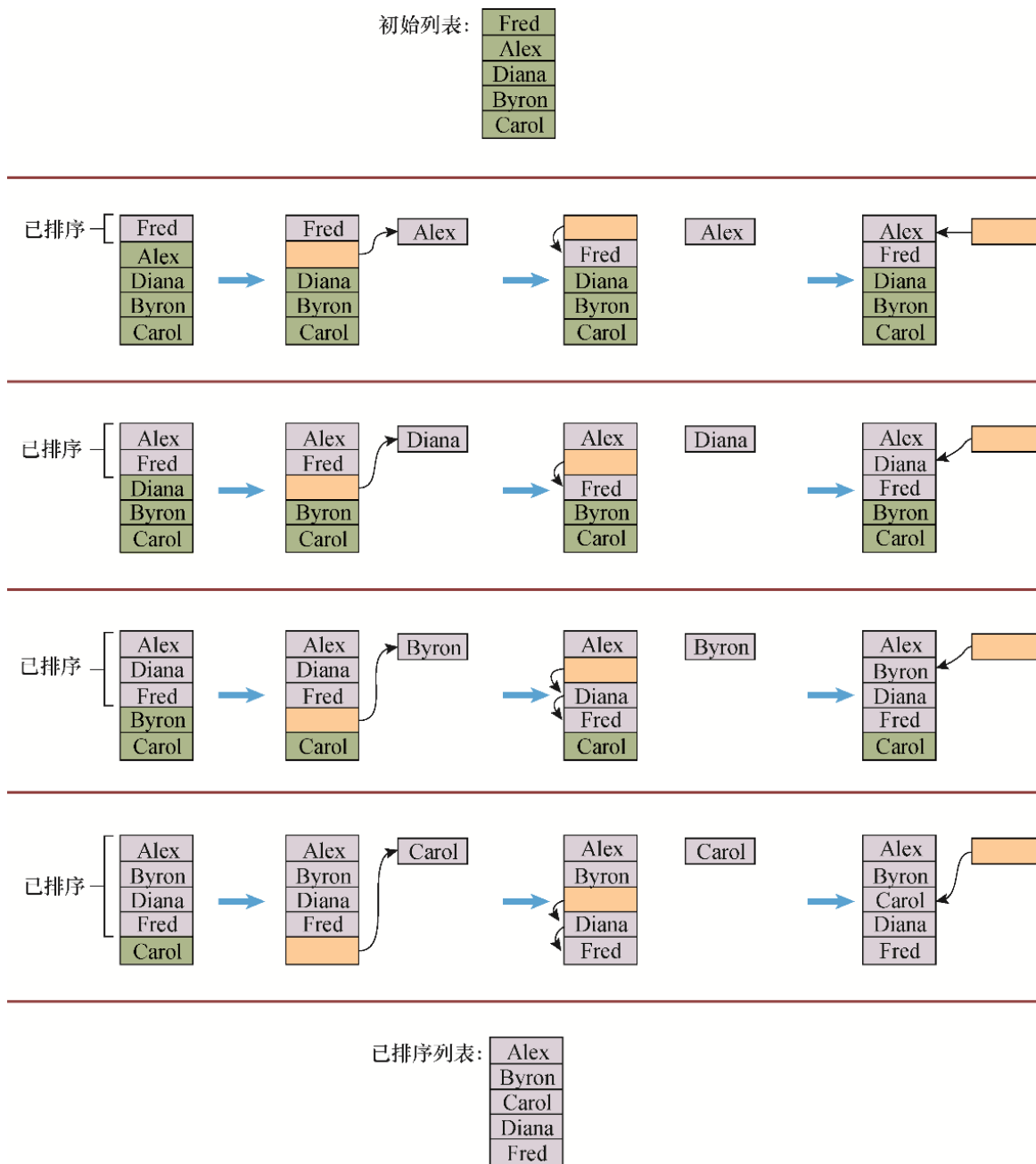


图5-10 按字母顺序对由Fred、Alex、Diana、Byron和Carol组成的列表进行排序

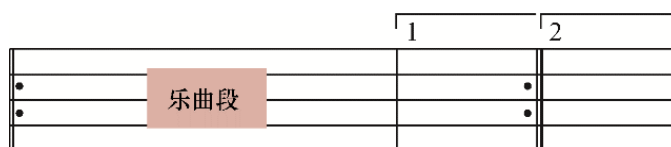
音乐中的迭代结构

音乐家比计算机科学家早几个世纪就已经开始使用和编写迭代结构了。实际上，一首歌曲的结构（谱写成多个小节，每个小节配以和

声) 可以用while语句示范：

```
while (有剩余小节):  
    唱下一小节  
    唱和声
```

此外，乐谱



只不过是谱曲者表达下面结构的方式：

```
N = 1  
while (N < 3):  
    演奏该乐曲段  
    演奏第N段尾曲  
    N = N + 1
```

接下来，我们来观察这个过程应该怎样重复执行。为了开始这个排序过程，主元应该是列表的第二个条目，然后，每当再次执行前，都应该将沿着列表向下的下一个条目选作主元，直到列表的最后一个条目已被定位。也就是说，随着前面例程的重复，主元的位置从第二个条目移进到第三个条目，然后到第四个条目，依次类推，直到例程定位到列表的最后一个条目。根据这个思路，我们可以使用下面的语句来控制这个所需的重复：

```
N = 2  
while (N的值没有超过List的长度):  
    选择List中的第N个条目作为主元  
    |  
    N = N + 1
```

其中N表示主元的位置，List的长度是指列表中的条目个数，而省略号则是指放置前面例程的位置。

完整的伪代码程序如图5-11所示。简而言之，这个程序是通过重复地移动条目并将其插入适当的位置来对列表进行排序的。因为这种插入过程是重复的，所以这种底层算法称为**插入排序**（insertion sort）。

```
def Sort(List):  
    N = 2  
    while (N的值没有超过List的长度):  
        选择List中第N个条目作为主元  
        把主元移到一个临时位置使该List留出一个空缺  
        while (在空缺位置的上面有名字and这个名字比主元大):  
            将空缺上面的名字下移到空缺上使该名字上面留出一个空缺  
            将主元移到List中的空缺上  
        N = N + 1
```

图5-11 用伪代码表达的插入排序算法

注意，图5-11中所示的结构是一个循环内部还有循环的结构，外层循环用第一个while语句表示，而内层循环用第二个while语句表述。每次执行外层循环体都导致内层循环体被初始化而且重复执行直到达到终止条件，因此外层循环体的一次执行将导致内层循环体多次执行。

外层循环控制的初始化部分由建立N的初始值的语句组成：

```
N = 2
```

修改部分通过下列语句每次给循环体末尾的N加1完成：


```
N = N + 1
```

当N的值超过列表的长度时，终止条件出现。

内层循环控制通过移走列表中的主元从而形成一个空缺位置来初始化。循环的修改步骤通过将条目向下移入空缺从而导致空缺位置上移来实现。终止条件有两个，要么空缺位置上方紧邻的名字不大于主元，要么空缺位置到达列表顶部。

5.4节问题与练习

1. 修改图5-6中的顺序搜索函数，使其可用于未排序的列表。
2. 把下面的伪代码例程转换为用repeat语句表述的等价例程：

```
Z = 0
X = 1
while (X < 6):
    Z = Z + X
    X = X + 1
```

3. 当今一些流行的程序设计语言语法

```
While (. . .) do (. . .)
```

来表示前测试循环，用语法

```
do (. . .) while (. . .)
```

来表示后测试循环。尽管设计上显得很优雅，但是这种相似的形式会导致什么问题？

4. 假设用图5-11所示的插入排序算法对由Gene、Cheryl、Alice和Brenda组成的列表进行排序。描述外层while结构的循环体每次执行结束时列表的组织结构。

5. 为什么不能把图5-11中while语句内的“大于”改为“大于等于”？

6. 插入排序算法的一个变体是**选择排序**（selection sort）。这一排序算法从选择列表中最小的条目并将其移到列表的最前面开始，然后选择列表中剩余的条目中最小的条目，并将其放到列表的第二个位置。通过重复从列表的剩余部分中选择并前移最小的条目，列表前面已排序部分逐渐变长，而后面未排序部分逐渐缩短。请使用我们的伪代码和选择排序算法来表达一个与图5-11中的列表排序函数类似的函数。

7. 另一个著名的排序算法是**冒泡排序**（bubble sort）。这一算法是重复对列表中相邻的两个条目进行比较，如果它们彼此之间的顺序不正确，就交换它们的位置。我们假设待排序的列表有 n 个条目。冒泡排序将从比较（并可能交换）位置 n 和 $n-1$ 上的条目开始，然后考虑位置 $n-1$ 和 $n-2$ 上的条目，并且继续向列表的前面移动，直到比较（并可能交换）完列表的第一个条目和第二个条目。可以看出，列表的这一遍操作能把最小条目放到列表的最前面。同样，再来一遍这样的操作能把仅大于最小的条目的条目放到列表的第二个位置。因此，重复 $n-1$ 遍这样的操作，就能完成整个列表的排序。（如果我们观察算法的工作过程，就会看到小条目像气泡一样冒到了列表的前面——这个算法

因此得名。) 请使用我们的伪代码和冒泡排序算法来表达一个与图5-11中的列表排序函数类似的函数。

基本知识点

- 可以开发不同的算法来解决同一个问题。

5.5 递归结构

递归结构为实现重复活动提供了除循环模型以外的另一种选择。循环涉及重复一个指令集，其方式是执行完成一组指令，然后重复执行，而递归则涉及将指令集作为自身的一个子任务来重复一个指令集。打个比方，考虑在有多个标签的网络浏览器中进行在线购物的过程。当用户在一个页面上研究产品时，可能会发现一个链接，这个链接指向一个或多个吸引用户在新的浏览器标签中打开链接的相关商品。那时，用户会把不完整的购物交易放在一边，去读另一个或另一系列页面。探索完离题商品后，用户关闭新标签返回原来的页面完成交易。结果是，有两个或更多个网页正在被浏览。但是，这些页面不是像线性循环结构那样被一个接一个地浏览，而是在一个中浏览另一个。当我们完成内部任务（可能有一个，也可能有多个）时，我们返回来继续外部任务。

5.5.1 二分搜索算法

作为介绍递归的一种方式，让我们再次处理在一个已排序的列表中搜索是否存在某特定条目的问题，但这次将通过考虑查字典时所遵循的步骤来迈出第一步。在这种情况下，我们不再按一条条的顺序甚至一页页的方式进行，而是开始就直接把字典翻到我们认为目标条目可能存在的那个区域中的一页。如果幸运的话，我们将在那里找到目标值；否则，就必须继续查找。但此时我们已经大大缩小了查找的范围。

当然，在查字典的时候，我们有“单词可能在哪里”的先验知识。例如，如果我们正在查找“somnambulism”这个单词，我们就会从字典的后面部分开始查找。但是对于一般列表，我们并没有这种先验知识，所以我们总是假定从列表的“中间”条目开始查找。这里“中间”这个词之所以用引号引起来，是因为一个列表可能包含偶数个条目，这种列表就没有准确意义上的中间条目。在这种情况下，我们将假定该“中间”条目为该列表后半部分的第一个条目。

如果列表的中间条目就是目标值，我们就可以声明搜索成功；否则，我们至少可以将搜索过程限定在列表的前半部分或者后半部分，具体取决于目标值是小于还是大于我们已考虑的中间条目。（记住，列表是已排序的。）

要搜索列表的剩余部分，我们本可以应用顺序搜索，但这里我们仍然应用搜索完整列表所用的方法。也就是说，我们选择列表剩余部分的中间条目作为下一个要考虑的条目。和前面一样，如果这个条目

就是目标值，搜索就完成了；否则我们可以把搜索限定在更小的列表部分中。

图5-12总结了 this 搜索方法的过程，这里我们考虑的任务是搜索图左边列表中的John条目。首先考虑中间条目Harry。因为我们的目标在此条目后面的列表中，所以接下来的搜索将考虑原始列表的下半部分。找到该子表的中间条目是Larry。因为我们的目标应该在Larry之前，所以我们将注意力转移到当前子表的前半部分。当我们询问第二个子表的中间条目时，我们找到了目标条目John，于是声明搜索成功。简而言之，我们的策略就是，将所讨论的列表连续分成更小的段，直到最终找到目标或者搜索范围缩小为空段为止。

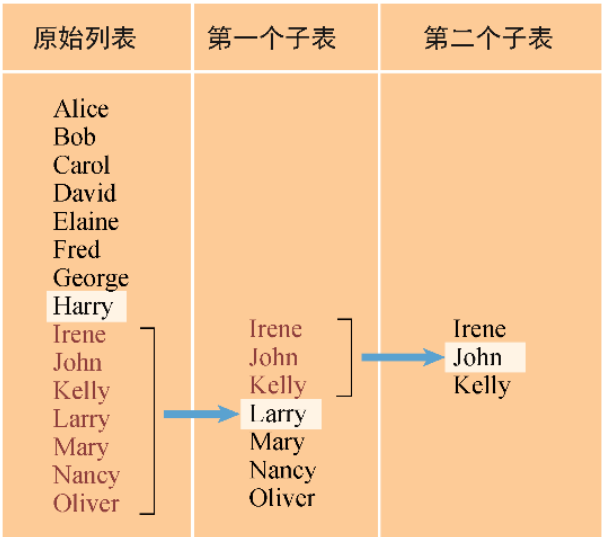


图5-12 应用我们的策略在列表中搜索条目John

这里需要强调最后一点。如果目标值不在原始列表中，那么我们的搜索方法会将列表不断地分成更小的段直到所考虑的段为空为止，此时，算法应该认为搜索失败。

图5-13展示的是我们这个想法的第一个伪代码草稿。这个草稿引导我们通过测试查看列表是否为空来开始搜索过程。如果为空，我们会被告知搜索失败；否则，我们会被告知考虑列表的中间条目。如果此条目不是目标值，我们就会被告知搜索表的前半部分或者后半部分。这两种可能都需要后继搜索。通过调用一个抽象工具的服务来执行这些搜索会很好。尤其是，我们的方法是应用一个名为Search的函数来执行这些后继搜索。因此，要完成我们的程序，必须提供这样一个函数。

```
if (List为空):  
    报告搜索失败  
else:  
    TestEntry = List的“中间”条目  
    if (TargetValue == TestEntry):  
        报告搜索成功  
    if (TargetValue < TestEntry):  
        用函数 Search() 在TestEntry前面的List部分里搜索  
        TargetValue, 并报告搜索结果  
    if (TargetValue > TestEntry):  
        用函数Search() 在TestEntry后面的List部分里搜索  
        TargetValue, 并报告搜索结果
```

图5-13 二分搜索技术的第一个草稿

但是这个函数应该执行的任务就是我们前面写的伪代码所表达的任务。它应该先检查给定列表是否为空，如果不为空，它应该继续考虑此表的中间条目。因此，我们可以提供这样一个函数：我们只需把当前例程视为名为Search的函数，并在需要后继搜索的地方插入对这个函数的引用。结果如图5-14所示。

```

def Search(List, TargetValue):
    if (List为空):
        报告搜索失败
    else:
        TestEntry = List的“中间”条目
        if (TargetValue == TestEntry):
            报告搜索成功
        if (TargetValue < TestEntry):
            Sublist = TestEntry前面的List部分
            Search(Sublist, TargetValue)
        if (TargetValue > TestEntry):
            Sublist = TestEntry后面的List部分
            Search(Sublist, TargetValue)

```

图5-14 二分搜索算法的伪代码

注意，这个函数包含了一个对自身的引用。如果顺着这个函数到达指令

```
Search(. . .)
```

我们将把用在原始列表上的那个函数用到较小的列表上。如果那次搜索成功，我们将返回并声明我们的原始搜索成功；如果这次后继搜索失败，我们将声明我们的原始搜索失败。

搜索和排序

顺序搜索算法和二分搜索算法仅是许多实现搜索过程的算法中的两个。同样，插入排序仅是许多排序算法中的一个。其他经典的排序算法包括归并排序（见第12章）、选择排序（见5.4节问题与练习6）、冒泡排序（见5.4节问题与练习7）、快速排序（对排序过程采取分治的方法）和堆排序（它使用一种巧妙的技术来查找列表里应该向

前移动的条目)。有关这些算法的讨论可以参阅本章末尾的“课外阅读”里面列出的书目。

为了看一下图5-14中的函数是如何执行其任务的，我们让它搜索一个包括Alice、Bill、Carol、David、Evelyn、Fred和George的列表，查找目标值Bill，看看它的执行过程。首先选择David（中间条目）作为考虑的测试条目。因为目标值（Bill）小于该测试条目，于是，我们对David前面的列表条目（即Alice、Bill和Carol）应用Search()函数。这样，我们便创建了搜索函数的第二个副本，并将它分配给这个后继任务。

现在我们拥有两个正在执行的搜索函数的副本，如图5-15所示。原始副本的进展将暂时停止在下面的指令处：

Search(Sublist, TargetValue)

同时我们把这第二个副本用于搜索Alice、Bill和Carol组成的列表的任务。在这个后继搜索后，我们将放弃该函数的第二个副本，将其结果报告给原始副本，然后继续原始副本的进展。以这种方式，函数的第二个副本被当作原始函数的从属函数执行，完成原始模块请求的任务，然后消失。

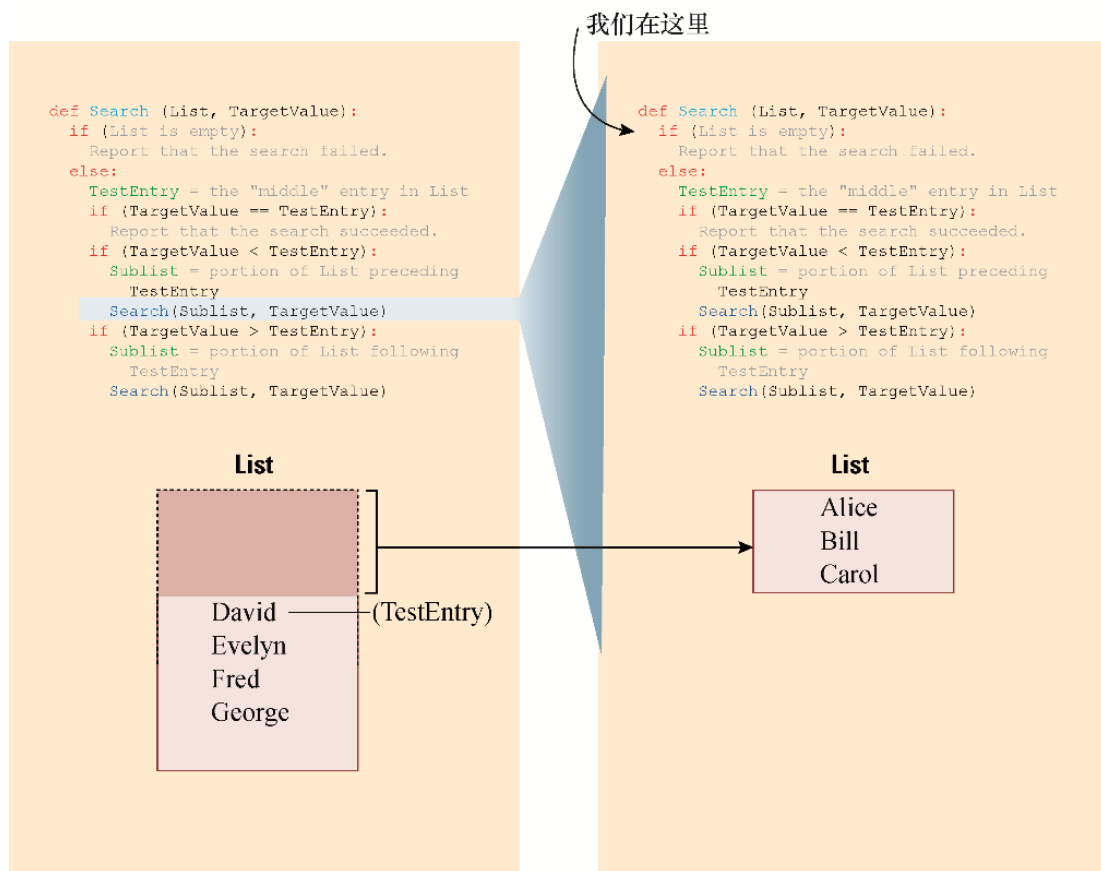


图5-15 递归搜索

这个后继搜索选择Bill作为它的测试条目，这是因为它是Alice、Bill和Carol组成的列表的中间条目。因为这个条目与目标值相同，所以函数声明其搜索成功并终止。

此时，我们已经完成了函数的原始副本所请求的后继搜索，所以可以继续原始副本的执行。这里，我们又被告知应该把后继搜索的结果作为原始搜索的结果报告。因此，我们报告原始搜索已经成功。我们的搜索已正确确定，Bill是Alice、Bill、Carol、David、Evelyn、Fred和George组成的列表的成员。

现在让我们考虑一下，如果用图5-14所示的函数在Alice、Carol、Evelyn、Fred和George组成的列表中搜索条目David，会出现什么情况。这次函数的原始副本选择Evelyn作为测试条目，并推断目标值肯定在列表的前半部分。因此，它请求函数的另外一个副本来搜索Evelyn之前的条目，即包含Alice和Carol的二条目列表。在这个阶段，我们遇到了图5-16所示的情况。

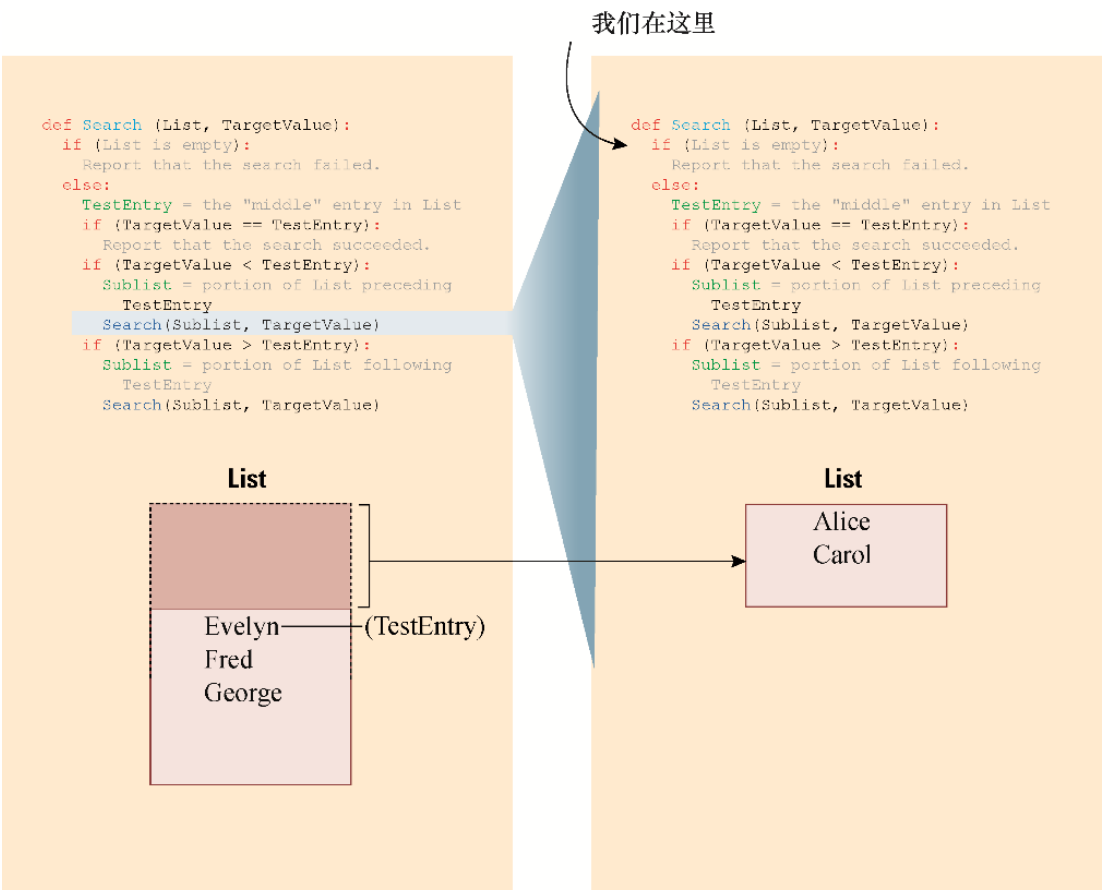


图5-16 第二次递归搜索，第一个快照

这个函数的第二个副本选择Carol当作当前条目，并推断目标值必定存在于该列表的后半部分。然后，它请求函数的第三个副本来搜索Alice和Carol组成的列表中Carol后面的名字组成的列表。该子表为

空，所以函数的第三个副本需要在一个空列表中搜索目标值David。图5-17显示了目前所处的情况。函数的原始副本负责处理Alice、Carol、Evelyn、Fred和George组成的列表的搜索，测试条目为Evelyn；第二个副本负责处理Alice和Carol组成的列表的搜索，测试条目为Carol；第三个副本要在一个空列表中开始搜索。

当然，函数的第三个副本很快就会声明它的搜索失败并终止。第三个副本的任务的完成使得第二个副本能够继续它的任务。第二个副本注意到它请求的搜索未成功，于是声明自己的搜索失败并终止。这个报告是原始副本一直在等待的，所以它现在可以继续执行了。因为它所请求的搜索失败，所以它声明自己的搜索已失败并终止。我们的例程已正确推断出David不在Alice、Carol、Evelyn、Fred和George组成的列表中。

总之，如果回顾前面的例子，我们就可以看到，图5-14所示的算法所采用的过程是，重复地将所考虑的列表分成两个较小的块，以这种方式将后继搜索严格地限制在其中一个块中。这种一分为二的方法就是该算法称为**二分搜索**（binary search）的原因。

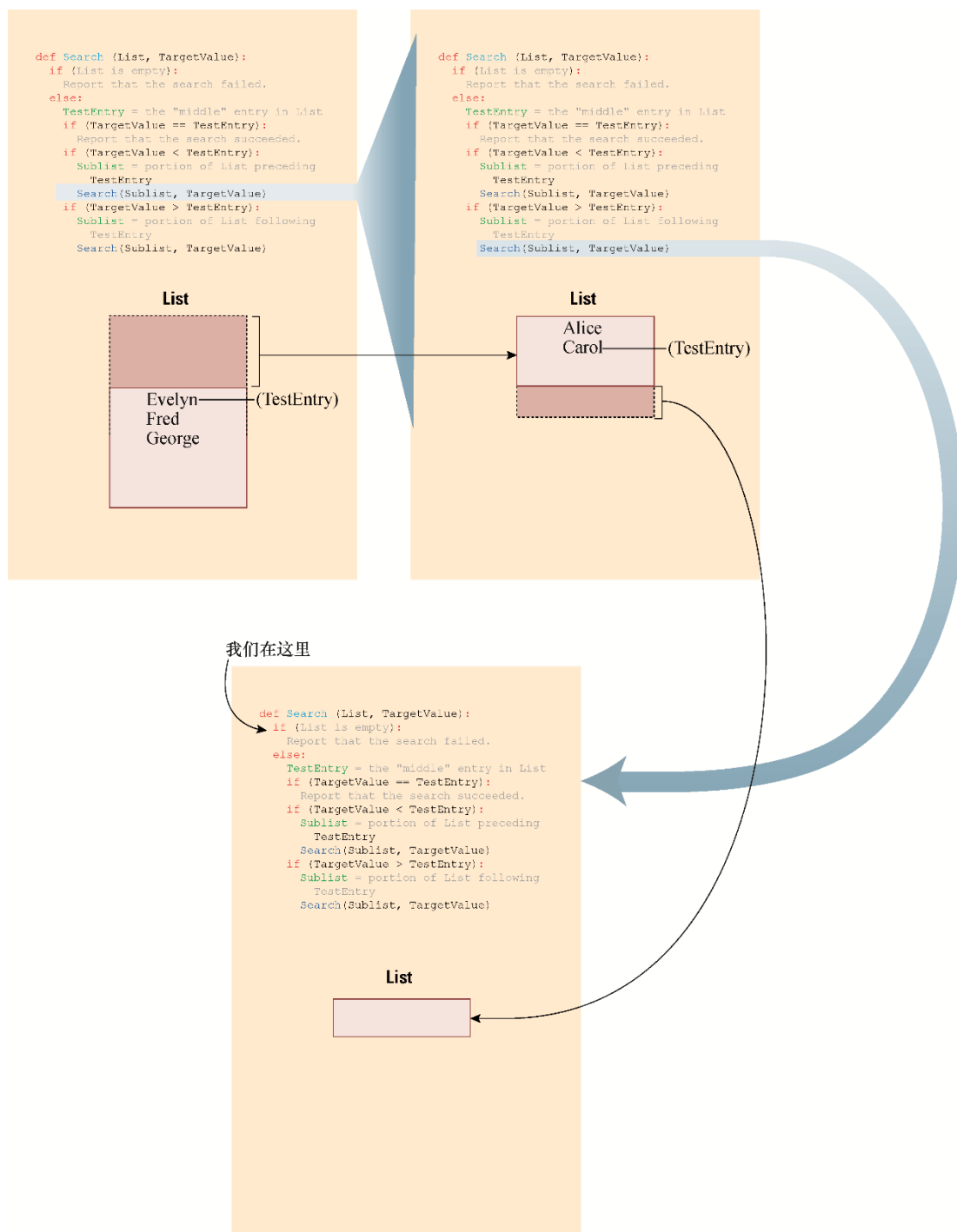


图5-17 第二次递归搜索，第二个快照

5.5.2 递归控制

二分搜索算法与顺序搜索算法相似，因为它们都要执行一个重复的过程。但是，这种重复的实现却是截然不同的。顺序搜索是以一种循环的方式重复执行一个过程，而二分搜索则是把每一阶段的重复当作前一阶段的子任务。这种技术称作**递归**（recursion）。

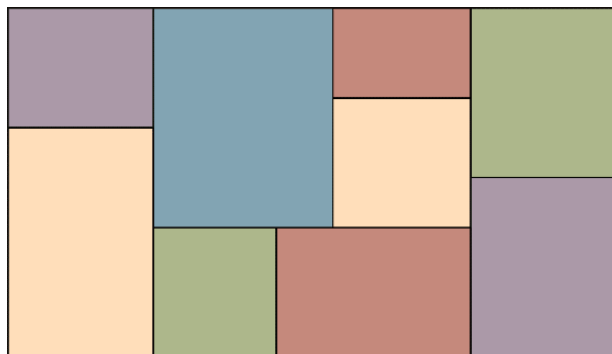
正如我们所见，执行递归函数产生的错觉是这个函数存在多个副本，每个副本称作这个函数的一次激活（activation）。这些激活通过一种伸缩方式动态创建，并随着算法的前进而最终消失。在任何给定时间存在的激活中，只有一次激活是活跃地进行的，其他激活都处于不确定的状态，每一次激活都要等待另一次激活终止后才能继续。

作为重复过程，递归系统也像循环结构一样依赖于正确的控制。就像循环控制一样，递归系统也依赖于对终止条件的测试，同时还依赖于保证终止条件能够达成的设计。事实上，正确的递归控制也涉及同样的3个要素——初始化、修改和终止测试——与循环控制的要求相同。

艺术中的递归结构

下面的递归函数可用于在一块长方形画布上绘制荷兰画家皮耶·蒙德里安（Piet Mondrian，1872—1944）风格的图画，他通过在一个长方形画布上将长方形连续划分成更小的长方形来绘画。你可以尝试自己实现这一函数来画出与下面所示图画类似的图画。把该函数应用到代表画布的一个长方形上，然后开始在上面绘制图画。（如果你正在怀疑由该函数表示的算法是否符合5.1节的定义，那么你的怀疑是有根据的。实际上，这是一个非确定性算法的例子，因为有几个地方需要

实现该函数的人或机器来做出“创造性”的决定。也许这就是为什么蒙德里安的结果被认为是艺术，而我们的却不是。)



```
def Mondrian (Rectangle):  
    if (用你的艺术眼光看Rectangle的尺寸太大):  
        把Rectangle划分成2个较小的长方形  
        把函数Mondrian应用到其中一个较小的长方形上  
        把函数Mondrian应用到另一个较小的长方形上
```

通常，递归函数会在请求进一步激活之前测试终止条件 [通常称作**基本条件** (base case) 或者**退化条件** (degenerative case)]。如果不满足终止条件，例程就创建该函数的另一次激活，并给它分配一项任务，让其解决一个修改后的问题；与分配给当前激活的任务相比，这个任务距离终止条件更近一些。但是，如果满足终止条件，就会采用导致当前活动终止而不创建额外激活的路径。

让我们来看看图5-14中的二分搜索函数是如何实现重复控制的初始化阶段和修改阶段的。在这个例子中，一旦目标值被找到或者任务被缩减到只是完成对空列表的搜索，额外激活的创建就会终止。整个过程是从简单地给出初始列表和目标值隐式开始的。从该初始配置开始，函数就会将分配给它的任务修改为在更小的列表中进行搜索。因

为原始列表的长度有限，并且每次修改步骤都会缩小考虑的列表长度，所以我们确信，最终能找到目标值，或者能将任务缩小到搜索空列表。因此，我们能够推断这个重复过程一定可以停止。

最后，由于循环控制结构和递归控制结构都是能让一系列指令重复执行的方法，我们可能会问：这两种结构在能力上是否等价？也就是说，如果一个算法是使用循环结构设计的，那么是否能设计出另一个只用递归技术就能解决同样问题的算法？反过来呢？这样的问题在计算机科学中是很重要的，因为它们的答案最终能告诉我们：为了获得尽可能强大的程序设计系统，应该在程序设计语言中提供什么特征。我们将在第12章中回到这些想法上来，到时，我们会考虑计算机科学及其数学基础的一些更具理论性的方面。在这种背景下，我们将能够证明附录E中提到的“迭代和递归结构的等价性”。

5.5节问题与练习

1. 在由Alice、Brenda、Carol、Duane、Evelyn、Fred、George、Henry、Irene、Joe、Karl、Larry、Mary、Nancy和Oliver组成的列表中，用二分搜索（见图5-14）搜索名字Joe的时候，会询问到哪些名字？

2. 当对一个200个条目的列表应用二分搜索时，最多要询问多少个条目？一个100 000个条目的列表呢？

3. 如果给N赋的初始值为1，那么下面的递归函数会打印出什么样的数列？

```
def Exercise (N):  
    print(N)  
    if (N < 3):  
        Exercise(N + 1)  
    print(N)
```

4. 上题中的递归函数的终止条件是什么？

5.6 效率和正确性

在本节中，我们介绍两个构成计算机科学中重要研究领域的主题。第一个是算法效率，第二个是算法正确性。

5.6.1 算法效率

尽管当今的机器每秒可以处理数百万或者数十亿条指令，但是效率仍旧是算法设计中所关注的一个主要问题。通常，在高效算法和低效算法之间的选择会产生问题的实用解决方案和不实用解决方案之间的区别。

让我们考虑这样一个问题，一个大学的教务主任需要面临检索和更新学生记录的任务。尽管在任何一个学期学校可能只有大约10 000名学生注册，但是它的“当前学生”文件包括30 000多名学生的记录，这些学生在过去的几年中至少注册了一门课程但还没有完成学业，所以在某种意义上来讲，他们都是当前学生。眼下，让我们假设这些学生的记录按学生标识号顺序以列表的形式存储于教务主任的计算机

中。因此，要找到一条学生记录，教务主任要在这个列表中查找特定的标识号。

我们已经介绍了两种用于搜索这种列表的算法：顺序搜索和二分搜索。现在的问题是，对教务主任来讲，这两种算法是否会带来不同的效果。我们首先考虑顺序搜索。

给定一个学生标识号，顺序搜索算法从列表的开头开始，将找到的条目与期望的标识号相比较。因为不知道关于目标值来源的任何信息，所以我们无法推断这个搜索要向列表中走多远。但是，我们可以说，在多次搜索之后，我们认为平均搜索深度是列表长度的一半；有的可能短一些，有的可能长一些。因此，我们估计，经过一段时间，顺序搜索平均每次大概需要检查15 000条记录。如果检索并检查每一条记录的标识号需要10毫秒（0.01秒），那么这样的搜索平均需要150秒（即2.5分钟）——这个时间对等待计算机屏幕显示学生记录的教务主任来说是不可忍受的。即使检索和检查每条记录所需的时间缩短到1毫秒，整个搜索仍然需要平均15秒，这个等待时间仍然较长。

相比之下，二分搜索算法通过比较目标值和列表的中间条目来进行搜索。即使中间条目不是期望的条目，至少也将剩余搜索限制在了原始列表的一半。因此，在询问了有30 000条学生记录的列表的中间条目之后，二分搜索最多还有15 000条记录需要考虑。在第二次询问之后，最多还剩7 500条记录，在第三次检索之后，要考虑的列表已降至不超过3 750条记录。照此继续，我们看到，最多检索15次之后，就能在这个有30 000条记录的列表中找到目标记录。因此，如果每次检

索需要10毫秒来完成，那么搜索一个特定记录的过程就只需0.15秒——这意味着，对教务主任来说，任何特定学生记录的访问都可以瞬间完成。我们得出结论：在顺序搜索算法和二分搜索算法之间的选择将对该应用产生巨大影响。

基本知识点

- 找到问题的高效算法可以帮助解决问题的更大实例。
- 确定算法的效率是通过对算法的形式化或数学推理来完成的。
- 同一个问题的不同的正确算法可能会有不同的效率。

这个例子表明了计算机科学领域中大家熟知的算法分析的重要性，算法分析包含了对资源（如时间或存储空间）的研究。这种研究的一个主要应用是评估替代算法的相对优劣。算法分析通常涉及最好情况、最差情况和平均情况。在上面的例子中，为了估计搜索一个30 000个条目的列表所需的时间，我们完成了顺序搜索算法的平均情况分析 and 二分搜索算法的最差情况分析。通常这种分析应该在更通用的情况下进行。也就是说，在考虑搜索列表的算法时，我们不能只考虑某个特定长度的列表，还要尝试找出一个公式，指示这种算法对于任意长度的列表的性能。将我们先前的推理推广到任意长度的列表并不难。具体而言，当应用于含有 n 个条目的列表时，顺序搜索算法将需要平均询问 $n/2$ 个条目，而二分搜索算法在最差情况下最多要询问 $\log_2 n$ 个条目。（ $\log_2 n$ 表示以2为底 n 的对数。除非另有说明，计算机科学家在谈论对数时，通常是指以2为底的对数。）

我们现在用类似的办法分析插入排序算法（如图5-11所示）。回想可知，这个算法需要选择一个称为主元的列表条目，将这个条目与这个条目之前的那些条目进行比较，直到找到正确的插入位置，然后将主元插入这个位置。因为该算法主要涉及两个条目之间的比较，所以我们的方法是计算在对长度为 n 的列表进行排序时需要进行的比较次数。

算法从把列表的第二个条目选作主元开始，然后继续选择后面的条目作为主元，直到到达列表的末尾。在可能的最好情况下，每个主元都已经在合适的位置，于是它只需与一个条目进行比较。因此，在最好情况下，将插入排序应用到一个有 n 个条目的列表需要进行 $n-1$ 次比较。（第二个条目与一个条目比较，第三个条目与一个条目比较，依次类推。）

相反，在最差情况下，每个主元都必须与列表中排在它前面的所有条目进行比较，然后才能找到合适的位置。这种情况发生在原始列表为倒序的时候。在这种情况下，第一个主元（列表的第二个条目）要与一个条目进行比较，第二个主元（列表的第三个条目）要与两个条目进行比较，依次类推（见图5-18）。因此，在对有 n 个条目的列表进行排序时，总共需要的比较次数为 $1 + 2 + 3 + \dots + (n-1)$ ，也就是 $(n^2 - n)/2$ 。具体而言，如果一个列表包含10个条目，那么在最差情况下，插入排序算法需要进行45次比较。

对每个主元所做的比较					
原始列表	第1个主元	第2个主元	第3个主元	第4个主元	已排序列表
Elaine David Carol Barbara Alfred	1 Elaine David Carol Barbara Alfred	3 David Elaine 2 Carol Barbara Alfred	6 Carol David 5 Elaine 4 Barbara Alfred	10 Barbara Carol 9 David 8 Elaine 7 Alfred	Alfred Barbara Carol David Elaine

图5-18 将插入排序应用于最差情况中

在插入排序的平均情况下，我们预计每个主元与其前面的一半条目进行比较。这样一来，一共需要执行的次数只是最差情况的一半，也就是总共只需 $(n^2-n)/4$ 次比较就可以完成对 n 个条目的列表的排序。例如，如果我们用插入排序算法对各种长度为10的列表进行排序，那么我们预计每种排序的平均比较次数是22.5。

这些结果的意义在于，插入排序算法在执行期间进行的比较次数给出了执行这种算法所需的近似时间量。使用这个近似值，图5-19显示的图表明了执行插入排序算法所需的时间是如何随着列表长度的增加而增长的。这张图是根据我们对这个算法的最差情况的分析画的，在此分析中，我们推算出对长度为 n 的列表进行排序最多需要在列表条目之间进行 $(n^2-n)/2$ 次比较。在图5-19中，我们标出了几个列表的长度，并给出了每种情况所需的时间。注意，随着列表的长度均匀增加，排序列表所需的时间增长得越来越快。因此，随着列表长度的增加，这个算法的效率会越来越低。

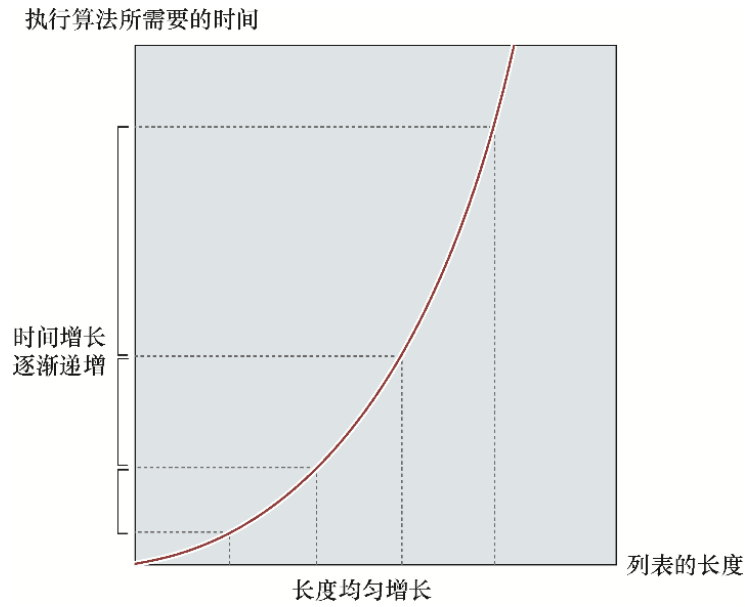


图5-19 插入排序算法的最差情况分析图

让我们使用相似的方法来分析一下二分搜索算法。回顾一下我们前面得到的结论，使用该算法搜索含有 n 个条目的列表最多需要询问 $\log_2 n$ 个条目，这又给出了对不同长度的列表执行这一算法时所需的近似时间量。图5-20给出了基于这个分析的曲线图，在这张图里，我们依旧标出了长度均匀增加的几个列表长度，并标出了每种情况下算法执行所需的时间。注意，随着列表长度的增加，算法所需时间量是逐渐递减的。也就是说，随着列表长度的增加，二分搜索算法的效率会越来越高。

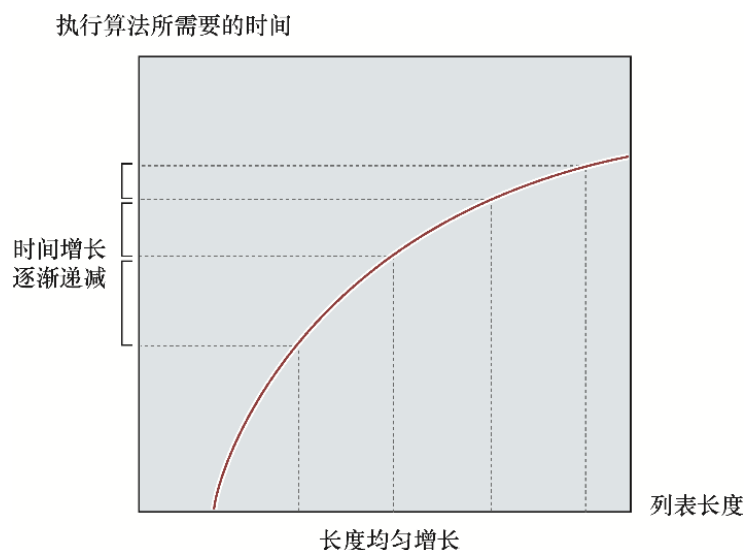


图5-20 二分搜索算法最差情况分析的曲线图

图5-19和图5-20的区别在于图的大体形状，该大体形状揭示了一个算法在应对越来越大的输入规模的情况下到底应该有多好。此外，图的大体形状是由表达式的类型而非具体的表达式所决定的：所有的线性表达式都会产生一条直线，所有的二次表达式都会产生一条抛物线，所有的对数表达式都会产生一条图5-20所示的对数曲线。习惯上，我们用可以产生一个形状的最简单的表达式来标识该形状。具体而言，我们用表达式 n^2 来标识抛物线形状，用 $\log_2 n$ 来标识对数曲线形状。

由于通过比较一个算法执行其任务所需的时间与其输入数据大小所得的图的形状反映了算法的效率特征，因此可以根据这些图的形状对算法进行分类——通常是基于算法的最差情况分析。用来标识这些类型的符号有时称作**大 Θ 符号**（big-theta notation）。所有图为抛物线形状的算法，如插入排序算法，都划归为 $\Theta(n^2)$ （读作“ n 的平方的大 Θ ”）表示的算法类型；所有图为对数曲线的算法，如二分搜索算法，

都划分到 $\Theta(\log_2 n)$ （读作“ $\log_2 n$ 的大 Θ ”）表示的算法类型中。知道特定算法归为哪一类，就可以预测它的性能，并且可以拿它与能够解决相同问题的其他算法进行比较。两个 $\Theta(n^2)$ 算法在输入大小增加的时候，对于时间将会有相近的需求变化。此外， $\Theta(\log_2 n)$ 算法的时间需求不会像 $\Theta(n^2)$ 算法那样增加得那么快。

5.6.2 软件验证

回想一下，波利亚对于问题求解的分析（见5.3节）的第4阶段是评估解决方案的准确性及其作为解决其他问题的工具的潜力。下面这个例子可以例证这个阶段第一部分的重要意义。

一位旅行者有一条七环的金链，他必须在一个偏僻的旅馆里住7晚。金链的一个环可以支付一晚的租金。必须最少切多少次，旅行者才能每天早上支付旅馆金链的一环而不用提前支付住宿费？

基本知识点

- 算法的正确性是通过算法的形式化或数学推理来确定的，而不是通过测试算法的实现来确定的。

要解决这个问题，我们首先要认识到并不是金链的每一环都要切开。如果只切开第2个环，就可以让第1个环和第2个环与另外5个环分开。按照这个想法，我们得到这样一种解，就是只需切开金链的第2、第4和第6个环，这个过程只切3环就分开了所有的环（见图5-21）。此

外，切任何更少的次数都会导致两环仍然连在一起，所以我们可能会得出结论，这个问题的正确答案是3次。

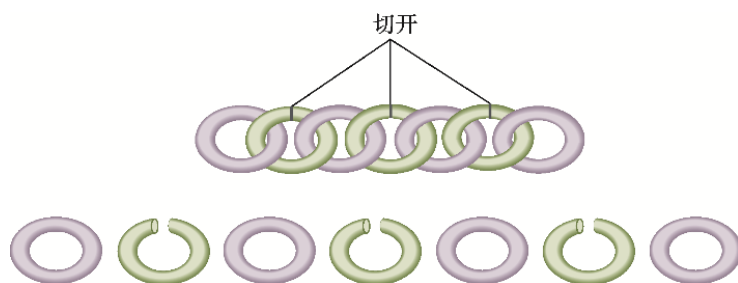


图5-21 只切3次就将金链分开

然而，当我们重新考虑这个问题时，我们可能会发现，在只切第3个环时，我们会获得3段金链，金链的长度分别是1、2和4（见图5-22）。对这些段，我们可以进行如下操作。

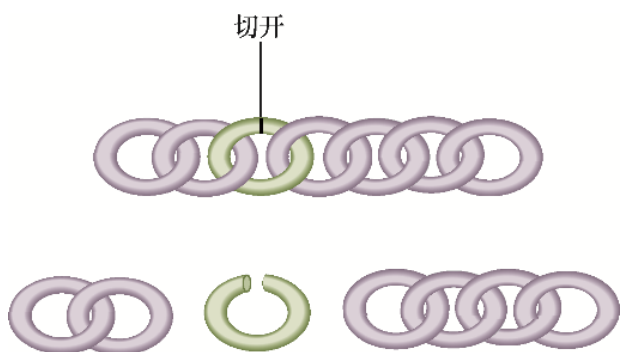


图5-22 只切1次就解决了这个问题

第1天早上：把1个环的那段给旅馆。

第2天早上：把2个环的那段给旅馆，同时拿回1个环。

第3天早上：把1个环的那段给旅馆。

第4天早上：把4个环的那段给旅馆，同时拿回原先给旅馆的那3个环。

第5天早上：把1个环的那段给旅馆。

第6天早上：把2个环的那段给旅馆，同时拿回1个环。

第7天早上：把1个环的那段给旅馆。

因此，第一个答案，即我们认为正确的那个答案，实际上是不正确的。然而，我们又如何认定我们这个新解决方案是正确的呢？可以这样论证：因为第1天早上必须给旅馆1个环，所以至少要切开一个环，同时因为新解决方案只需切一次，所以必定是最优的。

回到程序设计环境中，这个例子强调了一个被认为正确的程序和一个正确的程序之间的区别，二者并不一定相同。计算社区有很多涉及软件的可怕的故事，例如，尽管“已知”一个软件是正确的，但最终还是因为一些没有预料到的情况而在关键时刻发生故障。因此，软件验证是一项重要的保证，对有效验证技术的搜寻也成了计算机科学中一个活跃的研究领域。

在这个领域内，研究的一条主线尝试把形式逻辑技术用于证明一个程序的正确性。也就是说，目标是用形式逻辑来证明程序表达的算法确实做了它打算做的事。基本论点是通过将验证过程简化为一个形式过程，以防止那些可能与直觉有关的不准确结论，就像金链问题一样。下面来更详细地讨论如何把这个方法应用于程序验证中。

软件验证之外

文中所讨论的验证问题并不是软件独有的。确认执行程序硬件没有缺陷是一个同等重要的问题，这涉及电路设计和机器构造的验证。再强调一次，目前工艺水平严重依赖于测试，就像在软件的情况一样，这意味着任何一个细微的错误都可能出现在最终的产品中。据记载，在20世纪40年代，由哈佛大学建造的马克一号（Mark I）包含了很多布线错误，而这些错误很多年都没有被发现。在20世纪90年代，在早期奔腾微处理器的浮点部分出现过一个缺陷，这个缺陷导致在对某些数进行除法运算时计算出的答案是错误的。在这两个例子中，错误都是在产生严重后果之前被发现的。

像模型检查（model checking）这样的现代数学技术为硬件和某些类型的软件的验证提供了一条希望之路，只是在能力和表现方面有其自身的局限性。

就像形式数学证明要基于公理进行一样（几何证明通常基于欧几里得几何公理，而其他证明则基于集合论的公理），程序正确性的形式证明是基于设计程序所使用的规格说明进行的。为了证明一个程序可正确地对名字列表进行排序，我们可以从假设程序的输入是一个名字列表开始，或者说，如果一个程序是设计用来计算一个或者更多个正数的平均值的，就假设输入确实是由一个或多个正数组成的。简而言之，正确性的证明是从以下假设开始的：程序开始执行时满足某些条件 [这个条件称为**前置条件**（precondition）]。

正确性证明的下一步是考虑这些前置条件的结果是如何在程序中传播的。为了这个目的，研究人员分析了各种各样的程序结构来确定一条语句（已知在结构执行之前为真）是如何受到结构执行的影响的。作为一个简单的例子，如果在执行指令

```
X = Y
```

之前，已知一条关于Y值的语句成立，那么在该指令执行完之后，那条关于Y值的语句对X值也同样成立。更准确地讲，如果在指令执行之前已知Y的值不为0，那么可以推断，指令执行以后X的值也不为0。

一个稍微复杂点的例子发生在下面这样的if-else结构中。

```
if (条件):  
    指令A  
else:  
    指令B
```

这里，如果已知某个语句在结构执行之前成立，那么在紧接着执行指令A之前，我们知道那条语句和那个测试条件皆为真；相反，如果要执行的是指令B，我们就知道那条语句和测试条件的否定一定成立。

依照这些规则，可以通过识别称作**断言**（assertion）的语句来进行正确性的证明，断言可以建立在程序中的各个点上。结果是一个断言集合，每个断言都是程序前置条件的一个结果以及可以导致在程序中的某点建立断言的一组指令。如果在程序的末尾建立的断言可以得

到想要的输出规格说明 [称为**后置条件** (postcondition)] , 我们就能断定程序是正确的。

作为一个例子, 考虑图5-23中所示的典型的while循环结构。假设, 作为在A点给定的前置条件的结果, 循环过程中每次进行终止测试的时候 (B点) , 我们都能确认一个特定断言为真。 [一个存在于某个循环内某一点的断言, 如果在每次执行到这个循环内这一点时都为真, 则称作**循环不变式** (loop invariant) 。] 然后, 如果循环永远终止, 执行就会转移到C点, 在这里我们可以得出结论: 循环不变式和终止条件均成立。(循环不变式仍旧成立是因为终止测试不改变程序中的任何值, 终止条件成立是因为如果它不成立循环就不会终止。) 如果这些组合语句暗示着期望的后置条件, 我们的正确性证明仅仅通过初始化和修改最终导致终止条件的循环组件就可以完成。

应该拿这个分析与图5-11中插入排序的例子相比较。那个程序的外层循环依据的循环不变式是

每次执行终止测试的时候, 列表中从位置1到位置 $N-1$ 的条目都已排序

终止条件是

N 的值大于列表的长度

因此, 如果循环永远终止, 我们就知道两个条件一定都满足了, 这暗示着整个列表已排序。

程序验证技术发展的进展依然充满挑战。即便这样，还是取得了一些进展，其中一个比较重要的进展是在程序设计语言SPARK中发现的，SPARK语言与更为流行的Ada语言之间有着紧密的联系。（关于Ada语言，我们将在下一章举例说明。）除了允许程序用像伪代码这样的高级形式表示，SPARK还给程序员提供了包含断言的方法，如程序里的前置条件、后置条件和循环不变式。因此，用SPARK语言编写的程序不仅包含要应用的算法，还包含应用形式化正确性证明技术所需的信息。迄今为止，SPARK已经成功地应用于涉及关键软件应用的很多软件开发项目中，包括美国国家安全局的安全软件、美国洛克希德·马丁公司的C130J大力神运输机的内部控制软件以及关键铁路运输控制系统。

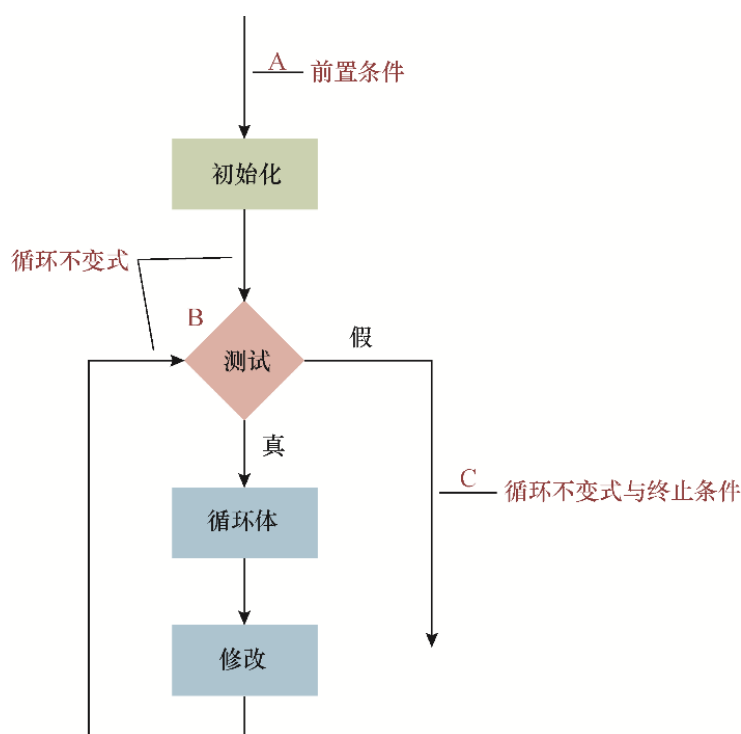


图5-23 与典型的while结构相关的断言

尽管有SPARK这样的成功案例，但形式化程序验证技术还没有得到广泛的应用，因此今天大多数的软件还是通过测试来进行“验证”，而这个过程是不可靠的。毕竟，通过测试进行的验证仅能证明程序对于测试用例是正确的。任何额外的结论都只是预测。程序中包含的错误往往都是测试和开发过程中没有注意到的一些细微疏忽导致的结果。因此，就像我们在金链问题中犯的错误一样，尽管做了很大的努力去避免程序中的错误，错误往往还是发现不了。2003年美国 and 加拿大出现了一个戏剧性的例子。发电站监测软件的竞争条件（回忆第3章）致使美国8个州和安大略省超过5000万人的长时间停电。

5.6节问题与练习

1. 假设有一台使用插入排序算法编程的机器对100个名字的列表进行排序平均需要1秒。估算一下对1 000个名字的列表进行排序需要多长时间？10 000个名字的列表呢？

2. 为下面每类算法列举一个例子： $\Theta(\log_2 n)$ 、 $\Theta(n)$ 和 $\Theta(n^2)$ 。

3. 将下列每类算法按效率从高到低的顺序排列： $\Theta(n^2)$ 、 $\Theta(\log_2 n)$ 、 $\Theta(n)$ 和 $\Theta(n^3)$ 。

4. 考虑下面的问题及建议的答案。建议的答案对吗？为什么对或者不对？

问题：假设一个盒子里有3张卡片，其中一张两面都涂成黑色，另一张两面都涂成红色，第三张一面涂成红色，另一面涂成黑色。抽出其中一张卡片，只允许看一面，那么另一面的颜色与你看到的那一面的颜色相同的概率有多大？

建议的答案：二分之一。假设你看到的卡片的那一面是红色的（如果是黑色的，讨论结果也是一样的）。三张卡片中只有两张有红色的一面，因此你看到的卡片一定是这两张中的一张。这两张中的一张背面是红色的，而另一张背面是黑色的。因此，你看到的卡片背面是红色的概率和是黑色的概率一样大。

5. 下面的程序段尝试计算两个正整数（一个被除数和一个除数）的商（不考虑余数），方法是计算从被除数中减去除数，直到余数比除数小时减了多少次。例如，7/3对应的结果应该为2，因为3可以从7中减2次。这段程序正确吗？证明你的结论。

```
Count = 0
Remainder = Dividend
repeat:
    Remainder = Remainder - Divisor
    Count = Count + 1
until (Remainder < Divisor)
Quotient = Count
```

6. 下面的程序段通过累计X个Y的和的方法来计算非负整数X和Y的积，也就是说，3乘以4就是计算3个4的和。这段程序正确吗？证明你的结论。

```
Product = Y
Count = 1
while (Count < X):
    Product = Product + Y
    Count = Count + 1
```

7. 假设有这样一个前置条件： N 的值是一个正整数，建立一个循环不变式，使得只要下面的例程终止，Sum就被赋值为 $0+1+2+\dots+N$ 。

```
Sum = 0
K = 0
while (K < N):
    K = K + 1
    Sum = Sum + K
```

提供一个能让例程实际终止的参数。

8. 假设一个程序及执行它的硬件都已被形式化地验证是准确的。这能保证准确性吗？

复习题

（带*的题目涉及选学章节的内容。）

1. 给出一组步骤的例子，使它符合5.1节开始部分给出的算法的非正式定义，但不符合图5-1给出的正式定义。

2. 解释建议的算法中的歧义性和算法表示中的歧义性的区别。

3. 描述如何使用原语来帮助消除算法表示中的歧义性。

4. 选择一个你比较熟悉的学科，并设计一个伪代码来描述它。具体来说就是，描述你要使用的原语以及你用于表示它们的语法。（如果你想不出什么熟悉的学科，可以考虑体育、艺术或者工艺。）

5. 从严格意义上讲，下面的程序表示的是一个算法吗？为什么是或者不是？

```
Count = 0
while (Count != 5):
```



```
Count = Count + 2
```

6. 从什么意义上讲，下列3个步骤并不构成一个算法？

步骤1：在直角坐标系的点(2, 5)和点(6, 11)之间画一条直线段。

步骤2：在直角坐标系的点(1, 3)和点(3, 6)之间画一条直线段。

步骤3：以上面两条线段的交点为中心，画一个半径为2的圆。

7. 用repeat结构代替while结构重写下面的程序段，确保它输出的值与原程序相同。

```
Count = 2
while (Count < 7):
    print(Count)
    Count = Count + 1
```

8. 用while结构代替repeat结构重写下面的程序段，确保它输出的值与原程序相同。

```
Count = 1
repeat:
    print(Count)
    Count = Count + 1
until (Count == 5)
```

9. 要把以

```
repeat:
    (. . .)
until (. . .)
```

形式表达的后测试循环转换为以

```
do:
  (. . .)
while (. . .)
```

形式表达的等价的后测试循环，必须做什么？

10．设计一个算法，当给定数字0, 1, 2, 3, 4, 5, 6, 7, 8, 9的一个排列时，重新排列这些数字，使产生的新排列表示的是这些数字的所有排列中与原排列最临近且比原排列大的值（或者，如果没有重新排列能产生更大的值，就报告不存在这样的重新排列）。例如，5647382901将产生5647382910。

11．设计一个算法，找出一个正整数的所有因子。例如，对于整数12，该算法应该报告值1、2、3、4、6和12。

12．设计一个算法，计算从1700年1月1日起的任意一天是星期几。例如，2001年8月17日是星期五。

13．正式的程序设计语言和伪代码的区别是什么？

14．语法和语义之间的区别是什么？

15．下面是一个传统的十进制加法题，每个字母代表一个不同的数字。问每个字母代表什么数字？你是怎样迈出第一步的？

$$\begin{array}{r} \text{XYZ} \\ + \text{YWY} \\ \hline \text{ZYZW} \end{array}$$

16．下面是一个传统的十进制乘法题，每个字母代表一个不同的数字。问每个字母代表什么数字？你是怎样迈出第一步的？

$$\begin{array}{r} XY \\ \times YX \\ \hline XY \\ YZ \\ \hline WVY \end{array}$$

17．下面是一个二进制加法题，每个字母代表一个不同的二进制数字。问哪个字母代表1，哪个字母代表0？请为解决此类问题设计一个算法。

$$\begin{array}{r} YXX \\ + XYX \\ \hline XYYY \end{array}$$

18．有4位采矿者，他们只有一个提灯，并且必须走路通过挖煤的坑道。他们最多可以两个人一起通过，并且其中一个人必须拿着提灯。这4位采矿者分别叫Andrews、Blake、Johnson和Kelly，他们单独通过矿井的时间分别是1分钟、2分钟、4分钟和8分钟。当两个人一起通过坑道时，要以速度慢的人的速度为准，如何安排才能使这4人在15分钟内通过坑道？在求解完这道题后，解释你是怎样迈出第一步的。

19．有一大一小两个酒杯，先往小酒杯中倒满酒，再把小酒杯中的酒倒入大酒杯。然后，把小酒杯中倒满水，再把小酒杯中的一些水倒入大酒杯。在大酒杯中均匀搅拌，再把混合液体倒回到小酒杯，直到倒满。请问此时大酒杯中的水比小酒杯中的酒多吗？解释你是怎样迈出第一步的。

20．两只蜜蜂，一只叫罗密欧，一只叫朱丽叶，它们住在不同的蜂房，但是它们相爱了。在一个无风的春天的早晨，它们同时离开各自的蜂房来相会。它们相遇的地点在距离最近的蜂房50米的地方，但它们都没看到对方，因此继续按各自的方向飞，直到飞到对方的蜂房。它们用了同样多的时间发现对方没在家，并开始返回。在距离最近蜂房20米的地方，它们又相遇了，这次它们看到了对方，并在回家之前去野餐了。请问这两个蜂房相距多远？在求解完这道题后，解释你是怎样迈出第一步的。

21．设计一个算法，给定两个字符串，测试第一个字符串是否是第二个字符串的子串。

22．下面这个算法是为打印所谓的斐波那契序列的开始部分而设计的。请标识这个循环体，哪儿是循环控制的初始化步骤？哪儿是修改步骤？哪儿是测试步骤？产生的数字列表是什么？

```
Last = 0
Current = 1
while (Current < 100):
    print(Current)
    Temp = Last
    Last = Current
    Current = Last + Temp
```

23．在下面的算法中，如果输入值分别以0和1开始，打印的数的序列是什么？

```
def MysteryWrite (Last, Current):
    if (Current < 100):
        print(Current)
        Temp = Current + Last
        MysteryWrite(Current, Temp)
```

24 . 修改上一题中的函数MysteryWrite , 使值以相反的顺序打印。

25 . 如果用二分搜索算法 (见图5-14) 从由给定的字母A、 B、 C、 D、 E、 F、 G、 H、 I、 J、 K、 L、 M、 N、 O组成的列表中搜索值J , 那么哪些字母会被询问到 ? 如果搜索值Z呢 ?

26 . 在对一个有6000个条目的列表进行许多次顺序搜索之后 , 你认为目标值与列表条目进行比较的平均次数是多少 ? 如果搜索算法是二分搜索呢 ?

27 . 确定下列每个迭代语句的终止条件。

a.while (Count < 5):

 . . .

b.repeat:

 . . .

until (Count == 1)

c.while ((Count < 5) and (Total < 56)):

 . . .

28．标识下列循环结构的循环体，计算它将执行多少次。如果把测试条件改变为“(Count != 6)”会怎样？

```
Count = 1
while (Count != 7):
    print(Count)
    Count = Count + 3
```

29．如果在计算机上实现下面这个程序，你觉得会发生什么问题？（提示：记得与浮点运算相关的舍入误差问题。）

```
Count = one_tenth
repeat:
    print(Count)
    Count = Count + one_tenth
until (Count == 1)
```

30．设计一个递归版本的欧几里得算法（见5.2节问题与练习3）。

31．假设Test1和Test2（在下面定义）的输入值都为1，那么下面两个例程的打印结果会有什么不同？

```
def Test1 (Count):
    if (Count != 5):
        print(Count)
        Test1(Count + 1)
def Test2(Count):
    if (Count != 5):
        Test2(Count + 1)
    print(Count)
```

32．确定上一题的例程中控制机制的重要组成要素。具体而言，什么条件会使这一过程终止？在哪里可以修改过程的状态，让它朝着终止条件变化？控制过程的状态是在哪里初始化的？

33 . 确定下面递归函数的终止条件。

```
def XXX (N):  
    if (N == 5):  
        XXX(N + 1)
```

34 . 用值3调用函数MysteryPrint (在下面定义) , 记录打印的值。

```
def MysteryPrint (N):  
    if (N > 0):  
        print(N)  
        MysteryPrint(N - 2)  
    print(N + 1)
```

35 . 用值2调用函数MysteryPrint (在下面定义) , 记录打印的值。

```
def MysteryPrint (N):  
    if (N > 0):  
        print(N)  
        MysteryPrint(N - 2)  
    else:  
        print(N)  
        if (N > -1):  
            MysteryPrint(N + 1)
```

36 . 设计一个算法 , (按递增顺序) 生成其素数因子为2和3的正整数的序列。也就是说 , 你的程序应该产生这样的序列 : 2, 3, 4, 6, 8, 9, 12, 16, 18, 24, 27, ...。在严格意义上讲 , 你的程序表示的是一个算法吗 ?

37 . 针对由Alice、Byron、Carol、Duane、Elaine、Floyd、Gene、Henry和Iris组成的列表回答下列问题。

- a . 哪种搜索算法（顺序或二分）能更快找到名字Gene？
- b . 哪种搜索算法（顺序或二分）能更快找到名字Alice？
- c . 哪种搜索算法（顺序或二分）能更快检测出名字Bruce不存在？
- d . 哪种搜索算法（顺序或二分）能更快检测出名字Sue不存在？
- e . 在用顺序搜索算法搜索名字Elaine时，需要询问多少个条目？用二分搜索呢？

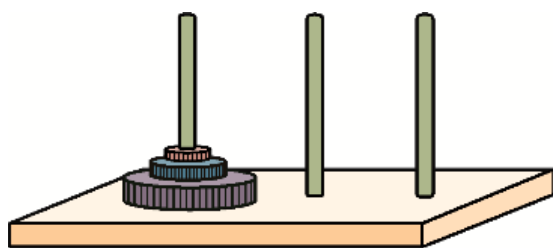
38 . 0的阶乘定义为1，正整数的阶乘定义为整数本身和比自己小的非负整数的阶乘。我们用符号 $n!$ 来表示整数 n 的阶乘。于是，3的阶乘（写作 $3!$ ）就是 $3 \times (2!) = 3 \times (2 \times (1!)) = 3 \times (2 \times (1 \times (0!))) = 3 \times (2 \times (1 \times (1))) = 6$ 。请设计一个递归算法来计算一个给定值的阶乘。

39 . a. 假设你必须给一个有5个名字的列表排序，而且你已经设计了一个能给有4个名字的列表排序的算法。请利用已经设计好的算法来设计一个能给有5个名字的列表排序的算法。

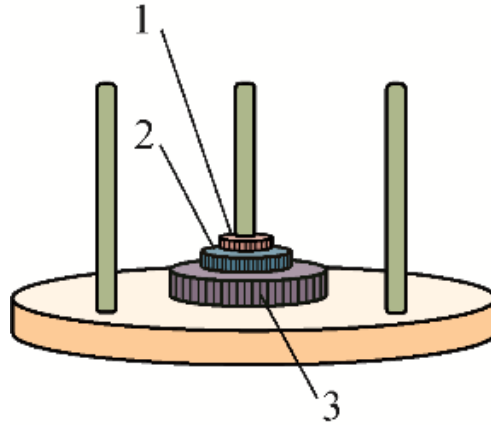
b . 基于问题a中使用的技术，设计一个能给任意长的名字列表排序的递归算法。

40 . 称为汉诺塔的谜题有3根柱子，每个柱子都可以放置若干个大小不同的环，这些环自底向上直径越来越小。这个问题是，如何将一个柱子上排列好的环移到另一个柱子上，规则是每次只能移动一个

环，较大的环不能放在较小的环上面。我们看到，如果总共就只有一个环，那么问题就非常容易。其次，当要移几个环的时候，如果你把除最大环之外的所有环都移到另一根柱子上，就能把这个最大的环移到第三根柱子上，然后问题就是把其余的环移到它上面。利用这个分析，开发一个递归算法来解决任意环数的汉诺塔谜题。



41. 求解汉诺塔谜题（见第40题）的另一种方法是把3根柱子想象成安装在一个圆形置物板上，它们分别位于4点钟、8点钟和12点钟位置。开始时，一根柱子上的环从小到大以1、2、3等依次编号，最小的环编号为1。看一根柱子最上面的环，如果它的编号是奇数，则允许它按照顺时针方向移到下一根柱子上；同样，如果它的编号是偶数，则允许它按照逆时针方向移到下一根柱子上（只要不把较大的环放在较小的环的上面就行）。在这个限制条件下，当几个柱子上有可移动的环时，总是移动编号最大的环。按照这个思路，开发一个非递归算法来解决汉诺塔谜题。



42．开发两个算法，用来打印一个工人30天的日薪，要求一个算法基于循环结构，另一个基于递归结构。这个工人每天的薪水是前一天的两倍（第一天的薪水设为1便士）。如果你在实际机器上实现你的算法，那么在数的存储上你可能会遇到什么问题？

43．设计一个算法，求一个正数的平方根。开始时，把这个正数本身作为根的第一个猜测值，然后通过将原正数除以前一个猜测值得到的结果与前一个猜测值取平均来从前一个猜测值反复产生新的猜测值。分析对这个重复过程的控制。具体来说，这个重复过程的终止条件应该是什么？

44．设计一个算法，列出一个由5个不同字符组成的字符串中符号的所有可能的重新排列。

45．设计一个算法，在给定的名字列表中找到最长的名字。使用for循环结构。如果列表里有多“最长”的名字，你的算法会做什么？特别地，如果列表里所有名字的长度都一样，你的算法会做什么？

46．设计一个算法，给定一个由5个数或更多数组成的列表，在不对整个列表进行排序的情况下，在列表中找出5个最小的数和5个最大的数。

47．将名字Brenda、Doris、Raymond、Steve、Timothy和William按顺序排列，要求在使用插入排序算法（见图5-11）进行排序时比较次数最少。

48．对于有4000个名字的列表，使用二分搜索算法（见图5-14）时最多要询问多少个条目？这与顺序搜索算法（见图5-6）相比如何？

49．使用大 Θ 符号对传统的小学加法和乘法的算法进行分类。也就是说，如果两个各有 n 位数字的数相加，那么要做多少次一位的加法？如果两个各有 n 位数字的数相乘，那么要做多少次一位的乘法？

50．有时对一个问题稍做变动就可能使它的解的形式发生重大改变。例如，找到一个简单的算法来求解下述问题，并用大 Θ 符号对它进行归类：

把一群人分为两个不相交的（任意大小的）子组，使得两个子组成员的年龄的总和的差尽可能大。

现在把问题改为，使得两个子组成员的年龄的总和的差尽可能小，再利用大 Θ 符号对这个问题的方法进行归类。

51．从下面的列表找出一组数，使其总和等于3165。你的方法的效率如何？

26 . 39, 104, 195, 403, 504, 793, 995, 1156, 1677

52 . 下面例程中的循环会终止吗？请解释你的答案。请解释，如果计算机实际执行这个例程，可能会发生什么情况（参见1.7节）。

```
X = 1
Y = 1 / 2
while (X != 0):
    X = X - Y
    Y = Y / 2
```

53 . 下面的程序段是用来计算两个非负整数X和Y的乘积的，方法是累加X个Y的和；也就是说，3乘以4是通过累加3个4得到的。这个程序段正确吗？请解释你的答案。

```
Product = 0
Count = 0
repeat:
    Product = Product + Y
    Count = Count + 1
until (Count == X)
```

54 . 下面的程序段是用来报告正整数X和Y中哪个大的。这段程序正确吗？请解释你的答案。

```
Difference = X - Y
if (Difference is positive):
    print('X is bigger than Y')
else:
    print('Y is bigger than X')
```

55 . 下面的程序段是用来从一个非空的整数列表中找到最大条目的。这个程序段正确吗？请解释你的答案。

```
TestValue = first list entry
CurrentEntry = first list entry
while (CurrentEntry is not the
    last entry):
    if (CurrentEntry > TestValue):
        TestValue = CurrentEntry
    CurrentEntry = the next list entry
```

56 . a . 标识图5-6中所示的顺序搜索算法的前置条件。为这个程序里的while结构建立一个循环不变式，当它与终止条件结合时，就意味着，在该循环终止时该算法将正确地报告成功或失败。

b . 给出一个论据，说明图5-6中的while循环事实上是会终止的。

57 . 基于“赋给X和Y的值是非负整数”这个前置条件，标识下面的while结构里的循环不变式，当它与终止条件结合时，就意味着，与Z关联的值在循环终止时一定是X - Y。

```
Z = X
J = 0
while (J < Y):
    Z = Z - 1
    J = J + 1
```

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1．因为现在完全验证复杂程序的正确性几乎是不可能的，所以在何种情况下（如果有这种情况）程序的创建者应该对错误负有责任？

2．假设你有一个想法，并把它开发成了一个供很多人使用的产品，而这已经耗费了你一年的时间和50 000美元的投资。可是，该产品的最终形式可能被大多数没有向你购买该产品的人使用。为了获得补偿，你具有哪些权利？盗版计算机软件道德吗？音乐和电影呢？

3．假设一个软件包非常昂贵，完全超出了你的预算，那么复制这个软件供自己使用道德吗？（毕竟，你无论如何都不可能去购买这个软件包，所以对供应商的销售额不会有影响。）

4．人们对河流、森林、海洋等的所有权一直争论不休，那么在什么意义上应该给某人或某机构一个算法的所有权？

5．一些人觉得新算法是被发现的，而另一些人觉得新算法是被创造出来的。你同意哪种说法？这些不同观点会导致关于算法的所有权和一般的所有权的不同结论吗？

6．设计一个实现非法行为的算法是道德的吗？它与该算法是否被实际执行有关吗？开发出这种算法的人应该具有该算法的所有权吗？如果应该具有，应该拥有哪些权利？算法的所有权应该与该算法的目的有关吗？大肆宣扬和散布破坏安全的技术道德吗？这与破坏的内容有关吗？

7. 尽管故事在电影版中经常被更改，作者仍会因小说的电影版权获得报酬。一个故事要改成一个不同的故事，必须改多少？一个算法要改成一个不同的算法，必须改多少？

8. 教育软件正在面向18个月或更小儿童销售。支持者认为，这些软件提供的视界和声音是许多孩子无法通过其他途径获得的；反对者认为，它是父母与子女之间交流的拙劣的替代品。你有什么看法？你会在对这个软件没有更多了解的情况下根据自己的意见采取行动吗？如果会，你会采取什么行动？

课外阅读

Aho, A. V., J. E. Hopcroft, and J. D. Ullman. *The Design and Analysis of Computer Algorithms*. Boston, MA: Addison-Wesley, 1974.

Baase, S. *Computer Algorithms: Introduction to Design and Analysis*, 3rd ed. Boston, MA: Addison-Wesley, 2000.

Barnes, J. *High Integrity Software: The SPARK Approach to Safety and Security*. Boston, MA: Addison-Wesley, 2003.

Gries, D. *The Science of Programming*. New York: Springer-Verlag, 1998.

Harbin, R. *Origami—the Art of Paper Folding*. London: Hodder Paperbacks, 1973.

Johnsonbaugh, R. and M. Schaefer. *Algorithms*. Upper Saddle River, NJ: PrenticeHall, 2004.

Kleinberg, J. *Algorithm Design*, 2nd ed. Boston, MA: Addison-Wesley, 2014.

Knuth, D. E. *The Art of Computer Programming*, Vol. 3, 2nd ed. Boston, MA: Addison-Wesley, 1998.

Levitin, A. V. *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Boston, MA: Addison-Wesley, 2011.

Polya, G. *How to Solve It*. Princeton, NJ: Princeton University Press, 1973.

Roberts, E. S. *Thinking Recursively*. New York: Wiley, 1986.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书
配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第6章 程序设计语言

本章我们来学习程序设计语言。虽然我们将继续在合适的地方使用Python示例，但是我们的目的不是专注于某一门特定的程序设计语言，而是学习程序设计语言。我们要了解程序设计语言及其相关联的方法之间的共性和个性。

本章内容

6.1 历史回顾

6.2 传统的程序设计概念

6.3 过程单元

6.4 语言实现

6.5 面向对象程序设计

*6.6 程序设计并发活动

*6.7 说明性程序设计

持久理解和学习成果

多层次抽象用于编写程序或创建其他计算制品。AP[®]CSP

学习成果 在编写程序或创建其他计算制品时开发抽象。

学习成果 使用多层次抽象来编写程序。

学习成果 确定编写程序时使用的多层次抽象。

算法是计算机可以执行的过程的精确指令序列，可以使用程序设计语言来实现。AP[®]CSP

学习成果 针对程序中的实现开发算法。

学习成果 用语言表达算法。

可以开发程序用于创造性表达、满足个人好奇心、创造新知识或解决问题（以帮助人、组织或社会）。AP[®]CSP

学习成果 开发正确的程序来解决问题。

人编写程序来执行算法。AP[®]CSP

学习成果 解释程序如何实现算法。

程序设计得益于适当的抽象。AP[®]CSP

学习成果 使用抽象来管理程序的复杂度。

如果人们被迫使用机器语言直接编写程序，很可能就无法开发出复杂的软件系统，如操作系统、网络软件和当今可用的大量应用软

件。我们至少可以说，在试图组织复杂系统的同时，处理这些与机器语言有关的错综复杂的细节将会是一项繁重的工作。

因此，人们开发出许多程序设计语言，使得算法的表达既便于人理解，又便于转换为机器语言指令。本章的目标是探究处理这些语言的设计和实现的计算机科学领域。

6.1 历史回顾

我们从追溯程序设计语言的历史发展开始学习。

6.1.1 早期程序设计语言

我们在第2章学过，现代计算机的程序由采用数字编码的指令序列组成。这样的编码系统称为机器语言。但是，用机器语言编写程序是一项烦琐的任务，而且经常出错，在工作完成之前，这些错误必须被找到并更正——这个过程称为**调试**（debugging）。

20世纪40年代，研究人员为了简化程序设计过程开发了计数制系统，使得指令可以用助记符表示，而不用使用数字形式表示。例如，指令

把寄存器5的内容移到寄存器6

可用第2章介绍的Vole机器语言表示为

0x4056

而使用助记符系统时，可以表示为

MOV R5, R6

作为一个更大一点的例子，机器语言例程

0x156C 0x166D 0x5056 0x306E 0xC000
--

将存储单元0x6C和0x6D的内容相加，将结果存储在地址0x6E（见第2章图2-7）中，可以使用助记符表达为

LD R5, Price LD R6, ShippingCharge ADDI R0, R5 R6 ST R0, TotalCost HLT
--

这里，我们使用LD、ADDI、ST和HLT分别表示加载、相加、存储和停止。此外，我们使用描述性名称Price、ShippingCharge和TotalCost分别指代地址为0x6C、0x6D和0x6E的存储单元。这些描述性名称经常被称为程序变量或标识符（identifier）。注意，助记符形式虽然有不足之处，但是与数字形式相比，它确实能更好地表达例程含义。

建立起这种助记符系统后，人们就开发了称为汇编器（assembler）的程序，用来将助记符表达式转换为机器语言指令。以

此方式，人们可以使用助记符的形式开发程序，然后再用汇编器把它转换为机器语言，而不必直接使用机器语言开发程序。

表示程序的助记符系统统称为**汇编语言**（assembly language）。汇编语言最初被开发出来的时候，它们代表了在研究更好的程序设计技术方面迈出了巨大一步。实际上，汇编语言非常具有革命性，因此被称为第二代程序设计语言，而第一代程序设计语言是机器语言本身。

尽管汇编语言与它们对应的机器语言相比有不少的优势，但是仍有一些不足——它们没有提供最终的程序设计环境。毕竟，汇编语言中使用的原语本质上和与之相对应的机器语言中的原语相同。这两者的区别仅体现在用于表示它们的语法上。因此，用汇编语言写的程序本质上是依赖于机器的，也就是说，程序中的指令都是遵循特定的机器特性来表达的。反过来，用汇编语言写的程序不能被简单地移植到另一种机器设计上，因为这个程序必须重写才能遵循新计算机的寄存器配置和指令集。

汇编语言的另一个缺点是，尽管程序员不再必须使用数字形式对指令进行编码，但仍不得不从机器语言的角度一小步一小步地思考。这种情况很类似于房屋设计——我们毕竟还是要根据木板、钉子和砖块等来设计房屋。确实，在实际的房屋建造中，最后的确还需要一个基于这些基本元素的描述，但是如果我们根据房间、窗户和门这样的更大一些的单元来思考设计，设计过程会更容易一些。

简而言之，构建产品最终要用的基本原语不一定是在产品设计过程中使用的原语。这个设计过程应该更适合使用高级原语，每一个原语都代表了一个与产品的主要特性相关的概念。一旦设计过程结束，这些原语就能够被翻译成与实现细节相关的较低层次的概念。

根据这个理念，计算机科学家开始开发比低层次的汇编语言更易于开发软件的程序设计语言。结果就出现了第三代程序设计语言，它们与前两代程序设计语言的不同之处在于，它们的原语不仅级别更高（因为它们所能表达的指令有较大幅度的增加），而且都是**机器无关**（machine independent）的（因为它们不依赖于特定机器的特性）。最著名的早期例子是FORTRAN（全称为FORmula TRANslator）和COBOL（全称为COMmon Business-Oriented Language）；前者是为科学和工程应用开发的；后者是由美国海军为商业应用开发的。

一般来说，第三代程序设计语言的方法就是标识一组高级原语（基本上和我们在第5章中开发伪代码的思路一致），而软件要用这些原语来开发。每一个原语都要能够实现为一个机器语言可用的低级原语序列。例如，语句

```
assign TotalCost the value Price + ShippingCharge
```

表达了一个高级活动，不涉及特定机器应该如何完成这项任务，但它可以由先前讨论过的机器指令序列来实现。因此，我们的伪代码结构

```
标识符 = 表达式
```

是潜在的高级原语。

一旦确定了这组高级原语，就可以编写称作**翻译器**（translator）的程序，这个程序能够把用高级原语表示的程序翻译成机器语言程序。这种翻译器类似于第二代汇编器，只是它常常必须将一些机器指令编译为短序列来模拟一个高级原语请求的活动。因此，这些翻译程序通常也称为**编译器**（compiler）。

翻译器的一种替代方案是**解释器**（interpreter），它是作为实现第三代程序设计语言的另一种方法出现的。这些程序类似于翻译器，只是它们在翻译出指令的同时执行指令，而不是把翻译出的指令记录下来供将来使用。也就是说，解释器不产生供以后执行使用的程序的机器语言副本，而是依据程序的高级形式实际执行它。

另一个枝节问题是，我们应当注意到，发展第三代程序设计语言的任务并没有想象得那么简单。用类似于自然语言的形式来编写程序的想法是革命性的，以至于许多管理人员起初都极力反对这个想法。格雷斯·赫珀（Grace Hopper）是公认的第一个编译器的开发者，她常常讲这样一个故事，她在演示第三代语言的翻译器时，使用的是德文词汇，而不是英文词汇。关键是，程序设计语言是围绕一小组原语来构造的，而这些原语只需要稍微修改一下翻译器就可以用各种自然语言来表达。但是，她惊讶地发现，许多听众对于她在第二次世界大战前后的几年里一直在教计算机“理解”德语感到震惊。今天，我们知道理解一门自然语言涉及很多东西，远远超过对不多几条严格定义的原语做出响应。的确，**自然语言**（natural language）（如英语、德语和

拉丁语) 不同于**形式语言** (formal language) (如程序设计语言), 因为后者是由语法严格定义的 (见6.4节), 而前者是随着时间的推移逐渐演变的, 没有涉及形式语法分析。

跨平台软件

典型的应用程序必须依赖操作系统来完成它的许多任务。它可能需要窗口管理程序提供的服务与计算机用户进行交互, 可能需要利用文件管理程序从海量存储器中检索数据。遗憾的是, 不同的操作系统可能要求以不同的方式请求这些服务。这样一来, 对于需要跨网络和互联网来传输和执行的程序而言, 网络和互联网涉及各种不同的机器设计和不同的操作系统, 因此程序必须要做到与操作系统无关, 同时与机器无关。“跨平台”这个术语用于反映这种额外的独立程度。也就是说, 跨平台软件是一个可以独立于操作系统设计和机器硬件设计的软件, 因此在整个网络上都是可执行的。

6.1.2 机器无关和超越机器无关

随着第三代程序设计语言的开发, 机器无关的目标基本实现了。由于第三代语言中的语句不再与任何特定机器的特性有关, 因此它们能够在不同的机器上被轻松编译。通过使用合适的编译器, 一个用第三代语言编写的程序理论上应该能够在任何机器上使用。

但是, 现实证明并不是这么简单。在设计编译器时, 底层机器的具体特征有时会作为要翻译的语言的条件反映出来。例如, 不同的机

器处理I/O操作有不同的方法，这在历史上导致了“相同的”语言在不同的机器上有着不同的特征或方言。因此，对于一个程序而言，从一台机器移植到另一台机器通常多少都要做些修改。

伴随可移植性问题而来的是，在某些情况下，对特定语言的正确定义应该包括什么缺乏一致性的认识。为此，美国国家标准化学会（American National Standards Institute，ANSI）和国际标准化组织（International Organization for Standardization，ISO）已经采用并发布了许多流行语言的标准。除此之外，出于某种语言的某个方言的流行以及其他编译器的作者生产兼容产品的意愿，还制定了一些非正式的标准。但是，即使是高度标准化了的语言，编译器的设计者通常也会提供一些不包括在标准版本之中的特性，这些特性有时也被称为语言扩展。如果一个程序员利用这些特性，那么他生产的程序就不能兼容采用其他厂商编译器的环境。

在程序设计语言的整个历史中，由于以下两个原因，第三代语言没有真正达到机器无关这个事实并不重要。第一，它们已足够接近机器无关，软件可以从一台机器相对容易地移植到另一台机器。第二，机器无关的目标已被证明只是更高要求目标的基础。

接踵而至的每一波新的较高级的程序设计语言都构建了更强大的抽象，提高了程序员使软件设计概念化的层次，使程序员能够构造日益复杂的软件系统。几行高级程序设计语言代码就可以利用数百万行库代码、调用内置的操作系统函数或者开始与网络和数据库的复杂交

互。结果，较高级的程序设计语言使计算机程序更易于人类作者编写，更易于后来的读者理解。

反过来，这些进步导致计算机科学家们梦想实现这样的程序设计环境：它允许人们用抽象的概念与机器进行交互，而不是强迫机器把这些概念翻译成与机器兼容的格式。此外，计算机科学家希望机器能够完成许多算法发现过程，而不是只能完成算法执行。结果是程序设计语言谱系的不断扩大，以至于按照不同世代清晰地对程序设计语言进行划分很难。

基本知识点

- 高级程序设计语言为程序员提供了更多的抽象，使人们读写程序更容易。

6.1.3 程序设计范型

按代划分程序设计语言的方法是基于线性尺度（见图6-1）的，在这个线性尺度上，语言的定位是由这个语言的使用者不受机器语言世界约束的程度，以及允许从解决问题的角度来进行思考的程度决定的。实际上，程序设计语言并不是严格遵照这种划分方式发展的，而是沿着不同的路径发展成了已经浮出水面并受追捧的程序设计过程 [称为**程序设计范型**或**编程范型**（programming paradigm）] 的替代方法。因此，图6-2所示的多轨迹图能更好地描述程序设计语言的发展历程，该图显示了来源于不同范型的不同路径的出现和发展。具体地

说，这幅图展示了4条路径，分别代表函数式范型、面向对象范型、命令式范型和说明性范型，图中通过与其他语言相对位置的关系，指出了与每种范型相关的各种语言的诞生时间。（但是这并不意味着一种语言必然是从一种早期语言中演化而来的。）

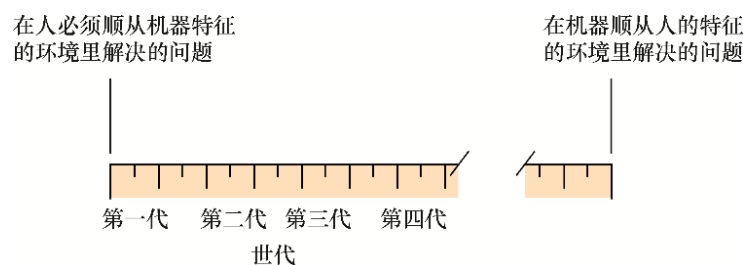


图6-1 程序设计语言的世代

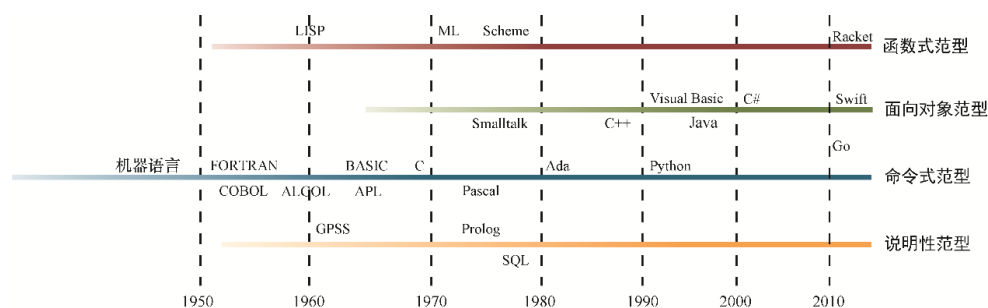


图6-2 程序设计范型的演变

我们应当注意到，尽管图6-2中标识的范型称为[程序设计范型](#)，但是这些替代方案的分支已经超出了程序设计过程的范畴。它们基本上代表了构建问题解决方案的不同方法，并且因此影响着整个软件开发过程。在这种意义上，[程序设计范型](#)这个术语有些使用不当，一个更现实的术语应该是[软件开发范型](#)（software development paradigm）。

在每种范型中，可以存在许多不同的语言，它们提供从非常低级的语言到非常高级的语言的不同层次的抽象。我们已经看到，自然语

言和伪代码都能足够耐心地充分表达算法，但文本和可视程序设计语言的目的是足够精确地显式描述计算机处理器自动执行的算法。有几个导致高级程序设计语言激增的因素。在各种人类问题求解的广阔领域中，已经开发出许多专用程序设计语言来更好地支持特定类型的算法。虽然几乎所有的程序设计语言都能够表达任何算法，但为支持图形用户界面构建而设计的语言，必然会提供来自支持统计分析的语言或嵌入式传感器反馈控制系统的不同算法快捷方式。从正确范型中选择一种适合特定问题领域（或开发团队特定问题求解风格）的程序设计语言会在软件包的清晰度和可维护性方面产生实质差异。

基本知识点

- 不同的程序设计语言提供不同层次的抽象。
- 用于算法的语言包括自然语言、伪代码，以及可视和文本程序设计语言。
- 不同的语言更适合表达不同的算法。
- 一些程序设计语言是为特定领域设计的，更适合表达那些领域中的算法。
- 用于表达算法的语言可能会影响清晰度或可读性之类的特性，但不会影响算法解决方案是否存在。
- 几乎所有的程序设计语言都能够表达任何算法。

- 在用语言表达算法时，清晰度和可读性是重要的考虑因素。

命令式范型（imperative paradigm），也叫**过程范型**（procedural paradigm），代表了程序设计过程的传统方法。命令式范型就是Python、第5章中的伪代码以及第2章中讨论的机器语言所基于的范型。顾名思义，命令式范型定义的程序设计过程是开发一个命令序列，遵照这个序列，对数据进行操作以产生所期望的结果。因此，命令式范型告诉我们，要处理程序设计过程，首先要找到解决手头问题的算法，然后将这个算法表示为一个命令序列。

与命令式范型相对的是**说明性范型**（declarative paradigm），它要求程序员描述的是要解决的问题，而不是要遵循的算法。更准确地说，说明性程序设计系统是应用预先设定的解决问题的通用算法来解决面临的问题的。在这种环境下，程序员的任务变成了开发问题的准确陈述，而不是描述一个解决问题的算法。

在开发基于说明性范型的程序设计系统时，一个主要障碍是需要一个潜在的解决问题的算法。正因为这样，早期的说明性程序设计语言本质上倾向于特定的用途，专为在特殊应用中的使用而设计。例如，许多年以来，说明性方法一直用于模拟系统（政治的、经济的、环境的等）来判定假设或获得预测。在这些情况下，潜在的算法本质上是通过重复计算参数的值（国内生产总值、贸易赤字等）来模拟时间推移的过程，其中所用的参数都基于以前计算得到的值。于是，用于这类模拟的说明性语言需要首先实现一个执行该重复函数的算法。然后，使用这个系统的程序员唯一的任务就是描述要模拟的情况。按

照这种方法，天气预报员不必开发一个预报天气的算法，只需要描述当前的天气情况，允许底层模拟算法产生未来几天的天气预报。

人们发现，数学里的形式逻辑学科提供了一种简单的、适用于通用说明性程序设计系统的问题求解算法，这个发现极大地促进了说明性范型的发展。结果是人们更加关注说明性范型，并且出现了**逻辑程序设计**（logic programming），这是6.7节要讨论的主题。

另一种程序设计范型是**函数式范型**（functional paradigm）。在这种范型下，程序被看作接受输入和产生输出的实体。数学家将这样的实体称为函数，这就是这种范型被称为函数式范型的原因。在函数式范型下，程序是通过连接较小的预定义程序单元（预定义函数）来构建的，其中每一个程序单元的输出都可以用作另一个程序单元的输入，通过这种方式可以获得所期望的整体上的输入—输出关系。简而言之，这种函数式范型下的程序设计过程就是把函数构造成较简单的函数的嵌套联合体。

举一个例子，图6-3说明了如何由两个较简单的函数构成一个计算支票簿余额的函数。其中一个称为Find_sum，它接收一些值作为输入，产生这些值的和作为输出；另一个称为Find_diff，它接收两个输入值，计算它们的差。使用LISP程序设计语言（一个著名的函数式程序设计语言）时，图6-3所示的结构可以用下列表达式表示：

```
(Find_diff (Find_sum Old_balance Credits) (Find_sum Debits))
```

该表达式的这个嵌套结构（如括号中指定的）反映了这样一个事实：函数Find_diff的输入是由Find_sum的两次应用产生的。Find_sum的第一次应用是产生所有Credits加Old_balance的结果，第二次应用是计算所有Debits的总和。然后，函数Find_diff使用这两个结果计算新的支票余额。

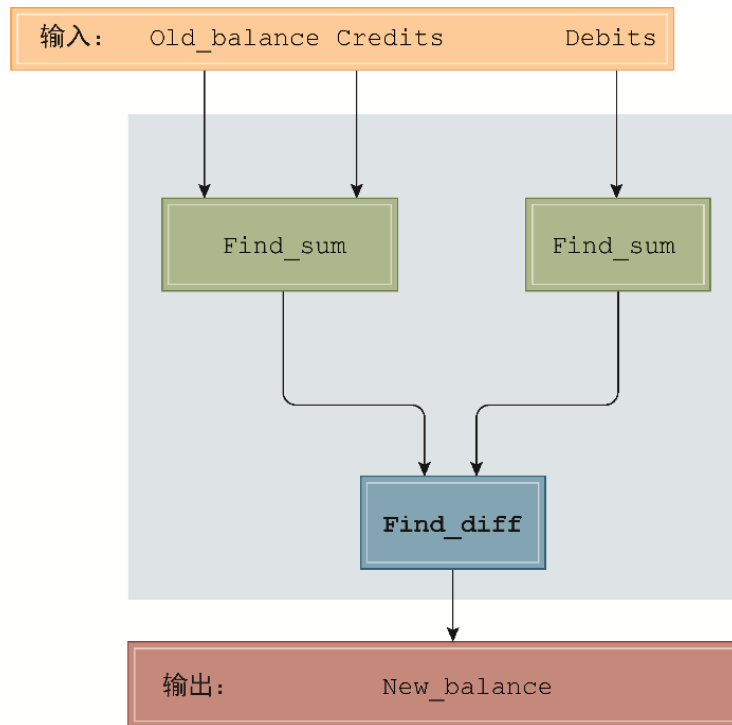


图6-3 用较简单的函数构造支票簿余额计算函数

为了更全面地理解函数式范型与命令式范型的区别，我们把求支票簿余额的函数式程序同下面遵循命令式范型的伪代码程序进行比较：

```
Total_credits = 所有Credits的总和
Temp_balance = Old_balance + Total_credits
Total_debits = 所有Debits的总和
Balance = Temp_balance - Total_debits
```


注意，这个命令式程序由多条语句组成，每条语句都请求执行计算，并请求把这个结果存储起来供以后使用。与命令式程序不同，函数式程序由单条语句组成，程序中的每个计算结果都会立即传送给下一个函数式程序。从某种意义上说，命令式程序可以看作若干工厂的集合，这些工厂中每家工厂都把原材料生产成产品，并把这些产品存放在仓库里；然后，产品从这些仓库被装运到其他需要这些产品的工厂。虽然函数式程序也可以看作若干协调工作的工厂的集合，但是这些工厂中每家工厂仅生产其他工厂订购的产品，然后立刻把这些产品运送到目的地而不需要中间存储。这种效率也是函数式范型的支持者宣称的优点之一。

另一种程序设计范型（当今软件开发领域中最著名的一个）是**面向对象范型**（object-oriented paradigm），它是与称为**面向对象程序设计**（object-oriented programming，OOP）的程序设计过程相关联的。遵照该范型设计的软件系统被看作称为**对象**（object）的单元的集合，其中每个对象都能够执行与自己直接相关的动作以及其他对象请求的动作。总之，这些对象相互作用共同解决手头的问题。

再举一个面向对象方法的例子。考虑一个开发图形用户界面的任务。在面向对象环境中，屏幕上的图标将作为对象来实现。每个对象包含一组函数，在面向对象环境中称为**方法**（method），用于描述对象是如何响应各种事件的发生的，如被鼠标单击选中或者是被鼠标在屏幕上拖动等。因此，整个系统将被构建为对象的集合，每个对象都知道如何响应与之有关的事件。

为了比较命令式范型与面向对象范型，这里考虑一个涉及名字列表的程序。在传统的命令式范型中，这个列表仅是一个数据的集合，任何访问这个列表的程序都必须包含执行所需操作的算法。然而，在面向对象方法中，这个列表将被构建成由列表和操作这个列表的方法的集合组成的对象。（这可能包括向列表中插入新条目的函数、从列表中删除条目的函数、检测列表是否为空的函数，以及对列表排序的函数。）因此，另外一个需要操作这个列表的程序单元不再包含执行相关任务的算法，而是要利用这个对象中提供的函数。从某种意义上说，程序单元要求列表自己把自己排好序，而不是像在命令式范型中那样由程序单元对列表排序。

尽管我们将要在6.5节更详细地讨论面向对象范型，但面向对象范型在当今软件开发领域的重要性要求我们在这里引入类的概念。为此，我们回顾一下，一个对象包含数据（如名字列表）和用于完成活动（如在列表中插入新的名字）的方法的集合。这些特性必须用所写的程序中的语句来描述。对象的属性的这个描述称为**类**（class）。一旦一个类被构造好了，就可以随时将它用在需要具有这些特征的对象上。因此，几个对象可以基于同一个类（即由同一个类构建）。就像同卵双胞胎一样，由于这些对象产生于相同的模板（相同的类），因此它们虽然具有相同的特征但却是不同的实体。[基于特定类的对象可以说是那个类的**实例**（instance）。]

因为对象是明确定义的单元，在可复用的类中，其描述是孤立的，所以面向对象范型受到了欢迎。事实上，面向对象程序设计的支

持者认为，面向对象范型为软件开发的“构建块”方法提供了一个自然的环境。他们设想了预定义的类的软件库，从这个库可以构造新的软件系统，就像用现成的组件构建许多传统产品一样。构建和扩展这样的库是一个持续的过程，我们将在第7章学习。

最后，我们应该注意，对象内的方法实质上是一些小的命令式程序单元。这就意味着，大多数基于面向对象范型的程序设计语言都包含许多可以在命令式语言中找到的特性。例如，流行的面向对象语言C++就是通过在C语言这个命令式语言中添加一些面向对象的特性开发出来的。此外，因为Java和C#是C++的衍生出来的，所以它们都继承了命令式语言的精髓。在6.2节和6.3节，我们将探究命令式语言的许多特性，同时，我们还会讨论贯穿于现今绝大多数面向对象软件里的概念。后面在6.5节中，我们将学习面向对象范型独有的特性。

6.1节问题与练习

1. 在什么意义上，用第三代语言编写的程序是机器无关的？在什么意义上，它们还是依赖于机器的？
 2. 汇编器和编译器的区别是什么？
 3. 我们可以将命令式程序设计范型概括为：它强调描述一个可以引出手头问题解决方案的过程。请给出说明性范型、函数式范型和面向对象范型的类似概括。
 4. 在什么意义上，第三代程序设计语言比前两代程序设计语言更高级？
-

6.2 传统的程序设计概念

高级程序设计语言包括从简单（常量、文字和变量）到中等复杂（语句、表达式和控制结构）到深奥（过程单元、模块和库）的多种抽象。

在本节中，我们将研究命令式程序设计语言和面向对象程序设计语言中的一些概念。为此，我们会从Ada、C、C++、C#、FORTRAN和Java等程序设计语言中取出一些例子。我们的目标不是要纠缠任何特定语言的细节，而是要说明常见的语言特性是如何出现在各种实际程序设计语言中的。因此，我们选择的程序设计语言集合都是具有代表性的。C是第三代命令式语言。C++是通过对C语言进行扩展得到的面向对象程序设计语言。Java和C#是从C++衍生的面向对象程序设计语言。（Java是Sun Microsystems公司开发的，而C#是微软公司的产品；Sun Microsystems公司后来被甲骨文公司收购了。）FORTRAN和Ada最初是作为第三代命令式语言设计的，尽管它们的新版本已扩展到包含大多数的面向对象范型。附录D简短地介绍了每种语言的背景。

尽管我们的示例中涉及了C++、Java和C#之类的面向对象语言，但是本节中将把程序想象成是用命令式范型编写的，因为面向对象程序（如描述对象应该怎样响应外部刺激的函数）中的许多单元本质上就是简短的命令式程序。后面在6.5节，我们将主要讨论面向对象范型独有的特性。

脚本语言

命令式程序设计语言的一个子集是称为**脚本语言**（scripting language）的语言集合。这些语言通常用来执行管理任务，而不是开发复杂的程序。这种任务的表述称为**脚本**（script），它解释了术语“脚本语言”。例如，计算机系统的管理员可能会写一个脚本来描述一系列应该在每晚执行的需要保持记录的活动，而PC的用户可能会写一个脚本来指导一系列程序的执行，以从数码相机中读取照片，通过日期对照片建立索引，以及在档案存储系统中存储照片的副本。脚本语言的起源可以追溯到20世纪60年代用于指导操作系统调度批处理作业（见3.1节）的作业控制语言。甚至在今天，仍有许多人认为脚本语言是指导其他程序执行的语言，这是对当前脚本语言的相当有限限制性的观点。脚本语言的例子包括Perl和PHP，它们在控制服务器端Web应用（见4.3节）和VBScript中很流行，VBScript是Visual Basic的一种方言，由微软公司开发并用于Windows的特定环境下。

通常，程序由一组语句组成，这些语句一般可以分成3类：声明语句、命令语句和注释。**声明语句**（declarative statement）定义程序中后面要用的自定义术语，如用来引用数据项的名字；**命令语句**（imperative statement）描述底层算法中的步骤；**注释**（comment）通过比较人性化的形式来解释程序中的一些难懂的特性，从而提高程序的可读性。通常，命令式程序（或者面向对象程序中的命令式程序单元）可以被认为具有图6-4描述的结构。它以描述程序要操作的数据的一组声明语句开始，紧接其后的是描述要执行的算法的命令语句。现在，很多语言都允许声明语句和命令语句自由交织，但其概念上的区

别依然存在。注释语句是很分散的，仅出现在需要对程序进行解释的地方。

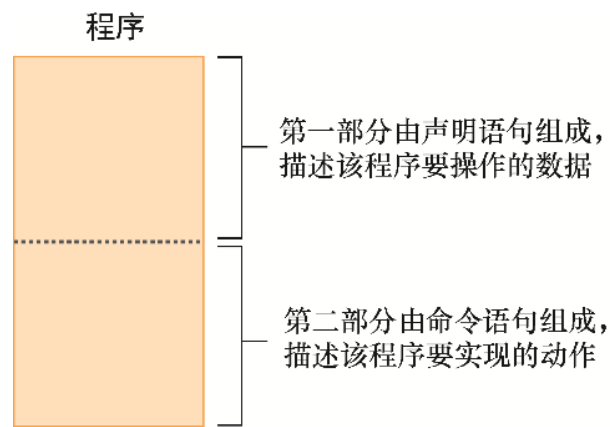


图6-4 一个典型的命令式程序
或程序单元的结构

基本知识点

- 软件是用多层次抽象（如常量、表达式、语句、过程和库）开发的。

根据上面的介绍，我们按照在程序中可能遇到语句类别的顺序考虑这些语句类别，从与声明语句有关的概念开始研究程序设计概念。

6.2.1 变量和数据类型

正如在1.8节中提到的那样，高级程序设计语言允许使用描述性的名字指向主存储器中的位置，而不必再使用数字地址，这样的名字称为变量（variable）。之所以这样取名是因为只要改变存储在这个位置

里的值，与该名字相关联的值就会随着程序的执行而改变。与Python不同，本章中我们的示例语言要求程序在使用变量之前先用声明语句标识变量。同时，这些声明语句也会要求程序员描述将要存储在与该变量相关联的存储器位置中的数据的类型。

基本知识点

- 程序指令可能涉及初始化、更新、读取和写入的变量。

这种类型称为**数据类型**（data type），它决定了数据项的编码方式以及可以在该数据上执行的操作。例如，**整型**（integer）是指由整数组成的数字数据，可能以二进制补码记数法存储。可以在整型数据上进行的运算包括传统的算术运算和相对大小的比较，如判断一个值是否比另一个值大。**浮点型**（float）类型，有时称为**实型**（real），是指可能包含整数以外的值的数字数据，可能以浮点记数法存储。可以在浮点型数据上进行的操作类似于那些可以在整型数据上进行的操作。但回想可知，两个浮点型项相加所需的操作与两个整型项相加所需的不一样。

假设我们想要在一个程序中使用变量WeightLimit来指向一个主存储器区域，这个区域包含一个用二进制补码记数法编码的数值。在C、C++、Java和C#语言中，我们可以通过在程序的开头插入下面的语句来声明我们的意图：

```
int WeightLimit;
```

这条语句的意思是：“名字WeightLimit将要在后面的程序用到，它指向一个存储器区域，该存储器区域中包含一个用二进制补码记数法存储的值。”同一类型的多个变量通常可以在同一个声明语句中声明。例如，语句

```
int Height, Width;
```

声明两个整型变量Height和Width。此外，大多数语言允许在声明变量时，为变量赋一个初始值。因此，语句

```
int WeightLimit = 100;
```

不仅声明了一个整型变量WeightLimit，还会为这个变量赋一个初始值100。与之相比，像Python这样的动态类型语言允许变量在不先声明类型的情况下被赋值，对这种变量的正确类型检查会在后面对它们执行操作时进行。

其他常见数据类型包括字符型和布尔型。字符型是指由符号组成的数据，可能使用ASCII或者Unicode进行编码存储。可以在这种数据上进行的操作包括比较，如按照字母顺序判断一个符号是否在另一个符号的前面，测试查看一个符号串是否在另一个符号串中，以及将一个符号串连接在另一个符号串的尾部从而形成一个更长的符号串。在C、C++、C#和Java语言中，可以用下面的语句来声明字符型变量Letter和Digit：

```
char Letter, Digit;
```


布尔型 (Boolean) 是指值仅为真或假的数据项。在布尔型数据上执行的操作包括判断当前值是真还是假。例如，如果将变量 LimitExceeded 声明为布尔型，那么下面这种形式的语句就是合理的：

```
if (LimitExceeded) then (...) else (...)
```

作为原语包括在程序设计语言中的数据类型（如用于整数的int和用于字符的char）称为**基本数据类型**（primitive data type）。我们所知道的整型、浮点型、字符型、布尔型都是常见的原语，其他数据类型（包括图像、音频、视频和超文本）目前还没有成为广泛使用的原语。但是，像GIF、JPEG和HTML这样的类型可能马上就要像整型和浮点型一样常见了。在6.5节和8.4节，我们将学习到面向对象范型是如何使程序员能够将可用数据类型库扩展到程序设计语言提供的原语类型之外的。事实上，这种能力是面向对象范型的一个著名特征。

总之，下面程序段是用C语言及其派生语言C++、C#和Java表达的，它将变量Length和Width声明为float类型，变量Price、Tax和Total声明为整型，变量Symbol声明为字符型：

```
float Length, Width;  
int    Price, Tax, Total;  
char   Symbol;
```

在6.4节中，我们会看到翻译器是如何利用从这些声明语句中收集到的知识，把一个程序从高级语言翻译为机器语言的。这里，我们要注意的是，这些信息可以用来识别错误。例如，对于两个早先声明为

布尔型的变量，如果翻译器发现一个要求对它们做加法的语句，那它很可能认为这个语句是错误的，并把这个发现报告给用户。

6.2.2 数据结构

除了数据类型，程序中的变量通常与**数据结构**（data structure）有关，数据结构是数据在概念上的形态或排列。例如，文本通常被看作一长串字符，而销售记录可能被想象为数值的矩形表格，其每一行代表某一位员工的销售额，每一列代表某一天的销售额。

一种常用的数据结构是**数组**（array），即由相同类型的元素组成的块，如一维列表、一个由行和列组成的二维表或具有更高维数的表。为了在程序中建立这样的数组，许多程序设计语言要求声明语句在声明数组名字的同时也指定数组每个维度的长度。例如，图6-5显示了由C语言语句

```
int Scores[2][9];
```

声明的概念结构，它的意思是：“变量Scores将要在后面的程序单元中使用到，指向一个2行9列的二维整型数组。”在FORTRAN中同样的声明语句要写成

```
INTEGER Scores(2,9)
```

数组一旦声明，就能够通过它的名字在程序中的其他地方引用它，或者通过一个称作**索引**（index）的整数值来标识这些数组的组成

元素，索引指定所需的行和列等信息。但是，在不同的语言中索引的范围是不同的。例如，在C语言（及其派生语言C++、Java和C#）中，索引是从0开始，也就是说对名为Scores的数组（上文已声明）的第2行第4列的条目将用Scores[1][3]来引用，而第1行第1列的条目将用Scores[0][0]来引用。与之相比，在FORTRAN程序中索引是从1开始的，所以第2行第4列的条目将用Scores[2][4]来引用（可再参考图6-5）。

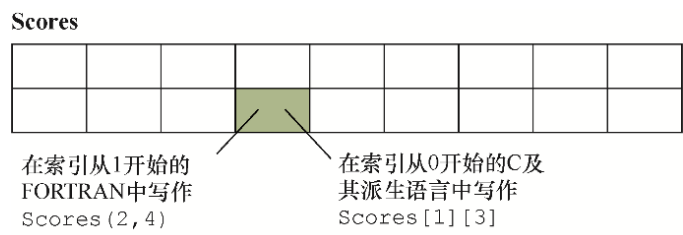


图6-5 拥有2行9列的二维数组

与所有数据项都是同一类型的数组相比，**聚合类型**（aggregate type），也称**结构**（structure）、**记录**（record）或者有时还称**异构数组**（heterogeneous array），是一个其中不同元素可以有不同类型的数
据块。例如，一个员工的数据块可能由一个名为Name的字符型条目、
一个名为Age的整型条目和一个名为SkillRating的实型条目组成。
这样的聚合类型用C语言声明如下：

```
struct { char Name[25];  
        int  Age;  
        float SkillRating;  
    } Employee;
```

上述声明的意思是：变量Employee指向一个结构（structure，简称为struct），这个结构有3个构成元素，它们分别是Name（包含25个字符的字符串）、Age和SkillRating（见图6-6）。一旦声明了这样一个聚合体，程序员就可以使用这个结构的名称（Employee）来引用整个聚合体，或者用结构的名称跟一个句点和字段名（如Employee.Age）来引用聚合体中的单个**字段**（field）。

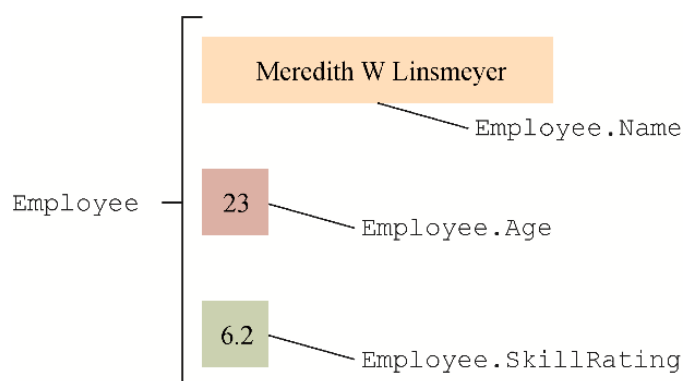


图6-6 结构Employee的概念布局

在第8章中，我们将会看到诸如数组这样的概念结构是如何在计算机内部真正实现的。特别是，我们将了解到，数组里的数据可能散布在主存储器或者海量存储器上的广大区域内，这就是我们将数据结构表达成数据在**概念上的**形态或者排列的原因。当然，计算机存储系统中的实际排列也许和它在概念上的排列完全不同。

6.2.3 常量和字面量

有时，程序中会用到预先确定的固定值。例如，一个管理机场附近空中交通的程序也许要无数次引用一些关于机场的海拔高度的数

据。当编写这样一个程序的时候，在每次需要这个数据时我们都可以从字面上包含这个值，如645米。一个值的这样一种显式出现称为**字面量**（literal）。字面量的使用导致了

```
EffectiveAlt = Altimeter + 645
```

这样的程序语句的出现，其中EffectiveAlt和Altimeter是假定的变量，而645是一个字面量。因此，这条语句的意思是将赋给变量Altimeter的值加上645的结果赋给变量EffectiveAlt。

回忆第1章，通过算术运算符和逻辑运算符组合的变量、文字和常量值称为**表达式**（expression）。在上面的示例中，Altimeter + 645就是这样的表达式。

在大多数程序设计语言中，由文本组成的字面量都是用单引号或者双引号来划界的，以便与其他程序部分相区分。例如，语句

```
LastName = "Smith"
```

可以用来把文本“Smith”赋给变量LastName，而语句

```
LastName = Smith
```

则用来把变量Smith的值赋给变量LastName。

通常，使用字面量并不是好的编程实践，因为字面量会掩盖包含字面量的语句的含义。例如，当一个读者读到语句

```
EffectiveAlt = Altimeter + 645
```

时，他如何知道645这个值代表的是什么呢？此外，字面量的使用会使在必要时修改程序的工作变复杂。如果将空中管制程序移植到另一个机场，那么所有对机场海拔高度的引用都要修改。如果每一处对海拔高度的引用都使用了字面量645，那么要在整个程序中定位每一个这样的引用并且加以修改。如果字面量645除了出现在对海拔高度的引用中，还出现在对数量的引用中，问题就更复杂了。我们如何知道哪个645要修改，哪个645要保持不变呢？

为了解决这些问题，程序设计语言允许给特定的不会改变的值分配一个描述性的名字。这种名字称为**常量**（constant）。例如，在C++和C#语言中，声明语句

```
const int AirportAlt = 645;
```

将标识符AirportAlt与固定值645（被认为是整型值）关联起来。在Java语言中，类似的概念表达为

```
final int AirportAlt = 645;
```

在Swift中表达为

```
let AirportAlt = 645
```

根据这些声明，描述性的名字AirportAlt能够用于字面量645出现的场合。若将这种常量用于伪代码中，语句

```
EffectiveAlt = Altimeter + 645
```

可以改写成

```
EffectiveAlt = Altimeter + AirportAlt
```

这条语句能够更好地表达语句的含义。此外，如果用这样的常量来代替字面量，当程序要移植到另一个海拔为267米的机场时，只修改这个定义常量的声明语句就可以将对机场海拔高度的所有引用改为新值。

6.2.4 赋值语句

一旦声明了用于程序的专门术语（如变量和常数），程序员就可以开始描述涉及的算法了，这要靠命令语句来实现。最基本的命令语句是**赋值语句**（assignment statement），它请求将一个表达式的值赋给一个变量（或者更确切地说，将一个表达式的值存储在该变量标识的存储区域中）。这种语句采用的语法形式通常为：一个变量，后跟一个代表赋值运算的符号，最后跟一个赋值表达式。这种语句的语义就是对表达式进行求值，把得到的结果存储为变量的值。例如，在C、C++、C#和Java语言中，语句

```
Z = X + Y;
```

是将X和Y相加的和赋给变量Z。行尾的分号在许多命令式语言中用来分隔语句，是这些语言的赋值语句与Python赋值语句在语法上的唯一区别。在一些其他语言（如Ada语言）中，等价的语句可以写成

```
Z := X + Y;
```

注意，这些语句仅在赋值运算符的语法上有所不同，在C、C++、C#和Java语言中，仅使用一个等号来表示，但在Ada语言中，要用一个冒号加等号的形式来表示。APL语言中的赋值运算符可能更好一些，APL语言是由肯尼思·艾弗森（Kenneth E. Iverson）在1962年设计的。（APL是A Programming Language的缩写。）它使用一个箭头来表示赋值。因此，前面的赋值在APL语言中可以表示为

$Z \leftarrow X + Y$

不幸的是，一直以来大多数键盘上都没有“ $\leftarrow$ ”符号。

赋值语句的许多功能都与语句右边的表达式的作用域关系密切。一般而言，任何代数表达式都可以用在赋值表达式中，包括通常用符号+、-、*和/分别代表算术运算中的加法、减法、乘法和除法。一些语言（包括Ada和Python，但不包括C或Java）用**组合来表示幂运算，因此表达式

$X ** 2$

表示 x^2 。但是，各种语言对这种表达式的解释方式是不一样的。例如，对于表达式 $2 * 4 + 6 / 2$ ，如果是从右向左求值，得到的值是14，而如果从左向右求值，得到的值就是7。这种不确定性通常是通过[运算符优先级](#)（operator precedence）规则来解决的，这意味着某些运算的优先级比其他运算的高。传统的代数规则规定乘法和除法的优先级比加法和减法的高，也就是说，乘法和除法要在加法和减法之前

执行。根据这个约定，前面的表达式的结果应该是11。在大多数语言中，括号的优先级比所有运算符的优先级都高。因此， $2 * (4 + 6) / 2$ 的结果应该是10。

许多程序设计语言允许使用一个符号表示多种运算。在这些情况下，符号的含义只能根据操作数的数据类型来确定。例如，当操作数是数值时，符号+传统上表示加法；但在某些语言里，如Java、Python和Swift，当其操作数是字符串时，符号+表示拼接。也就是说，表达式

```
'abra' + 'cadabra'
```

的结果是abracadabra。一个运算符的这种多种用法称为**重载**（overloading）。许多程序设计语言都提供了几个常见运算符的内置重载，而另外一些程序设计语言（如Ada、C++、C#和Swift）可能允许程序员定义额外的重载的含义，甚至添加额外的运算符。

6.2.5 控制语句

控制语句（control statement）可以改变程序的执行顺序。在所有的程序设计结构中，某些控制语句受到了极大的关注并且引发了很大的争议。罪魁祸首是最简单的控制语句——goto语句。它提供了一种把执行顺序转向另一个位置的手段，这个位置是用名字或数字标记的。因此，它不过是机器级的JUMP指令的直接应用。但在高级程序设

计语言中，这个特点意味着，它允许程序员写出像下面这样可读性很差的程序：

```
    goto 40
20 Evade()
    goto 70
40 if (KryptoniteLevel < LethalDose) then goto 60
    goto 20
60 RescueDamsel()
70 ...
```

而仅使用一条下面这样的语句就可以完成相同的工作：

```
if (KryptoniteLevel < LethalDose):
    RescueDamsel()
else:
    Evade()
```

为了避免产生这样的复杂性，现代程序设计语言设计出来的控制语句都允许在一个词法结构中表达整个分支模式。选择哪些控制语句放到语言中是一种设计决策。目标是要提供一种语言，它不仅允许算法用可读的形式表达，还能帮助程序员获得这种可读性。为了实现这个目标，可以限制使用会导致不良程序设计的特性，同时鼓励使用设计更好的特性。结果是产生了称为**结构化程序设计**（structured programming）的实践，它包含了有条理的设计方法学，以及对控制语句的合理使用。这个方法的中心思想是要设计一个既容易理解又容易证明其满足需求规格说明的程序。

程序设计语言文化

和自然语言一样，不同程序设计语言的使用者往往会开创出不同的文化，并且经常站在各自的观点上争论语言之间的优劣。有时候这

种差异很显著，如涉及不同的程序设计范型时。其他时候，这种差异却很微妙。例如，尽管Pascal语言对过程和函数进行了区分（见6.3节），但是C和Python程序员把这两者都称作函数。这是因为Python程序声明过程的方式与声明函数的方式是一样的，只是没有定义返回值。一个类似的例子就是，C++程序员把包含在对象内的过程称为成员函数，而通用术语称之为方法。造成这种差异的原因在于，C++是在C语言的基础上扩展而来的。另外一个文化差异是Ada程序中的保留字通常用大写字母或粗体排版，而这种习惯对于C、C++、FORTRAN或Java的使用者来说却不常见。

尽管本书大部分章节都是用Python做的范例，但是每一个具体的例子都采用了与所涉及的程序设计语言相适应的风格。当你遇见这些例子的时候，一定要记住的是，这些例子代表的实际上是程序设计语言中的通用思想，而不是传授某种程序设计语言细节的一种手段。不要只见树木不见森林。

在第5章中我们遇到过用if-else和while语句表示的两种常见的分支结构。这些结构几乎出现在所有的命令式语言、函数式语言或面向对象语言中。更准确地说，Python语句

```
if (condition):  
    statementA  
else:  
    statementB
```

和

```
while (condition):  
    body
```

在C、C++、C#和Java中将被写成

```
if (condition) statementA; else statementB;
```

和

```
while (condition) { body }
```

注意，这些语句在这4种语言中是相同的，这是因为C++、C#和Java都是命令式语言C的面向对象的扩展。与之相对应，在Ada语言中相应的语句将被写成

```
IF condition THEN  
    statementA;  
ELSE  
    statementB;  
END IF;
```

和

```
WHILE condition LOOP  
    body  
END LOOP;
```

另一个常见的分支结构常用switch或case语句表示。它提供了依据赋给指定变量的值，在多个选项中选择一个语句序列的方法。例如，在C、C++、C#和Java语言中，语句

```
switch (variable) {  
    case 'A': statementA; break;  
    case 'B': statementB; break;  
    case 'C': statementC; break;  
    default: statementD }  
}
```

分别根据variable的当前值是A、B还是C来请求执行statementA、statementB或statementC；如果variable的值是其他值，那将执行statementD。等价的Swift结构非常相似，但是没有分号或break语句。在Ada语言中，相同的结构将被写成

```
CASE variable IS
  WHEN 'A'=> statementA;
  WHEN 'B'=> statementB;
  WHEN 'C'=> statementC;
  WHEN OTHERS=> statementD;
END CASE;
```

另一个称为for循环的常见控制结构及其在C++、C#和Java语言中的表示如图6-7所示。这个结构不同于第5章中介绍的Python的for结构。对于迭代遍历数据列表的循环，C语言家族的for结构的初始化、修改和终止部分都不是隐式设置的，而是显式地包含在一条语句里的。当循环体对于特定范围内的每个值都要执行一次时，这样的语句就很方便。特别地，图6-7中的语句指示循环体被重复执行——第一次Count的值为1，第二次Count的值为2，第三次Count的值为3。包含这两种for结构的两类程序设计语言在语法上可能会有所区别，如foreach或for...in。

根据所引用的例子，我们可以得到这样一个结论：所有命令式程序设计语言和面向对象程序设计语言中都有常见的分支结构，只是略有不同。从计算机科学的理论中我们可以了解到一个有些令人惊讶的结果：只需要这些结构中的少数几个就足以保证程序设计语言能够提供表达任何有算法解决方案的问题的解决方案的方法。我们将会在第

12章中研究这个问题。现在，我们只是指出，学习程序设计语言不是一个无休止的学习各种控制语句的过程，在当今程序设计语言中可以找到的大多数控制结构本质上都是本书介绍的这些结构的变体。

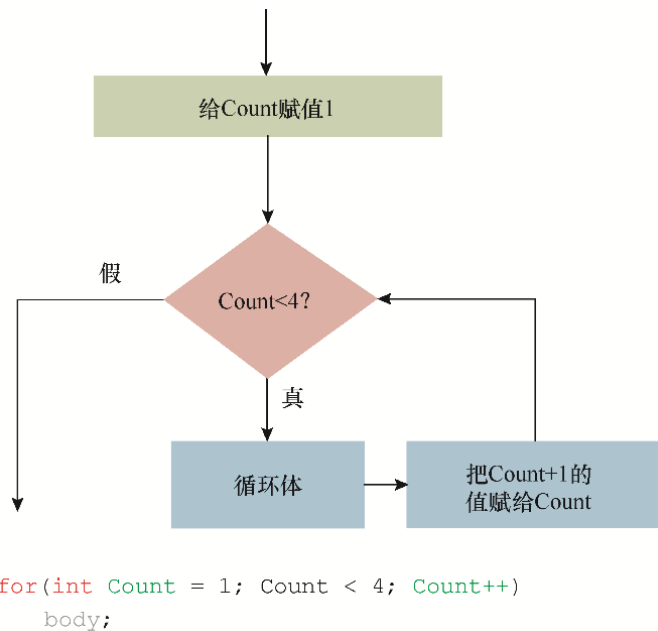


图6-7 C++、C#和Java语言中的for循环结构及其表示

Visual Basic

Visual Basic是面向对象程序设计语言，是微软公司开发的工具，微软Windows操作系统的用户可以通过它开发他们自己的GUI应用程序。实际上，Visual Basic不止是一种语言，它还是一个完整的软件开发包，程序员可以利用它的预定义组件（如按钮、复选框、文本框、滚动条等）构建应用程序，并且可以通过描述组件如何响应不同事件来定制这些组件。例如，对于按钮，程序员可以描述单击按钮时应该发生什么。在第7章中，我们将会学到，这种通过预定义组件构建软件的策略是当今软件开发技术的发展趋势。

在20世纪90年代，Windows操作系统的日益流行与Visual Basic开发包的便利性使得Visual Basic成为被广泛使用的程序设计语言。对于具有图形用户界面的软件快速原型设计来说，Visual Basic的后继者，如VB.NET，仍然是流行的语言选择。

Swift

Swift是苹果公司于2014年推出的多泛型程序设计语言。它结合了命令式程序设计语言、面向对象程序设计语言和函数式程序设计语言的特征。虽然Swift库和开发环境在Linux和苹果公司的操作系统上都能用，但是它们经过优化，能利用苹果公司的平台上已有的大量Objective C代码。在和Xcode开发环境一起使用时，Swift代码可以在虚拟机“操场”上编译和运行，这个“操场”模拟了在iOS、watchOS等苹果公司的操作系统上运行的许多智能手机和设备。许多智能手机应用现在都是用Swift开发的，Swift提供了有用的现代程序设计语言功能，如类型推断、模式匹配、自动存储管理以及其他安全功能。

6.2.6 注释

不管一种程序设计语言设计得多么好，也不管一个程序把该语言的特性应用得多么出色，当人们试图阅读和理解这个程序时，程序的附加信息通常都是有用的，或者是必须要有的。为此，程序设计语言提供了在程序中插入解释性语句的方法，这些语句称为**注释**

（comment）。翻译器会忽略这些注释语句，因此对机器来讲，有没有注释都不影响程序的执行。无论源程序有没有注释，翻译器生成的

程序的机器语言版本都是一样的，但对人来讲，这些语句提供的信息是程序的一个重要组成部分。若没有这些注释，复杂的大型程序就很容易阻碍程序员的理解。

基本知识点

- 无论单独工作还是在协作程序设计环境中工作，文档都有助于开发和维护程序。

在程序中插入注释的方法通常有两种。一种是用两个特殊标记将整个注释括起来，一个标记放在注释的开头，一个标记放在注释的末尾。另一种是标示出注释的开头，这一行标记右边的部分都属于注释。在C++、C#和Java中，我们可以同时看到这两种注释方法的应用。它们用/*和*/来包围注释，用//开始一个注释直至行末。因此，

```
/* This is a comment. */
```

和

```
// This is a comment.
```

都是有效的注释语句。

关于什么是有意义的注释简单说几句。当为了制作内部文档而要求一些初级程序员使用注释语句的时候，他们容易为

```
ApproachAngle = SlipAngle + HyperSpaceIncline;
```


这样的语句写出类似“通过把HyperSpaceIncline和SlipAngle相加计算ApproachAngle”这样的注释。这种注释是多余的，除了增加程序的长度，对程序的清晰度没一点帮助。注释的用途是解释程序，而不是重复程序。对于这条语句的更合适的注释应该是解释为什么要计算ApproachAngle（如果这一点不很明显的话）。例如，注释“ApproachAngle将会在后面计算ForceFieldJettisonVelocity时使用，此后就不再需要了”就比前面的注释更有用一些。

此外，分散在程序语句之中的注释有时会妨碍人跟踪程序流程的能力，从而使理解程序比没有注释的时候还要困难。一个好的办法是将关于某个单一程序单元的所有注释集中放在一个位置上，如放在该程序单元的开始位置。这就给程序单元的读者提供了查看注释的确切地点，也提供了可以用来描述此程序单元的目的和一般特征的地点。如果所有的程序单元都采用了这个格式，整个程序就会有一定程度的一致性——每个程序单元都包括一组解释性的语句，以及随后对该程序单元的正式介绍。程序中的这种一致性提高了程序的可读性。

基本知识点

- 关于程序组件（如段和过程）的文档有助于开发和维护程序。

基于块的程序设计

可视程序设计语言，有时也称为基于块的程序设计语言，是传统的基于文本的程序设计环境的替代方案，它允许开发人员通过操纵屏

幕上的图形块来创建程序。可以将基本语言结构（如循环、条件、函数等）表示为图形块，然后像虚拟乐高块一样将它们插在一起创建程序指令序列。

由于可视化程序设计语言的易用性，它们已成为入门计算机科学课程特别是中小学生和低年级大学生的热门选择。Alice、Scratch、StarLogo和App Inventor是教育中使用的几个广泛使用的基于块的语言的示例。

大体上，基于块的语言不亚于基于文本的程序设计语言，但是简化的教育视觉语言的流行使许多学生误以为它们只能用于基本任务。实际上，强大的块语言可用于编写嵌入式系统、操纵复杂的媒体制品以及构建仿真程序。

6.2节问题与练习

1. 为什么使用常量的程序设计风格比使用字面量的要好？
2. 声明语句和命令语句的区别是什么？
3. 列举一些常见的数据类型。
4. 给出命令式程序设计语言和面向对象程序设计语言里的一些常见控制结构。
5. 数组和聚合类型的区别是什么？

6.3 过程单元

在前面的章节中，我们已经看到了将大程序拆分成小的可管理的单元的一些好处。在本节中，我们将主要讨论函数的概念，函数是一

个命令式语言获得程序的模块化表示的主要技术。正如第5章中提到的，多年来程序设计语言用了许多术语来表述这个重要概念：子程序、子例程、过程、方法、函数，有时含义略有不同。在严格的命令式语言中，基本是用函数这个术语来表述这个概念，但在面向对象语言中，程序员们在指定对象应该如何响应外部刺激时，经常更喜欢使用方法（method）这个术语。

6.3.1 函数

从一般的意义上来说，函数（function）就是完成一个任务的一组指令，它能够用作其他程序单元的抽象工具。当请求了函数提供的服务时，程序的控制权就转移给了函数，在函数执行完之后，程序控制权又返回到原来的程序单元（见图6-8）。将控制权转移给函数的过程一般称为调用（call或者invoke）函数。我们将请求函数执行的程序单元称为调用单元。

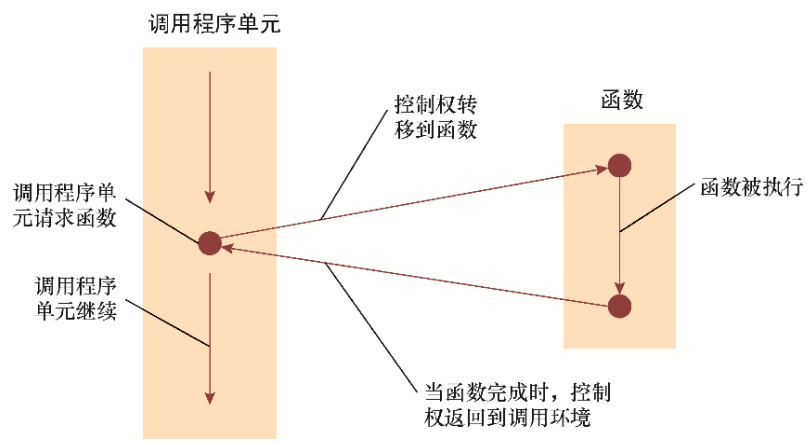


图6-8 一个函数的控制流

在第5章的Python示例中，函数通常被写成单个的程序单元，程序单元以一条称为函数**头**（header）的语句开始，其中标识了函数的名称。函数头后面是定义函数细节的语句。这些语句倾向于按照传统的命令式程序相同的方式排列，以声明语句开始（描述函数中使用的变量），接着是命令语句（描述执行函数时要进行的步骤）。

一般来说，在函数中声明的变量称为**局部变量**（local variable），意思是它只能在这个函数的内部引用。局部变量能够消除由于两个独立的函数碰巧使用同一名称的变量所产生的混淆。[一个程序中可以引用某个变量的部分称为该变量的**作用域**（scope）。因此，局部变量的作用域就是声明它的函数。作用域没有限定在程序中某个特定部分使用的变量称为**全局变量**（global variable）。大多数程序设计语言都提供了指定变量是局部变量还是全局变量的方法。]

大多数现代程序设计语言都允许通过只声明函数名来调用函数。例如，如果GetNames、SortNames和WriteNames分别是获取、排序及打印名字列表的函数的名字，那么获取、排序和打印该列表的程序就可以写成

```
GetNames()  
SortNames()  
WriteNames()
```

注意，通过为每个函数分配一个指明该函数功能的名字，这种简明扼要的形式看起来就像是反映该程序含义的命令序列。

对新程序员来说，开发新函数最初可能是具有挑战性的。虽然在大多数语言中定义新函数的语法很简单，但识别正确的抽象并将其简洁地编码成程序设计语言代码的过程需要实践。初学者可能会发现自己在编写了几个相关任务的具体示例之后才能识别出产生一个满足所有情况的函数的归纳。通过减少对具体情况细节的关注，往往更容易看到它们之间的共同点。构建好的函数的核心智力任务是抽象。

基本知识点

- 开发抽象的过程涉及去除细节和归纳功能。
- 为了归纳概念，抽象从具体示例中提取共同特征。

6.3.2 参数

函数通常会使用一些通用项编写，这些项在函数被应用的时候才能具体化。例如，第5章的图5-11中表达的就是一个通用列表而不是一个具体列表。在本书的伪代码中，这些通用项都是在函数头的括号中标识出来的。因此，图5-11中的函数头以

```
def Sort(List):
```

开始，然后使用List指向将要被排序的列表，从而进一步描述列表排序过程。如果想用这个函数来为一个婚礼宾客列表排序，我们只需按照函数中的指示，假设通用项List指向的是婚礼宾客列表。然

而，如果想排序一个会员列表，我们只需将通用项List解释成引用会员列表。

函数内部的这些通用项称作**参数**（parameter）。更准确地说，这些在函数内部使用的项称为**形参**或**形式参数**（formal parameter），当函数被调用的时候，赋给形参的精确含义称为**实参**或**实际参数**（actual parameter）。在某种意义上，形参就像是函数体上的槽口，当函数被请求的时候，实参就被塞入这个槽口中。

基本知识点

- 参数化可以归纳一个特定的解决方案。

就像在Python中一样，许多程序设计语言要求在定义函数时将形参列在函数头的括号里。再如，图6-9给出了用C语言编写的名字为ProjectPopulation的函数的定义。该函数期望在它被调用的时候给它一个具体的年增长率。在这个增长率的基础上，假设初始数量为100，这个函数计算出未来10年某个种群的数量，并且将结果存储在名为Population的全局数组中。

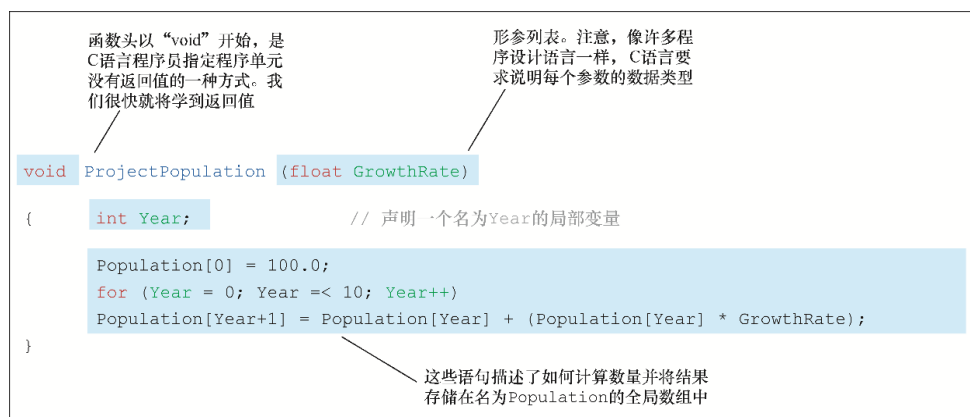


图6-9 用C语言编写的函数ProjectPopulation

许多程序设计语言在调用函数时也用括号来标识实参。也就是说，请求函数执行的语句由函数名和括号中的实参列表组成。因此，像

```
使用增长率0.03调用函数ProjectPopulation()
```

这样的伪代码语句可以用C语言语句

```
ProjectPopulation(0.03);
```

表示，它是用增长率0.03来调用图6-9中的函数ProjectPopulation。这个语法与Python中的语法是相同的，只是尾部没有分号。

当函数不只包含一个参数的时候，实参要与函数头中所列的形参一一对应，即第一个实参对应第一个形参，依次类推。然后，实参的值就可以有效地传递给它们对应的那个形参，从而函数得以执行。

为了强调这一点，假设函数PrintCheck使用这样的函数头来定义：

```
def PrintCheck(Payee, Amount):
```

其中，Payee和Amount是函数的形参，分别指向收支票的人以及支票的数额。那么，用语句

```
PrintCheck('John Doe', 150)
```

调用该函数，将会使形参Payee关联实参'John Doe'，形参Amount关联实参150，从而使函数得以执行。但是，使用语句

```
PrintCheck(150, "John Doe")
```

调用函数会将值150赋给形参Payee，将'John Doe'赋给形参Amount，而这会导致错误的结果。

对于形参和实参之间的数据传递，不同的程序设计语言有不同的处理方法。在一些语言中，对于实参所表示的数据会产生一个副本并传给函数。使用这种方法，函数对数据所做的任何修改仅仅是对副本的修改——调用程序单元中的数据并没有被修改，我们称这种参数传递方式为**按值传递**（pass by value）。注意，按值传递参数可以保护调用单元中的数据，使之不会被设计不当的函数错误地修改。例如，如果调用单元传递一个员工的名字给一个函数，我们当然希望函数不要改变这个名字。

遗憾的是，当参数表示很大的数据块时，按值传递参数的效率不高。一个更高效的给函数传递参数的方法是在调用程序单元中告诉函数它所需的实参的地址，从而使函数可以对实参进行直接存取。在这种情况下，我们称这种参数传递方式为**按引用传递**（pass by reference）。注意，按引用传递参数允许函数修改调用环境中的数据。这种方法很适合对列表进行排序的函数，因为调用这样的函数的目的就是改变列表。

举一个例子。假设函数Demo的定义为


```
def Demo (Formal):  
    Formal = Formal + 1
```

此外，假设变量Actual被赋予一个值5，我们用下面的语句调用Demo：

```
Demo(Actual)
```

那么，如果参数是按值传递的，在函数中对Formal的改变不会影响变量Actual的值（见图6-10）。但是，如果参数是按引用传递的，Actual的值就会增加1（见图6-11）。

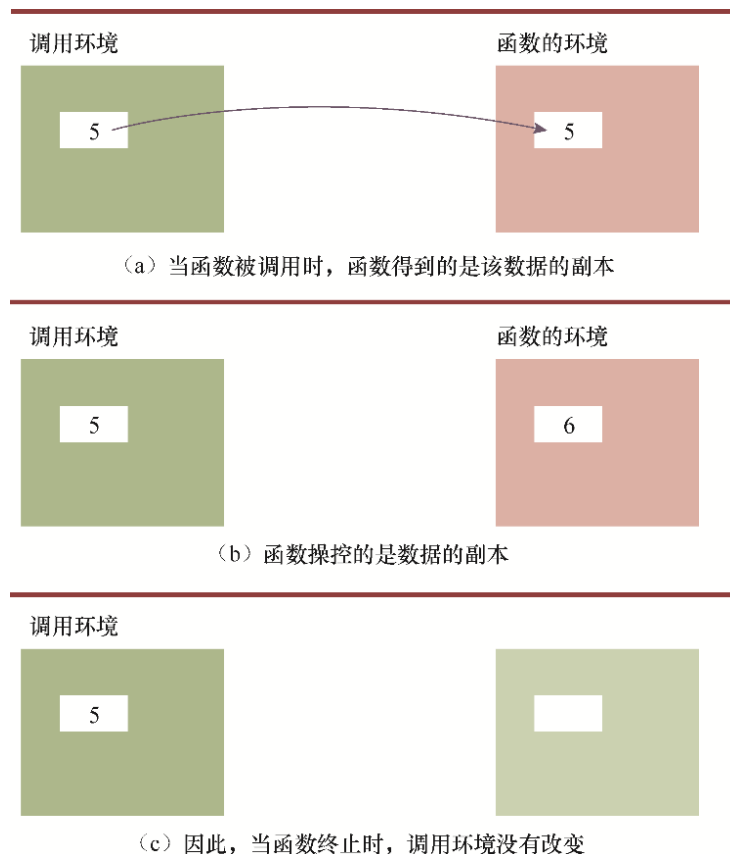


图6-10 执行函数Demo，按值传递参数

不同的程序设计语言提供了不同的参数传递技术，但是在所有情况下，参数的使用都允许编写一般意义上的函数，并在适当的时候应用于具体的数据。广义函数是有效软件复用的基石；有经验的程序员经常会保存一些触手可及的最喜欢的、经过良好测试的函数，用来解决常见问题。

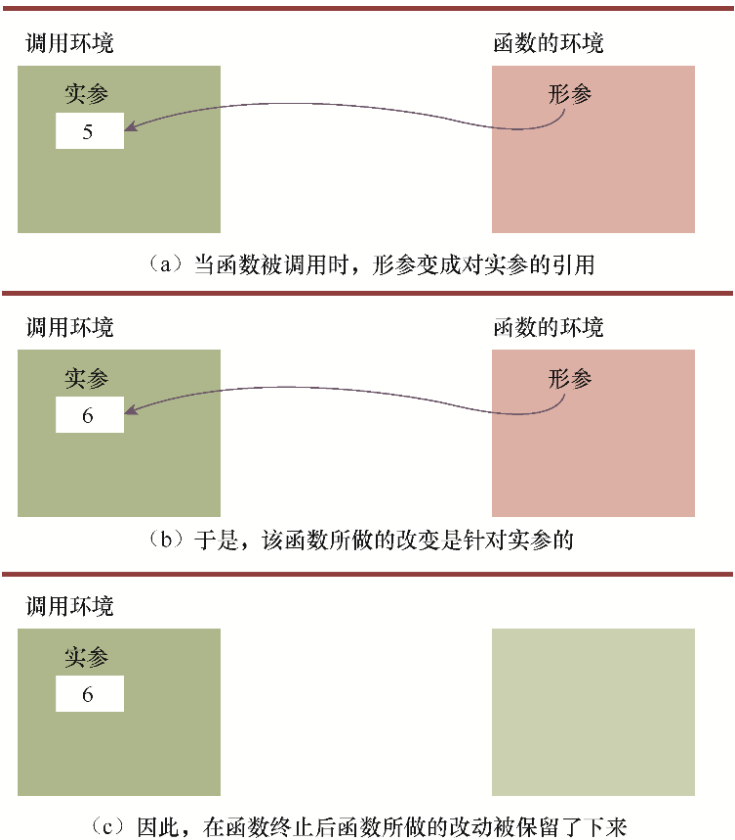


图6-11 执行函数Demo，按引用传递参数

基本知识点

- 抽象用允许软件复用的输入参数来概括功能。

6.3.3 有返回值的函数

让我们暂停一下，考虑许多程序设计语言中的函数概念的微小变化。有时，函数的目的是要产生一个值，而不是完成一个动作。（考虑这样两个函数之间的细微差别，一个函数是估计售出的小商品的数量，另一个函数用来玩一个简单的游戏；前者的重点是产生一个值，后者的重点是完成一个动作。）实际上，计算机科学中的函数一词源于数学中的函数概念，默认情况下，它是一个输入集和一个输出集之间的关系。从这个意义上讲，到目前为止，我们定义的所有Python示例和伪代码函数示例都是函数的特例，这些例子没有考虑输出值或返回值。（从技术上讲，所有Python函数都会返回值，因此都适合用数学术语函数来称呼。不包含return语句的函数返回默认值None。）

在程序设计语言的家族树中，有许多用来区分这两种不同类型的程序单元的术语。在有影响力的Pascal语言中，术语过程（procedure）专门用来指没有返回值的子程序。C和Java使用关键字void来标记没有返回值的函数或方法。Python程序员有时用术语有返回值的函数（fruitful function）来称呼那些有返回值的函数，以示区分。不管特定于语言的术语是什么，大多数语言起名的时候都有同样的想法——一个程序单元将值作为“函数值”传递回调用程序单元。也就是说，执行函数的结果就是计算出一个值并将这个值发送回调用程序单元。这个值可以存储在一个变量里供以后引用，也可以立即用于计算。例如，C、C++、Java或者C#的程序员可能会编写

```
ProjectedJanSales = EstimatedSales(January);
```

来请求把调用函数EstimatedSales产生的结果赋给变量ProjectedJanSales，以确定一月份预计出售多少件小商品。或者，程序员可能会编写

```
if (LastJanSales < EstimatedSales(January)) ...  
else ...
```

依据今年1月份的销售是否好于去年同期来产生不同的动作。注意，在第二种情况中，由函数计算出来的值是用来确定应该执行哪个分支的，不会被存储起来。

在程序中定义有返回值的函数的方式与定义无返回值的函数的方式基本相同。它们的不同在于，前者的函数头通常以指定要返回的值的数据类型开始，函数定义通常以明确指定要返回的值的return语句来结束。图6-12给出了一个名为CylinderVolume的函数的定义，它是仿照C语言写的。（实际上，C程序员会使用一种更简洁的形式，我们之所以使用这个稍冗长的版本是出于教学的需要。）当函数被调用的时候，函数的形参Radius和Height接收确定的值，函数使用这些尺寸计算汽缸容积并返回这个计算结果。因此，类似下面的语句可用在程序的其他地方来确定一个半径为3.45、高为12.7的汽缸内所盛物体的成本：

```
Cost = CostPerVolUnit * CylinderVolume(3.45, 12.7);
```

在前几章中，我们使用了几个有返回值的Python内置函数的例子，包括input()函数math.sqrt()函数，但是我们还没有定义一

个我们自己的函数。图6-12中的CylinderVolume函数的Python版本为：

```
def CylinderVolume(Radius, Height):  
    Volume = math.pi * Radius * Radius * Height  
    return Volume
```

程序设计语言允许软件开发人员创建自己的函数作为手段，用于实现软件复用、提高程序清晰度和可读性以及提升正确代码。复杂的整体算法比明智分解为较小函数的等价程序更难测试。编写良好的函数可以独立于较大的算法进行测试，增加发现隐藏软件bug的机会。一旦设计者更加确定组成函数块是正确的，他就会更加确信整个系统是正确的。

基本知识点

- 使用现有的正确算法作为构建块来构造新算法有助于确保新算法是正确的。

```
float CylinderVolume (float Radius, float Height)  
{  
    float Volume;  
    Volume = 3.14 * Radius * Radius * Height;  
    return Volume;  
}
```

函数头以要返回数据的类型作为开始

声明一个名为Volume的局部变量

计算汽缸的容积

终止函数并返回变量Volume的值

图6-12 用C语言编写的有返回值的函数CylinderVolume

事件驱动软件系统

文中，在我们考虑过的情况中，函数的激活都是程序中其他位置的语句显式调用该函数的结果。此外，还有一些情况中函数是由一个事件的发生隐式激活的。例如，在GUI中，有种函数描述单击按钮时应该产生什么动作，这种函数不是由其他程序单元的调用激活的，而是由单击按钮这一事件的结果激活的。这种其函数是通过事件而不是显式请求来激活的软件系统称作**事件驱动**（event-driven）系统。简而言之，事件驱动软件系统是由这样的函数组成的：它们描述各种事件发生时应该做什么。当系统执行时，这些函数一直处于休眠状态，直到与它们对应的事件发生，然后它们被激活，并在完成它们的任务后回到休眠状态。

6.3 节问题与练习

1. 变量的“作用域”是什么意思？
2. 函数和有返回值的函数的区别是什么？
3. 为什么许多程序设计语言实现I/O操作的方式很像是调用函数？
4. 形参和实参的区别是什么？
5. 按引用调用传递参数的函数与按值调用传递参数的函数的区别是什么？

6.4 语言实现

在本节中，我们将研究把高级语言编写的程序转换为机器可执行形式的过程。

6.4.1 翻译过程

将程序从一种语言转换为另一种语言的过程称为**翻译**（translation），原始形式的程序称作**源程序**（source program），翻译后的版本称作**目标程序**（object program）。翻译过程包括3个活动，它们分别是词法分析、语法分析和代码生成，翻译器中完成这3个活动的相应单元分别称为**词法分析器**（lexical analyzer）、**语法分析器**（parser）和**代码生成器**（code generator）（见图6-13）。

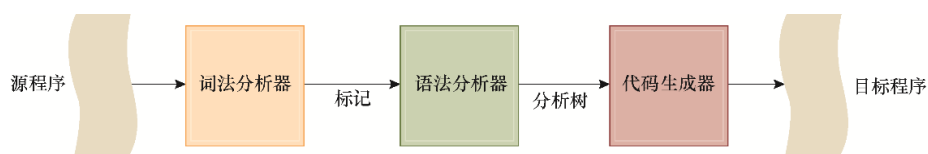


图6-13 翻译过程

词法分析是识别源程序中构成单个实体——**标记**（token）的符号串的过程。例如，3个符号的153不应该解释成一个1、一个5和一个3，而是应该识别为一个数值。同样，程序中出现的一个单词，尽管由独立的符号组成，也应该解释成一个单元。大多数人进行词法分析都是下意识的。当要求大声朗读的时候，我们读出来的都是词，而不是单个的字母。

因此，词法分析器逐个符号地读源程序，识别出哪些符号的组合代表一个标记，然后根据它们是否是数值、单词、算术运算符等对这

些标记分类。词法分析器对每一个标记及其分类进行编码，并将它们提交给语法分析器。在此过程中，词法分析器会跳过所有的注释语句。

因此，语法分析器将程序看作是由词法单元（标记）组成的，而不是由独立符号组成的。语法分析器的工作就是将这些单元组合成语句。实际上，语法分析是标识程序中的语法结构和辨认每个成分作用的过程。正是语法分析技术的细节使人在读句子

The man the horse that won the race threw was not hurt.

时，会犹豫一下。（试一试这句话：“That that is is. That that is not is not. That that is not is not that that is.”！）

Java和C#的实现

在某些情况下，如控制动画网页时，软件必须通过因特网传输并在远程机器上执行。如果软件是以源程序形式提供的，那么在目的地将产生额外的延迟，因为软件在执行前要被翻译成合适的机器语言。但是，以机器语言的形式提供软件意味着要根据远程计算机上使用的机器语言提供不同版本的软件。

Sun Microsystems公司和微软公司通过设计“通用机器语言”（在Java中称为字节码，在C#中称为.NET通用中间语言）解决了这个问题，源程序可以翻译成这种通用机器语言。虽然这些语言不是真实的机器语言，但它们被设计成了可快速翻译的。因此，只要把用Java或C#编写的软件翻译成合适的“通用机器语言”，这个软件就能被传送到

因特网中的另外一台机器上，在那里高效地执行。在某些情况下，这个执行是由解释器完成的，而在其他情况下，通用机器语言在执行前会被快速翻译，这个过程称为即时编译（just-in-time compilation）。

为了简化语法分析过程，早期程序设计语言坚持将每条程序语句以一种特定的方式定位在打印页上，这种语言称为**固定格式语言**（fixed-format language）。现在，许多程序设计语言都是**自由格式语言**（free-format language），这意味着不再苛求语句的位置安排了。自由格式语言的优势在于程序员可以以增强人类可读性的方式组织编写的程序。在这种情况下，通常使用缩进来帮助读者更好地把握语句的结构。程序员不应该写

```
if Cost < CashOnHand then pay with cash else use credit card
```

而应该写

```
if Cost < CashOnHand
    then pay with cash
    else use credit card
```

要想让机器来分析以自由格式语言编写的程序，就必须设计一种程序设计语言的语法，使得无论源程序中使用多少空格，机器都能识别出程序的结构。为此，大多数自由格式语言都使用诸如分号这样的标点符号来表示语句的结束，另外还使用诸如if、then和else这样的**关键字**（keyword）来表示单个短语的开始，这些关键字通常都是保留字（reserved word），即程序员在程序中不能把它们用于其他目的。Python在这方面与众不同，因为它有自由格式语言的特点，但它

又严格要求用缩进来表示结构，而不是用分号和大括号这样的标点符号。

语法分析过程是根据一系列定义程序设计语言语法的规则进行的。总的来说，这些规则称为**文法**（grammar）。表达这些规则的一种方法是借助**语法图**（syntax diagram），它是语言文法结构的图形化表示。图6-14给出了第5章中Python的if-else语句的语法图。这个图表明if-else结构的开始是单词if，然后是一个**布尔表达式**，接着是冒号，随后是缩进语句。此结构的后面有没有单词else及其后面的冒号和缩进语句，都是允许的。注意，实际出现在if-else语句中的项都在椭圆形框中，而需要进一步描述的项，如**布尔表达式**和**缩进语句**，都在矩形框中。需要进一步描述的项（矩形框中的那些项）称为**非终结符**（nonterminal），而出现在椭圆形框中的项称为**终结符**（terminal）。在一个程序设计语言语法的完整描述中，非终结符由额外的图来描述。

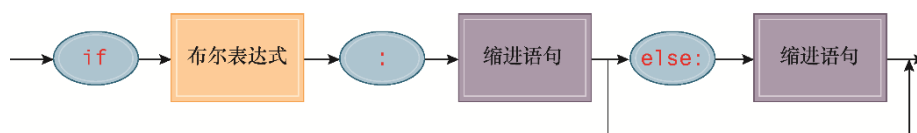


图6-14 Python的if-else语句的语法图

作为一个更完整的例子，图6-15给出了一组语法图，描述了一个称为**表达式**的结构的语法，该结构被规定为简单的算术表达式结构。第一个图描述了一个**表达式**，这个表达式由一个**项**（term）组成，后面可以跟（也可以不跟）一个+号或-号，+号或-号的后面是另一个**表达式**。第二个图描述了一个**项**，这个项由单个**因子**组成，或者由一个

因子后面跟一个 $\times$ 号或 $\div$ 号，然后再跟另一个项组成。最后一个图描述了一个因子，这个因子由x、y或z中的一个符号组成。

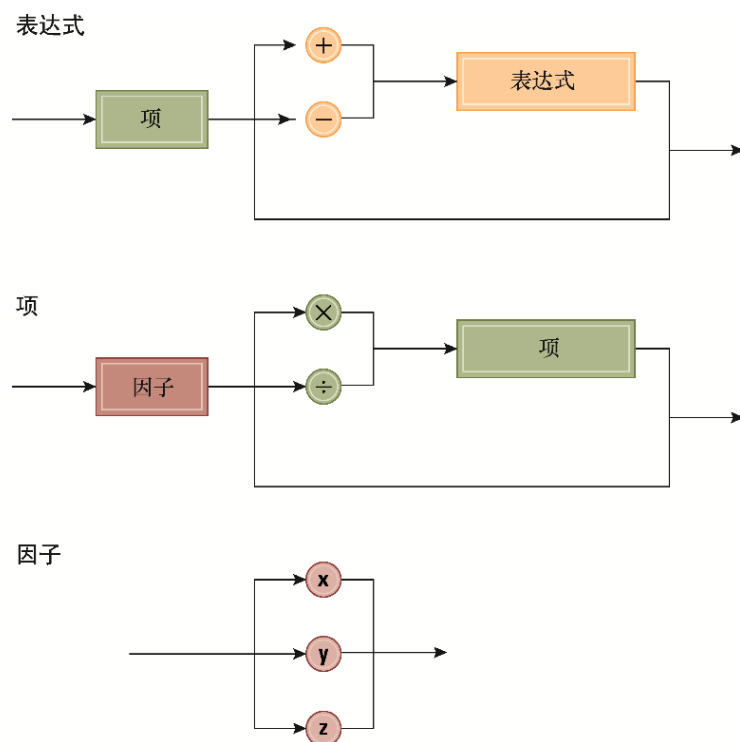


图6-15 描述简单代数表达式的语法图

特定的字符串符合一组语法图的方式还可以用[语法分析树](#) (parse tree) 以图形化的形式表示，根据图6-15中所示的语法图，图6-16描述了字符串

$x + y \times z$

的语法分析树。注意，树的顶端以非终结符[表达式](#)开始，并且在每一层都给出了本层的非终结符是如何分解的，这个过程直到得到字符串本身的全部符号才结束。具体来说，该图还说明（根据图6-15中的第一个图），一个[表达式](#)可以分解成一个[项](#)，后面跟有+号，然后

再跟一个表达式。接着，这个项可以分解成（用图6-15中的第二个图）一个因子（结果是符号x），最后的表达式可以分解（用图6-15中的第三个图）为一个项（结果是 $y \times z$ ）。

一个程序的语法分析过程本质上就是为源程序构建语法分析树的过程。的确，一个语法分析树代表了语法分析器对程序文法构成的理解。因此，描述程序文法结构的语法规则是不允许同一个字符串出现两个不同的语法分析树的，因为这将导致语法分析器内部产生歧义。我们称允许同一个字符串有两个不同的语法分析树的方法为多义文法（ambiguous grammar）。

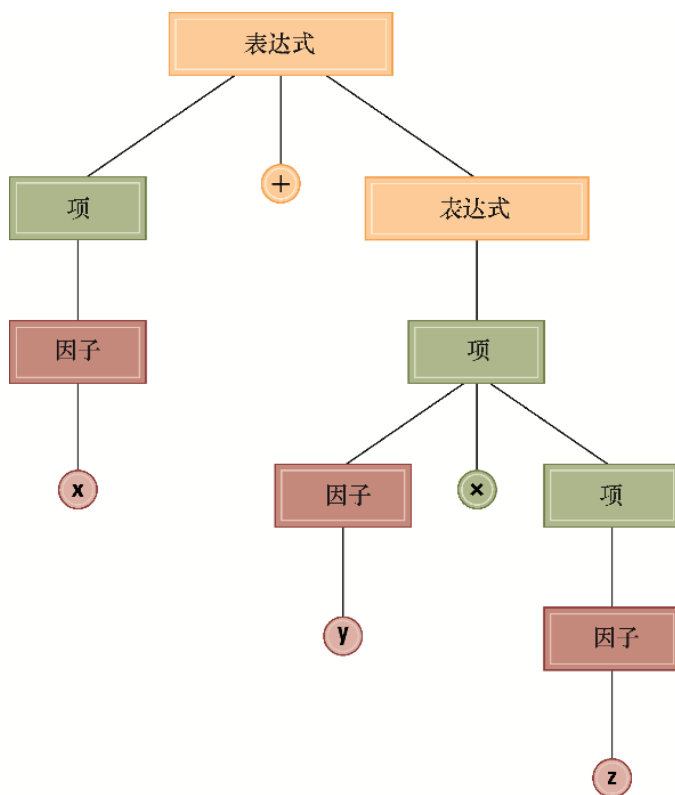


图6-16 基于图6-15中语法图的字符串 $x+yz$ 的语法分析树

文法中的歧义性可能非常微妙。事实上，图6-14中展示的规则存在这样的缺陷，对于下面的语句可以生成如图6-17所示的两个语法分析树：

```
if B1: if B2: S1 else: S2
```

注意，这两个解释明显不同。第一个表示语句S2在B1为假时执行，而第二个表示语句S2仅当B1为真且B2为假时执行。

形式化程序设计语言的语法定义旨在避免这种歧义性。在Python中，我们通过使用缩进来避免这样的问题。特别是，我们可以写

```
if B1:
    if B2:
        S1
    else:
        S2
```

和

```
if B1:
    if B2:
        S1
else:
    S2
```

来区分这两种可能的解释。

当语法分析器分析程序的文法结构时，它能够标识单独的语句，并且能够区分声明语句和命令语句。当它**识别**声明语句时，它将这些声明的信息记录在一个称为**符号表**（symbol table）的表中。因此，符

号表中包含这样的信息：程序中出现的变量名以及与这些变量相关的数据类型和数据结构。然后，当语法分析器分析到

z = x + y

这样的命令语句时，会以这些信息为依据来进行分析。

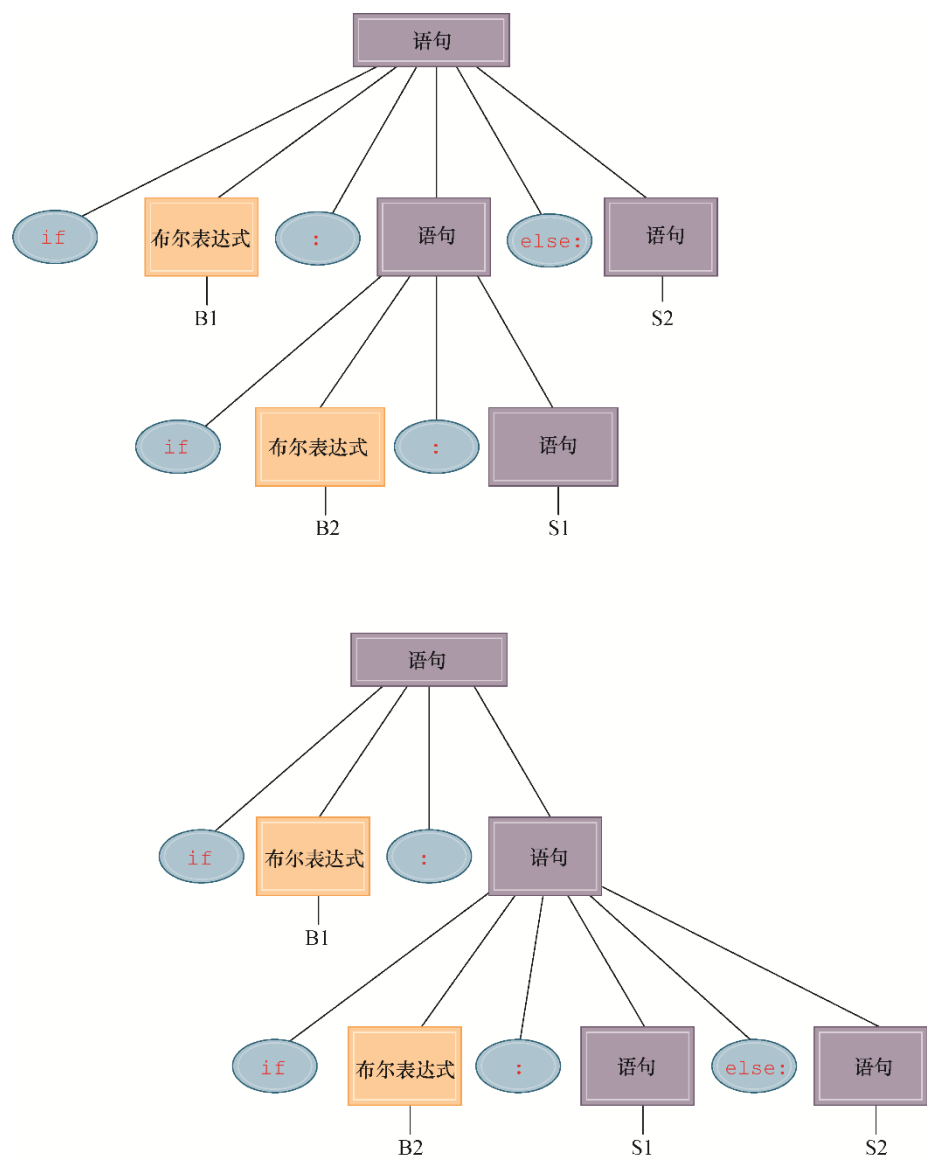


图6-17 语句if B1: if B2: S1 else: S2的两个不同的语法分析树

具体来说，为了确定符号+的含义，语法分析器必须要知道x和y的数据类型。如果x是浮点型而y是字符型，x和y相加就没有任何意义，应该报告为错误。如果x和y都是整型，语法分析器就会请求代码生成器用机器的整数加法操作码构建相应的机器语言指令；如果x和y都是浮点型，语法分析器就会请求使用浮点数加法操作码；如果都是字符型，语法分析器就会请求代码生成器构建一串机器语言指令来完成相应的拼接操作。

如果x是整型而y是浮点型这种特殊的情况，那么加法的概念是可用的，但是值不能以兼容的形式编码。在这种情况下，语法分析器可能会选择让代码生成器构建指令，把其中的一个值转换为另一种类型，然后再执行加法运算。这种类型之间的隐式转换称为**强制类型转换**（coercion）。

许多程序设计语言的设计者都反对强制类型转换，因为隐式类型转换会改变数据项的值，并因此导致微妙的程序bug。他们认为，需要强制类型转换通常就意味着程序在设计上存在缺陷，所以不应该让语法分析器来迁就。因此，大多数现代程序设计语言都是**强类型**（strongly typed）的，这意味着一个程序请求的所有活动都必须包含一致类型的数据。一些程序设计语言（如Java）允许进行强制类型转换，只要它是**类型提升**（type promotion），意思是将一个低精度值转换为一个较高精度的值。可能改变一个值的隐式强制类型转换将被报告为错误。在大多数情况下，程序员仍可以请求这种类型转换，方法

是进行显式**类型转换**（type cast）。显式类型转换会通知编译器：程序员知道将应用类型转换。

翻译过程的最后一步是**代码生成**（code generation），它是构造机器语言指令来实现语法分析器能识别的语句的过程。这个过程涉及许多问题，其中一个就是要产生高效的机器语言版本的程序。例如，我们考虑一个翻译下面的双语句序列的任务：

```
x = y + z  
w = x + z
```

如果这两条语句作为两个独立的语句来进行翻译，那么在执行加法操作之前，每一条语句都需要数据从主存储器传送到CPU。然而，效率可以通过这样的认识获得：一旦第一条语句被执行之后，x和z的值就已经在CPU的通用寄存器中了，执行第二条语句的时候就不再需要从内存中加载这两个数了。这种方法就称作**代码优化**（code optimization），是代码生成器的重要任务。

基本知识点

- 用程序设计语言编写的代码经常会被翻译成另一种（较低级的）语言的代码，以便在计算机上执行。

最后，我们应该注意的是，词法分析、语法分析和代码生成这3个活动并不是严格按照顺序执行的。相反，这些活动是交织在一起的。词法分析器首先从源程序中读取字符，并且标识出第一个标记。然后，它将这个标记传递给语法分析器。每当语法分析器从词法分析器

接收到一个标记时，就开始分析正在读取的文法结构。此时，它可能会向词法分析器请求另一个标记，而如果语法分析器认为已经读到了一个完整的短语或者语句，它就会请求代码生成器产生相应的机器指令。每一个这样的请求都会使代码生成器生成机器指令，并且将其加入目标程序中。接着，将一个程序从一种语言翻译成另一种语言的任务自然符合面向对象范型。源程序、词法分析器、语法分析器、代码生成器以及目标程序都是对象，每一个对象都在完成自己的任务的同时通过来回发送消息与其他对象进行交互（见图6-18）。

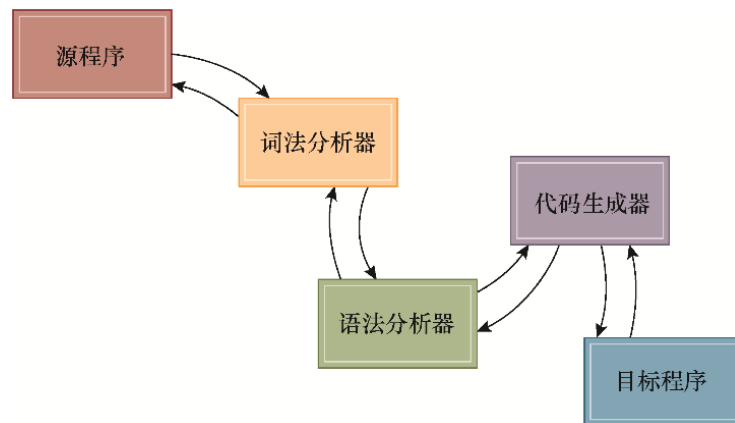


图6-18 翻译过程的面向对象方法

6.4.2 软件开发包

像编辑器和翻译器这样的在软件开发过程中应用的软件工具，通常会组合成一个包，起到集成软件开发系统的作用。根据3.2节中的分类方案，这样的系统属于应用软件。通过使用该应用包，程序员可以随时使用编辑器编写程序，使用翻译器将程序转换成机器语言，使用

各种各样的调试工具来跟踪故障程序的执行，以发现哪里出现了问题。

使用这样的集成系统的好处很多，可能最明显的就是程序员在需要修改和测试程序时，可以很容易地在编辑器和调试工具之间来回切换。此外，许多软件开发包允许开发中的相关程序单元以合适的方式链接，以便简化对相关程序单元的存取。一些包还维护这样的记录，即自上次基准制定以来一组相关程序单元中哪些单元已经做了修改。对许多相关程序单元由不同程序员开发的大型软件系统的开发来说，这些功能是非常有利的。

从小范围讲，软件开发包中的编辑器通常根据正在使用的程序设计语言进行定制。这样的编辑器通常会提供自动的行缩进功能，这已成为目标语言的事实标准，在某些情况下，程序员键入前面几个字符之后，编辑器就能够识别并自动补全关键字。此外，编辑器可以突出显示源程序中的关键字（也许是使用颜色），使程序更易读。

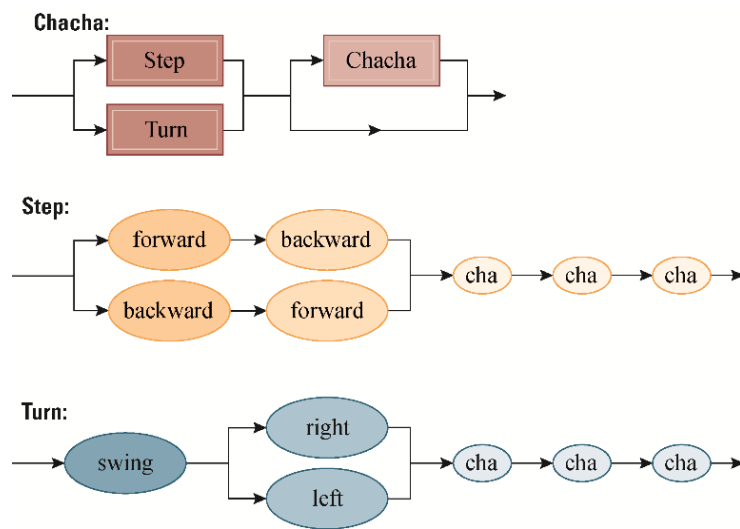
在下一章中，我们将会学到软件开发人员越来越多地研究这样的方法，即如何用预制的称为组件的块来构建新软件系统，这导致了一种新的称为组件架构的软件开发模型的产生。基于组件架构模型的软件开发包通常使用图形界面，在显示器屏幕上用图标表示各种组件。在这种环境下，程序员（或者组件装配人员）可以用鼠标选择所需的组件。选好的组件可以用软件开发包的编辑器进行定制，然后用鼠标进行定位和单击就可以加到其他组件上。这种包代表了在寻求更好的软件开发工具的方向上前进了一大步。

6.4节问题与练习

1. 描述翻译过程的3个主要步骤。
2. 什么是符号表？
3. 终结符和非终结符的区别是什么？
4. 基于图6-15中的语法图，画出下面这一表达式的语法分析树：

$x \times y + x + z$

5. 根据下面的语法图，描述遵循结构Chacha的字符串。



6. 现代程序设计语言编辑器为了让代码更易于阅读，对语法做了分色显示。你在自己用的编辑器中看到了几种颜色？每种颜色都代表什么意思？

6.5 面向对象程序设计

在6.1节中，我们了解到面向对象范型必然需要开发称为**对象**（object）的活动程序单元，每个对象都包含描述对象如何响应各种刺激的函数。一个问题的面向对象解决方法就是标识出涉及的对象，并将其作为一个独立的单元来描述。接着，面向对象程序设计语言提供描述对象及其行为的语句。在本节中，我们将引入一些以C++、Java和C#语言形式出现的语句，这3种语言都是当今使用的比较著名的面向对象程序设计语言。

6.5.1 类和对象

我们可以考虑开发一个简单的计算机游戏的任务，在这个游戏中，玩家要用高功率激光器射击坠落的流星来保卫地球。每个激光器都有一个有限的内部能源，每次射击都将消耗一部分能源。一旦能源耗尽，激光器就没用了。每一个激光器都应该能响应瞄准右面一点、瞄准左面一点和发射激光束的命令。

在面向对象范型中，计算机游戏中的每一束激光都将实现为一个对象，每个这样的对象都包含了它的剩余能量的记录以及修改目标和发射激光束的函数。由于所有的激光对象都有同样的属性，所以可以使用一个通用的**模板**来描述它们。在面向对象范型中，这样的一组对象的模板称作**类**（class）。

在第8章，我们将探究类和数据类型之间的相似点。现在，我们只需要注意类描述的是一组对象的共同特征，这与基本数据类型整型的

概念包含像数字1、5和82这样的数的一般特征类似。一旦程序员在程序里包含一个类的描述，就可以用模板来构建和操作那一“类型”的对象，这相当于基本的整型允许操作整型的“对象”。

在C++、Java和C#语言中，描述类的语句形式为

```
class Name
{
    .
    .
    .
}
```

其中，Name是一个名字，在程序的其他地方可通过这个名字来引用该类。大括号中是被描述的类的属性。具体来讲，图6-19展示了一个描述计算机游戏中激光结构的名为LaserClass的类。这个类包含1个名为RemainingPower的（整型）变量以及3个分别名为turnRight、turnLeft和fire的函数的声明，这些函数描述了要执行的完成相应动作的步骤。因此，任何一个从该模板构造的对象都包含如下特性：1个名字为RemainingPower的变量以及3个名字分别为turnRight、turnLeft和fire的函数。

对象内部的变量，如RemainingPower，称为**实例变量**（instance variable），对象里的函数称为**方法**（method，在C++中称作成员函数）。注意，在图6-19中，描述实例变量RemainingPower的声明语句类似于6.2节中讨论的声明语句，描述方法的方式和6.3节中描述函数的方式也比较相似。毕竟，实例变量的声明和方法的描述基本上都是命令式程序设计的概念。

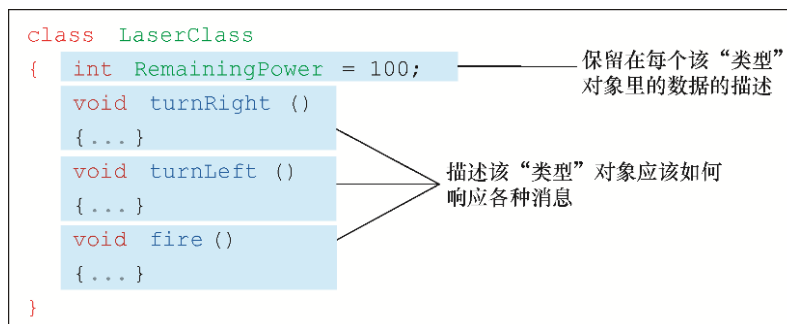


图6-19 描述计算机游戏中一种激光武器的类结构

一旦我们在游戏程序中描述了LaserClass类，我们就可以通过下列形式的语句来声明3个LaserClass“类型”的变量Laser1、Laser2和Laser3：

```
LaserClass Laser1, Laser2, Laser3;
```

注意，这与我们在前面6.2节中所学的声明3个整型变量x、y和z的语句

```
int x, y, z;
```

的格式是一样的，它们都是由一个“类型”名后跟一个将要声明的变量列表组成的，区别在于，后者所说的变量x、y和z将用于在程序中指向整型项（这是基本类型），而前者所说的变量Laser1、Laser2和Laser3将用于在程序中指向LaserClass“类型”的项（这是程序中定义的“类型”）。

一旦我们将变量Laser1、Laser2和Laser3声明为LaserClass“类型”，就可以给它们赋值。在这种情况下，所赋的值

必须是与LaserClass“类型”相一致的对象。这些赋值可以通过赋值语句来进行，但是在声明变量的语句中给变量赋开始值通常是很方便的。在C++语言的声明中，这种初始赋值是自动的。也就是说，语句

```
LaserClass Laser1, Laser2, Laser3;
```

不仅创建了变量Laser1、Laser2和Laser3，而且还创建了3个LaserClass“类型”的对象，其中每一个作为每个变量的值。在Java和C#语言中，这种初始赋值与对一个基本类型的变量赋初值的方式基本相同。具体而言，语句

```
int x = 3;
```

不仅声明了一个整型变量x，而且还把值3赋给了这个变量；语句

```
LaserClass Laser1 = new LaserClass();
```

不仅声明了一个LaserClass“类型”的变量Laser1，而且还用LaserClass模板创建了一个新的对象，作为开始值赋给了Laser1。

在进一步讨论之前，我们应该停下来强调一下类和对象的区别。类是一个模板，对象是从这个模板构造出来的。用一个类能创建许多对象。我们经常称对象为构造它的类的**实例**（instance）。因此，在我们的计算机游戏中，Laser1、Laser2和Laser3都是变量，其值都是LaserClass类的实例。

在用声明语句创建变量Laser1、Laser2和Laser3并将对象赋给它们之后，可以通过编写命令语句来激活这些对象中合适的方法来继续我们的游戏程序。（在面向对象术语中，这称为“向对象发送消息”。）具体而言，我们可以让赋给变量Laser1的对象用语句

```
Laser1.fire();
```

来执行它的fire方法，我们也可以让赋给Laser2的对象通过语句

```
Laser2.turnLeft();
```

来执行它的turnLeft()方法。

这些实际上只不过是函数的调用。事实上，前面的语句是调用赋给变量Laser1的对象内部的函数（方法）fire()，后面的语句是调用赋给变量Laser2的对象内部的函数turnLeft()。

在这个阶段，我们的流星游戏例子已经给出了掌握典型面向对象程序总体结构的背景知识（见图6-20）。它会包含与图6-19相似的各种类的描述，每一个都描述程序中使用的一个或多个对象的结构。另外，程序会包含一个命令程序段（通常和名字“main”关联），这个命令程序段包括运行程序时最初要执行的步骤序列。这个程序段会包含与我们激光器声明类似的声明语句，用来建立程序中使用的对象，还会包含调用那些对象中执行方法的命令语句。

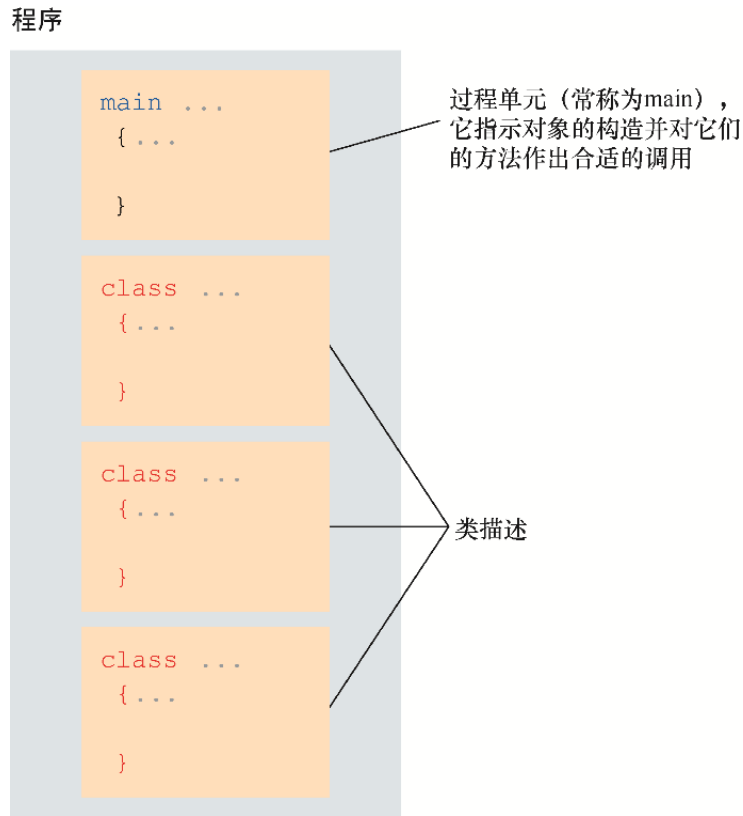


图6-20 典型的面向对象程序的结构

6.5.2 构造器

当构造对象时，经常需要进行一些个性化的定制。例如，在我们的流星计算机游戏中，我们可能想让一些不同的激光器具有不同的初始能量设置，这就意味着应该给不同对象中名为RemainingPower的实例变量赋不同的初始值。这种初始化的需要可以通过在合适的类中定义特殊的称为**构造器**（constructor）的方法来处理。构造器是在构建类的对象时自动执行的。构造器是通过在类的定义中使用与类同名的方法来标识的。

图6-21中给出了图6-19中的LaserClass类定义的扩展。注意，它包含一个构造器，该构造器的形式是一个名为LaserClass的方法。这个方法将它接收的参数值赋给实例变量RemainingPower。因此，当从这个类构造对象时，这个方法就会执行，使得RemainingPower被初始化为一个合适的设置。

```
class LaserClass
{ int RemainingPower;
  LaserClass(InitialPower)
  { RemainingPower = InitialPower;
  }
  void turnRight()
  { ... }
  void turnLeft()
  { ... }
  void fire()
  { ... }
}
```

当一个对象被创建时，构造器给RemainingPower赋一个值

图6-21 带有构造器的类

构造器使用的实参是由实际创建对象的语句里的参数列表标识的。因此，基于图6-21中给出的类的定义，C++程序员将会编写语句

```
LaserClass Laser1(50), Laser2(100);
```

来创建两个LaserClass类型的对象，一个是Laser1，初始能量值为50；另一个是Laser2，初始能量值为100。使用Java和C#的程序员完成同样任务的语句是：

```
LaserClass Laser1 = new LaserClass(50);
LaserClass Laser2 = new LaserClass(100);
```

6.5.3 附加特性

现在假设我们想要增强我们的流星计算机游戏，以使得玩家达到一定的分数时，可以获得奖励为其现有的激光器补充能量。除可以补充能量以外，这些激光器与其他激光器有一样的属性。

为了简化对具有相似但不完全相同特征的对象描述，面向对象语言允许一个类通过称为**继承**（inheritance）的技术包含其他类的属性。例如，假设我们正在使用Java来开发我们的游戏程序，我们可以先用之前描述的类语句定义一个名为LaserClass的类，描述程序中所有激光器的共同属性。接下来，我们使用语句

```
class RechargeableLaser extends LaserClass
{
    .
    .
    .
}
```

来描述另一个名为RechargeableLaser的类。（C++和C#语言用冒号代替单词extends。）这里extends子句指明了这个类不仅继承了LaserClass类的特性，还包含了大括号中出现的特性。在这个示例中，这些大括号将包含新的方法（名字也许是recharge），描述将实例变量RemainingPower重置为其初始值所需的步骤。一旦这些类定义好了，就可以使用语句

```
LaserClass Laser1, Laser2;
```

来将变量Laser1和Laser2声明为指向传统激光器的变量，并且使用语句

```
RechargeableLaser Laser3, Laser4;
```

来将变量Laser3和Laser4声明为指向拥有RechargeableLaser类中描述的额外属性的激光器的变量。

继承的使用导致了各种具有相似但不完全相同特征的对象的存在，也因此导致了6.2节中讨论的重载现象。（让我们回忆一下，重载是指使用一个符号，如+号，代表不同的运算，具体取决于它的操作数的类型。）假设一个面向对象图形包包含各种对象，每个代表一种形状（圆形、矩形、三角形等）。一个特定的图像可能由一组这类对象构成，每个对象都“知道”自己的大小、位置、颜色，都有自己响应消息的方式，例如，移到一个新的位置或者在显示器上画出自己。要画图像，我们只需给图像中的每个对象发送“画自己”的消息。但是，对象形状的不同决定了所使用的画一个对象的例程是不同的——画正方形的过程和画圆形的是不同的。这种对消息的自定义解释称为**多态**（polymorphism），这种消息被说成是多态的。

封装（encapsulation）是与面向对象程序设计相关的另一个特性，它是指限制对一个对象内部属性的访问。说一个对象的某些特性被**封装**的，意思是只有对象自己能够访问它们。被封装的特性被认为是私有的，可以从对象外部访问的特性被认为是公有的。

作为一个示例，让我们回到图6-19中的LaserClass类。回想一下，它描述了1个实例变量RemainingPower和3个方法turnRight()、turnLeft()和fire()。这些方法可以被其他程序单元访问，以使LaserClass的实例执行合适的动作。但是RemainingPower的值应该只能由实例的内部方法来修改，其他程序单元应该不能直接访问这个值。为了强制实现这些规则，我们只需将RemainingPower指定为private而将turnRight()、turnLeft()和fire()指定为public，如图6-22所示。通过这种设计，在翻译程序期间任何试图从RemainingPower值所在的对象的外部访问RemainingPower值的操作都会被标识为错误——强制程序员在继续之前改正该错误。

类中的成分是指定为公有还是私有取决于它们是否应该是从其他程序单元可访问的

```
class LaserClass
{
    private int RemainingPower ;
    public LaserClass (InitialPower )
    {RemainingPower = InitialPower;
    }
    public void turnRight ( )
    { ... }
    public void turnLeft ( )
    { ... }
    public void fire ( )
    { ... }
}
```

图6-22 在Java或C#程序中使用封装的LasserClass的定义

封装是一个强大的抽象工具，它允许软件开发人员从对象的实现（将变量命名为什么、它们当前保存了什么值等内部细节）中分离对象的行为（方法实际做什么）。通过这种方式，系统中的其他对象仅

依赖对象的公有方法和字段显式提供的服务，这使得整个系统在后面修改对象代码的内部细节时更不容易出错。

基本知识点

- 数据抽象提供了从实现中分离行为的手段。

6.5节问题与练习

1. 对象和类的区别是什么？
2. 在本节的计算机游戏示例中，除了LaserClass，还可以找到什么对象类？除了RemainingPower，还可以在LaserClass类中找到什么实例变量？
3. 假设类PartTimeEmployee和FullTimeEmployee继承了Employee类的属性。你希望在每个类中找到哪些特性？
4. 什么是构造器？
5. 为什么类中的一些项要指定为private？

*6.6 程序设计并发活动

假设有人要求我们为多路攻击敌人飞船的计算机游戏设计一个生成动画的程序。一种办法就是只设计一个用来控制整个动画屏幕的程序，由它来负责绘制每艘飞船，这（如果动画要做得很逼真）将意味着该程序必须掌握许多飞船的各自特征。另一种办法就是设计一个程序来控制单艘飞船的动画，每艘飞船的特征由程序执行的开始阶段赋

给参数的值决定。然后，动画可以通过创建这个程序的多个激活（activation）来构建，每一个激活都使用各自的一组参数。同时执行这些激活，我们就能产生有许多飞船从屏幕上同时飞过的错觉。

这种多个激活的同时执行称为**并行处理**（parallel processing）或**并发处理**（concurrent processing）。真正的并行处理需要多个CPU内核，每个CPU内核执行一次激活。当仅有一个CPU可用时，并行处理的错觉是通过允许多个激活共享一个处理器的时间来获得的，所采用的方式与通过多道程序设计系统（见第3章）来实现相似。

与较传统的单指令序列语境相比，许多现代计算机应用程序在并行处理语境中解决起来更容易。于是，较新的程序设计语言提供了表达并行计算所涉及的语义结构的语法。这种语言的设计需要对这些语义结构的识别以及描述它们的语法的开发。

每种程序设计语言都倾向于从自己的角度来处理并行处理范型，结果就有了不同的术语。例如，“激活”这个非正式的称谓在Ada语言中称为任务（task），而在Java语言中称为线程（thread）。也就是说，在Ada程序中，同时发生的动作是通过创建多个**任务**来完成的，而在Java程序中是通过创建多个**线程**来完成的。无论哪一种情况，结果都是生成和执行多个活动，其方式与多任务处理操作系统控制下的进程是一样的。我们将采用Java中的术语，把这种“进程”都称为线程。

也许，在涉及并行处理的程序中必须表达的最基本的动作就是创建新线程。如果我们想要同时执行飞船程序的多个激活，就需要有能说明这一点的语法。这种新线程的产生处理与请求一个传统函数的执行类似。不同之处在于，在传统设置下，请求函数激活的程序单元在所请求的函数终止之前不会继续进行（见图6-8），而在并行语境中，请求程序单元在被请求函数执行其任务的同时继续执行（见图6-23）。因此，要创建多艘飞船飞过屏幕的场景，我们可以写一个主程序，该主程序只生成飞船程序的多个激活，每个激活都配有描述不同飞船特征的参数。

一个与并行处理相关的更复杂的问题是处理两个线程之间的通信。例如，在飞船的例子中，代表不同飞船的线程可能需要相互之间通告它们的方位以协调行动。在其他情况下，一个线程需要等待，直到另一个线程到达其计算的某个位置，或者一个线程在实现特定的任务之前需要停止另一个线程。

这种通信需求一直是计算机科学家研究的课题，并且许多新的程序设计语言都有不同的解决线程交互问题的办法。作为一个示例，让我们考虑两个线程操作同一个数据时面临的通信问题。（这个例子在选读的3.4节中有更详细的描述。）如果并发执行的两个线程都需要给一个公用的数据项值加3，就需要一个方法来保证在允许另一个线程执行任务之前先允许一个线程完成任务，否则它们会使用相同的初始值开始各自的计算，这意味着最后的结果将会是只增加3而不是增加6。我们说一次只能由一个线程访问的数据必须互斥访问。

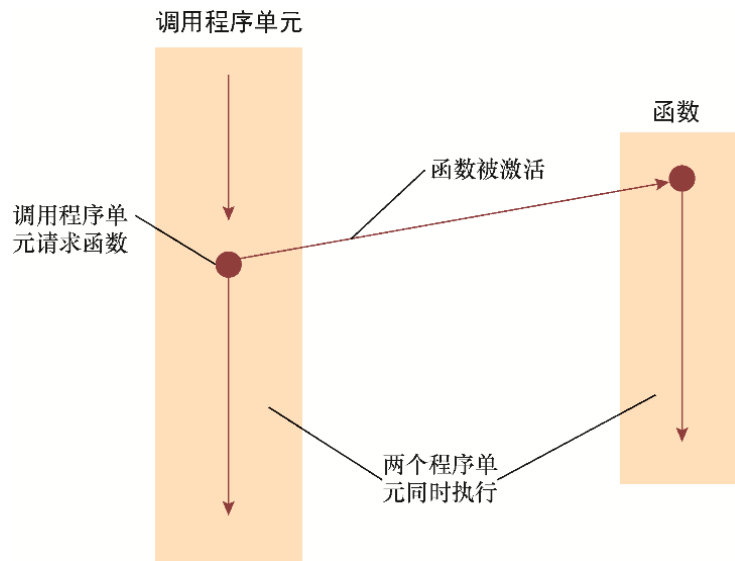


图6-23 线程的产生处理

一种实现互斥访问的方式是编写描述涉及线程的程序单元，以便在一个线程正在使用共享数据时，它可以阻止其他线程访问这个数据，直到这样的访问是安全的。（这个办法在3.4节中描述过，在那里我们把一个进程中访问共享数据的那部分标识为临界区。）经验表明，这个办法是有缺陷的，它把保证互斥的任务分散在程序各处——每个访问该数据的程序单元都必须是正确设计的才能确保这种互斥，因此一个程序段中的错误就能破坏整个系统。由于这个原因，许多人认为更好的解决办法是使数据项有控制对自身访问的能力。简而言之，不再依赖于访问数据的线程来防止多重访问，而是赋予数据项本身这个能力。结果是访问控制集中在程序中的一个点，而不是分散在许多程序单元中。增加了对自身访问的控制能力的程序项经常称作**监控程序**（monitor）。

我们的结论是，用于并行处理的程序设计语言的设计，涉及开发表达诸如线程的创建、线程的暂停和重启、临界区的标识以及监控程序的组成等的办法。

智能手机编程

用于手持设备、移动设备和嵌入式设备软件开发的通用程序设计语言通常和用于其他开发的相同。若有较大的键盘和额外的耐心，我们可以使用智能手机编写某些智能手机应用。不过，大多数情况下，智能手机的软件都是使用特殊软件系统在台式计算机上开发的，而这些特殊软件系统提供了用于编辑、翻译和测试智能手机软件的工具。简单的应用通常用Java、Swift、C++和C#编写。不过，若要编写较复杂的应用或核心系统软件，我们可能需要额外支持并行处理和事件驱动的编程。

最后，我们应当注意，尽管动画提供了一个探索并行计算问题的有趣场景，但是这只是受益于并行处理技术的一个领域，其他领域包括天气预报、空中交通管制、复杂系统（从核反应到行人交通）的模拟、计算机联网以及数据库维护。

6.6节问题与练习

1. 可以进行并发处理的程序设计语言中哪些特性是比较传统的程序设计语言中没有的？
 2. 描述两种可以确保对数据进行互斥访问的方法。
 3. 说出除动画以外可受益于并行计算的其他领域。
-

*6.7 说明性程序设计

在6.1节中，我们断言形式逻辑提供了一个通用的解决问题的算法，围绕这个算法可以构建一个说明性程序设计系统。在本节中，我们将研究这个断言，首先介绍这个算法的基本原理，然后简要地看一看基于这种算法的说明性程序设计语言。

6.7.1 逻辑推演

假设我们知道科米不是在舞台上就是病了，并且我们得知科米不在舞台上。于是我们推断出科米一定是病了。这个演绎推论的例子称为**消解**（resolution）。消解是许多称为**推理规则**（inference rule）的技术中的一种，用于从大量的陈述中推导出结果。

为了更好地理解消解，我们首先约定用单个字母来表示简单语句，用符号 $\neg$ 来表示语句的否定。例如，我们用 A 来代表“科米是一位王子”，用 B 来代表“佩奇小姐是一位演员”。那么，表达式

$A \text{ OR } B$

的意思是“科米是一位王子或者佩奇小姐是一位演员”，而

$B \text{ AND } \neg A$

的意思是“佩奇小姐是一位演员，科米不是一位王子”。我们将用箭头来表示“蕴涵”关系。例如，表达式

$A \rightarrow B$

的意思是“科米是一位王子蕴涵着佩奇小姐是一位演员”。

以这种通式，消解原理意味着从形式为

$P \text{ OR } Q$

和

$R \text{ OR } \neg Q$

的语句可以归结出语句

$P \text{ OR } R$

这样，我们就说原来的两条语句消解形成了第三条语句，我们称之为**消解式**（resolvent）。重要的是要看到，这个消解式是原始语句的逻辑结论。也就是说，如果原始语句是真的，那么消解式也一定是真的。（如果 Q 是真的，那么 R 一定是真的；但如果 Q 是假的，那么 P 一定是真的。因此，不管 Q 是真是假， P 或者 R 一定是真的。）

我们用图形表示两个语句的消解，如图6-24所示，其中我们写了原始语句，画了从原始语句向下到消解式的线。注意，消解只能用于以**子句形式**（clause form）出现的成对语句，即其基本组成部分通过布尔运算符OR连接的语句。因此

$P \text{ OR } Q$

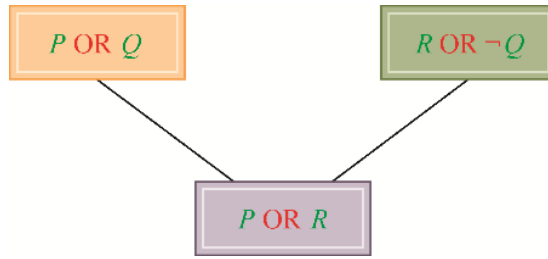


图6-24 消解语句($P \text{ OR } Q$)和($R \text{ OR } \neg Q$)得出($P \text{ OR } R$)

是子句形式，而

$$P \rightarrow Q$$

就不是子句形式。事实是这个潜在的问题并没有引起高度关注，因为这是数理逻辑中的一个定理的结论，这个定理指出，任何用一阶谓词逻辑（一个用扩展表达能力表示语句的系统）表达的语句都可以用子句形式来表达。我们就不在这里继续讨论这个重要的定理了，但是为了将来参考，我们发现语句

$$P \rightarrow Q$$

等价于子句形式的语句

$$Q \text{ OR } \neg P$$

如果一组语句中的所有语句不可能同时为真，就说这组语句就是**不相容的**（inconsistent）。换句话说，一组不相容的语句是一组自相矛盾的语句。一个简单的例子是语句 P 与语句 $\neg P$ 的组合。逻辑学家已经证明，反复消解提供了验证一个不相容子句集合的不相容性的系统化方法。这个方法的规则是，如果反复进行消解产生了一个空子句

(消解 P 形式的子句和 $\neg P$ 形式的子句的结果)，那么原来的那组语句必定是不相容的。例如，图6-25证明下面这组语句是不相容的：

$P \text{ OR } Q$

$R \text{ OR } \neg Q$

$\neg R$

$\neg P$

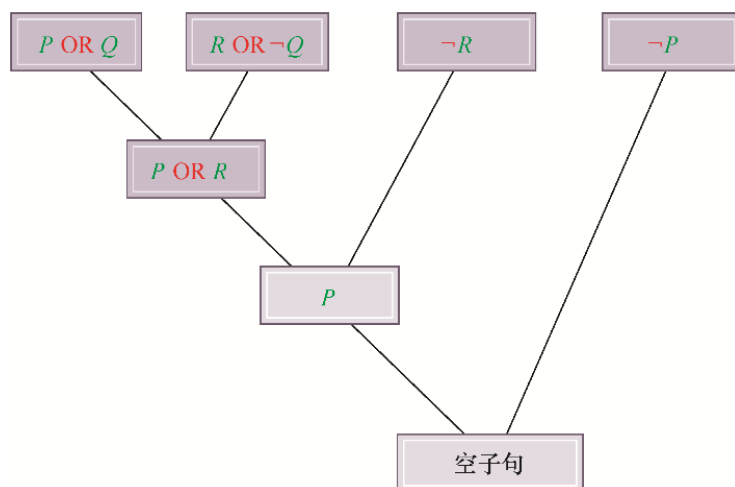


图6-25 消解语句 $(P \text{ OR } Q)$ 、 $(R \text{ OR } \neg Q)$ 、 $\neg R$ 和 $\neg P$

假设现在我们先想确认一组语句蕴涵了语句 P 。蕴涵语句 P 就相当于对语句 P 的否定 $\neg P$ 。因此，要证明原来的那组语句蕴涵 P ，只需将原来的语句和 $\neg P$ 进行消解，直到产生一个空子句。基于获得的空子句，我们就可以得出结论： $\neg P$ 与原来的语句组是不相容的，因此，原来的那组语句一定蕴涵 P 。

在将消解应用于一个实际的程序设计环境之前，还有最后一个问题。假设我们有两个语句（其中X代表任何地方）

$(\text{Mary is at } 'X') \rightarrow (\text{Mary's lamb is at } X)$
--

和

Mary is at home

按照子句形式，这两个语句变为

$(\text{Mary's lamb is at } X) \text{ OR } \neg(\text{Mary is at } X)$
--

和

(Mary is at home)

乍一看，似乎没有可以消解的部分。此外， (Mary is at home) 和 $\neg(\text{Mary is at } X)$ 近乎相互对立。问题是要认识到： $\text{Mary is at } X$ 是一个关于位置的一般语句，而关于home的语句则是 $\text{Mary is at } X$ 的特定形式。因此，上面第一条语句的特殊情况是

$(\text{Mary's lamb is at home}) \text{ OR } \neg(\text{Mary is at home})$
--

它可以与语句

(Mary is at home)

消解产生语句

```
(Mary's lamb is at home)
```

把值赋给变量（如将值home赋给X），从而使消解可以进行的过程称为**合一**（unification）。在演绎系统中，这个过程允许将一般语句用于特定的应用。

6.7.2 Prolog

程序设计语言Prolog（PROgramming in LOGic的简写）是一个说明性程序设计语言，它解决问题的底层算法的基础就是反复地进行消解，这样的语言称为**逻辑程序设计**（logic programming）语言。一个Prolog程序由一组初始语句组成，底层算法在这组语句的基础上进行演绎推理。构成这些语句的成分称为**谓词**（predicate）。一个谓词由一个谓词标识符和一个带括号的语句组成，括号里列有该谓词的变元。一个谓词代表一个与它的变元相关的事实，因而谓词标识符的选取通常都反映了该事实的基本语义。因此，如果想表达Bill是Mary的家长的这一事实，我们可以使用谓词形式

```
parent(bill, mary)
```

注意，尽管这个谓词里的变元表示的是专有名词，但是它们仍然是以小写字母开始的，这是因为Prolog区分常量变元和变量变元的依据是常量以小写字母开始，而变量以大写字母开始。这里，我们使用了Prolog的术语，用其中的术语**常量**代替更通用的术语**字面量**。更准确地说，在Prolog里用于表示字面量的词语bill（注意b是小写）可

能会被更通用的表示法表示为Bill，而词语Bill（注意B是大写）在Prolog中表示变量。

Prolog程序中的语句有事实和规则两种，每一种都是以一个句点来结束的。一个事实包括一个谓词。例如，乌龟比蜗牛快这个事实用Prolog语句可以像下面这样表示：

```
faster(turtle, snail).
```

而兔子比乌龟快的事实可以表示为

```
faster(rabbit, turtle).
```

一个Prolog规则就是一个“蕴涵”语句。但是，Prolog程序员并不是将语句写成 $X \rightarrow Y$ 这种形式，而是写成“Y if X”，只是单词if用符号：-（一个冒号和一个连字符）代替了。因此规则“X is old implies X is wise”对于一个逻辑学家来说可能要这样来表述：

```
old(X)  $\rightarrow$  wise(X)
```

而在Prolog语言里则表述为

```
wise(X) :- old(X).
```

又如，规则

```
(faster(X, Y) AND faster(Y, Z))  $\rightarrow$  faster(X, Z)
```

在Prolog里表达为：

```
faster(X, Z) :- faster(X, Y), faster(Y, Z).
```

[分隔faster(X, Y)和faster(Y, Z)的逗号代表合取符AND。] 尽管像这样的规则不是子句形式的，但是它们在Prolog里是被允许的，因为它们很容易转化为子句形式。

记住，Prolog系统并不知道程序中谓词的含义，它只是根据消解推理规则以完全的符号方式来对语句进行操作。因此，用事实和规则来描述谓词的有关特性完全是程序员的职责。从这一点来看，Prolog的事实倾向于用来标识谓词的特殊实例，而规则用来描述一般原理。这就是前面有关谓词faster的语句所使用的方法。这两个事实描述了“快”的特定实例，而规则描述了一个一般属性。注意，兔子比蜗牛快的事实，尽管没有明说，但这是两个事实通过规则结合的结论。

当使用Prolog语言进行软件开发时，程序员的任务就是开发一组事实和规则来描述已知的信息。这些事实和规则构成了要在演绎系统中使用的一组初始语句。一旦这组语句确定了，就可以向系统提出一些猜测（在Prolog术语中称为目标）——通常是用计算机的键盘键入它们。当向Prolog系统提交这样的目标后，系统利用消解来试图证明这个目标是初始语句的结论。因为每一个都是初始语句的逻辑结论，所以基于我们描述更快关系的那组语句，可以证明下面的每一个目标：

```
faster(turtle, snail).  
faster(rabbit, turtle).  
faster(rabbit, snail).
```

最开始的两个与出现在初始语句中的事实相同，而第三个需要系统的某种程度的演绎。

如果我们提供的目标的变元不是常量而是变量，那么可以得到更为有趣的例子。在这种情况下，Prolog试图从初始语句中推导出目标，同时跟踪推导所需要的合一。然后，如果这个目标达到了，Prolog就报告这些合一。例如，考虑目标

```
faster(w, snail).
```

Prolog对于它的响应是报告

```
faster(turtle, snail).
```

的确，这是初始语句的一个结论，并且通过合一与这个目标一致。此外，如果要求Prolog提供更多的结论，那么它会找到并报告结论

```
faster(rabbit, snail).
```

而我们能通过提出目标

```
faster(rabbit, w).
```

来要求Prolog寻找一些比兔子慢的动物的实例。事实上，如果我们以目标

```
faster(v, w).
```

开始，Prolog会最终寻求所有可以从初始语句中推导出来的更快关系。这意味着可以用一个简单的Prolog程序来证明某种特定的动物比另一种跑得快，从而找出那些比某种给定的动物跑得快的动物，找出那些比某种给定的动物跑得慢的动物，或者找出所有跑得更快的关系。

这个潜在的多样性是激发计算机科学家想象力的特性之一。遗憾的是，当在Prolog系统中实现时，消解过程继承了“其理论形式中并没有呈现出来”的限制，因此，Prolog程序无法实现它们被预期的灵活性。要理解我们的意思，首先要注意图6-25中的示意图，它只显示了与手头任务相关的那些消解，此外，还有一些其他的消解方式，例如，消解最左子句和最右子句可产生消解式Q。因此，除描述应用所涉及的事实和规则的语句外，Prolog程序经常必须包含一些额外的能正确指导消解过程的语句。由于这个原因，实际的Prolog程序可能无法获得我们先前示例中提出的目的多样性。

6.7节问题与练习

1. 语句 R 、 S 、 T 、 U 和 V 中哪些是 $(\neg R \text{ OR } T \text{ OR } S)$ 、 $(\neg S \text{ OR } V)$ 、 $(\neg V \text{ OR } R)$ 、 $(U \text{ OR } \neg S)$ 、 $(T \text{ OR } U)$ 和 $(S \text{ OR } V)$ 这组语句的逻辑结论？
2. 下面这组语句是相容的吗？请解释你的答案。

$$P \text{ OR } Q \text{ OR } R$$

$$\neg R \text{ OR } Q$$

$R \text{ OR } \neg P$

$\neg Q$

3. 补全下面的Prolog程序末尾的两个规则，以使得谓词 `mother(X, Y)` 的含义是“X是Y的母亲”，`father(X, Y)` 的含义是“X是Y的父亲”。

```
female(carol).  
female(sue).  
male(bill).  
male(john).  
parent(john, carol).  
parent(sue, carol).  
mother(X, Y):-  
father(X, Y):-
```

4. 根据第3题的Prolog程序，下面的规则想要表述这样的含义：
如果X和Y有共同的父母，X是Y的同胞。

```
sibling(X, Y) :- parent(Z, X), parent(Z, Y).
```

这个同胞关系的定义会使Prolog得出什么意外的结论？

复习题

（带*的题目涉及选读章节的内容。）

1. 说一种程序设计语言是机器无关的是什么意思？

2. 将下面的Python程序翻译成附录C中描述的Vole机器语言：

```
x = 0
while (x < 3):
    x = x + 1
```

3 . 将语句

```
Halfway = Length + Width
```

翻译成Vole机器语言（见附录C），假设Length、Width和Halfway都以浮点记数法表示。

4 . 将高级语言语句

```
if (X == 0):
    Z = Y + W
else:
    Z = Y + X
```

翻译成Vole机器语言（见附录C），假设W、X、Y和Z的值都是用二进制补码记数法表示的，每个值使用存储器中的一个字节。

5 . 为什么在第4题中为了翻译语句必须标识变量的数据类型？为什么许多高级程序设计语言需要程序员在程序的开头标识每一个变量的类型？

6 . 说出4种不同的程序设计范型的名称并描述它们。

7 . 假设函数 f 需要两个数值作为它的输入，并且返回这两个值中较小的一个作为输出值。如果 w 、 x 、 y 和 z 都代表数值，那么 $f(f(w, x), f(y, z))$ 返回的结果是什么？

8. 假设 f 是一个函数，它返回一个输入字符串经过反转得到的结果； g 是一个函数，它返回两个输入字符串经过拼接得到的结果。如果 x 是字符串 $abcd$ ，那么 $g(f(x), x)$ 的返回值是什么？

9. 假设你要写一个面向对象程序来维护自己的财务记录。在表示活期存款账户的对象里应该存储什么数据？这个对象应该能响应什么消息？程序中可能使用的其他对象是什么？

10. 概述机器语言和汇编语言的区别。

11. 为附录C中描述的Vole机器语言设计一个汇编语言。

12. 程序员John认为在程序中声明常量的功能是不必要的，因为可以用变量来代替它。例如，在6.2节的AirportAlt的例子中，可以把AirportAlt声明为变量，然后在程序的开头为其赋需要的值。为什么这种方法不如使用常量好？

13. 概述声明语句和命令语句的区别。

14. 解释字面量、常量和变量的区别。

15. a. 什么是运算符优先级？

b. 根据运算符优先级，表达式 $6+2\times 3$ 的值是什么？

16. 什么是结构化程序设计？

17. 语句

```
if (X == 5):  
    ...
```

中的双“等”号和赋值语句

```
X = 2 + Y
```

中的单“等”号在含义上的区别是什么？

18．画一个流程图来表示下面的for语句所表达的结构。

```
for(int x = 2; x < 8; ++x)  
{ ... }
```

19．用Python的while语句把下列的for语句翻译成等效的程序段。

```
for(int x = 2; x < 8; ++x)  
{ ... }
```

20．如果你熟悉乐谱，就把音乐符号当成程序设计语言来分析。控制结构是什么？插入程序注释的语法是什么？什么音乐符号有类似于图6-7中的for语句的语义？

21．画一个流程图来表示下面的语句所表达的结构。

```
switch (suit)  
{  
    case 'clubs': bid(1);  
    case 'diamonds': bid(2);  
    case 'hearts': bid(3);  
    case 'spades': bid(4);  
}
```


22 . 重写下面的程序段 , 使用一个case语句代替嵌套的if-else语句。

```
if (W == 5):  
    Z = 7  
else:  
    if (W == 6):  
        Y = 7  
    else:  
        if (W == 7):  
            X = 7
```

23 . 使用一个if-else语句概括下面的例程 :

```
if X > 5 then goto 80  
X = X + 1  
goto 90  
80 X = X + 2  
90 stop
```

24 . 概述在命令式语言和面向对象语言中找到的用于实现下述每个活动的基本控制结构。

- a. 判断接下来应该执行哪一条命令。
- b. 重复一组命令。
- c. 改变变量的值。

25 . 概述翻译器和解释器的区别。

26 . 假设程序中的变量X声明为整型。当执行程序语句

```
X = 2.5
```

时，会发生什么错误？

27．说一种程序设计语言是强类型的是什么意思？

28．为什么一个大的数组不太可能按值传递给函数？

29．假设Python函数Modify的定义为

```
def Modify (Y):  
    Y = 7  
    print(Y)
```

如果参数是按值传递的，那么执行下面这个程序段会打印出什么结果？如果参数是按引用传递的呢？

```
X = 5  
Modify(X)  
print(X)
```

30．假设Python函数Modify的定义为

```
def Modify (Y):  
    Y = 9  
    print(X)  
    print(Y)
```

其中X是全局变量。如果参数是按值传递的，那么执行下面这个程序段会打印出什么结果？如果参数是按引用传递的呢？

```
X = 5  
Modify(X)  
print(X)
```

31. 有时，实参是通过产生一个函数要使用的副本来传递给函数的（如按值传递时），但在函数完成时，函数副本里的值会在调用函数继续执行之前传送给实参。在这种情况下，称参数是按“值—结果”传递的。如果参数是按值—结果传递的，那么执行第30题中的程序段会打印出什么结果？

32. a. 按值传递参数相对于按引用传递参数有什么优点？

b. 按引用传递参数相对于按值传递参数有什么优点？

33. 下面的语句存在什么歧义？

```
X = 3 + 2 × 5
```

34. 假设一个小公司有5名员工，并且计划增加到6名。而且假设下面的赋值语句是该公司的一个程序中的：

```
DailySalary= TotalSal/5;  
AvgSalary  = TotalSal/5;  
DailySales = TotalSales/5;  
AvgSales   = TotalSales/5;
```

那么，如果该程序原来是用名为NumberOfEmp和WorkWeek（值都为5）的常量编写的，那么如何简化更新程序的任务才能把赋值语句表达为：

```
DailySalary = TotalSal/DaysWk;  
AvgSalary   = TotalSal/NumEmpl;  
DailySales  = TotalSales/DaysWk;  
AvgSales    = TotalSales/NumEmpl;
```

35 . a. 形式语言和自然语言的区别是什么？

b. 分别举一个例子。

36 . 用语法图来表示第5章中Python的while语句的结构。

37 . 设计一套用来描述你本地电话号码的语法的语法图。例如，在美国，电话号码由一个分区号码（area code）、一个地区号码（regional code）和一个4位数字组成，如（444）555-1234。

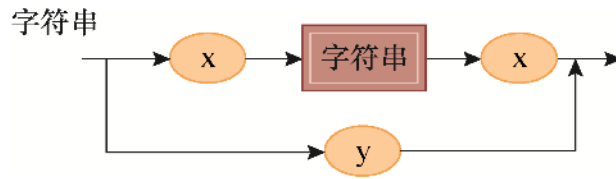
38 . 设计一套用来描述你母语里简单句的语法图。

39 . 设计一套用来描述不同的日期表示方式的语法图，如“月/日/年”或是“月 日，年”。

40 . 设计一套用来描述“句子”的文法结构的语法图，其中“句子”要由单词yes后跟相同数量的单词no组成。例如，句子“yes yes no no”符合要求，而句子“no yes”“yes no no”和“yes no yes”就不符合要求。

41 . 有一种“句子”是这样的：单词yes的后面跟有相同数量的单词no，其后又跟有相同数量的单词maybe。例如，“yes no maybe”“yes yes no no maybe maybe”就是这样的句子，而“yes maybe”“yes no no maybe maybe”“maybe no”就不是。给出一个论据说明，设计不出一套描述上述句子的文法结构的语法图。

42 . 写一个句子来描述下面语法图定义的字符串的结构，然后画出字符串xxyxx的语法分析树。



43．为6.4节中的第5题增加语法图，以得到一套定义Dance结构为Chacha或Waltz的语法图，其中Waltz包括一个或多个以下模式的副本：

forward diagonal close

或

backward diagonal close

44．基于图6-15中的语法图，为表达式 $x \times y + y \div x$ 画出语法分析树。

45．当为下面的语句生成机器代码时，代码生成器可以实现哪些代码优化？

```

if (X == 5):
    Z = X + 2
else:
    Z = X + 4
  
```

46．简化下面的程序段：

```

Y = 5
if (Y == 7):
    Z = 8
else:
    Z = 9
  
```

47．简化下面的程序段；

```
while (X != 5):  
    X = 5
```

48．在面向对象程序设计环境中，类型和类有哪些相似，又有哪些不同？

49．描述如何使用继承来开发描述不同类型建筑的类。

50．类的私有部分与公有部分的区别是什么？

51．a. 给出一个实例变量应该是私有的例子。

b. 给出一个实例变量应该是公有的例子。

c. 给出一个方法应该是私有的例子。

d. 给出一个方法应该是公有的例子。

52．描述在模拟酒店大厅里行人交通的程序中可能发现的一些对象。包括一些对象应该能够完成的动作的解释。

*53. 在程序设计语言语境中，术语[监控程序](#)指什么？

*54. 并发处理的什么属性使得使用支持并发的程序设计语言是可取的？

*55．画一个图（类似于图6-25）表示证明 $(Q \text{ OR } \neg R)$ 、 $(T \text{ OR } R)$ 、 $\neg P$ 、 $(P \text{ OR } \neg T)$ 和 $(P \text{ OR } \neg Q)$ 这组语句是不相容的所需的消解。

*56 . $\neg R$ 、 $(T \text{ OR } R)$ 、 $(P \text{ OR } \neg Q)$ 、 $(Q \text{ OR } \neg T)$ 和 $(R \text{ OR } \neg P)$ 这组语句是相容的吗？请解释你的答案。

*57 . 扩展6.7节中第3题和第4题描述的Prolog程序，包含另外的家庭关系：叔叔（舅舅）、姑姑（姨）、祖父母和堂兄弟姐妹（表兄弟姐妹）。还要增加一条定义 $\text{parents}(X, Y, Z)$ 表示X和Y是Z的父母的规则。

*58. 假设下列Prolog程序中的第一条语句的意图是表达“Alice喜欢运动”，请翻译程序中的最后两个语句。然后，基于这个程序，列出Prolog能够得出的Alice喜欢的所有事情的列表。请解释你的列表。

```
likes(alice, sports).
likes(alice, music).
likes(carol, music).
likes(david, X) :- likes(X, sports).
likes(alice, X) :- likes(david, X).
```

*59. 如果下面的程序段在一台用1.7节中描述的8位浮点格式表示数值的机器上执行，会遇到什么问题？

```
X = 0.01
while (X != 1.00):
    print(X)
    X = X + 0.01
```

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回

答，以及你的判断是否对每个问题都标准如一。

1．通常，版权法支持与一个想法的表达有关的所有权，而不是这个想法本身的所有权。因此，一本书中的段落是受版权法保护的，但是这一段落表述的思想就不受保护。这种权利应该如何扩展到源程序和它们表达的算法呢？一个了解某商业软件包中所使用的算法的人，应该在多大程度上被允许编写他们自己的程序来表达这些相同算法，并将这个软件推向市场？

2．通过使用高级程序设计语言，程序员可以使用诸如if、else和while这样的单词来表达算法。计算机对那些词的含义能理解到什么程度？正确响应这些词的使用的能力意味着理解词语吗？你怎么知道另一个人理解了你所说的？

3．一个开发有用的新程序设计语言的人应该有从这个语言的使用中获利的权利吗？如果有，如何保护这样的权利？一种程序设计语言可以在多大程度上被拥有？公司对其员工的创新智力成果有多大程度上的所有权？

4．在临近最后期限时，一个程序员打算放弃用注释语句编制文档以使程序能够按时完成，这是可接受的吗？（初学者在得知文档对于专业的软件开发人员是何等重要时，往往非常惊讶。）

5．许多程序设计语言的研究都是为了开发出一种能让程序员编写出易于人类阅读和理解的程序的语言。应该在多大程度上要求程序员

使用这些能力？也就是说，对于能够正确实现功能但从人的角度来看写得不好的程序，在什么程度上才算是好程序？

6．假设一个业余程序员写了一个程序供自己使用，但程序的构造比较草率。这个程序没有使用能让程序更易读的程序设计语言特性，效率不高，并且用了利用特殊情况（这个程序员试图使用这个程序的特殊环境）的省事方法。一段时间以后，这个程序员把他的程序复制给了想使用这个程序的朋友，而他的朋友又把这个程序复制给了他们的朋友。这个程序员要为他的程序可能出现的问题负多大责任？

7．计算机专业人员对于各种程序设计范型应该精通到什么程度？某些公司坚持在公司开发的所有软件中使用同一种预先确定好的程序设计语言。如果某计算机专业人员在这种公司工作，你对前面问题的回答是否会发生变化？

课外阅读

Aho, A. V., M. S. Lam, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2007.

Barnes, J. *Programming in Ada 2012*. Cambridge, England: Cambridge University Press, 2014.

Clocksin, W. F. and C. S. Mellish. *Programming in Prolog*, 5th ed. New York: Springer-Verlag, 2013.

Friedman, D. P. and M. Felleisen. *The Little Schemer*, 4th ed.
Cambridge, MA: MIT Press, 1995.

Kernighan, B.W. and D.M. Ritchie. *The C Programming Language*,
2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

Metcalf, M., J. Reid and M. Cohen. *Modern Fortran Explained*, 4th
ed. Oxford, England: Oxford University Press, 2011.

Pratt, T. W. and M. V. Zelkowitz. *Programming Languages, Design
and Implementation*, 4th ed. Upper Saddle River, NJ: Prentice-Hall, 2001.

Richards, D. and H. Hamburger. *Logic and Language Models for
Computer Science*, 3rd ed. Hackensack, NJ: World Scientific, 2017.

Savitch, W. and K. Mock. *Absolute C++*, 6th ed. Essex, England:
Pearson, 2016.

Savitch, W. and K. Mock. *Absolute Java*, 6th ed. Essex, England:
Pearson, 2016.

Savitch, W. and K. Mock. *Problem Solving with C++*, 10th ed. Essex,
England: Pearson, 2017.

Scott, M. L. *Programming Language Pragmatics*, 4th ed. Waltham,
MA: Morgan Kaufmann, 2016.

Sebesta, R. W. *Concepts of Programming Languages*, 11th ed. Essex, England: Pearson, 2016.

Wu, C. T. *An Introduction to Object-Oriented Programming with Java*, 5th ed. Burr Ridge, IL: McGraw-Hill, 2009.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第7章 软件工程

本章讨论的是在开发大型的复杂软件系统过程中遇到的问题。本章的主题之所以称为**软件工程**，是因为软件开发是一个工程化的过程。研究软件工程的目标就是要找到一些原则来指导软件开发过程，进而生产出高效可靠的软件产品。

本章内容

7.1 软件工程学科

7.2 软件生命周期

7.3 软件工程方法学

7.4 模块化

7.5 行业工具

7.6 质量保证

7.7 文档

7.8 人机界面

7.9 软件所有权和责任

持久理解和学习成果

可以开发程序用于创造性地表达、满足个人好奇心、创造新知识或解决问题（以帮助人们、组织或社会）。AP[®]CSP

学习成果 开发正确的程序来解决问题。

学习成果 协作开发程序。

人们出于不同的目的开发、维护和使用程序。AP[®]CSP

学习成果 评估程序的正确性。

软件工程是与构建满足用户需求的可靠软件系统有关的知识体。

学习成果 区分软件开发的各个阶段。

学习成果 描述大规模程序设计与个人程序设计有何不同。

学习成果 解释软件生命周期的概念。

学习成果 解释软件系统中模块化的重要性。

软件工程是计算机科学的一个分支，致力于寻找指导大型复杂软件系统的开发原则。开发这类系统所面对的问题并非只是编写小程序所面对问题的放大。例如，开发大型系统的时候，要求许多人工作很长时间，而在这期间，预期的系统需求可能会改变，参与该项目的人员也可能会变动。因此，软件工程包括诸如人员管理和项目管理之类

的主题，与计算机科学相比，这样的主题与业务管理的关系更密切。当然，我们的侧重点还是放在那些与计算机科学密切相关的主题上。

7.1 软件工程学科

为了有助于理解软件工程中涉及的问题，这里可以想象构造一个大型的复杂设施（一辆汽车、一幢多层办公大楼或者一座教堂），对其进行设计，然后监管其构建过程。如何估算完成该项目所需的时间、费用以及其他资源？如何把项目划分为几个便于管理的模块？如何保证构建的模块是兼容的？工作在不同模块的人如何沟通？如何衡量进度？如何妥善处理更广泛的细节（如门把手的选择、滴水兽的设计、用于彩色玻璃窗的蓝色玻璃的可用性、柱子的强度、供暖系统的管道设计）？在一个大型软件系统的开发过程中，同样需要面对如此繁多的问题。

因为工程是一个很成熟的领域，所以有人可能认为有大量现成的可以用来解决软件工程中的这些问题的工程技术。这种推理有一定的道理，但是忽略了软件的特性与其他工程领域特性之间的根本区别。这些区别已经向软件工程项目提出了挑战，导致成本超支、推迟交付软件产品和软件产品不能满足用户的需求等后果。所以，在发展软件工程学科上，首先要做的就是弄清这些差别。

差别之一涉及通过通用预制组件来构建系统的能力。一些传统的工程领域长期受益于这种能力，即在构建复杂设备时用“现成的”组件作为构建块。例如，设计新车时，不必设计新的引擎或变速器，利用

这些部件以前的设计方案即可。然而，软件工程在这方面是很落后的。过去，以前设计的软件组件一般用于特定的领域（也就是说，它们的内部设计是基于特殊应用的），所以将它们作为通用组件来使用是受限的。结果是，复杂的软件系统历来都是从头做起。正如在本章中我们将看到的那样，在这一点上已经取得了重大进展，尽管还有很多工作要做。

软件工程学科与其他工程学科之间的另一个差别在于，软件工程学科缺少用来衡量软件属性的定量技术——**度量学**（metrics）。例如，为了计算开发一个软件系统的成本，人们希望能够估算出拟建产品的复杂性，但是测量软件“复杂性”的方法还不太成熟。同样，评价软件产品的质量也具有挑战性。对于机器设备，质量的重要量度是平均无故障时间，这是对设备耐损耗性的一个基本衡量指标。而软件与此相反，它没有损耗，所以这种方法在软件工程中并不适用。

以定量方式测量软件属性涉及的困难是软件工程难以找到与机械和电子工程相同意义上的严格、坚实的立足点的一个原因。尽管机械和电子工程是建立在成熟的物理科学的基础上的，但是软件工程仍然在找寻其自身的根基。

因而，现在的软件 engineering 研究在两个层面上进行：一部分研究者（有时也称为实践派）的工作指向开发直接应用的技术；另一部分研究者（称为理论派）则致力于探寻软件工程的基础原理和理论，为将来构建更稳定的技术而努力。基于主观基础，实践派以前开发和提出

的许多方法已经被其他方法代替，新方法本身可能也将随着时间的推移而被淘汰。与此同时，理论派的进展一直很缓慢。

实践派和理论派这两方面进展的需求都是巨大的。我们这个社会已经沉迷于计算机系统及其相关软件。我们的经济、医疗保健、政府、执法、交通运输和国防系统等都依赖于大型软件系统。然而，这些系统的可靠性依然是主要问题。软件错误已经导致了一些大的灾难，新近的灾难如月亮的升起被误以为是核攻击、纽约银行造成的一天损失500万美元、空间探测器的失踪、过量辐射导致的人员伤亡，还有电话通信在同一时间大面积瘫痪等。

这并不是说情况都很悲观。我们已经在解决诸如缺少预制组件和衡量标准等问题方面取得了很大的进展。此外，计算机技术在软件开发过程中的应用，导致了**计算机辅助软件工程**（computer-aided software engineering，CASE）的出现，这使软件开发流程化，简化了软件的开发过程。CASE已经促进了各种称为**CASE工具**（CASE tool）的计算机化系统的发展，这些系统包括项目设计系统（用来辅助成本估算、项目调度以及人员分配等）、项目管理系统（用来辅助监控项目的开发进度）、文档工具（用来辅助编写和组织文档）、原型与仿真系统（用来辅助原型的开发）、界面设计系统（用来辅助图形用户界面的开发）和程序设计系统（用来辅助编写和调试程序）等。其中一些工具的功能和字处理程序、电子表格软件和电子邮件通信系统等差不多，最开始是为一般应用开发的，现在已为软件工程师所采用。另外的一些工具主要是为软件工程环境专门设计的相当复杂

的软件包。实际上，称为**集成开发环境**（integrated development environment，IDE）的系统把软件开发工具（编辑器、编译器、调试工具等）都组合到了单个的集成包中，其中，智能手机开发所应用的系统便是这类系统的典型代表。这类系统不仅提供编写和调试软件所必需的程序设计工具，而且提供模拟器（借助于图形显示）让程序设计人员查看正在开发的软件在手机上是如何实际执行的。

除了研究人员、专业人士和标准化组织（包括ISO）的努力，美国计算机协会（ACM）和美国电气电子工程师学会（IEEE）也已经加入了改善软件工程状态的战斗中。这些努力包括采用职业操守和职业道德增强软件开发人员的职业精神，反对对个人职责的漠视态度；建立衡量软件开发组织质量的标准，提供帮助这些组织提高其排名的指导方针。

美国计算机协会

美国计算机协会（Association for Computing Machinery，ACM）成立于1947年，是致力于推动艺术、科学及信息技术应用的国际性科学与教育组织。其总部在纽约，下设许多特别兴趣组（SIG），分别致力于计算机体系结构、人工智能、生物医学计算、计算机与社会、计算机科学教育、计算机图形学、超文本/超媒体、操作系统、程序设计语言、模拟与建模，以及软件工程等主题。

美国电气电子工程师学会

美国电气电子工程师学会（Institute of Electrical and Electronics Engineers，IEEE，读作“i-triple-e”）是一个电气、电子和制造工程师

的组织，成立于1963年，由美国电气工程师学会（由包括托马斯·爱迪生在内的25名电气工程师于1884年创建）和美国无线电工程师学会（创建于1912年）合并而成。今天，IEEE的运营中心位于新泽西州的皮斯卡塔韦镇。学会由许多技术分会组成，如航空电子系统学会、激光与电光学学会、机器人与自动化学会、车载技术学会以及计算机学会。IEEE也参与了各种标准的制定。例如，IEEE的努力产生了今天在大多数计算机上使用的单精度浮点数标准和双精度浮点数标准（第1章中介绍过）。

设计构建大型软件系统的首要挑战之一是这项工作必然涉及许多人。虽然小型系统可以由一个确定的人设计，但许多工作人员必须能够有效沟通和协作是大型软件工程项目本质。分阶段合作大型软件项目需要一套不同于独立编码的但可以降低每个个人任务复杂度的技能。事实上，这是因为大型软件系统非常复杂，所以可能有必要将系统分解成更小的组成部分，这么做只是为了让开发人员彻底了解他们的那部分工作。

基本知识点

- 协作可以减小单个程序员需要完成的任务的大小和复杂度。
- 程序迭代开发中的协作需要与单独开发程序不同的技能。
- 开发程序时的成功协作需要参与者之间的有效沟通。

本章其余部分将讨论软件工程的基本原理（如软件生命周期和模块化），介绍软件工程的发展动向（如设计模式的定义与应用，以及可复用软件组件的出现），考察面向对象范型对这个领域产生的影响。

7.1 节问题与练习

1. 为什么程序的代码行数不是好的衡量程序复杂性的标准？
 2. 提出一种测量软件质量的度量标准，并说明这种度量标准有什么缺点？
 3. 什么样的方法能用来确定一个软件单元中有多少错误？
 4. 列举出在软件工程领域中已经或当前正在改进的两个语境。
-

7.2 软件生命周期

软件工程最基础的概念是软件生命周期。

7.2.1 周期是个整体

图7-1展示的是软件生命周期，这个图表明了一个事实，即软件一旦开发完成，就进入了一个使用和维护的周期，这个周期将一直持续到软件生命结束。这种模式在许多工业产品中也很常见。不同之处在于，对于其他产品，维护阶段往往是一个修复过程，而对于软件，维护阶段往往包括改错和更新。实际上，软件进入维护阶段是因为发现

了错误，软件应用中发生的变化需要在软件中做相应的修改，或者上一次修改中的变更导致软件中其他地方出现了问题。

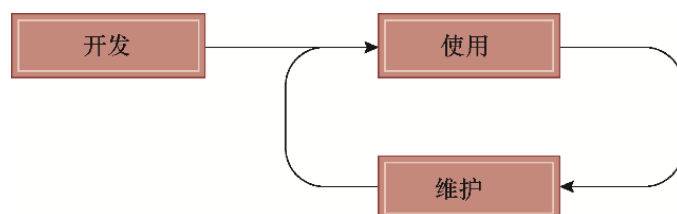


图7-1 软件生命周期

无论软件因为什么样的原因进入维护阶段，这个过程都需要有人（通常不是原作者）研究底层的程序及其文档，直至把这个程序或者至少是这个程序的相关部分理解清楚；否则，任何的修改只会带来比它解决的问题更多的问题。即使软件设计精良并有良好的文档，要达到这种理解也是一件困难的事情。事实上，到了这个阶段，软件的某部分往往会因为“从头开发一个新系统要比成功修改现有软件包更容易”这样一个借口（这个借口通常是真实的）而弃之不用。

经验表明，在软件开发期间稍作努力，就可能会在需要对软件进行修改时产生巨大的影响，例如，在第6章对数据描述语句的讨论中，我们可以看出，与使用字面量相比，使用常量会大大简化未来的修改工作。结果是，软件工程的大部分研究工作都集中在软件生命周期的开发阶段，其目标就是利用这个努力与收益之间的杠杆作用。

7.2.2 传统的开发阶段

传统的软件开发生命周期的主要步骤是需求分析、设计、实现和测试（见图7-2）。

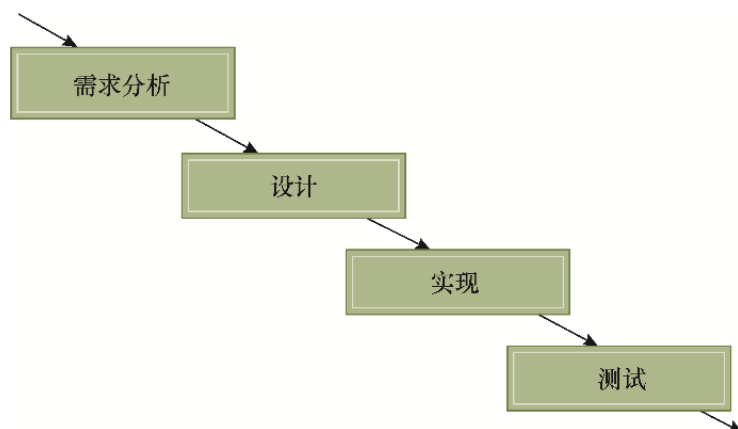


图7-2 传统的软件生命周期的开发阶段

基本知识点

- 程序员在解决问题时，要设计、实现、测试、调试和维护程序。

1. 需求分析

软件生命周期从需求分析开始，需求分析的目标是：指定预期系统要提供的服务，确认这些服务的条件（如时间限制、安全性等）；定义外界与系统的交互方式。

需求分析涉及拟建系统的**利益相关者**（stakeholder，将来的使用者，还有其他有关联的人，如法律上或者财务上相关的人）提供的重要数据。事实上，在一些情况下，终端用户是一个实体（如公司或政府机构），他们倾向于为软件项目的实际执行雇用软件开发人员，需

求分析可能会开始于用户独自进行的可行性研究。在其他一些情况下，软件开发人员可能会为大众市场生产**商用现货**（commercial off-the-shelf，COTS）软件，这些软件可能会在零售商店销售，也可能通过因特网下载。在这种情况下，用户是不太准确定义的实体，需求分析可能要从软件开发者的市场调研开始。

无论是哪一种情况，需求分析过程都包括：收集和分析软件用户的需求；和项目的利益相关者协商，在一般需求、核心需求、费用和可行性之间权衡；最终确定的需求要明确最终的软件系统必须具有的特性和服务。这些需求被记录在一个称为**软件需求规格说明**（software requirements specification）的文档中。从某种意义上讲，这个文档是所涉及的各方之间达成的书面协议，它的目的是指导软件开发，为日后开发过程中可能产生的分歧提供解决方法。像IEEE这样的专业组织和美国国防部这样的大型软件客户都已经采用了软件需求规格说明文档编写的标准，这些事实证明，软件需求规格说明文档十分重要。

从软件开发人员的角度来看，软件需求规格说明文档应该能够为软件开发的顺利进行制定严格的目标。然而，大多数情况下，需求文档很难提供这种稳定性。事实上，软件工程领域里的大多数实践者都认为：在软件工程产业中，导致成本超支和产品交付延期的最主要原因是不良沟通以及不断变化的需求。在地基已经打好的情况下，很少有客户会坚持对楼盘的建设计划做大的修改；但是许多组织机构在软件开始构建以后很久，仍然不断扩大或变更软件系统需求的实例比比皆是。这可能是由于公司决定把原本仅为附属机构开发的软件系统应

用到整个公司，或者是技术的进步取代了初始需求分析中可用的功能。无论如何，软件工程师们发现，必需频繁与项目的利益相关者直接沟通。

基本知识点

- 与程序用户的协商和沟通是程序开发解决问题的重要方面。
- 程序开发包括确定影响问题解决方案的程序员和用户问题。

2 . 设计

如果说需求分析提供了对一个预期软件产品的描述，那么设计主要是为预期系统的构建制订计划。从某种意义上讲，需求分析是指明要解决的问题，而设计则是要制订问题的解决方案。从一个外行人的视角来看，需求分析常常等同于决定软件系统应该做些什么，而设计则是决定系统如何做。虽然这种描述是有启发性的，但很多软件工程师认为它是有缺陷的，因为实际上在需求分析阶段有很多如何要考虑，在设计阶段也有很多什么要考虑。

软件系统的内部结构是在设计阶段建立的。设计阶段的结果是可以被转化为程序的软件系统结构的详细描述。

如果项目是建造一座办公大楼，而不是构建一个软件系统，那么在设计阶段应该为大楼制订能满足指定需求的详细结构计划。例如，这样的计划应该包含在各个细节层次上描述拟建大楼的蓝图汇总。有

了这些文档，才能建造实际的大楼。制订这些计划的技术已经发展了很多年，包括标准的符号系统以及大量的建模和图形化方法学。

同样，在软件的设计中，图形化和建模也发挥着重要的作用。然而，软件工程师所用的方法学和符号系统不如建筑领域里的稳定。与建筑学这个成熟的学科相比，随着研究人员努力寻找软件开发过程中更好的方法，软件工程的实践显得非常有活力。我们将在7.3节探究这个不断变化的领域，在7.5节研究一些当前的符号系统以及与它们相关的图形化/建模方法学。

3 . 实现

实现涉及程序的实际编写、数据文件的创建和数据库的开发。在实现阶段，我们会看到**软件分析员**（software analyst，有时称为系统分析员）和**程序员**（programmer）之间任务的不同。软件分析员参与整个开发过程，其工作重点可能在需求分析和设计步骤，而程序员的主要工作是实现这些步骤。狭义地说，程序员负责写程序来实现软件分析员提出的设计。尽管我们做了这样的区分，但我们还要注意，在计算机领域里并没有一个中央机构控制术语的使用。许多有着软件分析员头衔的人，本质上就是程序员，而许多有着程序员（也许是高级程序员）头衔的人，从完全意义上讲实际是软件分析员。我们很快就可以看到，术语上的这种模糊是因为现在软件开发过程中的步骤经常会交叉重叠。

4 . 测试

在过去传统的开发阶段中，测试本质上等同于调试程序和确认最终的软件产品是否与软件需求规格说明文档相一致的过程。但是如今，这样的测试观念被认为太过狭隘。程序不是唯一在软件开发过程中被测试的人工制品，实际上，整个开发过程中的每个中间步骤的成果都必须进行准确性“测试”。此外，我们将在7.6节中看到，测试现在被认为是为全面保证质量所做努力中的一部分，这一目标渗透于整个软件生命周期。因此，很多软件工程师认为测试不应该再被看作软件开发过程中独立的一步，而应该将其及其许多表现形式纳入其他步骤中，形成一个三步开发过程，其中每一步都应该有自己的名称，如“需求分析和确认”、“设计和验证”和“实现和测试”。

遗憾的是，尽管有现代的质量保障技术，大型软件系统还是会有错误，即使经过大量的测试也不能避免。其中许多错误可能在软件的生命周期内都检测不出来，但是另一些错误却可能会造成重大的故障。消除这种错误是软件工程的目标之一。错误仍然普遍存在的事实意味着还有许多研究要做。

7.2节问题与练习

1. 软件生命周期的开发阶段是如何影响维护阶段的？
2. 简要说明软件生命周期的开发阶段的4个步骤（需求分析、设计、实现和测试）。
3. 软件需求规格说明的作用是什么？

7.3 软件工程方法学

软件工程早期的方法强调以一个严格的顺序进行需求分析、设计、实现和测试。理由是，允许大型软件系统在开发过程中进行变更很危险。因此，软件工程师坚持：在开始设计之前必须先完成整个系统的需求分析，而且，在开始实现之前必须先完成设计。结果产生了一个现在称为**瀑布模型**（waterfall model）的软件开发过程，之所以称这种开发过程为“瀑布”模型，是因为它跟瀑布一样，只能向一个方向“流动”。

近年来，瀑布模型规定的高度结构化环境与“自由发挥”的试错过程（通常对创造性的问题求解至关重要）之间的矛盾，带来了软件工程技术的变化。软件开发中出现的**增量模型**（incremental model）就说明了这一点。依据这个模型，所需的软件系统以一种增量的模式构建，即软件产品先是以功能有限的简化版本出现，一旦这个版本的系统通过了测试并且可能还经过了未来用户的评估，就可以以递增的方式不断地向系统中添加更多的功能并测试，直至整个系统全部完成。例如，要为医院开发一个病人记录系统，第一个增量版本只需要能够查看整个记录系统中的一小部分病人的记录样本就可以了，一旦这个版本能够工作，就可以逐步向系统中加入其他功能，如增加和更新记录的功能。

基本知识点

- 给正确的工作程序逐渐增加测试程序段，有助于创建正确的大型程序。

另外一种与严格遵循瀑布模型不同的模型是**迭代模型**（iterative model），尽管它与增量模型是两个不同的概念，但它们是相似的，实际上有时它们是相同的。增量模型使用的是**扩展**产品的每个前期版本到更大版本的概念，而迭代模型使用的是**改进**每个版本的概念。实际上，增量模型包含一个基本的迭代过程，而迭代模型可以增量增加功能。

基本知识点

- 程序开发的迭代过程有助于开发解决问题的正确程序。

一个重要的迭代技术的例子是Rational软件公司创造的**统一软件开发过程**（rational unified process，RUP，与“cup”谐音），现在这家公司是IBM的一个分公司。RUP在本质上是一种软件开发范型，它重新定义了软件生命周期中开发阶段的每一个步骤，并提供了执行这些步骤的指导。IBM公司将这些指导和支持这些指导的CASE工具当作商品出售。今天，RUP已广泛应用于整个软件行业。实际上，它的流行促进了非专利版本——**统一过程**（unified process）的发展，这是非商业的。

增量模型和迭代模型有时会利用软件开发朝着**原型开发**（prototyping）发展的趋势。原型开发是构建并评估预期系统的非完整版本（称为原型）的过程。在增量模型中，将这些原型发展为一个最终的完整系统的过程称为**演化式原型开发**（evolutionary prototyping）。在迭代性更强的情况下，可能会丢弃原型，以使最后

设计有全新的实现，这种方法称为**抛弃式原型开发**（throwaway prototyping）。**快速原型开发**（rapid prototyping）通常属于抛弃式原型开发这个范畴，这种方法是在开发的早期阶段快速构建一个预期系统的简单原型。这个原型可能只由几个屏幕图像组成，用来演示系统将如何与其用户交互以及系统将有哪些功能。其目标不是产生一个可运行的产品版本，而是获得一个示范工具，用来澄清软件开发过程所涉及的各个部分之间的通信。例如，事实证明，快速原型有利于在需求分析阶段澄清系统需求，也能帮助在销售期间向潜在客户进行推销介绍。

计算机爱好者/业余爱好者使用多年的增量和迭代思想的一种不太正式的变体，称为**开源开发**（open-source development）。这是今天许多自由软件开发采用的一种方式。最著名的例子也许就是Linux操作系统，该系统的开源开发工作最初是由林纳斯·托瓦兹（Linus Torvalds）领导的。软件包的开源开发遵循以下过程：先是单个作者编写软件的初始版本（通常是用于满足作者自己的需求），然后将其源代码和相关文档发布到因特网上。其他用户可以从因特网上免费下载和使用这个软件。由于这些其他用户拥有该软件的源代码和文档，所以他们能够修改或增强这个软件的功能使之满足自己的需要，或者改正他们发现的错误。他们将这些改动报告给原作者，原作者再将这些改动整合到自己发布的软件版本中，使软件的扩展版本可用于进一步的修改。实际上，软件包可能在一个星期内就要经过几次扩展演变。

由瀑布模型转化而来的变化最显著的方法是称为**敏捷方法**（ agile method ）的方法学集合，这些方法学都建议在增量的基础上进行早期快速的实现，响应需求变更，降低对严格需求分析和设计的强调。敏捷方法的一个例子是**极限编程**（ extreme programming , XP ）。根据XP模型，由一个不到12人的团队在一个公共的工作场所自由地交换想法，在开发项目过程中相互协作，通过每天不断重复“非正式需求分析、设计、实现和测试”这样一个周期的方式，增量开发软件。这样，软件包的新扩展版本就能定期出现，每个新版本都能由项目的利益相关者进行评估，并以此为基础做进一步的增量开发。概括说来，敏捷方法具有灵活性的特点，这与瀑布模型形成鲜明对比，瀑布模型让人联想到经理和程序员在各自的办公室工作，各自严格地完成整个软件开发任务中明确定义的那部分工作。

通过瀑布模型与XP模型的对比，揭示了软件工程方法学的广度，这些方法学正被应用于软件开发过程，希望能找到一种更好的高效构建可靠软件的方式。这个领域的研究仍在继续，虽然取得了一定的进展，但还有许多工作要做。

7.3节问题与练习

- 1．概述软件开发的传统的瀑布模型与较新的增量和迭代范型的区别。
- 2．说出3种不再严格遵循瀑布模型的开发范型。
- 3．传统的演化式原型开发与开源开发的区别是什么？

4. 对于通过开源方法开发的软件的所有权，你认为可能会出现什么样的潜在问题？

7.4 模块化

7.2节中的一个关键点是：要修改软件，就必须理解这个程序，或者至少理解这个程序的相关部分。即使是小程序，要达到这样的理解一般已经够难了，对于大型软件系统，如果没有**模块化**（modularity），那几乎是不可能的。模块化就是把软件分割成多个易于处理的通常称为**模块**（module）的单元，每个单元仅承担整个软件的一部分职责。

7.4.1 模块式实现

模块可以以不同的方式实现。我们已经看到（在第5章和第6章中），在命令型范型的语境中，模块表现为函数。与之对应的是，面向对象范型则是利用对象作为其基本的模块要素。这些差别很重要，因为它们决定了最初的软件设计过程中的潜在目标。这个目标是将全部任务表示为个别的、易于管理的过程，还是确定系统中的对象并理解它们之间是如何相互作用的？

为了说明这一点，我们来考虑用命令型范型和面向对象范型是如何开发一个模拟网球比赛的简单模块化程序的。在命令型范型中，我们先考虑肯定会发生的动作。因为每场网球比赛都是从一名选手发球

开始，所以我们可以先考虑构造名为Serve()的函数（基于选手的特点，可能还有一点儿概率），用来计算球的初始速度和方向。接下来，我们需要确定球的路径。（球会撞在网上吗？它将弹回到什么地方？）我们可以把这些计算放在另外一个名为ComputePath()的函数中。下一步可能就要确定另外一名选手是否能击回这个球。如果能够击回这个球，我们还必须计算球的新的速度和方向。我们可以把这些计算放在名为Return()的函数中。

照这样继续，我们可以构造出图7-3所示的**结构图**（structure chart）所描述的模块化结构。在这个图中，矩形表示函数，箭头表示函数之间的依赖关系（由函数调用来实现）。具体来说，这个图表明了整个比赛是由名为ControlGame()的函数来控制的。为了完成任务，ControlGame()函数又调用了函数Serve()、Return()、ComputePath()和UpdateScore()这4个函数的服务。

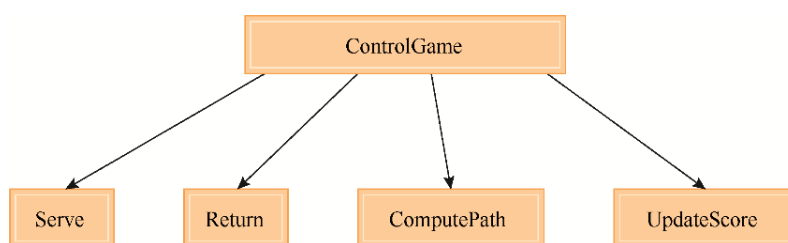


图7-3 一个简单的结构图

注意，这个结构图中并没有描述每个函数是如何完成自己的任务的，确切地说，这个图只是确定了函数并描述了函数之间的依赖关系。事实上，ControlGame()函数要完成自己的任务可能会先调用Serve()函数，然后反复调用ComputePath()函数和Return()函

数，直到有一名选手没有击中球为止。最后，调用UpdateScore()函数，之后通过再次调用Serve()函数重复整个过程。

至此，我们只获得了所需程序的一个极其简单的轮廓，但思路已经建立起来了。按照命令型范型，通过构思系统必须实现的活动，我们已经完成了程序的设计并因此得到了其中模块为函数的设计。

现在，我们重新考虑这个程序的设计，这次是在面向对象范型的语境中考虑的。我们开始的想法就是用两个对象PlayerA和PlayerB来表示两位选手。这两个对象将具有相同的功能但具有不同的特点。（两名选手应该都能发球和截击球，但是其技巧和力度不同。）因此，这两个对象将是同一个类的实例。[回忆一下，我们在第6章介绍了类的概念：类是定义与每个对象相关联的函数（称为方法）和属性（称为实例变量）的模板。]我们把这个类称为PlayerClass，它将包含Serve()方法和Return()方法，用来模拟选手的相应动作。这个类中还将包含属性（如skill和endurance等），这些属性的值反映了选手的特点。到目前为止，我们的设计结果如图7-4所示。从图中可以看出，PlayerA和PlayerB是PlayerClass类的两个实例，而这个类包含了Skill属性和Endurance属性，也包含了serve()方法和returnVolley()方法。（注意，在图7-4中，为了区分对象名和类名，我们在对象名下面加了下划线。）

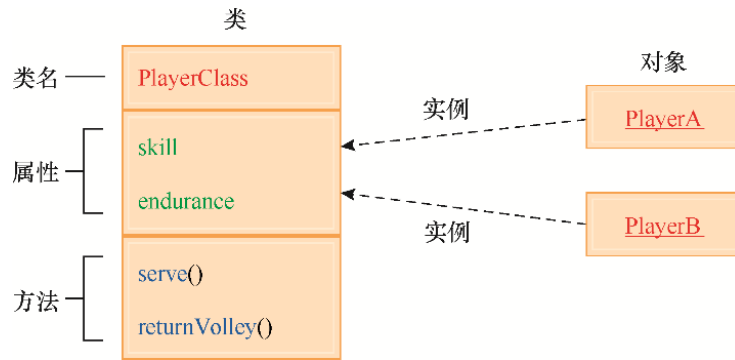


图7-4 PlayerClass类的结构和它的实例

接下来，我们需要一个对象来扮演裁判的角色，判定选手完成的动作是否合乎规则。例如，发球过网并落在球场的合适区域内了吗？为此，我们可以建立一个名为Judge的对象，该对象包含 `evaluateServe()` 方法和 `evaluateReturn()` 方法。如果Judge对象判定发球或截击球合乎规则，那么比赛继续；否则，Judge对象会给另一个名为Score的对象发消息，告知它记录相应的结果。

此时，我们的网球程序的设计包括4个对象，即PlayerA、PlayerB、Judge和Score。为了阐明我们的设计，考虑一下在截击中可能发生的事件序列，如图7-5所示，图中我们用方框表示涉及的对象。这个图是要把这些对象之间的通信表示成调用对象PlayerA的 `serve()` 方法的结果。当我们从上向下依次看图时，会发现事件是按次序发生的。就如同第一个水平箭头所表示的那样，PlayerA通过调用 `evaluateServe()` 方法向Judge对象报告它的发球，然后Judge对象判定发球是否有效，并通过调用PlayerB的 `returnVolley()`

方法请求PlayerB截击球。当Judge判定PlayerA出现错误并请求Score对象记录结果时，截击结束。

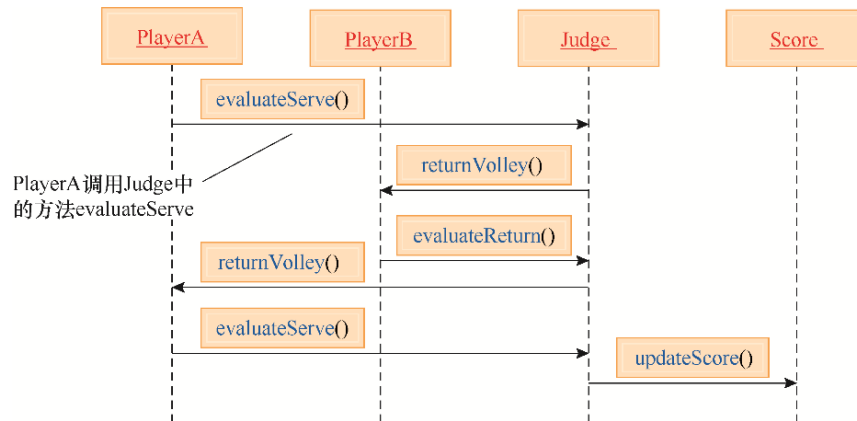


图7-5 由PlayerA的发球引发的对象间的交互

和我们命令型范型例子的情况一样，我们的面向对象程序现阶段也是极其简单的。然而，我们已经取得了很大的进步，能够很清楚地理解面向对象范型是如何产生模块化设计的，而在此模块化设计中其基本组件是对象。

7.4.2 耦合

通过前面的介绍我们知道，模块化是开发出易于管理的软件的一条途径。其基本思想是，以后的任何修改可能只适用于少数几个模块，允许个人对系统的修改只集中在系统的有关部分，而不是整个系统。当然，这里有个前提，就是对一个模块的修改不会无意中影响到系统的其他模块。因此，在设计一个模块化系统的时候，其目标就应该是做到模块之间的最大独立性，或者换句话说，就是使两个模块之

间的联系尽可能少，这种联系称为模块间的**耦合**（coupling）。事实上，用于度量软件系统复杂性（并且因此获得了一种估算维护软件的开销的手段）的一个指标就是度量该系统模块间耦合。

模块间耦合有多种形式。一种是**控制耦合**（control coupling），出现在一个模块移交执行控制给另外一个模块的时候，如函数调用的情况。图7-3中的结构图就表示了存在于两个函数之间的控制耦合。具体来说，从ControlGame()模块到Serve()模块的箭头表明前者将控制权交给后者。图7-5中表示的也是控制耦合的情况，图中的箭头所描绘的路径代表了控制权在对象间的传递。

模块间耦合的另一种形式是**数据耦合**（data coupling），它指的是两个模块之间的数据共享。如果两个模块是通过共享同一个数据项相互作用的，那么对一个模块的修改可能会影响另一个模块，并且对数据本身格式的修改在这两个模块中都会有反映。

两个函数之间的数据耦合有两种形式。一种是以参数的形式从一个函数到另一个函数进行显式的数据传送。这种耦合在结构图中是用两个函数之间的箭头指示数据的传送，箭头的方向表明数据项传递的方向。例如，图7-6是图7-3的扩展版本，从图7-6中我们可以看出：当ControlGame()函数调用Serve()函数时，ControlGame()函数会将需要模拟的选手的特点告知Serve()函数；当Serve()函数完成它的任务时，会将球的轨迹报告给ControlGame()函数。

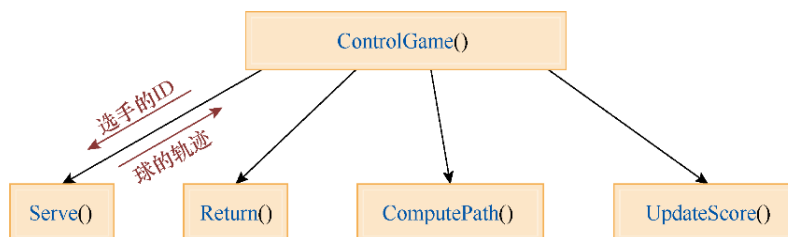


图7-6 一个包含数据耦合的结构图

类似的数据耦合也发生在面向对象设计中的两个对象之间。例如，当PlayerA对象请求Judge对象对其发球进行判定时（见图7-5），它必须将球轨迹信息传递给Judge对象。另一方面，面向对象设计范型的其中一个优势就在于它从本质上倾向于将两个对象之间的数据耦合减小到最低。这是因为一个对象中的方法倾向于包括操作这个对象内部数据的所有函数。例如，PlayerA对象将包括有关该对象特点的信息和需要这个信息的方法。因此，没有必要将这些信息传递给其他对象，这样对象间数据耦合就最小化了。

与将数据作为参数显式传递相反，数据可以以**全局数据**（global data）的形式在模块之间进行隐式共享。全局数据是可以自动被整个系统中的所有模块使用的数据项。这与局部数据项不同，局部数据项只能在某个特定的模块中使用，除非显式地传递给了另一个模块。大多数高级语言都提供了全局数据和局部数据的实现方式，但是对全局数据的使用应当谨慎。使用全局数据的问题在于，如果某个人试图修改依赖于全局数据的一个模块，那么他就很难确定正被修改的模块与其他模块之间有怎样的相互关系。简而言之，全局数据的使用降低了模块作为一种抽象工具的使用价值。

7.4.3 内聚

正如模块间的耦合应最小化，同样重要的是，每个模块的内部绑定程度应该最大化。术语**内聚**（cohesion）就用来表示这种内部绑定，或者说模块内部各部分的关联程度。要充分理解内聚的重要性，我们必须考察系统的最初开发并考虑这个软件的整个生命周期。如果有必要在模块中做出修改，那么存在于模块中的各种不同的活动会搅乱原本简单的一个过程。因此，软件设计人员在寻求模块间的低耦合的同时，还力求做到模块内部的高内聚。

一种较弱的内聚形式称为**逻辑内聚**（logical cohesion）。模块内的逻辑内聚是由其内部元素本质上实现逻辑上相似的活动所引起的。例如，考虑一个模块，它完成整个系统与外界进行通信的功能。黏合这个模块的“胶水”是模块中的所有活动都用以处理通信。然而，通信的各个主题差别很大，有些可能是用来获取数据的，其他一些可能是用来报告结果的。

一种较强的内聚形式称为**功能内聚**（functional cohesion），即模块中所有部分都聚焦于某一活动的性能。在命令型范型的设计中，功能内聚的程度通常可以通过把其他模块中的子任务独立出来用作抽象工具来增强。这一点在网球模拟的例子中得到了很好的说明（再次参见图7-3），在该示例中ControlGame()模块将其他模块用作抽象工具，以便它能专注于监督比赛，而不是把精力分散在实现发球、截击球和维护比分这样的细节上。

在面向对象设计中，因为对象中的方法常常执行松散相关的活动，这些活动唯一的共同“纽带”就是它们都是由同一个对象执行的活动，所以全部的对象通常都是逻辑上内聚的。例如，在网球模拟示例中，每个选手对象都包含发球和截击球的方法，这些方法是明显不同的活动。因此这样一个对象只是一个逻辑上内聚的模块。然而，软件设计人员应该力求做到使一个对象中的每个方法都在功能上内聚。也就是说，即使对象在整体上仅仅是逻辑上内聚，对象里的每个方法也应当只实现一个功能内聚的任务（见图7-7）。

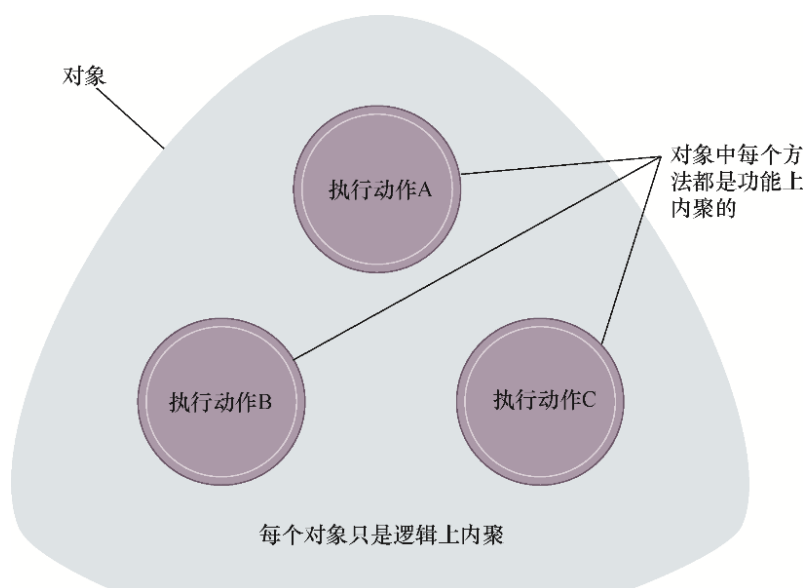


图7-7 一个对象的逻辑内聚和功能内聚

7.4.4 信息隐藏

信息隐藏（information hiding）是好的模块化设计的基石，它指的是限制软件系统的指定部分的信息。这里的术语信息应该从广义阐释，它包括关于程序单元结构和内容的任何知识。因此，它包括数

据、用到的数据结构类型、编码系统、模块的内部组成结构、过程单元的逻辑结构和任何涉及模块内部特性的其他因素。

信息隐藏的关键是阻止模块的动作对其他模块产生不必要的依赖或影响。否则，模块的有效性就可能会受到影响，可能是受其他模块开发中错误的影响，也可能是受软件维护期间不正确的维护的影响。例如，如果一个模块不限制其他模块对其内部数据的使用，那么这个模块内部的数据可能就会被其他模块破坏。或者，如果一个模块被设计成要利用另一个模块的内部结构，那么后面在另一个模块的内部结构被修改时，这个模块可能会产生错误。

重要的是要注意到信息隐藏具有两个化身：一个作为设计目标，另一个作为实现目标的。应当这样设计一个模块：使其他模块不需要访问它的内部信息，并且应当以强化模块边界的方式实现模块。前者的例子是最大化内聚和最小化耦合，后者的例子涉及使用局部变量、应用封装和使用定义明确的控制结构。

最后，我们应该注意到，信息隐藏是抽象主题和抽象工具使用的核心。实际上，抽象工具的概念是“黑盒”的概念，其用户可以忽略它的内部特性，这样就允许用户集中考虑手头更大的应用。从这个意义上讲，信息隐藏相当于封装抽象工具的概念，就像安全罩可以用来保护复杂的、具有潜在风险的电子设备一样。既保护它们的用户远离内部危险，又保护它们的内部免受其用户侵扰。

7.4.5 组件

我们已经提到，软件工程领域里的一个障碍就是缺乏预制的“现成的”组件构建块来构建大型的软件系统。在这一点上，软件开发中的模块化方法让我们看到了希望。特别是，事实证明，面向对象程序设计范型尤其有用，因为对象来自完备的、自我包含的单元，这些单元明确定义了与其外部环境的接口。一旦将一个对象，或者更准备地说是一个类，设计成能完成某种特定功能，就可以用它来在任何需要那个服务的程序中实现这个功能。此外，在对象定义必须定制以符合一个特定应用的需要的情况下，继承提供了一种对预制对象的定义进行改进的手段。因此，面向对象程序设计语言C++、Java和C#都带有多套预制的“模板”这一点就不足为奇了。通过这些模板，程序员可以很方便地实现对象并用来完成特定功能。具体来说，C++有C++标准模板库，Java编程环境有Java应用程序员接口（application programmer interface，API），C#程序员可以访问.NET框架类库。

虽然对象和类实际有可能为软件设计提供预制的构建块，但这并不意味着它们就是理想的选择。一个问题是，它们提供相对较小的模块来构建系统，所以对象实际上是更通用的**组件**（component）概念中的一个特例，组件在概念上就是软件的一个可复用单元。实际上，大多数组件都是基于面向对象范型的，并且表现为一个或多个对象组成的集合的形式，其功能是作为一个自包含单元。

组件的开发和使用的研究导致了称为**组件架构**（component architecture，也就是通常所说的基于组件的软件工程）的领域的出现。在这个领域中，传统的程序员被**组件装配员**（component

assembler) 所代替, 由组件装配员把预制的组件装配成软件系统。在许多开发环境中, 常常用图形界面中的图标来表示预制的组件。组件装配员并不涉及组件内部的编程, 而是在预先定义好的组件集合中选择相关的组件, 然后将它们进行最小化的定制并连接, 从而获得所需要的功能。事实上, 精心设计的组件的一个属性就是无须经过内部修改就可以扩展为包含一些特定应用的特性。

基本知识点

- 开发正确的程序组件, 然后将它们组合起立, 有助于创建正确的程序。

智能手机系统为组件架构提供了沃土。由于这些设备的资源限制, 应用实际是一套协作组件, 其中每一个都会为其应用提供一个具体的功能。例如, 一个应用中的每个显示屏通常都是一个独立的组件。幕后可能存在其他服务组件, 用于存储和访问存储卡上的信息、执行某个连续的功能(如播放音乐), 或通过因特网访问信息。每一个这样的组件都按需独立启动和停止, 以便高效地服务用户; 然而, 应用本身的显示和动作看起来像是一体的。

除限制系统资源使用的动机之外, 智能手机的组件架构为应用之间的集成带来了好处。例如, Facebook在智能手机上执行时, 可能会使用联系人应用组件将所有Facebook好友添加为联系人。此外, 电话应用(处理电话功能的应用)也可以访问联系人组件查找来电人。因

此，在接到Facebook朋友的电话时，手机屏幕上可以显示朋友的照片（以及他最后的Facebook帖子）。

7.4节问题与练习

1. 小说在章、节和条目等单元之间的耦合程度方面与百科全书有什么不同？内聚方面呢？
2. 一项体育赛事通常分为几个单元。例如，棒球比赛分成几局，网球比赛分成几盘。试分析两个这类“模块”之间的耦合。这类单元在什么意义上是内聚的？
3. 最大化内聚与最小化耦合的目标是否一致？也就是说，随着内聚的增加，耦合会自然减少吗？
4. 定义耦合、内聚和信息隐藏。
5. 扩展图7-3所示的结构图，使其包括模块ControlGame()与UpdateScore()之间的数据耦合。
6. 绘制一幅类似于图7-5的图，表示PlayerA的发球因违反规则被视为无效时将发生的事件序列。
7. 传统的程序员与组件装配员之间有什么区别？
8. 假设大多数智能手机都有一些个人组织应用（如日历、联系人、闹钟、社交网络、电子邮件系统、地图等），你认为组件功能的哪些组合实用而有趣？

7.5 行业工具

在本节中，我们研究软件开发的分析与设计阶段使用的一些建模技术和符号系统。其中一些技术和符号系统是在软件工程学科中以命令型范型为主导的年代里开发的。现在，在面向对象范型的语境中也可以找到它们中某些的身影，而另一些，如结构图（再看图7-3），则是专门用于命令型范型的。我们首先考虑一些从其命令型范型发展而来的技术，然后研究一些较新的面向对象的工具和设计模式的扩展功能。

版本控制

版本控制系统现在是大多数大型软件工作的一部分。基于万维网和网络的版本控制系统允许团队成员通过提供一致的跟踪代码个别修改的机制合作编写源代码。像Git、Subversion和Mercurial这样的版本控制工具允许团队成员在共享项目的本地副本上工作、合并其他成员并发进行的修改，以及准备好共享时提交自己的新修改。版本控制系统可以准确跟踪共享项目的每个特定修改的修改时间和修改人，并且可以通过将代码回滚到较早版本来撤销不满意的修改。版本控制也可用于文档、配置数据和构建过程的结构。

事实上，基本的版本控制功能现在已经包含在许多流行的软件包（实际上任何带有“撤销”按钮的程序都可以回滚多个更改）以及基于云的服务中，如Google Docs。

7.5.1 较老的工具

尽管命令型范型致力于依据过程或函数来构建软件，但确定这些函数的方式是考虑要操作的数据，而不是函数本身。具体思路是，通过研究数据在系统中如何流动，就能确定在哪儿更改数据格式，或者在哪儿对数据的路径进行合并或拆分。因此，就确定了进行处理的位置，这样一来，通过数据流的分析就能确定函数。**数据流图**（dataflow diagram）是表示从数据流分析中所获得的信息的一种手段。在数据流图中，箭头表示数据路径，椭圆表示数据操控发生的位置，矩形表示数据源和数据存储。作为一个示例，图7-8展示了一个用于表示医院病人计费系统的基本数据流图。注意，该图表明“支付”（从病人流出）和“病人记录”（从医院文件流出）在椭圆“处理支付”处合并，并从此处将“已更新记录”流回医院文件。而“病人记录”（从医院文件流出）和“账单”（从处理账单处流出）在椭圆“处理账单”处合并，并从此处将“账单”流回病人。

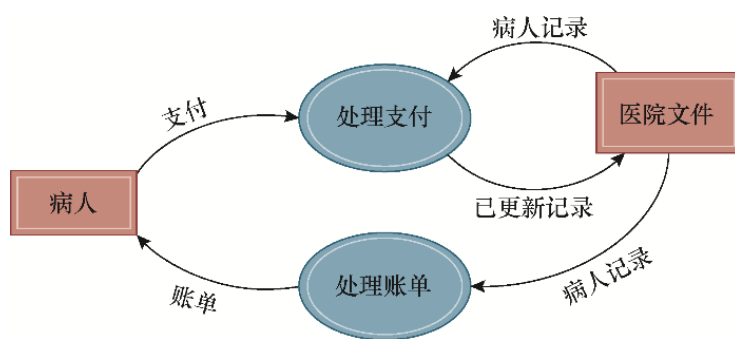


图7-8 一个简单的数据流图

数据流图不仅能在软件开发的设计阶段帮助确定过程，还能在分析阶段帮助理解预期系统。实际上，构建数据流图可以作为一种用来改善客户与软件工程师之间的交流的手段（因为软件工程师一直为理解客户想要什么而努力并且客户努力描述个人愿望），其次，即使在命令型范型已经不太流行的情况下，这些数据流图仍然在使用。

软件工程师已经用了很多年的另一种工具就是**数据字典**（data dictionary），它是整个软件系统中出现的数据项的一个中央信息库。这些信息包括为引用每个数据项所采用的标识符、每个数据项里的有效条目的构成情况（数据项一直是数值型的，还是一直是字符型的？分配给该数据项的值的可能范围是什么）、数据项存储在什么地方（数据项要存储在文件中或数据库中吗？如果要，具体存储在哪个里面）、软件在什么地方引用这些数据项（哪些模块需要数据项的信息）。

构建数据字典的一个目标是，增强软件系统的利益相关者与软件工程师之间的沟通，其中软件工程师负责将利益相关者的需求转化为需求规格说明。在这种语境下，构建数据字典有助于确保这样一个事实，即如果部分数字不是真正的数值型的，那么在软件的分析阶段就能发现，而不用等到在后面的设计或实现阶段才发现。构建数据字典的另一目标是确立整个系统的一致性。借助构建字典常常可发现冗余和矛盾。例如，一个数据项在库存记录中称为PartNumber，而在销售记录中可能就改称为PartId，还有，人事部门可能会用数据项Name来表示一名员工，而库存记录可能会用数据项Name来表示一个零件。

7.5.2 统一建模语言

在面向对象范型出现以前，数据流图和数据字典是软件工程宝库中的工具，在命令型范型已经不太流行的情况下（这些工具最初是为

命令型范型开发的），这些工具仍然能找到用武之地。现在，我们转而研究更为现代的称为**统一建模语言**（Unified Modeling Language，UML）的工具集。统一建模语言是基于面向对象范型思想发展而来的。然而，在这个工具集中，我们讨论的第一个工具是**用例图**（use case diagram），无论这个工具潜在的范型如何都是非常有用的，因为它仅尝试着从用户的视角来捕捉拟建系统的画面。图7-9展示的就是用例图的一个例子。

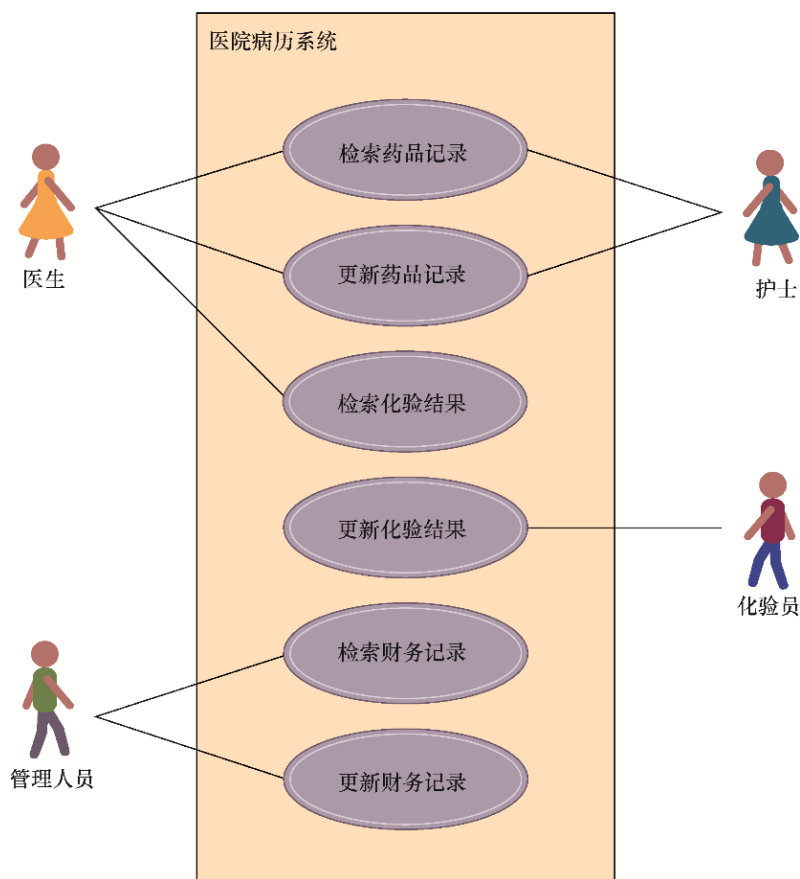


图7-9 一个简单的用例图

用例图是用大的矩形框来描述拟建系统的，在这个矩形框中，系统与其用户之间的交互，称为**用例**（use case），是用椭圆来表示的，

而系统中的用户，称为**参与者**（actor），是用火柴人表示的（即使角色可能不是一个人，也这么表示）。这样，图7-9中的图表示的是，“医生”和“护士”将用拟建的“医院记录系统”来“检索病例记录”。

基本知识点

- 程序的功能经常通过用户与之交互的方式来描述。

鉴于用例图是从外部考虑拟建的软件系统的，UML提供了许多种工具，用于表示系统内部的面向对象设计。其中一种工具是**类图**（class diagram），它是一个符号系统，用来表示类的结构和两个类之间的关系——在UML的术语中称为**关联**（association）。举一个例子，考虑医生、病人和病房之间的关系。我们假定表示这些实体的对象是分别从Physician、Patient和Room类构建的。

图7-10表明了这3个类之间的关系在UML类图中是如何表示的，其中矩形框表示类，线表示关联。关联线上可以做标记，也可以不做标记。如果做标记，可以用实心箭头来指明标记被读的方向。例如，在图7-10中，标记“照顾”后面的箭头表示医生照顾病人，而不是病人照顾医生。有时关联线上会带有两个标记，可以从任意一个方向读取关联。图7-10中Patient和Room类之间的关联例证了这一点。



图7-10 一个简单的类图

除了指明两个类之间的关联，类图还能传达这些关联的多样性。也就是说，它能指明一个类的多少个实例与另一个类的实例相关联。这个信息被记录在关联线的两端。具体来说，图7-10表明每位病人可以占用一个房间，而每个房间能容纳0位或1位病人。（我们假设每个房间都是私人房间。）星号（*）用于表示一个任意的非负数，因此，图7-10中的*表示每位医生可以照顾多位病人，而在关联的医生端的1表示每位病人只由一位医生照顾。（我们的设计只考虑主治医师这个角色。）

为了完整性起见，我们应该注意到关联的多样性有3种基本形式：一对一关系、一对多关系和多对多关系，如图7-11所示。

一对一关系（one-to-one relationship）的一个例子就是病人和已占用的私人病房之间的关联，因为每位病人只能分配到一个房间，而且每个病房只分配给一位病人。

一对多关系（one-to-many relationship）的一个例子就是医生和病人之间的关联，因为每位医生可以照顾多位病人，而每位病人只有一位（主治）医生照顾。在这个例子中，当我们考虑将病人与咨询医生之间的关联加入病人与医生之间的关系时，就形成了**多对多关系**（many-to-many relationship），即每位病人可以有多位咨询医生来辅助治疗，而每位咨询医生可以帮助多位病人。

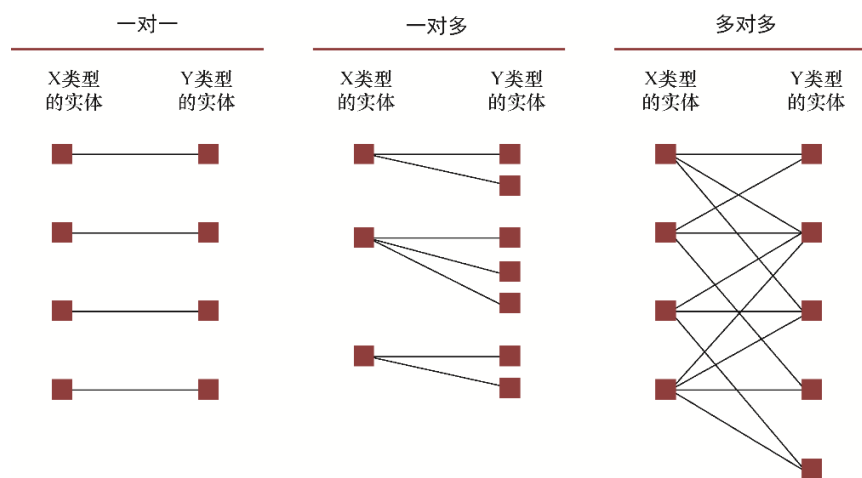


图7-11 X类型的实体与Y类型的实体之间的一对一、一对多以及多对多关系

在面向对象设计中，有一种经常出现的情况：一个类表示另一个类的更具体的版本。在这种情况下，我们说后者是前者的泛化。UML提供了一种特殊的表示泛化的表示法。图7-12中给出了一个例子，它描述了MedicalRecord、SurgicalRecord和OfficeVisitRecord类之间的泛化。两个类之间的关联用箭头表示，这是UML表示泛化的关联符号。注意，每个类都由一个矩形表示，里面包含类的名称、属性和方法，格式见图7-4。这是UML在类图中表示类的内在特征的方式。图7-12中描述的信息是：

MedicalRecord类既是SurgicalRecord类的泛化，也是OfficeVisitRecord类的泛化。也就是说，SurgicalRecord和OfficeVisitRecord类包含MedicalRecord类的所有特性，并附加了那些明确地列在它们矩形框中的特性。因此，SurgicalRecord和OfficeVisitRecord类都包含病人、医生和记录日期，只是SurgicalRecord类还包含手术流程、医院、出院日期和准许病人出院的权力，而OfficeVisitRecord类还包含了症状和诊断。这3个

类都有打印病历记录的功能。SurgicalRecord和OfficeVisitRecord类中的PrintRecord()方法是MedicalRecord类中PrintRecord()方法的特化，它们都可以打印其类特有的信息。

回顾第6章（6.5节），在面向对象程序设计环境中实现泛化的一个很自然的方式是利用继承。然而，许多软件工程师都警告说，继承并不是对所有的泛化情况都适合。原因在于，继承导致了两个类之间的强耦合度，这种耦合可能是软件生命周期的后期不希望出现的。例如，因为类的改变会自动反映在它的所有继承类中，所以软件维护阶段可能看起来很小的修改就会导致不可预见的后果。作为一个例子，我们假设一个公司为其员工开放了一个娱乐设施，这就意味着娱乐设施里所有具有会员资格的人都是该公司的员工。为了给这个设施做一个会员表，程序员可以利用继承依据早先定义的Employee类构建一个RecreationMember类。但是，如果公司后来发展顺利，公司决定对员工家属或公司退休员工也开放娱乐设施，就必须断开Employee类和RecreationMember类之间嵌入的耦合。因此，使用继承的时候不应该只考虑方便性，而应该将继承的使用严格限制在正在实现的泛化永远不变的情况下。

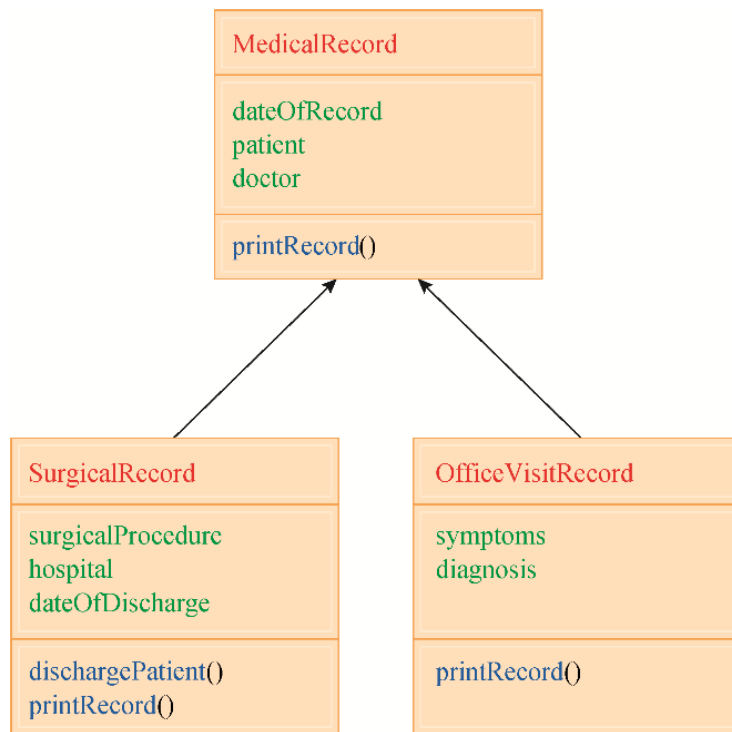


图7-12 描述泛化的一个类图

类图表示的是程序设计中的静态特征。它们不表示程序在执行过程中发生的事件序列。为了表示这种动态特征，UML提供了一系列的图表类型，它们统称为**交互图**（interaction diagram）。交互图的一种是**序列图**（sequence diagram），它描述了完成一个任务所涉及的个体（如参与者、完整的软件组件或个体对象）之间的通信。这些图与图7-5类似，因为它们都用带有向下延伸的虚线的矩形表示个体。每个矩形连同它的虚线称为**生命线**（life line）。两个个体之间的通信用连接合适生命线的带标记的箭头表示，这里的标记指示的是正被请求的动作。当自顶向下读图时，这些箭头是按时间先后次序出现的。当个体完成请求的任务，并把控制返回给发出请求的个体（就像传统的从一个过程返回）时，这时的通信是用一个指回原始生命线的无标记箭头表示的。

因此，图7-5本质上是一个序列图。但是，图7-5的语法本身有几个缺点。一个是它不允许我们捕获两名选手之间的对称性。我们必须画一个单独的图来表示开始于PlayerB发球的截击球，即使其交互序列与PlayerA发球的非常相似。此外，图7-5只描述了一次具体的截击球，一次一般的截击球活动肯定可以无限延展。形式化序列图有捕获单个图中这些变化的技术，虽然我们不需要仔细研究这些，但还是应该简要地看一下图7-13中展示的形式化序列图，它描述了基于我们的网球比赛设计的一个一般的截击球活动。

还要注意，图7-13说明了整个序列图是包含在一个矩形——称为**帧**（frame）——中的。帧的左上角是一个五角形，其中包含字符sd（意思是“sequence diagram”）和一个标识符，这个标识符可以是一个标识整个序列的名字，也可以是图7-13中那样被调用来初始化序列的方法的名字。注意，与图7-5相比，图7-13中表示选手的矩形并不是指特定的选手，只是表明了它们代表PlayerClass“类型”的对象。其中一个被指定为self，意思是激活它的发球方法（serve方法）来初始化序列。

图7-13中需要注意的另外一点是，它处理了两个内部矩形。这些矩形是用来表示一个图中的候选序列的**交互段**（interaction fragment）。图7-13中包含两个交互段，一个标记为“loop”，另一个标记为“alt”。这些本质上是我们在5.2节Python中首次遇到的while和if-else结构。“loop”交互段表明只要Judge对象判定validPlay的

值为真，就要重复其边界内的事件；“alt”交互段表明根据 fromServer 的值是真是假，执行其中一个候选序列。

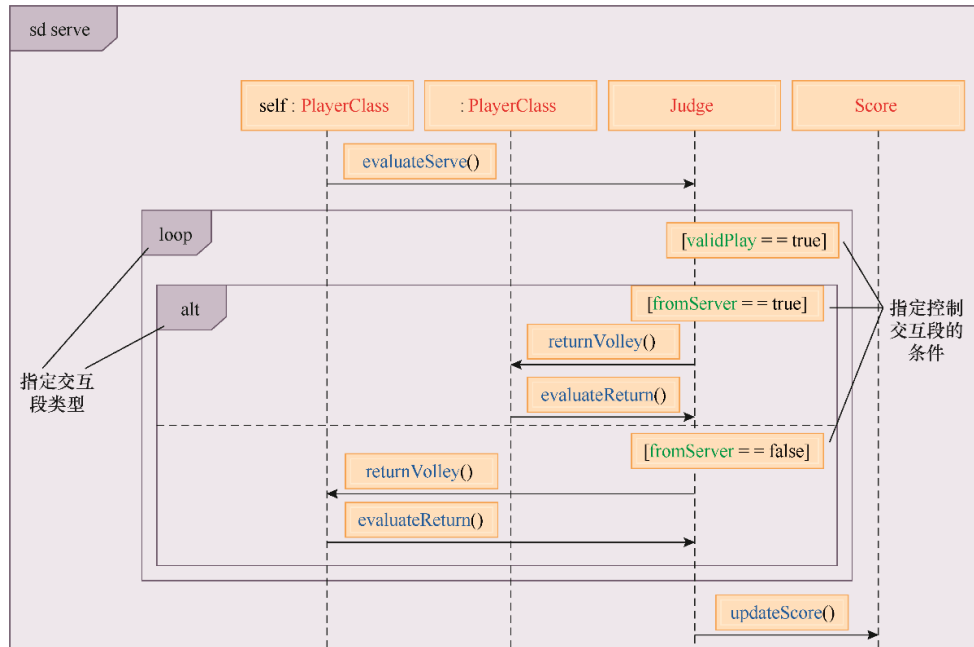


图7-13 描述一般截击球的序列图

最后，在这里介绍**类-职责-协作**（Class-Responsibility-Collaboration card，简称**CRC卡**）的作用是恰当的，尽管这部分内容不属于UML，但它们在验证面向对象设计方面起着很重要的作用。CRC卡是一张简单的卡，如索引卡，上面写着有关对象的描述。利用CRC卡的方法，软件设计师为预期系统的每个对象做一张卡，然后用这些卡在系统模拟中表示对象，模拟可能在桌面上进行，也可能通过一个“戏剧”实验进行，在实验中，设计团队的每个成员手持一张卡，扮演卡上描述的对象的角色。这样的模拟通常称为**结构化走查**（structured walkthrough），人们发现在设计实现之前的设计阶段找错很有用。

7.5.3 设计模式

对软件工程师而言，越来越强大的工具是不断发展的设计模式集。**设计模式**（design pattern）是用来解决软件设计过程中反复出现的问题的一种预先开发的模型。例如，适配器模式提供了一个解决办法，用来解决通过预制模块来构建软件时经常出现的问题。具体来说，预制模块可能已经具备了解决手边问题的功能，但可能还没有与当前应用兼容的接口。在这种情况下，适配器模式可以用作一种将一个模块“封装”在另一个模块里的标准方法，仅需要为原始模块的接口与外部世界提供翻译功能，这样一来，原始的预制模块就能用于应用中了。

另一种成熟的设计模式是装饰者模式，它提供了一种用来设计系统的手段，而所设计的系统依据当时的情况完成一些相同活动的不同组合。这种系统会产生大量的选择，如果没有经过仔细的设计，很可能导致软件极其复杂。但是，装饰者模式提供了一个实现这类系统的标准化方式，从而产生了一种易于管理的解决办法。

反复出现的问题的识别和用于解决这类问题的设计模式的创建和分类在软件工程领域里是一个不断进步的过程。然而，其目标不仅仅是找到设计问题的解决办法，还要找到能在软件生命周期的后期提供灵活性的高质量解决方案。因此，对诸如耦合最小化和内聚最大化这样的好的设计原则的考虑，在设计模式的发展过程中起着重要的作用。

设计模式在发展过程中所取得的进展成果，在今天的软件开发包所提供的工具库中得到了体现，如Oracle公司提供的Java编程环境以及微软公司提供的.NET框架等。事实上，在这些工具包中找到的许多模板本质上就是设计模式的框架，它们为设计问题提供了现成的、高质量的解决方案。

最后，我们要提的是，软件工程中设计模式的出现是不同领域相互促进的一个很好的例子。设计模式起源于克里斯托弗·亚历山大（Christopher Alexander）在传统建筑领域里的研究，他的目标是确定有助于建筑物或建筑群的高质量建筑设计的特征，然后开发包含这些特征的设计模式。今天，软件设计中已经包含了他的许多思想，并且许多软件工程师仍继续从他所做的工作中汲取灵感。

7.5 节问题与练习

1. 画一个数据流图，表示一名读者从图书馆登记借一本书时发生的数据流。
2. 画出图书馆记录系统的用例图。
3. 画一个类图，表示旅客与其所住酒店之间的关系。
4. 画一个类图，表示人是员工的泛化这一事实，包括一些可能每个人都有的属性。
5. 把图7-5转化为完整的序列图。
6. 设计模式在软件工程过程中扮演什么角色？

7.6 质量保证

软件故障、成本超支、错过截止时间等现象的激增，对软件质量控制的方法提出了要求。在本节中，我们考虑一些在此方面的努力方向。

7.6.1 质量保证的范围

在计算机发展的早期，生产优质软件的关注点主要集中在去除在实现过程中产生的编程错误上。在本节的后面，我们将讨论在这个方向上取得的进步。然而，如今软件质量控制的范围远超出了调试过程，它的分支包括软件工程过程的改进、培训课程的开设（在很多情况下是为了认证），以及标准的制定（建立完善软件工程的基础）。在这方面，我们已经注意到ISO、IEEE和ACM这些组织在提升专业性和制定用于评估软件开发公司内部质量控制的标准方面所起的作用。一个具体的例子是ISO 9000系列标准，它涉及许多工业活动，如设计、生产、安装和维修。另一个例子是ISO/IEC 33001，它是由ISO和国际电工委员会（IEC）联合制定的一套标准。

现在许多主要软件承包商要求他们雇用的软件开发组织符合这些标准。因此，软件开发公司正在建立**软件质量保证**（software quality assurance，SQA）小组，负责监督和强制执行组织所采用的质量控制系统。例如，在传统的瀑布模型下，SQA小组将负责在设计阶段开始之前批准软件需求规格说明，或在实现阶段开始之前批准设计及相关的文档。

基本知识点

- 协作可以让找到和纠正开发程序时的错误更容易。

许多主题构成了当今质量控制工作的基础，其中之一就是保存记录。最重要的是将开发过程中的每一步都准确地记入文档以供将来参考。但是，这个目标与人类的本性相冲突，人的本性使得人们倾向于在做出决定或改变决定时不更新相关文档。因此，记录有可能是不正确的，因而在未来阶段使用它们时很有可能会产生误导。在这一方面，CASE工具具有非常大的优势。它们使得像重画图表和更新数据字典这类任务与手工方法相比，要更加容易。因此，记录更可能会被更新，最终的文档也更可能是准确的。（这只是软件工程必须应对人性弱点的多个实例之一，其他例子包括当人们共事时，不可避免的个性冲突、嫉妒和自我抵触。）

另一个面向质量的主题是评审的使用，其中涉及软件开发项目的各方聚在一起，考虑一个具体主题。评审贯穿整个软件开发过程，采用的形式有需求评审、设计评审和实现评审。它们可能表现为：需求分析早期的原型演示；软件设计团队成员间的结构化走查；或者实现设计相关部分的程序员间的协调。这样的评审（基于重复的基础）提供了沟通的渠道，通过它们可以避免误解，在错误造成灾难之前更正错误。在称为IEEE 1028的IEEE软件评审标准中专门提到了它们，这一事实例证了评审的重要性。

有些评审在本质上是关键的。一个例子就是，项目利益相关者的代表和软件开发团队之间针对批准最终的软件需求规格说明而进行的评审。实际上，这个批准标志着正式需求分析阶段的结束，同时它也是后续要进行的开发过程的基础。但是，从质量控制的角度来说，所有的评审都是重要的，它们都应该记入文档，作为不断进行的记录维护过程的一部分。

7.6.2 软件测试

软件质量保证现在被认为是贯穿整个开发过程的一个主题，程序的测试和验证本身一直是研究的主题。在5.6节中，我们讨论了用严格的数学方法验证算法正确性的技术，但结论是如今大多数软件要使用测试来“验证”。遗憾的是，这种测试在最好的情况下也不精确。除非我们对一个软件做足够多的测试，穷尽所有可能的情况，否则还是不能保证这个软件是正确的。但是，即使是在简单的程序中，也可能有无数条可以遍历的路径。因此，对一个复杂程序的所有可能的路径进行测试是一项不可能的任务。

另外，软件工程师已经开发出了一些测试方法，通过有限次的测试来提高发现软件错误的可能性。其中一种是以软件中的错误倾向于扎堆这个观察为基础的。也就是说，经验表明，一个大型软件系统中会有一小部分模块比其他模块更容易出问题。因此，与其把所有的模块都进行相同的、不彻底的测试，还不如确定那些容易出错的模块，对它们进行彻底的测试，这样可以发现系统的更多错误。这就是所谓

的**帕累托法则**（Pareto principle）的一个实例，该法则是意大利经济学家、社会学家维尔弗雷多·帕累托（Vilfredo Pareto，1848—1923）提出来的，他发现意大利的一小部分人口控制了意大利的大部分财富。在软件工程领域，帕累托法则指出，通过对一个集中区域施加作用，往往就可以明显地改变结果。

软件测试的另一种方法称为**基本路径测试**（basis path testing），这种方法要开发出一组测试数据，保证软件中的每条指令都能至少执行一次。人们已经用数学领域的图论开发出了确定这种测试数据集的技术。因此，虽然不可能保证通过软件系统的每条路径都得到测试，但是可以做到在测试过程中，系统的每条语句都至少能执行一次。

基于帕累托法则和基本路径测试的技术都依赖于对被测试软件的内部构成的理解，因此这类测试都属于所谓的**白盒测试**（glass-box testing）这一类，这也就意味着软件测试人员要了解软件的内部构成，在设计测试的时候要用到这个知识。与之相反，还有一类测试称为**黑盒测试**（black-box testing），这类测试并不依赖于对软件内部构成的了解。简而言之，黑盒测试是从用户的角度来完成的。在黑盒测试中，测试人员并不关心软件本身是如何执行其任务的，只关心软件在精确度和时间性方面是否能正确执行。

黑盒测试的一种方法是称为**边界值分析**（boundary value analysis）的技术，该技术包括确定数据范围 [即**等价类**（equivalence class），软件应以类似的方式操作它们]，然后用接近这些范围的边界的数据测试软件。例如，如果软件需要接受指定范围内的输入值，

那么就可以用这个范围内的最低值和最高值对该软件进行测试；或者如果软件需要协调多个活动，那么就应在尽可能大的活动集合上测试。基于的理论是：通过确定等价类，测试用例的数量可以最小化，因为对一个等价类内的几个例子的正确操作往往会验证整个类的软件。此外，确定一个类内错误的最佳机会是使用类边界上的数据。

黑盒测试的另一种方法是**β测试**（beta testing），在这种测试中，在产品的最终版本稳定下来向市场发布之前，软件的初步版本会被提供给一部分预期受众，以了解软件在现实环境中的表现如何。[开发者现场进行的类似测试称为<στρογγ>α测试</στρογγ>（alpha testing）。] β测试的优点远远不止传统的错误发现。通过这种测试获得的普通用户的反馈意见（无论正面的还是负面的）将有助于调整市场策略。此外，早期发布的beta版的软件有助于其他软件开发人员设计出与之兼容的产品。例如，就台式计算机市场的新操作系统来说，beta版本的发布会鼓励与之兼容的实用软件的开发，这样最终版的操作系统上市时，就已经有与之配套产品了。此外，β测试的存在会在市场上造成一种期待感——营造一种增加推广和销量的氛围。

7.6节问题与练习

1. 软件开发组织内的SQA小组的作用是什么？
2. 人性是以什么方式与质量保证对立的？
3. 说出两种在整个开发过程中用来加强质量的课题。
4. 在测试软件时，成功的测试是指发现了错误的测试，还是没有发现错误的测试？

5. 为了确定系统中的哪些模块应该接受比其他模块更为彻底的测试，你会建议采用什么技术？

6. 一个软件包设计用来对有不超100个项的列表进行排序，请问，对此软件包采用什么样的测试最为合适？

7.7 文档

如果人们不能学会使用和维护软件系统，那么这个软件系统也就没多大的用处。因此，文档是最终软件包的一个重要部分，因而文档的编写也就成了软件工程领域里的一个重要课题。

软件文档有3种用途，因而可以将文档划分为3类：用户文档、系统文档和技术文档。**用户文档**（user documentation）的用途是解释软件的特性，并描述如何使用软件。用户文档是给软件用户阅读的，因而其编写采用的是应用方面的术语。

今天，用户文档被公认为是一种重要的营销工具。好的用户文档加上精心设计的用户界面，能使软件更容易为人们所接受，从而提高软件销量。正因为认识到这一点，许多软件开发人员聘请熟悉技术的文档撰写人编制产品文档，或者将自己的软件产品的初级版本提供给独立作者，以便在向公众发布软件时书店里同时有关于如何使用该软件的书籍。

用户文档传统上是纸质书或小册子，但在线电子版或软件本身含有文档的情况越来越多。这使读者在使用软件时能参考文档。在这种

情况下，信息可能会分割成小的单元，有时称为帮助页面，如果用户在两个命令之间犹豫不决时，帮助页面可能会自动出现在屏幕上。

系统文档（system documentation）的用途是描述软件的内部构成，便于在软件的生命周期内维护软件。系统文档的一个主要部分是系统中所有程序的源代码版本。这些源代码版本应以易读的格式呈现，这一点很重要，这就是为什么软件工程师支持“采用精心设计的高级程序设计语言、采用注释语句对程序进行注释，以及采用以一致单元表示每个模块的模块化设计”的原因。实际上，大多数软件开发公司都有一定的要求其员工编写程序时要遵循的约定。例如，在书面上组织程序的缩进约定，为不同程序结构（如变量、常量、对象和类）建立有差别的命名约定，以及保证所有程序都能有效地文档化的文档约定。这些约定在整个公司的软件中都是统一的，这样最终就能简化软件的维护过程。

基本知识点

- 程序文档有助于程序员开发和维护正确的程序，以有效地解决问题。

系统文档的另外一个组成部分是设计文档的记录，其中包括软件需求规格说明和显示这些规格说明在设计期间被如何获得的记录。这些信息对软件的维护是有帮助的，因为它指明了软件为何要这样实现，同时这些信息也降低了维护阶段所做的修改破坏系统完整性的可能性。

技术文档（technical documentation）的用途是描述软件系统应该如何安装和维护（如调整操作参数、安装更新以及将出现的问题反馈给软件开发人员）。软件的技术文档与汽车工业中提供给汽车修理工的文档类似。这种文档不讨论汽车是怎样设计和构造的（这类似于软件的系统文档），也不解释如何驾驶汽车和操作汽车加热/制冷系统（这类似于软件的用户文档），而是描述如何维护汽车配件，例如，如何更换变速器，或者如何追查间歇性电气问题。

在台式计算机领域里，软件的技术文档和用户文档之间的区别是比较模糊的，这是因为用户经常也是安装和维护软件的人。然而，在多用户环境中，区别更明显。因此，技术文档是专供系统管理员使用的。系统管理员要负责其管辖范围内所有软件的维护，允许用户将软件包作为抽象工具访问。

7.7节问题与练习

1. 软件可以以哪些形式文档化？
2. 系统文档应在软件生命周期的哪个（或哪些）阶段进行准备？
3. 程序和它的文档相比，哪个更重要？

7.8 人机界面

回顾一下7.2节，其中讲到需求分析阶段的一项任务是定义要开发的软件系统将如何与它的环境进行交互。本节我们将考虑与这种交互相关的主题，那就是当它涉及与人交流时的情况，这是一个意义深远

的主题。毕竟，应该允许人把软件系统当作一个抽象工具来使用。这个工具应该易于使用和设计，能最小化（理想情况下消灭）人与系统间的交流错误。这意味着系统界面的设计应该对人方便，而不仅是对软件系统方便。

良好的界面设计非常重要，因为与系统的其他特征相比，系统界面更容易给用户留下深刻的印象。毕竟，人往往会从系统可用性的角度来审视一个系统，而不是从它如何巧妙地执行其内部任务的角度。从人的角度来看，很可能会根据系统界面在两个具有竞争性的系统之间做出选择。因此，系统界面的设计可能会成为判定一个软件工程项目是否成功的最终决定因素。

由于这些原因，人机界面在软件开发项目的需求分析阶段已经成为一个很重要的关注点，并且是软件工程的一个不断成长的子领域。事实上，有些人认为人机界面的研究是一个完全独立的领域。

智能手机界面便得益于这一领域的研究。为了能够实现一种方便的袖珍设备，传统人机界面的元素（标准尺寸的键盘、鼠标、滚动条、菜单）已被新的方式所取代；新方式的例子包括在触摸屏上操作的手势、语音命令以及具有高级的单词和短语自动输入功能的虚拟键盘。尽管这些标志着重大的进展，但大多数智能手机用户认为进一步创新的空间还很大

对人机界面设计的研究主要来自称为**人机工程学**（ergonomics）和**知行学**（cognetics）的工程领域。人机工程学研究的是与人类身体

能力相协调的系统的设计，知行学研究的是与人类精神能力相协调的系统的设计。这两个学科中，人机工程学更好理解一些，主要是因为人类已经跟机器打了几个世纪的交道。在古代工具、武器和运输系统中都能找到示例。这些历史大部分是不证自明的，但是有时人体工程学的应用与直觉是相反的。一个经常被提到的例子就是打字机键盘（现在已经衍生为计算机键盘）的设计，其中键被有意排列，以降低打字员的速度，这样早期机器上使用的分层机械系统就不会卡住。

相反，与机器的精神交互是一个相对较新的现象，因此，正是知行学为富有成效的研究和启发性的见解提供了更大的可能性。通常这些研究成果更具有精妙之处。例如，从表面上看人类的良好习惯有助于提高效率，但有些习惯也会导致一些错误，即使界面设计本意上是要解决问题的。考虑一下用户要求一个典型操作系统删除一个文件的过程。为了防止误删，大部分界面都会要求用户确认一个请求，也许是通过一个消息确认，如“你真的想删除这个文件吗？”乍一看，这个确认信息好像解决了误删的问题，但是使用这个系统一段时间后，用户会养成自动回答“是”的习惯。这样，这个删除文件的任务就从包含删除命令和对问题思考后的响应的两步过程，变成了“删除-是”的一步处理过程，这意味着，当用户意识到提交了错误的删除请求时，这个请求其实已被确认，文件也已经被删除。

当人们需要使用几个应用软件包时，习惯的形成可能也会带来问题。这些软件包的界面可能是相似的，但还是有些不同。相似的用户操作可能会导致不同的系统响应，或相似的系统响应可能需要不同的

用户操作。在这种情况下，在一种应用上养成的操作习惯可能会在其他应用软件上导致错误的发生。

另外一个与人机界面设计研究人员有关的人类特质是人类注意力的狭隘性，注意力有随着专心程度的增加而越来越专注的趋势。随着人类越来越专注于手头上的工作，打破这种专注也越来越困难。1972年，一架商务飞机因为飞行员太过专注于起落架问题（实际上，是在改变起落架指示灯的过程中），尽管驾驶舱里的警报一直在响，但飞机还是笔直地撞向了地面，造成了空难。

台式计算机界面中经常会出现一些小状况。例如，大多数键盘都有“大写锁定”（Caps Lock）灯，用于指示键盘是否处在“大写锁定”模式下（即大写锁定键已被按了）。但是，如果有人不小心按了大写锁定键，那么直到奇异的字符出现在屏幕上，他才会注意到指示灯的状态。即使如此，用户经常依然会迷茫一会儿才会发现问题原因。从某种意义上来说，用户看不到大写灯的变化是很正常的，因为键盘上的指示灯不在用户的视线范围内。但是，通常用户也不能注意到直接放在他们视线内的指示灯。例如，用户会特别专注于他们的任务，以至于观察不到显示屏上光标的变化，即使观察光标是他们的任务之一。

还有另外一个在界面设计阶段必须预先考虑的人类特质，即并行处理多个事情时大脑有限的思考能力。在1956年《心理学评论》

（*Psychological Review*）的一篇文章中，George A. Miller的一项研究表明，人类大脑同时最多能处理大约7个细节问题。因此，重要的是界

面要设计成这样的：在需要做决定时要将所有相关的信息都呈现在界面上，不要依赖人类用户的记忆。特别地，若要求人类记住先前屏幕图像中的精确细节，这就是很糟糕的设计。更进一步地，如果界面需要用户在屏幕图像间进行大量导航，用户会变得很迷惑。因此，屏幕图像的内容和排列成了一个重要的设计问题。

尽管人机工程学和知行学的应用使得人机界面设计折射出独特的韵味，但这个领域还是包含着软件工程中很多更加传统的主题。特别地，和较传统的软件工程领域一样，搜寻度量标准在界面设计领域中同样重要。界面可以度量的特性包括学习一个界面所需的时间、在界面上完成任务所需的时间、用户界面出错率、一段时间不用后用户使用界面的熟练程度，甚至是一些主观特性，如用户喜欢界面的程度。

GOMS（与“Toms”押韵）模型最初是在1954年提出的，是人机界面设计领域里搜寻度量标准的代表。这个模型的基础方法学是依据用户的目标（如删除文本中的某个字）、操作（如单击鼠标按键）、方法（如双击鼠标按键和按下删除键）和选择规则（实现同一目标的两种方法间的选择）来分析任务的。实际上，这就是GOMS缩写的起源——goal（目标）、operator（操作）、method（方法）和selection rule（选择规则）。简而言之，GOMS是一种把人使用一个界面的动作分析成基本步骤序列（按键、移动鼠标、做决定）的方法学。给每个基本步骤的性能分配一个精确的时间段，这样，GMOS就能通过把分配给任务步骤的时间相加，根据每个界面完成相似任务所需的时间，提供一种比较不同拟建界面的方法。

理解类似于GMOS的系统的技术细节不是我们当前的研究目的，我们这个示例的要点是，GOMS是以人类行为（移动手、做决定等）特性为基础的。事实上，GMOS的发展起初被认为是心理学中的主题。因此，GMOS重新强调了人类特质在人机界面设计领域甚至那些从传统软件工程里延伸出来的主题中所起的作用。

在可预见的未来，人机界面设计肯定是一个活跃的研究领域。涉及当今GUI的许多问题依然没有得到解决，大量附加问题潜存在现在刚刚兴起的三维界面的使用中。实际上，因为这些界面承诺结合语音和与三维视觉的触摸交流，所以潜在问题的范围是巨大的。

7.8节问题与练习

1. a. 说出人机界面设计领域中人机工程学的一个应用。
b. 说出人机界面设计领域中知行学的一个应用。
2. 对比智能手机与台式机的人机界面，一个显著的区别便是滚动显示区域所用的技术。在台式机上，滚动通常通过用鼠标拖曳显示区域右侧或下方的滚动条，或者使用鼠标内置的滚轮来实现，而智能手机通常不用滚动条。（就算用，它们通常也显示为细线，以表明底层显示的哪一部分当前是可见的。）智能手机界面上的滚动是通过在显示屏上进行的滑动触摸手势实现的。

a. 根据人机工程学，可以提出什么论据来支持这一区别？

b. 根据知行学，可以提出什么论据来支持这一区别？

3. 人机界面设计领域与较传统的软件工程领域有什么不同？
4. 说出在设计人机界面时应该考虑的3种人类特质。

7.9 软件所有权和责任

大多数人都会同意这样一个观点：应该允许公司或个人收回开发高质量软件的投资或者从中获利。否则，就不可能有许多人愿意承担生产社会所需软件的工作了。简而言之，软件开发人员需要对他们生产的软件拥有一定的所有权。

提供这种所有权的法律工作属于**知识产权**（intellectual property）法范畴，其中许多依据的都是版权法和专利法的既定原则。实际上，版权或专利的目的是允许产品的开发者在保护其所有权的同时向公众发布产品（或其部分产品）。同样，产品开发人员（无论是个人还是公司）要主张其对该产品的所有权就要在其创作的所有生产产品（包括需求规格说明、设计文档、源代码和测试计划）中和最终产品的某个可见位置上包含版权声明。版权声明要清晰地确定所有权、有权使用该作品的人以及其他限制。此外，开发人员的权利在**软件许可**（software license）中是用法律术语正式表述的。

软件许可是软件所有者与软件产品用户之间的一份法律协议，它授予用户使用产品的特定权限，而不转移知识产权的所有权。这些协议非常详细地解释了双方的权利和义务。因此，在安装和使用某一软件产品之前，仔细阅读和理解软件许可中的条款是非常重要的。

尽管版权和软件许可协议为禁止直接复制和非授权使用软件提供了法律保护，但它们通常不足以禁止另一方独立开发与该产品功能几近相同的软件产品。令人遗憾的是，多年来很多真正具有创新性的软件产品的开发人员却无法从他自己的发明中充分获利（其中两个著名的例子便是电子表格软件和Web浏览器）。在大多数情况下，往往是另一家公司成功地开发了具有竞争力的产品，并占据了具有绝对优势的市场份额。就这一方面来说，阻止竞争对手侵扰的一个法律途径就是专利法。

专利法的制定是为了允许发明者从其发明中获得商业上的利益。为了获得专利，发明者必须透露发明的细节，并说明对具有类似背景的其他人来说，这是新的、有用的、不是轻而易举做到的（对软件来说，这是一个非常具有挑战性的要求）。如果一个专利被授权，那么在一段有限的时期（一般是自专利申请提出之日起的20年）内发明者就被赋予了阻止其他人制造、使用、销售或引进发明的权力。

采用专利的一个缺点是，获得专利是一个既昂贵又费时的过程，通常要历时几年。在这段时间内，软件产品可能会过时，直到专利被授权，申请人只有不确定的权限阻止别人盗用其产品。

在软件工程过程中，明确版权、软件许可和专利是极其重要的。在开发某一软件产品时，软件工程师通常会选择集成来自其他产品的软件，它可能是一个完整的产品、一部分组件，也可能是通过因特网下载的一部分源代码。然而，如果在此过程中未能尊重知识产权，则将可能导致巨大的损失和严重的后果。例如，2004年，一个不太出名

的公司—NPT有限公司—成功地赢得了一场法律诉讼，它控告 Research In Motion (RIM，黑莓智能手机制造商) 公司在电子邮件系统中侵犯了它的一些关键技术的专利权。这一诉讼的判决结果包括暂停RIM向美国所有黑莓用户提供电子邮件服务的禁令！最终，RIM与NPT达成协议并支付给NPT总共6.125亿美元，才避免了关停的命运。

最后，我们应该讨论责任的问题。软件开发人员为了保护自己免于承担责任，通常会在其产品上附带免责声明，用以说明其责任的限制。诸如“因使用本软件所造成的任何损失，本公司概不负责”这样的声明比较常见。然而，如果原告能够举出被告的疏忽之处，法庭很少会认可这类声明。因此，责任案件倾向于关注被告对所生产的产品是否给予了相应的关照程度。一个在开发字处理系统的情况下认为可以接受的关照程度，如果放在核反应堆的控制软件的开发上，就可能被认为是一种疏忽。因此，其中一种最好的防止软件责任索赔的办法是，在软件的开发过程中，运用合理的软件工程准则，采用与软件应用相适应的关照程度，生成并维护验证这些努力的记录。

7.9节问题与练习

- 1．在需求规格说明、设计文档、源代码和最终产品中，版权声明的意义是什么？
- 2．版权法和专利法的制定对社会有何益处？
- 3．免责声明中的什么内容将不会被法庭认可？

复习题

(带*的题目涉及选学章节的内容。)

1. 举出一个例子，说明软件开发时所做的努力是如何在日后的软件维护中得到回报的。

2. 什么是演化式原型开发？

3. 试解释缺少度量某些软件特性的度量学是如何影响软件工程学科的。

4. 你是否认为度量软件系统复杂性的度量标准是积累的？积累的意思是：整个系统的复杂性是其各部分的复杂性之和。请解释你的答案。

5. 你是否认为度量软件系统复杂性的度量标准是可交换的？可交换的意思是：不论系统是最初开发了X特性，后来加入了Y特性，还是最初开发了Y特性，后来加入了X特性，整个系统的复杂性都是相同的。请解释你的答案。

6. 软件工程与电子、机械工程等较传统的工程领域有何区别？

7. a. 给出软件开发中采用传统瀑布模型的缺点。

b. 给出软件开发中采用传统瀑布模型的优点。

8. 开源开发是一个自顶向下或者自底向上的方法学吗？请解释你的答案。

9．试描述为何使用常量代替字面量能简化软件维护。

10．耦合和内聚的区别是什么？哪个应该最小化？哪个应该最大化？为什么？

11．从日常生活中选取一个对象，依据功能内聚或逻辑内聚来分析其组成部分。

12．试将由一条简单的goto语句造成的两个程序段之间的耦合与由函数调用引起的耦合进行比较。

13．在第6章中我们已经知道，参数可以通过按值传递或按引用传递这两种方式传递给函数。哪一种提供了更为复杂的数据耦合形式？请解释你的答案。

14．如果一个大型软件系统中的数据元素都设计成全局数据，那么在维护阶段中可能会出现什么问题？

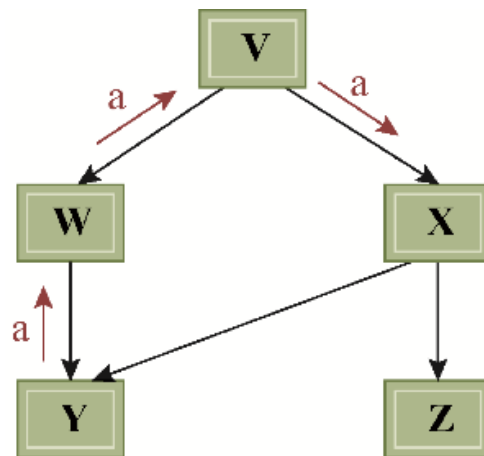
15．在面向对象程序中，声明一个实例变量是公有的或是私有的对数据耦合意味着什么？将实例变量声明为私有的这一偏好背后的基本原理是什么？

*16．举出一个会在并行处理语境中出现的涉及数据耦合的问题。

17．按下面的结构图回答下列问题。

a. 模块Y把控制返回到哪个模块？

- b. 模块Z把控制返回到哪个模块？
- c. 模块W和模块X是通过控制耦合链接起来的吗？
- d. 模块W和模块X是通过数据耦合链接起来的吗？
- e. 哪些数据是由模块W和模块Y共享的？
- f. 模块W和模块Z是以什么方式关联的？



18．用一个结构图来表示为小商店（也许是在人流量较大的社区开的一家私人古董店）开发的一个简单库存/账务系统的过程结构。

请问，由于销售税法的变动，你必须修改系统中的哪些模块？如果你决定要维护一个老顾客的记录，以便能给他们邮寄广告，那么需要对哪些模块进行修改？

19．使用类图，为上题设计一个面向对象的解决方案。

20．画出一个简单的类图，表示杂志出版商、杂志和订阅者彼此之间的关系。只要在表示类的每个矩形框中描述类名即可。

21．什么是UML？它是用来做什么的？请详细解释与“M”这一字母对应的词。

22．画一个简单的用例图，描述图书馆的读者使用图书馆的方式。

23．画一个序列图，表示当公用事业公司给客户发送账单时将发生的交互序列。

24．画一个简单的数据流图，描述当一个交易完成时，自动库存系统里出现的数据流。

25．对类图所表示的信息与序列图所表示的信息进行比较。

26．说明一对多关系与多对多关系有何不同？

27．举出一个本章中没有提到的一对多关系的例子。举出一个本章中没有提到的多对多关系的例子。

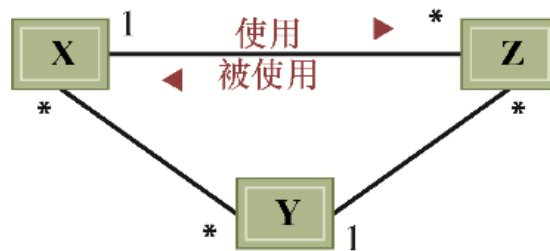
28．基于图7-10中的信息，想象一下在看望病人的过程中医生和病人之间可能发生的交互序列。画出一个表示这个序列的序列图。

29．画出一个类图，表示饭店里服务生和顾客之间的关系。

30．画出一个类图，表示杂志、杂志出版商和订阅者彼此之间的关系，并提供每个类的实例变量和方法。

31．扩展图7-5中的序列图，显示下面这种情况下将会发生的交互序列：PlayerA成功回击PlayerB的截击球，但PlayerB未能成功回击那个截击球。

32．下面的类图表示的是工具、工具的用户以及工具的生产厂商彼此之间的关联，根据这个类图回答下列问题。



a. 工具、用户和厂商分别是由哪个类（X、Y和Z）表示的？验证你的答案。

b. 一个工具能被1个以上的用户使用吗？

c. 一个工具能被1个以上的厂商制造吗？

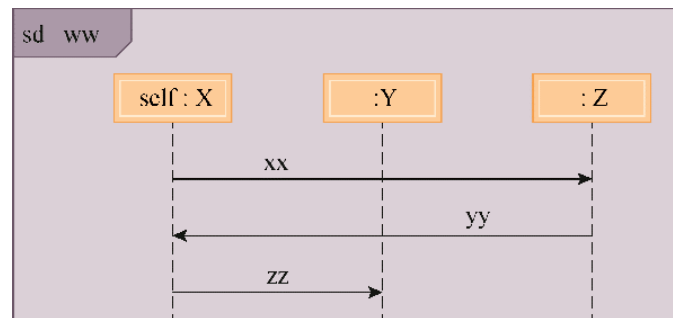
d. 每个用户都要使用由多个厂商制造的多个工具吗？

33．根据下面的各种情况，判断所述的活动是与序列图有关、与用例图有关，还是与类图有关。

a. 表示用户将与系统交互的方式。

- b. 表示系统中两个类之间的关系。
- c. 表示完成某一任务时对象采用的交互方式。

34．基于下面的序列图，回答下列问题。



- a. 什么类含有名为ww()的方法？
- b. 什么类含有名为xx()的方法？
- c. 在序列中，Z类型的对象会一直与Y类型的对象直接通信吗？

35．画出一个序列图，表明对象A调用对象B中的方法bb()，B执行请求的动作并将控制返回给A，然后A再调用对象B中的cc()方法。

36．扩展前一个问题的解决方法，表明仅当变量“continue”为真时A才能调用方法bb()，在B返回控制后，只要“continue”继续为真，A就继续调用bb()方法。

37．画出一个类图，描述这样一个事实，即Truck（卡车）类和Automobile（小汽车）类都是Vehicle（汽车）类的泛化。

38．依据图7-12，SurgicalRecord类型的对象中会包含哪些额外的实例变量？OfficeVisitRecord类型的呢？

39．解释为什么继承并不总是实现类泛化的最佳方式。

40．举出软件工程领域以外的一些设计模式。

41．总结设计模式在软件工程中的作用。

42．在多大程度上来说，一个典型的高级程序设计语言中的控制结构（如if-else、while等）是小规模的设计模式？

43．下列情况哪个涉及了帕累托法则？请解释你的答案。

a. 一粒老鼠屎搞坏一锅粥。

b. 每个电台都专注于一种特定的风格，如摇滚乐、古典音乐或谈话。

c. 在选举活动中，候选人非常明智地将其重点放在之前投他们票的那部分选民上。

44．软件工程师希望大型软件系统在错误的内容上是同种类型的，还是不同类型的？请解释你的答案。

45．黑盒测试与白盒测试的区别是什么？

46．试举出一些在软件工程以外的领域中发生的与黑盒测试和白盒测试类似的事件。

47．开源开发与 β 测试有何区别？（考虑白盒测试和黑盒测试。）

48．假定在一个大型系统快要完成最后的测试之前，故意放入100个错误。此外，还假定在最后的测试期间发现并纠正了200个错误，而其中50个错误属于故意放入系统中的。请问，如果接下来剩下的50个已知错误也被纠正了，那么你估计系统中还有多少个未知的错误？为什么？

49．什么是GOMS？

50．什么是人机工程学？什么是知行学？

51．对比智能手机与台式机的人机交互界面，它们之间的一个区别就在于改变显示屏上图像比例以获得更多或更少细节（此过程称为“缩放”）的技术。在台式机上，缩放通常通过拖动一个独立于显示区的滑块来实现，或者通过菜单或工具栏选项实现。在智能手机上，实现缩放要用大拇指和食指同时触摸显示屏，然后改变两个触摸点间的距离（即实现“放大”功能的“双指触击-展开”过程，或者实现“缩小”功能的“双指触击-收缩”过程）。

a. 根据人机工程学，可以提出什么论据来支持这一区别？

b. 根据知行学，可以提出什么论据来支持这一区别？

52. 在什么情况下，传统的版权法无法保护软件开发者的投资？

53. 在什么情况下，软件开发者不能成功获得专利？

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1. a. 分析员玛丽被分配了一个实现一个系统的任务。通过该系统可以将医疗档案存放在与大型网络相连接的计算机上。依据她的观点来看，系统安全性方面的设计存在缺陷，但是由于财务方面的原因她的想法被否决了，而且公司要求使用她认为不合格的安全系统来继续该项目。这种情况下，她该怎么办？为什么？

b. 假设分析员玛丽按照吩咐实现了该系统，并且现在她发现有未经授权的人在检索医疗档案。这时她该怎么办？对于这样一种侵犯安全的情况，她要负多大责任？

c. 假设分析员玛丽没有听从老板的安排，拒绝继续开发这个系统，并且义无反顾地将设计缺陷公布于众，结果导致公司的财务紧张，许多无辜的员工失去工作。分析员玛丽的行为对吗？如果事实证明，玛丽只是整个设计团队的一名成员，她并不知道公司正在花费大的精力在别的地方开发一套有效的安全系统，而这套系统将会用在玛

丽正在开发的系统上，那么又会怎么样？这种情况是如何改变你对玛丽行为的判断的？（记住，玛丽对这种情况的观点和以前一样。）

2．当一个大型软件系统由许多人一起开发时，如何分担责任？有责任的等级吗？有不同程度的责任吗？

3．我们知道，大型的复杂软件系统通常是由许多人一起开发的，其中很少有人能够对整个项目有全面的了解。那么对一名员工来说，他在对项目功能没有完全了解的情况下为项目出力是合乎道德的吗？

4．一个人对其成果最终如何为他人所用应该负多大的责任？

5．在计算机专业人员与客户之间的关系中，专业人员的责任是实现客户的要求，还是对客户的要求加以指导？如果专业人员预见到客户的要求会导致缺乏职业道德的结果发生，该怎么办？例如，客户可能希望为了提高效率而走捷径，而专业人员预见到采用这些捷径可能会成为数据错误或系统误用的根源。如果客户坚持这么做，专业人员就没责任了吗？

6．如果技术的发展太过迅猛，发明者还未来得及从他的发明中获利，新的发明就已紧随而来，取而代之，将会发生什么？这种获利对激励发明者是必需的吗？开源开发的成功与你的答案有什么关系？免费的高质量软件是可持续现实吗？

7．计算机革命能改善（或者说帮助解决）世界能源问题吗？对其他的大规模问题（如饥饿和贫穷）情况又会如何？

8. 技术会无限期地发展下去吗？什么（如果有）会逆转社会对技术的这种依赖？如果社会继续推进技术无限期地发展下去，结果将会怎样？

9. 如果你有一台时间机器，你想要生活在哪个历史时间段中？有你想带走的当前技术吗？一种技术能与另一种技术分开吗？一边反对全球变暖，一边接受现代医疗，这现实吗？

10. 智能手机上的许多应用会自动集成其他应用提供的服务。这种集成可能会将进入一个应用的信息与另一个应用共享。这种集成的好处是什么？过多的集成会导致什么问题吗？

课外阅读

Alexander, C., S. Ishikawa, and M. Silverstein. *A Pattern Language*. New York: Oxford University Press, 1977.

Beck, K. *Extreme Programming Explained: Embrace Change*, 2nd ed. Boston, MA: Addison-Wesley, 2004.

Braude, E. *Software Design: From Programming to Architecture*. New York: Wiley, 2004.

Bruegge, B. and A. Dutoit. *Object-Oriented Software Engineering Using UML, Patterns, and Java*, 3rd ed. Boston, MA: Addison-Wesley, 2010.

Cockburn, A. *Agile Software Development: The Cooperative Game*, 2nd ed. Boston, MA: Addison-Wesley, 2006.

Fox, C. *Introduction to Software Engineering Design: Processes, Principles and Patterns with UML2*. Boston, MA: Addison-Wesley, 2007.

Gamma, E., R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley, 1995.

LaViola, Jr., J. J., E. Kruijff, R.P. McMahan, D.A. Bowman, and I. Poupyrev. *3D User Interfaces Theory and Practice*, 2nd ed. Boston, MA: Addison-Wesley, 2017.

Pfleeger, S. L. and J. M. Atlee. *Software Engineering: Theory and Practice*, 4th ed. Upper Saddle River, NJ: Prentice-Hall, 2010.

Pilone, D. and N. Pitman. *UML 2.0 in a Nutshell*. Cambridge, MA: O'Reilly Media, 2005.

Pressman, R. S. and B. Maxim. *Software Engineering: A Practitioner's Approach*, 8th ed. New York: McGraw-Hill, 2014.

Schach, S. R. *Classical and Object-Oriented Software Engineering*, 8th ed. New York: McGraw-Hill, 2010.

Shalloway, A. and J. R. Trott. *Design Patterns Explained*, 2nd ed. Boston, MA: Addison-Wesley, 2005.

Shneiderman, B., C. Plaisant, M. Cohen, S. Jacobs, N. Elmqvist, and N. Diakopoulos. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6th ed. Essex, England: Pearson, 2016.

Sommerville, I. *Software Engineering*, 10th ed. Essex, England: Pearson, 2016.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第8章 数据抽象

本章将要研究的是如何对数据排列进行模拟，这门学科称为数据结构，它不同于由计算机的主存储器提供的以一个个单元来组织数据的方式。目标是让数据的使用者将数据集视为一种抽象工具来访问，而不是强迫用户从计算机的主存储器组织的角度去思考。这方面的研究工作将向我们展示，构造这种抽象工具的需求是如何产生对象和面向对象编程概念的。

本章内容

8.1 基本数据结构

8.2 相关概念

8.3 数据结构的实现

8.4 一个简短的案例研究

8.5 定制的数据类型

8.6 类和对象

*8.7 机器语言中的指针

持久理解和学习成果

程序设计得益于适当的抽象。AP[®]CSP

学习成果 使用抽象来管理程序的复杂性。

程序设计使用数学和逻辑概念。AP[®]CSP

学习成果 在程序设计中使用适当的数学和逻辑概念。

数据结构是表示和操纵程序中数据集合的普遍抽象。

学习成果 讨论内置数据结构的适当使用。

学习成果 描述列表、栈、队列、树和其他抽象数据类型的常见应用。

第6章已经介绍了数据结构这个概念，在第6章中我们了解到：程序员可以利用高级程序设计语言提供的技术来表达算法，就好像正被操作的数据不是按照一个个单元存储在计算机的主存储器中的，而是以其他方式存储的。我们还了解到：传统程序设计语言支持的数据结构称为基本结构。本章将讨论一些技术，利用这些技术能够构建和操纵语言的基本结构以外的数据结构，该研究能够使我们从传统数据结构过渡到面向对象范型。贯穿这一过程的底层主题是抽象工具的构建。

8.1 基本数据结构

我们先介绍一些基本的数据结构作为后续几节的例子。用于把数据收集到列表和其他结构的抽象是常见的解决问题工具。

基本知识点

- 计算方法可以用列表和集合来解决问题。

8.1.1 数组和聚合

在6.2节中，已经介绍了数组和聚合类型这两种数据结构。回顾一下，**数组**（array）是一种“矩形的”数据块，其项具有相同的类型。最简单的数组形式是一维数组，由一行元素组成，每个元素的位置由一个下标确定。例如，带有26个元素的一维数组可用于存储每个字母表字母在一页文本中出现的次数。一个二维数组由多行多列组成，其中，项的位置由一对下标确定，即第一个下标值确定项的行位置，第二个下标值确定项的列位置。例如，用一个矩形数组表示销售人员的每月销售额，每行的项代表的是某个销售人员每月的销售额，每列的项代表的是某个月每个销售人员的销售额。这样一来，第3行第1列的项就可以表示第3个销售人员1月的销售额。

与数组不同，**聚合类型**（aggregate type）是一个由数据项组成的块，其中的各个数据项可能具有不同的类型和大小。块里的项通常称为**字段**（field）。聚合类型的一个示例是与单个员工相关的数据块，其字段可能有3项：员工的名字（字符型数组）、年龄（整型）和技能

等级（浮点型）。聚合类型中的字段通常都不是通过数字下标来访问的，而是通过字段名来访问的。

8.1.2 列表、栈和队列

另一种基本数据结构是**列表**（list），它是一个集合，其表项按顺序排列（见图8-1a）。列表的开头称为**表头**（head），列表的结尾称为**表尾**（tail）。

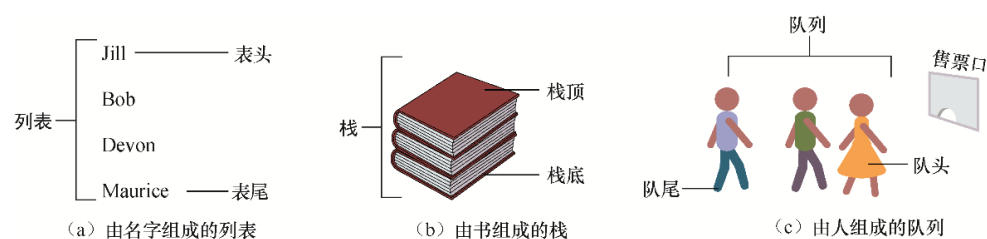


图8-1 列表、栈和队列

几乎所有的数据集合都可以看成列表。例如，文本可以看成符号的列表，二维数组可以看成行的列表，CD上记录的音乐可以看成声音的列表。比较传统的例子包括宾客名单、购物清单、班级注册表和库存清单。与列表相关的操作视情况而定。在某些情况下，我们可能需要从列表中移除项，向列表中添加新项，每次“处理”列表中的一个项，改变项在列表中的排列，或者搜索查看某个特定的数据项是否在列表中。我们将在本章的后面讨论这些操作，但列表及其变体的基本概念可能是算法中最普遍的数据抽象。

基本知识点

- 列表和列表操作（如添加、移除和搜索）在许多程序中很常见。

- 集合的基本操作包括添加元素、移除元素、迭代所有元素以及确定元素是否在集合中。

通过严格限制列表中项的访问方式，我们可以得到两种特殊类型的列表——栈和队列。**栈**（stack）是这样一种列表，其项只能在表头进行添加和删除。例如，一个由书组成的栈，其物理限制决定了所有的添加和删除都只能发生在顶部（见图8-1b）。用通俗的术语来讲，栈的头称为**栈顶**（top），栈的尾称为**栈底**（bottom或base）。在栈顶添加一个新项称为**入栈**（pushing），在栈顶删除一个项称为**出栈**（popping）。注意，最后放入栈中的项总是被最先移除，因此我们说栈是一种**后进先出**（Last-In，First-Out，LIFO，读作“LIE-foe”）结构。

Python列表

内置的Python列表（list）数据结构是程序设计语言抽象的“瑞士美工刀”，可以在更现代的语言中找到。Python列表的属性与较传统的语言中的数组很像——可以通过基于0的索引来访问和分配元素。然而，像聚合类型一样，列表的元素可以是不同的类型和大小，如下例所示：

```
listOfStuff = [42, 'blue', 3.14592]
```

它创建一个名为“listOfStuff”的新列表对象，它包含一个整数、一个字符串和一个浮点数。（和正确的聚合类型不一样，具有异构数据的Python列表仍然可以通过索引访问。Python对象字段可以通过名称访问。）

Python列表具有内置机制，允许添加和删除项、搜索和处理集合中的每个项。

```
# Add a new item to the end of the collection.
listOfStuff.append('clear')
# Print each item in the collection, one at a time.
for item in listOfStuff:
    print(item)
# See if a particular value is in the collection.
if 'clear' in listOfStuff:
    print('clear is in the list')
```

通过只以索引为0使用内置pop()和insert()函数，Python列表可以像后进先出栈数据结构一样工作。

```
# Push a new item onto the front (index 0) of the collection.
listOfStuff.insert(0, 'fraggle')
# Pop it back off.
front = listOfStuff.pop(0)
```

通过只使用内置的pop()和append()函数，Python列表可以像先进先出队列一样工作。

这种后进先出特性意味着，对那些必须以与存储顺序相反的顺序进行检索的数据项而言，栈是理想的选择，因此栈经常被用作回溯活动的基础结构。术语**回溯**（backtracking）是指退出系统的过程，它与进入系统的顺序相反。一个经典的例子是：为了找到走出森林的路而

原路返回。例如，考虑支持递归过程所需的基本结构，每次新的激活开始时，之前的激活都必须搁置一旁。此外，随着每次激活完成，必须检索被搁置的最后一次激活。这样，如果激活在被搁置时被压入栈中，那么每次需要检索激活时，正确的激活将位于栈的顶部。

队列（queue）是这样一种列表，其项只能从表头移除，新项只能从表尾插入。在剧院排队等待购票的一排（或一队）人（见图8-1c）就是这种数据结构的例子，其中排在队伍最前面的人先购票，新到的人必须到队尾排队。在第3章中，我们已经遇到过队列结构，在那节我们看到，批处理操作系统将等待执行的作业存储在称为作业队列的队列中。在那节我们还了解到，队列是一种**先进先出**（first-in，first-out，FIFO，读作“FIE-foe”）结构，这意味着表项是按其存储的顺序从队列中移除的。

第1章中介绍过，队列常被用作缓冲区的基本结构，缓冲区是从一处传送到另一处的数据的临时存储区域。当数据项到达缓冲区时，会被放在队列的末尾。然后，当数据项需要被转发到其最终的目的地时，会按其在队头出现的顺序被转发。因此，数据项转发的顺序就是它们到达的顺序。

8.1.3 树

树（tree）是一种集合，其项具有层次化的组织形式，很像一个典型公司的组织关系图（见图8-2）。在这种组织关系图中，顶部表示总

裁，由分支线向下连到副总裁，副总裁下面是地区经理，等等。对于树结构的这种直观定义，我们还要加上一个额外的限制条件，即（就组织关系图而言）公司的任何一个员工只有一个直接上司。也就是说，组织中的不同分支不会在下一层相遇。（第6章已经举过几个树的例子，是以语法分析树的形式介绍的。）

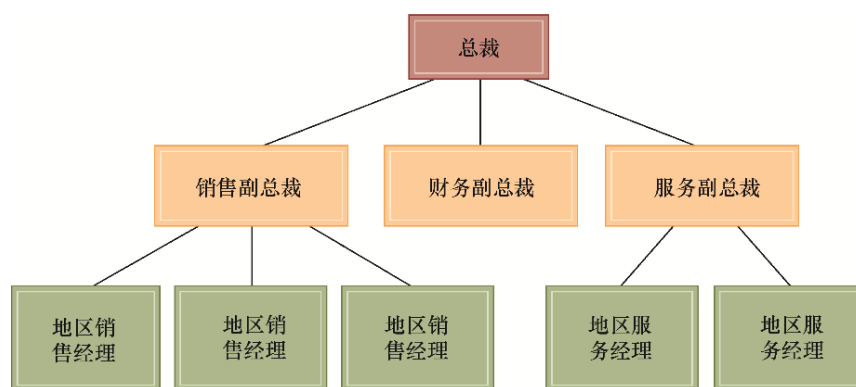


图8-2 组织关系图的一个例子

树中的每一个位置称为一个**节点**（node）（见图8-3）。树顶部的那个节点称为**根节点**（root node）。（如果我们把图倒过来看，这个节点就表示了树的根。）另一端点处的节点称为**终端节点**（terminal node），有时称为**叶节点**（leaf node）。我们常将从根到叶子的最长路径上的节点数称为树的**深度**（depth）。换句话说，一棵树的深度就是该树所包含的水平层数。

有时，我们在提及树结构时，就感觉像是每个节点直接派生出了它下一层的节点。从这个意义上讲，我们常常会说一个节点的祖先或后代。我们称一个节点的直接后代为它的**孩子**（children），称其直接祖先为它的**双亲**（parent）。此外，我们说有同一个双亲的那些节点称

为**兄弟**（sibling）。每个双亲的孩子都不超过两个的树称为**二叉树**（binary tree）。

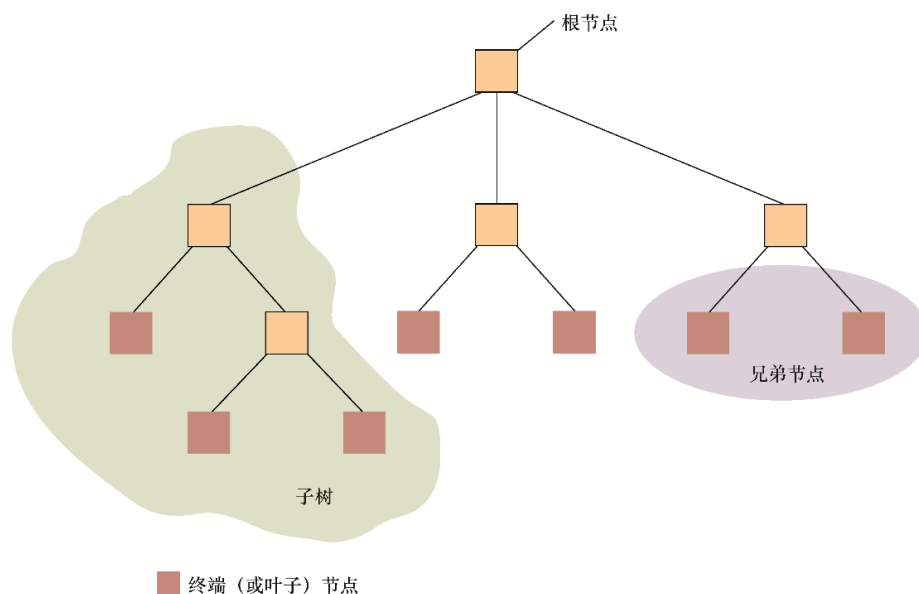


图8-3 树的术语

如果我们选择一棵树中的任意一个节点，就会发现该节点与其下面的那些节点也能构成树结构，我们称这些较小的结构为**子树**（subtree）。这样一来，每个孩子节点都是这个孩子的双亲下面的子树的根，每个这样的子树称为双亲的一个**分支**（branch）。在二叉树中，提到树的显示方式时我们经常会说节点的左分支和右分支。

8.1节问题与练习

1. 举出下列每种结构的一些（计算机科学以外的）例子：列表、栈、队列和树。
2. 总结列表、栈及队列的区别。

3. 假设将字母A放入一个空栈中，然后依次放入字母B和C。再假设一个字母出栈，字母D和E入栈。请将栈中的字母按照从栈顶到栈底出现的顺序列出来。如果此时一个字母要出栈，那么得到的将是哪个字母？

4. 假设将字母A放入一个空队列中，然后依次放入字母B和C。再假设从队列中移除一个字母，插入字母D和E。请将队列中的字母按照从队头到队尾出现的顺序列出来。如果此时从队列中移除一个字母，应该是哪个字母？

5. 假设一棵树有4个节点：A、B、C和D。如果A和C是兄弟，并且D的双亲是A，那么哪些节点是叶节点？哪个节点是根节点？

8.2 相关概念

在本节中，我们将分别讨论3个与数据结构紧密相关的主题：抽象、静态结构与动态结构的区别以及指针的概念。

8.2.1 再谈抽象

前一节中介绍的结构通常与数据有关。然而，计算机的主存储器并不是按照数组、列表、栈、队列和树这样的结构来组织的，而是由一组可寻址的存储单元顺序组成的。这样一来，所有的其他结构都必须进行模拟。如何完成这种模拟是本章的主题。到现在为止，我们只是指出，数组、列表、栈、队列和树这样的组织都是被创建的抽象工

具，之所以构造这些抽象工具，是为了便于对数据用户屏蔽实际数据存储的细节，允许用户访问，就像这些信息是以一种较为方便的形式存储的一样。

在这里，用户这个术语并不一定是指人，这个词的含义取决于我们当时的视角。如果从一个使用PC来维护保龄球联赛记录的人的角度考虑，那么用户就是一个人。在这种情况下，应用软件（也许是电子表格软件包）将负责把数据用人觉得方便访问的抽象形式表示出来（很可能用数组来表示）。如果从因特网上的一个服务器的角度考虑，那么这时的用户可能是一个客户机。在这种情况下，服务器将负责把数据表示成便于客户机访问的抽象形式。如果从程序的模块化结构来考虑，那么用户应该是需要访问这些数据的任何模块。在这种情况下，包含数据的模块应该负责把数据表示成便于其他模块访问的抽象形式。所有这些情况中，有一条共同的主线，那就是用户拥有将数据作为一个抽象工具来访问的特权。

8.2.2 静态结构与动态结构

构建抽象数据结构中的一个重要区别是，所模拟的结构是静态的还是动态的，也就是说，结构的形状或大小是否会随时间改变。例如，如果这个抽象工具是一个名字列表，那么考虑以下情况将非常重要：这份名字列表是会一直保持固定的大小，还是会因名字的添加和删除而扩大和缩小。

就一般规律而言，静态结构比动态结构更容易管理。如果一个结构是静态的，那么只需要提供一种能够访问结构中不同数据项的方法，或许还需要提供一种能改变指定位置的值的方法。但是，如果结构是动态的，就必须处理添加和删除项的问题，还要找到因数据结构增长所需的存储空间。在结构设计不合理的情况下，添加一个新项可能会导致对结构进行大规模的重排，而且结构的过度增长可能会迫使整个结构转移到另一个可用空间更大的存储区域。

垃圾收集

随着动态数据结构的扩大和缩小，存储空间会被使用和释放。回收不用的存储空间以备将来使用的过程称为**垃圾收集**（garbage collection）。许多场合都需要垃圾收集机制。操作系统里的内存管理程序在分配和回收存储空间时必须进行垃圾收集。文件管理程序在计算机的海量存储器上进行文件的存储和删除操作时，也要进行垃圾收集。此外，在分派程序控制下运行的任何进程，可能需要在分配给它的存储空间中进行垃圾收集。

垃圾收集涉及一些微妙的问题。对于链式结构，每次改变指向数据项的指针，垃圾收集程序都必须决定是否回收指针原先指向的那个存储空间。在涉及多路径指针交叉的数据结构中，问题会变得尤为复杂。不准确的垃圾收集例程会导致数据丢失，或者存储空间的低效使用。例如，如果垃圾收集无法成功回收存储空间，可用空间就会慢慢减少，这种现象称为**内存泄漏**（memory leak）。

8.2.3 指针

我们知道计算机的主存储器中的各个单元是由数字地址来标识的。作为数值，这些地址本身就可以进行编码并存储在存储单元中。**指针**（pointer）就是包含这种编码地址的一个存储区。就数据结构来说，指针是用来记录存储数据项的位置的。例如，如果我们必须要重复地将一个数据项从一个位置移到另一个位置，那么可以指定一个固定的位置来作为一个指针。这样一来，我们每次移动该项时，就能够通过更新这个指针来反映数据的新地址。之后，当我们需要访问该数据项时，就可以通过指针来找到该项。事实上，指针将一直“指向”该数据。

在第2章学习CPU的过程中，我们已经遇到过指针这个概念。在那一章中，我们看到，称为程序计数器的寄存器被用来存放下一条要执行的指令的地址。因此，程序计数器就起到了指针的作用。事实上，程序计数器的另一个名字是**指令指针**（instruction pointer）。

举一个指针应用的例子，假设我们有一份小说的清单，按照小说名的字母顺序存储在计算机的存储器中。虽然在许多应用中这样的排列比较方便，但是要找到某个作者的所有小说作品就比较难了，因为它们分散在整个列表中。为了解决这个问题，我们可以在表示一本小说的每个存储单元块中预留一个额外的存储单元，并将该存储单元用作指针，指向表示这一作者的书的另一个存储块。通过这种方法，同一作者的所有小说就可以链接成一个环（见图8-4）。一旦找到给定作者的一本小说，就可以循着指针一本接另一本地找到该作者的所有小说。

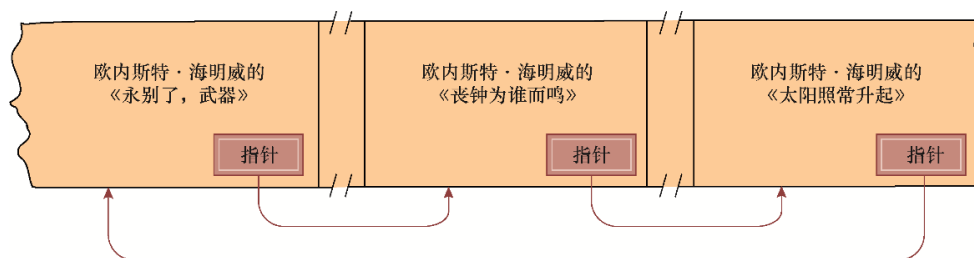


图8-4 按小说名排列、根据作者链接的小说

许多现代程序设计语言都把指针作为一种基本数据类型。也就是说，这些语言允许对指针进行声明、分配和操作，就像对整数和字符串那样。利用这种语言，程序员就能在机器的存储器中，把相关的项用指针相互链接起来，从而设计出精巧的数据网。

8.2节问题与练习

1. 数组、列表、栈、队列和树等数据结构在何种意义上是抽象的？
2. 请举一个涉及静态数据结构应用的例子。再举出一个涉及动态数据结构应用的例子。
3. 请举一个在计算机科学领域外出现指针这个概念的例子。

8.3 数据结构的实现

现在我们来讨论8.2节介绍的一些数据结构在计算机的主存储器中的存储方式。正如第6章中介绍的，在高级程序设计语言中，常常将这些结构作为基本结构来提供。我们这里的目标是要理解如何将处理这

些结构的程序翻译成用来处理存放在主存储器中的数据的机器语言程序。

8.3.1 存储数组

我们先讨论存储数组的技术。

假设我们想存储一个24小时温度读数的序列，每个读数需要存储空间的一个存储单元。此外，假设依据它们在序列中的位置来确定这些读数。也就是说，我们想要能够访问第1个读数或者第5个读数。简单来说，就是想要按照一维数组的方式来处理这个序列。

这里，只要将这些读数按顺序存储在具有连续地址的24个存储单元中，就可以实现这个目标了。在这种情况下，如果这个序列中第1个单元的地址是 x ，那么任何一个指定温度读数可以通过“用所需读数的下标减去1，然后将计算的结果加上 x ”来得到。具体来说，第4个读数就放在 $x + (4 - 1)$ 这个地址中，如图8-5所示。

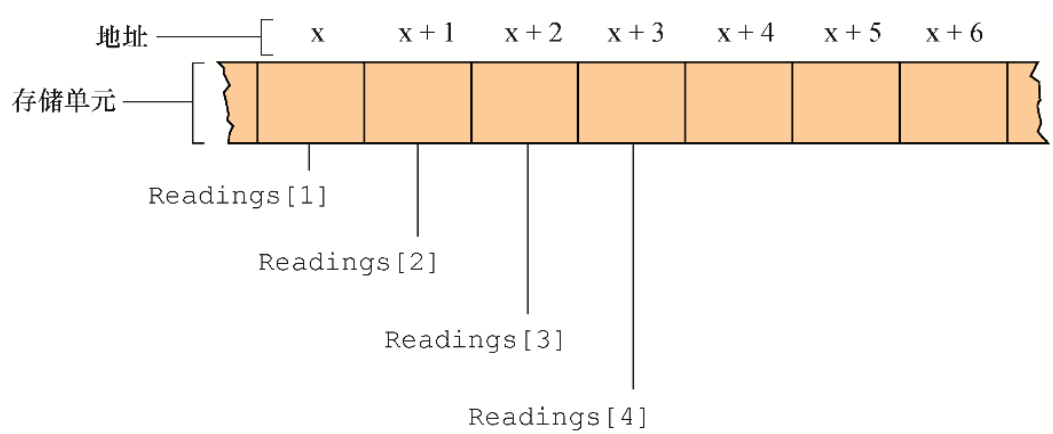


图8-5 存储在存储器中的温度读数数组，起始地址为 x

这种技术为大多数高级程序设计语言的翻译器所采用，用以实现一维数组。当翻译器遇到下面这样的声明语句时：

```
int Readings[24];
```

就表明，Readings这个术语是指可以存放24个整数值的一维数组，这时，翻译器会安排预留24个连续的存储单元。在程序的后面，如果遇到赋值语句

```
Readings[4] = 67;
```

就要求将值67放入数组Readings的第4项中。此时，翻译程序就生成了一串机器指令，把值67放入地址为 $x + (4 - 1)$ 的存储单元中，其中 x 为与数组Readings相关的存储块中第一个单元的地址。通过这种方式，程序员在编写程序的时候，就可以认为温度读数确实存储在一个一维数组中。（注意，在Python、C、C++、C#和Java语言中，数组的下标是从0开始的，而不是从1开始的，这样一来，第4个读数应该由Readings[3]表示。见本节末的问题与练习3。）

现在，假设我们要记录一个公司的销售人员一周内的销售额。在这种情况下，我们可以想象将数据排列成一个二维数组。在该数组中，每行的值表示某个员工的销售额，而每列的值表示某一天内所有的销售额。

标准模板库

本章所讨论的数据结构已经成为标准的编程结构，事实上，因为其标准性，许多编程环境都把它们当作原语一样对待。在C++编程环境中就能找到一个示例，即通过标准模板库（Standard Template Library，STL）使该环境的功能更为强大。标准模板库中有一组描述常用数据结构的预定义的类。因此，通过在C++程序中并入标准模板库，程序员就可以从描述这些结构细节的任务中解放出来。他们只需用8.6节中将StackOne声明为StackOfIntegers类型的方式，声明标识符是这些类型的即可。

为了满足这种要求，首先要认识到这个数组是静态的，即使它的内容得到更新，其大小也不会改变。这样我们就可以计算出整个数组所需的存储区的总数，从而预留出一块这个大小的连续存储单元。接下来，我们把数据一行一行地存入数组。从所预留的存储块的第1个存储单元起，把数组第1行的值存入连续的存储器位置；接着存放下一行，再下一行，以此类推（见图8-6）。我们说这种存储系统采用的存储方式是**行主序**（row major order），与之相对的是**列主序**（column major order），列主序是将数据一列一列地存入数组。

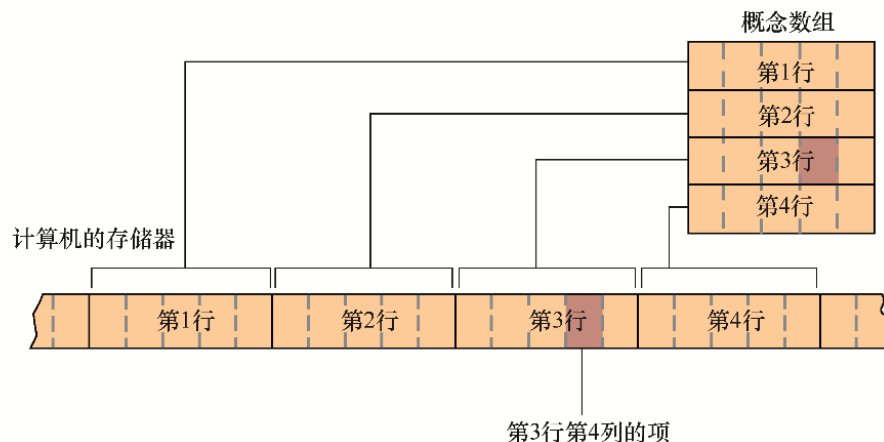


图8-6 以行主序方式存储的一个4行5列的二维数组

如果数据以这种方式存储，那么考虑一下，如何找到数组中第3行第4列的值。设想一下，我们处在计算机的存储器的预留块的第1个单元。从这个位置起，我们可以依次找到数组第1行的数据、第2行、第3行，依次类推。要得到第3行的数据，我们必须先经过第1行和第2行。由于每一行有5项（从星期一到星期五，每天1项），因此我们必须总共经过10项才能到达第3行的第1项。从那里，我们还要再经过3项，才能到达第3行第4列的那个项。这样，为了到达第3行第4列的项，我们必须经过从存储块的开始处算起的13项。

上述计算过程可以概括为一个公式，即可以将按行列位置的引用转换为实际的存储地址。具体来说，如果令 c 表示一个数组的列数（也就是每行所包含的项数），那么第 i 行第 j 列项的地址就可以表示为

$$x + (c \times (i - 1)) + (j - 1)$$

其中， x 是包含第1行第1列的项的单元的地址。也就是说，我们必须经过 $i-1$ 行（其中每行包括 c 项），才能到达第 i 行，然后再经过 $j-1$ 项，才能到达这行的第 j 项。在我们之前的例子中， $c = 5$ ， $i = 3$ ， $j = 4$ ，所以，如果数组从地址 x 开始存储，那么第3行第4列的项将在地址 $x + (5 \times (3 - 1)) + (4 - 1) = x + 13$ 处。表达式 $(c \times (i - 1)) + (j - 1)$ 有时称为**地址多项式**（address polynomial）。

再强调一次，这也是大多数高级程序设计语言的翻译器所采用的技术。当遇到声明语句

```
int Sales[8,5];
```

时，则表明Sales是一个8行5列的二维整数数组，因此翻译器会留出40个连续的存储单元。以后如果遇到赋值语句

```
Sales[3,4] = 5;
```

请求将值5放到数组Sales的第3行第4列的那个项中，翻译器就会构建一串机器指令，将值5放到地址为 $x + 5 \times (3 - 1) + (4 - 1)$ 的存储单元中，其中， x 是与数组Sales相关联的存储块的第1个单元的地址。通过这种方式，程序员编写程序时，就好像销售额确实存储在一个二维数组中一样。

8.3.2 存储聚合

现在，假设要存储名为Employee的聚合，它包含3个字段组成：Name（字符型数组）、Age（整型）和SkillRating（浮点型）。如果每个字段所需的存储单元的数目是固定的，那么就可以将聚合存储在一个连续的单元块中。例如，假设Name字段最多需要25个单元，Age只需要一个单元，SkillRating也只需要一个单元。那么，我们就可以预留出一个有27个连续单元的存储块，开始的25个存储单元用来存储员工的名字，第26个存储单元用来存储员工的年龄，最后一个存储单元用来存储员工的技能等级（见图8-7a）。

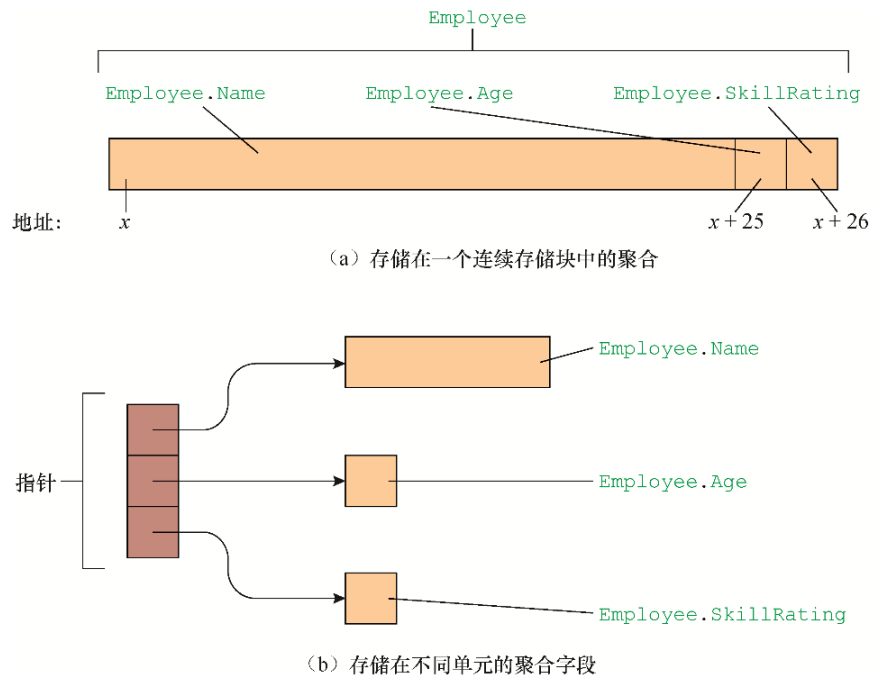


图8-7 存储聚合类型Employee

用这种排列就可以很容易地访问该聚合中不同的字段。在知道聚合开始地址以及聚合中所需字段的偏移量的情况下，字段的引用可以被翻译成存储单元。例如，如果第一个存储单元的地址是 x ，那么指向`Employee.Name`（意思是`Employee`聚合中的`Name`字段）的任何引用都将翻译成从地址 x 开始的25个存储单元，而指向`Employee.Age`（意思是`Employee`聚合中的`Age`字段）的引用将翻译成地址为 $x+25$ 的存储单元。具体来说，如果翻译器遇到了高级语言程序中的这样一条语句：

```
Employee.Age = 22;
```

那么翻译器会构建所需的机器语言指令，用以将值22放入地址为 $x+25$ 的存储单元。

在一个连续存储单元块中存储聚合的另一种方法是，将聚合的每个字段分别存放在不同的位置，然后通过指针的方式将它们链接在一起。更准确地说，如果这个聚合包含3个字段，那么就在存储器中找到一个位置，用以存储3个指针，每个指针指向一个字段（见图8-7b）。如果这些指针存储在以 x 为起始地址的存储块中，那么通过存储在地址 x 处的指针就可以找到第1个字段，通过存放在地址 $x + 1$ 处的指针就可以找到第2个字段，依次类推。

这种排列对聚合字段的大小是动态的情况尤其适用。举例来说，利用这种指针系统，只需要在存储器中找到一个区域来存放较大的字段，然后调整相关的指针指向这个新位置，就可以增加第一个字段的大小。但是，如果聚合存放在一个连续的存储块中，就不得不修改整个结构。

8.3.3 存储列表

现在来讨论将一个名字列表存储在计算机的主存储器中的技术。一种策略是将整个列表存入具有连续地址的一整块存储单元中。假定每个名字不超过8个字母，我们可以把这个大块的整块存储单元分成一组子块，每个子块包含8个存储单元。每个子块中存储一个用ASCII码记录的名字，一个单元存储一个字母。如果一个名字不足以填满分配给它的子块的所有存储单元，只需用空格的ASCII码将剩余的单元填满就行。利用这个系统，存储一个10个名字的需要一个有80个连续存储单元的存储块。

图8-8所概括的就是刚刚描述的存储系统。重点是，整个列表都存储在一大块存储单元中，其连续的项依次存放在相邻的存储单元中。这种组织称为**邻接表**（contiguous list）。

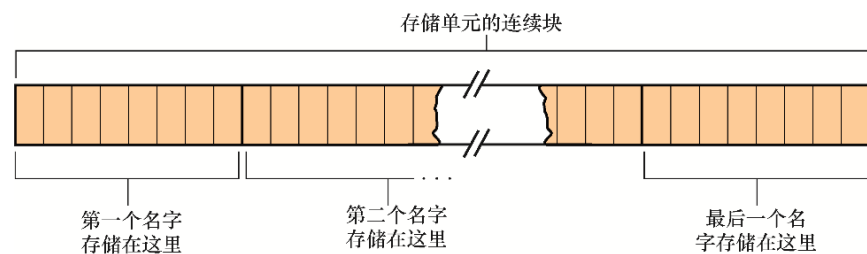


图8-8 名单作为一个邻接表存储在存储器中

邻接表的实现

大多数高级程序设计语言都提供了构建和操作数组的原语，它们都是构建和操作邻接表的方便工具。如果列表的项都是相同的基本数据类型，那么该列表就是一个一维数组。一个更复杂一点儿的例子是文中讨论过的一个有10个名字的列表，其中每个名字不超过8个字符。这种情况下，程序员可以将一个邻接表构建为一个10行8列的二维字符数组，该表可以用图8-6所表示的结构来表示（假设该数组是以行主序进行存储的）。

许多高级语言都含有支持这种列表实现的功能。例如，假设将上面提到的二维字符数组称为MemberList，那么在传统的表示法中用表达式MemberList[3, 5]来指代第3行第5列的那个字符；但在有些程序设计语言中用表达式MemberList[3]来指代整个第3行，也就是列表的第3项。

邻接表这种存储结构用来实现静态列表很方便，但用来实现动态列表就不方便了，因为名字的添加和删除都会导致不断地对项进行移位操作，这样就比较耗时。在最差情况下，添加项需要把整个列表移到一个新位置，才能获得足够大的可用存储单元块来存储扩展过的列表。

如果我们允许一个列表的各个项存储在不同的存储区中，而不是存储在一个大的连续存储块中，这些问题就可以得到简化。为了说明这个问题，仍然考虑存储名字列表的例子（其中每个名字不超过8个字母）。这次，将每个名字存储在一个有9个连续存储单元的存储块中。前8个存储单元用来存放名字本身，最后一个单元用作指针，指向列表中的下一个名字。根据这个思路，整个列表可以分散在几个小的由指针链接起来的9单元块中。正是因为这个链接系统，这种组织方式称为**链表**（linked list）。

为了记录链表的起点，我们另外再设一个指针，用来存储第一项的地址。因为这个指针指向链表的起点，或者叫头，所以将这个指针称为**头指针**（head pointer）。

为了标记链表的结尾，我们使用了**空指针**（null pointer），在一些程序设计语言中也称为**NIL指针**（NIL pointer），在Python中称为None对象，它只是放在最后一项的指针单元中的一个特殊位模式，用来表明链表中不会再有别的项。例如，如果约定从不在地址0处存储列表项，那么值0就永远不会作为合法的指针值出现，因而就可以将值0用作空指针。

最终的链表结构如图8-9所示，图中用几个单个的矩形表示存放链表的分散存储块。每个矩形都标识出了它们的组成部分。每个指针由一个从指针本身指向指针被访问地址的箭头表示。要遍历整个链表，就需跟随头指针找到第一个表项，由此项开始，跟随项中存储的指针从一项跳到下一项，直至遇到空指针。

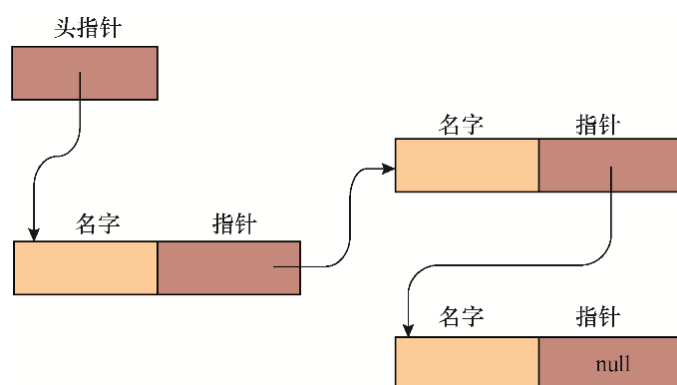


图8-9 链表的结构

为了说明链表相对于邻接表的优势，这里考虑一个任务——删除一个项。在邻接表中，删除一个项就会产生一个空缺，这就意味着，被删除项后面的项必须向前移动才能保持列表连续。然而，在链表的情况下，删除一个项只需改变一个指针。也就是说，将原本指向被删除项的指针修改成指向被删除项后面的那个项就可以做到（见图8-10）。这样一来，当遍历这个链表时由于被删除项已不再是链的一部分，因而会被忽略。

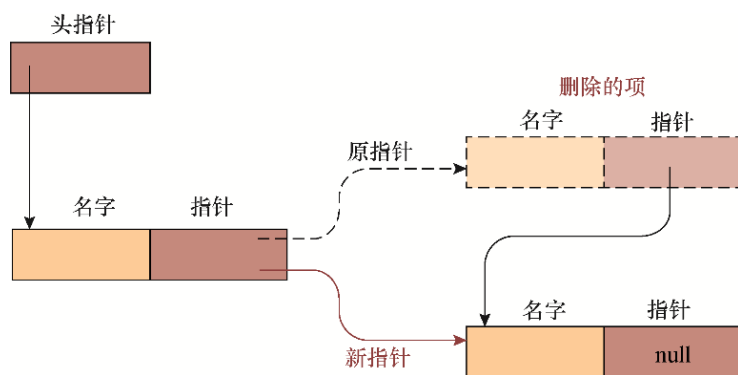


图8-10 从链表中删除一个项

在链表中插入一个新项就稍微麻烦一点。我们要先找到一个足够容纳这个新项及其指针的未用存储块，然后存入该项，并将该项的指针填为应接在新项后面的那个项的地址。最后，修改该新项的前一项的指针，令其指向新项（见图8-11）。做了这样的修改后，每次遍历链表的时候，就能在合适的位置找到新项。

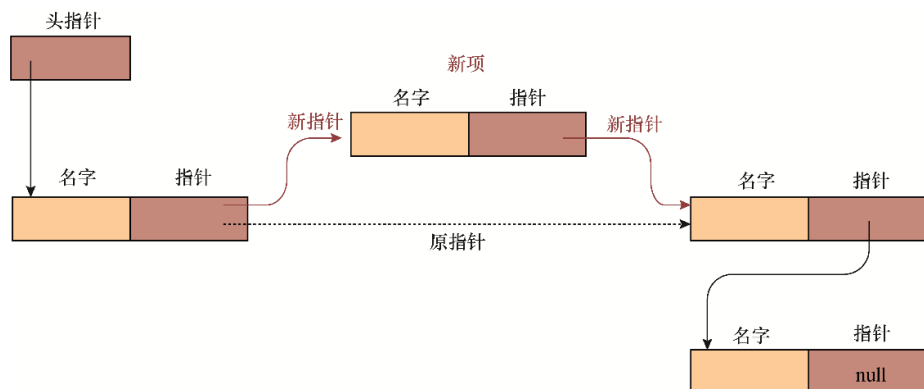


图8-11 向链表中插入一个项

指针的一个问题

就像使用流程图会导致杂乱的算法设计（见第5章）、随意使用goto语句会导致蹩脚的程序（见第6章）一样，乱用指针也会产生不必要的复杂性和易错的数据结构。为了克服这种混乱，许多程序设计

语言严格限制了指针的灵活性。例如，Java语言不允许一般形式的指针，只允许称为引用的受限形式的指针。一个区别在于，引用不能被算术运算修改。举例来说，如果Java程序员想把Next引用前移至邻接表中的下一项，那么他应该用等价于下面这条语句的语句：

将Next重新定位到下一个列表项

而C程序员则会用等价于下面这条语句的语句：

将Next的值赋给Next + 1

注意，Java语句能更好地反映其根本目标。此外，为了执行这条Java语句，必须有另一个列表项，但是，如果Next已经指向了列表中的最后一项，那么这条C语句将导致Next指向列表外的某样东西——这是C程序员新手甚至经验丰富的老手经常犯的一个错误。

8.3.4 存储栈和队列

为了存储栈和队列，通常采用一种类似于邻接表的组织方式。就栈而言，所预留的存储块的大小要足够容纳其大小达到最大时的栈。（确定这个存储块的大小往往是很关键的设计决定。如果预留的空间太小，则栈最终可能会超出所分配的存储空间；然而，如果预留的空间太多，则会浪费存储空间。）将这个存储块的一端指定为栈底，入栈的第一个项就存储在这里。于是，再入栈的项就放在其上一个入栈的项的旁边，这样一来栈就向着预留块的另一端生长。

注意，在项入栈和出栈时，栈顶的位置会在预留的存储单元块中来回移动。为了跟踪这个位置，栈顶地址会存储在一个称为**栈指针**（stack pointer）的额外存储单元中。也就是说，栈指针是指向栈顶的指针。

如图8-12所示，整个栈系统是这样工作的：为了向栈中压入一个新项，首先要调整栈指针，令其指向刚好超出栈顶部的空闲处，然后将新项存放在这个位置。为了从栈顶弹出一个项，先读取栈指针指向的那个数据，然后调整栈指针，令其指向栈中的下一个项。

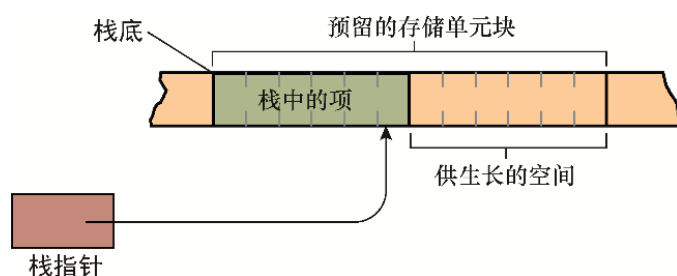


图8-12 存储器中的一个栈

队列的传统实现方法类似于栈的实现方法，也是在主存储器中预留一块连续的存储单元，其大小要足以容纳其大小达到最大时的队列。然而，就队列而言，需要在队列的两端进行操作，所以不能像栈那样只用一个指针，这里需预留两个存储单元用作指针。一个指针称为**头指针**（head pointer），用来记录队列的队头；另一个指针称为**尾指针**（tail pointer），用来记录队列的队尾。当队列为空时，这两个指针指向同一个位置（见图8-13）。每次向队列中插入项时，就将该项放在由尾指针指向的位置，然后调整尾指针，令其指向下一个未用位置。通过这种方式，尾指针始终指向队尾的第一个空闲处。要从队列

中移除一个项，需要先读取头指针指向的那个项，然后调整头指针，令其指向队列中的下一项。

至此，所描述的这种存储系统存在一个问题，即随着项的插入和移除，队列会像冰川一样在存储器中慢慢移动（见图8-13）。这样一来，就需要一种机制，使队列保持在预留的存储块中。这个问题的解决方案很简单，就是让队列自始至终都在所分配的存储块中移动。于是，当队尾到达队列的末端时，插入新项的操作就要回到存储块的起始端进行，这时的起始端是空闲的。同样，当存储块中最后一项成为队头并被移除时，就将头指针调回到存储块的开头，此时其他项正在那里等待。用这种方式，队列在存储块内按环状依次排列，就像存储块的尾部被连接起来形成了一个环（见图8-14）。这种技术所实现的队列称为**环状队列**（circular queue）。

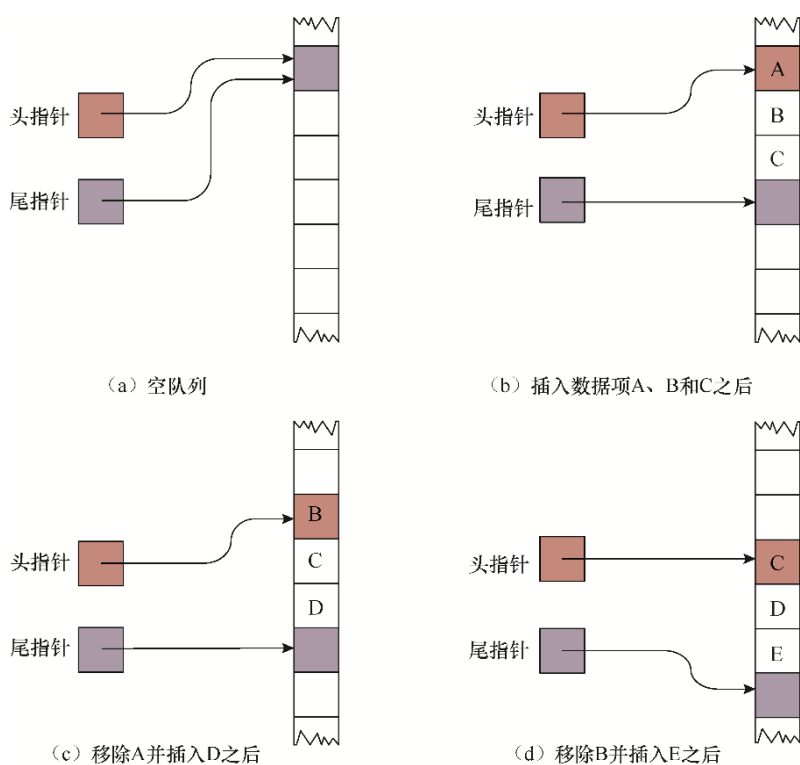


图8-13 用头指针和尾指针实现一个队列。注意，随着项的插入和移除，队列是如何在存储器中慢慢移动的

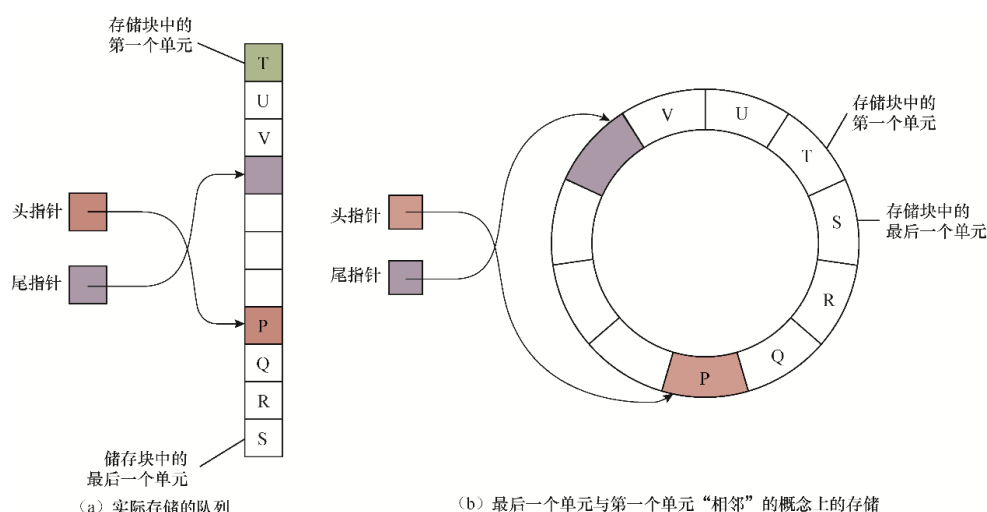


图8-14 包含字母P到字母V的环状队列

8.3.5 存储二叉树

为了讨论树存储技术，这里我们将注意力集中在二叉树上。我们说过，二叉树的每个节点至多有两个孩子。这种树通常采用类似于链表所用的链式结构存储在存储器中。然而，二叉树的每个项（或称为节点）不是由两个元素（依次为数据和指向下一个节点的指针）组成的，而是由3个元素组成：（1）数据；（2）指向该节点的第一个孩子的指针；（3）指向该节点的第二个孩子的指针。尽管在机器中不存在左右之分，但这里为了方便，称第一个指针为**左孩子指针**（left child pointer），而称另一个指针为**右孩子指针**（right child pointer），这样我们就可以方便地在纸上画出树的图形了。因此，二叉树的每个节点可由一个短的、连续的存储单元块来表示，其格式如图8-15所示。

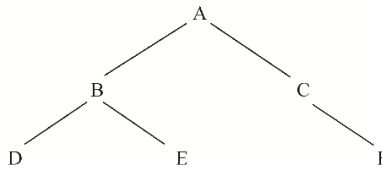


图8-15 二叉树中的一个节点的结构

要在存储器中存储树，首先要找到可用的存储单元块来存放节点，然后依据想要的树结构将这些节点链接起来。每个指针必须设置成指向其相关节点的左孩子或者右孩子，如果树的某个方向不再有节点，则将相应的指针赋值为null。（这意味着，终端节点的特征是其两个指针的值都为null。）最后，我们留出一个专门的存储器位置，称为**根指针**（root pointer），用来存储根节点的地址。对树的访问就是从根指针开始的。

图8-16所示的就是这样一种链式存储系统的例子，图中既展示了二叉树的概念结构，又展示了该树在计算机的存储器中的实际存储形式。注意，树的节点在主存储器中的实际排列与概念上的排列有很大的不同。不过，沿着根指针就能找到根节点，然后跟着相应的指针从一个节点到另一个节点跟踪树的任何路径。

概念树



实际存储结构

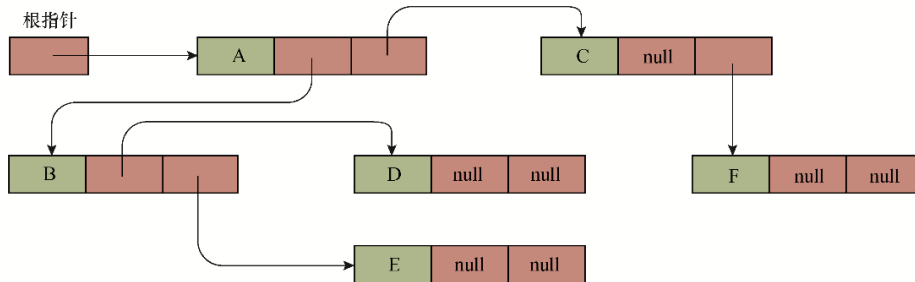
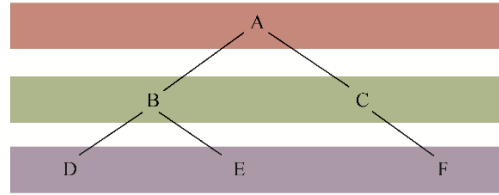


图8-16 二叉树的概念组织形式和使用链式存储系统的实际组织形式

对于二叉树存储，除了用链式结构，还可以用单个的连续存储单元块来存储整棵树。我们用这种方法把树的根节点存储在这个存储块的第1个单元中。（为了简单起见，假设树的每个节点只需要一个存储单元。）然后，把根的左孩子存储在第2个单元中，把根的右孩子存储在第3个单元中，一般而言，继续把在单元 n 中找到的节点的左孩子和右孩子分别存储在单元 $2n$ 和 $2n+1$ 中。在存储块中，没有被树用到的单元就用一个特别的位模式来标识，表示这个位置没有数据。利用这种技术，图8-16中的树可以像图8-17所示的那样存储。注意，这个存储系统本质上是自上到下将树中各层的节点作为存储段来存储，一层接着一层。也就是说，存储块中的第1项是根节点，接下来是根的孩子，然后是根的孙子，依次类推。

概念树



实际存储结构

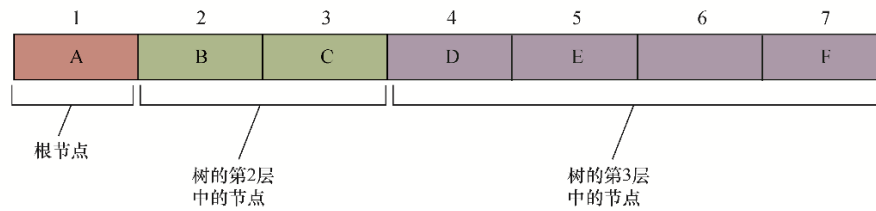


图8-17 一棵没用指针存储的树

与前面描述的链式结构相比，这种存储系统在查找某个节点的双亲或兄弟方面更为有效。一个节点的双亲的位置可以这样确定：将该节点的位置除以2，然后丢掉余数（如位置为7的节点的双亲将是位置为3的节点）。一个节点的兄弟的位置可以这样确定：如果该节点的位置为偶数，则用兄弟的位置加上1；如果该节点的位置为奇数，则用兄弟的位置减去1。例如，位置为4的节点的兄弟是位置为5的节点；位置为3的节点的兄弟是位置为2的节点。此外，当二叉树接近平衡（也就是说，根节点下的两个子树具有同样的深度）和完全平衡（也就是说，该树没有瘦长的分支）的情况下，这种存储系统对存储空间的利用更为有效。不过，对于不具备这些特征的树，该系统的效率就会变得非常低，如图8-18所示。

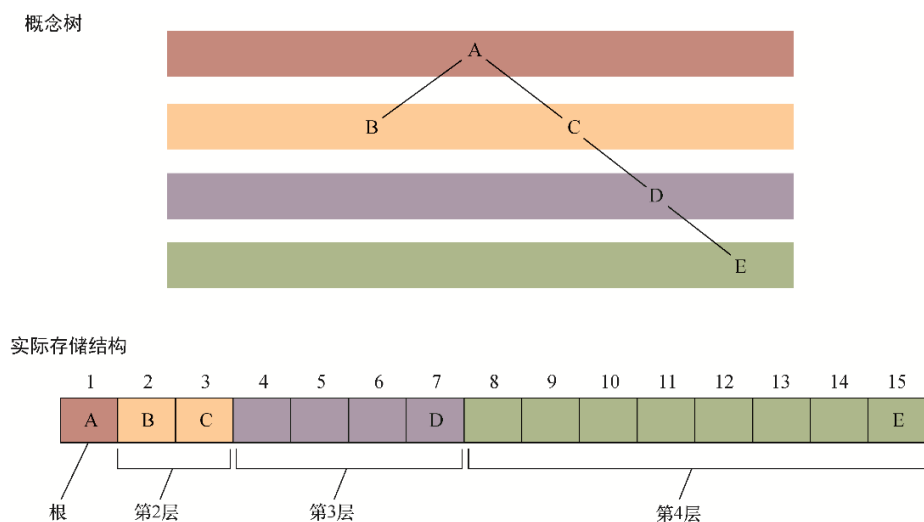


图8-18 一棵稀疏不平衡树的概念形式以及不用指针的存储方式

8.3.6 操控数据结构

我们已经看到，数据结构在计算机的存储器中的实际存储方式与用户想象的概念结构是不同的。二维数组实际上并不是存储在一个二维的矩形存储块中，列表或树实际上可能由分散在较大的存储区域内的小片段组成。

因此，为了允许用户将数据结构当作一种抽象工具来访问，必须对用户屏蔽实际存储系统的复杂性。这意味着，用户给出的（以及按照抽象工具的方式规定的）指令必须转换成适合实际存储系统的步骤。就数组而言，我们已经看到，这是如何利用地址多项式将行、列下标转换成存储单元地址来做到的。具体来说，我们已看到，打算用抽象数组的程序员编写的下列语句是如何被转换为完成对主存储器进行正确修改的步骤的：

```
Sales[3, 4] = 5;
```

同样，我们已看到，下面这种涉及抽象聚合类型的语句是如何根据该聚合的实际存储情况被翻译成合适的动作的：

```
Employee.Age = 22;
```

在列表、栈、队列和树这些情况下，根据抽象结构定义的指令通常是通过函数转换为合适的动作的，其中的函数在向用户屏蔽底层存储系统的细节的同时还完成了所需的任务。例如，如果提供 `insert()` 函数是用来向链表中插入新项的，那么只要执行下面的函数调用，就可以将 J. W. Brown 插入 Physics 208 班的注册学生列表中：

```
insert("Brown, J.W.", Physics208)
```

注意，这个函数调用完全是根据抽象结构声明的，通过这种方式可以把表的实际实现过程隐藏起来。

作为一个更详细的例子，图8-19展示了一个名为 `printList()` 的用来打印值的链表的函数。这个函数假设称为 `List` 的聚合中的 `Head` 字段指向了链表中的第一项，并且链表中的每个项都由两部分组成：值（`Value`）和指向下一项的指针（`Next`）。在图8-19中，Python 的特殊值 `None` 被用作空指针。一旦这个函数开发完成，就可以作为一个抽象工具用来打印链表，而无须关心打印链表所需的实际步骤。例如，要获得 Economics 301 班的打印班级列表，用户只需执行下面的函数调用就能得到想要的结果：

```
printList(Economics301ClassList)
```

此外，如果我们以后想改变列表的实际存储方式，那么只需改变 `printList()` 函数的内部动作，用户仍然可以继续使用以前的那个函数调用来完成列表打印。

```
def PrintList(List):  
    CurrentPointer = List.Head  
    while (CurrentPointer != None):  
        print(CurrentPointer.Value)  
        CurrentPointer = CurrentPointer.Next
```

图8-19 一个打印链表的函数

8.3节问题与练习

1. 说明下面的数组是如何以行主序的方式存储在主存储器中的。

5	3	7
4	2	8
1	9	6

2. 如果一个二维数组是以列主序而不是行主序的方式存储的，那么请给出一个能找到该二维数组中第 i 行第 j 列的项的公式。

3. 在Python、C、C++、Java和C#这些程序设计语言中，数组的下标都是从0开始的，而不是从1开始。因此，名为Array的数组的第1行第4列的项可通过 `Array[0][3]` 来引用。在这种情况下，翻译器将

用什么样的地址多项式把`Array[i][j]`这样的引用格式转换成存储器地址呢？

4．什么条件表明链表为空？

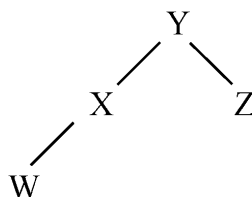
5．修改图8-19中的函数，使其打印出某个指定的名字之后就停止打印。

6．根据本节提到的在一个连续的存储单元块中实现栈的技术，什么条件表明栈为空？

7．请说明，在高级语言中，如何用一维数组来实现栈？

8．如果队列是用本节描述的循环方式实现的，那么当队列为空时，头指针和尾指针之间的关系如何？队列满时，关系又如何？如何检测队列是满的还是空的？

9．依据本节所讲的内容，画出下面这棵树在采用左孩子指针和右孩子指针来存储时，其在存储器中存放的情况。然后，利用本节所讲的树的另一种存储系统，画一幅图，表示利用连续存储块存放时的情况。



8.4 一个简短的案例研究

现在考虑一个按字母顺序存储名字列表的任务。假设要对这个列表进行如下操作：

搜索 (search) 一个项是否存在， 按字母顺序打印 (print) 列表， 插入 (insert) 一个新项

我们的目标是开发一个带有一组函数的存储系统来实现这些操作，从而产生一个完整的抽象工具。

首先考虑存储这个列表的几种可选的方法。如果按照链表方式来存储，就需要以顺序的方式对列表进行搜索，在第5章中就讨论过这样一个过程。如果列表很长，这个处理过程的效率就会非常低。因此我们要寻找另外一种实现方法，使得我们在搜索过程中能够利用到二分搜索算法（见5.5节）。要利用这种算法，我们必须能从所采用的存储系统中找到这个列表较小部分里的中间项。我们的解决办法是，将列表用二叉树的形式进行存储。首先让列表的中间项成为根节点，然后把列表余下部分的头一半的中间项作为根的左孩子，把列表余下的后一半的中间项作为根的右孩子。接下来，将列表余下的每个四分之一部分的中间项变为根节点的孩子的孩子，依次类推。例如，图8-20中的树就表示了包含字母A、B、C、D、E、F、G、H、I、J、K、L和M的一个字母列表。（我们约定，当所讨论的列表的一部分有偶数个项时，取中间两项里较大的那个为中间项。）

为了搜索以这种方式存储的列表，要先将目标值与根节点进行比较。如果它们相等，则搜索成功；如果它们不相等，则根据目标值是小于根节点还是大于根节点，决定是移到根的左孩子还是移到右孩子。这样我们找到了继续搜索所必需的列表部分的中间项。这个“比较并移到孩子”的过程会一直继续，直到找到目标值（说明搜索成功了）

或者遇到了空指针（None）但却没有找到目标值（说明搜索失败了）为止。

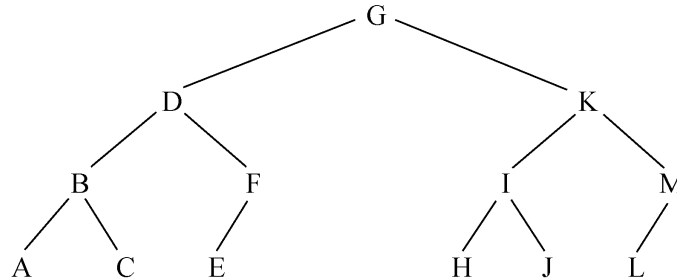


图8-20 字母A到M排列成一个有序树

图8-21给出了用链式树结构表示这种搜索过程的方式。Python中的elif关键字是“else: if ...”的简写。注意，这里的这个函数只是图5-14中那个函数的一个细化，图5-14表示的是二分搜索算法的原始陈述。这两个图的区别主要是表面上的。原先描述的算法是对列表的相继变小的片段进行搜索，而这里描述的算法是对相继变小的子树进行搜索（见图8-22）。

```
def Search(Tree, TargetValue):
    if (Tree is None):
        return None          # Search failed
    elif (TargetValue == Tree.Value):
        return Tree          # Search succeeded
    elif (TargetValue < Tree.Value):
        return Search(Tree.Left, TargetValue)
    # Apply the function Search to see if TargetValue is in the
    # subtree identified by the root's left child pointer, and
    # report the result of that search.
    elif (TargetValue > Tree.Value):
        return Search(Tree.Right, TargetValue)
    # Apply the function Search to see if TargetValue is in the
    # subtree identified by the root's right child pointer, and
    # report the result of that search.
```

图8-21 当将列表实现为链式二叉树时二分搜索的样子

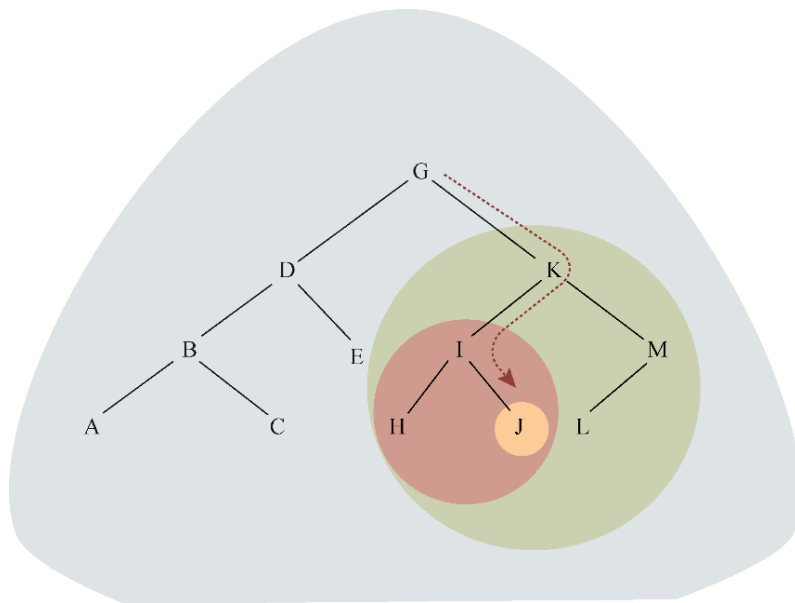


图8-22 利用图8-21中的函数搜索字母J所考虑的相继变小的树

当把“列表”存储为二叉树时，你可能会认为，现在按照字母顺序打印这个列表的过程变难了。然而，要按照字母顺序打印这个列表，只需先按字母顺序打印出左子树，然后打印出根节点，接下来再按字母顺序打印出右子树即可（见图8-23）。毕竟，左子树包含的所有元素都小于根节点，而右子树包含的所有元素都大于根节点。到目前为止，该算法的逻辑框架如下所示：

```
if (树非空):  
    按照字母顺序打印左子树  
    打印根节点  
    按照字母顺序打印右子树
```

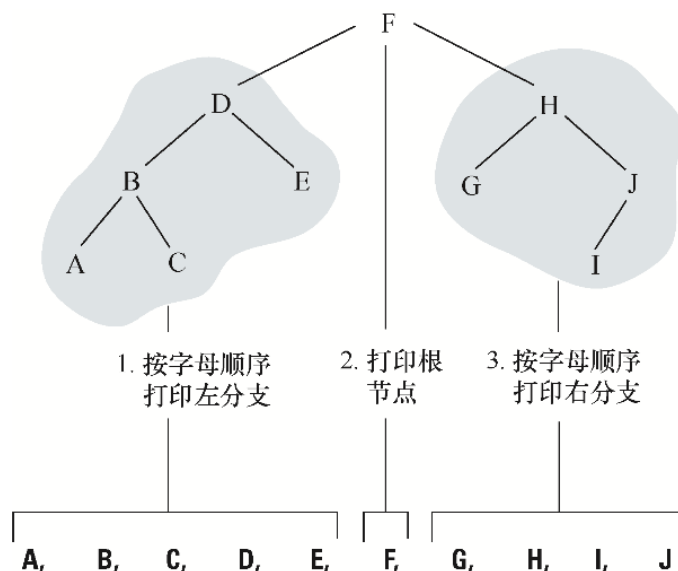


图8-23 按字母顺序打印一棵搜索树

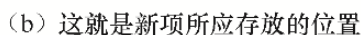
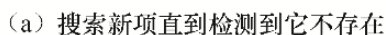
这个框架中包含按照字母顺序打印左子树和右子树这两项任务，这两项任务本质上是原始打印任务的缩小版本。也就是说，解决打印一棵树的问题涉及打印子树的更小的任务，这就使人想到运用递归方法来解决我们这棵树的打印问题。

沿着这个思路，可以把我们最初的设想扩展为一个完整的打印二叉树的Python函数，如图8-24所示。这里，将该函数命名为PrintTree()，然后调用PrintTree()打印左子树和右子树。注意，递归过程的终止条件（遇到一个空子树None）肯定能达成，因为函数的每次后续激活都是在比引起这次激活的树更小的树上进行的。

```
def PrintTree(Tree):  
    if (Tree is None):  
        PrintTree(Tree.Left)  
        print(Tree.Value)  
        PrintTree(Tree.Right)
```

图8-24 用于打印二叉树中数据的函数

在树中插入一个新项的任务也比起初看起来的要容易。凭直觉你也许会认为，插入新项只需先将树切开，为新项留出空间，但实际上，不论要添加的节点的值如何，总是可以作为一个新的叶节点插入到树中。为了给新项找到合适的位置，要沿着搜索该项时所遵循的那条路径往下走。因为该项并不在树中，所以我们最终会遇到一个空指针——这个位置就是存放新节点的合适位置（见图8-25）。事实上，这就是搜索新项时会到达的位置。



对于链式树结构，表达这个过程的函数如图8-26所示。它首先在树中搜索要插入的值（称为NewValue），然后把包含NewValue的一个新叶节点放到合适的位置。注意，如果在搜索过程中发现要插入的项已经在树中，则不进行插入操作。图8-26中的Python代码使用函数调用TreeNode()创建了一个新聚合，用作这个链式树结构的一个新叶节点。这需要图中没展示的额外代码来把TreeNode标识为用户定义的类型，用户定义的类型将在下节中介绍。

我们得出这样一个结论：一个由链式二叉树结构连同用于搜索、打印、插入操作的函数组成的软件包提供了一个完整的包，这个包可以作为我们假想应用的一个抽象工具。事实上，如果实现恰当，用户

在使用这个包时将无须关心底层的实际存储结构。利用这个包中的过程，用户可以想象名字列表是按字母顺序存储的，而事实上，这些“列表”项实际分散在链接为二叉树的存储单元块中。

```
def Insert(Tree, NewValue):
    if (Tree is None):
        # Create a new leaf with NewValue
        Tree = TreeNode()
        Tree.Value = NewValue
    elif (NewValue < Tree.Value):
        # Insert NewValue into the left subtree
        Tree.Left = Insert(Tree.Left, NewValue)
    elif (NewValue > Tree.Value):
        # Insert NewValue into the right subtree
        Tree.Right = Insert(Tree.Right, NewValue)
    # If NewValue was neither less than or greater than root,
    # value is already present, so make no change.
    return Tree
```

图8-26 一个用于向存储为二叉树的列表中插入新项的函数

8.4节问题与练习

1. 画一个二叉树，可以用来存储由R、S、T、U、V、W、X、Y和Z组成的列表，以备将来搜索之用。
2. 简要说明，将图8-21中的二分搜索算法应用到图8-20中的树中查找项J时所经历的路径。查找项P时的路径又是怎样的？
3. 画一个图，表示将图8-24中的递归打印树算法用在图8-20中的有序树上打印K节点时的活动状态。
4. 有一个树结构，它的每个节点有26个孩子。试解释这样一个结构是如何对英语中拼写正确的单词进行编码的。

8.5 定制的数据类型

在第6章中，我们介绍了数据类型的概念，并讨论了几种基本数据类型，如整型、浮点型、字符型和布尔型。大多数程序设计语言都提供了这些基本数据类型。此外，为了更好地满足某个具体应用的需要，程序员可以定义一些自己的数据类型，本节将考虑几种定义方式。

8.5.1 用户定义的数据类型

如果除了程序设计语言中提供的那些基本数据类型，还有其他数据类型可用，那么表达一个算法就会变得更容易。出于这个原因，许多现代程序设计语言都允许程序员利用基本数据类型作为构建块，定义一些附加的数据类型。这种“自制”数据类型的最基本的例子被称为**用户定义的数据类型**（user-defined data type），其本质就是几个基本数据类型组合而成的具有同一名字的聚合体（conglomerate）。

为了对此进行解释，这里假设我们想要开发一个涉及许多变量的程序，每个变量都有相同的聚合结构，都由名字、年龄和技能级别组成。一种方法是将每个变量分别定义成聚合类型（见6.2节）。然而，更好的办法是将这个聚合定义成一种新的（用户定义的）数据类型，然后就把这种新的数据类型作为一种基本类型来使用。

回顾6.2节中的例子，在C语言中，下面的语句定义了一个称为Employee的新聚合，其中包含3个字段Name（字符型）、Age（整型）和SkillRating（浮点型）：


```
struct
{
    char    Name[25];
    int     Age;
    float   SkillRating;
} Employee;
```

与之相反，下面的C语句并没有定义新聚合变量，而是定义了一个新聚合类型EmployeeType：

```
struct EmployeeType
{
    char    Name[25];
    int     Age;
    float   SkillRating;
};
```

这样一来，就可以采用与基本数据类型相同的变量声明方式，用这个新的数据类型来声明变量了。也就是说，与C语言允许用语句

```
int x;
```

将变量x声明为整型的方式一样，可以用语句

```
struct EmployeeType Employee1;
```

将变量Employee1声明为Employee Type类型。然后，在程序的后面，变量Employee1将引用一整块存储单元，该存储块中包括员工的姓名、年龄和技能级别。存储块中的各个项可以通过诸如Employee1.Name和Employee1.Age这样的表达式来引用。因此，下面这样的语句可能会被用来将值26赋给Employee1块中的Age字段：

```
Employee1.Age = 26;
```

此外，下面这样的语句形式可以用来将3个变量DistManager、SalesRep1和SalesRep2声明为EmployeeType类型：

```
struct EmployeeType DistManager, SalesRep1, SalesRep2;
```

就像下列形式的语句通常被用作将变量Sleeve、Waist和Neck声明为基本的float类型一样：

```
float Sleeve, Waist, Neck;
```

区分用户定义的数据类型和这个类型的一个实际项非常重要。后者称为类型的一个**实例**（instance）。用户定义的数据类型本质上是一个用来构建类型实例的模板。该模板描述了这种类型的所有实例所具有的属性，但是它本身并非这种类型的一个实例（这就好比，饼干成型切割刀是做饼干的模板，但其本身不是饼干）。在上面的例子中，用户定义的数据类型EmployeeType构建了3个该类型的实例——DistManager、SalesRep1和SalesRep2。

8.5.2 抽象数据类型

在许多程序设计语言里，用户定义的数据类型（如C语言的结构和Pascal语言的记录）都扮演着重要的角色，帮助软件设计者根据特定程序的需要量身定制数据表示。然而，传统的用户定义的数据类型

仅允许程序员定义新的存储系统，而没有提供对具有这些结构的数据进行的操作。

抽象数据类型（abstract data type，ADT）是一种用户定义的数据类型，可以同时包含数据（表示）和函数（行为）。支持抽象数据类型创建的程序设计语言通常都有两个特征：（1）具有“用于将抽象数据类型定义为单个单元”的语法；（2）具有“将抽象数据类型的内部结构隐藏起来，不让程序的其他部分利用”的隐藏机制。第一个特征是一个重要的组织工具，能将抽象数据类型的数据和函数组织在一起，这简化了维护和调试。第二个特征提供了可靠性，它能阻止抽象数据类型以外的其他代码访问其数据，无须通过专为此目的提供的函数。

基本知识点

- 列表和其他集合可被视为开发程序过程中的抽象数据类型（ADT）。

为了对此进行说明，这里假设我们想要在一个程序中创建和使用几个整数值的栈。方法可以是，将每个栈实现成一个有20个整数值的数组。将栈底的项放在（压入）数组的第一个位置，将栈的其他项相继放入（压入）数组的高位项（见8.3节的问题与练习7）。再用一个额外的整型变量作为栈指针，用来存放数组项的下标，而下一个栈项将会被压入该数组中。因此，每个栈都由一个包含栈本身的数组和一个起着栈指针作用的整数组成。

为了实现这个构想，首先要用下列形式的C语句来建立一个称为StackType的用户定义的类型：

```
struct StackType
{
    int StackEntries[20];
    int StackPointer = 0;
};
```

（回顾一下，在C、C++、C#和Java等语言中，数组StackEntries的下标范围是0~19，因此这里将StackPointer的值初始化为0。）做了这个声明之后，我们就可以通过下列语句声明名为StackOne、StackTwo和StackThree的栈：

```
struct StackType StackOne, StackTwo, StackThree;
```

此时，变量StackOne、StackTwo和StackThree中的每一个都可以引用一个唯一的存储单元块，用于实现各自的栈。但是，如果现在想把值25压入StackOne会怎么样呢？当然，我们希望能屏蔽以栈的实现为基础的数组结构的细节，仅将栈作为一种抽象工具来使用，即可能会用类似于

```
push(25, StackOne);
```

这样的一个函数调用。但是，如果不定义称为push()的相应函数，这样的一条语句就不可用。我们想对StackType类型的变量进行的其他操作有：从栈中弹出项、检查栈是否为空以及检查栈是否已满。所有这些操作都要求定义相应的函数。简而言之，我们的StackType数据类型的定义并不包括我们想让这个类型关联的所有属

性。此外，程序里的任何函数都有可能访问我们的StackType变量的StackPointer字段和StackEntries字段，绕开我们设计到函数push()和pop()中的仔细检查。程序另一部分中的草率的赋值语句会重写存储在栈数据结构中间的数据元素，甚至会破坏所有栈共有的特征——后进先出行为。

我们需要一种机制，既能定义允许对StackType进行的操作，又能保护内部变量不被外部引用。Java语言的interface语法就是这样一种机制。例如，在Java中，可以写

```
interface StackType
{
    public int      pop(); /* Get the item at top of stack */
    public void push(int item); /* Add a new item to stack */
    public boolean  isEmpty(); /* Check if stack is empty */
    public boolean  isFull(); /* Check if stack is full */
}
```

单独来讲，这个抽象数据类型没有指定如何存储栈，或者用什么算法来执行push()函数、pop()函数、isEmpty()函数和isFull()函数。这些细节（已被这个interface抽象出来）会被其他Java代码在其他地方指定。然而，像我们前面的用户定义的数据类型一样，程序员能把变量或函数参数声明为StackType类型的。

我们可以用下列语句把StackOne、StackTwo和StackThree声明为栈：

```
StackType StackOne, StackTwo, StackThree;
```

在程序的后面（这3个变量开始是空引用，在使用前必须用具体的Java类实例化，但这里我们不关心这些细节），我们可以用下面这样的语句将这些项压入这些栈中：

```
StackOne.push(25);
```

该语句表示用值25作为实际参数，执行与StackOne相关联的push()函数。

相对于更基本的用户定义的数据类型而言，抽象数据类型是完整的数据类型。在20世纪80年代，抽象数据类型在Ada等语言中的出现，代表着程序设计语言在设计方面前进了一大步。今天，面向对象语言提供了称为类的抽象数据类型的扩展版本，在下一节中将会介绍。

8.5节问题与练习

1. 数据类型与该数据类型的实例的区别是什么？
2. 用户定义的数据类型与抽象数据类型的区别是什么？
3. 请描述一种用来实现列表的抽象数据类型。
4. 请描述一种用来实现支票账户的抽象数据类型。

8.6 类和对象

在第6章中我们讨论过，面向对象范型使得系统可由称为对象的单元组成，这些对象通过彼此交互来完成任务。每个对象都是一个响应

来自其他对象的消息的实体。对象由称为类的模板描述。

在许多方面，这些类实际上就是抽象数据类型（其实例称为对象）的描述。例如，图8-27展示了，Java语言和C#语言是如何定义StackOfIntegers类的。（C++语言中的等价类的定义具有相同的结构，只是语法略有不同。）注意，这个类为每个在抽象数据类型StackType中声明的函数都提供了一个函数体。此外，这个类包含一个称为StackEntries的整型数组和一个用来确定数组中栈顶位置的整数StackPointer。

```
class StackOfIntegers implements StackType
{
    private int[] StackEntries = new int[20];
    private int StackPointer = 0;

    public void push(int NewEntry)
    { if (StackPointer < 20)
      StackEntries[StackPointer++] = NewEntry;
    }

    public int pop()
    { if (StackPointer > 0) return StackEntries[--StackPointer];
      else return 0;
    }

    public boolean isEmpty()
    { return (StackPointer == 0); }

    public boolean isFull()
    { return (StackPointer >= MAX); }
}
```

图8-27 用Java和C#实现的整型栈

用这个类作为模板，在Java或C#程序中用语句

```
StackType StackOne = new StackOfIntegers();
```

或者在C++程序中用语句

```
StackOfIntegers StackOne();
```

来创建一个名为StackOne的对象。在后面的程序中，可以用语句

```
StackOne.push(106);
```

将值106压入StackOne中，或者用语句

```
OldValue = StackOne.pop();
```

检索StackOne的栈顶项放到变量OldValue中。

这些特征与抽象数据类型的那些特征本质上是一样的。不过，类与抽象数据类型之间还是有区别的。前者是后者的扩展。例如，我们在6.5节中介绍过，面向对象语言允许类从其他类继承属性，并允许类包含称为构造函数的特殊方法——当创建对象时用其来定制个性化的对象。此外，类会有不同程度的封装性（见6.5节），这样就可以使其实例的内部属性免受非正常的影响，同时公开其他字段给外部访问。

我们得出结论：类和对象的概念体现了程序中数据抽象的表示技术又前进了一步。事实上，正是由于这种以方便的方式定义和使用抽象的能力，才有了面向对象设计范型的流行。

8.6节问题与练习

1. 抽象数据类型与类在哪些方面相似？在哪些方面不同？
2. 类与对象的区别是什么？
3. 请描述一个类，用该类作为构建整型队列类型的对象的模板。

*8.7 机器语言中的指针

本章已经介绍过指针，并说明了如何用指针来构建数据结构。本节将讨论如何在机器语言中处理指针。

假设我们想要用附录C中描述的Vole机器语言写一个程序，从图8-12中描述的栈中弹出一项，然后将其放入一个通用寄存器中。换句话说就是，我们想要将含有栈顶项的存储单元中的内容加载到一个寄存器中。我们的机器语言提供了两条用于加载寄存器的指令：一条是用操作码0x2，另一条是用操作码0x1。回想一下，在操作码0x2的情况下，操作数字段包含要加载的数据，而在操作码0x1的情况下，操作数字段则包含要加载的数据的地址。

因为不知道内容会是什么，所以用操作码0x2达不到目的。而且，因为不知道地址会是什么，所以用操作码0x1也达不到目的。毕竟，栈顶的地址会随着程序的执行而变化。不过，我们知道栈指针的地址。也就是说，我们知道所要加载的数据的地址的位置。于是，我们需要的就是用于加载寄存器的第3个操作码，其中操作数字段包含了指向要加载的数据的指针的地址。

为了实现这个目标，我们对附录C中的Vole语言进行扩展，使其包含操作码0xD。使用这个操作码的指令的形式可能是DRXY，表示将地址为XY的存储单元的内容加载到寄存器R中（见图8-28）。因此，如果栈指针在地址0xAA的存储单元中，指令0xD5AA就能将栈顶的数据加载到寄存器5中了。

然而，这条指令并没有完成出栈操作。我们还必须将栈指针减1，以便让它指向新的栈顶。这意味着，在加载指令之后，机器语言程序还必须将栈指针加载到一个寄存器中，将其减去1，再把结果存回到存储器。

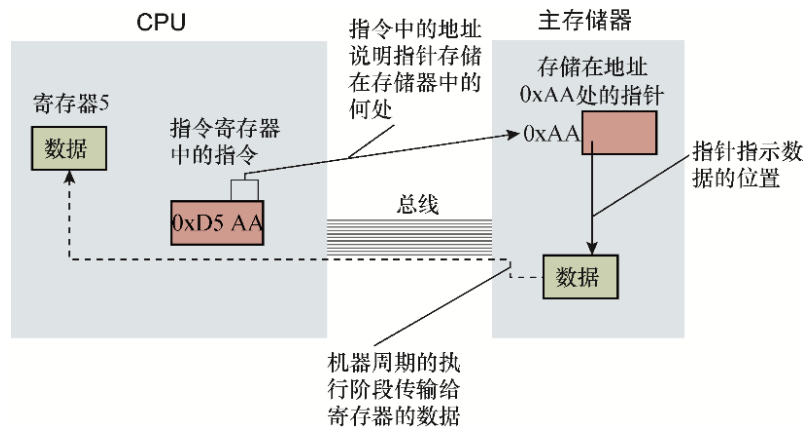


图8-28 首次尝试扩展附录C中的Vole机器语言来利用指针

如果不用存储单元，而用某个寄存器来作为栈指针，就可以减少栈指针在寄存器与存储器之间的来回移动。但是，这意味着我们必须重新设计加载指令，以便它期望指针在寄存器中，而不是在主存储器中。这样一来，我们就不用早些时候的那个方案了，使用操作码0xD定义一条指令，令其具有DR0S的形式即可，这就表示将由寄存器S指向的存储单元的内容加载到寄存器R（见图8-29）。于是，一个完整的

出栈操作就可以这样来完成：在这条指令之后添加一条指令（或几条指令），将存储在寄存器S中的值减去1。

注意，要实现压入操作，还需要一条类似的指令。因此，还要进一步扩展Vole机器语言，引入操作码0xE，以便0xER0S形式的指令将表示把寄存器R的内容存储到由寄存器S指向的存储单元中。同样，为了完成压入操作，在这条指令后面添加一条指令（或几条指令），将寄存器S中的值加上1。

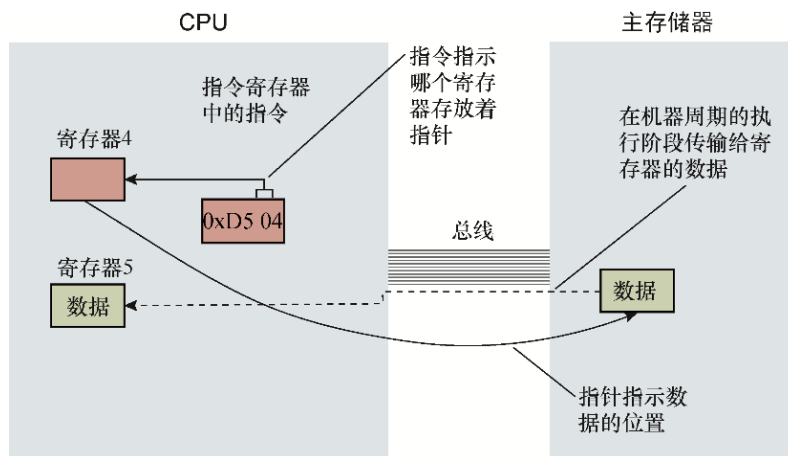


图8-29 把存储在寄存器中的一个指针指向的存储单元的内容加载到一个寄存器中

我们提出的这些新操作码0xD和操作码0xE，不仅说明了机器语言是如何被设计来操纵指针的，还说明了在最初的机器语言中没有提到的寻址技术。正如附录C中所提到的，机器语言用两种方式来确定一条指令中涉及的数据。第一种方式是用一条操作码为0x2的指令来表示。这里，操作数字段就明确包含了所涉及的数据。这种寻址技术称为**立即寻址**（immediate addressing）。确定数据的第二种方式是用操作码为0x1和0x3的指令来表示。这里，操作数字段包含的是所涉及的

数据的地址。这种寻址技术称为**直接寻址**（direct addressing）。然而，我们提出的新操作码0xD和0xE则表明还有一种确定数据的形式。这些指令的操作数字段包含的是数据的地址的地址。这种寻址技术称为**间接寻址**（indirect addressing）。所有这3种寻址技术在今天的机器语言中都是比较常见的。

8.7节问题与练习

1. 假设Vole机器语言（见附录C）按照本节最后的建议进行了扩展。此外，假设寄存器8中的内容为模式0xDB，地址为0xDB的存储单元中的内容为模式0xCA，地址为0xCA的存储单元的内容为模式0xA5。在执行完下面的每一条指令之后，寄存器5中的位模式将是什么？

a. 0x25A5 b. 0x15CA c. 0xD508

2. 利用本节最后描述的扩展，写一段完整的Vole机器语言例程，完成出栈操作。假设栈是按图8-12所示的方式实现的，栈指针在寄存器F中，并且栈顶出栈后存入寄存器5中。

3. 利用本节最后描述的扩展，写一个程序，将从地址0xA0开始的5个连续的存储单元的内容复制到从地址0xB0开始的5个存储单元。这里假设程序的起始地址为0x00。

4. 本章中介绍了一条0xDR0S形式的机器指令。假设将这个形式扩展为0xDRXS，意思是：把寄存器S中的值加上值X，然后将结果所指向的数据加载到寄存器R中。这样一来，通过读取寄存器S中的值，再加上值X，就可以得到指向数据的指针。寄存器S中的值不会发生变

化。（如果寄存器F的内容为0x04，那么指令0xDE2F就把地址为0x06的存储单元的内容加载到寄存器E中，而寄存器F的值保持0x04不变。）这条指令的优点是什么？如果一条指令的形式为0xDRTS，表示“把寄存器S的值和寄存器T的值相加，然后把所得的结果所指向的数据加载到寄存器R中”，那么这条指令又有什么优点？

复习题

（带*的题目涉及选读章节的内容。）

1．当下列数组分别以行主序和列主序在机器的存储器中存储时，画图说明该数组的存储情况。

A	B	C	D
E	F	G	H
I	J	K	L

2．假设一个6行8列的数组是按照行主序存储的，其起始地址为20（十进制）。如果数组中的每个项只需要一个存储单元，那么数组中第3行第4列的项的地址是什么？如果每个项需要两个存储单元，那么结果又如何？

3．假设第2题中的数组是以列主序存储的而不是以行主序存储的，重做第2题。

4. 如果想利用传统的一维数组来实现动态列表，会带来什么样的复杂情况？

5. 描述一种用来存储三维数组的方法。请问这里用来定位第*i*面、第*j*行、第*k*列的项的地址多项式是什么？

6. 假设字母列表A、B、C、D、E、F和G存储在一个连续的存储单元块中。在保持列表字母顺序不变的情况下，向列表中插入字母D需要进行哪些操作？

7. 下面的表格表示的是计算机的主存储器中的一些存储单元的内容以及每个存储单元的地址。注意，其中有些存储单元包含字母表中的字母，而每个这样的存储单元后面都跟随一个空存储单元。在这些空存储单元中填入适当的地址，使每个包含一个字母的存储单元和它后面的存储单元一起，构成一个链表中的项，其中字母要按字母顺序排列。（用0表示空指针。）这里的头指针包含的地址是什么？

地 址	内 容
0x11	'C'
0x12	
0x13	'G'

地 址	内 容
0x14	
0x15	'E'
0x16	
0x17	'B'
0x18	
0x19	'U'
0x1A	
0x1B	'F'
0x1C	

8．下面的表格表示的是计算机的主存储器中一个链表的一部分。链表中的每项由两个单元组成：第一个单元包含的是字母表中的一个字母；第二个单元包含的是一个指向链表下一项的指针。请更改指针，使字母N不再出现在链表中；然后，用字母G代替字母N，并更改相应的指针，使新字母按字母顺序出现在链表中的合适位置。

地 址	内 容
0x30	'J'
0x31	0x38
0x32	'B'
0x33	0x30
0x34	'X'
0x35	0x46
0x36	'N'
0x37	0x40
0x38	'K'
0x39	0x36
0x3A	'P'
0x3B	0x34

9．下面的表格使用与前两题相同的格式表示了一个链表。如果头指针包含的值是0x44，那么这个链表表示的名字是什么？改变指针，使得这个链表包含名字Jean。

地 址	内 容
0x40	'N'
0x41	0x46
0x42	'T'
0x43	0x40
0x44	'J'
0x45	0x4A
0x46	'E'
0x47	0x00
0x48	'M'
0x49	0x42

地 址	内 容
0x4A	'A'
0x4B	0x40

10．下面的哪个例程能够正确地将NewEntry项紧挨着插到链表中名为Previous-Entry的项的后面？另外一个例程有什么问题？

例程1：

- 1．将PreviousEntry指针字段的值复制到NewEntry的指针字段。
- 2．将PreviousEntry指针字段的值改成NewEntry的地址。

例程2：

- 1．将PreviousEntry指针字段的值改成NewEntry的地址。
- 2．将PreviousEntry指针字段的值复制到NewEntry的指针字段。

11．设计一个函数，将两个链表拼接起来（也就是说，把一个链表放到另一个链表的前面，形成一个链表）。

12．设计一个函数，将两个已排序的邻接表合并成一个已排序的邻接表。如果列表是链表呢？

13．设计一个函数，对一个链表进行反向排列。

14 . a. 设计一个算法，用栈作为辅助存储结构，以相反的顺序打印一个链表。

b . 设计一个递归函数，在不显式使用栈结构的情况下完成同样的任务。这个递归解决方案涉及的栈会采用什么形式？

15 . 有时，一个单链表可以有两种不同的顺序，只要为每个项附加两个指针，而不是一个指针即可。请填写下面的表格，使得通过紧跟每个字母的第一个指针，就可以找到名字Carol；而通过紧跟每个字母的第二个指针，就可以按照字母顺序找到字母。所表示的这两个链表的头指针分别包含什么值？

地 址	内 容
0x60	'O'
0x61	
0x62	
0x63	'C'
0x64	
0x65	

地 址	内 容
0x66	'A'
0x67	
0x68	
0x69	'L'
0x6A	
0x6B	
0x6C	'R'
0x6D	
0x6E	

16．下面的表格表示的是文中讨论过的，存储在连续存储单元块中的一个栈。如果这个栈的栈底地址是0x10，而栈指针包含值0x12，那么出栈指令取出的是何值？执行出栈操作后，栈指针中的值是什么？

地 址	内 容
0x10	'F'
0x11	'C'
0x12	'A'
0x13	'B'
0x14	'E'

17．如果第16题中的指令是向栈中压入字母D，而不是弹出一个字母，那么请画出一个表格来展示存储单元中最后的内容。在执行入栈指令后，栈指针中的值是什么？

18．设计一个函数，从一个栈中移除栈底的项，而栈中的其他项保持不动。这里只能用出栈和入栈操作来访问栈。为了解决这个问题，应该用什么样的辅助存储结构？

19．设计一个函数，比较两个栈的内容。

20．假设给你2个栈，如果一次只允许你从一个栈移动一个项到另一个栈，那么原始数据可能会重新排列成什么样？如果给你3个栈，那么原始数据可能会排列成什么样？

21. 假设给你3个栈，并且一次只允许你从一个栈移动一个项到另一个栈。设计一个算法，颠倒其中一个栈中的两个相邻项。

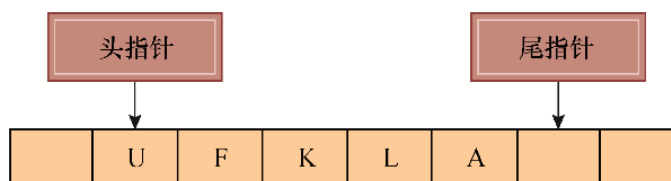
22. 假设要创建一个存储名字的栈，其中名字的长度不同。那么，为什么说“把名字存储在分散的存储区域，再建立一个指向这些名字的指针的栈，而不是在栈中存储名字本身”更有利？

23. 队列在存储器中是向其头部的方向慢慢移动，还是向其尾部的方向慢慢移动？

24. 假设要实现一个“队列”，该队列中的新项都有相应的优先级。这样，一个新项就会被放在那些优先级较低的项前面。请描述一个实现这种“队列”的存储系统，并证明你的结论。

25. 假设队列中的每个项都需要一个存储单元，其头指针包含值11，尾指针包含值17。当向队列中插入一项并移走两项之后，这些指针的值又是多少？

26. a. 假设一个队列是以环状队列的形式实现的，其状态如下图所示。请画图表示在插入字母G和R、移除3个字母、插入字母D和P之后队列的结构。



b. 在(a)中，如果在移除任何字母之前插入字母G、R、D和P，会出现什么错误？

27. 描述如何在用高级语言编写的程序中用数组来实现队列。

28. 假设有两个队列，一次只允许从一个队列的队头移一个项到任何一个队列的队尾。请设计一个算法，颠倒其中一个队列中的两个相邻项。

29. 下面的表格表示的是存储在机器的存储器中的一棵树。树的每个节点有3个单元。第一个单元包含的是数据（字母），第二个单元包含的是指向该节点左孩子的指针，第三个单元包含的是指向该节点右孩子的指针。0x0值代表空指针。如果根指针的值是0x4F，那么请画出这棵树。

地 址	内 容
0x40	'G'
0x41	0x0
0x42	0x0
0x43	'X'

地 址	内 容
0x44	0x0
0x45	0x0
0x46	'J'
0x47	0x49
0x48	0x0
0x49	'M'
0x4A	0x0
0x4B	0x0
0x4C	'F'
0x4D	0x43
0x4E	0x40
0x4F	'W'

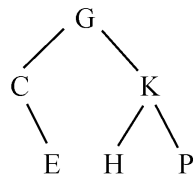
地 址	内 容
0x50	0x46
0x51	0x4C

30 . 下面的表格表示的是计算机的主存储器中一个单元块的内容。注意，一些单元包含的是字母表中的字母，并且每个这样的单元后面都跟有两个空单元。填充这些空单元，使得这个存储块表示下面的树。这里用字母后的第一单元作为指向节点左孩子的指针，用接下来的单元作为指向节点右孩子的指针。用0x0表示空指针。根指针中的值应该是什么？

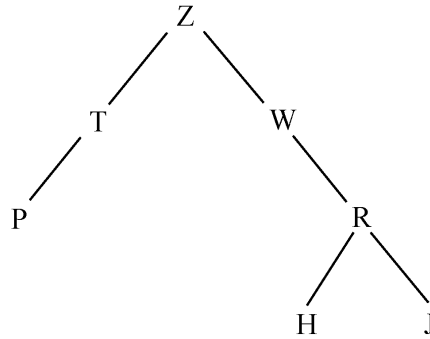
地 址	内 容
0x30	'C'
0x31	
0x32	
0x33	'H'
0x34	

地 址	内 容
0x35	
0x36	'K'
0x37	
0x38	
0x39	'E'
0x3A	
0x3B	
0x3C	'G'
0x3D	
0x3E	
0x3F	'P'
0x40	

地 址	内 容
0x41	



- 31．设计一个非递归算法来代替图8-21所示的递归算法。
- 32．设计一个非递归算法来代替图8-24所示的递归算法。用一个栈来控制可能需要的任何回溯。
- 33．应用图8-24中的打印树的递归算法。画图表示在打印X节点时该算法的嵌套激活（以及每次激活的当前位置）。
- 34．在保持根节点相同且不改变数据元素的物理位置的同时，改变第29题中树的指针，使得图8-24中的树的打印算法按字母顺序打印节点。
- 35．如果下面的二叉树不用指针存储，而是用8.3节中描述的连续存储单元块来存储，那么请画图表示该二叉树在存储器中是如何存储的。



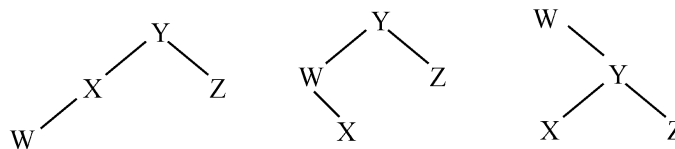
36．假设如8.3节中所描述的，用连续存储单元来表示包含值A、B、C、D、E、F和G的二叉树。请画出这棵树的图。

37．举一个可以把一个列表（概念结构）实现为一棵树（实际的底层结构）的例子。再举一个可以把一棵树（概念结构）实现为一个列表（实际的底层结构）的例子。

38．文中讨论的链式树结构包含指针，这就允许访问者沿着树从双亲下移到孩子。请描述一个指针系统，允许沿着树从孩子上移到双亲。兄弟之间的移动又如何？

39．描述一个适合于表示国际象棋游戏期间棋盘布局的数据结构。

40．通过图8-24中的算法，判断下列几棵树中哪棵树的节点会按照字母顺序打印出来？



41．修改图8-24中的函数，使得能以相反的顺序打印“列表”。

42．描述一个可以用于存储一个家族的家谱史的树结构。对该树会进行一些什么样的操作？如果该树是用链式结构来实现的，每个节点应该关联一些什么指针？假设该树就是按照你刚才描述的指针，并以链式结构实现的，请设计相应的过程来完成你上面确定的操作。利用你设计的存储系统，解释一下如何才能找到一个人的所有兄弟。

43．如果一棵树是按图8-20中的形式存储的，请设计一个过程来从这棵树中找到并删除一个给定的值。

44．在树的传统实现中，构造的每个节点都有一个单独的指向每个可能孩子的指针。这种指针的数量是一个设计决定，表示任何节点可以有的最大孩子数。如果一个节点的孩子数比指针数少，就将它的一些指针简单地设置为null。但是，这样的一个节点的孩子决不会比指针多。描述一下，如何在不限其节点所拥有的孩子数的情况下实现一棵树。

45．使用仿照8.5节介绍的C结构语句编写的伪代码，定义一个用户定义的数据类型，表示关于一家公司员工的数据（如名字、地址、工作职务、工资级别等）。

46．利用与图8-27中的Java的类语法相似的伪代码，拟定一个抽象数据类型的定义，表示一个名字列表。具体来说，用什么结构来包含这个列表？提供什么函数来操纵这个列表？（不需要包括函数的详细说明。）

47．利用与图8-27中的Java的类语法相似的伪代码，拟定一个抽象数据类型的定义，表示一个队列。然后再给出伪代码语句，说明如何创建这个类型的实例，以及如何在这些实例中插入和删除项。

48．a. 抽象数据类型和基本数据类型的区别是什么？

b．抽象数据类型与用户定义的数据类型的区别是什么？

49．确定用来表示地址簿的抽象数据类型中可能会出现的数据结构和过程。

50．确定用来表示视频游戏中一个简单航天器的抽象数据类型中可能出现的数据结构和过程。

51．修改图8-27和8.5节中的StackType接口，使该类定义的是一个队列，而不是栈。

52．类与传统的抽象数据类型相比，在哪个方面更通用？

*53. 利用8.7节最后描述的0xDR0S形式和0xER0S形式的指令，写一个完整的机器语言例程，向图8-12中实现的栈中压入一个项。这里假设栈指针在寄存器F中，而要压入的项在寄存器5中。

*54. 假设链表中的每个项都由一个存放数据的存储单元后跟一个指向下一项的指针构成。而且，假设一个要将位于存储地址0xA0处的新项插到位置0xB5处的项和位置0xC4处的项之间。利用附录C中描述

的语言，以及8.7节最后描述的附加操作码0xD和0xE，写一个机器语言例程来实现这个插入操作。

*55. 8.7节描述的0xDR0S形式的指令与0xDRXY形式的指令相比有什么优势？在8.7节的问题与练习4中描述的0xDRXS形式的指令与0xDR0S形式的指令相比有什么优势？

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的伦理、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1. 假设一位软件分析师设计了一种数据组织方式，能在特定的应用中有效地处理数据。那么该如何保护对这个数据结构的权益呢？数据结构是一种思想的表达（好比一首诗），因而可以通过版权来进行保护，还是数据结构也像算法一样存在同样的法律漏洞？用专利法呢？

2. 在何种程度上，错误的数据比没有数据更糟糕？

3. 在许多应用程序中，栈可以扩展到多大取决于可用的存储器容量有多大。如果可用空间被耗尽，那么所设计的软件要能产生一条像“栈溢出”这样的消息并终止。在大多数情况下，这种错误从不会发生，而且用户也从不会意识到这个错误。但是，如果这种错误发生

了，并且丢失了敏感数据，那么谁该对此负责？软件的开发者如何最大限度地减轻自己的责任？

4．在基于指针系统的数据结构中，删除一个项通常是通过改变指针来办到的，而不是通过擦除存储单元来办到的。这样一来，当链表的一项被删除后，这个删除的项实际上还留在存储器中，直到有其他数据需要它的存储空间。这种被删除的数据的存在会产生什么样的道德和安全方面的问题？

5．将数据和程序从一台计算机传送到另一台计算机很容易。因此，很容易将一台机器上存储的知识传输给许多机器。相比之下，一个人要把知识传输给另一个人有时要花很长的时间。例如，一个人要教会另一个人一种新语言是要花时间的。如果机器的能力开始挑战人的能力，那么这种在知识传输率上的反差将意味着什么？

6．利用指针可以将相关的数据在计算机的存储器中链接起来，其链接方式能让人联想到，信息在人脑中也是采用这种方式关联起来的。那么，这样一种在计算机的存储器中的链接与人脑中的链接有何相似之处？它们的不同点是什么？如果尝试着把计算机建造得更接近人类思维在伦理上是可取的吗？

7．计算机技术的普及已经产生了新的伦理问题，还是只是提供了一种新的环境，而在这样的环境之中，原来的那些伦理学理论是否还适用？

8. 假设计算机科学导论教材的作者想用一些程序的例子来说明文中的概念。不过，为了简明，许多例子必须使用实际的专业优质软件的简化版本。作者知道，这些例子会被不持怀疑态度的读者所使用，并最终用到一些重要的软件应用中去，而这些应用更适合采用较为健壮的技术。作者应该采用这些简化版的例子，即使因为简化降低了它们的演示价值，仍坚持认为所有的例子都是健壮的，还是应该拒绝使用这些例子，除非能同时获得清晰性和健壮性？

课外阅读

Carrano, F. M. and T. Henry. *Data Abstraction and Problem Solving with C++: Walls and Mirrors*, 7th ed. Boston, MA: Addison-Wesley, 2016.

Goodrich, M.T., R. Tamassia, and M.H. Goldwasser. *Data Structures and Algorithms in Python*. Hoboken, NJ: Wiley, 2013.

Gray, S. *Data Structures in Java: From Abstract Data Types to the Java Collections Framework*. Boston, MA: Addison-Wesley, 2007.

Lee, K.D. and S. Hubbard. *Data Structures and Algorithms with Python*. Cham, Switzerland: Springer, 2015.

Main, M. *Data Structures and Other Objects Using Java*, 4th ed. Boston, MA: Addison-Wesley 2011.

Main, M. and W. Savitch. *Data Structures and Other Objects Using C++*, 4th ed. Boston, MA: Addison-Wesley, 2010.

Prichard, J. and F. M. Carrano. *Data Abstraction and Problem Solving with Java: Walls and Mirrors*, 3rd ed. Boston, MA: Addison-Wesley, 2010.

Shaffer, C. A. *Practical Introduction to Data Structures and Algorithm Analysis*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2001.

Weiss, M. A. *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA: Addison-Wesley, 2013.

Weiss, M. A. *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA: Addison-Wesley, 2011.

Weiss, M. A. *Data Structures and Problem Solving Using Java*, 4th ed. Boston, MA: Addison-Wesley, 2011.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第9章 数据库系统

数据库是一个将大量数据转化成抽象工具的系统，允许用户以一种简便的方式搜索和提取相关的信息项。本章将讨论这个主题，还将讨论与数据挖掘相关领域和传统的文件结构。其中，数据挖掘寻求揭示大型数据收集中隐藏模式的技术，传统的文件结构提供了今天的数据库和数据挖掘系统的许多基本工具。

本章内容

9.1 数据库基础

9.2 关系模型

*9.3 面向对象数据库

*9.4 维护数据库的完整性

*9.5 传统的文件结构

9.6 数据挖掘

9.7 数据库技术的社会影响

持久理解和学习成果

人们使用计算机程序来处理信息从而获得深入理解和知识。

AP[®]CSP

学习成果 找到数字化地处理信息的模式和测试假设来获得深入理解和知识。

计算有助于探索和发现信息中的联系。 AP[®]CSP

学习成果 从数据中提取信息来发现和解释联系或趋势。

学习成果 确定大型数据集是如何影响使用计算过程来发现信息和知识的。

计算几乎在每个领域都能实现创新。 AP[®]CSP

学习成果 解释计算是如何影响其他领域创新的。

计算对人和社会有全球性影响——既有有利的又有有害的。

AP[®]CSP

学习成果 分析计算的有利影响和有害影响。

数据库系统应用抽象将大型数据集合体转换为有用的信息源。

学习成果 引用数据库系统的基本目标、功能和模型。

学习成果 描述数据库系统的组成，举出它们的使用示例。

学习成果 确定主要的数据库管理系统功能，描述它们在数据库系统中的作用。

学习成果 解释数据独立的概念及其在数据库系统中的重要性。

学习成果 使用说明性查询语言（SQL）从数据库中获取信息。

当今的技术已经能够存储极其大量的数据，但是这种大型数据集合是无用的，除非我们能够使用计算工具提取那些与手头任务相关的特定信息项。本章将研究数据库系统，弄清这些系统是怎样利用抽象从大型数据集合体中提取出有用信息来源的。作为相关主题，还要研究快速发展的数据挖掘领域，这个领域的目标是开发用于识别和探索数据集合模式的技术。此外，我们还将学习传统文件结构的原理，因为它支撑了当今的数据库和数据挖掘系统。

基本知识点

- 大型数据集的有效使用需要计算解决方案。

9.1 数据库基础

数据库（database）是一种多维的数据的所有作曲家的所有作品，以及改写其他作曲家作品的那些作曲家的所有作品。的集合，之所以说是多维的，是因为在这种集合中，通过两个数据项之间的内部链接，可以从不同的角度来获取信息。这与传统的文件系统（见9.5节）不同，传统的文件系统，有时称为**平面文件**（flat file），是一种一维

的存储系统，即它只从一个角度来展示它的信息。例如，一个包含作曲家及其作品信息的平面文件可能会提供一个按作曲家组织的作品清单，而数据库可能会呈现某一作曲家的所有作品、某一类音乐。

9.1.1 数据库系统的重要性

从历史发展角度来看，计算机广泛应用到信息管理领域时，每个应用都是作为独立系统来实现的，都有一套自己的数据。例如，工资用工资单文件处理，人事部门维护其自己的员工记录，库存通过库存文件来管理。这就意味着，许多只是一个部门需要的信息会在整个公司中重复出现，而许多不同但相互关联的项又存储在不同的系统中。在这种背景下，数据库系统应运而生，它作为一种信息集成的手段，通过特定的组织来存储和维护数据（见图9-1）。利用这样一个系统，可以根据相同的销售数据生成补进存货订单，创建市场趋势报告，给最有可能积极响应广告和产品公告的客户定向发布这种信息，并为销售人员生成奖金核查。

基本知识点

- 数据库能帮助有效组织和发现信息趋势。

如果能以有意义的方式访问信息，就可以用这种信息集成池提供的有价值的资源做出管理决策。因此，数据库的研究重点在于开发一些技术，把数据库中的信息提供给决策过程。在这方面已经取得了很大的进展。如今，数据库技术与数据挖掘技术相结合，已成为一种重

要的管理工具，使组织的管理层从涵盖组织及其环境的各个方面的大量数据中提取出新的知识和见解。

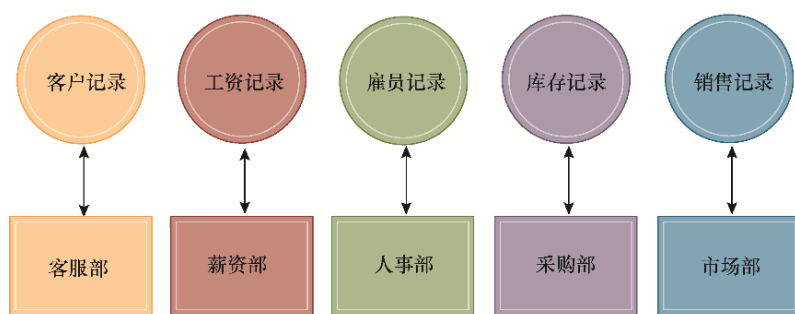
基本知识点

- 可以从翻译和转换数字表示信息中获得深入理解和知识。

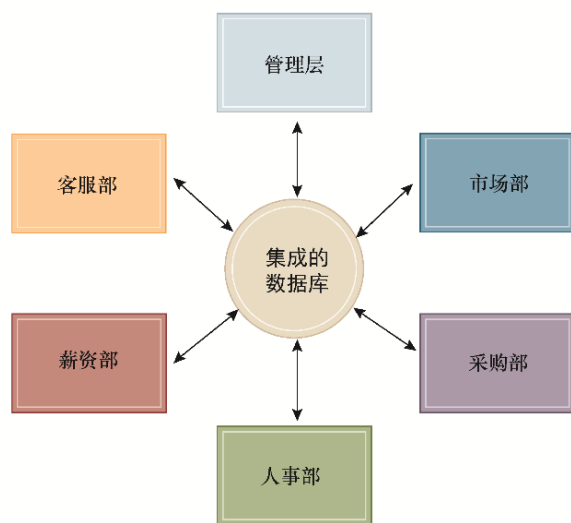
此外，数据库系统已经成为支撑万维网中许多流行网站的基础技术。谷歌、eBay和亚马逊等站点的基础主题是提供客户端与数据库之间的接口。为了响应客户端的请求，服务器询问数据库，以网页的形式组织查询结果，并把网页发送给客户端。这种Web接口已经使数据库技术承担了新角色，数据库不再是存储公司记录的一种手段，而是公司的产品。现代数据库不仅存储交易数据，还存储所有类型的文本多媒体内容和元数据。事实上，通过结合数据库技术和Web接口，因特网已经成为主要的全球信息源。

基本知识点

- 大型数据集包括交易、测量、文本、声音、图像和视频等数据。



(a) 面向文件的信息系统



(b) 面向数据库的信息系统

图9-1 文件结构与数据库结构的比较

元数据是关于数据的数据

术语元数据（metadata）指的是关于其他类型数据的信息。在计算机化目录普及之前的日子里，常见的元数据形式包括图书馆卡目录抽屉，用于通过作者、标题、主题或其他更深奥的标准快速定位书架上的书籍。计算机元数据可用于描述图像（关于照片在哪里拍摄的、用什么设备拍的、由谁拍的信息）或媒体文件（作者、专辑、歌曲标题等），还可用于提供网页语义队列。许多常见的数据文件格式现在

包括嵌入文件的元数据，如文档的原创作者信息或云资源的修订历史。

元数据可以通过为分析提供额外的结构和信息线索来进一步增强计算工具分析数据的能力。

当爱德华·斯诺登在2013年公开披露美国国家安全局多年来一直在大量收集美国公民的电话元数据时，该实践的支持者很快指出没有记录电话本身的内容。批评者列举了一些方式作为回应，其中在没有法庭命令的情况下元数据本身（被叫电话号码，通话时长等）就是对隐私的重大侵犯。

基本知识点

- 元数据是关于数据的数据。
- 元数据可以是关于图像、网页或其他复杂对象的描述性数据。
- 元数据可以通过提供数据各个方面的附加信息来增加那个数据或数据集的有效使用。

9.1.2 模式的作用

数据库技术的迅速发展有一个缺点，即潜在的敏感数据可能会被未经授权的人访问。一个在公司网站上下订单的人不应该有接触该公司财务数据的机会；类似地，公司福利部门的员工可能需要访问公司

的员工记录，但不应该有接触公司库存或销售记录的机会。因此，对数据库中信息的访问控制能力和共享它的能力同等重要。

为了让不同的用户访问数据库中的不同信息，通常数据库系统都依赖模式和子模式。**模式**（schema）是整个数据库结构的一个描述，数据库软件用它来维护数据库。**子模式**（subschema）只是与特定用户需求相关的那部分数据库的一个描述。例如，一个大学数据库的模式应当说明，每个学生记录包含的项除学生的学习成绩外，还有学生的当前住址和电话号码。此外，还要说明每个学生记录与学生指导教师的记录有关联。反过来，每个教师的记录将包含个人地址、工作经历等。基于这种模式，要维持一个链接系统，最终使得学生的信息与教师的工作经历相关联。

为了使大学的注册人员不能利用这种链接关系获取全体教员的所有信息，就必须限制注册人员只能访问数据库的子模式，这个子模式的全体教员记录描述不包括工作经历。在这种子模式下，注册会员可以找出哪个教职工是某个学生的导师，但得不到该教职工的其他信息。相反，薪资部的子模式应该提供每个教职工的工作经历，但不应该包括学生与导师之间的链接关系。因此，薪资部可以修改教职工的工资，但无法得到这个人指导的学生的名单。

虽然子模式是管理敏感信息使用权的有用工具，但实践中，非常大型的动态分布式数据库很难保护。重叠子模式之间的交互、系统的物理或网络安全缺漏以及不可靠的人类操作员都为受保护的数据库内容被滥用提供了机会。

基本知识点

- 隐私和安全问题出现在计算系统和制品的开发和使用中。
- 维护包含个人信息的大型数据集的隐私可能是具有挑战性的。

9.1.3 数据库管理系统

一个典型的数据库应用涉及多个软件层，我们将其分为两个主要的层：应用层和数据库管理层（见图9-2）。应用软件负责处理数据库与用户之间的通信，它可能相当复杂，用户通过网站访问数据库的应用就是其中一个例子。在这种情况下，整个应用层包括遍及因特网的客户和使用数据库满足客户请求的服务器。

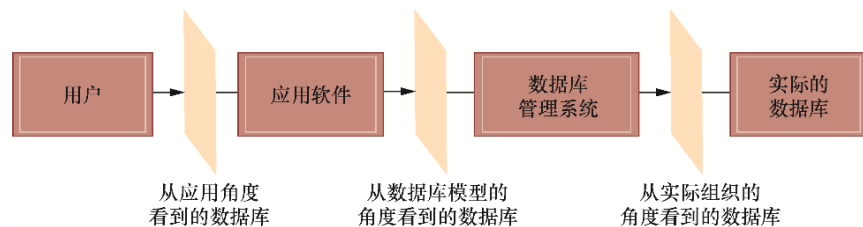


图9-2 一个数据库实现的概念层

分布式数据库

随着网络能力的提高，数据库系统已发展为包含称为分布式数据库的数据库，其数据驻留在不同的机器上。例如，一个跨国公司可以在本地站点存储和维护本地员工记录，但通过网络链接这些记录，创建一个分布式数据库。

分布式数据库可能包含碎片数据和/或重复数据。前面提到的员工记录的例子就属于第一种情况，即数据库的不同片段存储在不同的地方。在第二种情况中，不同的地方存储着同一数据库部分的几个副本，这种副本的存在可以减少信息的获取时间。两种情况都提出了较传统的集中式数据库所没有遇到过的问题：如何掩饰这种数据库的分布式特性，使它像一个连贯的系统那样工作；如何确保数据库更新时，数据库中的各个副本仍保持一致。所以，分布式数据库是当前的一个研究领域。

注意，应用软件并不直接操纵数据库，数据库实际是由**数据库管理系统**（Database Management System，DBMS）操纵的。一旦应用软件确定了用户正在请求的操作，它就会利用数据库管理系统作为抽象工具来得到结果。当请求是添加或删除数据时，实际更改数据库的是数据库管理系统。当请求是检索信息时，实际完成所需搜索的也是数据库管理系统。

应用软件与数据库管理系统分离有几个好处。一个好处就是允许构建和使用抽象工具，在软件设计中，我们已反复看到这个重要的简化的概念。如果数据库实际是如何存储数据的这样一个细节被数据库管理系统所屏蔽，那么应用软件的设计就可以大大简化了。例如，在使用精心设计的数据库管理系统时，应用软件就不必关心数据库到底是存储在单台机器上，还是像**分布式数据库**（distributed database）那样分散存储在网络中的许多机器上。相反，数据库管理系统自己就能

处理这些问题，因此应用软件可以直接访问数据库，而不用关心数据实际存储在哪里。

应用软件与数据库管理系统分离的第二个好处是，这样的结构提供了一种对数据库访问进行控制的手段。通过规定由数据库管理系统来执行对数据库的所有访问，数据库管理系统能实施由各种子模式施加的限制。具体来说，数据库管理系统可以对其内部需要采用整个数据库模式，但要求将每个用户使用的应用软件限制在由该用户子模式描述的范围內。

把用户界面与实际数据操作分离成两个不同的软件层的另一个原因是，可以获得**数据独立性**（data independence），即改变数据库组织本身而不改变应用软件的能力。例如，人事部需要在每个员工记录中增加一个字段，以说明相应员工是否选择参加了本公司新的健康保险计划。如果是由应用软件来直接处理数据库，那么这种数据格式的改变就需要修改与该数据库有关的所有应用程序。这样一来，原本由人事部提出的修改，可能就变成了，不但要修改薪资程序，还要修改用于公司业务通信服务的邮寄标签打印程序。

应用软件与数据库管理系统的分离消除了这种重新编程的需要。要实现单个用户所需的数据库修改，只需修改总体模式以及涉及这个变更的那些用户的子模式，所有其他用户的子模式都保持不变。因此，基于没有改变的子模式的应用软件也不需要修改。

9.1.4 数据库模型

我们已多次看到如何用抽象来隐藏内部复杂性。数据库管理系统又给出了一个例子。它们隐藏了数据库内部结构的复杂性，允许数据库的用户想象数据库中存储的信息是以更有用的格式组织的。具体来说，数据库管理系统包含许多例程，它们把按数据库的概念视图表达的命令，翻译为实际数据存储系统所要求的操作。这种数据库的概念视图称为**数据库模型**（database model）。

在接下来的几节中，我们将讨论关系数据库模型和面向对象数据库模型。在关系数据库模型中，数据库的概念视图是一组由行和列组成的表。例如，关于公司员工的信息可以看成这样的表格，即每行表示一名员工，各列分别表示姓名、地址、员工工号等。于是，数据库管理系统会包含一些例程，让应用软件从表的某一行中选取某些项，或者输出工资列中找到的值的范围，尽管信息并没有实际按行和列存储。

这些例程构成了应用软件用来访问数据库的抽象工具。更准确地说，通常应用软件用一种通用程序设计语言（在第6章讨论过）来编写。这些语言为算法的表达提供了基本元素，但缺少操纵数据库的指令。然而，用这些语言编写的程序可以把数据库管理系统提供的例程作为预先编好的子例程来使用，这实际上扩充了该语言的能力，从而支持了数据库的概念模型。

寻找更好的数据库模型的工作永无止境，其目标就是想要找到能够容易地把复杂的数据库系统概念化的模型，从而形成简明的表达信息请求的方式，产生高效的数据库管理系统。

9.1节问题与练习

1. 确定制造厂中对相同或者相似库存信息有不同用途的两个部门。然后，说明两个部门的子模式可能会有何不同。
2. 数据库模型的目标是什么？
3. 概述应用软件和数据库管理系统的作用。

9.2 关系模型

本节将更详细地讨论关系数据库模型，它描绘的是用矩形表存放的数据，这种表称为**关系**（relation），这类似于电子制表程序显示信息的格式。例如，在关系模型中，可以将一个公司员工的信息表示为如图9-3所示的关系。

EmplId	Name	Address	SSN
25X15	Joe E. Baker	33 Nowhere St.	111223333
34Y70	Cheryl H. Clark	563 Downtown Ave.	999009999
23Y34	G. Jerry Smith	1555 Circle Dr.	111005555
⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮

图9-3 包含员工信息的一个关系

关系中的一行称为一个**元组**（tuple）（有人读作“TOO-pul”，也有人读作“TU-pul”）。在图9-3所示的关系中，元组由某个特定员工的信息组成。因为每列描述的是对应的元组所表示的实体的一些特征或属性，所以关系中的列称为**属性**（attribute）。

9.2.1 关系设计中的问题

设计关系数据库的关键步骤是设计构成这个数据库的关系。尽管这个工作看上去很简单，但对于粗心的设计者来说，仍有不少难以捉摸的陷阱。

假定除了图9-3所示的关系中所包含的那些信息，我们还想包含员工的职位信息。我们可能想包含与每个员工相关的工作经历，包括如下一些属性：职位名称（秘书、办公室经理、楼层主管）、职位标识码（每种岗位特有的）、与每个职位相关的技能代码、该职位所在部门，以及员工任职的开始日期和结束日期的时间段（如果该职位代表的是员工的当前岗位，我们就用*号表示结束日期）等。

解决这个问题的一种方法是扩展图9-3所示的关系，在表格中加进图9-4所示的这些属性列。然而，仔细检查这个结果会发现一些问题。一个问题是，信息的冗余导致了效率低下。这个关系中不再是每个员工对应一个元组，而是员工的每次职位的工作指派对应一个元组。如果一个员工在公司里历任好几个职位，那么新关系中的几个元组一定包含该员工的相同信息（姓名、地址、工号及社会保险号）。例如，因为Baker和Smith担任过多个职位，所以他们的个人信息会重复。还有，如果某个特定的职位由多位员工担任过，那么与此职位相关的部门以及相应的技能代码也会在表示职位的指派的每个元组中重复。例如，因为楼层经理这个职位由多位员工担任过，所以这个职位的描述会重复。

对于这样一种扩展的关系，如果考虑从数据库中删除信息的话，会暴露另外一个更为严重的问题。例如，假设只有Joe E. Baker是唯一一个担任标识为D7这个职位的员工。如果他离开公司了，并被从图9-4表示的数据库中删除，那么有关D7职位的信息就会丢失，因为包含D7职位需要K2技能等级这个事实的元组，只有与Joe E. Baker有关的那个元组。

EmplId	Name	Address	SSN	JobId	JobTitle	SkillCode	Dept	StartDate	TermDate
25X15	Joe E. Baker	33 Nowhere St.	111223333	F5	Floor manager	FM3	Sales	9-1-2009	9-30-2010
25X15	Joe E. Baker	33 Nowhere St.	111223333	D7	Dept. head	K2	Sales	10-1-2010	*
34Y70	Cheryl H. Clark	563 Downtown Ave.	999009999	F5	Floor manager	FM3	Sales	10-1-2009	*
23Y34	G. Jerry Smith	1555 Circle Dr.	111005555	S25X	Secretary	T5	Personnel	3-1-1999	4-30-2010
23Y34	G. Jerry Smith	1555 Circle Dr.	111005555	S26Z	Secretary	T6	Accounting	5-1-2010	*
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.

图9-4 包含冗余的关系

你也许会认为，能做到只删除元组中一部分信息就可以解决这个问题，但是这又会引起新的麻烦。例如，与F5职位相关的信息是应该留存在一个部分元组中，还是应该留存在关系中的其他什么地方？此外，这种使用部分元组的想法正好说明了该数据库的设计还能够进一步完善。

所有这些问题产生的原因就在于我们在一个单一关系里融进了多个概念。图9-4中的扩展关系包含了员工的直接信息（姓名、工号、地址、社会保险号）、有关公司现有职位的信息（职位标识码、职位名称、部门、技能代码），以及关于员工和职位之间关系的信息（开始日期、结束日期）。基于以上的观察，我们可以通过用3个关系重新设

计这一系统来解决我们的问题，每个关系对应前面的一类信息。我们可以保留图9-3中的那个原始关系（现在我们称其为EMPLOYEE关系），再以称为JOB和ASSIGNMENT的两个新关系的形式插入附加信息，就产生了图9-5所示的数据库。

EMPLOYEE关系			
EmplId	Name	Address	SSN
25X15	Joe E. Baker	33 Nowhere St.	111223333
34Y70	Cheryl H. Clark	563 Downtown Ave.	999009999
23Y34	G. Jerry Smith	1555 Circle Dr.	111005555

JOB关系			
JobId	JobTitle	SkillCode	Dept
S25X	Secretary	T5	Personnel
S26Z	Secretary	T6	Accounting
F5	Floor manager	FM3	Sales
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.

ASSIGNMENT关系			
EmplId	JobId	StartDate	TermDate
23Y34	S25X	3-1-1999	4-30-2010
34Y70	F5	10-1-2009	*
23Y34	S26Z	5-1-2010	*
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.

图9-5 由三个关系组成的员工数据库

这样由这3个关系组成的数据库就包含了员工信息（在EMPLOYEE关系中）、职位信息（在JOB关系中）和工作经历信息（在ASSIGNMENT关系中），其他信息则隐含在不同关系信息的组合中。例如，如果知道一个员工的工号，就可以先用ASSIGNMENT关系找到该员工任职过的所有职位，再用JOB关系找到与这些职位有关的部门（见图9-6），这样就可以找到这个员工工作过的部门。通过这样一些

步骤，任何原先可以从单个大型关系中获得的信息，都可以在不出现前面提到的那些问题的情况下从3个较小的关系中获得。

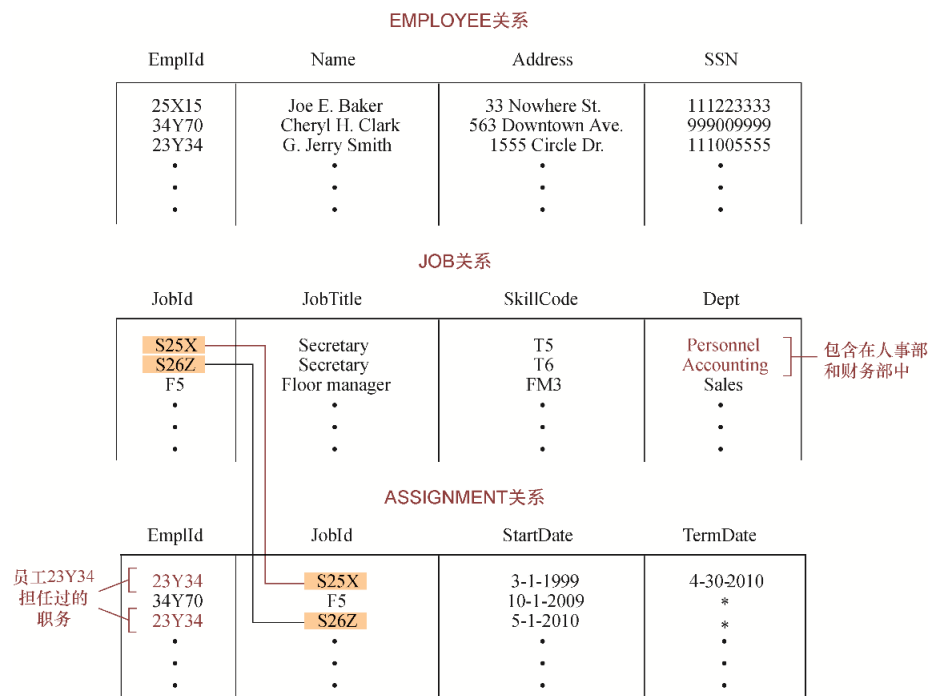


图9-6 查找员工23Y34工作过的部门

遗憾的是，把信息划分到不同的关系中并不总是像前面的例子那样顺利。例如，比较图9-7中的原始关系（具有EmpId、JobTitle和Dept属性）和提议分解成的两个关系。乍看起来，双关系系统与单关系系统好像包含相同的信息，但事实并非如此。例如，考虑要查找某员工工作过的部门这个问题。这在单关系系统中很容易，只需询问包含该员工的工号的那个元组，提取相应的部门即可。然而，在双关系系统中，所要的信息未必存在。我们可以找到该员工的职位名称和具有这个职位的一个部门，但这并不意味着该员工在这个部门工作，因为几个部门可以有相同名称的职位。

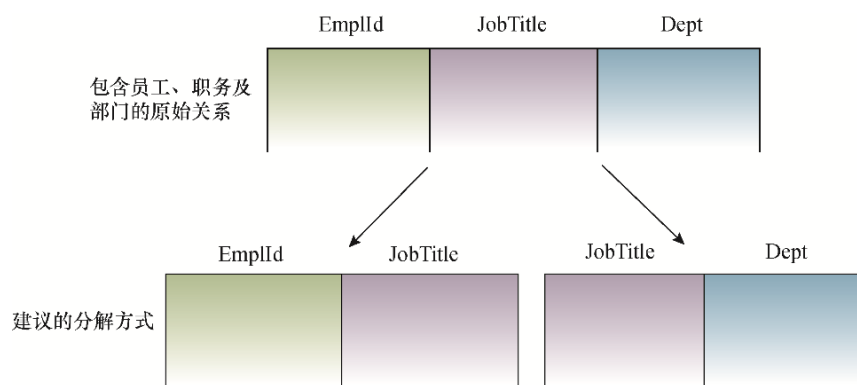


图9-7 一个关系和提议的分解

于是，我们看到，把一个关系分解成几个比较小的关系时，信息有时会丢失，有时不会丢失，后者称为**无损分解**（lossless decomposition或nonloss decomposition）。这种关系特征是重要的设计考量因素。目标就是找出会在数据库设计中引起问题的一些关系特征，并找到重新组织这些关系以消除这些问题特征的方法。

9.2.2 关系运算

现在我们对数据是如何按照关系模型来组织的有了基本的了解，是时候看看如何从由关系组成的数据库中提取信息了。我们先看看我们可能要对关系执行的一些操作。

有时我们需要从一个关系中选取某些元组。例如，要检索某个员工的信息，我们就必须从EMPLOYEE关系中选取包含适当身份属性值的元组，要得到某个部门的职位名称列表，我们就必须从JOB关系中选择具有该部门属性的元组。这样选取的结果是，从父关系中选取的元组构成了另一个关系。选择某一特定员工信息的结果是产生一个只

包含来自EMPLOYEE关系的一个元组的关系，而选择与某个部门相关的元组是产生一个可能包含来自JOB关系的几个元组的关系。

简而言之，在一个关系上想要执行的一种运算就是要选取具有某些特征的元组，并把这些选出的元组放到一个新关系中。为了表示这种运算，我们采用下面的语法：

NEW ← SELECT from EMPLOYEE where EmpId = '34Y70'
--

此语句的语义是：创建一个名为NEW的新关系，它包含从EMPLOYEE关系选得的其EmpId属性等于34Y70的那些元组（本例中应该只有一个元组）（见图9-8）。

SELECT运算是从一个关系中提取行，与此相反，PROJECT运算是提取列。例如，假定在查找某部门的职位名称时，我们已经从在JOB关系中用SELECT（选取）得到与该部门对应的元组，并把这些关系放到一个叫NEW1的新关系中。我们要查找的列表是这个新关系里的JobTitle列。PROJECT运算就是提取这个列（或者必要时是几个列），并把结果放到一个新关系中。这个运算表达为

NEW2 ← PROJECT JobTitle from NEW1

结果是创建了另一个新关系（名为NEW2），它包含从NEW1关系中JobTitle列得到的那些值所构成的一个列。

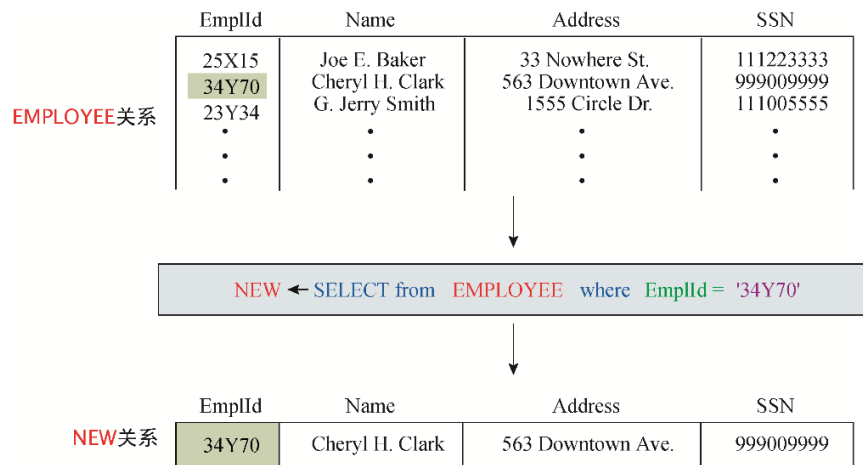


图9-8 SELECT运算

再举一个PROJECT运算的例子。语句

MAIL ← PROJECT Name, Address from EMPLOYEE

可以用来获取所有员工的姓名和地址的列表。这个列表是新创建的（两列）关系，名为MAIL（见图9-9）。

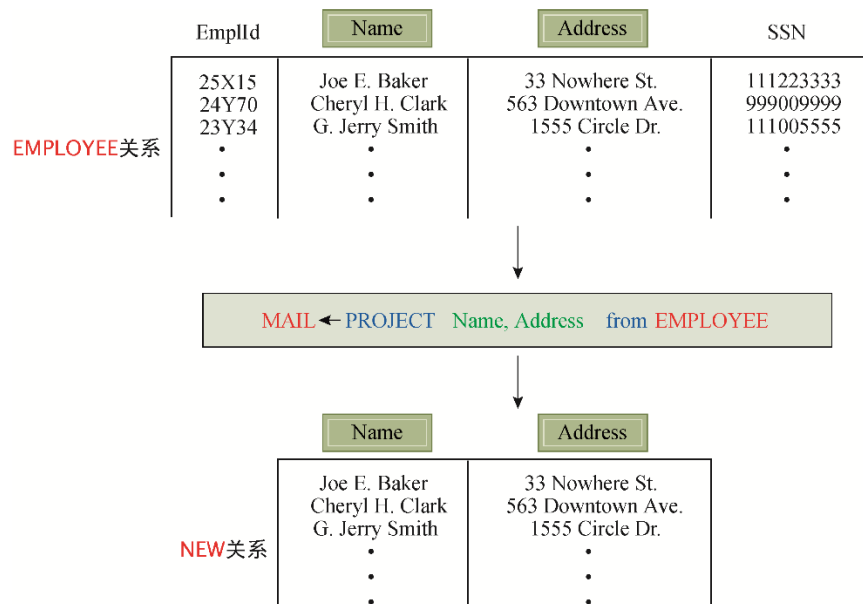


图9-9 PROJECT运算

另外一个用于连接关系数据库的操作是JOIN运算。它用来把原来不同的关系组合成一个关系。两个关系的连接（JOIN）产生一个新关系，而新关系的属性由原来两个关系的属性组成（见图9-10）。这些属性的名称与原关系中的名称一样，只是每个都加上了原关系作为前缀。（如果包含属性V和W的关系A与包含属性X、Y和Z的关系B做JOIN运算，那么结果就有名为A.V、A.W、B.X、B.Y和B.Z的5个属性。）这种命名约定保证了新关系的属性名称唯一，即使原先的几个关系中有相同的属性名也没关系。

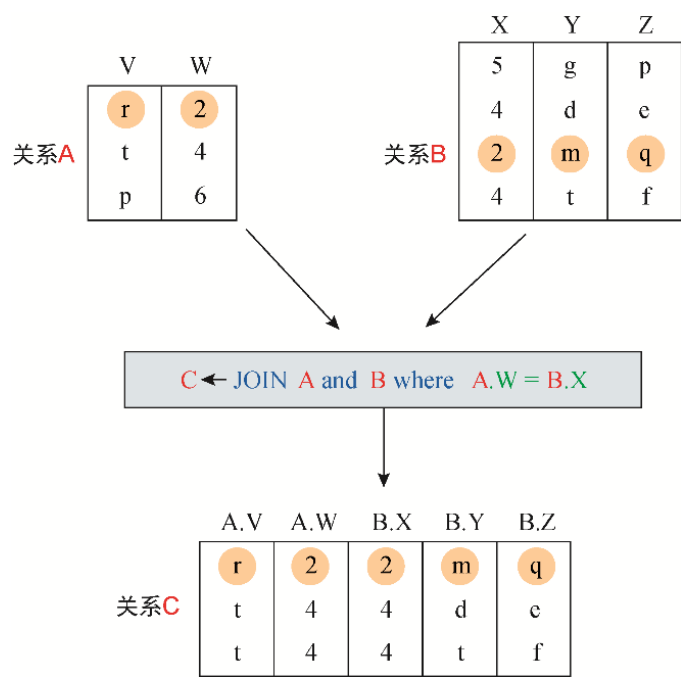


图9-10 JOIN运算

新关系的元组（行）由原来两个原始关系的元组拼接而成（见图9-10）。哪几个元组会实际连接成新关系的元组取决于构建JOIN的条件。一个条件是指定的属性要有相同值。事实上，图9-10表示的就是这种情况，它演示了执行下列语句的结果：

C ← JOIN A and B where A.W = B.X

在这个例子中，关系A的一个元组与关系B的一个元组拼接，其条件正是两个元组的属性W和X值相等。因此，关系A的元组(r, 2)与关系B的元组(2, m, q)的拼接出现在结果中，因为第一个元组中的W属性的值等于第二个元组中X属性的值。另一方面，在最后的系统中并没有关系A的元组(r, 2)与关系B的元组(5, g, p)拼接的结果，这是因为这些元组在属性W和X中没有共享共同的值。

再看一个例子，图9-11展示的是执行下列语句的结果：

C ← JOIN A and B where A.W < B.X

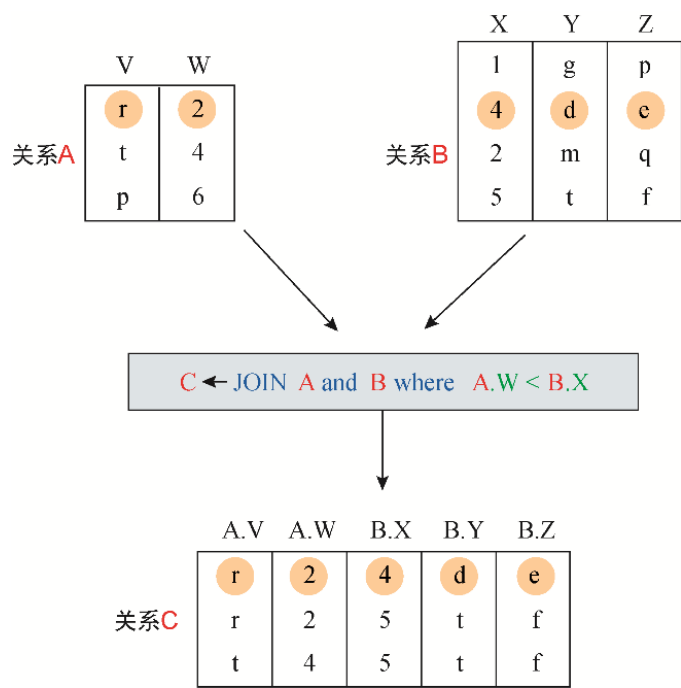


图9-11 JOIN运算的另一个例子

注意，结果中的元组正是其中关系A中的属性W小于关系B中的属性X的那些元组。

现在来看如何对图9-5中的数据库用JOIN运算来获取一个列表，其中包括所有员工的工号和每个员工的工作部门。我们首先观察到，所需信息分散在一个以上的关系中，因此，检索信息单靠SELECT和PROJECT是不行的。实际上，我们所需的工具是语句

```
NEW1 ← JOIN ASSIGNMENT and JOB  
      where ASSIGNMENT.JobId = JOB.JobId
```

如图9-12所示，它产生了一个关系NEW1。通过这个关系，我们的问题就能得到解决，即先用SELECT选取其中ASSIGNMENT.TermDate等于 '*'（ '*'表示“还在任职期”）的那些元组，然后在ASSIGNMENT.EmplId和JOB.Dept属性上进行PROJECT运算。简而言之，我们需要的信息可以通过执行以下序列从图9-5所示的数据库中获得：

```
NEW1 ← JOIN ASSIGNMENT and JOB  
      where ASSIGNMENT.JobId = JOB.JobId  
NEW2 ← SELECT from NEW1 where ASSIGNMENT.TermDate = '*'  
LIST ← PROJECT ASSIGNMENT.EmplId, JOB.Dept from NEW2
```

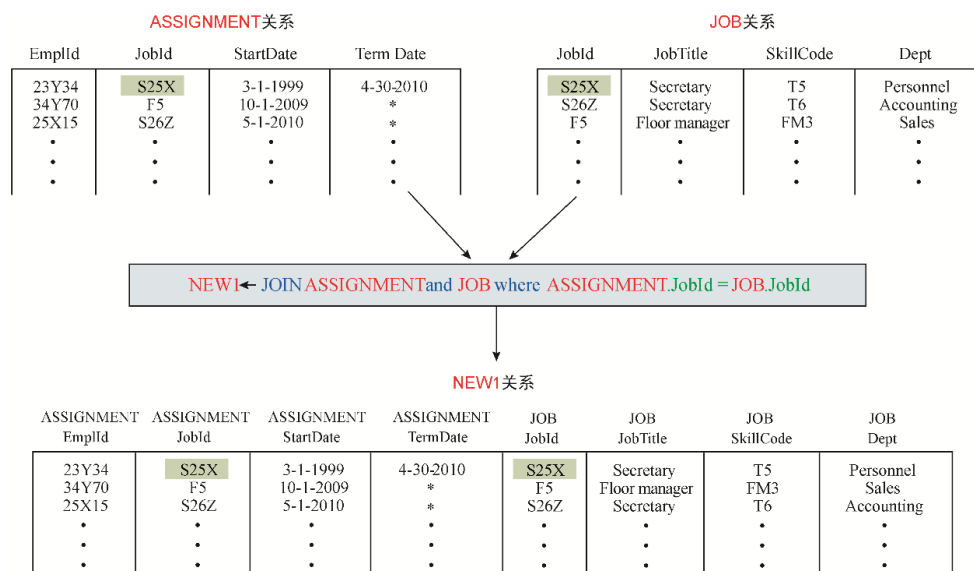


图9-12 JOIN运算的一个应用

9.2.3 SQL

介绍了基本的关系运算之后，接下来我们要考虑的是数据库系统的总体结构。我们知道，数据库实际上是存储在海量存储系统中的。为了让应用程序员免于关注这种系统的细节，人们设计了数据库管理系统，使应用软件能够按照数据库模型（如关系模型）来编写。数据库管理系统根据模型接受命令，并把这些命令转换为与实际存储结构有关的操作。这种转换由数据库管理系统提供的一组例程来处理，应用软件将它们用作抽象工具。因此，基于关系模型的数据库管理系统会包括完成SELECT、PROJECT和JOIN运算的例程，应用软件可以调用它们。通过这种方式，编写应用软件就好像数据库真的是存储在关系模型的简单表格中。

今天的关系数据库管理系统不一定提供执行原始形式的SELECT、PROJECT和JOIN运算的例程，而是提供一些组合了这些基本步骤的例程。一个例子就是结构化查询语言（structured query language，SQL），它构成了绝大多数关系数据库查询系统的主干。例如，SQL是因特网上许多数据库服务器采用的关系数据库系统MySQL（读作“My-S-Q-L”）的基础语言。

SQL流行的一个原因是美国国家标准学会已经将它标准化了，另一个原因是它起初是由IBM公司开发和推出的，因而备受媒体关注，就流行起来了。本节将解释如何用SQL表达关系数据库的查询。

虽然看起来用SQL表述的查询好像是以命令的形式来完成的，但本质上它是一种陈述性的语句。应当把一条SQL语句看作是对所需信息的一种描述，而不是一串要执行的操作。这样的意义在于，有了SQL，应用程序员就不必为开发处理关系的算法而花费精力，他们只要描述所需的信息就可以了。

作为SQL语句的第一个例子，我们现在来考虑上面提到的那个查询的例子。在那个例子中，为了获取所有员工的工号及其所在部门开发了一个3步的处理过程。在SQL中，整个查询用下面这样一条语句就可以表示：

```
SELECT EmplId, Dept
FROM Assignment, Job
WHERE Assignment.JobId = Job.JobId
AND Assignment.TermDate = '*'
```

这个示例表明，每条SQL查询语句可包含3条子句：一条select子句、一条from子句和一条where子句。粗略地说，其实这样一条语句就是请求如下运算的结果：from子句中列出的所有关系的JOIN运算，然后是选出满足where子句中条件的那些元组的SELECT运算，最后是P在select子句中列出的元组上进行PROJECT运算。（注意，因为SQL语句中的select子句确定的是PROJECT运算中所用的属性，所以术语有些颠倒。）让我们来看一些简单的例子。

下面的语句产生包含在Employee关系中的所有员工的姓名和地址的列表：

```
SELECT Name, Address  
FROM Employee
```

注意，这仅仅是一个PROJECT运算。

下面的语句产生Employee关系中Cheryl H. Clark相关的元组的所有信息：

```
SELECT EmplId, Name, Address, SSN  
FROM Employee  
WHERE Name = 'Cheryl H. Clark'
```

这其实是一个SELECT运算。

下面的语句产生包含在Employee关系中的Cheryl H. Clark的姓名和地址：

```
SELECT Name, Address  
FROM Employee  
WHERE Name = 'Cheryl H. Clark'
```

这是SELECT和PROJECT运算的组合。

下面的语句产生所有员工姓名及其开始工作日期的列表：

```
SELECT Employee.Name, Assignment.StartDate  
FROM Employee, Assignment  
WHERE Employee.EmplId = Assignment.EmplId
```

注意，这是如下几个操作的结果：首先对Employee和Assignment两个关系进行JOIN运算，然后对在where子句和select子句中指定的元组和属性进行SELECT运算和PROJECT运算。

最后需要指出的是，SQL语句除了可以执行查询，还可以定义关系的结构，创建关系，以及修改关系的内容。例如，下面是INSERT INTO、DELETE FROM和UPDATE语句的例子。

下面的语句在Employee关系中增加给定值的元组：

```
INSERT INTO Employee  
VALUES ( '42Z12', 'Sue A. Burt', '33 Fair St.', '444661111');
```

下面的语句从Employee关系中移除与G. Jerry Smith有关的元组：

```
DELETE FROM Employee  
WHERE Name = 'G. Jerry Smith'
```

而下面的语句修改Employee关系中 Joe E. Baker 有关的元组中的地址：

```
UPDATE Employee
SET Address = '1812 Napoleon Ave.'
WHERE Name = 'Joe E. Baker'
```

9.2节问题与练习

1. 根据图9-5中的EMPLOYEE关系、JOB关系和ASSIGNMENT关系给出的部分信息，回答下列问题：
 - a. 谁是有人事部经验的财务部秘书？
 - b. 谁是销售部的楼层经理？
 - c. G. Jerry Smith当前任职的职位是什么？
2. 根据图9-5所示的EMPLOYEE关系、JOB关系和ASSIGNMENT关系，写出一个关系运算序列，获取人事部内所有职位名称的列表。
3. 根据图9-5所示的EMPLOYEE关系、JOB关系和ASSIGNMENT关系，写出一个关系运算序列，获取员工姓名及其工作部门的列表。
4. 把你的第2题和第3题的答案转换成SQL语句。
5. 说明关系模型是如何提供数据独立性的。
6. 说明在一个关系数据库中不同的关系是如何联系在一起的。

*9.3 面向对象数据库

另一种数据库模型是基于面向对象范型的。这种方法构建的数据库称为面向对象数据库，由对象构成，对象之间通过相互链接来反映

它们之间的关系。例如，上一节的员工数据库的面向对象实现可以包含3个类（对象的类型）：Employee、Job和Assignment。

Employee类的对象可以包含EmplId、Name、Address及SSN这些属性，Job类的对象可以包含JobId、JobTitle、SkillCode和Dept这些属性，Assignment类的对象可以包含StartDate和TermDate这些属性。

面向对象数据库的概念表示如图9-13所示，其中两个不同对象之间的链接用连接两个相关对象的线来表示。如果我们关注Employee类型的对象就会发现，它链接到了Assignment类型的一组对象，表示某员工承担的不同任务。同样，Assignment类型的每个对象也都链接到了Job类型的一个对象，表示某个职位与某项工作指派相关。因此，顺着表示某员工的对象的链接查找，就能找到该员工的所有工作指派。类似地，顺着表示某职位的对象的链接查找，就能找到担任过该职位的所有员工。

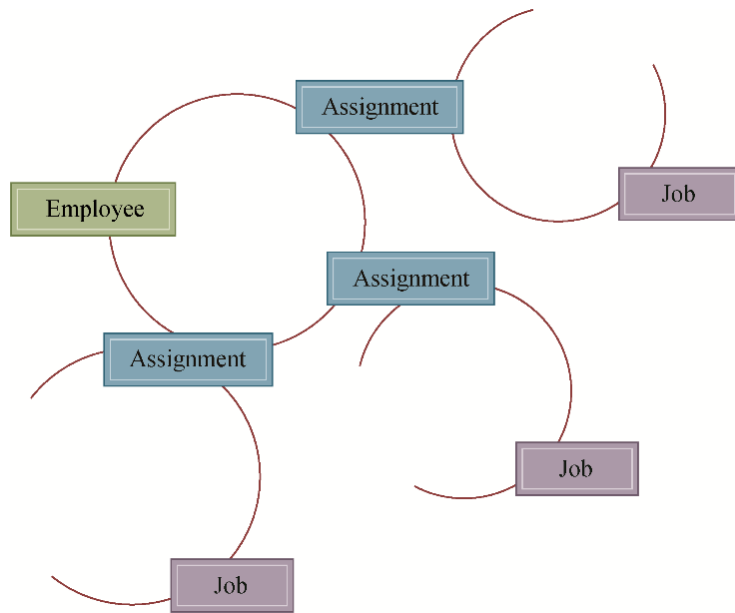


图9-13 面向对象数据库中两个对象之间的关联

面向对象数据库中两个对象之间的链接通常是由数据库管理系统来维护的，所以编写应用程序的程序员不需要关心如何实现这些链接的细节。相反，当向数据库中添加新对象时，应用软件只要指定这个新对象应该链接的其他对象就可以了。然后数据库管理系统会创建记录这些关联信息可能需要的链接系统。具体来说，数据库管理系统可能会以类似于链表的形式，把表示某给定员工的工作指派的对象链接起来。

面向对象数据库管理系统的另一个任务是，为其托管的对象提供永久的存储空间，这个要求看起来显而易见，但它与处理对象的常规方式有本质的区别。通常，在执行面向对象程序时，程序执行期间创建的对象在程序终止时就会被丢弃。从这个意义上看，可以认为对象是临时的。但是，在数据库中创建或添加的对象，在创建它们的程序

终止后必须保存。这样的对象称为**持久的**（persistent）。因此，创建持久对象与创建常规对象有很大不同。

面向对象数据库的支持者提出许多论据来说明为什么面向对象的数据库设计方法比关系方法好。其中一个论据是，面向对象方法使整个软件系统（应用软件、数据库管理系统和数据库本身）用同样的范型来设计。这与以往开发询问关系数据库的应用软件通常是采用命令式程序设计语言不同。在这样的任务中，命令范型与关系范型之间的冲突是不可避免的。就我们学习的水平而言，这种差别是很微妙的，但是多年以来这种差别正是许多软件错误的根源。即使以现有的知识水平，我们也能够理解，面向对象数据库与面向对象应用程序的结合产生了一种整个系统中的对象相互间进行通信的景象。另外，关系数据库与命令式应用程序结合，会让人联想到两个本质上不同的组织企图寻找共同接口的画面。

为了理解面向对象数据库相对于关系数据库的另一个优势，这里我们先来看一个在关系数据库中存储员工姓名的问题。如果把全名存储在一个关系的单个属性中，那么查询姓氏就会比较麻烦。然而，如果将全名分存于分开的单个属性中，如名字、中间名和姓氏，那么属性的数量也会成问题，因为不是所有的名字都符合特别结构——即使仅处理单一文化中的人名也是如此。在面向对象数据库中，这些问题可以隐藏在存储员工姓名的对象里。一个员工的姓名可以存储为一个智能对象，它能以不同的格式输出有关员工的姓名。因此，从这些对象

外部看，对于处理姓氏、全名、结婚前的姓氏或者绰号等，都是一样简单。每种视角所涉及的细节都会被封装在对象中。

这种封装不同数据格式技术特性的能力在其他情况下也很有优势。在关系数据库中，关系中的属性是数据库整体设计的一部分，所以与这些属性相关的数据类型将遍及整个数据库管理系统。（临时存储的变量必须声明为适当的类型，并且必须设计出处理不同类型数据的函数。）因此，扩展关系数据库来包含新类型（音频和视频）的属性可能会有问题。具体来说，贯穿数据库设计的各种函数都需要被扩展，才能包含这些新的数据类型。然而，在面向对象设计中，用来检索表示员工姓名的对象的函数，同样可以用来检索表示电影的对象，因为类型的差异可以被隐藏在所涉及的对象里。因此，面向对象方法显得与多媒体数据库的构建更协调，而这个特性已经被证明具有极大的优势。

面向对象范型对数据库设计而言还有一个好处是，不仅可以存储数据，还可以存储智能对象。也就是说，对象能够包含一些方法，用来描述它应如何响应有关它的内容和关系的消息。例如，图9-13中的Employee类的每个对象都能够包含用于报告和更新这个对象信息的方法，也有用于显示员工工作经历的方法，甚至还可能有用于修改员工职位指派的方法。同样，Job类的每个对象都可以有用于报告职位细节的方法，还可以有用于报告担任过此职位的那些员工的方法。这样一来，要检索员工的工作经历，就不需要再编写复杂的函数了，只需要询问相应的员工对象就能报告其工作经历。如果能够构建其组成

部分能智能地响应查询请求，那么它就能提供一些比较传统的关系数据库无法提供的令人兴奋的功能。

9.3节问题与练习

1. 在本节讨论的员工数据库中，Assignment类的对象的实例应该包含哪些方法？
 2. 什么是持久对象？
 3. 试确定在一个处理仓库库存的面向对象数据库中，要用到哪些类以及这些类的哪些内部特征。
 4. 试确定面向对象数据库优于关系数据库的地方。
-

*9.4 维护数据库的完整性

个人使用的廉价数据库管理系统是相对简单的系统。它们倾向于有一个单纯的目标——对用户屏蔽数据库实现的技术细节。用这类系统维护的数据库相对较小，一般包含的信息也不会非常重要，信息的丢失或破坏只会带来不便，而不至于造成灾难性的后果。当问题真的出现时，用户通常可以直接改正错误的项，或者从备份中重新加载数据库，并手工做些必要的修改，使备份能够及时更新。这样的处理过程也许不太方便，但是避免这种麻烦的代价往往比麻烦本身还大。无论如何，这种麻烦只局限在少数人身上，并且经济损失通常也有限。

然而，就大型的多用户商用数据库系统来说，利害关系就大得多。数据不准确或数据丢失的代价巨大，甚至会带来毁灭性的后果。

此外，随着数据库并发用户数量的不断增长，允许的数据库操作必须能够相应扩展。在这样的环境下，数据库管理系统的主要作用是通过防范问题来维护数据库的完整性，例如那些因为某种原因只完成了一部分的操作，或者那些因为疏忽的交互导致数据库中信息不准确的操作。本节要讨论的就是数据库管理系统的这种作用。

基本知识点

- 当数据集很大时，系统的可伸缩性是一个重要的考虑因素。

9.4.1 提交/回滚协议

单个事务（如从一个银行账户到另一银行账户的资金转账、预订航班的取消、大学课程中的学生注册）可能在数据库层次涉及多个步骤。例如，两个银行账户之间的资金转账，要求一个账户的余额减少，同时另一个账户的余额增加。在这些步骤之间，数据库中的信息可能会不一致。事实上，在第一个账户余额已减少，而另一个账户余额尚未增加的这段短暂的时间里，资金有可能会不见。类似地，当为一个乘客重新安排航班座位时，可能会有一瞬间这个乘客没有座位，或者有一瞬间乘客名单中的乘客数会比实际乘客数多一位。

在大型数据库遭受繁重事务负载的情况下，在任意一个瞬间，数据库极有可能处于某个事务的中间状态。一个执行事务的请求或者一个设备的故障，很可能会发生在数据库处于不一致状态的时候。

我们首先考虑故障的问题。数据库管理系统的目标是确保这种问题不会把数据库冻结在不一致的状态。为了做到这一点，经常通过非易失性存储系统（如磁盘）中维护一个包含每个事务活动记录的日志来实现。在允许事务更改数据库之前，先将要执行的更改记录到日志中。这样，这个日志就包含了每个事务操作的永久记录。

把一个事务的所有步骤记录在日志中的那个点，称为**提交点**（commit point）。正是在这个点上，数据库管理系统拥有在必要时靠自己重建事务所需的信息。在这个点上，数据库管理系统在一定意义上为事务提供了保证，即它负责确保事务活动在数据库中得到反映。在设备出现故障的情况下，数据库管理系统可以利用其日志中的信息，重建自上一次备份以来已经完成的（提交的）事务。

如果问题出现在事务达到其提交点之前，那么数据库管理系统可能会发现自己不能完成已经执行了一部分的事务。这种情况可以利用日志**回滚**（roll back）（也称为撤销）实际上已被事务执行的活动。例如，在出现故障的情况下，数据库管理系统可以通过回滚那些在故障发生时未完成（或称为未提交）的事务来恢复。

然而，事务的回滚并不局限于设备故障恢复的处理。它们常常也是数据库管理系统正常操作的一部分。例如，由于有试图访问特权信息的情况发生，事务可能会在完成全部步骤前被终止，或者可能是遇到死锁情况，即竞争资源的几个事务发觉自己一直在等待数据，而等待的数据正好被对方使用。在这些情况下，数据库管理系统能够利用日志回滚事务，从而避免未完成的事务使数据库出错。

为强调数据库管理系统设计的精妙特性，我们应该注意到，回滚过程中隐藏着许多微妙的问题。一个事务的回滚可能会影响到其他的事务已用过的数据库项。例如，正被回滚的事务可能已经更新了一个账户余额，而另一个事务可能已经在这个更新值的基础上进行了它的活动。这可能意味着这些另外的事务也得回滚，从而又会影响到别的事务，结果就产生了称为**级联回滚**（cascading rollback）的问题。

9.4.2 锁定

现在我们来考虑这样一个问题，即在执行某个事务的同时，数据库正因另一个事务而处于变迁状态，这种情况会造成两个事务之间的相互影响，从而产生错误的结果。例如，如果一个事务正处在从一个账户转账资金到另一个账户的中途，此时另一个事务试图计算银行存款总额，就会产生**错误决算问题**（incorrect summary problem）。这可能会造成存款总数不是太大，就是太小，具体取决于执行转账步骤的次序。另一个可能出现的问题称为**更新丢失问题**（lost update problem）。例如，有两个事务，每个事务都是完成从同一账户扣除金额的操作。如果一个事务读取账户的当前余额时，正值另一个事务刚读取过余额但尚未计算好新余额的时候，那么这两个事务都会在一个初始余额上进行扣除操作，这样一来，其中一个扣除的效果将不会反映在数据库中。

为了解决这样的问题，数据库管理系统可以强制以一次执行一个整体事务的方式来处理事务，即每个新事务要进行排队等待，直到它

前面的事务全部完成后才能得到执行。但是事务常常要花费很多时间来等待海量存储操作的完成。这里可以采用这样一种方式来解决这个问题，即通过事务之间的交叉执行，可以实现把一个事务等待的时间分配给另一个事务，用来处理它已经获得的数据。大多数大型数据库管理系统都有一个调度程序来协调事务间的分时，这非常像多道程序设计操作系统中协调进程的交叉处理（见3.3节）。

为了防止错误决算问题和更新丢失问题这一类异常情况的出现，这些调度程序都包含了一个**锁定协议**（locking protocol），该协议规定，数据库中当前正在被某个事务使用的项都要按协议加以标记。这些标记称为锁，已标记的项目称为被锁定。常见的锁有两种——**共享锁**（shared lock）和**排他型锁**（exclusive lock）。它们分别对应于访问事务所需数据的两种形式——共享访问和互斥访问。如果一个事务不打算改变数据项，那么它请求的就是共享访问，这就意味着允许其他事务查看该数据项。但是，如果一个事务打算改变数据项，那么它请求的就必须是互斥访问，这就意味着只有该事务才能访问该数据项。

在锁定协议中，每次事务请求访问数据项时，它还必须告诉数据库管理系统它需要的访问类型。如果事务请求的是对一个数据项的共享访问，那么不论这个数据项是否用共享锁锁定，这个访问都将得到批准，并且将该数据项用共享锁进行标记。但是，如果请求的数据项已经用排他型锁标记了，那么别的访问都将会被拒绝。如果事务请求的是对数据项的互斥访问，那么只有在这个数据项没有被锁定的情况下，请求才会被批准。通过这种方式，一个打算改变数据项的事务就

可以通过获得的互斥访问来防止别的事务对该数据项的干预。反之，如果几个事务都不打算改变数据项，那么它们就能共享访问这个数据项。当然，一旦事务完成对数据项的访问，就会通知数据库管理系统，从而解除相关的锁定。

当出现事务的访问请求被拒绝的情况时，可以用许多不同的算法进行处理。一种算法是只强制事务等待，直至所请求的项可用为止，但是这种方法容易造成死锁。因为两个事务请求对同样的两个数据项进行互斥访问的时候，如果每个事务都获得了对其中一个数据项的互斥访问权，并且又坚持等待另一个数据项，那么它们就会出现阻塞情况。为了避免这种死锁的发生，有些数据库管理系统会优先处理较旧的事务。也就是说，如果一个较旧的事务请求访问被较新的事务锁定的数据项，那么那个较新的事务就会被迫释放其所有的数据项，并回滚它的活动（依据日志）。于是，较旧的事务就获得了对所需数据项的访问权限，而较新的事务只得重新开始。如果这个较新的事务被反复抢占，那么随着过程的进展它也会变旧，并最终成为一个具有高优先级的较旧的事务。这个协议，称为**受伤等待协议**（wound-wait protocol）（旧的事务将新的事务挂起，而新的事务等待变成旧的事务），能保证每个事务最终都能完成它的任务。

9.4节问题与练习

1. 到达提交点的事务与没到达的事务之间有什么区别？
2. 数据库管理系统是怎样防止大量的级联回滚的？

3. 假定一个账户的初始余额是400美元。有两个事务，一个事务从这个账户中支取100美元，另一个事务从同一账户中支取200美元。请说明，这两个不加控制的相互穿插的事务怎样才能使账户的最终余额为100美元、200美元和300美元。

4. a. 概述事务对数据库中的数据项请求共享访问的可能结果。

b. 概述事务对数据库中的数据项请求互斥访问的可能结果。

5. 试描述会导致在数据库系统上执行操作的事务间出现死锁的一系列事件。

6. 请说明如何打破第5题答案中的死锁情况。你的解决方案需要使用数据库管理系统的日志吗？请解释你的答案。

*9.5 传统的文件结构

在本节中，我们抛开多维数据库系统的研究来讨论传统的文件结构。这些结构代表着数据存储和检索系统的历史开端，现在的数据库技术就是由此发展而来的。为这些结构开发的许多技术（如索引技术和散列技术）是构建今天大规模复杂数据库的重要工具。

9.5.1 顺序文件

顺序文件（sequential file）是一种从头到尾以顺序方式访问的文件，好像文件中的信息都排成一行。这种文件的例子有音频文件、视频文件、包含程序的文件和包含文本文档的文件等。事实上，大多数

典型个人计算机用户创建的文件都是顺序文件。例如，在保存电子表格时，它的信息就会被编码和存储为顺序文件，电子表格应用软件能够重新构建电子表格。

文本文件是顺序文件，它的每个逻辑记录是用ASCII码或Unicode码编码的单个符号。文本文件经常用作构建较复杂的顺序文件（如员工记录文件）的基本工具。为此，只需建立一个统一格式，把每个员工的信息表示为一串文本，然后按照格式对这些信息进行编码，接下来就把这些员工记录一个接一个地记录成一个文本串。例如，可以构建这样的一个简单的员工文件，即每个员工记录为可以输入31个字符的字符串，其中25个字符作为一个字段，用来表示员工的姓名（填充足够多的空格补全25字符的字段），随后6个字符作为一个字段，用来表示员工的工号。最终的文件将会是一个很长的编码过的字符串，其中每31个字符组成的块代表一个员工的信息（见图9-14）。我们可以根据由31个字符的块组成的逻辑记录来检索文件中的信息。在每个这样的块中，各个字段是根据构成块的统一格式来识别的。

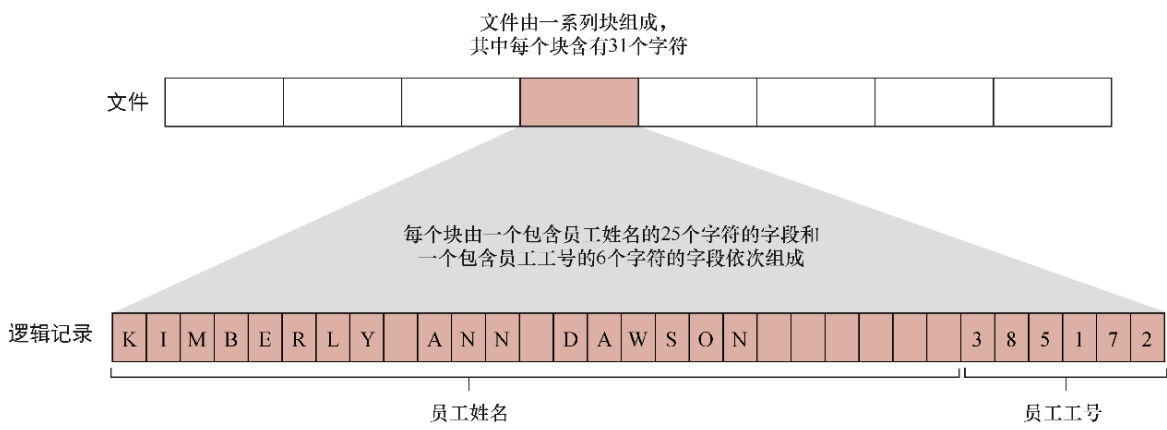


图9-14 以文本文件实现的一个简单的员工文件的结构

顺序文件中的数据必须以能保持文件顺序性的方式记录在海量存储器里。如果海量存储系统本身是有顺序的（就像磁带或CD），就可以直接做到这一点。我们只需根据存储介质的顺序特性，将文件记录到存储介质上。然后，处理文件的任务就是以找到它们的顺序读取和处理文件内容。播放音频CD就是这么一个过程，其中音乐被存储为顺序文件，沿着连续的螺旋轨道存储在一个接一个的扇区上。

然而，在用磁盘存储的情况下，文件将分散在不同的扇区上，因而会以各种顺序来检索。为了保持正确的顺序，大多数操作系统（更准确地说是文件管理程序）都会维护一个存有文件的扇区的列表。这个列表作为磁盘目录系统的一部分与文件记录在同一磁盘上。即使文件实际分布在磁盘的不同部分，利用这个列表，操作系统也能以正确的顺序检索扇区，就好像文件真的是按顺序存储的一样。

处理顺序文件的一个固有问题就是需要检测何时到达文件的末尾。通常我们把顺序文件的末尾称为**文件结束**（end-of-file，EOF）。有许多种标识EOF的方法：一种是在文件的末尾放置一个特殊的记录——**哨兵**（sentinel）；另一种是利用操作系统的目录系统中的信息来标识一个文件的EOF。也就是说，由于操作系统知道哪些扇区包含有此文件，因此它也就知道这个文件在什么地方终止。一个小公司的工资单处理就是使用顺序文件的一个典型例子。这里我们想象一个由一系列逻辑记录组成的顺序文件，每条记录都包含一个员工的薪水信息（如姓名、员工工号、工资等级等），依据这些信息就能定期打印出支票，每检索一个员工的记录，就能计算出该员工的工资，然后再打

出相对应的支票。处理这样一个顺序文件的活动，可以用下面这样的语句来完成：

```
while (未到达EOF):  
    从文件中检索下一条记录并处理这条记录
```

当顺序文件中的逻辑记录用键字段来标识时，文件通常就可以这样安排，即按照由键（可能是字母键，也可能是数字键）确定的顺序来安排文件中的记录。这样一种安排简化了文件信息的处理工作。例如，假设处理工资单要求依据考勤单的信息更新每个员工的记录。如果包含考勤单记录的文件和包含员工记录的文件都根据同样的键按照同样的次序存储，就能通过顺序地访问两个文件来进行更新处理，即用从一个文件检索到的考勤单来更新另一个文件中的对应记录。这是一个重大改进，因为如果文件不按照相应次序存储，就必须反复地搜索，而上述方法就克服了这个缺点。因此，更新典型的顺序文件通常需要多个步骤来处理。首先，将新信息（如考勤单中的信息）记录在一个称为事务文件的顺序文件中，对这个事务文件按照要被更新的文件（称为主文件）的次序进行排序，然后，通过从两个文件中顺序地读取记录来对主文件记录进行更新。

这个更新过程的一个微小变化是将两个顺序文件归并成一个包含原来两个文件记录的新文件。假定两个输入文件中的记录是按照公共的键字段升序排列的，并假定归并产生的输出文件也是按键的升序来排列的。图9-15总结了典型的归并算法。其基本思想是顺序地扫描两个输入文件，从而构建出输出文件（见图9-16）。

```
def MergeFiles(InputFileA, InputFileB, OutputFile)
    if (两个输入文件都在EOF):
        停止, OutputFile为空.
    if (InputFileA不在EOF):
        声明它的第一个记录为它的当前记录
    if (InputFileB不在EOF):
        声明它的第一个记录为它的当前记录
    while (两个输入文件都不在EOF):
        将键字段值 “较小的” 当前记录放在OutputFile中
        if (该当前记录是其对应输入文件的最后一个记录):
            声明该输入文件在EOF
        else:
            声明该输入文件中的下一个记录是该文件的当前记录
    从不在EOF的输入文件的当前记录开始复制
    其余记录到OutputFile
```

图9-15 归并两个顺序文件的函数

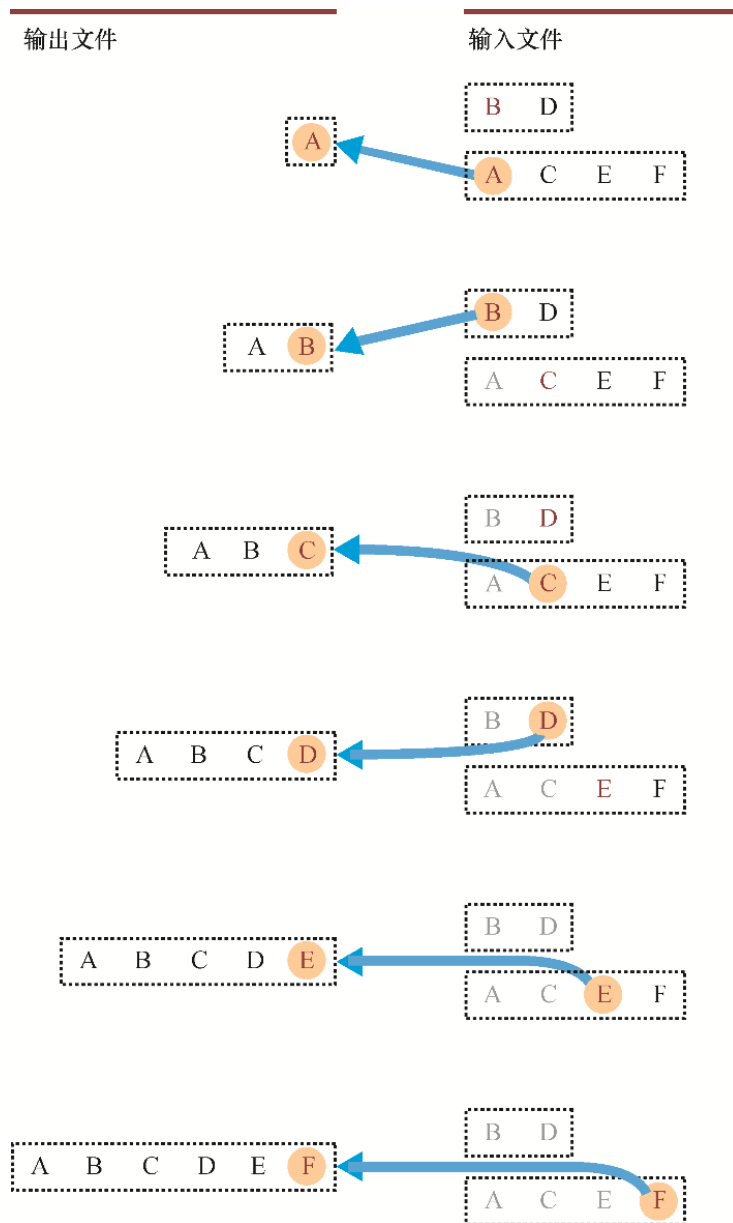


图9-16 归并算法的应用（字母用于代表整条记录，具体的字母表示记录的键字段的值）

9.5.2 索引文件

顺序文件是存储那种处理的次序就是其文件存储次序的数据的理想选择。然而，当文件中的记录必须以一种不可预测的次序进行检索

时，顺序文件的效率就不高了。在这种情况下，需要一种方法来快速确定所需逻辑记录的位置。一种流行的解决方案是对文件使用索引，这与书本里的索引用来定位主题在书中位置的方式非常一致。这种文件系统称为**索引文件**（indexed file）。

文件的索引包含存储在该文件中的键的列表和指示包含每个键的记录存储位置的项。因此，要找到某个记录，需要先在索引中找到标识键，再检索存储在与那个键相关的位置处的信息块。

文件的索引通常作为一个单独的文件与被索引的文件存储在同一个海量存储设备里。在文件处理开始之前，通常要先将索引传送到主存储器中，以便在需要访问文件中的记录时，能很容易地访问到（见图9-17）。

索引文件的一个典型的例子出现在维护员工记录的上下文中。想检索单个员工记录时，可以使用索引来避免冗长的搜索。具体来说，如果员工记录的文件是用员工工号进行索引的，那么只要知道员工的工号就能很快检索到该员工的记录。另一个例子出现在音频CD上，使用索引能相对快速地访问到单个录音。

多年以来，在基本索引概念的基础上已经使用了许多不同的索引技术。构建索引的一种方式是运用层次化的方式，以便索引呈现出分层结构或树结构。一个突出的例子就是大多数操作系统为组织文件存储所采用的分层目录系统。在这种情况下，目录（或文件夹）起的是

索引的作用，而每个索引又包含了指向其子索引的链接。从这个角度来看，整个文件系统只是一个大型的索引文件。

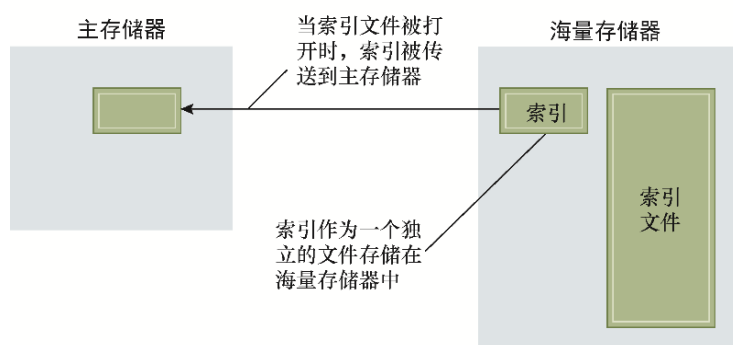


图9-17 打开索引文件

9.5.3 散列文件

尽管索引技术为访问数据存储结构中的数据项提供了一种相对快速访问机制，但这么做维护索引的开销也很大。**散列**（hashing，也称哈希）是一种能提供类似的访问效果，但没有这种开销的技术。与索引系统的情况一样，散列允许通过键值来定位记录。但是，散列并不是在索引中查找键，而是直接通过键确定记录的位置。

散列系统可以概括如下：数据存储空间被分成若干称为**桶**（bucket）的区，每个桶能放几条记录。记录根据将键的值转换为桶号的算法分散存储在这些桶中。这里，将键的值转换为桶号的算法称为**散列函数**（hash function）。每条记录就存储在通过这样处理标识的桶中。因此，要检索一条已经置于这种存储结构中的记录，首先要对

该记录的标识键应用散列函数，以确定相应的桶，然后检索桶中的内容，最后从检索到的数据中搜索所需的记录。

散列不仅能用作从海量存储器中检索数据的手段，也能用作从存储在主存储器中的大数据块中检索数据项的手段。当将散列用于海量存储器中的存储结构时，得到的结果称为**散列文件**（hash file）。当将散列用于主存储器中的存储结构时，得到的结果通常称为**散列表**（hash table）。

通过散列认证

散列不只是用来作为构建高效数据存储系统的一种手段。例如，散列还可以用作认证因特网上传送消息的一种手段。散列的基本思想是：以一种秘密方式对消息进行散列运算，然后将得到的值与消息一起传送。为了认证消息，接收方对收到的消息进行散列处理（以同样的秘密方式），并确认得到的值与原始值一致。（这里假定经过散列处理后得到相同的值，而消息发生改变的可能性非常小。）如果得到的值与原始值不一致，就表明该消息已被破坏。那些对此感兴趣的人可能希望在因特网上搜索有关MD5的信息，MD5是一种在认证应用领域有着广泛使用的散列函数。

将错误检测技术看作散列在认证领域的一种应用，已经是一目了然的事了。例如，校验位的使用本质上就是一个散列系统，在此系统中，位模式只散列为0或1，然后将这个值与原始模式一起传送。如果最终收到的位模式不能散列成同样的值，就认为这个位模式已被破坏。

现在我们把散列技术运用在典型的员工文件上，文件中的每条记录包含的是一家公司发一位员工的信息。首先，在海量存储器中创建几个可用的区域，用来扮演桶的角色。至于如何设计决定桶的数量和每个桶的大小，稍后再讨论。现在，我们假定已创建了41个桶，桶号从0一直到40。（我们选择41个桶，而不是偶数40个桶，原因稍后再解释。）

现在我们假设用员工工号作为标识员工记录的键。然后，下一步工作就是开发一个散列函数，把这些键转换成桶号。虽然员工工号可能是25X3Z或J2X35这样的格式，不是数字型的，但它们是以位模式存储的，我们能够将这些位模式解释成数字。利用这个数字解释，我们就可以用任何一个键除以可用桶的桶号，然后记下余数。在这个例子中，余数将是0～40范围内的一个整数。因此，我们就可以用每次做除法得到的余数来确定41个桶中的一个（见图9-18）。

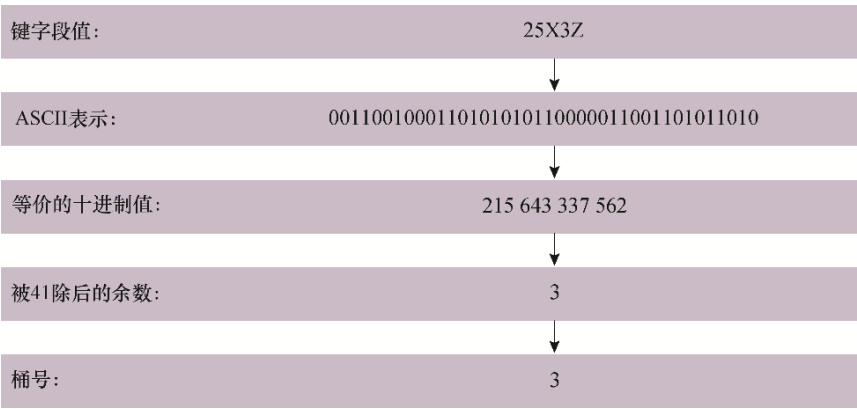


图9-18 将键字段值25X3Z散列到41个桶中的一个

以此作为我们的散列函数，接下来分别考虑每条记录，继续构建文件。通过使用散列函数对其键除以41得到一个桶号，然后把该记录

存储在这个桶中（见图9-19）。后面，我们需要检索一条记录时，只需将这个散列函数应用到该记录的键上就能识别相应的桶，然后在那个桶里搜索要检索的记录。

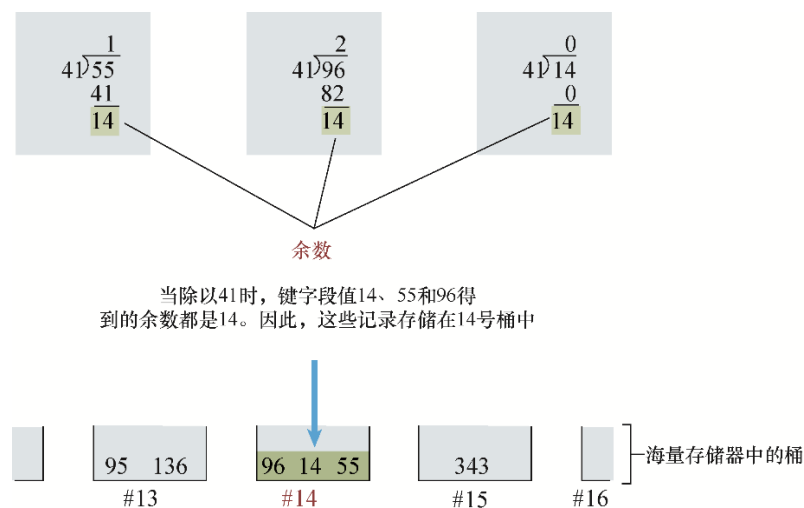


图9-19 散列系统的雏形

现在，重新考虑一下把存储区分成41个桶的问题。首先注意，要想得到一个有效的散列系统，应该将要存储的记录均匀地分布在这些桶中。如果数量不成比例的键恰巧散列到同一个桶里——这种现象被称为**聚类**（clustering）——那么一个桶里就会存储数量不成比例的记录。结果是，从这个桶里检索一条记录会花费更多的搜索时间，这也就失去了散列的好处。

现在再来看，如果我们选择把存储区域分成40个桶，而不是41个桶，那么我们的散列函数就会用键值除以40，而不是41。但是，如果被除数和除数有一个公共因子，那么这个公共因子也会出现在余数中。具体来说，如果存储在散列文件中的数据项的键碰巧都是5的倍数（也是40的除数），那么当用40来除时，5这个因子就会出现在余数

中，并且数据项就会聚类在与余数0、5、10、15、20、25、30和35相对应的那些桶里。类似的情况还会出现在键是2、4、8、10和20的倍数的情况中，因为它们也都是40的因子。因此，我们选择把存储区域划分为41个桶，因为41是素数，选择它可以消除公共因子，从而降低聚类的可能性。

但是，聚类的可能性绝对不能完全消除，即使是用精心设计的散列函数，在文件构建过程的早期也是非常有可能存在两个键经过散列后得到同一个值的情况。这种现象称为碰撞。为了理解其中的原因，考虑下面的情况。

假设我们已经找到一个在41个桶中任意分配记录的散列函数，这时候的存储系统是空的，并且打算一次插入一条新记录。当我们插入第一条记录时，它会被放进一个空桶里。然而，当插入第二条记录时，41个桶中还有40个桶是空的，这样一来，第二条记录被放进空桶的概率只有40/41。假设第二条记录被放进了一个空桶，那么当放第三条记录时就只能找到39个空桶，所以被放进空桶的概率是39/41。继续这个过程就可以发现，如果前7条记录都被放进了空桶，那么第八条记录被放进余下空桶的概率就只有34/41。

基于上述分析，我们就能算出所有前8条记录都被放进空桶的概率，即每条记录被放入空桶概率的乘积，这里假设前面的记录都已被放入空桶。那么，这个概率为

$$(41/41)(40/41)(39/41)(38/41)\dots(34/41) = 0.482$$

重点是这个结果小于一半。也就是说，当在41个桶里分配记录时，很可能在存储到第八条记录时碰撞就已经发生了。

发生碰撞的高概率表明，不管如何精心选择散列函数，设计任何散列系统时都必须考虑聚类。具体来说，一个桶有可能会装满或者溢出。这个问题的一种解决方法是允许扩展桶的大小，另一种解决方法是允许桶溢出到一个专门为解决这种问题而保留的溢出区。无论如何，聚类和溢出情况的出现都会使散列文件的性能明显降低。

研究表明，作为一般规律，只要记录的数量与文件中总的记录容量的比率——这个比率称为**负载因子**（load factor）——保持在50%以下，那么散列文件就会表现良好。但是，如果负载因子开始攀升超过75%，那么系统的性能通常就会降低（出现严重的聚类现象，造成某些桶装满甚至溢出）。由于这个原因，如果散列存储系统的负载因子接近75%这个值，那么这个系统通常会以一个更大的容量重建。最后我们得出结论：通过实现一个散列系统来获得记录检索的高效率是要付出一定代价的。

9.5节问题与练习

1. 假设一个输入文件包含键字段值等于B和E的记录，而另一个输入文件包含A、C、D和F，试遵循图9-15所示的归并算法归并这两个文件。

2. 归并算法是一种称为归并排序的流行排序算法的核心。你能说明一下这种算法吗？（**提示**：任何非空文件都可以看作单项文件的

集合。)

3. 文件是顺序的，这是物理属性还是概念属性？

4. 从索引文件中检索记录需要哪些步骤？

5. 试说明：散列函数选择不当是如何导致散列存储系统的性能不比顺序文件优越的。

6. 假设一个散列存储系统是用文中提到的除法散列函数构建的，但这里只用6个桶。对下列每个键值，确定具有该键的记录应该放入哪个桶。会出现什么问题？为什么？

a. 24

b. 30

c. 3

d. 18

e. 15

f. 21

g. 9

h. 39

i. 27

j. 0

7. 要想出现一组中有两个人的生日在同一天的可能性，至少需要聚集多少人？这个问题与本节的内容有何关系？

9.6 数据挖掘

一个与数据库技术紧密相关的迅速发展的学科就是数据挖掘，它包括用于发现数据集模式的技术。数据挖掘已经成为市场营销、库存管理、质量控制、借贷风险管理、欺诈检测和投资分析等众多领域的重要工具。数据挖掘技术甚至可以应用在那些似乎不大可能会用到的场合，如用于鉴定以DNA分子编码的特定基因的功能以及描绘生物体的特性。

基本知识点

- 模式会出现在使用计算工具转换数据的时候。
- 数据挖掘已经使医学、商业和科学领域的创新成为可能。

数据挖掘活动与传统的数据库询问不同，原因在于数据挖掘所做的工作是设法确定以前未知的模式，而传统数据库需要做的只是检索已经存储好的事实。此外，数据挖掘操作的是静态的数据集合，称为**数据仓库**（data warehouse），而不是经常要更新的“联机”运行的数据库。这些仓库往往是数据库或数据库集合的“快照”。因为在静态系统中寻找模式要比在动态系统中简单，所以用它们来替代实际运行的数据库。

我们还应该注意的是，数据挖掘的主题不单局限于计算领域，还涉及统计学领域。事实上，很多人认为，由于数据挖掘源自试图对大量不同的数据集合进行统计分析，因而它是统计学的一种应用，而非计算机科学的一个领域。

数据挖掘的两种常见形式是**类描述**（class description）和**类辨别**（class discrimination）。类描述用来找出描绘一组给定数据项的特性，而类辨别用来找出区分两组数据项的特性。例如，类描述技术可以用来找出购买经济型轿车的人的特性，而类辨别技术可以用来找出区分买二手车的顾客与买新车的顾客的特性。

生物信息学

数据库技术和数据挖掘技术的进步，正在扩展生物学家在涉及模式识别和有机化合物分类研究领域中的可用工具库。结果产生了生物

学的一个新领域——生物信息学。生物信息学起源于解码DNA的研究工作，它现在包括了蛋白质分类和理解蛋白质相互作用序列（称为生物化学路径）这样的一些研究。尽管通常认为生物信息学是生物学的一部分，但它很好地例证了计算机科学是如何影响其他领域甚至扎根于其他领域的。

另一种数据挖掘的形式是**聚类分析**（cluster analysis），它用来发现类。注意，这与类描述不同，类描述是用来发现已确定的类中成员的特性的。更准确地说，聚类分析试图找到那些能引导发现分组的数据项的特性。例如，在分析观看某部电影的观众年龄信息的过程中，通过聚类分析可能会发现，观众会分成两个年龄组，即4～10岁年龄组和25～40岁年龄组。（也许这部电影吸引了孩子和他们的父母？）

还有一种数据挖掘的形式，称为**关联分析**（association analysis），它的工作是寻找两个数据组之间的联系。关联分析可以揭示买土豆片又买啤酒和饮料的顾客，或者既在正常的工作日购物又能享受退休优惠的顾客。

孤立点分析（outlier analysis）是数据挖掘的另一种形式。它试图识别不符合规则的数据项。孤立点分析可以用于识别数据集中的错误，用于通过检测信用卡是否突然偏离客户的正常消费模式来识别信用卡是否被盗用，甚至可以用于通过发现反常行为来识别潜在的恐怖分子。

基本知识点

- 组合数据源、聚类数据和数据分类是使用计算机处理信息的过程的一部分。

最后，还有一种数据挖掘的形式称为**序列模式分析**（ sequential pattern analysis ），它试图识别随时间变化的行为模式。例如，序列模式分析可以揭示股票市场等经济系统中的趋势，或者气候环境等环境系统中的趋势。

最后这个例子表明，数据挖掘的结果可以用来预测未来的行为。如果一个实体具有表征某个类的特性，那么这个实体就可能表现得像这个类的成员。然而，许多数据挖掘项目只是为了获得对数据的更好理解，如利用数据挖掘来解开DNA之谜就证明了这一点。无论如何，数据挖掘具有巨大的潜在应用领域，并且有望成为未来几年的一个活跃的研究领域。

注意，数据库技术和数据挖掘之间的关系就像堂兄弟一样，因此一个领域的研究会影响另一个领域。数据库技术广泛运用，使得数据仓库具有以**数据立方**（ data cube ）（ 从多角度看待数据——**立方**这个术语用来推测多维图像 ）的形式表示数据的能力，这就使得数据挖掘成为可能。反过来，当数据挖掘方面的研究人员改进了实现数据立方的技术时，这些成果也会给数据库设计领域带来了好处。

最后，我们应当认识到，成功的数据挖掘远不仅包括数据集范围内的模式识别，还必须应用明智的判断来确定哪些模式是有实际意义的，哪些只是巧合。某家便利店出售大量中奖彩票对计划买彩票的人

来说可能并不重要，但是发现买快餐的顾客也常会买冷冻食品对杂货店老板来说就是一条很有意义的信息。同样，数据挖掘也包括大量的伦理问题，这些问题涉及数据仓库代表的个人的权利、所得结论的准确性和使用，甚至涉及数据挖掘初衷是否恰当。

9.6节问题与练习

1. 为什么数据挖掘不在“联机”数据库上实施？
2. 给出通过文中提到的每种类型的数据挖掘可以发现的模式的另外一个例子。
3. 给出数据立方在挖掘销售数据时可能允许的几种不同视角。
4. 数据挖掘与传统的数据库查询有何不同？

9.7 数据库技术的社会影响

随着数据库技术的发展，以往那些埋藏在秘密记录里的信息都变成可获取的了。许多情况下，自动化图书馆系统会将读者的阅读喜好放在触手可及的地方，零售商保存其客户的购买记录，因特网搜索引擎保留其客户端请求的记录。而且，这些信息可能会提供给营销公司、执法机构、政党、雇主和个人。

基本知识点

- 安全和隐私问题会与包含个人信息的数据同时出现。

这是渗透到整个数据库应用范围的潜在问题的代表。技术使收集大量的数据以及通过合并或比较不同的数据集来获得它们之间的关系变得非常容易，而这些信息以前是隐藏起来的。所带来的法律和道德的影响（既有正面的也有负面的）是巨大的。这些影响不仅是学术界辩论的主题，更是现实。

基本知识点

- 通过计算实现的创新会引起法律和道德关注。

现在的数据收集过程在有些情况下比较明显，而在有些情况下就显得比较微妙了。前一种情况的例子是直接要求某人提供信息。这可能以自愿的方式进行，如民意调查或竞赛登记等形式，也可能以非自愿的方式进行，如以政府规定强制进行等。有时，自愿与否取决于个人的观点。申请贷款时提供个人信息是自愿的还是非自愿的？区别取决于接受贷款是为了方便还是必需的。现在有些零售商在你使用信用卡时，会要求你允许以数字化格式记录你的签名。同样，提供这种信息是自愿的还是非自愿的也取决于所处的环境

数据收集比较微妙的情况下，就避免了与主体直接进行交流。这样的例子有：信用卡公司记录了信用卡持有者的所有购物活动，网站记录了访问者的身份，社会活动家记录了停在目标机构停车场的汽车的车牌号。在这些情况下，数据收集的主体可能不会意识到他们的信息被收集，更不大可能意识到存在为此建立的数据库。

有时，只要一个人停下来想想，这种潜在的数据收集活动就很清楚了。例如，杂货店可能会为已经登记过的常客提供折扣。登记过程中可能要发一个身份识别卡，在购物时要出示该卡才能享受折扣。这样商店就可以收集大量客户的购物记录，而这种记录的价值远远超出了折扣的价值。

当然，推动数据收集繁荣发展的动力就是数据的价值，数据的价值因数据库技术的发展而得到放大，这些数据库技术使数据能够联系起来，这就揭示出了原本模糊的信息。例如，对信用卡持有者的消费模式进行分类和交叉列出，就能获得极具市场价值的顾客资料概况。健身杂志的订阅单可以寄给那些最近买过健身器材的人，而驯狗杂志的订阅单则可以瞄准那些不久前买过狗食的人。有时，信息的组合方式实在是很有想象力。例如，将犯罪记录与社会福利记录进行对比，可以找到并抓获假释期间的违法者；1984年美国的义务兵役注册机构利用从一家著名的冰淇淋店获得的生日登记表，找出了那些逃避兵役登记的公民。

有若干种方法可以用来防止社会滥用数据库。一种方法是应用法律手段。遗憾的是，通过一部法律来反对一种行为只能规定一种行为是非法的，但阻止不了这种行为的发生。最好的例子是1974年美国通过的《隐私权法案》（Privacy Act），其目的是保护公民免受政府数据库滥用的危害。该法案中一个条款规定，政府机构要在《联邦公报》（Federal Register）上公布其数据库通告，允许公民访问和纠正他们的个人信息。然而，政府机构却迟迟不能遵照这个条款。这倒并

并不意味着那些部门有恶意的目的。在许多情况下，问题出在官僚作风。但是，官僚机构可能正在构建它无法识别的人事数据库这个事实是令人不安的。

另一种（也许更强大的）控制数据库滥用的方法是公众舆论。如果惩罚超过好处，数据库就不会被滥用，并且企业最害怕的惩罚就是负面的公众舆论，因为这直击要害。20世纪90年代初期，正是公众舆论阻止了一些主要征信局为营销目的出售邮件列表。更近一点儿的例子，谷歌公司于2011年停止了它的Google Buzz社交网络工具，此前，这个工具自动共享流行的Gmail工具中的联系人信息，这受到了公众的严厉批评。即使是政府机构也会向公众舆论妥协。1997年，美国社会保障局（Social Security Administration）修改了可以通过因特网查询社会保障记录的计划，这是因为公众舆论对信息的安全性产生了质疑。有些情况下，几天就能得到结果，这与需要经过冗长法律程序才能得到结果的情况形成了鲜明的对比。

基本知识点

- 如果忽视隐私和其他保护，信息的企业和政府管理可能会被利用。

当然，在许多情况下，数据的持有者和数据的主体都受益于数据库应用，但是在所有情况下，都不应该轻视丧失隐私权。当信息很准确时，这种隐私问题是很严重的；而当信息有错误时，隐私问题会变得硕大无比。当意识到自己的信用评级受到了错误信息的负面影响

时，你可以想象出那种无望的感觉。不难想象，在这个错误信息很容易被共享给其他机构的环境下，你的问题会被怎样地放大。一般来说，隐私问题是并且仍将是技术（特别是数据库技术）进步带来的一个主要副作用。要解决这些问题就需要我们成为有素质的、警觉的、积极的公民。

9.7节问题与练习

- 1．应该为了识别有犯罪倾向的个人而给执法机构访问数据库的权利吗？尽管这些个人可能没有犯过罪？
- 2．应该为了识别有潜在健康问题的个人而给保险公司访问数据库的权利吗？尽管这些个人并无任何症状？
- 3．假设你的经济状况良好。如果这个信息被各种机构分享，你能从中得到什么好处？你需要忍受同一信息发布的什么不利后果？如果你经济状况不好，结果又会如何？
- 4．新闻自由在控制数据库滥用方面起什么作用？（例如，新闻对公众舆论或滥用曝光有多大影响？）

复习题

（带*的题目涉及选读章节的内容。）

- 1．概述平面文件与数据库之间的不同。
- 2．数据独立性是什么意思？

3. 在数据库实现的层次化方法中，数据库管理系统的作用是什么？

4. 模式和子模式有什么不同？

5. 指出把应用软件与数据库管理系统分离的两个好处。

6. 描述抽象数据类型（见第8章）与数据库模型的相似之处。

7. 说出下列情况或活动发生在数据库系统（用户、应用程序员、数据库管理系统软件的设计者）的哪个层次。

a. 数据在磁盘上怎样存储效率才最高？

b. 243航班还有空位吗？

c. 在海量存储器中关系应当如何组织？

d. 在终止会话之前应该允许用户敲错几次密码？

e. 怎样才能实现PROJECT运算？

8. 下列哪项工作是由数据库管理系统完成的？

a. 确保将用户对数据库的访问权限限制在相应的子模式内。

b. 把基于数据库模型的命令翻译成适合实际数据存储系统的活动。

c. 隐藏数据库中的数据实际分散在网络中的许多计算机内的这一事实。

9. 描述在关系数据库中如何表示下列关于航空公司、（某一天的）航班和乘客的信息。

航空公司：Clear Sky、Long Hop和Tree Top

Clear Sky的航班：CS205、CS37和CS102

Long Hop的航班：LH67和LH89

Tree Top的航班：TT331和TT809

Smith已预订CS205（座位12B）、CS37（座位18C）和LH89（座位14A）。

Baker已预订CS37（座位18B）和LH89（座位14B）。

Clark已预订LH67（座位5A）和TT331（座位4B）。

10. 从何种程度上讲，对一个关系应用SELECT和PROJECT运算的次序是有意义的？或者说，在什么条件下，先用SELECT再用PROJECT的结果和先用PROJECT再用SELECT的结果是一样的？

11. 给出一个论据，证明9.2节描述的JOIN运算中的where子句是不必要的。（也就是说，要证明任何使用where子句的查询都应该能够

通过下面这样的方式重新表示：用JOIN运算把一个关系中的每个元组与另一个关系中的每个元组连接起来。)

12. 根据下面显示的关系，执行下列每个指令之后，关系RESULT是怎样的：

X关系			Y关系	
U	V	W	R	S
A	Z	5	3	J
B	D	3	4	K
C	Q	5		

a. $RESULT \leftarrow PROJECT\ W\ from\ X$

b. $RESULT \leftarrow SELECT\ from\ X\ where\ W = 5$

c. $RESULT \leftarrow PROJECT\ S\ from\ Y$

d. $RESULT \leftarrow JOIN\ X\ and\ Y\ where\ X.W \geq Y.R$

13. 根据以下数据库，利用SELECT、PROJECT和JOIN命令，写出指令序列来回答下列关于零件及其生产商的问题。

Part关系	
PartName	Weight
Bolt 2X	1
Bolt 2Z	1.5
Nut V5	0.5

Manufacturer关系

CompanyName	PartName	Cost
Company X	Bolt 2Z	.03
Company X	Nut V5	.01
Company Y	Bolt 2X	.02
Company Y	Nut V5	.01
Company Y	Bolt 2Z	.04
Company Z	Nut V5	.01

a . 哪些公司生产了Bolt 2Z ?

b . 获取Company X生产的零件以及每种零件成本的列表。

c . 哪些公司生产重量为1的零件 ?

14 . 用SQL回答第13题。

15 . 利用SELECT、PROJECT和JOIN命令 , 编写指令序列来回答下面关于图9-5中EMPLOYEE、JOB和ASSIGNMENT关系中的信息的问题。

a . 获取公司员工的姓名和地址列表。

b . 获取曾在人事部工作过或正在人事部工作的人员的姓名和地址列表。

c . 获取正在人事部工作的人员的姓名和地址列表。

16 . 用SQL回答上一题。

17 . 设计一个包含作曲家、作曲家生平及其作品信息的关系数据库。(避免类似于图9-4中的冗余。)

18．设计一个包含音乐表演者、其唱片以及其所录乐曲的作曲者信息的关系数据库。（避免类似于图9-4中的冗余。）

19．设计一个包含计算设备生产商及其产品信息的关系数据库。（避免类似于图9-4中的冗余。）

20．设计一个包含有关出版商、杂志及订户信息的关系数据库。（避免类似于图9-4中的冗余。）

21．设计一个包含有关零件、供应商及客户信息的关系数据库。每种零件可由几个供应商供应，并可有多个客户订购。每个供应商可以供应许多种零件，也可有多个客户。每个客户可以向多个供应商订购多种零件；实际上，同一种零件可向一个以上的供应商订购。（避免类似于图9-4中的冗余。）

22．写出一个指令序列（利用SELECT、PROJECT和JOIN运算），检索图9-5中描述的关系数据库中财务部每个职位的JobId、StartDate及TermDate。

23．用SQL回答第22题。

24．编写一个指令序列（利用SELECT、PROJECT和JOIN运算），检索图9-5中描述的关系数据库中每位在职员工的Name、Address、JobTitle及Dept。

25．用SQL回答第24题。

26．编写一个指令序列（利用SELECT、PROJECT和JOIN运算），检索图9-5中描述的关系数据库中每个在职员工的Name及JobTitle。

27．用SQL回答第26题。

28．由单个关系

Name	Department	TelephoneNumber
Jones	Sales	555-2222
Smith	Sales	555-3333
Baker	Personnel	555-4444

和两个关系

Name	Department
Jones	Sales
Smith	Sales
Baker	Personnel

Department	TelephoneNumber
Sales	555-2222
Sales	555-3333
Personnel	555-4444

提供的信息有什么不同？

29．设计一个包含汽车零件及其子零件信息的关系数据库。务必考虑到这个事实：一个零件可能包含更小的零件，同时它本身可以是更大零件的零件。

30．选择一个常用的网站，设计一个你打算用作该网站支持数据库的关系数据库。

31．基于图9-5所示的数据库，说明以下程序段回答的问题：

```
TEMP ← SELECT from ASSIGNMENT
      where TermDate = '*'
RESULT ← PROJECT JobId, StartDate
      from TEMP
```

32．把第31题中的查询翻译成SQL语句。

33．基于图9-5所示的数据库，说明以下程序段回答的问题：

```
TEMP1 ← JOIN EMPLOYEE and ASSIGNMENT
      where EMPLOYEE.EmplId =
            ASSIGNMENT.EmplId
TEMP2 ← SELECT from TEMP1 where
      TermDate = '*'
RESULT ← PROJECT Name, StartDate
      from TEMP2
```

34．把第33题中的查询翻译成SQL语句。

35．基于图9-5所示的数据库，说明以下程序段回答的问题：

```
TEMP1 ← JOIN EMPLOYEE and JOB
      where EMPLOYEE.EmplId = JOB.EmplId
TEMP2 ← SELECT from TEMP1 where
      Dept = 'SALES'
RESULT ← PROJECT Name from TEMP2
```

36．把第35题中的查询翻译成SQL语句。

37 . 把下面的SQL语句翻译成SELECT、PROJECT和JOIN运算的序列。

```
SELECT Job.JobTitle
FROM Assignment,Job
WHERE Assignment.JobId = Job.JobId
      AND Assignment.EmplId = '34Y70'
```

38 . 把下面的SQL语句翻译成SELECT、PROJECT和JOIN运算的序列。

```
SELECT Assignment.StartDate
FROM Assignment , Employee
WHERE Assignment.EmplId =
      Employee.EmplId
      AND Employee.Name = 'Joe E . Baker'
```

39 . 描述将以下SQL语句用到第13题中的数据库上的效果。

```
INSERT INTO Manufacturer
VALUES('Company Z', 'Bolt 2X', .03)
```

40 . 描述将以下SQL语句用到第13题中的数据库上的效果。

```
UPDATE Manufacturer
SET Cost = .03
WHERE CompanyName = 'Company Y'
      AND PartName = 'Bolt 2X'
```

*41. 确定你期望在用来维护杂货店库存的面向对象数据库中找到的几个对象，并说明你期望在其中的每个对象中包含哪些方法？

*42 . 确定你期望在用来维护图书馆馆藏记录的面向对象数据库中找到的几个对象，并说明你期望在其中的每个对象中包含哪些方法？

*43．如果T1和T2两个事务按如下安排来调度，会产生什么样的错误信息？

设计T1来计算账户A和B的总和，设计T2来将账户A的100美元转账到账户B。T1先检索账户A的余额，然后T2执行转账，最后T1检索账户B的余额，并输出它检索到的两个值的总和。

*44．说明如何用文中介绍的锁定协议来解决第43题中产生的错误。

*45．如果第43题中的T1是较新的事务，那么受伤等待协议会对事件序列有什么影响？如果T2是较新的事务呢？

*46．假设有一个事务试图向一个余额为200美元的账户中存入100美元，同时另一事务试图从同一账户中取出100美元。描述如何通过这些事务的交叉处理，使得最后余额为100美元。描述如何通过这些事务的交叉处理，使得最后余额为300美元。

*47．一个事务对数据库中的一个数据项有互斥访问和有共享访问有什么不同？为什么这种差别很重要？

*48．9.4节中讨论的涉及并发事务的问题并不限于数据库环境。用多个字处理程序访问同一个文档时会发生什么类似的问题？（如果你的PC中有字处理程序，试着激活两个实例来访问同一文档，看看会发生什么。）

*49．假设一个顺序文件有50 000条记录，询问一条记录需要5毫秒。请问：检索一条处在文件中间部位的记录，需要等待多长时间？

*50．见图9-15中的归并算法，如果其中一个输入文件一开始就是空的，请列出执行该算法的步骤。

*51．修改图9-15中的算法来处理这样一种情况：两个输入文件都包含一个有相同键字段值的记录。假定这些记录完全相同，并且输出文件中应该只出现一个。

*52．设计一个系统，利用这个系统，存储在磁盘上的一个文件能够作为顺序文件以两种不同的顺序进行处理。

*53．描述如何能够利用一个文本文件作为基本结构来构建包含杂志订阅者信息的顺序文件。

*54．设计一种技术，通过这种技术将逻辑记录大小并不一致的顺序文件实现为文本文件。例如，假设你想构建一个顺序文件，其中每条逻辑记录都包含一位小说家的信息及其作品的列表。

*55．索引文件与散列文件相比有什么优势？散列文件与索引文件相比又有什么优势？

*56．本章描述了传统文件索引与由操作系统维护的文件目录系统之间的相似之处。在哪些方面操作系统的文件目录与传统索引有不同？

*57. 如果将散列文件分到10个桶里，那么任意3条记录中至少有2条记录散列到同一个桶里的概率是多少？（假定散列函数没给任何桶分配高于其他桶的优先级。）文件中必须存储多少条记录，才更可能发生碰撞？

*58. 假设将文件分到100个桶里而不是10个桶，重解第57题。

*59. 如果我们使用本章讨论的除法技术作为散列函数，并且将文件存储区分成23个桶，那么应该搜索哪个区域来查找其键（当把键翻译为一个二进制值时）为整数124的记录？

*60. 比较散列文件的实现与同构二维数组的实现。散列函数与地址多项式的作用有什么类似之处？

*61. 试给出下列比较的一个优势。

a. 顺序文件优于索引文件。

b. 顺序文件优于散列文件。

c. 索引文件优于顺序文件。

d. 索引文件优于散列文件。

e. 散列文件优于顺序文件。

f. 散列文件优于索引文件。

*62 . 从哪个方面可以看出顺序文件类似于链表？

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1 . 在美国，所有联邦囚犯的DNA记录现在都存储在一个数据库中，供刑事侦查用。如果发布这些信息用于其他用途，如用于医学研究，道德吗？如果道德，可用于什么目的？如果不道德，为什么？每种情况的利与弊又是什么？

2 . 应该允许大学能将其学生的相关信息公开到何种程度？可以公布他们的姓名和地址吗？可以在不标识学生身份的情况下公布他们的成绩分布呢？你的答案和第1题的答案一致吗？

3 . 构建有关个人的数据库时，采取什么样的限制比较合适？政府有权持有公民的哪些信息？保险公司有权持有其客户的哪些信息？公司有权持有其员工的哪些信息？在这些设定的情况下应该实施控制吗？如果应该，如何实施？

4 . 信用卡公司把它的客户的消费模式卖给营销公司是否合适？赛车邮购公司把它的邮购清单卖给赛车杂志是否可以接受？美国国税局把那些有巨额收入的纳税人的姓名和地址信息卖给股票经纪人是否可

以接受？如果你没有充分的把握回答是或否，那么你能提出什么可接受的策略吗？

5．数据库的设计者对如何使用数据库中的信息要负责到什么程度？

6．假设数据库错误地允许未经批准的对数据库信息的访问。如果信息被不当获得和使用，那么数据库设计者应分担多大的信息误用责任？你的回答与作恶者为发现数据库设计漏洞并获取未授权信息所花费的精力大小有关吗？

7．数据挖掘的盛行带来了大量的道德问题和隐私问题。如果数据挖掘揭示了你所在社区的所有居民的某些特征，那么你的隐私受到侵犯了吗？数据挖掘的使用是促进了商业的发展还是鼓励了盲从？已知从人口普查数据中提取的信息比从个人问卷调查中提取的信息多，强制公民参加人口普查在多大程度上是适当的？数据挖掘给营销公司带来的好处对于毫不知情的受众是否不公平？这样一种状况的好与坏到了什么程度？

8．应该允许个人或公司在多大程度上收集和持有私人信息？假使这些信息已经能公开地收集但是分散在多个来源中会怎样？应该期望个人或公司在何种程度上保护这类信息？

9．许多图书馆提供查阅服务，以便读者在搜索信息时可以得到图书管理员的帮助。因特网和数据库技术的存在会使这项服务过时吗？

如果会，那是前进还是倒退？如果不会，为什么？因特网和数据库技术的存在对图书馆本身的存在有怎样的影响？

10．你的身份信息被盗用的可能性到了什么程度？你会采取哪些措施使被盗用机会最小？如果你是身份盗用的受害者，你可能会受到什么损害？如果发生身份盗用，你自己有责任吗？

课外阅读

Berstein, A., M. Kifer and P. M. Lewis. *Database Systems*, 2nd ed. Boston, MA: Addison-Wesley, 2006.

Connolly, T. and C.E. Beg. *Database Systems: A Practical Approach to Design, Implementation and Management*, 5th ed. Boston, MA: Addison-Wesley, 2009.

Date, C. J. *An Introduction to Database Systems*, 8th ed. Boston, MA: Addison-Wesley, 2004.

Date, C. J. *Databases, Types and the Relational Model*, 3rd ed. Boston, MA: Addison-Wesley, 2007.

Elmasri, R. and S. Navathe. *Fundamentals of Database Systems*, 7th ed. Essex, England: Pearson, 2015.

Patrick, J. J. *SQL Fundamentals*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2009.

Silberschatz, A., H. Korth, and S. Sudarshan. *Database Systems Concepts*, 6th ed. New York: McGraw-Hill, 2009.

Ullman, J. D. and J. D. Widom. *A First Course in Database Systems*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2008.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第10章 计算机图形学

本章将探索计算机图形学领域——一个对电影和交互式视频游戏的制作具有重大影响的领域。实际上，计算机图形学的发展正使视觉媒体脱离现实的限制，许多人认为计算机生成的动画可能很快就会取代整个影视业对传统的演员、布景和摄影的需求。

本章内容

10.1 计算机图形学的范围

10.2 3D图形学概述

10.3 建模

10.4 渲染

*10.5 处理全局光照

10.6 动画

持久理解和学习成果

计算使人能够采用创造性的开发过程创建用于创造性表达或用于解决问题的计算工件。AP[®] CSP

学习成果 通过组合或修改现有制品来创建新的计算制品。

计算可以扩展传统形式的人类表达和体验。AP[®]CSP

学习成果 使用计算工具和技术进行创造性表达。

模型和模拟利用抽象生成新的理解和知识。AP[®]CSP

学习成果 使用模型和模拟来表示现象。

学习成果 使用模型和模拟来制定、细化和测试假设。

可以开发程序进行创造性表达、满足个人好奇心、创造新知识或者解决问题（来帮助个人、组织或社会）。AP[®]CSP

学习成果 开发程序进行创造性的表达、满足个人好奇心或者创造新知识。

计算几乎能在所有领域实现创新。AP[®]CSP

学习成果 解释计算是如何影响其他领域创新的。

计算机图形学是计算机科学的分支，它应用计算机技术制作和操控视觉表现。它与各种各样的主题相关联，包括文本的呈现、图形和图表的构建、图形化用户界面的开发、照片的处理、视频游戏的制作、动画电影的创作等。然而，“计算机图形学”这一术语越来越多地被用来指代3D图形学这一特定领域，因此本章大部分内容将集中在3D

图形学这个主题上。我们将从定义3D图形学开始，阐明它在较广义的计算机图形学中的作用。

10.1 计算机图形学的范围

随着数字摄像机的出现，处理数字编码图像的软件迅速流行起来。这类软件让人可以通过去除瑕疵和可怕的“红眼”等操作达到“润色”照片的目的，也可以从不同的照片裁剪和粘贴一部分来“修饰”照片，创造不必反映现实的图像。

在影视业中，类似的技术经常被用来制造特效。事实上，这些应用是影视业从模拟系统（如胶卷）转向数字编码图像的主要推动因素。这类应用可用于改变最初由摄像机捕捉的动作，包括去除支撑钢丝、叠加多个图像或产生新图像序列帧等。

基本知识点

- 数字图像可以通过生成像素模式、处理现有数字图像或者组合图像来创建。

除了处理数字照片和电影帧的软件外，现在还有各种各样的工具/应用软件包，可帮助制作从简单线条到复杂艺术作品的二维图像。

（一个众所周知的最基本的例子就是微软的“画图”应用程序。）这类程序的基本操作包括让用户绘制点和线、插入椭圆和矩形这类简单的几何图形、给区域填充颜色以及剪切和粘贴图画指定部分。电影、

电视和计算机游戏制作人不仅可以使⽤现有的商业工具，还可以创建定制的软件工具来推动新画面和动画的创造。

基本知识点

- 数字特效和动画可以使⽤现有软件创造，也可以使⽤包含实现特效和动画功能的软件来修改。

注意，上面所有的应⽤都是⽤于处理平面的二维图形和图像的。因此，有两个相关的研究领域：一个是**2D图形学**（2D graphics），另一个是**图像处理**（image processing）。二者的区别在于：2D图形学侧重于把二维图形（圆、矩形、文字等）转化为像素模式来产⽣图像，⽽图像处理（第11章研究⼈⼯智能时会介绍）侧重于分析图像中的像素来识别模式，以达到增强或“理解”图像的目的。简而言之，2D图形学是⽤来处理生成图像的，⽽图像处理是⽤来处理分析图像的。

基本知识点

- 计算能够对现实和虚拟现象进⾏创造性探索。

与2D图形学中把二维图形转换为图像相对应，**3D图形学**（3D graphics）领域是把三维图形转换成图像。这个过程是：建造三维场景的数字编码版本，然后模拟摄影过程来制作这些场景的图像。这与传统的摄影类似，不同之处在于，场景是使⽤3D图形学技术“拍摄”出来的，不是真实存在的，⽽是以数据和算法的集合的形式“存在的”。因

此，3D图形学“拍摄”的是虚拟世界，而传统的摄影拍摄的是真实世界。

注意到使用3D图形创建图像要经历两个不同的步骤很重要。这两个步骤中一个是创建、编码、存储和处理被拍摄出来的场景，另一个是生成图像的过程。前者是创造性的艺术过程，而后者是密集的计算过程。这些是我们在接下来的4节中要探索的主题。

3D图形学可以制作虚拟场景的“照片”的事实，使它非常适用于交互式视频游戏和动画电影的制作。否则，其中的现实束缚会限制动作。公平地说，用于创造新画面、音乐和特效的强大计算工具甚至正在改变娱乐产业试图讲述的各种故事。交互式视频游戏是由编码的三维虚拟环境构成的，游戏玩家与之进行交互，玩家看到的图像是通过3D图形学技术制作出来的。动画电影是用类似的方法创作的，只是与虚拟环境交互的是人类动画师，而不是最终观众。公众最终看到的则是导演/制片人确定的二维图像序列。

基本知识点

- 数字特效、图像、音频、视频和动画的创造已经改变了相关产业。

- 计算方面的发展已经产生和提高了其他领域的创造力。

我们将在10.6节中更全面地研究3D图形学在动画中的应用。现在，让我们通过预测随着3D图形技术的发展，这些应用可能会走向何

方来结束本节。如今，电影是作为二维图像序列发布的。尽管显示这些信息的放映机已经从使用成卷胶片的模拟设备发展到使用DVD播放机和平板显示器的数字技术，但它们仍然只能处理二维的。

但是，想象一下，随着创建和操作真实的三维虚拟世界的能力得到改善，会发生什么改变。我们将不仅能“拍摄”这些虚拟世界和以二维图像的形式发布电影，而且能发布虚拟世界。观众不仅可以观看电影，还可以置身于电影布景中。通过“3D图形放映机”来观看这一三维场景，就像通过专用的“游戏盒子”来观看视频游戏一样。观众可能先看到导演/制片人预定的“推荐情节”，与此同时也可以与虚拟布景进行互动产生其他的新情节，就像玩视频游戏一样。这种可能性非常大，特别是在我们也考虑到正在被开发的三维人机界面的潜力的时候。

10.1 节问题与练习

1. 概述图像处理、2D图形学和3D图形学之间的区别。
2. 3D图形学与传统摄影有何不同？
3. 应用3D图形学制作“照片”的两个主要步骤是什么？

10.2 3D图形学概述

我们从创建和显示图像的整个过程开始对3D图形学的学习，这个过程由3步构成：建模、渲染和显示。建模步骤（将在10.3节中详细介绍）与传统电影业中设计和构建布景类似，只是3D图形场景是用数字

编码数据和算法构建的。因此，在计算机图形学背景下产生的场景可能在现实中永远都不存在。

下一步就是通过计算场景中的物体如何出现在由特定位置的摄像机拍摄的照片中，来制作场景的二维图像。这个步骤称为**渲染**（rendering），是10.4节和10.5节的主题。渲染的概念是运用解析几何计算场景中的物体到一个称为**投影平面**（projection plane）的面上会形成的投影，这种方式与摄像机将场景投影到胶卷上的方式类似（见图10-1）。这种投影称为**透视投影**（perspective projection），在这种投影方式下，所有的物体都沿一条称为**投影线**（projector）的直线投影，这条直线是从一个称为**投影中心**（center of projection）或**视点**（view point）的公共点延伸出来的。这与**平行投影**（parallel projection）不同，平行投影中的投影线是平行的。透视投影产生的投影与人眼看到的类似，而平行投影产生的是物体“真正”的剖面，这在工程制图中通常非常有用。

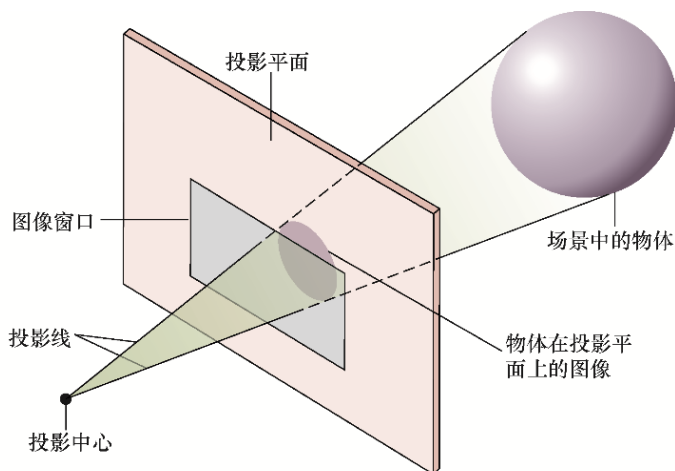


图10-1 3D图形学范例

定义最终图像边界的投影平面的受限部分称为**图像窗口**（image window）。它相当于大多数模拟摄像机取景器中显示的矩形，用于指明潜在图像的边界。实际上，人从大多数模拟摄像机的取景器中看到的摄像机投影平面都比它的图像窗口大。（举个例子，你可以在取景器中看到玛莎阿姨的头顶，但是如果她的头顶没出现在图像窗口内，它就不会出现在最终图像中。）

一旦确定了投影到图像窗口的场景部分，就可以计算出最终图像上的每个像素的显示情况，这种逐个像素的计算过程会很复杂，因为它需要确定场景中的物体如何与光线融合（在明亮光线下坚硬且有光泽的表面渲染与在间接光线下柔软且透明的表面渲染应该有所不同）。因此，渲染过程大量借鉴了包括材料科学和物理学在内的众多领域。此外，在确定一个物体的外观时经常需要了解场景中的其他物体。这个物体可能处在另一个物体的阴影中，或者这个物体是一面镜子，它的外观实质上就是另一个物体的外观。灯光——在强光下，坚硬、有光泽的表面的渲染方式应与在间接光下的柔软、透明表面的渲染方式不同。

当确定了每个像素的外观后，结果被表示成图像的位图，集中存储在称为**帧缓冲区**（frame buffer）的存储区域中。这个缓冲区可能是主存储器的一个区域，或者在有专门为图形应用设计的硬件的情况下，它也可能是一块专用存储电路。

最后，存储在帧缓冲区中的图像要么为了观看而显示，要么为了以后显示传送给更持久的存储器。如果正在制作的图像是用于电影

的，那么在最终展示之前它可能会被存储甚至修改。但是，在交互式视频游戏或飞行模拟器中，必须随着图像的实时生成来显示图像，这个要求经常限制了创建的图像的质量。这就是由电影业发布的功能齐全的动画制作的图像质量要超过当今交互式视频游戏的图像质量的原因。

我们通过分析一个典型的视频游戏系统来结束对3D图形的介绍。这个游戏实际上就是一个编码的虚拟世界和软件的结合体，它允许游戏玩家操控这个虚拟世界。玩家操控这个虚拟世界时，游戏系统会不断地渲染场景并把图像存储到图像缓冲区中。为了克服真实世界的时间限制，大多数渲染过程都是由专用硬件处理的。实际上，正是这些硬件使游戏系统和一般个人计算机之间有了显著差别。最后，游戏中的显示设备显示帧缓冲区中的内容，给玩家以变化场景的幻觉。

10.2节问题与练习

1. 总结使用3D图形生成图像时涉及的3个步骤。
2. 投影平面和图像窗口有何不同？
3. 什么是帧缓冲区？

10.3 建模

3D计算机图形项目开始的方式与戏剧舞台制作方式十分相似——必须设计布景并收集或者搭建所需的道具。在计算机图形学术语中，布景称为**场景**（scene），道具称为**物体**（object）。记住，3D图形场

景是虚拟的，因为组成它的物体是由数字编码模型“构建”而成，并不是实际的物理结构。

在本节中，我们将探讨与“构建”物体和场景有关的话题。我们从单个物体的建模问题开始，并以考虑收集这些物体来形成场景这个任务结束。

10.3.1 单个物体的建模

在舞台制作中，道具与现实的契合程度取决于它在场景中的使用方式。我们可能不需要一辆完整的汽车，电话并不需要能用，背景可能只是画在背景幕布上的。同样，就计算机图形学而言，一个物体的软件模型能准确地反映物体真实属性的程度依赖于情境的需要。前景中物体的建模比背景中的物体的建模需要更多的细节。而且，在那些不受严格实时限制的情况下，会产生更多的细节。然而，伴随细节而来的是计算工作（时间）和数据表示（空间）上的相关成本，所以我们有抽象掉不必要细节的强烈动机。物体的计算机模型和相互作用的模拟必须从抽象开始，这些抽象比现实世界中的抽象要简单。

基本知识点

- 模型和模拟是较复杂的物体或现象的简化表示。

因此，一些物体模型可能相对简单，而另一些可能极其复杂。作为一个通用规则，模型越精确，制作出的图像的质量越高，但渲染所

需要的时间也越长。因此，现在进行的大多数对于计算机图形学的研究都是在寻求用于构建高度细致但高效的物体模型的技术的开发。这些研究中有些涉及开发可以根据场景中物体的最终作用来提供不同的细节层次的模型，这样可以在变化的场景中使用同一个物体模型。

描述一个物体所需的信息包括物体的形状和其他特性，如决定物体如何与光相互作用的表面特性。现在，让我们考虑形状建模这个任务。

1. 形状

3D图形学中物体的形状通常被描述成称为**平面片**（planar patch）的小平面的集合，其中每一个平面片都是一个多边形。总体来说，这些多边形构成一个**多边形网格**（polygonal mesh），它近似于被描述物体的形状（见图10-2）。通过使用小平面片，近似值可达到所需的精度。

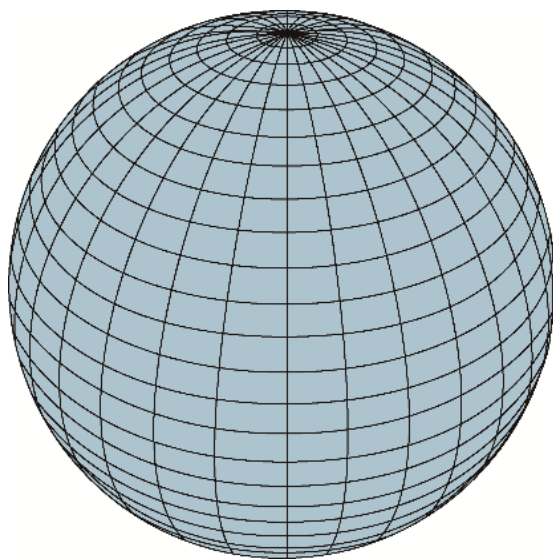


图10-2 球的多边形网格

多边形网格中的平面片经常选择为三角形，因为每个三角形都能用它的3个顶点表示，这是三维空间中确定一个平面所需点的最少数目。在任何情况下，多边形网格都可表示成其平面片顶点的集合。

一个物体的多边形网格表示可以通过多种途径获得。其中一种是以所需形状的精确的几何描述开始，然后用这个描述构建一个多边形网格。例如，解析几何中半径为 r 的球体（中心在原点）用方程来描述是：

$$r^2 = x^2 + y^2 + z^2$$

基于这个公式，我们可以建立球体上经线和纬线的方程，确定这些线的交叉点，然后用这些点作为多边形网格的顶点。类似的技术可以用到其他传统的几何形状上，这就是比较便宜的计算机生成的动画中的人物常常感觉像是用球体、圆柱体和锥体等结构拼凑而成的原因。

越常规的形状可以用越复杂的分析方法来描述。一种方法是使用**贝塞尔曲线**（Bezier curve）（以皮埃尔·贝塞尔命名，他在20世纪70年代早期提出了这个概念，当时他是雷诺汽车公司的工程师），它允许在三维空间中只用几个称为控制点的点来定义曲线段（其中有两个点表示曲线段的端点，而其他的点则指明曲线的变形方式）。例如，图10-3展示了一个由4个控制点定义的曲线。注意曲线是如何被拉向两个不是线段端点的控制点的。移动这些点，曲线被扭曲成不同的形状。

（当你用微软的“画图”这样的绘图软件包来构建曲线时就能体验到这种技术。）尽管我们不会在这里继续探讨这个话题，但描述曲线的贝塞尔方法可以被扩展为描述三维曲面，称为**贝塞尔曲面**（Bezier surface）。相应地，贝塞尔曲面被证明是获得复杂表面多边形网格的过程中高效的第一步。

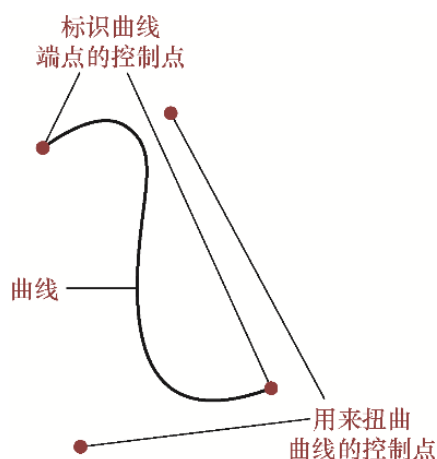


图10-3 贝塞尔曲线

你可能会问：为什么有必要把形状的精确描述（如球体的简明公式或描述贝塞尔曲面的公式）转换成使用多边形网格的形状近似？答案是用多边形网格表示所有物体的形状确立了渲染过程的统一方法——可以更高效地渲染整个场景的技巧。这样，尽管几何公式提供了形状的精确描述，但它们只是作为构建多边形网格的工具。

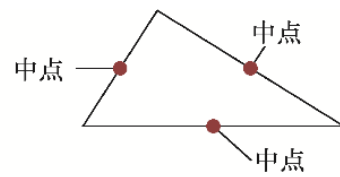
另外一种获得多边形网格的方法是用蛮力的方式构建网格。在无法用优雅的数学技术表示形状的情况下，这种方法很流行。这个过程是先构建物体的物理模型，然后通过用能记录自身三维空间位置的笔式设备来触摸表面，记录下模型表面点的位置——这个过程就称为**数**

字化（digitizing）。然后将得到的点的集合用作顶点来获得描述形状的多边形网格。

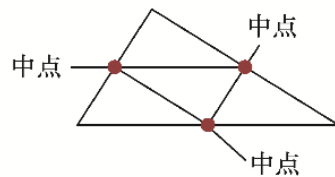
遗憾的是，有些形状非常复杂，难以用几何建模或手工数字化获得真实模型。这些例子包括复杂的植物结构（如树）、复杂的地形（如山脉）和气态物质（如云、烟或火苗）。在这些情况下，多边形网格可以通过编写自动构建所需形状的程序来获得。这样的程序统称为**程序化模型**（procedural model）。换言之，程序化模型是应用算法生成所需结构的程序单元。

例如，通过执行下列步骤，程序化模型已被用来生成山脉：以一个三角形开始，确定三条边的中点（见图10-4a）；然后，连接这些中点，形成4个较小的三角形（见图10-4b）；现在，在把原三角形的3个顶点固定住的同时，在三维空间里移动中间点（允许三角形的边线延长或缩短），从而扭曲三角形的形状（见图10-4c）；对每个更小的三角形重复这个过程（见图10-4d）；继续重复这个过程，直到获得所需的精度。

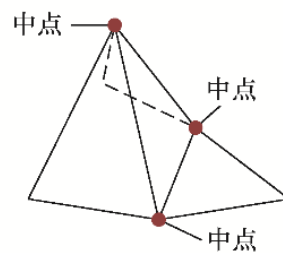
程序化模型提供了一种生成多个相似但又独特的复杂物体的有效方式。例如，可以用一个程序化模型构建各种各样逼真的树对象——每棵树都有自己的（但很相似的）分支结构。构建这些树模型的一种方法是应用分支规则来“长出”树，所用的方式与语法分析器（见6.4节）按语法规则构建语法分析树大致相同。事实上，在这些情况下使用的分支规则的集合经常被称为语法。一种语法可能是专为“长出”松树设计的，而另一种语法可能是专为“长出”橡树设计的。



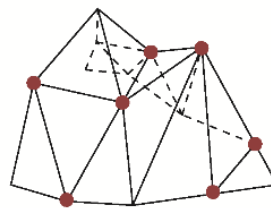
(a) 标识中点



(b) 连接中点



(c) 移动中点



(d) 在更小的三角形上重复这个过程

图10-4 产生一个山脉的多边形网格

另一种构建程序化模型的方法是将物体的基础结构模拟为一个大的粒子集合。这种模型称为**粒子系统**（particle system）。粒子系统通常会应用某些预定义的规则来移动系统中的粒子（可能所用的方式会让人想起分子间相互作用），产生所需的形状。例如，粒子系统已经被用来生成水面晃动的动画，我们将在后面介绍动画时看到。（想象一下，把一桶水建模为一桶弹珠。随着桶的摇晃，弹珠也随之到处翻

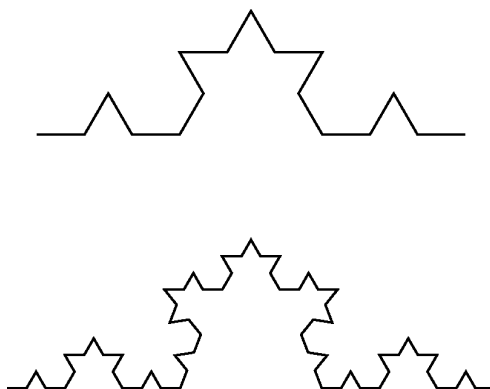
滚，模拟水的运动。）粒子系统应用的其他示例包括闪烁的炉火火焰、云和人群场景。

分 形

用文中描述的程序化模型构建山脉（见图10-5）是分形在3D图形学的例子。从技术上讲，分形（fractal）是“豪斯多夫维数大于其拓扑维数”的几何物体。直观上讲，这意味着物体是通过将较低维数的物体的副本“打包”而形成的。（想象一下宽度就是通过“打包”多条平行线段而创建的这一错觉。）分形通常是用递归过程来形成的，而递归中的每次激活就是重复“打包”另外用来建立分形模式（更小）的副本。分形的结果是其每个部分都是自相似的，当放大时，都表现为其自身的一个副本。分形的一个传统的示例就是科赫雪花，它是通过重复地用相同结构的较小版本替换结构中的直线段而形成的：



生成的细化序列如下：



分形在3D图形学领域经常是程序化模型的支柱。实际上，它们已经被用来制作逼真的山脉、蔬菜、云和烟的图像。

程序化模型的输出通常是一个近似于所需物体形状的多边形网格。在一些情况下，如使用三角形生成山脉，网格就是生成过程的自然结果。在另一些情况下，如根据分支规则长出树，网格可能就是额外的、最终的步骤。例如，在粒子系统中，系统外边缘上的粒子自然会被选作最终多边形网格的顶点。

由程序化模型生成的网格有多精确视具体情况而定。一个场景的背景中的树的程序化模型可能产生一个只反映树的基本形状的粗糙网格，而前景中的树的程序化模型可能产生能分清各个枝叶的网格。

2. 表面特征

仅由多边形网格构成的模型只捕获物体的形状。大多数渲染系统能在渲染过程中丰富这些模型，从而根据用户的需求模拟各种表面特征。例如，利用使用不同的着色技术（我们将在10.4节介绍），用户可以指定将球的多边形网格渲染成光滑的红球或是粗糙的绿球。在某些情况下，这种灵活性是可以做到的。但在需要如实渲染原始物体的情况下，关于物体的更具体的信息必须包含在模型中，这样渲染系统才会知道该做什么。

除了物体形状，还有多种用于编码物体信息的技术。例如，人可以沿着多边形网格的每个顶点，在物体的这一点上编码原始物体的颜

色。然后在渲染过程中用这一信息重新创建原始物体的外观。

在其他情况下，通过称为**纹理映射**（texture mapping）的过程，颜色模式可以与物体表面相关联。纹理映射类似于贴墙纸，因为它是将一个预定义的图像与一个物体的表面相关联。这个图像可能是数字照片、艺术家的绘画，也可能是计算机生成的图像。传统的纹理图像包括砖墙、木纹表面和大理石表面。

例如，假设我们想要对石墙建模，我们可以用描述长方体的简单多边形网格来表示墙的形状；然后，用这些网格，我们就能提供石砌体的二维图像。随后，在渲染过程中，可以将这个图像用在长方体上，产生石墙的外观。更准确地，每当渲染过程需要确定墙上的点的外观时，它就只需简单地用砌体图像中对应的点的外观。

纹理映射应用到相对平坦的表面时效果最好。如果必须大幅扭曲纹理图像去覆盖弯曲的表面（想象成试图给一个沙滩球贴墙纸的问题），或者如果纹理图像完全裹着一个物体，并导致了接缝，在接缝处纹理模式可能不与它本身融合，那么效果看上去会不够逼真。尽管如此，纹理映射已经被证明是一种模拟纹理的有效方法，被广泛用于实时敏感的场所（一个基本的例子就是交互式视频游戏）。

3. 寻求逼真的效果

构建可以生成逼真图像的物体模型是一个正在研究的主题，特别有趣的是活体角色的材质，如皮肤、头发、毛皮和羽毛等。这些研究

大多是针对特殊物质的，并且包括建模和渲染技术。例如，为了获得人类皮肤的逼真模型，有些学者研究光穿透表皮层和真皮层的程度以及这些层的厚度对皮肤外观的影响。

另一个例子是人类头发的建模。如果头发是从远距离看，那么传统的建模技术就足够了。但是，对于特写镜头，显示出头发真实的质感将会是一个挑战。其中的问题包括半透明特性、纹理深度、悬垂特征和头发对像风这样的外力的反应方式。为了解决这些棘手的问题，有些应用程序转向对单缕头发建模（这是一项艰巨的任务，因为一个人大约有100 000根头发）。然而，更令人惊奇的是，有些研究者已经建立了头发模型，这些模型给出了每一缕头发的缩放纹理、颜色变化和机械动力学特征。

还有一个已经发展到相当精确的建模程度的例子，就是布的建模。在这个例子中，利用了编织模式的复杂细节来生成织物类型（如斜纹布与缎布）之间特有的纹理差别，并且将纱线的细节特性与编织模式数据组合在一起，创建出编织物的模型，产生逼真的特写图像。另外，还将物理学和机械工程学的知识应用到计算织物材料图像的单根线上去，以说明线的拉伸和织物修剪方面的特性。

正如我们所说，生成逼真图像是一个活跃的研究领域，它综合了建模和渲染过程的技术。一般来说，当取得进步时，新技术首先会整合到那些不受实时限制影响的应用中，如电影制作工作室中的图形软件，其中建模/渲染过程与图像的最终显示之间有明显的延迟。当这些新技术得到进一步发展完善和优化，它们就可以进入实时应用中了，

而且这些环境中的图形质量也就得到了提高。真正实现与虚拟世界的实时交互可能已经不太远了。

10.3.2 整个场景的建模

一旦对场景中的物体进行了充分的描述和数字编码，它们就都被赋予了场景内的位置、大小和朝向。然后，这些信息的集合会被链接起来，形成一个称为**场景图**（scene graph）的数据结构。此外，场景图还包含到表示光源的特殊物体及表示摄像机的特定物体的链接。在这里摄像机的位置、朝向和焦点等属性被记录下来。

因此，场景图类似于传统摄影中的工作室布置，它包括摄像机、灯光、道具和背景布景（当按下快门时，所有的东西都将对照片的外观产生影响），它们全都在适当的位置上。不同的是，传统摄影布置包含的是真实的物体，而场景图包含的是物体的数字编码表示。简而言之，场景图描述的是一个虚拟世界。

计算纤维

出于娱乐目的，对计算机生成图像的真实感的需求驱动了计算机建模在布、毛和头发纤维方面的进步。例如，注意下图中蓝精灵的帽子和服装的织物细节，以及蓝妹妹的金色头发的真实感。这种对逼真的数字细节的关注现在已经司空见惯了，即使是应用于眉毛伸展到其帽子上的明显虚构的卡通人物，也是如此。



蓝精灵：失落的村庄（2017），索尼影视动画

图片来源：Alamy/Alamy Stock Photo



图片来源：Lifstyle pictures/Alamy Stock Photo

蓝精灵周围禁忌森林繁茂的细节需要一个计算密集的场景图。在一些场景中，可以看到60多棵树，还有许多其他种类的植被。一些树木模型由大约40 000片叶子组成，每片叶子有60 ~ 90个多边形。其他植被元素，如无处不在的草甸排草蔓，是程序生成的地被植物。

摄像机在场景中的位置会产生许多影响。如前所述，用于物体建模的细节取决于物体在场景中的位置。前景物体需要的细节比背景物体需要的细节多，前景和背景的区别依赖于摄像机的位置。如果使用的场景环境类似于戏剧舞台布景，那么前景和背景就很好区分，物体模型也能被相应地构建。但是，如果环境要求摄像机的位置要根据不同的图像而改变，那么由物体模型提供的细节就需要在“照片”间进行调整，这是当前研究的一个领域。一种假设是，场景由“智能”模型构成，当摄像机在场景中移动时，这些模型改进了它们的多边形网格和其他特性。每个物体的模型所需的抽象级别取决于底层物体及其在场景中的语境。

基本知识点

- 模型可以根据要呈现的物体或现象使用不同的抽象或不同的抽象层次。

移动摄像机情景的一个有趣的例子出现在虚拟现实系统中，人类用户可以借助它来体验在虚构的三维世界里走来走去的感觉。虚构世界用场景图表示，人通过在由人的移动指定的场景中移动的摄像机来观察这个环境。实际上，为了提供三维的深度感知用了两部摄像机：一部表示人的右眼，另一部表示人的左眼。通过显示由每只眼睛前的摄像机得到的图像，人们能产生身在三维场景中的错觉，当在体验中增加音频和触觉时，这种错觉会变得十分逼真。

最后，我们应该注意到，场景图的构建在3D图形处理中有举足轻重的位置。因为它包含制作最终图像所需的所有信息，它的完成标志着艺术建模过程的结束和渲染图像的计算密集型过程的开始。实际上，场景图一旦构建好，图形学的任务就变成了计算投影、确定特定点的表面细节和模拟光效——这些任务在很大程度上与特定应用无关。

3D电视

现在有一些技术能够在电视中生成3D影像，但都依赖于同一种立体视觉效果，即对于左右眼看到的两个稍稍不同的图像，大脑可以依此判断出深度。就这方面而言，现今最便宜的装置需要具有滤光镜。较老的彩色镜片（20世纪50年代用于电影院）或更现代的偏光镜片会过滤掉屏幕上同一图像的不同方面，从而使左右眼看到不同的图像。较昂贵的技术涉及“主动式”眼镜，它与快速切换左右图像的3D电视同步地交替开关左右镜片。最后，不需要特殊眼镜或头戴式设备的3D电视正在开发中。它们在屏幕表面精心排放一组滤光镜或放大镜，从而

将左右图像以稍稍不同的角度投射给观看者从而使观看者的左眼和右眼看到不同的图像。

10.3节问题与练习

1. 下面是表示平面片顶点的4个点（是用传统的直角坐标系编码的）。请描述面片的形状。（对于没有解析几何背景知识的人，这里解释一下。每个三元组表示的是如何从房间中的一个角落开始到达问题中的那个点。第一个数表示沿着地板和位于你右边的墙之间的接缝走多远；第二个数表示沿着与位于你左边的墙平行的方向上向房间里走多远；第三个数表示从地板向上爬多高。如果有一个数是负数，你就不得不假装你是一个幽灵，可以穿过墙和地板。）

$(0, 0, 0)$ $(0, 1, 1)$ $(0, 2, 1)$ $(0, 1, 0)$

2. 什么是程序化模型？
3. 列出一些在用于制作公园图像的场景图中可能会出现的物体。
4. 既然可以用几何方程更精确地表示形状，为什么还要用多边形网格来表示？
5. 什么是纹理映射？

10.4 渲染

现在让我们考虑渲染过程，它决定了场景图中的物体投影到投影平面时将如何显示。有几种方法可以完成渲染任务。本节着重介绍当

今消费者市场上大多数较流行的图形系统（视频游戏、家用计算机等）使用的传统方法。10.5节将研究这种方法的其他两个可供选择的解决方案。

我们从一些关于光和物体之间相互作用的背景信息开始。毕竟，物体的外观是由从物体发出的光决定的，因此确定物体的外观这一任务最终变成了对光的行为的模拟。

10.4.1 光 - 表面相互作用

根据物体的材料属性，照射到物体表面的光可能会被吸收，被表面反射成反射光，或者穿过表面（被弯曲）形成折射光。

1 . 反射

让我们考虑从一个平坦不透明的表面反射的光线。光线沿直线传播，以一个角度照射到表面上，这个角度称为**入射角**（incidence angle）。光线的反射角与入射角相同，如图10-5所示，这些角度是相对于垂直（或**正交**）于表面的线来测量的。（正交于表面的线经常简称为“法线”，如在“入射角是相对于法线度量的”中。）入射光线、反射光线和法线全都在同一平面中。

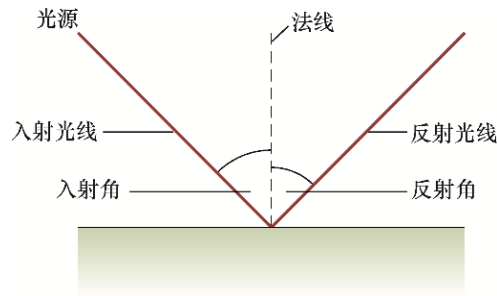


图10-5 反射光

如果表面是光滑的，在同一区域照射到表面的平行光线（如那些来自同一光源的光线）就会以基本相同的方向反射，并作为平行光线远离物体。这种反射光称为**镜面光**（specular light）。注意，只有当表面和光源的朝向使光沿观察者的方向反射时，镜面光才能被观察到。因此，它通常显示为表面上明亮的高亮区。此外，因为镜面光与表面的接触时间最短，所以它非常接近原始光源的颜色。

但是，物体表面很少是绝对光滑的，因此许多光线在表面照射点的朝向会与大多数普通表面的不同。此外，光线经常穿透表面的直接边界，在表面的粒子间跳弹，最后作为反射光线离开。结果是许多光线将向不同的方向散开。这种散开的光称为**漫射光**（diffuse light）。与镜面光不同，漫射光在一定范围的方向内是可见的。此外，因为它倾向于延长与表面接触的时间，更易受材料吸收特性的影响，因此它更接近物体的颜色。

图10-6展示了一个被单个光源照射的球。球上明亮的高亮区是镜面光产生的。通过漫射光，可以看见面向光源的半球的其他部分。注意，球面背着主光源的半球无法通过直接反射光源而被看见。球的这部分能够被看见是由于**环境光**（ambient light）的存在，环境光是与任

何特定光源或方向无关的“漂泊的”或发散的光。被环境光照射的表面部分往往显示为均匀的深色。

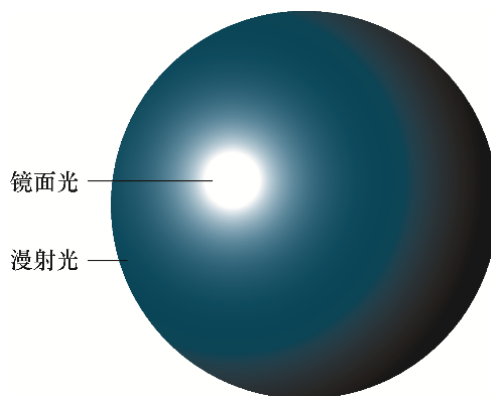


图10-6 镜面光与漫射光

大多数表面既反射镜面光又反射漫射光。表面的特点决定了镜面光和漫射光的比例。平滑表面看起来发亮是因为它们反射的镜面光多于漫射光。粗糙表面看起来灰暗是因为它们反射的漫射光多于镜面光。此外，由于某些表面具有细微的特性，入射光的方向不同，镜面反射光和散射光的比例也会有所不同，从一个方向入射的光照射到这种表面上可能反射的主要是镜面光，而从另一个方向入射的光照射到这种表面上的光可能反射的主要为漫射光。因此，当表面旋转时，它的外观将从明亮变化为灰暗。这种表面称为**各向异性表面**（anisotropic surface），与各向异性表面相对的是**各向同性表面**（isotropic surface），后者的反射模式是对称的。各向异性表面的例子可以在织物（如缎子）中找到，其中织物的细毛根据它的朝向改变材料的外观。另外一个例子是运动场的草地表面，那里草的排列方向（通常是由草被裁剪的方式决定的）产生了各向异性视觉效果，如同明暗相间的条纹图案。

2 . 折射

现在考虑光照射到透明的物体上，而非不透明的物体上。在这种情况下，光线是穿过物体而非从其表面反射的。当光线穿透表面时，它们的方向改变了，这种现象称为**折射**（refraction），如图10-7所示。折射程度是由所涉及材料的折射率决定的。折射率与材料的密度有关。高密度材料的折射率往往比低密度材料的折射率高。当光线进入折射率较高的材料中时（如从空气进入水中），会在入射点处向靠近法线的方向弯曲；当光线进入折射率较低的材料中时，会向远离法线的方向弯曲。

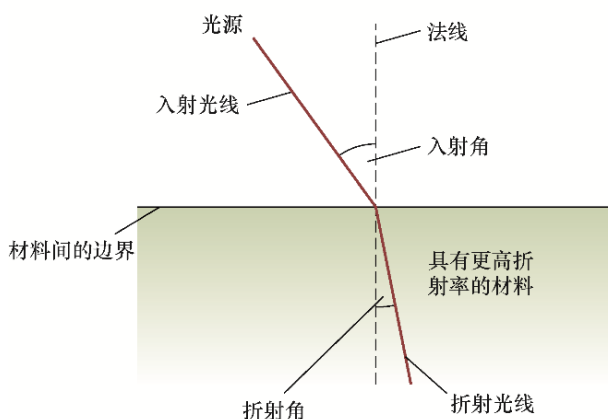


图10-7 折射光

为了准确地渲染透明物体，渲染软件必须知道所涉及材料的折射率。但这还不够，渲染软件还必须知道物体表面的哪一面表示物体的里面，而另一边为外面。光线是进入物体还是离开物体？获得这些信息的技术有时相当巧妙。例如，如果我们约定，从外部观察物体时，总是按逆时针顺序将多边形网格中每个多边形的顶点依次存放在一个

列表中，那么通过给出的列表我们可以很巧妙地编码面片的哪一边表示物体的外面。

10.4.2 裁剪、扫描转换和隐藏面消除

现在我们着重考虑一下从场景图生成图像的过程。我们目前使用的方法正是在大多数交互式视频游戏系统中使用的技术。综合应用这些技术形成了一个固定下来的范式，称为**渲染流水线**（rendering pipeline）。在本节的末尾我们将介绍这种方法的一些优缺点，在下一节我们将探讨两种备选方法。现在，注意到渲染流水线涉及的是不透明物体因而折射就不是问题，是有帮助的。此外，它忽略了两个物体之间的相互影响，因此目前我们不必担心镜像和阴影问题。

渲染流水线首先要确定包含摄像机能“看到”的物体（或物体的一部分）的三维场景中的区域。这个区域称为**视体**（view volume），是棱锥内的一个空间，这个棱锥是由从投影中心出发向图像窗口每个角延伸的直线定义的（见图10-8）。

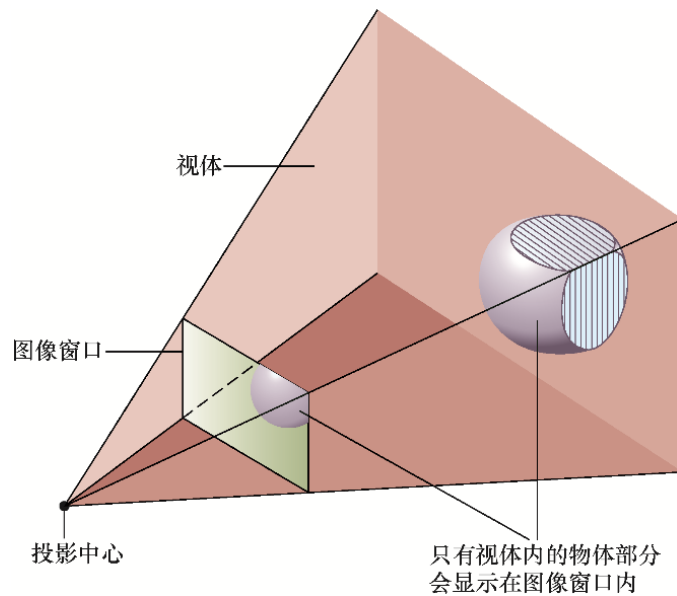


图10-8 确定视体里面的场景区域

一旦视体被确定下来，就不用考虑那些与视体不相交的物体或物体部分了。毕竟，那部分场景的投影将落在图像窗口的外面，因此不会出现在最终图像中。第一步就是去除完全在视体外面的物体。为了简化这个过程，可以将场景图组织成一个树结构，其中处在场景不同区域的物体被存储在不同的分支上。因此，仅需忽略树中的整个分支，就可以去除场景图的大部分。

确定和去除那些与视体不相交的物体后，剩余的物体通过称为**裁剪**（clipping）的过程修剪，裁剪实际上就是去掉每个物体在视体外面的部分。更准确点说，裁剪首先把每个平面片与视体边界进行比较，然后去除平面片落在外面的部分，最终得到完全处在视体里面的多边形网格（可能是修剪过的平面片）。

渲染流水线的下一步是确定剩余平面片上要与最终图像中的像素位置相关联的点。重要的是认识到只有这些点会对最终图像有所贡献。如果物体上的细节落在两个像素位置的中间，那它就不能用像素来表示，因此在最终图像中是不可见的。这就是像素数在数字摄像机市场被着重宣传的原因。像素越高，小的细节就越有可能在照片中被捕捉到。

图形走样

你有注意到在电视屏幕上条纹衬衫和条纹领带会出现奇怪的“闪烁”现象吗？这是由一种称为**图形走样**（aliasing）的现象导致的结果，当期望的图像网格中的模式与组成图像的像素密度不匹配时，就会出现图形走样现象。例如，假设期望的图像的一部分由黑白相间的条纹组成，但是所有像素的中心碰巧都落在了黑色条纹上，那么物体将被当作全黑色的来渲染。但是，如果物体稍微移动一下，所有像素的中心可能都落在了白色条纹上，那么物体就会突然变成白色的。有许多种方法可以改善这种令人心烦的效果，一种就是使用图像中一个小区域的平均值而不是精确的单个点来渲染每一个像素。

像素位置与场景中的点相关联的过程称为**扫描转换**（scan conversion）（因为它涉及把面片转换为称为扫描线的水平像素行）或**光栅化**（rasterization）（因为一个像素阵列称为一个光栅）。用扫描转换是这样实现的：通过将直线（投影线）从投影中心延伸到图像窗口中的每个像素位置，然后找到这些投影线与平面片相交的点；最

后，根据这些点我们得到物体在图像上的外观。实际上，这些点在最终图像中是由像素表示的。

图10-9描述了单个三角面片的扫描转换。图10-9a显示了如何通过投影线实现一个像素位置与面片上一个点的关联，图10-9b显示了通过扫描转换得到的面片的像素图像。整个像素阵列（光栅）用网格表示，与三角形相关的像素已经被着色。注意，这个图还说明了扫描转换的形状的特征点小于像素尺寸时会产生失真。大多数个人计算机用户会经常在显示屏上看到这种锯齿状的边缘。

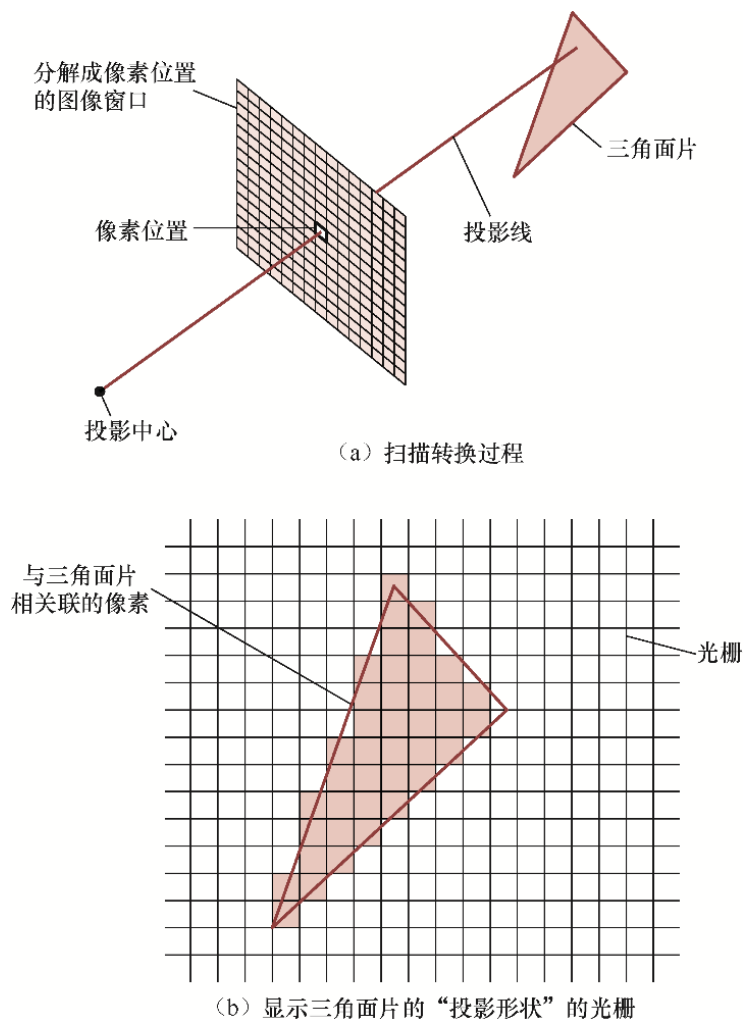


图10-9 三角面片的扫描转换

遗憾的是，整个场景（乃至单个物体）的扫描转换不像扫描转换单个面片那样直观明了。这是因为，当涉及多个面片时，一个面片可能会遮住另一个面片。这样，即使投影线与平面片相交，面片上的这个点在最终图像上也不一定可见。识别和去除场景中被遮挡的点的过程称为**隐藏面消除**（hidden-surface removal）。

隐藏面消除的一个具体版本是**背面消除**（back face elimination），也就是不考虑那些在多边形网格中表示物体“后面”的面片。注意，背面消除相对简单，因为可以认为那些朝向背着摄像机的面片是处在物体后面的。

但是，解决整个隐藏面消除问题需要的不仅仅是背面消除。例如，想象一辆汽车在大楼前的场景，来自汽车和大楼的平面片将会投影到图像窗口的相同区域。在重叠发生的地方，最终存储在帧缓冲区中的像素数据对应的是前景物体（汽车）的外观，而不是背景物体（大楼）的外观。简而言之，如果投影线与多个平面片相交，那么最靠近图像窗口的面片上的点应该被渲染。

解决这个“前景/背景”问题的一种简单方法称为**画家算法**（painter's algorithm），就是根据物体与摄像机的距离在场景中依次放置物体，然后先扫描转换较远的物体，允许较近物体的扫描转换结果覆盖先前的任何结果。遗憾的是，画家算法不能处理物体交织在一起的情况。例如，树的一部分可能在一个物体的后面，而另一部分可能在这个物体的前面。对于“前景/背景”问题的更彻底的解决方案是聚

焦单个像素，而不是整个物体。一种流行的技术是使用称为**z缓冲区**（z-buffer，也称深度缓冲区）的额外存储区域，z缓冲区中包含图像中每个像素的条目（也可以说是帧缓冲区中的每个像素条目）。z缓冲区中的每个位置被用来存储沿着相应的投影线从摄像机到帧缓冲区中相应条目当前表示的物体间的距离。因此，在z缓冲区的帮助下，要解决“前景/背景”问题，必须在还没将像素数据放在帧缓冲区中或者当前观察的物体上的点比先前渲染的物体上的点更近时（这是由z缓冲区记录的距离信息决定的），计算和存储像素的外观。

更准确地说，当使用z缓冲区时，渲染过程可以按如下步骤进行：为z缓冲区中的所有条目设置一个值，表示从摄像机到要渲染的物体的最大距离。然后，每次考虑渲染平面片上的任何一个新点时，首先将其与相机间的距离和z缓冲区中关联当前像素位置的值进行比较。如果距离小于z缓冲区中的值，则计算点的外观，在帧缓冲区中记录结果，用到刚渲染的点的距离替换z缓冲区中的旧条目。（注意，如果到这个点的距离大于在z缓冲区中找到的值，则不需要采取任何行动，因为该面片上的点太远不用考虑，或者它被已经渲染的更近的点遮住了。）

总之，因为最后的渲染计算成本很高，所以我们先采用各种技术丢弃不影响最后结果的物体模型细节。这是一个一般策略，在计算机图形学以外的许多建模和模拟问题中也有效。

基本知识点

- 模型经常忽略正在建模的物体或现象的不必要特征。

10.4.3 着色

一旦扫描转换确定了要显示在最终图像中的平面片上的点，渲染任务就变成了确定这个点所在面片的外观。这个过程称为**着色**（shading）。注意，着色涉及计算从相应的点投影到摄像机的光的特征，反过来，它又取决于该点表面的朝向。毕竟，是该点表面的朝向决定了镜面光、漫射光和环境光被摄像机拍摄的效果。

平面着色（flat shading）是解决着色问题的一个简单方法，它是把平面片的朝向作为其上每个点的朝向，也就是说，假设每个面片上的表面都是平坦的。但是，结果是最终图像将显示为图10-10中那样有棱角的，而不像图10-6中显示的那样圆。从某种意义上讲，平面着色生成的图像是多边形网格本身，而不是用网格建模的物体。

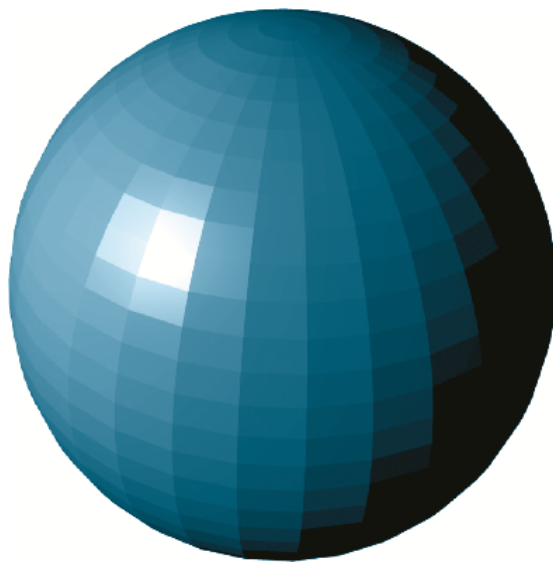


图10-10 用平面着色渲染时可能出现的球体

为了生成更加逼真的图像，渲染过程必须将每个平面片的外观融合到平滑曲面显示的表面。这是通过估算每个渲染点出原始表面的真实朝向来完成的。

这种估算模式通常始于指示多边形网格顶点处表面朝向的数据。获得此数据有多种方法。一种方法是编码每个顶点处原始表面的朝向，并把这个数据附着在多边形网格上，作为建模过程的一部分。这会产生一个带箭头的多边形网格，其中箭头称为**法向量**（normal vector），这些箭头附着在每个顶点上。每个法向量沿垂直于原始表面的朝向指向外部。这样的多边形网格可以想象成图10-11所示的样子。（另一种方法是计算与顶点相邻的每个面片的朝向，然后使用这些朝向的“平均值”来估计顶点表面的朝向。）

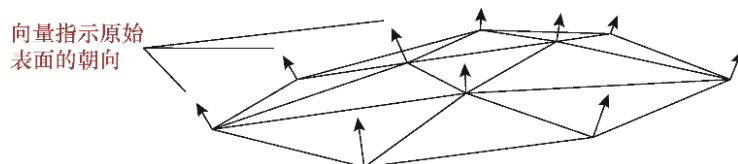


图10-11 带有顶点法向量的多边形网格的概念视图

不管多边形网格顶点的原始表面朝向是如何获得的，都可以用几种策略来对基于这些数据的平面片进行着色。这些策略包括**高洛德着色**（Gouraud shading）和**冯氏着色**（Phong shading）。它们之间的区别是微妙的。二者首先都用面片顶点处的表面朝向信息来粗略估计沿着面片边界的表面朝向。然后，高洛德着色用这些信息确定沿面片边界的表面的外观，最后，插入这个边界外观来估算面片内部的点处的表面的外观。与之相比，冯氏着色则是沿面片边界的表面的朝向插入

估算面片内部的点处的表面朝向，然后只考虑外观的问题。（简而言之，高洛德着色将朝向信息转换为颜色信息，然后插入颜色信息，而冯氏着色插入朝向信息，直至估计出所考虑点的朝向，然后将朝向信息转换成颜色信息。）结果是冯氏着色更有可能检测到面片内的镜面光，因为它对表面朝向的变化更敏感。（见本节结尾的问题与练习3。）

最后，应该注意到可以扩充基本着色技术，为表面添加纹理外观。例如，称为**凹凸映射**（bump mapping）的方法，实际上就是把生成的表面外观朝向加上一个小的变量，这样表面看起来就是粗糙的。更准确地说，凹凸映射在传统着色算法所使用的插值算法中加了一个自由度，这样整个表面看起来是有纹理的，如图10-12所示。

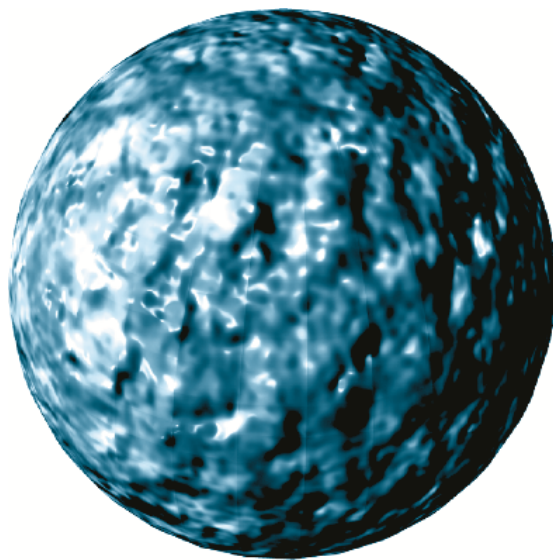


图10-12 使用凹凸映射渲染时可能出现的球体

10.4.4 渲染 - 流水线硬件

我们已经说过，渲染流水线就是依次执行裁剪、扫描转换、隐藏面消除和着色处理这些操作。此外，执行这些任务的高效算法是众所周知的，并且已经直接在电子电路中实现，通过超大规模集成电路（VLSI）技术已经被微缩成芯片，自动完成整个渲染流水线。如今，即使是低廉的产品也具备每秒渲染几百万个平面片的能力。

大多数为图形应用（包括视频游戏机）设计的计算机系统都在它们的设计中加入了这些设备。在更通用的计算机系统中，可以以**图形卡**（graphics card）或**图形适配器**（graphics adapter）的形式加入这种技术，图形卡或图形适配器是作为专用控制器连接到计算机总线上的（见第2章）。这样的硬件大大减少了执行渲染过程所需的时间。渲染流水线硬件还降低了图形应用程序的复杂度。从本质上讲，所有软件需要做的就是给图形硬件提供场景图，然后硬件执行流水线步骤，并把结果放在帧缓冲区中。因此，从软件角度来看，整个渲染流水线被简化为一个步骤，将硬件用作抽象工具。

我们再次以交互式视频游戏为例。为了初始化游戏，游戏软件把场景图传送给图形硬件，然后硬件渲染场景，把图像放在帧缓冲区中，在这里图像被自动显示在监视屏上。随着游戏的进行，游戏软件只需更新图形硬件中的场景图，反映出不断改变的游戏情景，而硬件反复渲染场景，每次都更新过的图像放在帧缓冲区中。

许多图形卡上的现代**图形处理器单元**（graphical processor unit，GPU）现在就是这种优秀的抽象工具。只要它们可以用类似的渲染计算方式来表达，就可以用于计算其他类型的密集模拟。当不用GPU实

时渲染屏幕输出时，可以用它计算一般数字结果。渲染一个复杂图形输出或数值模拟所需的时间不仅取决于底层模型的复杂度，还取决于所用的硬件和软件系统。

基本知识点

- 模拟需要的时间受模型的细节和质量的层次以及用于模拟的硬件的影响。

但是应该注意到，不同图形硬件的处理能力和通信特性是完全不同的。这样，如果像视频游戏这样的应用是针对特定图形平台开发的，那么在将它转移到另一个环境中时，就不得不对其进行修改。为了减少对特定图形系统的依赖，人们已经开发了在图形硬件和应用软件之间起中介作用的标准软件接口。这些接口由把标准化命令转换为特殊图形硬件系统需要的特定指令的软件例程组成。这类接口的示例包括**开放图形库**（open graphics library，OpenGL）和**Direct3D**。OpenGL是由Silicon Graphics开发的非专有系统，被广泛用于视频游戏行业；**Direct3D**是微软为微软Windows环境开发的。

最后应该注意到，虽然与渲染流水线关联的优点很多，但它也有缺点，其中最显著的就是流水线只实现了局部光照模型，这意味着流水线渲染的每个物体是独立于其他物体的。也就是说，在局部光照模型下，每个物体都是相对于光源被渲染的，仿佛它是场景中的唯一物体。结果是，没有捕获到两个物体之间相互作用的光（如阴影和反射）。与之相对，全局光照模型考虑了物体间相互作用。在10.5节中

我们将讨论两种实现全局光照模型的技术。目前我们只需注意，这些技术超出了当前技术的实时能力。

但是，这并不意味着使用渲染流水线硬件的系统不能够产生一些全局光照效果。实际上，人们已经开发出了巧妙的技术，用来克服局部光照模型强加的一些限制。特别是，可以在局部光照模型的环境里模拟**阴影**（drop shadow）的外观（是投在地上的阴影），方法是制作投射阴影物体的多边形网格的副本，把网格副本变成平坦的，然后放在地面上，涂以黑色。换句话说，把阴影成另一个物体，可以用传统的渲染流水线硬件来渲染，产生阴影的幻觉。这种技术在“大众水平”应用（如交互式视频游戏）和“专业水平”应用（如飞行模拟）中都很流行。

10.4节问题与练习

- 1．总结镜面光、漫射光和环境光之间的区别。
 - 2．定义术语**裁剪**和**扫描转换**。
 - 3．高洛德着色和冯氏着色可以总结为：高洛德着色使用沿着面片边界的物体表面的朝向确定沿边界的表面的外观，然后把这些外观插值到面片的内部，确定所考虑的特定点的的外观；冯氏着色通过对边界朝向的插值计算面片内部的点的朝向，然后使用这些朝向确定所考虑的特定点的的外观。物体的外观会有什么不同？
 - 4．渲染流水线的意义是什么？
 - 5．描述如何用局部光照模型来模拟镜子中的反射。
-

*10.5 处理全局光照

研究人员目前正在研究渲染流水线的两种替代方案，这两种方案都实现了全局光照模型，因此都提供了克服传统流水线固有的局部光照模型局限性的潜力。其中一种方案是光线跟踪，另一种方案是辐射度。这两种方案都是极为精确且费时的，这一点我们即将看到。

10.5.1 光线跟踪

光线跟踪（ray tracing）本质上是沿着光线进行反方向跟踪以找到它的光源的过程。这个过程首先选择要渲染的像素，确定经过这个像素和投影中心的直线，然后跟踪沿着这条直线射入图像窗口的光线。这个跟踪过程沿着此直线进入场景，直至它接触到一个物体。如果这个物体是光源，则终止光线跟踪过程，将像素渲染为光源上的一个点；否则，计算物体表面的特性，以此确定入射光的方向，该光线被反射产生了该过程正在追溯的光线。然后继续跟踪入射光线反向找到它的光源，在这个点上可能还会有另一条需要识别和跟踪的光线。

图10-13给出了一个光线跟踪的例子，在图中我们看到被跟踪的光线向后穿过图像窗口，到达镜子的表面。从这里跟踪光线到一个有光泽的球，再向后到镜子，然后从镜子到光源。基于从这个跟踪过程中获得的信息，图像中的像素应该显示为镜子中反射的光源照亮的球上的一点。

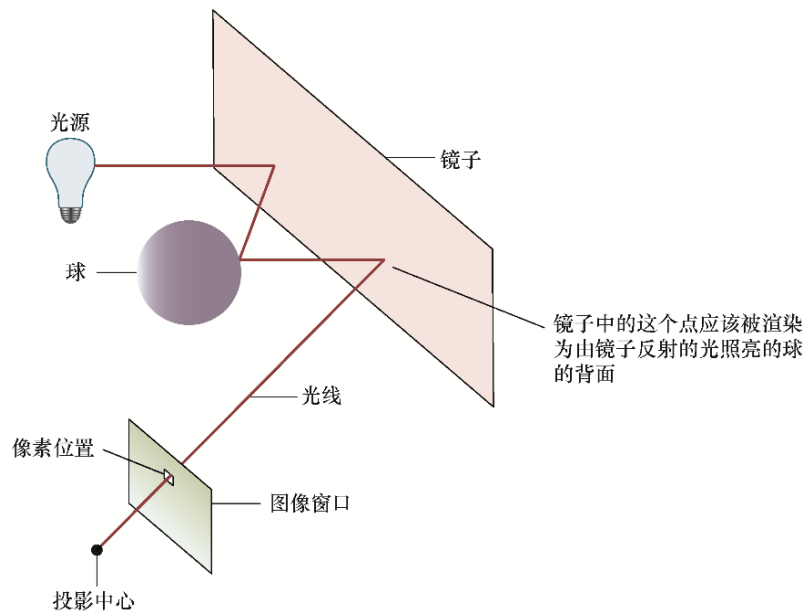


图10-13 光线跟踪

光线跟踪的一个缺点是它仅跟踪镜面光。因此，通过这种方法渲染的所有物体往往都具有有光泽的外观。为了抵消这个影响，可以应用光线跟踪的一个变种——**分布式光线跟踪**（distributed ray tracing）。分布式光线跟踪与光线跟踪的区别在于，它不是从反射点向后跟踪单条光线，而是同时跟踪从这个点出发的多条光线，每条光线的延伸方向略有不同。

当涉及透明的物体时，可应用基本光线跟踪的另一个变体。在这种情况下，每当向后跟踪光线至表面时，必须考虑两种效果，一种是反射，另一种是折射。在这种情况下，跟踪原始光的任务分成了两个：向后跟踪反射光和向后跟踪折射光。

光线跟踪通常是递归实现的，每次递归都被要求跟踪光线到它的源头。第一次递归可能跟踪它的光线至一个有光泽的不透明表面。在

这一点上，此光线可能被识别为一条入射光线的反射，计算这个入射光的方向，并调用另一次递归去跟踪这个入射光线。第二次递归将执行一个类似的任务，去查找它的光线的源头。这个过程可能还会递归调用。

可以用各种条件来终止递归光线跟踪。被跟踪的光线与光源相交，被跟踪的光线可能存在于不与物体相交的场景中，或者是递归次数达到了预定的限制。还有一种终止条件是基于所遇到的表面的吸收特性的，如果表面是高吸收性的（如灰暗不光滑的表面），那么任何入射光线几乎都不会对表面外观的产生影响，光线跟踪停止，累积的吸收会产生相似的效果。也就是说，访问多个适度可吸收性的表面后，光线跟踪可以终止。

正是基于全局光照模型，光线跟踪才避免了传统渲染流水线中许多固有的限制。例如，隐藏面消除问题和阴影检测问题通常可以在光线跟踪过程中自然地解决。遗憾的是，光线跟踪有一个很大的缺点，那就是费时。当跟踪每条反射光线至其源点时，需要的计算量会急剧增加——当把折射或者应用分布式光线跟踪考虑进去时问题就复杂了。因此，光线跟踪没有在“消费者级”实时系统（如交互式视频游戏）中实现，但在实时性要求不高的“专业水平”的应用（如电影工作室中使用的图像软件）中能找到它。

10.5.2 辐射度

另一种可以取代传统渲染流水线的方法是**辐射度**（radiosity）。光线跟踪通过跟踪单条光线，采用点到点的方法；而辐射度通过考虑两个平面片对之间的辐射光能的总和，采取更加区域化的方法。这个辐射的光能本质上就是漫射光。一个物体辐射的光能，可能是这个物体产生的（如光源这种情况），也可能是这个物体反射的。然后，每个物体的外观都要通过考虑从其他物体接收的光能来确定。

一个物体辐射的光对另一个物体外观的影响程度是由称为**形状因子**（form factor）的参数来确定的。场景中要渲染的每对面片都关联唯一的形状因子。这些形状因子考虑所涉及的两个面片之间的几何关系，如间距和相对朝向。为了确定场景中一个面片的外观，要对每个计算使用一个合适的形状因子来计算从场景中所有其他面片接收的光能总量。然后将结果结合起来，产生每个面片的单个颜色和强度。最后使用类似于高洛德着色的技术，将这些值插入相邻的面片之间，从而获得光滑没有棱角的表面。

因为要考虑许多面片，所以辐射度的计算量非常大。因此，就像光线跟踪一样，辐射度的应用无法满足当前消费市场上图形系统的实时要求。辐射度的另一个问题是，由于它处理的单元包含的是整个面片，而不是单个点，所以它无法捕获镜面光的细节，这意味着用辐射度渲染的所有表面倾向于有灰暗的外观。

但是，辐射度的确有它的优点。一个优点是使用辐射度确定物体的外观是不受摄像机影响的。这样，一旦计算出一个场景的辐射度，场景不同摄像机位置的渲染就能快速完成。另一个优点是辐射度捕获

了光的许多细微特征，如**渗色**（color bleeding），即一个物体的颜色影响其周围其他物体的色调。因此，辐射度有它的独特之处。其中之一就是用在建筑设计设置的图形软件中。实际上，拟议建筑内的光主要是由漫射光和环境光组成的，因此镜面光的影响不大，并且新的摄像机位置可以得到有效处理，这一事实意味着建筑师能很快从不同的视角看到不同的房间。

10.5 节问题与练习

1. 为什么光线跟踪是沿着光线向后（从图像窗口到光源），而不是向前（从光源到图像窗口）？
 2. 简单的光线跟踪和分布式光线跟踪的区别是什么？
 3. 辐射度的两个缺点是什么？
 4. 光线跟踪和辐射度在哪些方面是相似的？在哪些方面是不同的？
-

10.6 动画

现在让我们进入**计算机成像**（computer-generated imagery，CGI）动画这个主题。计算机成像就是利用计算机技术来制作和显示呈现运动的图像。计算机动画的相对便宜但功能强大的软件工具使创建者可以构建比使用可能节省成本的传统手绘动画技术更精确、更复杂的视觉艺术。CGI动画正变得越来越流行，这不仅是因为潜在成本较低，

还因为这种更强大的工具可以达成更雄心勃勃、更新颖的故事的目标。

基本知识点

- 计算有助于创建和修改计算制品。计算促进了计算制品的创建和修改，并提高了细节和精度。
- 计算作为一种使能技术，其发展产生和提升了其他领域的创造力。

10.6.1 动画基础

我们先介绍一些基本的动画概念。

1 . 帧

动画是通过快速连续地显示一系列称为**帧**（frame）的图像来实现的。这些帧以固定时间间隔捕获变化的场景的外观，因此，帧的顺序展示就产生了一直观看连续场景的错觉。电影业的标准显示速率是每秒24帧。广播视频的标准是每秒60帧（尽管每隔一帧的视频帧被设计为与前一帧交织来产生完整详细的图像，但是视频也可以被归类为每秒30帧的系统）。

帧可以用传统摄影制作，也可以用计算机图形学人工生成。而且，这两种技术还可以结合起来应用。例如，2D图形软件经常用于修

改通过摄影获得的图像，消去支撑钢丝的外观、叠加图像和创建**变形**（morphing）的幻象（变形就是一个物体看起来要变成另一个物体的过程）。

仔细观察变形可以深入了解动画过程。构建变形效果首先要确定一对包含变形序列的关键帧：一个是变形出现之前的最后一个图像，另一个是变形出现之后的第一个图像。（在传统的电影制作中，这需要“拍摄”两个动作序列：一个在变形出现之前，另一个在变形出现之后。）然后，将变形前的帧的特征，如称为**控制点**（control point）的点和线，与变形后的帧的相似特征相关联，并通过应用数学技术同时使用控制点作为向导将一个图像逐渐变形为另一个图像来构造变形。通过记录变形过程中所产生的图像得到一个短的人工生成的图像序列，把它填充到当初定义的那两个关键帧之间，就创建出了变形幻象。

Kineograph

Kineograph是一本动画帧的书，通过快速翻动书页来模拟运动。读完这部分内容，你就能制作自己的Kineograph（假设你还没有在书的空白处填满涂鸦）。在第一页的空白处放置一个圆点，然后根据第三页上的印记位置，在第三页位置上放置另一个位置与第一页略有不同的圆点。在每个连续的奇数页上重复这个过程，直到你到达书的末尾。现在，快速翻动书页，观察点的跳动情况。一转眼，你已经制作了自己的Kineograph，这也许是你迈向动画事业的第一步！作为运动学的一个实验，你可以试着画一个粗线条人物代替简单的圆点，使粗

线条人物看起来像在走路。现在，你也可以通过制作水滴撞击地面的图像来进行动力学实验。

2 . 故事板

一个典型的动画项目从创建**故事板**（storyboard）开始。故事板是一个二维图像序列，用关键点处显示的场景草图的形式讲述一个完整的故事。故事板的最终角色取决于动画项目是用2D技术实现，还是用3D技术实现。在使用2D图形的项目中，故事板一般会演变为帧中的最终场景，与20世纪20年代迪士尼工作室所做的大致相同。那时候，艺术家（称为主动画师）会把故事板细化为称为**关键帧**（key frame）的详细帧，建立动画固定时间间隔的角色和场景的外观。然后，助理动画师将绘制额外的帧来填充两个关键帧之间的间隔，以使动画看起来连续且平滑。这个填充间隔的过程称为**中间渐变**（in-betweening）。

这一过程与如今使用的过程的主要区别是，动画师现在使用图像处理 and 2D图形软件绘制关键帧，而且大量的中间渐变过程已经实现自动化，因此助理动画师这个角色已基本消失。

模 糊

在传统摄影领域，人们在生成快速运动物体的清晰图像方面做出了大量的努力。在动画领域，却出现了相反的问题。如果描述运动物体的序列帧中的每一帧都把物体渲染成清晰的图像，那么运动可能显得急促不流畅。然而，清晰的图像是将帧创建为场景图中静止物体的单独图像的自然副产品。因此，动画师经常需要手动扭曲计算机生成

的帧中的运动物体的图像。有一种技术称为**超取样**（supersampling），它会产生多幅图像，其中运动物体只被稍微移动，然后覆盖这些图像产生单个帧。另一种技术是改变运动物体的形状，使它看起来像是沿着运动方向被拉长的。

3 . 3D动画

大多数视频游戏动画和功能完备的动画制作现在是使用3D图形创建的。在这些情况下，项目仍旧从创建由二维图像组成的故事板开始。但是，与发展成为2D图形项目的最终产品不同，故事板被用作构建三维虚拟世界的蓝图。当其中的物体按照脚本或视频游戏的进展进行的移动时，这个虚拟世界将反复地被“拍摄”。

也许我们应该停下来，弄清楚物体在计算机生成的场景内移动的含义。记住，“物体”实际上是存储在场景图中的数据集合。这个集合中的数据是一些指示物体位置和朝向的值。因此，只要改变这些值就能完成物体的“移动”。一旦完成这些改变，新值就会被用在渲染过程中。结果，物体看起来像是在最终的二维图像中已经移动。

10.6.2 运动学和动力学

3D图形场景中的运动究竟是自动的，还是受动画师控制的，这个自动化的程度随应用的不同而变得不尽相同。当然，目标是整个过程的自动化。为了实现这个目标，许多研究已经直接面向找寻能识别和

模拟自然发生现象的运动的方法。机械学中的两个领域被证明在这一点上特别有用。

一个机械学分支是**动力学**（dynamics），它研究的是通过应用物理定律确定力作用于物体的效果来描述物体的运动。例如，除位置外，场景中的物体可能会被赋予运动的方向、速度和质量。然后使用这些项来确定重力或与其他物体的碰撞对物体产生的影响，让软件可以计算下一帧中物体的合理位置。

作为一个例子，我们考虑构建一个描述容器中水的晃动的动画序列的任务。在场景图中我们可以使用粒子系统来表示水，每个粒子表示一个小单位的水。（把水看成是由弹珠大小的大“分子”构成的。）那么，当容器从一边晃到另一边时，我们就可以应用物理定律来计算重力对粒子的影响，以及粒子间的相互作用。这就让我们可以在固定时间间隔计算每个粒子的位置，并且通过使用外层粒子的位置作为多边形网格的顶点，我们可以获得描绘水的表面的网格。于是，模拟过程中反复“拍摄”这个网格就得到了动画。

用来模拟运动的另一个机械学分支是**运动学**（kinematics），它研究的是根据物体各部分间的相对运动方式来描述物体的运动。当需要通过移动手臂和腿之类的附属物来模拟有关节的角色时，运动学的应用很突出。与计算肌肉和重力施加的力的影响相比，通过模拟关节运动模式更容易对运动进行建模。因此，尽管在确定反弹球的路径时，动力学可能是可选的技术；但是动画角色的手臂的运动将使用运动学计算出合理的肩膀、肘和手腕的旋转来确定。因此，许多模拟生物特

性的研究集中在解剖学以及关节与附属物的结构影响运动的方式的问题。

应用运动学的一个典型方法是，首先用粗线条人物画来表示角色，模拟要描述的角色骨骼结构；然后用一个表示围绕角色部位的角色外表的多边形网格覆盖角色的每个部分，并且建立相应的规则来确定相邻的网格相互连接的方式。通过重新定位关节在骨骼结构中的位置，就能够操纵人物（通过软件或动画师来操纵），就像一个人操纵提线木偶时使用的方式一样。“线”连接到模型的点称为**关节变量**（avars），它是“articulation variables”（或最近提出的“animation variables”）的简写。

运动学应用的许多研究都是为了开发自动计算模拟自然运动的附属物位置序列的算法。沿着这些方向，生成逼真的行走序列的算法现在已经可用了。

但是，基于运动学的大量动画还是通过指示人物通过一个预设的关节附着物位置的序列来产生的。这些位置可能是由动画师创造性地给出的，也可能是由**运动捕捉**（motion capture）获得的，运动捕捉记录了生物模型在执行相应的动作时的位置。更准确地说，在人身体上的主要关节上应用反光带后，就可以在人投掷棒球的时候从多个角度拍摄人的动作。然后，通过观察各种照片中反光带的位置，就可以标识出人在进行投掷动作时手臂和腿的精确朝向，并将这些朝向传送给动画中的人物。

10.6.3 动画制作过程

动画研究的最终目标是自动化整个动画制作过程。想象一下，给定合理的参数，软件将自动生成所需的动画序列。电影业现在通过单个虚拟“机器人”来制作人群、战争场景和惊逃动物的图像，这些“机器人”在场景中自动移动，每个都执行分配给它的任务，这充分印证了人类在动画制作领域中取得的进步。可能需要数百或数千名临时演员才能在现实世界中拍摄的场景，可以被虚拟建模，其成本只有现实场景的一小部分，而且不会有真实的演员受到烟火或道具伤害。

基本知识点

- 模拟模仿真实世界的事件，而无须构建和测试现实世界现象的成本或风险。

在为《指环王》三部曲拍摄幻想的半兽人军队和人类军队时，出现了一个有趣的情况。每一个屏幕上的战士都被建模成不同的“智能”物体，它们都具有自己的体型特征和随机赋予的个性，这种个性赋予它攻击或者逃跑的倾向。在第二部圣盔谷之战的模拟测试中，半兽人的逃跑倾向设置得过高，它们刚遇到人类战士就会逃跑。（这也许是虚拟群众演员认为工作太危险的第一个实例。）

当然，如今许多动画还是由人类动画师制作的。但是，不同于20世纪20年代的手工绘制二维帧，如今动画师都使用软件操纵场景图中的三维虚拟物体，这种方式让人想起前面讨论运动学时介绍的控制提

线木偶的方式。用这种方式，动画师能够创建一系列的虚拟场景，这些场景然后被“拍摄”成动画。在某些情况下，这种技术只用于为关键帧制作场景，然后在软件应用动力学和运动学把场景图中的物体从一个关键帧场景位置移到下一个关键帧场景位置时，再使用软件通过自动渲染场景来产生中间渐变帧。

随着计算机图形学的进步以及技术的不断提高，肯定会有更多的动画制作过程变成自动化的。是否有一天人类动画师这一角色以及人类演员和实物布景会不复存在还有待观察，但许多人认为这一天不会太远。实际上，与把无声电影转化为有声电影相比，3D图形学对影视业的影响可能更为深远。

10.6 问题与练习

1. 人类看见的图像往往在人类的感知上有约200 ms的视觉暂留，基于这个近似值，每秒必须呈现多少幅图像才能产生动画？这个近似值是如何对应于电影中使用的每秒的帧数的？
2. 什么是故事板？
3. 什么是中间渐变？
4. 定义术语[运动学](#)和[动力学](#)。

复习题

1. 下列哪些是2D图形学的应用，哪些是3D图形学的应用？

- a . 设计杂志页面的布局。
- b . 用微软画图软件绘制图像。
- c . 为视频游戏制作虚拟世界的图像。

2 . 3D图形学中的哪些术语对应于下列传统摄影领域的术语？解释你的答案。

- a . 胶片。
- b . 取景器中的矩形。
- c . 要拍摄的场景。

3 . 当使用透视投影时，场景中的球体在什么条件下不会在投影平面上产生一个圆？

4 . 当使用透视投影时，直线段的图像能变成投影平面上的曲线段吗？请证明你的答案。

5 . 假设一根8英尺直杆的一端距离投影中心4英尺。此外，假设从投影中心到直杆一端的直线与投影平面相交于一点，这一点距离投影中心1英尺，如果直杆与投影平面平行，那么投影平面上直杆的图像长度是多少？

6 . 说明平行投影和透视投影之间的区别。

7. 说明图像窗口和帧缓冲区之间的关系。

8. 应用3D图形学产生电影和应用3D图形学产生交互式视频游戏的动画，二者之间有什么明显的不同？请解释你的答案。

9. 说出可能出现在3D图形场景使用的物体模型中的物体的一些属性。说出可能不会出现在物体模型中的物体的一些属性。请解释你的答案。

10. 说出没被只含一个多边形网格的模型捕获的物体的一些物理特性。（因此，一个多边形网格不能单独构成一个完整的物体模型。）请解释如何把这些物理特性中的一个添加到物体模型中。

11. 三维空间中的任意4个点能是多边形网格中一个面片的顶点吗？请解释你的答案。

12. 下面的每个集合表示多边形网格中一个面片的顶点（使用传统的直角坐标系）。请描述网格的形状。

面片1：(0, 0, 0) (0, 2, 0) (2, 2, 0) (2,0,0)

面片2：(0, 0, 0) (1, 1, 1) (2, 0, 0)

面片3：(2, 0, 0) (1, 1, 1) (2, 2, 0)

面片4：(2, 2, 0) (1, 1, 1) (0, 2, 0)

面片5：(0, 2, 0) (1, 1, 1) (0, 0, 0)

13. 下面的每个集合表示多边形网格中一个面片的顶点（使用传统的直角坐标系）。请描述网格的形状。

面片1： $(0, 0, 0)$ $(0, 4, 0)$ $(2, 4, 0)$ $(2, 0, 0)$

面片2： $(0, 0, 0)$ $(0, 4, 0)$ $(1, 4, 1)$ $(1, 0, 1)$

面片3： $(2, 0, 0)$ $(1, 0, 1)$ $(1, 4, 1)$ $(2, 4, 0)$

面片4： $(0, 0, 0)$ $(1, 0, 1)$ $(2, 0, 0)$

面片5： $(2, 4, 0)$ $(1, 4, 1)$ $(0, 4, 0)$

14. 设计一个表示长方体的多边形网格，使用传统的直角坐标系来编码顶点，画一幅草图表示你的解决方案。

15. 使用不超过8个三角面片，设计一个多边形网格，近似表示半径为1的球体形状。（只有8个面片，你的网格将是非常粗略的近似的球体，但是目标是让你理解什么是多边形网格，而不是让你生成球体的精确表示。）使用传统的直角坐标系表示你的面片的顶点，画一个网格的草图。

16. 为什么下面4个点不是一个平面片的顶点？

$(0, 0, 0)$ $(1, 0, 0)$

$(0, 1, 0)$ $(0, 0, 1)$

17 . 假设顶点 $(1, 0, 0)$ 、 $(1, 1, 1)$ 和 $(1, 0, 2)$ 是一个平面片的顶点，下面哪些线段是面片表面的法线？

a . 从 $(1, 0, 0)$ 到 $(1, 1, 0)$ 的线段

b . 从 $(1, 1, 1)$ 到 $(2, 1, 1)$ 的线段

c . 从 $(1, 0, 2)$ 到 $(0, 0, 2)$ 的线段

d . 从 $(1, 0, 0)$ 到 $(1, 1, 1)$ 的线段

18 . 说出程序化模型的两种“类型”。

19 . 在建模过程和渲染过程之间，哪一个是

a . 更标准化的任务？

b . 更计算密集型的任务？

c . 更具创造性的任务？

证明你的答案。

20 . 下列哪一个可能在场景图中表示？

a . 光源

b . 不动的道具

c . 角色/演员

d . 摄像机

21 . 从何种意义上说 , 场景图的创建是3D图形学过程中的关键步骤 ?

22 . 场景图中的摄像机可能会改变位置和朝向 , 这个事实引入了何种复杂性 ?

23 . 假设带有顶点 $(0, 0, 0)$ 、 $(0, 2, 0)$ 、 $(2, 2, 0)$ 和 $(2, 0, 0)$ 的平面片的表面是平滑且闪亮的。如果一条光线从点 $(0, 0, 1)$ 发出 , 在 $(1, 1, 0)$ 处入射表面 , 那么反射光线将经过下列哪个点 ?

a . $(0, 0, 1)$

b . $(1, 1, 1)$

c . $(2, 2, 1)$

d . $(3, 3, 1)$

24 . 假设一个浮标上支撑着一个离静止水面高3米的灯。如果有一个观察者距浮标4.5米 , 在水面以上1.5米 , 那么观察者在水面的哪个点上能看到灯光的反射 ?

25 . 如果一条鱼在静止的水面下游动 , 观察者从水面上看鱼 , 那么从观察者的位置来看 , 鱼好像是在哪里 ?

a . 在其真实位置的上方且朝向背景方向。

b . 在真实位置。

c . 在其真实位置的下方且朝向前景方向。

26 . 假设点 $(1, 0, 0)$ 、 $(1, 1, 1)$ 和 $(1, 0, 2)$ 是一个平面片的顶点，并且从物体外面看时顶点是按逆时针顺序列出的。在以下每种情况下，指明从所给出的点发出的光线是从物体的外面照到面片的表面的，还是从物体的里面照到面片的表面的。

a . $(0, 0, 0)$

b . $(2, 0, 0)$

c . $(2, 1, 1)$

d . $(3, 2, 1)$

27 . 举一个视体外的物体仍然能够出现在最终图像中的例子。请解释你的答案。

28 . 描述z缓冲区的内容和用途。

29 . 在针对隐藏面消除的讨论中，我们描述了借助z缓冲区解决“前景/背景”问题的过程。用第5章介绍的伪代码表达这个过程。

30 . 假设物体的表面被交替的橙色和蓝色竖条纹覆盖，其中每个竖条纹的宽度都是1厘米。如果这个物体被放置在场景中，以便其像素

位置与物体上间隔2厘米的点相关联，那么在最终图像中，物体的外观可能会是什么样的？解释你的答案。

31．虽然纹理映射和凹凸映射都是与表面“纹理”相关的手段，但它们是截然不同的技术。用一小段话来对它们进行比较。

32．列出渲染流水线的4个步骤，并给出每个步骤的简要定义。

33．使用硬件/固件实现渲染流水线有哪些优点？

34．为交互式视频游戏设计的计算机的硬件与通用PC的硬件有什么不同？

35．传统渲染流水线的主要限制是什么？

36．局部光照模型与全局光照模型的区别是什么？

37．光线跟踪与传统的渲染流水线相比有哪些优点，有哪些缺点？

38．分布式光线跟踪与传统的光线跟踪相比有哪些优点，有哪些缺点？

39．辐射度与传统的渲染流水线相比有哪些优点，有哪些缺点？

40．如果用传统光线跟踪制作的场景图像与用辐射度制作的相同场景的类似图像进行比较，你该如何比较这两类图像？

41．制作一部能在电影院放映的90分钟动画作品需要多少帧？

42．描述粒子系统是如何用于制作闪烁的火焰动画的。

43．在创建一个描述单个物体在场景中移动的动画序列时，解释z缓冲区的使用有何帮助。

44．当今人类动画师的工作与以前人类动画师的工作有哪些不同？

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1．假设计算机动画达到了在影视业中不再需要真实演员的地步，那结果会是什么？不再有“电影明星”会带来什么样的连锁反应？

2．随着数字摄像机和相关软件的发展，普通大众已经具备了修改或伪造照片的能力。这将给社会带来何种变化？会引起哪些道德问题和法律问题？

3．对照片应该拥有到什么程度？假设一个人在网站上放一张自己的照片，其他人下载了这张照片并对其进行修改，以致主体名声受损，并在流行的社交媒体上传播修改后的版本。那么照片的主体应该

拥有什么追索权？如果原始照片中的那个人代表一个公众广泛信任的组织会怎么样——那个组织或公众必须有什么追索权才能获得真相？

基本知识点

- 计算机程序或运行程序的结果可以被迅速分享给大量用户，会对个人、组织和社会产生广泛影响。

4. 帮助开发暴力视频游戏的程序员应该对此游戏产生的任何后果负有何种程度的责任？应该限制儿童接触此类游戏吗？如果应该，应该如何限制以及由谁来限制？对社会上其他一些特殊的团体（如已定罪的罪犯）呢？

课外阅读

Angel, E. and D. Shreiner. *Interactive Computer Graphics, A Top-Down Approach with WebGL*, 7th ed. Boston, MA: Addison-Wesley, 2014.

Hill, Jr., F. L. and S. Kelley. *Computer Graphics Using OpenGL*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2007.

Hughes, J.F., A. Van Dam, M. McGuire, D.F. Sklar, J.D. Foley, S.K. Feiner, and K. Akeley. *Computer Graphics, Principles and Practice*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2014.

LaViola, Jr., J. J., E. Kruijff, R.P. McMahan, D.A. Bowman, and I. Poupyrev. *3D User Interfaces Theory and Practice*, 2nd ed. Boston, MA: Addison-Wesley, 2017.

McConnell, J. J. *Computer Graphics, Theory into Practice*. Sudbury, MA: Jones and Bartlett, 2006.

Parent, R. *Computer Animation, Algorithms and Techniques*, 3rd ed. San Francisco, CA: Morgan Kaufmann, 2012.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第11章 人工智能

本章探讨计算机科学的一个分支：人工智能。尽管该领域相对年轻，但它已经产生了一些令人惊讶的结果，例如，电子游戏节目的参赛者、用来学习和推理的计算机、协调一致来完成一个共同目标（如赢得一场足球比赛）的多台机器。在人工智能领域，今天的科学幻想可能就是明天的现实。

本章内容

11.1 智能与机器

11.2 感知

11.3 推理

11.4 其他研究领域

11.5 人工神经网络

11.6 机器人学

11.7 后果的思考

持久理解和学习成果

计算有助于探索和发现信息中的联系。AP[®]CSP

学习成果 从数据中提取信息来发现和解释联系或趋势。

计算对人和社会有全球性影响——既有有利的又有有害的。

AP[®]CSP

学习成果 分析计算的有利影响和有害影响。

人工智能挑战包括处理图像和语言、构建自主智能体和机器人这样的障碍。

学习成果 区分机器推理/行为和人类推理/行为的概念。

学习成果 描述启发式在复杂搜索中的作用。

学习成果 了解3种主要的机器学习方式（监督学习、强化学习和无监督学习）的区别。

人工智能是计算机科学的一个领域，旨在设法构建自主机器——无须人为干预就能完成复杂任务的机器。这个目标要求机器能够感知和推理，这两种能力属于常识性活动，虽然对人脑来说是自然而然的，但历史证明对机器来说是有难度的。结果是该领域的工作一直富于挑战性。本章就来探讨这个广阔研究领域中的一些主题。

11.1 智能与机器

人工智能领域相当大，并且是与心理学、神经学、数学、语言学以及机电工程等学科相融合的。为了集中思路，我们先来考虑智能体的概念以及智能体可能呈现的智能行为类型。实际上，人工智能的许多研究都可按智能体的行为来分类。

11.1.1 智能体

智能体（agent）是一种能对其环境的刺激做出响应的“设备”。把一个智能体想象为一个单个的机器（如机器人）是很自然的，尽管智能体可能还有别的形式，如自主飞机、交互式视频游戏里的角色，或是通过因特网与其他进程通信的进程（可能作为客户机、服务器或対等端）。大多数智能体都具有传感器和执行器，前者接收来自其环境的数据，后者对其环境做出相应。常见的传感器包括麦克风、照相机、距离传感器以及空气或土壤采样设备等，执行器的例子有车轮、腿、翅膀、夹子以及语音合成器。

很多人工智能的研究都可以在构建智能体的上下文中进行描述，这意味着智能体执行器的动作必须对通过其传感器接收的数据做出合理的响应。按照响应的级别，我们可以对这些研究进行分类。

最简单的响应是反射作用（reflex action），这只是对输入数据的一个预定的响应。更高级的响应用于获取更“智能的”行为。例如，我们可能会给智能体赋予其所在环境的知识，要求其相应地调整其行为。投掷棒球的过程在很大程度上是反射作用，但决定如何投掷以及

向哪儿投掷就需要有当前环境的知识。（有一个人出局，跑垒员在一垒和三垒。）如何存储、更新和处理这种现实世界的知识，并最终应用到决策过程中，一直是人工智能领域具有挑战性的问题。

如果我们希望智能体的目标是赢得一场棋赛或是通过一条拥挤的通道，就需要另一种级别的响应。这种目标导向的行为需要智能体的响应（或一系列响应），应当是周密考虑的结果，构成一个行为计划，或是在当前的各种选项中选取最好的行为。

在有些情况下，智能体的响应能够随着时间的推移和智能体学习而得到改进。智能体的学习可以采取不断发展[过程性知识](#)（procedural knowledge）的形式（学习“怎样”），或者储备[陈述性知识](#)（declarative knowledge）的形式（学习“什么”）。学习过程性知识通常涉及一个反复试验的过程，在这个过程中，智能体从出错受罚、正确受奖的过程中学会恰当的动作。人们根据这个方法开发了一些智能体，它们能够随着时间的推移逐步提高它们在竞赛性游戏（如西洋跳棋和国际象棋）中的能力。学习陈述性知识采取的形式通常是扩充或变更智能体的知识库中的“事实”。例如，一个棒球运动员必须不断地重复调整其知识数据库（虽然还是有一人出局，但现在跑垒员在一垒和二垒），根据它来确定未来事件的合理响应。

一个智能体要对刺激做出合理的响应，就必须“理解”由其传感器接收的刺激。也就是说，智能体必须能够从其传感器产生的数据里提取信息，或者换句话说，智能体必须能够感知。在有些情况下，这是一个简单的过程。从一个陀螺仪获取的信号很容易以与确定响应的计

算兼容的形式编码。但在其他情况下，从输入数据中提取信息很难。例如，理解语音和理解图像就很难。同样，智能体也必须能够将它们的响应以与其执行器兼容的形式表达。这可以是一个简单的过程，也可能需要智能体将其响应表达为一个完整的口语句子——这意味着智能体必须生成语音。所以，像图像处理和分析、自然语言理解以及语音生成这些主题都是重要的研究领域。

我们这里标识的智能体的属性既表示以前的研究范畴，又表示当今的研究范畴。当然，它们彼此之间并不是完全无关的。我们希望最终能够开发出处理所有这些属性的智能体，产生出理解来自环境的数据并能够通过学习过程来开发新的响应模式的智能体，学习的目的是最大限度地提高智能体的能力。然而，研究人员通过单独考虑各类推理行为和独立研究它们，获得了一个可以在以后与其他领域的进步相结合产生更智能的智能体的立足点。

本节的最后我们介绍一个智能体，为11.2节和11.3节的讨论提供一个背景。设计这个智能体是为了求解八数码游戏（eight-puzzle），该游戏由标有1~8的8个方块组成，安装在一个总共能容纳9个方块的3行3列的框架内（见图11-1）。这样，框架内的方块间有一个空位，挨着空位的方块可以推动，允许框架内的方块随意排布。问题是要把杂乱排布的方块移回到它们的初始位置（见图11-1）。

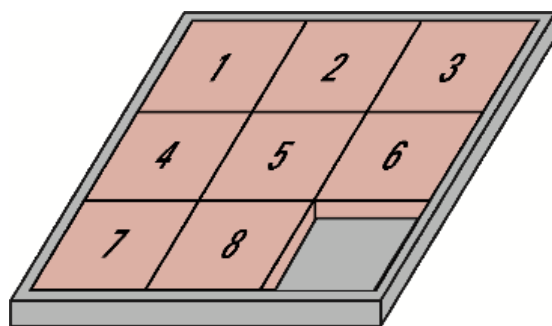


图11-1 八数码游戏的已解布局

我们的智能体采用的装置配有一个夹具、一部照相机和一根带橡皮头的指杆，其中橡皮头是为了推移物品时不会打滑（见图11-2）。当首次开启该智能体时，夹具会一开一合，好像它在索要方块一样。当我们把一个随意排布的八数码游戏放进夹具时，夹具就会接近它。不一会儿，机器的指杆就会降低，并开始在框架内推移方块，直到所有的方块回到它们的原始位置。这时，机器会放开拼板并自己关掉电源。

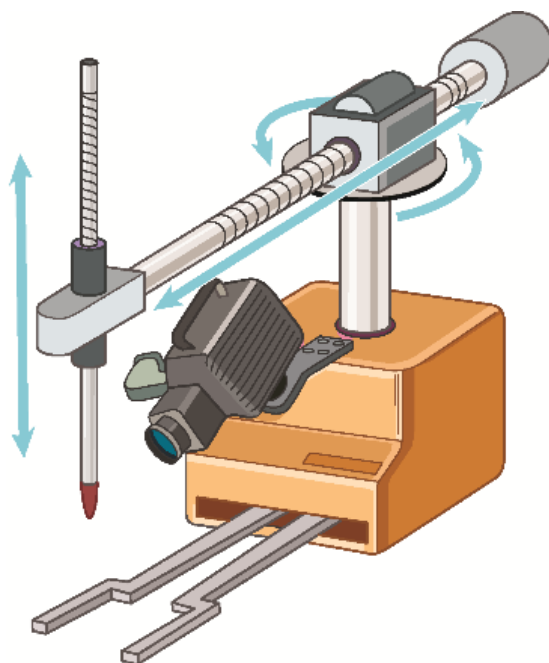


图11-2 我们的八数码游戏求解机器

这个解决八数码游戏的机器展现了前面提过的两个智能体的属性。第一，它必须能够感知，就是必须从其照相机拍摄的图像中获取当前方块的状态。我们将在11.2节阐述理解图像的问题。第二，它必须制订并实施一个实现目标的计划。这些问题将在11.3节中阐述。

11.1.2 研究方法

要对人工智能领域做出客观的评价，理解它是将沿两条路线发展会有所帮助。一条是工程路线，其中研究人员试图开发展现智能行为的系统。另一条是理论路线，其中研究人员正在尝试从计算的角度来研究动物（尤其是人类）的智能。我们通过考虑这两种路线的执行方式，来说明这种二分的研究方法。在工程路线指导下，产生的是面向性能的方法论，因为根本目标是生产符合某些性能目标的产品在理论路线指导下，产生的是面向模拟的方法论，因为根本目标是增进人类对计算智能的理解，因而关注重点是底层过程，而不是外在性能。

作为一个例子，考虑自然语言处理和语言学领域。这些领域关系密切，各领域的研究可互相取长补短，但它们的根本目标不同。语言学家的兴趣在于弄明白人类如何处理语言，因此更倾向于理论性的研究，而自然语言处理领域的研究者的兴趣在于开发能处理自然语言的机器，因此倾向于工程方向。因此，语言学家以面向模拟的模式行事——构建用来检验理论的系统。与其相比，自然语言处理的研究者以面向性能的模式行事——构建执行任务的系统。后一种模式产生的系

统（如机器用于响应口头命令的文档翻译程序和系统）在很大程度上依赖于语言学家获取的知识，但在特定系统的限定环境中经常使用恰好能工作的“简化方法”。

作为一个基本的例子，考虑为一个操作系统开发shell的任务，它通过口述英语命令从外部世界接收指令。在这种情况下，shell（一个智能体）不需要考虑完整的英语语言。更准确地说，shell不需要区分copy这个词的不同意思。（是名词还是动词？应该含有抄袭的言外之意吗？）相反，shell只需将copy这个词与其他命令（如rename、delete）区分开来。因此，shell可以通过将输入与预先确定的音频模式相匹配来完成它的任务。这种系统的性能可能会令工程师满意，但从美学角度看，其实现方式也许并不能取悦理论家。

11.1.3 图灵测试

过去，**图灵测试**（Turing test，由艾伦·图灵在1950年提出）一直作为衡量人工智能领域发展的一种基准。现在，图灵测试的重要性已不及从前，但它仍是人工智能领域重要的一部分。图灵的提议是，允许一个人（我们称其为询问者）通过打字机系统与一个测试对象进行通信，但不告知询问者测试对象究竟是一个人还是一台机器。在这种环境下，如果询问者没能将机器与人区分开，就可以称这台机器的行为是智能的。图灵预测，到2000年，机器将会有30%的机会通过一个5分钟的图灵测试——这个预见惊人地准确。

人工智能的起源

构建能够模仿人类行为的机器的探索有很长的历史，不过很多人会认同现代人工智能领域起源于1950年。就是在这一年，艾伦·图灵发表了论文“Computing Machinery and Intelligence”，其中提出机器能够通过编程来展现智能的行为。这个领域的名字——人工智能——就在几年后由约翰·麦卡锡（John McCarthy）在一个现在看来颇具传奇色彩的提案中提出，他建议“1956年夏天在达特茅斯学院（Dartmouth College）开展人工智能的研究”，以探索“一个假设，即学习的每个方面或智能的任何其他特征原则上都能够被描述得特别精确以至于能够制造出模拟它的机器”。

图灵测试已不再被认为是对智能的有效度量，其中一个原因是，相对简单的手法会产生（produce）怪异的智力表现。图灵测试场景的一个著名示例是20世纪60年代中期由Joseph Weizenbaum开发的DOCTOR程序（更通用的系统版本称为ELIZA）。这个交互程序被设计用来反映罗杰斯分析师指导心理治疗的场景，计算机扮演分析师的角色，而用户扮演病人的角色。在内部，DOCTOR所做的一切是根据某些定义明确的规则将病人的陈述重新构造并反馈给病人。例如，回应病人的陈述“我今天觉得很累”，DOCTOR可能回答：“为什么你今天觉得很累？”如果DOCTOR不能识别句子结构，它就只能做出像“继续”或者“这很有趣”这样的响应。

Weizenbaum开发DOCTOR的目的是研究自然语言的交流。心理疗法的课题只提供了一个程序可以“交流”的环境。但令Weizenbaum沮丧

的是，个别的心理学家建议把这个程序用在实际的心理治疗中。（罗杰斯的论点是，在治疗期间，应该是病人来主导对话，而不是分析师。因此，他们认为计算机也可能能像治疗师那样引导对话。）而且，DOCTOR投射出极强的理解能力，以至于许多与它“交流”的人都愿意顺从机器的问答式对话。在某种意义上，DOCTOR通过了图灵测试。结果是，在道德及技术上都产生了争议，Weizenbaum成为一位在技术进步的世界中维护人类尊严的倡导者。

较新的图灵测试“成功”的例子包括一些因特网病毒，为了诱骗人类放松对恶意软件的防护，这些病毒与人类受害者进行“智能”对话。此外，与图灵测试类似的现象出现在国际象棋博弈程序这样的计算机游戏的场景中。尽管这些程序是通过应用蛮力技术来选择棋子的走法（这与我们将在11.3节讨论的内容相似），但与计算机竞赛的人却经常感觉机器拥有创造力甚至个性。相似的感觉在机器人技术领域也有，其中根据物理属性建造的机器能表现出智能特征。例如，玩具机器狗仅通过点头或者竖耳朵来响应声音，就表现出了可爱的个性。

11.1节问题与练习

1. 指出一个智能体可能会做的几种“智能”行为。
2. 一棵放在只有一个光源的暗室里的植物，会朝着光源生长。这是一种智能反应吗？植物拥有智能吗？那么，你对智能的定义是什么？
3. 假设一台售货机根据所按的按钮售出各种物品，你认为这样的一台机器“知道”哪个按钮被按了吗？那么，你对“知道”的定义是什么？

么？

4．如果一台机器通过了图灵测试，你会认同这台机器是智能的吗？如果不认同，你会认同它看上去是智能的吗？

5．假设你使用发短信或者交互式社交媒体网站进行了长达10分钟的有意义的连贯对话。如果你后来发现与你对话的是一台机器，那么你会得出结论说这台机器是智能的吗？为什么会或者为什么不会？

11.2 感知

一个智能体要想智能地响应从它的传感器接收的输入，就必须能够理解那个输入。也就是说，智能体必须能够感知。本节我们来探讨感知的两个研究领域，即理解图像和理解语言，这两个领域被证明特别具有挑战性。

11.2.1 理解图像

让我们考虑一下11.1节介绍的求解八数码游戏的机器提出的问题。因为我们的应用要求的精度不高，在开合的过程中检测夹具中是否有方块的功能也很简单，所以机器上夹具的开合没有出现严重的障碍。即使是把照相机聚焦在方块上的问题，也能通过将夹具的位置设计在一个便于观察的特定的预定位置上简单地解决。因此，机器需要的第一个智能行为是从视觉媒介中提取信息。

重要的是要认识到，我们的机器在看方块时面对的问题不单是生成和存储图像。很多年前技术就能做到这一点，像传统的摄影及电视系统的情况一样。相反，问题是为了提取方块的当前状态，要理解这个图像（可能随后还要监控方块的移动）。

在八数码游戏求解机器的案例中，对方块图像的可能解释是相对有限的。我们可以假定，出现的东西总是一个排列整齐的数字1~8的图像。问题只是去提取这些数字的排列。为此，我们想象方块的图片已经在计算机内存中，按照位进行编码。编码中的每个位表示一个特定像素的亮度。假定图像的大小统一（机器把方块放在照相机前的预定位置），把图像的不同部分与由位模式构成的预定模板相比较（位模式是由方块中使用的单个数字生成的），机器就能够检测出哪个方块在哪个位置。如果发现匹配，则说明方块达到了要求的条件。

这种识别图像的技术是光学字符阅读器中使用的一种方法。不过，它是有缺点的，它对被读取的符号的类型、大小和朝向要求一定程度的统一性。具体来说，即使对于相同的符号，外形也相同，字体较大的字符产生的位模式与字体较小的模板也不匹配。此外，可以想象，在设法处理手写材料时问题会变得非常困难。

解决字符识别问题的另一种方法是基于匹配几何特征，而不是基于符号的精确外形的。在这种情况下，数字1可能会被描述为一条单竖线，数字2可能是一条不封闭的曲线和一条横跨底部的水平直线相连，等等。这种识别符号的方法分两步：第一步是从要处理的图像中提取图像特征；第二步是把这些特征与那些已知符号的特征进行比较。与

模板匹配方法一样，这种符号识别技术并不可靠。例如，图像中的小错误会产生一组完全不同的几何特征，就像区分字母O和C的情况，或者区分八数码游戏里的数字3和8的情况。

幸好在八数码游戏中不需要理解一般的三维场景的图像。例如，我们的优势是能保证识别的形状（数字1~8）相互孤立地处在图像上的不同部分，而不是像更一般的布置中常见的重叠图像那样出现。例如，在一张普通的照片中，我们面对的不仅是从不同的角度识别一个物体的问题，还有物体的某些部分会被隐藏起来看不到这个事实。

理解一般图像的任务通常要分两步进行：（1）**图像处理**（image processing），指标识图像的特征；（2）**图像分析**（image analysis），指理解这些特征代表什么意思的过程。在利用符号的几何特征识别符号的情况中，我们已经提出了二分法。在这个过程中，我们发现：图像处理由标识在图像中发现的几何特征的过程来表示，图像分析由标识那些特征的含义的过程来表示。

图像处理带来了大量的研究课题。一个是轮廓增强，这是一个使用数学技术使图像中区域间的边界变得更清晰的过程。在某种意义上，轮廓增强试图将照片转换成线条画。图像分析的另一个活动是所谓的区域发现。这是标识图像中具有共同属性（如亮度、颜色或者纹理）的那些区域的过程。这种区域可能代表图像中某个物体的一部分。（这种识别区域的能力使计算机可以给老式的黑白电影着成彩色。）图像处理领域还包含另一个活动——平滑（smoothing），即去

除图像中缺陷的过程。平滑使图像中存在的错误不会混淆其他图像处理步骤，但是过度平滑也会导致重要信息的丢失。

平滑、轮廓增强和区域发现都是用来标识图像中各种成分的步骤。图像分析是确定“这些成分代表什么以及最终这个图像代表什么”的过程。这里人要面对从不同视角识别被部分遮挡的物体这样的问题。图像分析的一种方法是先假定一个图像大概是什么，然后尝试把图像中的成分与那些猜测到的物体相联系。这看起来是人类所用的方法。例如，有时我们会发现，在我们视觉模糊的情况下，识别一个没有想到的物体是很困难的，但是一旦有了该物体可能是什么的线索，我们就能很容易地识别它。

与一般图像分析相关的问题很复杂，但在图像分析领域取得的巨大进步已经帮助推动了很多方面的发展，从机器人吸尘器到超过95%的美国邮件的自动分拣。不过，该领域的许多研究仍有待完成。事实上，图像分析是其中一个说明那些人类智力完成起来又快又明显容易的任务如何不断挑战机器能力的领域。

强人工智能与弱人工智能

能够通过对机器进行编程来展现智能行为的那种推测能力，被认为**弱人工智能**（weak AI），并且今天在不同程度上已经被大众所接受。但是，机器能够通过编程获得智力和（实际上）意识的推测能力，则被认为是**强人工智能**（strong AI）。强人工智能引发了广泛的争论。反对者认为，机器与人有本质的区别，因而它永远不能像人那样感受爱、判断对错和考虑自我。然而，支持者认为，人脑的是由许

多小的部件构成的，每个部件都不是人，都没有意识，但是，当它们组合起来就成了人，有了意识，他们辩称，为什么同样的现象就不可能出现在机器身上呢？

解决强人工智能争论的难点在于，智能和意识这样的属性是内在特征，不能够直接界定。正如艾伦·图灵指出的那样，我们认为其他人有智能的原因是他们的行为是智能的——即使我们无法观察到它们内部的智力状态。那么，如果机器也呈现外在的意识特征，那么我们准备认可机器具备同样的水准吗？为什么认可，为什么不认可？

11.2.2 语言处理

理解语言是感知问题的另一个已证明的颇具挑战性的问题。把形式化的高级程序设计语言翻译成机器语言（见6.4节）方面获得成功，使早期的研究人员相信，只需几年就能实现通过编程使计算机理解自然语言。实际上，翻译程序的这种能力会给人一种错觉——机器真能理解正在翻译的语言。（回忆6.1节中格雷斯·赫珀讲的那个故事，管理者们以为她正在教计算机理解德语。）

这些研究人员无法理解的是，形式化的程序设计语言与英语、德语以及拉丁语这些自然语言之间在深度上的差异。程序设计语言由精心设计的原语组成，每条语句只有一种语法结构和一个意思。相反，自然语言的一个句子会因为上下文的不同，甚至交流方式不同而有多种意思。因此，为了理解自然语言人在很大程度上依靠额外的知识。

例如，句子

Norman Rockwell painted people.

和

Cinderella had a ball.

都有多种意思，但通过语法分析或单独翻译每个词并不能区分这些意思。实际上，要理解这些句子需要具有理解产生句子的上下文的能力。在有些场合，一个句子的真实意思与它的字面翻译完全不同。例如，

Do you know what time it is?

通常的意思是“请告诉我现在几点了”，如果说话的人已经等候了很长时间，那么这句话意思可能是“你来得太晚了”。

因此，要弄明白一种自然语言中的一个句子的意思需要几个层次的分析。第一层是**句法分析**（syntactic analysis），其主要成分是语法分析。在这里，句子

Mary gave John a birthday card.

的主语被认为是Mary，而句子

John got a birthday card from Mary.

的主语被发现是John。

分析的第二层称为**语义分析**（semantic analysis）。语法分析仅标识每个词在语法上的作用，与语法分析不同，语义分析的任务是标识句子中每个词在语义上的作用。语义分析试图标识的内容有：描述的动作、动作的主体（不一定是句子的主语）以及动作的目标等。正是通过语义分析，我们认为句子“玛丽给约翰一张生日贺卡”和“约翰收到玛丽赠送的一张生日贺卡”是在说同一件事。

分析的第三层是**上下文分析**（contextual analysis）。在这一层，句子的上下文被引入理解过程中。例如，很容易分辨出句子

The bat fell to the ground.

中的每一个单词在语法上的作用。我们甚至能通过识别动作“falling”和动作主体“bat”等来实现语义分析。但直到我们考虑到句子的上下文后，句子的意思才能变得明确。具体来说，这句话在棒球比赛的上下文中和在洞穴中探险的上下文中有着不同的意思^[1]。而且，只有在上下文这一层，问题“Do you know what time it is?”的真正意思才能最终揭晓。

[1] 英文单词bat有“蝙蝠”和“球棒”两种意思，在棒球比赛中应理解为“球棒”，在洞穴探险中则应理解为“蝙蝠”。——译者注

我们应该注意，各个层次分析（句法、语义及上下文）并不一定是相互独立的。对于句子

Stampeding cattle can be dangerous.

如果我们想象是一群牛在惊跑，那么主语是名词cattle（stampeding是修饰它的形容词）。但是，如果上下文是哪个捣蛋鬼以惊吓牛群取乐，那么主语就是动名词stampeding（宾语是cattle）。因此，这个句子不只有一个语法结构——究竟哪一个正确取决于上下文。

自然语言处理的另一个研究领域涉及整个文档，而不是单个句子。这里涉及的问题分成两类：**信息检索**（information retrieval）和**信息提取**（information extraction）。信息检索的任务是标识与手头主题有关的文档。例如，万维网的用户设法找到与特定主题相关的站点时所面对的问题。该技术的当前状态是为关键字搜索站点，但这经常产生大量的假结果，并且经常会因为它处理的是“automobiles”而不是“cars”而忽略重要的站点。因此，我们就需要一种能理解所考虑的站点内容的搜索机制。要获得这样的理解很困难，这也就是许多搜索机制都转向采用XML这样的技术来产生万维语义网（如4.3节所述）的原因。

掌心中的人工智能

人工智能技术在智能手机应用中越来越突出。例如，谷歌公司研发了一款提供可视搜索引擎的智能手机应用Google Goggles。只要用智能手机的照相机对某一本书、某一地标性建筑或某一标记拍照，Goggles就会进行图像处理、图像分析和文本识别，然后启动一次Web搜索来识别这个对象。对于许多常见的语言组合，谷歌翻译能用一种

语言记录口头词句，将它们翻译成另一种语言讲给你。随着人工智能不断以创新方式被使用，智能手机无疑会越来越智能。

信息提取的任务是从文档中提取信息，以便它采用一种在其他应用中中用的形式。这可以理解为一个特定的问题确定答案，或者以某种形式记录信息供以后回答问题时使用。有一种这样的格式称作**框架**（frame），框架实质上是一种记录细节的模板。例如，考虑一个读报系统。该系统可能会利用各种各样的框架，为报纸上可能出现的每类文章用一个。如果系统认定一篇文章是关于入室盗窃的报道，那么它会继续设法将它填写到入室盗窃框架的位置上。该框架可能要求填写入室盗窃的地点、入室盗窃的时间和日期以及盗走的物品等数据项。与之相比，如果系统认定一篇文章是关于自然灾害的报道，那么它会填写自然灾害框架，引导系统确定灾害类型、损失大小等。

信息提取器记录信息的另一种形式称为**语义网**（semantic net）。这实质上是一个大的链式数据结构，结构中的指针用来指示数据项之间的联系。图11-3展示了一个语义网的一部分，其中突出显示的部分是从下面的句子中得到的信息：

Mary hit John.

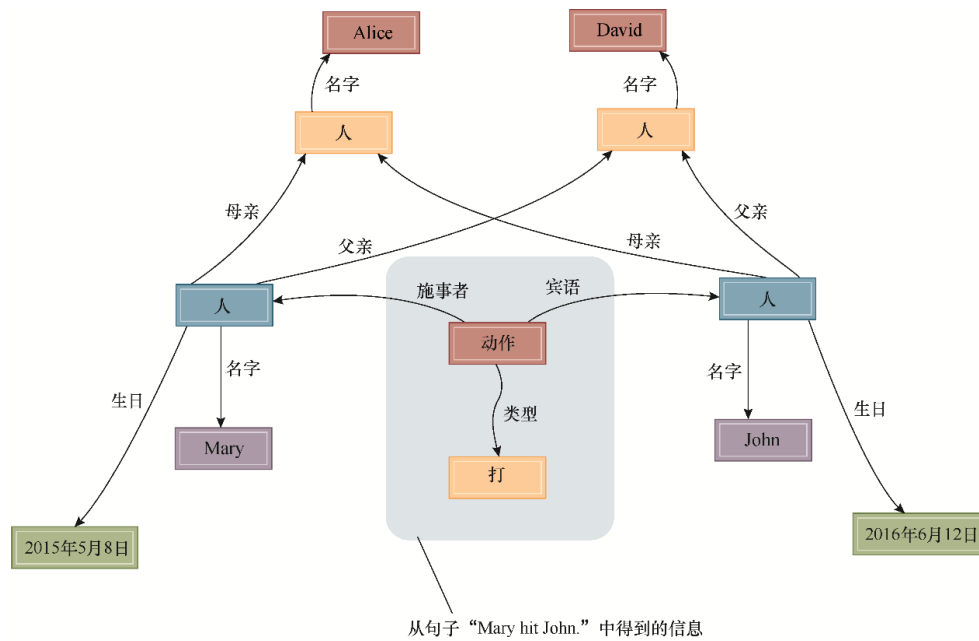
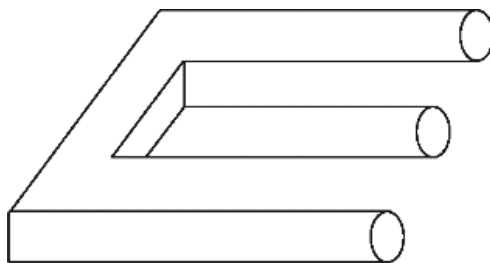


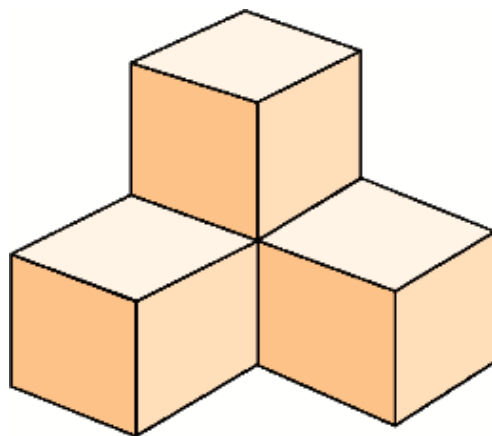
图11-3 一个语义网

11.2节问题与练习

1. 如果机器人使用自己的视频系统来控制自己的活动，而不是把视频转播给远程控制机器人的人，那么对机器人视频系统的要求会有什么不同？
2. 是什么让你知道下图是不合情理的？这样的洞察力怎样编程到机器中？



3. 下面表示的堆中有几个方块？如何对机器进行编程使其能准确回答这种问题？



4. 你是如何知道句子“Nothing is better than complete happiness”和句子“A bowl of cold soup is better than nothing”不是暗指“A bowl of cold soup is better than complete happiness”的？如何将你的这种区分能力移植到机器上？

5. 指出翻译句子“They are racing horses”时涉及的二义性。

6. 比较下列两个句子的语法分析结果，然后解释二者在语义上的差异。

The farmer built the fence in the field.

The farmer built the fence in the winter.

7. 根据图11-3中的语义网，Mary和John之间是什么家庭关系？

11.3 推理

现在，让我们使用11.1节介绍的求解八数码游戏的机器来探讨开发具有基本推理能力的智能体的技术。

11.3.1 产生式系统

一旦我们的求解八数码游戏的机器从看到的图像中解读出方块的位置，它的任务就变成了算出怎么走能解开谜题。解决这个问题的一种马上可以想到的方法是，把方块的所有可能排列对应的解决方案都预先编程到机器中。然后，机器的任务就只是选择和执行合适的程序。然而，因为八数码游戏有100 000多种布局，所以为每一种布局提供一个直接的解决方案显然不可取。因此，我们的目标是对机器编程，让机器能够自己构建八数码游戏的解决方案。也就是说，必须对机器编程使其能够完成基本的推理活动。

机器推理能力的开发是一个研究多年的主题。有关这方面的研究的一个结果是认识到，有一大类具有共同特征的推理问题。这些共同特征被单独放在一个称为**产生式系统**（production system）的抽象实体中。产生式系统由以下3个主要部分组成。

（1）**状态集合**。每个**状态**（state）是一种可能在应用环境中发生的情形。最初的状态称作**开始状态**（start state）（或者初始状态），期望的状态称作**目标状态**（goal state）。（在我们的案例中，一个状态就是八数码游戏的一种布局，开始状态就是八数码游戏提交给机器时的布局，目标状态就是图11-1所示的已经解决了难题的布局。）

（2）**产生式集合**（又称**规则**或者**移动**）。**产生式**（production）是指能在应用环境中完成的让系统从一种状态转换到另一种状态的操作。每个产生式可能都会关联一些先决条件，也就是说，在产生式被应用之前，环境中必须出现一些可能存在的条件。（对于八数码游戏

的例子，产生式就是方块的移动。方块每次移动的先决条件是其相邻的位置必须有空位。)

(3) **控制系统**。**控制系统** (control system) 是由解决从开始状态转换到目标状态问题的逻辑组成的。在这个过程的每一步中，控制系统都必须决定接下来应该应用满足前置条件的那些产生式中的哪一个。(对于八数码游戏的例子，给定八数码游戏一个特定状态，在空位旁有几个方块就存在几个可用的产生式。控制系统必须决定移动哪一个方块。)

注意，在一个产生式系统的上下文中，赋予求解八数码游戏的机器的任务可以被公式化。在这种情况下，控制系统采用程序的形式。该程序检查八数码游戏的当前状态，确定实现目标状态的一系列产生式，并执行这一系列产生式。因此，我们的任务就是为求解八数码游戏设计一个控制系统。

控制系统开发中的一个重要概念是**问题空间** (problem space)，它是产生式系统中的所有状态、产生式和先决条件的集合。问题空间通常会概念化成**状态图** (state graph) 的形式。这里，图这个术语是指一种数学家称为**有向图** (directed graph) 的结构，即一组由箭头连接起来的称为**节点** (node) 的位置。一个状态图由一组用箭头连接的节点组成，其中节点表示系统中的状态，箭头表示系统从一个状态转换到另一个状态的产生式。当且仅当有一个把系统从箭头起点处的状态转换到箭头终点处的状态的产生式时，状态图中两个节点由一个箭头连接。

我们应该强调的是，正像在求解八数码游戏时方块的可能状态的数量使我们无法明确地提供预先编程好的解决方案一样，数量问题也使我们无法明确地表示整个状态图。因此，状态图是概念化手头问题的一种方法，但不能用来表示全部内容。尽管如此，你可能会发现，它有助于考虑（并可能扩展）图11-4中展示的八数码游戏的一小部分状态图。

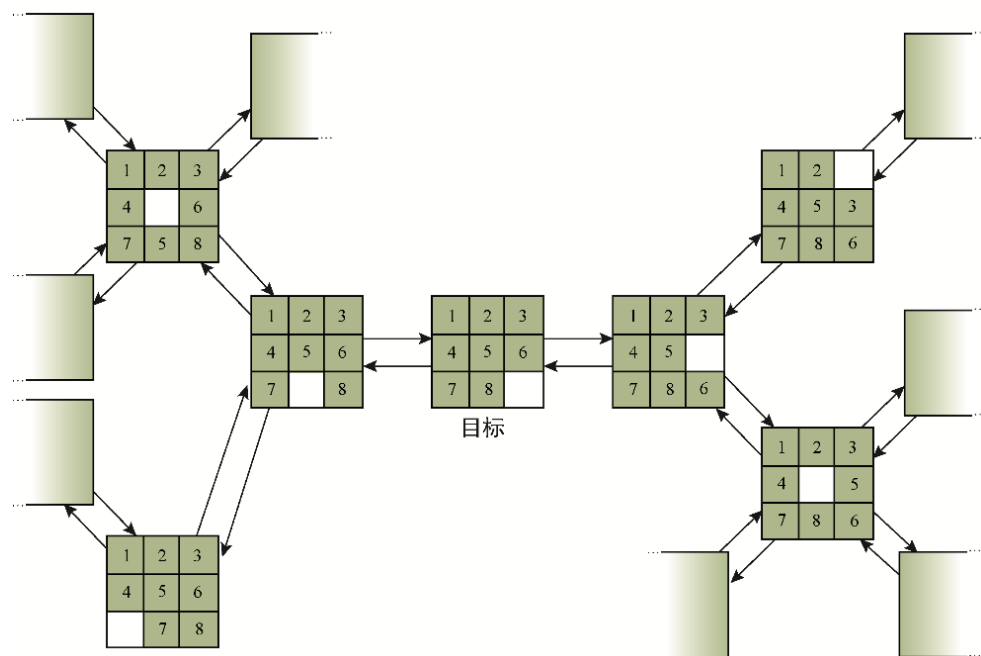


图11-4 八数码游戏状态图的一小部分

当从状态图的角度来看，控制系统面临的问题变成了寻找一连串从开始状态指向目标状态的箭头。毕竟，这一连串的箭头代表了解决初始问题的一系列产生式。因此，不管应用是什么，控制系统的任务都可以看作寻找一条贯穿状态图的路径。对控制系统的这种普遍观点是根据产生式系统对要求推理的问题进行分析得到的成果。如果一个

问题能够根据产生式系统来描述，那么它的解决方案就能够根据搜索一条路径来明确表达。

为了强调这一点，我们先来考虑其他任务是如何根据产生式系统来设计，并在寻找贯穿状态图的路径的控制系统上下文中完成的。人工智能的经典问题之一就是下国际象棋这样的游戏，这类游戏在一个明确规定的上下文中属于中等复杂度，因此它为测试理论提供了一个理想的环境。在国际象棋中，底层产生式系统的状态是可能的棋盘布局，产生式是棋子的走法，控制系统就具体为选手（人或别的）。状态图的开始节点表示棋子在初始位置时的棋盘。从该节点出发的分支是一些箭头，这些箭头指向游戏中第一步棋之后会形成的那些棋盘布局，而从这些布局的每一个出发的分支仍然是一些箭头，指向下一步棋会形成的那些布局；依次类推。通过这种明确的表达，我们可以把一个国际象棋游戏想象成由两个选手组成，每个选手都试图通过在一个大的状态图中寻找一条通向自己选择的目标节点的路径。

或许从给定事实得出逻辑推论的问题是一个不太明显的产生式系统的例子。这个上下文中的产生式是称为**推理法则**（inference rule）的逻辑规则，这些规则允许从旧语句中形成新语句。例如，语句“所有超级英雄都是崇高的”和“超人是超级英雄”可以合并产生“超人是崇高的”。这样一个系统中的状态由各种在演绎过程的特定点处为真的一组语句组成：开始状态是可以从中得出结论的一组基本语句（常称为公理），目标状态是包含了提出结论的任意一组语句。

作为一个示例，图11-5展示了以下过程所经历的状态图的一部分：从一组语句“苏格拉底是男人”“所有男人都是人”“所有人都是凡人”可以得出结论“苏格拉底是凡人”。

从图11-5中可以看出，随着推理过程应用合适的产生式生成新的语句，我们看到知识的主体从一个状态转换到另一个状态。当今，这种推理系统经常用逻辑程序设计语言（见6.7节）来实现，是大多数**专家系统**（expert system）的骨干。专家系统是为模拟因果推理而设计的软件包，这些推理是人类专家面对相同情形时所遵从的。例如，医疗专家系统用来协助疾病诊断或改良治疗。

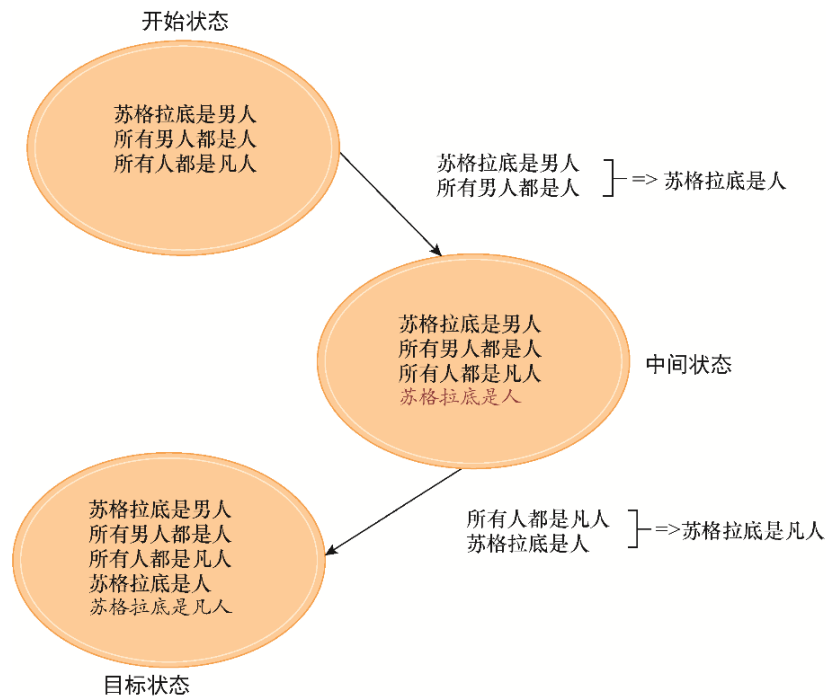


图11-5 产生式系统上下文中的演绎推理

11.3.2 搜索树

我们已经看到，在产生式系统的上下文中，控制系统的工作涉及搜索状态图，找出一条从开始节点到目标节点的路径。完成这种搜索的一种简单方法是遍历每一个从初始状态出发的箭头，在每种情况下，记录目标状态，然后继续遍历从这些新状态出发的箭头，再次记录下结果，以此类推。对目标的搜索像向水中滴入一滴墨水一样，从开始状态扩散开来。这个过程会一直持续到其中一个新状态就是目标状态，这时，解决方案就找到了，控制系统只需沿着找到的这条从开始状态到目标状态的路径应用产生式即可。

这种策略的结果实际上是构建了一棵称作**搜索树**（search tree）的树，它由控制系统经过分析得到的部分状态图构成。搜索树的根节点是开始状态，每个节点的孩子是那些通过应用一个产生式从双亲节点可到达的状态。搜索树中节点之间的每条线段代表一个产生式的应用，从根到叶子的每条路径代表状态图中相应状态之间的一条路径。

解决图11-6所示布局的八数码游戏将会产生的搜索树如图11-7所示。该搜索树最左边的分支代表试图通过先向上移动方块6来解决问题，中间分支表示以向右移动方块2来开局，而最右边的分支表示以向下移动方块5来开局。该搜索树进一步显示，如果以向上移动方块6来开局，那么下一个允许的产生式只能是右移方块8。（实际上，这时还可以向下移动方块6，但这只会倒退回上一个产生式，因而是毫无意义的移动。）

1	3	5
4	2	
7	8	6

图11-6 一个未解的八数码游戏

目标状态出现在图11-7所示的搜索树的最下面一层。因为这表明解决方案找到了，所以控制系统可以结束其搜索过程并开始构建用来解决外部环境中的方块的指令序列。这实际上是一个简单的过程：从目标节点的位置上行，同时在遇到由树枝线表示的产生式时将它们压入栈中。将这种方法应用到图11-7中的搜索树上，就会产生图11-8中的产生式栈。现在，只要把指令从该栈中弹出并执行，控制系统就能够解决外界的问题。

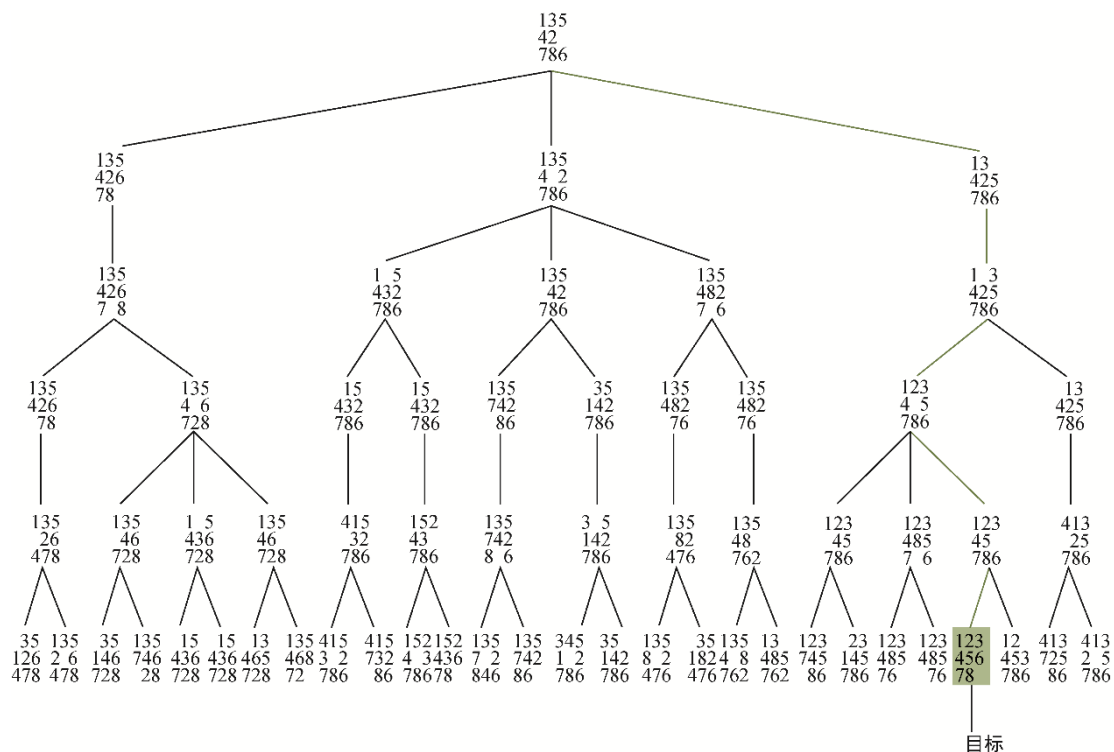


图11-7 搜索树的一个示例

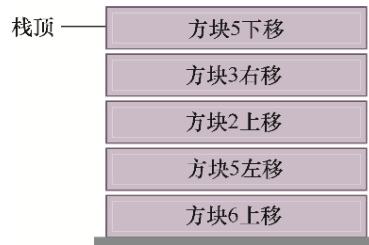


图11-8 被推入栈的用于稍后执行的产生式

还有一点我们应当注意，回想第8章我们讨论的树，利用一个指针系统**向下**指示一棵树，由此可以从一个双亲节点移到它的子节点。但是在搜索树的情况下，当控制系统顺着树从目标状态**向上**移到开始状态时，它必须能从一个孩子移到它的双亲。构建这种树的指针系统要向上指，而不是向下指。也就是说，每个子节点包含一个指向其双亲节点的指针，而不是双亲节点包含指向其子节点的指针。（有些应用会用两组指针允许在树中的双向移动。）

11.3.3 启发式

对于图11-7中的例子，我们选择了一个产生可管理搜索树的开始布局。但是在试图解决更复杂的问题时，产生的搜索树就会变得非常庞大。在国际象棋中，第一步移动就有20种可能，因此在这样的情况下，搜索树的根节点将会有20个子节点，而不是我们例子中的3个。而且，一局棋双方都移动30~35次是常有的情况。即使在八数码游戏的情况下，若不能快速到达目标，搜索树也会变得非常大。因此，开发一个完整搜索树像表示整个状态图一样不切实际。

应对这个问题的一种策略是改变搜索树构建的次序。我们不用**广度优先**（breadth-first）的方式（这意味着树是一层一层地构建的），而是沿着更有希望的路径往深度发展，只在原来的选择失败以后才考虑其他选择。这样做的结果就是以**深度优先**（depth-first）的方法构建搜索树，也就是说，树是以建立纵向路径而不是横向层次的方式构建的。更确切地说，这种方法通常被称为**最佳优先**（best-first）构建，因为在搜索中被选中的纵向路径看起来是最优的。

基本知识点

- 诸如“找到最佳”或“找到最小”这样的优化问题无法在合理的时间内解决，但近似的最优解可以。

最佳优先方法类似于人类面对八数码游戏时采用的策略。我们一般不会像广度优先方法那样，同时沿着几个可能的路径进行。相反，我们也许会选择看上去最有希望的路径并先沿着这条路径走下去。注意，我们说的是**看上去**最有希望的。在一个特定点，很难确定哪个选择是最佳的。仅凭感觉当然可能“误入歧途”。但不管怎样，相比“一视同仁”地关注每种选择的蛮力方法，这种凭直觉的方法似乎更具优势，因此在自动控制系统中应用直觉的方法似乎是明智的。

为此，我们需要一种方式来确定几个状态中哪一个看上去是最有希望的。我们的方法是采用**启发式**（heuristic），在我们的示例中，启发式是一个与每个状态相关的数值，用来衡量这个状态与最近目标之间的“距离”。在某种意义上，启发式是规划代价的一个量度。给定两

个状态之间的一种选择，具有较小启发值的状态是那个可以明显以最小代价到达目标的状态。因此，该状态代表了应追求的方向。

启发式应具备两个特征。第一，如果到达相关状态，它必须对该解决方案中剩余的工作量有一个合理的估计。这意味着它在多个选项中做出选择时能提供有意义的信息——启发式提供的估计越好，根据此信息所做的决定就越好。第二，启发式应该易于计算。这意味着它的使用应有益于搜索过程而非成为一种负担。如果计算启发式非常复杂，那还不如把时间花在进行广度优先搜索上。

在八数码游戏中，一个简单的启发式是，通过计算不在合适位置上的方块数目来估计到达目标的“距离”——这种推测指的就是一个有4个方块不在合适位置上的状态，相对于只有2个方块不在合适位置上的状态来说，离目标更远（因此更缺少吸引力）。然而，这个启发式并没有考虑方块离其位置有多远。假如这两个方块离它们的正确位置太远，就需要许多产生式来移动它们。

于是，一个比较好的启发式是测量每个方块距其目的地的距离，并把这些值相加得到一个量。一个直接挨着其目的地的方块的距离值为1，而有一个角与其目的地方块相接触的方块的距离值为2（因为它至少要在垂直方向和水平方向各移动一次）。这种启发式容易计算，并能得到一个方块从其当前状态到目的状态过程中需要移动的步数有一个粗略的估计。例如，图11-9所示布局相对应的启发值为7（因为方块2、5和8与其最终目的地的距离都为1，而方块3和6与目的地的距离

都为2)。实际上，它的确需要移动7步才能将方块布局恢复到已解布局。

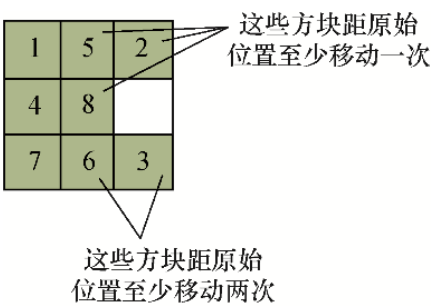


图11-9 一个尚未解决的八数码游戏

既然我们有了八数码游戏的一个启发值，下一步就是把它结合到决策过程。我们知道，人在做决定的时候倾向于选择看起来更接近目标的选项。因此，我们的搜索过程应该考虑搜索树中每个叶节点的启发式，并且从启发值最小的叶节点开始搜索。这就是图11-10中采用的搜索策略，图中给出了用于开发一个搜索树并执行得到的解决方案的算法。

```
创建状态图的开始节点作为搜索树的根节点，并记录它的启发值
while (目标节点还没有到达):
    选择所有叶节点中有最小启发值的最左边的叶节点
    将这个选定的节点作为子节点附加到通过单个产生式能到达的那些节点上
    在搜索树中这个节点的旁边记录每一个新节点的启发值
    从目标节点向上遍历搜索树一直到根节点，把与每个经过的线段相关联的产生式压入栈
    通过执行从栈中弹出的产生式解决原始问题
```

图11-10 采用启发式的控制系统的一个算法

让我们把这个算法用到八数码游戏上，从图11-6给出的初始布局开始。首先，我们建立初始状态并将它作为根节点，记录下它的启发值5。然后，如图11-11所示，while语句的第一次循环添加了3个从初

始状态能达到的节点。注意，我们已经在节点下的括号里记录了每个叶节点的启发值。

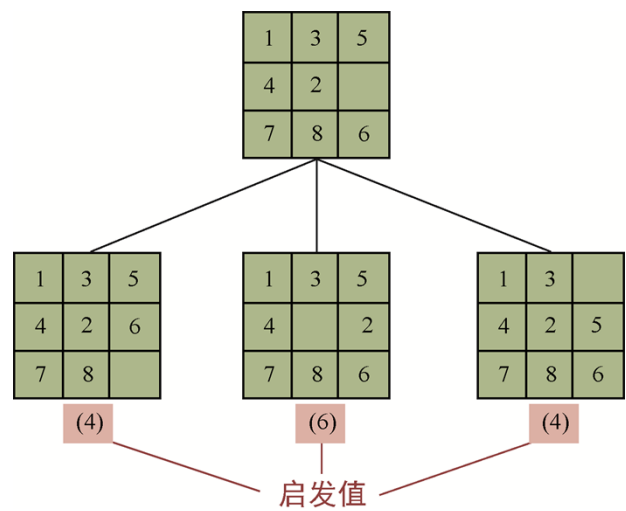


图11-11 我们的启发式搜索的开始

因为没有达到目标节点，所以我们再次执行while语句循环体，这次是从最左边的节点（“有最小启发值的最左边的叶节点”）扩展搜索。结果，搜索树呈图11-12所示的形式。

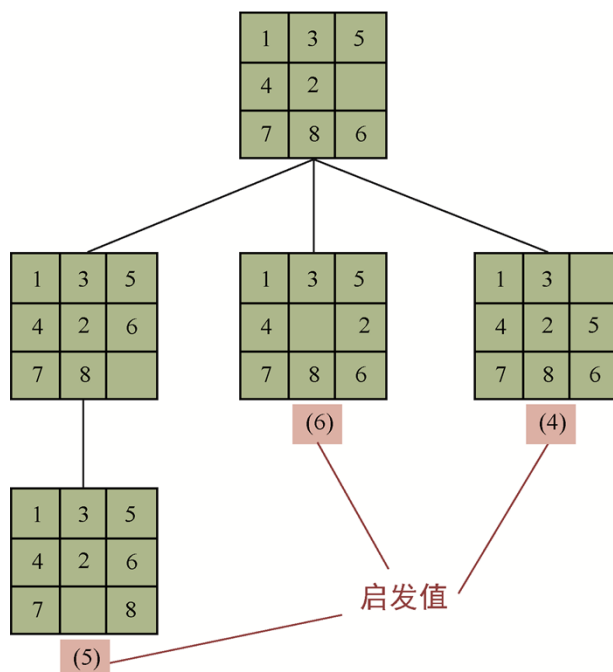


图11-12 两次搜索后的搜索树

现在，最左边叶节点的启发值是5，说明这个分支也许根本不是一个好选择。算法注意到这一点，在下一次循环时，while语句会指示我们从最右边的节点（它现在是“有最小启发值的最左边叶节点”）开始扩展树。这样扩展后的搜索树如图11-13所示。

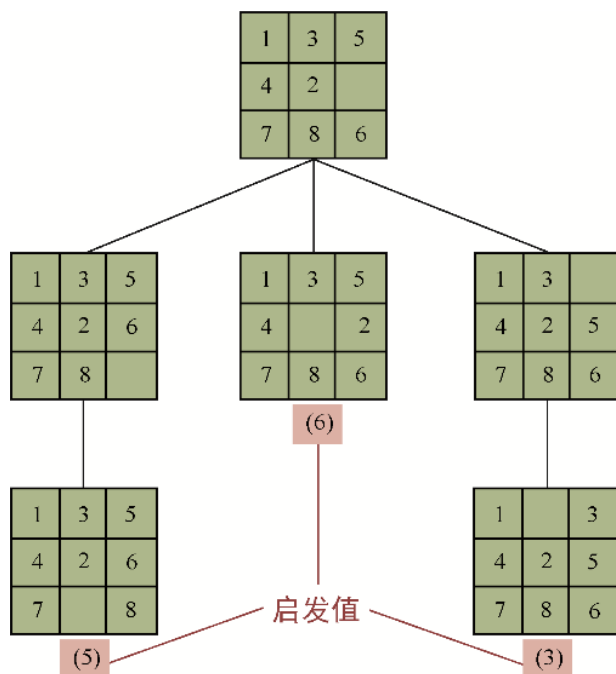


图11-13 3遍搜索后的搜索树

这时，算法好像走上了正轨。因为最右边节点的启发值只有一个3，while语句指示我们继续沿着这条路径进行，搜索直瞄目标，产生图11-14所示的搜索树。这棵树和图11-7中的树的比较表明，尽管新算法早期走了点儿弯路，但启发式信息的使用还是大大减小了搜索树的规模，并形成了更加高效的过程。

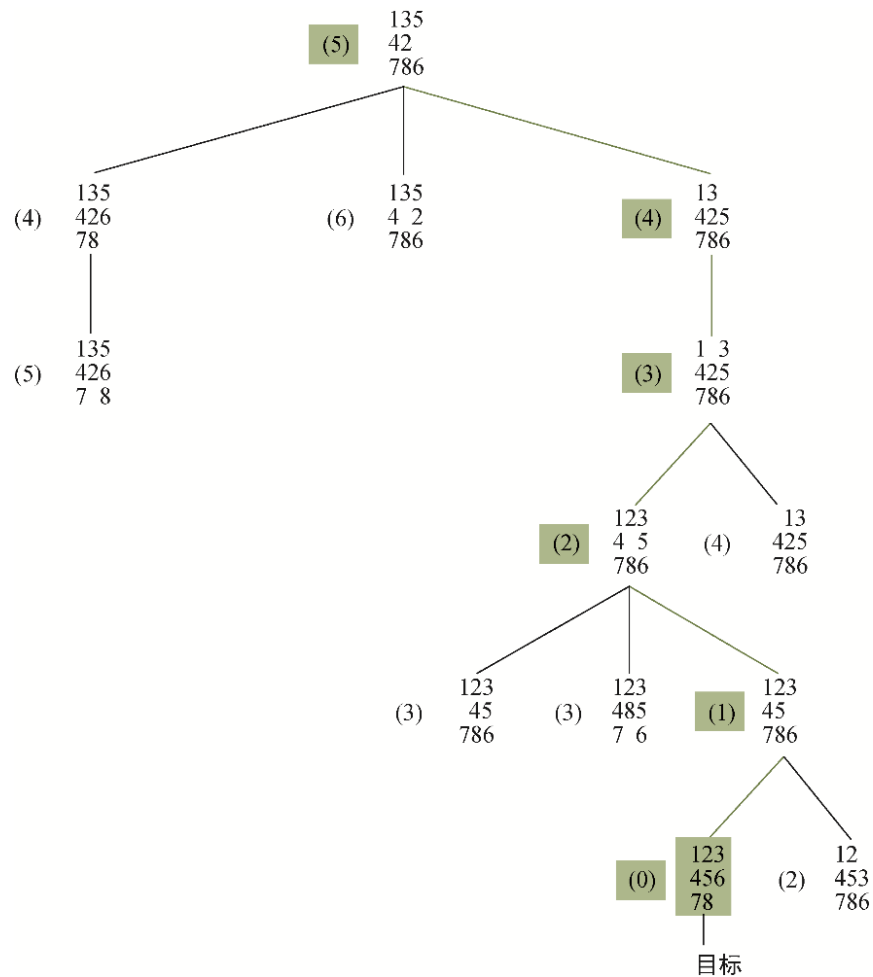


图11-14 用启发式系统形成的完整搜索树

在到达目标状态之后，while语句终止，我们继续从目标节点向上遍历到根节点，把沿途遇到的产生式压入一个栈中。结果栈如前面的图11-8所示。

最后，当这些产生式从栈中弹出时，我们得到指示，执行它们。这时，我们可以看到求解八数码问题的机器放下它的指杆，开始移动方块。

人工智能早期工作研究的课题涉及直接编写程序来模拟智能。但是，今天许多人认为，人类的智能并非基于复杂程序的执行，而是在经历了世代进化后形成的简单的刺激反应机能。这种关于“智能”的理论称为“基于行为的智能”，因为“智能的”刺激反应机能似乎是一些行为的结果，这些行为导致某些个体在其他个体遇难时得以幸免并能繁衍后代。

基于行为的智能似乎能回答人工智能范畴的若干问题，例如，为什么基于冯·诺依曼体系结构的机器在计算能力上能轻易地胜过人类，却难以展现常识性的判断力。因此，基于行为的智能有希望成为人工智能研究中的一个重要的影响因素。正如文中描述的那样，基于行为的技术已经应用在训练神经元如何按所期望的方式表现的人工神经网络领域、较传统的程序设计过程提供替代方法的遗传算法领域，以及通过反应策略改进机器性能的机器人领域。

关于启发式搜索的最后一点说明是按顺序。我们在本节中提出的算法通常称为最佳适配算法，但它并不能保证找到所有应用的最佳解决方案。例如，当使用汽车的全球定位系统（GPS）寻找到达某一城市的路线时，人们通常希望找到最近的路线，而不是任意一条路。**A*算法**（读作“A星算法”）是最佳适配算法的修改版，它可找到一个最优方案。这两种算法的一个主要区别是，除启发值外，A*算法还会考虑在选择下一个要展开的节点时到达每一个叶节点的“累计成本”。（对于汽车的GPS，这一成本是GPS从其内部数据库中获得的行进距离。）因此，A*算法的决策基础是完整潜在路径的成本估计，而不仅仅是剩余成本的预测。

11.3节问题与练习

1. 产生式系统在人工智能中有什么重要意义？
2. 画出八数码游戏中，围绕着代表下图状态的节点的那部分状态图：

4	1	3
	2	6
7	5	8

3. 使用广度优先方法，画出以下图为开始状态求解八数码游戏时控制系统构建的搜索树：

1	2	3
4	8	5
7	6	

4. 用笔、纸以及广度优先方法，构建出以下图为开始状态求解八数码游戏时所产生的搜索树（不必做完），你会碰到什么问题？

4	3	
2	1	8
7	6	5

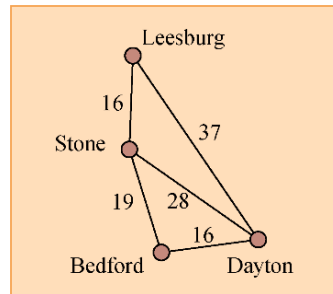
5. 登山者为到达顶峰，只考虑局部地形，并总是沿着最陡峭的上坡行进，解决八数码游戏的启发式系统与登山者之间有什么相似之处？

6. 利用本节所讲的启发式方法，采用图11-10所示的最佳适配算法，解决下面的八数码游戏：

1	2	3
4		8
7	6	5

7. 改进我们计算八数码问题中一个状态的启发值的方法，使图11-10所示的搜索算法不会做出错误的选择，就像在本节中的例子那样。你能举出一个改进的启发值仍然会导致搜索“误入歧途”的例子吗？

8. 下面是寻找从Leesburg到Bedford的路线时由最佳适配算法得到的搜索树（见图11-10）。这一搜索树中的每个节点都是地图上的一座城市，从代表Leesburg的节点开始。当扩展一个节点时，仅添加与正被扩展的城市直接相连的城市。在每个节点中记录它到Bedford的直线距离，并将此距离用作启发值。通过最佳适配算法找到的解决方案是什么？到的解决方案是最短路线吗？



从以下城市到Bedford的直线距离:

Dayton	16
Leesburg	34
Stone	19

9. A*算法从两个主要方面修改了最佳适配算法。首先，A*算法记录了到达每个状态的实际成本。对于地图上的路线，实际成本便是行进的距离。其次，当选择一个要扩展的节点时，A*算法选择其实际成本与启发值之和最小的节点。请根据这两个修改，绘制第8题的搜索树。在每个节点中记录行进至这一城市的距离、到达目标城市的启发值及它们的和。找到的解决方案是什么？找到的解决方案是最短路线吗？

11.4 其他研究领域

在本节中，我们探究一直在挑战人工智能领域研究人员的主题：知识处理、学习以及处理非常复杂的问题。这些活动对人脑来说似乎很简单，但对机器来说就明显负担沉重。目前，开发“智能”智能体方面的许多进展基本都是通过避免直接面对这些问题（或许通过应用巧妙的简化方法或限制问题出现的范围）来实现的。

11.4.1 知识的表达和处理

在关于感知的讨论中，我们看到：理解图像需要大量的关于图像中事物细节的知识，理解一句话的意思可能要依赖其所处的上下文。这些都是知识仓库起作用的例子，知识仓库经常称为**真实世界的知识**（real-world knowledge），是由人脑维护的。不知何故，人能存储大量信息，并以非凡的效率从这些信息中汲取有用的信息。给机器赋予这种能力是人工智能面临的一个主要挑战。

同时，现代计算机系统擅长存储和检索海量信息——任何可以离散化、数字化以及存储为1和0的数据。虽然数据和知识之间的确切关系既复杂又难捉摸，但是随着巨量计算机可访问数据的增长，增加了从所谓的“大数据”系统中提取新信息和知识的机会。软件工具在人工智能、机器学习和统计技术的帮助下，可以发现以前在大型数据集中看不见的模式或连接。

基本知识点

- 大型数据集为提取信息和知识提供了机会和也提出了挑战。
- 大型数据集提供了确定趋势、建立数据连接和解决问题的机会。
- 计算工具有助于发现大型数据集内部信息的连接。

根本目标是找到表示和存储知识的方法。这被我们已经看到的知识以陈述性和过程性出现的事实复杂化了。因此，知识表示不只是事

实或数据的表示，而是包含更广泛的范围。因此，能否最终找到一种用来表示所有形式知识的单一方案是值得怀疑的。

然而，问题不仅仅是知识表示和知识存储，知识还必须是容易理解的，但获取这种理解是一个挑战。11.2节介绍的语义网经常被用作知识表示和知识存储的手段，但是从中提取信息会有问题。例如，句子“Mary hit John.”的意思取决于Mary和John的相对年龄（是2岁和30岁还是相反？）。这个信息将会被存储在图11-3给出的完整的语义网中，但是在上下文分析期间提取这种信息需要对语义网进行大量的搜索。对于极大数据集，无论信息是如何组织的，高效的搜索算法或启发式都是必解的难题。

基本知识点

- 搜索工具是高效查找信息必不可少的。

处理知识获取还有另一个问题是，知识的确定不是明确的，而是含蓄的，是与手头的任务相关的。相对于直接用一个“没有”来回答问题“亚瑟赢了比赛吗？”，我们希望系统可以回答“没有，他因为流感病倒了，没能参加比赛。”然而，任务不只是获取相关信息。我们需要系统能够区分有关系的信息和相关联的信息。例如，“没有，他是1月份出生的，他妹妹的名字叫丽莎”这样的回答对前面的问题来说是没有意义的，即使该信息在某个方面是相关的。因此，高效获取知识不只是搜索问题，还是信息过滤或模式识别问题。

基本知识点

- 信息过滤系统是查找信息和识别信息模式的重要工具。

开发更好的知识提取系统的另一种方法是向提取过程插入各种各样的推理，结果产生了所谓的**元推理**（meta-reasoning）的方法，元推理即关于推理的推理。最初用于数据库搜索的一个示例是应用**封闭世界假设**（closed-world assumption）。这种假设先假设一条语句为假，除非它能够从可用的信息中明确得出。例如，封闭世界假设允许数据库在根本不包含任何关于Nicole的信息的情况下得出“Nicole Smith没有订阅特定杂志”这样的结论。过程就是观察到Nicole Smith不在订阅列表中，然后应用封闭世界假设得出Nicole Smith没有订阅的结论。

封闭世界假设表面上看起来是可行的，但是结果表明，单纯的元推理技术可能会产生微妙的不良影响。例如，假设我们仅有的知识是一条语句：

Mickey is a mouse OR Donald is a duck.

只看这个句子，我们无法推断出Mickey实际上是一只老鼠。因此，封闭世界假设会迫使我们得出语句

Mickey is a mouse.

为假的结论。以相似的方式，封闭世界假设会迫使我们得出语句

Donald is a duck.

为假的结论。因此，封闭世界假设导致我们得出矛盾的结论：尽管至少有一条语句必须为真，但两条语句都为假。理解这种看似单纯的元推理技术的结果是人工智能和数据库这两个领域研究的一个目标，它也强调了智能系统开发所涉及的复杂性。

最后，有一个在变化的环境中保持存储知识最新的问题，称为**框架问题**（frame problem）。如果一个智能体打算使用其知识来决定其行为，那么该知识必须是最近的。但是，支持智能行为所需知识的数量是庞大的，在变化的环境中维护这些知识是一项繁重的工作。一个复杂的因素是，在一个环境中发生的改变经常会间接地改变信息中的其他细节，而且说明这种间接影响的结果是很困难的。例如，如果一个花瓶被撞碎了，那么你对这个情况的了解将不再包括水在花瓶中的事实，尽管水洒了只是打碎花瓶间接造成的。因此，解决框架问题不仅需要以一种有效的方式来存储和获取海量信息的能力，还需要存储系统能够对间接结果有正确的反应。

11.4.2 学习

除了表示和处理知识，我们还希望赋予智能体获取新知识的能力。我们总是可以通过编写和安装新程序或者显式地向存储数据中添加新知识来“教”基于计算机的智能体，但是我们更希望智能体能够自己学习。我们希望智能体能够适应环境的变化并完成我们无法预先简单地编写程序就能够完成的任务。为家务劳动设计的机器人将要面对

新家具、新设备、新宠物甚至新主人，自主自动驾驶汽车必须能适应道路边界线的变化，博弈智能体应该能够制订和应用新策略。

对**机器学习**（machine learning）方法进行分类的一种方式是根据所需的人为干涉的级别进行分类。第一级是通过**模仿**（imitation）学习，在这一级人直接演示任务的步骤（方法可能是执行一系列的计算机操作或者通过一系列动作移动机器人），计算机只需记录这些步骤。这种形式的学习在电子表格和字处理程序等应用程序中已经用了很多年了，这些应用程序记录频繁发生的命令序列，然后通过一个请求就可以重放。注意，通过模仿学习，智能体承担的责任很少。

学习的下一层是**监督训练**（supervised training）。在监督训练中，人对一连串的示例确定正确的反应，然后智能体对这些示例进行归纳，开发出一种适用于新案例的算法。这一连串的示例称为**训练集**（training set）。监督训练的典型应用包括学习识别一个人的笔迹或声音，学习区分垃圾邮件和受欢迎的邮件，以及学习如何通过一组症状判断疾病。

学习的第三层是**强化**（reinforcement）。在强化学习中，智能体能通过对一个任务的反复试验的成功与失败得到一个一般规则，指导自己进行判断。强化学习对于学习如何玩国际象棋或西洋跳棋这样的游戏是很有帮助的，因为胜负容易界定。相对于监督训练，当智能体学习改善自身的行为时，强化学习允许智能体自主地做出反应。

一类称作无监督训练的机器学习技术与强化学习相似，但目标不同。聚类算法、主成分分析和无监督神经网络（将在下一节中描述）是几个可用于在人为干预最少的情况下从数据中推断模式或含义的方法示例。

因为还没有发现覆盖所有可能的学习行为的通用的学习规则，所以学习仍然是一个有挑战性的研究领域。不过，有大量的例子说明人们已经取得了进展，其中一个就是在卡内基梅隆大学开发的[基于神经网络的陆地自动驾驶汽车](#)（Autonomous Land Vehicle in a Neural Net，ALVINN）系统，该系统学习如何驾驶一辆配有一台车载计算机的厢式车，车载计算机使用一台摄像机作为输入。这里所采用的方法就是监督训练。ALVINN从人类驾驶员那里收集数据，并利用这些数据调整自己的驾驶决策。通过学习，该系统可以预测向哪里驾驶，对照人类驾驶员的数据来检查自己的预测，然后修改自己的参数以更接近人类的驾驶选择。ALVINN获得了很大的成功，它能以每小时约120公里的速度驾驶厢式车，同时带动了其他方面的研究，已经产生了可以成功在道路上高速行驶的控制系统。

逻辑程序设计中的知识

知识表示和存储中很重要的一点就是，要采用与存取知识的系统相兼容的方式来实现知识表示和存储。只有这样逻辑程序设计（见6.7节）通常才是有利的。在这样的系统中，知识用

Dumbo is an elephant.

和

X is an elephant implies X is gray.

这样的“逻辑”语句来表达。这样的语句可以用容易应用推理规则的符号系统来表示。反过来，我们在图11-5所示的演绎推理序列就可以采用直接的方式实现。因此，在逻辑程序设计中，知识的表达和存储与知识的提取和应用过程很好地整合在了一起。可以说，逻辑程序设计系统为知识的存储和应用提供了完美的衔接。

最后，我们应该认识一个与学习密切相关的现象——发现。二者的区别是，学习是“基于目标的”而发现不是。术语发现含有意料之外的意思，不是现有的可以学习的。我们可以着手去学习一门外语或学习驾驶汽车，但我们可能会发现那些任务比我们想象得难。探索者可能会发现一个大湖，但目标仅仅是了解那里究竟有什么。开发具有有效发现能力的智能体，要求该智能体能够识别潜在的富有成效的“思维训练”。这里，发现严重依赖推理能力及启发式的使用。此外，许多用于发现的潜在应用要求智能体能够区分有意义的结果和无意义的结果。例如，数据挖掘智能体不应该报告它发现的每一个不重要的关系。

计算机发现系统中成功的例子包括以哲学家弗朗西斯·培根爵士（Sir Francis Bacon）命名的Bacon，它已经能发现（或许应该说“重新发现”）电学上的欧姆定律、行星运动的开普勒第三定律和动量守恒定律。更有说服力的可能是AUTOCLASS系统，它采用红外光谱数据，

已经发现了天文学中以前未知的新型恒星，这是由计算机完成的一个真实的科学发现。

11.4.3 遗传算法

A*算法（见11.3节）可以找到许多搜索问题的最优解，但有一些问题因太过复杂无法通过这些搜索技术求解（当执行会超出可用内存或者无法在合理时间内完成时）。对于这些问题，有时可以通过一个涉及多代试探解的演进过程获得解。这一策略是**遗传算法**（genetic algorithm）的基础。从本质上来说，遗传算法通过将随机行为与繁殖理论的模拟和自然选择的进化过程相结合来发现解。

遗传算法首先生成试探解的一个随机池，其中的每个解都只是一种猜测。（对于八数码游戏，试探解可以是方块的一个随机移动序列。）每个试探解称为一个**染色体**（chromosome），而构成染色体的每个独立个体称为**基因**（gene）——对应八数码游戏中的一次方块移动。

因为每条初始染色体都是一个随机猜测，所以它不太可能表示手头问题的一个解。因此，遗传算法会继续生成一个新的染色体池，其中每条染色体是前一个池中两条染色体（双亲）的后代（孩子）。这些双亲是从池中随机选择的，而这个池子也会概率性地优先选择那些似乎最有可能生成解的染色体，于是这就模拟了适者生存的进化原则。（确定哪些染色体是最佳的双亲身份候选人，可能是遗传算法过

程中最困难的一步。) 每个后代都是双亲基因随机组合的产物。此外，最终的后代偶尔可能以某种随机的方式变异（如交换两次移动）。我们希望，通过反复重复这一过程，能够演变出越来越好的试探解，直到找出非常好的（如果不是最佳的）试探解。遗憾的是，遗传算法并不保证最终一定能够找到解，但研究表明，遗传算法可以有效解决种类繁多的复杂问题。

当被用于程序开发的任务时，遗传算法方法称为**进化规划**（evolutionary programming）。这里的目标是通过模拟进化过程开发程序，而不是直接编写程序。研究人员已经使用函数式程序设计语言将进化规划技巧应用于程序开发过程。运用这个方法，要从建立一个包含各式各样的函数的程序集合开始。这个开始集合中的函数构成了“基因池”，而之后的各代程序将通过“基因池”来构建。接下来，我们可以允许进化过程展开并延续许多代，期望每次通过上一代的最佳组合生成新一代，从而让目标问题的解逐步演进。

11.4节问题与练习

1. 术语**真实世界的知识**是什么意思？它在人工智能领域有何重要意义？
2. 一个关于杂志订阅者信息的数据库通常包含一个每种杂志订阅者的列表，但是不包含那些没有订阅的人的列表。那么，这种数据库如何判断一个人没有订阅一种特定的杂志呢？
3. 概述框架问题。

4. 给出训练一台计算机的3种方式。哪一种不涉及直接的人为干预？
5. 进化技术如何区别于较传统的问题求解技术？

11.5 人工神经网络

尽管人工智能取得了所有这些进步，但是这个领域里的许多问题仍然让使用传统算法的计算机负担沉重。指令序列的感知和推理能力水平看来不能与人的大脑相匹敌。由于这个原因，许多研究人员的目标正在转向利用自然界中观察到的现象的方法。这种方法中一种是11.4节介绍的遗传算法，另一种是人工神经网络。

11.5.1 基本特性

人工神经网络提供了模仿活体生物系统中神经网络的计算机处理模型。一个生物神经元是一个单个的细胞，它有一些称为树突的输入触角和一个称作轴突的输出触角（见图11-15）。经由一个细胞的轴突传递的信号反映了细胞是处于抑制状态还是处于兴奋状态。这种状态由细胞的树突收到的信号的组合来决定。这些树突通过称为突触的小间隙从其他细胞轴突中获得信号。研究表明，突触的传导性是受突触的化学成分控制的。也就是说，特定的输入信号对神经元是有兴奋作用还是有抑制作用由突触的化学成分决定。因此，可以认为，生物神经网络是通过调整神经元之间的这些化学连接来学习的。

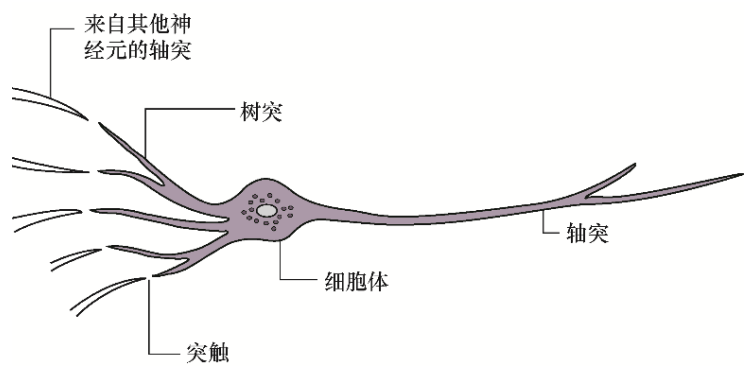


图11-15 活体生物系统中的一个神经元

人工神经网络中的一个神经元是模仿对生物神经元这个基本了解的一个软件单元。它产生输出0还是输出1，取决于其有效输入是否超过了一个给定的值——这个给定的值称为神经元的**阈值**（threshold value）。如图11-16所示，这个有效输入是许多实际输入的一个加权和。图中，神经元由椭圆表示，两个神经元之间的连接由箭头表示。从其他神经元（记为 v_1 、 v_2 和 v_3 ）的轴突获得的值被用作所描述神经元的输入。除了这些值，每个连接都与关联一个**权**（weight）（记为 w_1 、 w_2 和 w_3 ）。神经元把每个输入值与关联的权相乘，再把这些乘积相加形成有效输入（ $v_1w_1 + v_2w_2 + v_3w_3$ ）。如果这个和超过该神经元的阈值，那么该神经元就产生一个输出1（模拟神经元的兴奋状态）；否则就产生一个输出0（模拟神经元的抑制状态）。

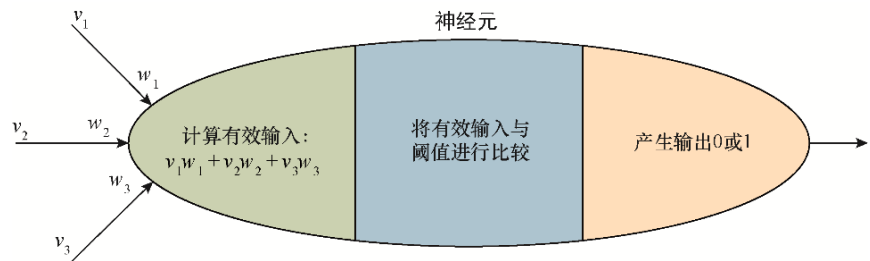


图11-16 一个神经元中的活动

如图11-16所示，我们用椭圆形作为表示神经元的约定符号，其中每个输入连接一个神经元，并在椭圆内写上与这个输入关联的权，最后，在椭圆的中央写上这个神经元的阈值。图11-17中的例子表示了一个有3个输入且阈值为1.5的神经元，与每个输入连接关联的权分别为-2、3和-1。因此，如果神经元接收的输入分别为1、1和0，那么其有效输入为 $(1) \times (-2) + (1) \times (3) + (0) \times (-1) = 1$ ，所以其输出为0。但是，如果神经元接收的输入分别为0、1和1，那么其有效输入为 $(0) \times (-2) + (1) \times (3) + (1) \times (-1) = 2$ ，这超出了阈值，所以神经元的输出为1。

权可以为正也可以为负，这一事实意味着相应的输入对接收神经元的作用可以是兴奋也可以是抑制。（若权为负，接收的输入为1就减少了加权和，故有效输入倾向低于阈值；相反，正的权使相应输入对加权和起增加作用，故增加了加权和超过阈值的机会。）此外，权的实际大小控制了相应输入单元对接收单元起抑制作用或是兴奋作用的程度。因此，通过调节整个人工神经网络中的权值，我们就能够对网络编程，以预定的方式对不同的输入做出响应。

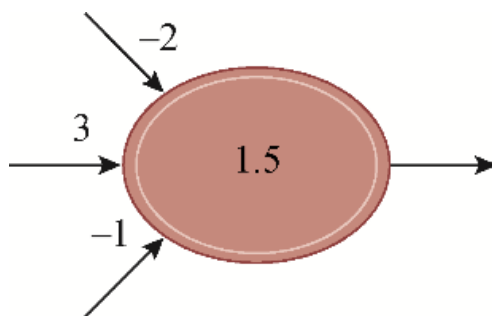


图11-17 一个神经元的表示

人工神经网络通常按拓扑结构分层排列。输入神经元位于第一层，输出神经元位于最后一层，其他神经元层（称为隐藏层）可以包含在输入层与输出层之间。一个层中的每个神经元与随后层中的每个神经元互相连接。图11-18给出了一个简单的神经网络的例子。将图11-18a中展示的简单网络编程为：若两个输入不同，则产生输出1；否则产生输出0。但是，如果将权改成图11-18b所示的权，那么我们将获得一个网络，如果网络的两个输入均为1，则网络响应1，否则响应0。

我们应该注意，图11-18中的神经网络配置与实际的生物神经网络相比实在是太过简单。一个人的大脑大约包含 10^{11} 个神经元，每个神经元约有 10^4 个突触。事实上，一个生物神经元的树突非常多，以至于它们看起来更像一个纤维网格，而不像图11-15中所表示的一个个触角。

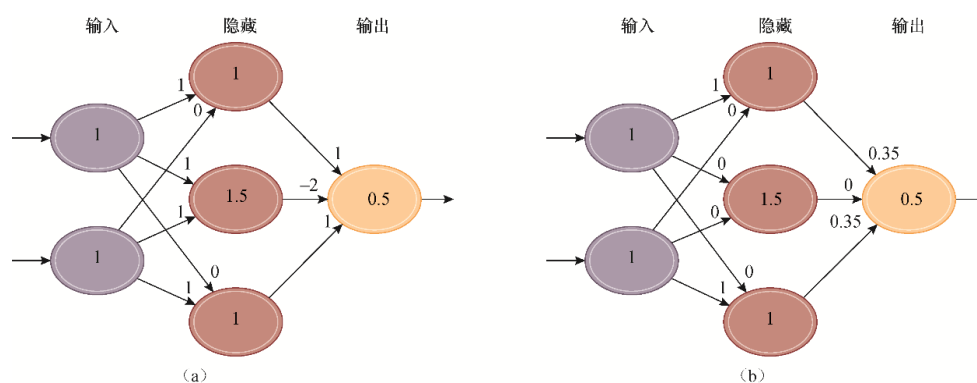


图11-18 有两个不同程序的神经网络

11.5.2 训练人工神经网络

人工神经网络的一个重要特征是，它们不再是通过传统意义上的编程来实现，而是通过训练来实现。也就是说，不再由程序员决定解决一个特定问题所需的权值，并把这些值“插”入网络中；相反，由人工神经网络通过监督训练学习获得合适的权值（见11.4节），监督训练涉及一个反复的过程，在这个过程中来自训练集的输入被应用到网络，然后用小的增量调整权，以便网络的性能接近期望行为。

值得注意的是，遗传算法技术是如何应用在训练人工神经网络的任务上的。具体来说，要训练一个神经网络，可以随机生成许多网络权集，其中每个权集用作遗传算法的一条染色体。接下来，网络会一步一步地被赋予由每条染色体表示的权，并通过各种输入被测试。在这个测试过程中，产生错误最少的染色体更有可能被选为下一代染色体的父母。在大量的实验中，这个方法最终都产生了成功的权集。

来看这样一个例子。在这个例子中，训练人工神经网络来解决问题已经成功，并且可能比尝试通过传统程序设计技术来提供解决方案更具有成效。问题是，机器人在尝试通过其自身摄像头收到的信息理解其所处环境时可能会面临的问题。例如，假设机器人必须区分一个房间的白色墙面和黑色地面。乍一看，这似乎是一个很简单的任务：只需简单分类，将白色像素归为墙面的一部分，将黑色像素归为地面的一部分。然而，当机器人从不同的方向观看或者在房间中走来走去时，在某些情况下不同的光照条件会使墙面呈现灰色，而在其他情况下地面可能会呈现灰色。因此，机器人需要学会在各种各样的光照条件下区分墙面和地面。

要做到这一点，我们可以构建一个人工神经网络，其中输入由表明图像中单个像素颜色特征的值和表明整个图像总亮度的值组成。构建完成之后，我们可以通过为这一网络提供代表各种光照条件下的墙面部分和地面部分的像素的许多示例，并以此来训练这一网络。

除了简单的学习问题（如像素分类），人工神经网络已用于学习复杂的智能行为，11.4.2节引用的ALVINN项目就是一例。实际上，ALVINN是一个人工神经网络，如图11-19所示，其构成出奇的简单。ALVINN从 30×32 的传感器阵列获得输入，其中每个传感器负责观察前方道路视频图像的一个特定部分，并将其发现报告给隐蔽层的4个神经元中的每一个。（因此，这4个神经元中每个都有960个输入。）这4个神经元中每个神经元的输出都与30个输出神经元的每一个相连接，其输出表明了驾驶的方向。这30个神经元形成一行，一端处于兴奋的神经元表示向左急转弯，而另一端处于兴奋的神经元则表示向右急转弯。

ALVINN的训练方式是：在“观察”人类驾驶的同时做出自己的驾驶决定，将自己的决定与人的决定进行比较，并且对其权稍作修改，使其决策更接近人的决定。然而，存在一个有意思的次要问题，尽管ALVINN通过这个简单的技术学习了驾驶，但是它没有学习如何从错误中恢复。因此，从人那里收集的数据也要人为加强恢复状态。（最初考虑的这种恢复训练的一种方法是使人令车辆偏离方向，以便ALVINN可以观察人如何恢复，从而学会如何靠自己恢复。但在人进

行初始的偏离过程时如果不关闭ALVINN，ALVINN除了学习了恢复还学习了偏离——这显然是不想要的特质。)

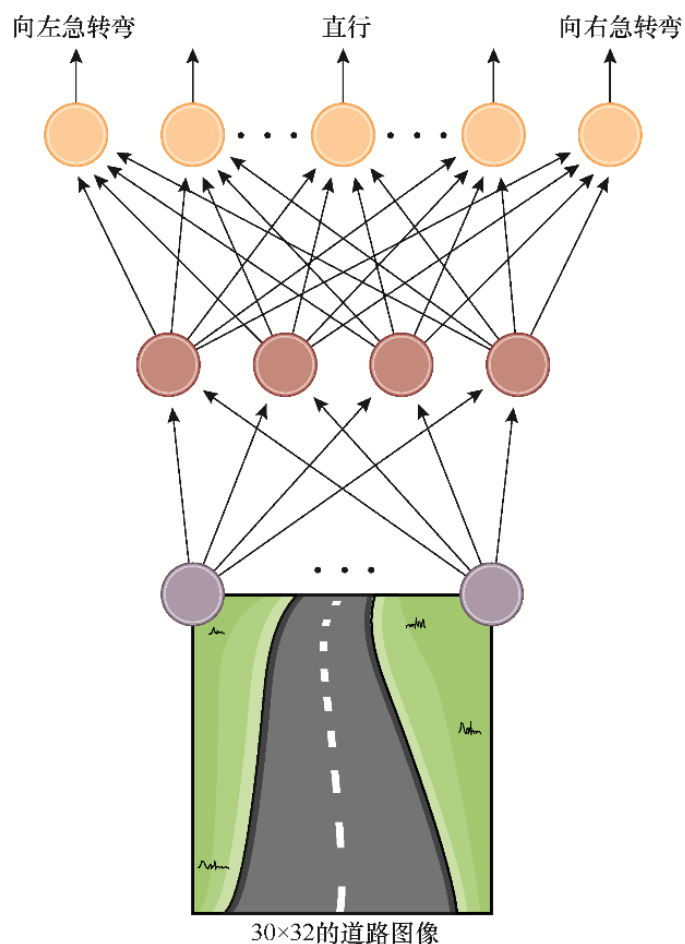
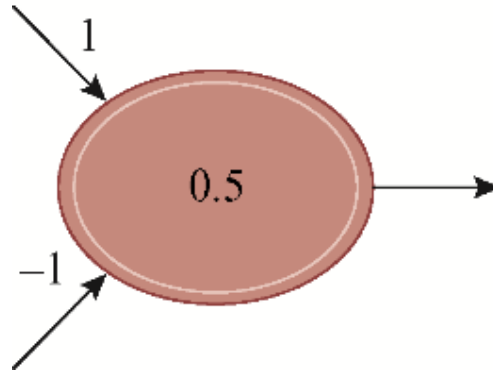


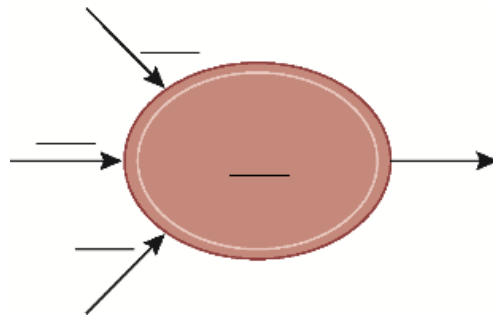
图11-19 ALVINN（带神经网络的自主陆地车辆）的结构。基于迪安·波默洛（Dean A. Pomerleau）的ALVINN结构

11.5节问题与练习

1. 对于下面的神经元，当它的两个输入均为1时，输出是多少？
如果输入模式为0, 0、0, 1和1, 0呢？



2. 调整下列神经元的权和阈值，使得当且仅当至少有两个输入为1时输出1。



3. 确定一个在训练人工神经网络时可能会发生的问题。

11.6 机器人学

机器人学 (robotics) 研究的是具有智能行为的物理上的自主智能体。与所有智能体一样，机器人在所处的环境中必须能够感知、推理和行动。因此，机器人学涵盖了人工智能的所有研究范围，还借鉴了很多机械和电气工程方面的知识。

为了与外界交互，机器人需要用机构来操作目标物体和移动。在早期的机器人学中，该领域与机械手的发展联系紧密，这些机械手通

常是带有肘、腕和手或工具的机械臂。研究不仅涉及这样的装置如何操作，而且涉及如何维护和应用有关它们的位置和方向的知识。（你闭着眼睛也能用自己的手指摸到自己的鼻子，因为你的大脑保存着你的鼻子和手指在什么地方方的记录。）随着时间的推移，机器人的手臂已经变得越来越灵巧，通过基于力反馈的触觉，它们能够成功地握住鸡蛋和纸杯。

最近，更快速、更轻便计算机的发展促进了移动机器人方面的更重大的研究。实现这种灵活性导致了大量富有创意的设计。在机器人运动能力方面，研究人员已经开发出可以像鱼一样游动的机器人、像蜻蜓一样飞的机器人、像蝗虫一样跳的机器人，以及像蛇一样蜿蜒爬行的机器人。

因为轮式机器人相对容易设计和制造，所以非常受欢迎，但是轮式机器人受限于它们可以穿越的地形类型。克服这种限制，组合使用轮子和导轨，使机器人能够爬楼梯或翻越岩石是当前的研究目标。例如，NASA（美国国家航空航天局）的火星漫游者（Mars rover）就是使用特殊设计的轮子在火星的岩石层上行走。

有腿的机器人移动性大大提高，但明显更复杂。例如，被设计成像人一样行走的双足机器人必须持续地监视和调整姿态，否则它就会跌倒。但是，这种困难能够被克服，本田公司开发的具有人类特征的双足机器人Asimo能够上楼梯，甚至能够跑，就是一个例证。

尽管在机械手和运动能力方面取得了巨大的进步，但是大多数机器人仍然不是非常自主的。通常工业机器人手臂是为每项任务严格编程的，工作时不用传感器，假设零件会按照指定的位置精确地传送给它们。其他移动机器人，如NASA火星漫游者和军用无人机（unmanned aerial vehicle，UAV），其智能都是依靠人的操作来实现的。

克服这种对人的依赖是当前研究的一个主要目标。问题涉及一个自主机器人需要知道关于其所处环境的哪些知识，以及需要预先计划其行为到什么程度。一种方法是建造能维护所处环境详细记录的机器人，该记录中包含目标物体及其位置的详细清单，机器利用这个清单就能够制订详细的行动计划。这个方向的研究很大程度上依靠知识表示和知识存储方面取得的进展以及推理和规划技术的改进。

另一个可选择的方法是开发反应式机器人，该方法不用维护复杂的记录，也不用耗费精力去构建详细的行动计划，只要应用简单的与外界交互的规则时时刻刻指导它们的行为即可。反应式机器人技术的支持者认为：当计划一个长途汽车旅行时，人们不会预先制订全面而详细的计划。相反，他们只是选择主要路线，而对于像到哪吃饭、走哪些出口，以及如何绕道行驶等细节则是到时候再考虑。同样，一个反应式机器人若要通过一条拥挤的走廊，或从一栋大楼走到另一栋大楼，也不会预先制订非常详细的计划，而是在遇到障碍时应用简单的规则避开。这是历史上最畅销的机器人——iRobot Roomba真空吸尘器所采用的方法，iRobot Roomba真空吸尘器以反应模式在地面上来回移

动，而不会费心去记家具和其他障碍物的细节。毕竟，这次碰到的家庭宠物下次不可能还待在同一个地方。

当然，没有一种方法可能证明对所有情况都是最好的。真正的自主机器人最有可能使用多层次推理和规划，应用高级技术设定和达到主要目标，应用较低级的反应系统完成次要子目标。这种多层次推理的例子可在机器人世界杯（RoboCup）比赛（机器人足球队的国际性比赛）中找到，该比赛是一个研究论坛，致力于开发到2050年能够对抗世界级人类足球队的机器人足球队。这里，重点是不仅要制造能够“踢”球的移动机器人，还要设计能够相互协作达到共同目标的机器人足球队。这些机器人不仅要能移动并推理自己的行为，还要能推理其队友及对手的行为。

家用机器人



机器人真空吸尘器，如iRobot颇受欢迎的Roomba系列，已成为广泛使用的消费设备。高级模型可以利用光学、红外线、碰撞和跌落检

测传感器的组合，导航通过整个房间的地面，记住它成功绕过家具腿、楼梯间和宠物时的位置。控制机器人的自主智能体软件足够智能，可以在随身携带的垃圾箱已满或电池电量不足时返回其充电站。它的导航算法使用房间长度扫描来覆盖大的开放区域，结合第二遍边缘探索绕着障碍物进行清扫，可能还能找到未到过的房间。

相关的智能手机应用可用于监视机器人的状态、规划自动运行，甚至可用于生成机器人已清洁的地面空间的最终地图，并注明特别脏的地方。

机器人学研究领域的另一个例子是进化机器人学，该领域把进化理论应用于开发低级反应规则所对应的方案和开发高级推理所对应的方案。这里我们发现，适者生存理论被用到了设备的开发上，经过若干代的学习，这些设备就能够获得自己的平衡或移动的方式。这个领域的许多研究各有侧重，其不同之处在于机器人的内部控制系统（主要是软件）及其机体的物理结构。例如，把一个能游泳的蝌蚪机器人的控制系统换在一个有腿的类似的机器人身上，然后在控制系统中应用进化技术，就得到一个能爬行的机器人。还有一些例子中，进化技术已经被应用在机器人的物理形体上，用于发现执行特定任务的最佳传感器位置。更具挑战性的研究正在寻求软件控制系统与身体结构同时进化的途径。

机器人学中令人印象深刻的研究成果举不胜举。目前的机器人与科幻电影和小说中的超能机器人相差甚远，但是在执行特定任务上已经取得了重大成功。我们有能够在交通中行驶的机器人、行为像宠物

狗的机器人，以及引导武器瞄准其目标的机器人。然而，在享受这些成功的同时我们应该注意到，我们对人造宠物狗及智能武器可怕威力的喜爱引发了一系列社会问题和道德问题，这些都向社会发出了挑战。我们的未来是我们自己造就的。

机器人制造史

（a）2010年4月15日，在德国东部马格德堡举行的2010机器人世界杯德国公开赛上，一个足球机器人在踢球（图片来源：Jens Schlueter/GettyImages）。



（b）Tartan Racing团队的赛车“Boss”——DARPA举办的无人驾驶汽车城市挑战赛Urban Challenge的大奖得主（© DARPA）。



(c) NASA的一台探测车——探测火星表面地质情况的机器人
(由NASA/JPL-Caltech提供)。



11.6节问题与练习

1. 机器人行为的反应式方法在何种方式上有别于较传统的“基于计划”的行为？
 2. 机器人学领域当前的研究主题有哪些？
 3. 进化理论被用在了机器人开发的哪两个层次？
-

11.7 后果的思考

毫无疑问，人工智能取得的进步有造福人类的潜力，人们很容易热衷于这些潜在的好处。然而，这也为将来留下了后患，它的破坏性后果与其带来的好处同样巨大。这种差异常常只是一个人的观点或者一个人的社会地位的不同——彼之所得，此之所失。所以花一点时间从另一个角度观察正在进步的技术对我们来说是比较恰当的。

有些人把技术进步看作给予人类的一份厚礼——将人类从枯燥的、单调的任务中解放出来，为更令人愉悦的生活方式打开大门。但对于同一个现象，另一些人则把它看作剥夺公民就业机会、把财富引向权势人物的祸根。实际上，这正是印度忠诚的人道主义者圣雄甘地传递的信息。甘地认为，如果用农民家中的纺车来代替大型纺织工厂，那么印度人的生活将会变得更好。他宣称，通过这种方式，用一个分散的大宗生产系统取代只雇佣少数人的集中式大宗生产，将有利于平民大众。

历史上有很多因财富和权力分配不均而引起的革命。如果允许今天正在进步的技术增强这种差异，可能会导致灾难性的后果。

但是，制造越来越智能的机器的后果，比那些涉及不同社会阶层之间权力斗争的后果更加微妙，也更加根本。这些问题触及了人类自身形象的核心。19世纪，查尔斯·达尔文的进化论及人类可能由更低等的生命形式进化而来的想法震惊了整个社会。那么，面对机器的智力向人类智力挑战的冲击，社会会如何应对呢？

过去，技术发展缓慢，让我们有时间重新调整智能的概念，维护人类自身形象。19世纪，我们的老祖宗会认为当时的机械装置具有超自然的能力，而今天我们决不会认为这些机械有什么智能。因此，如果机器真的挑战了人类的智能，或者更有可能的是，机器能力的进步远远超过了我们的适应能力，那么人类会如何应对呢？

考虑一下20世纪中叶社会对当时IQ测试的反应，也许可以从中得到一些线索，看看人类面对挑战我们智能的机器时可能的反应。这些测试被用来确定儿童的智力水平。美国常常依据儿童在这些测试中的表现来对他们进行分类，并据此将他们纳入相应的教育计划。反过来，只向那些在测试中表现好的孩子开放教育机会，而安排那些测试表现差的孩子去参加补习。简而言之，当给出一种衡量个体智能的标准时，社会倾向于漠视被认为是低于这个标准的人的能力。那么，如果机器的“智能”能力已经变得可与人类相匹敌，甚至只是看上去可以相匹敌，社会将怎样应对这种局面呢？抑或社会也漠视那些能力被认为“不如”机器的人吗？如果这样，对于这些社会成员来说，后果会是什么？难道一个人的尊严会受他与机器的比较结果的影响吗？

基本知识点

- 通过计算实现的创新引发了法律和道德关注。

我们已经看到，在一些特定场合，人类的智力正面临机器的挑战。机器现在有能力打败国际象棋专家；计算机化的专家系统能够给出治疗意见；管理证券投资的简单程序常常比投资专家做得更好。这

样的系统是怎样影响所涉及人员的自我形象的呢？随着在越来越多的领域里机器的表现优于人类，个人的自尊心会受到怎样的影响？

许多人认为机器拥有的智能与人类的智能有着本质的区别，因为人是生物，而机器不是。因此，他们认为，机器永远不会重现人类的决策过程。机器也许会做出与人同样的决策，但是机器做出这些决策的基础与人类的不同。那么，在多大程度上存在不同类型的智能？对社会来说，如果按照非人类智能的“思维”方式来发展，是否合乎道德？

Joseph Weizenbaum在他的*Computer Power and Human Reason*一书中坚决反对不加抑制地应用人工智能，他这样写道：

计算机能够做出司法判决，计算机能够做出精神病判定。它们能够以比最有耐心的人更加老练的方式投掷硬币。关键在于不应该给它们这种任务。在某些场合，它们甚至能够得到“正确的”决策——但是其依赖的基础总是而且一定不是人类乐于接受的。

已经有很多有关“计算机与人脑”的争辩。我在这里的结论是，问题的实质不在于技术，甚至也不在于数学，而在于道德。设定计算机去做什么，不能用“能不能”这样的问题，计算机适用性的限制最终只能用“应该”来表达。最基本的认识应该是：因为我们现在还没有办法让计算机有智慧，所以我们现在不应该让计算机去做需要智慧的工作。

也许你会认为本节所述的许多内容近乎科幻小说，而不是计算机科学。就在不久前，许多人因抱有同样的“这永远不会发生”的态度，而拒绝考虑“如果计算机操纵了社会，会发生什么？”这样的问题。但从许多方面来看，这一天已经到来。如果一个计算机化的数据库错误地报告了你有不良的信用评级、有犯罪记录或者银行账户透支，那么是计算机的报告会奏效还是你自己的清白申诉会奏效？如果一个出了故障的导航系统错误地指示了大雾笼罩的跑道的位置，那么飞机将降落在何处？如果一个用机器来预测公众对不同政治决策的反应，那么政治家应该采取何种决策？你遇到过多少次因为“计算机坏了”所以办事员无法为你服务的情形？那么，究竟谁（或什么）掌管着这个社会？难道我们还没有把社会交给机器吗？

11.7节问题与练习

1. 如果去掉过去100年里发明的所有机器，那么今天的人有多少能幸存？如果是过去50年呢？20年呢？幸存者将会在何处？
2. 你的生活被机器控制到了什么程度？谁控制着影响你生活的这些机器？
3. 你从哪里获得用于日常决策的信息？对于你的重大决策呢？对这些信息的准确度你有多少把握？为什么？

复习题

（带*的题目涉及选读章节的内容。）

1. 正如11.2节说明的那样，人类可能会将问题用于提问之外的目的。另一个例子，“你知道你的轮胎瘪了吗？”这个问题是用来提醒的，而不是用来提问的。给出一些用来表达安慰、警告或责备的问题的例子。

2. 把一台汽水自动售货机当作智能体来进行如下分析：它的传感器是什么？它的执行器是什么？它展现了什么级别的响应（反射响应、基于知识的响应、基于目标的响应）？

3. 确定下列每个响应是反射响应、基于知识的响应还是基于目标的响应。证明你的回答。

a. 一个计算机程序把文本从德文翻译成英文。

b. 当室内温度低于当前设定值时，恒温器打开暖气。

c. 一位飞行员驾驶飞机安全地在跑道上着陆。

4. 如果研究人员用计算机模型来研究人脑的记忆能力，那么为机器开发的程序必须记住机器的最佳能力吗？请解释原因。

5. 举出几个陈述性知识的例子。举出几个过程性知识的例子。

*6. 在面向对象程序设计的环境中，对象的哪些部分用来存储陈述性知识？哪些部分用来存储过程性知识？

7. 下列活动中，你认为哪些是面向性能的？哪些是面向模拟的？

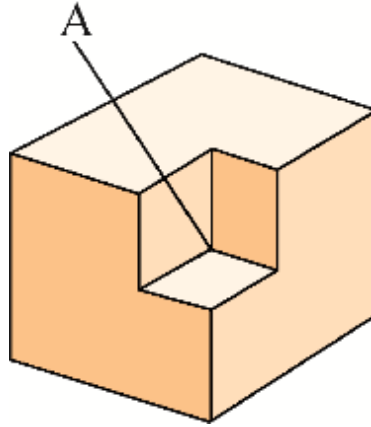
- a. 自动往返系统的设计（通常用在机场两个航站楼之间）。
- b. 用于预测台风路径的模型的设计。
- c. 用于提取和维护存储在万维网上的文档索引的Web搜索数据库的设计。
- d. 用于测试理论的国家经济模型的设计。
- e. 用于监视病人生命体征的程序的设计。

8．当今，某些打给企业的电话都由自动应答系统来处理，该系统利用语音识别与打电话的人进行通话。这些系统通过图灵测试了吗？请解释你的答案。

9．确定一个可用于区分符号F、E、L和T的小型几何特征集。

*10．描述通过将特征与模板进行比较来识别特征的技术与第1章中讨论的利用纠错码识别特征的技术之间的相似之处。

11．根据下面线条画中标记为A的“角落”是凸起的还是凹下的来描述这个图的两种解释。



12 . 比较下列两个句子中介词短语的作用（仅有一个词不同）。
如何对机器进行编程，让它做这样的区分？

The pigpen was built by the barn.

The pigpen was built by the farmer.

13 . 下面两个句子的语法分析结果有什么不同？语义分析结果有什么不同？

An awesome sunset was seen by Andrea.

Andrea saw an awesome sunset.

14 . 下面两个句子的语法分析结果有什么不同？语义分析结果有什么不同？

If $X < 10$ then subtract 1 from X else add 1 from X.

If $X > 10$ then add 1 to X else subtract 1 from X.

15．在文中，我们对照形式化程序设计语言，简要地讨论了理解自然语言的问题。作为自然语言情况下涉及的复杂性的一个示例，确定问题“Do you know what time it is?”有不同含义的情形。

16．一个句子上下文的改变能够改变这个句子的含义以及意思。在图11-3的上下文中，如果两个人都出生在21世纪晚期，那么句子“Mary hit John.”的意思会如何变化？如果一个出生在20世纪80年代，而另一个出生在21世纪晚期，含义又会如何改变？

17．画一个语义网，把下列段落的意思表示出来。

Donna threw the ball to Jack, who hit it into center field. The center fielder tried to catch it, but it bounced off the wall instead.

18．有时候回答问题的能力既取决于本身，也取决于对知识的局限性的了解。例如，假设数据库A和B都包含公司健康保险计划的一个完整的雇员名单，但只有数据库A知道名单是完整的。那么关于一个不在名单里的员工，数据库A能够推断出的什么信息是数据库B推断不出来的？

19．举出一个封闭世界假设导致矛盾的例子。

20．举出两个共用一个封闭世界假设的例子。

21．在产生式系统上下文中，状态图和搜索树有什么区别？

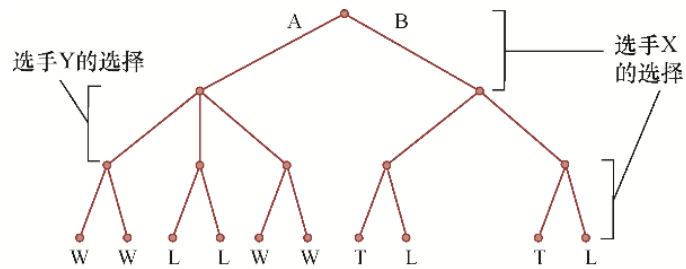
22．依照产生式系统分析解决魔方问题的任务。（状态、产生式等都是什么？）

23．a．假定搜索树是一棵二叉树，达到目标需要8个产生式。如果树是以广度优先的方式构建的，那么当达到目标状态时，树中的节点数最大是多少？

b．解释如何通过同时进行两个搜索（一个从初始状态开始，另一个从目标状态反向进行）直到这两个搜索会合来减少搜索过程中要考虑的节点总数。（假定记录反向搜索过程中发现的状态的搜索树也是一棵二叉树，并且两个搜索以相同的速率进行。）

24．在文中我们提到，产生式系统经常被用作从已知事实中得出结论的一种技术。系统的状态是推理过程的每个阶段已知为真的事实，产生式是用于操控已知事实的逻辑规则。标识几个逻辑规则，允许从事实“John is a basketball player”“Basketball players are not short”和“John is either short or tall”中得出“John is tall”的结论。

25．下面的树表示一个竞赛游戏中可能的移动，选手X当前可在移动A和移动B中选择其一。选手X移动后，选手Y才能选择移动，然后由选手X选择最后一步移动。树的叶节点标记为W、L还是T取决于结局表示的是选手X最后是赢、输还是平局。选手X应选择移动A还是移动B？为什么？在竞争氛围中选择“产生式”和八数码游戏这样的单人游戏中选择有什么不同？



26．按照产生式系统分析西洋跳棋游戏，并描述一个用于确定两个状态中哪一个更接近目标的启发式。这种情况下的控制系统与八数码游戏这样的单人游戏的控制系统有什么区别？

27．把代数定律看作产生式，代数表达式简化的问题就能在产生式系统的上下文中解决。确定一组代数产生式，使等式 $3/(2x-1)=6/(3x+1)$ 可简化为 $x=3$ 的形式。当进行这种代数简化时，经验法则（即启发法则）是什么？

28．不用任何启发信息的辅助，画出利用广度优先搜索方法解决下面的初始状态的八数码游戏时生成的搜索树。

	1	3
4	2	5
7	8	6

29．利用图11-10中的最佳适配算法尝试解决第28题中的八数码游戏，用不在正确位置的方块的数目作为启发式，画出这种情况下生成的搜索树。

30．利用图11-10中的最佳适配算法尝试解决下列初始状态的八数码游戏，假定所用的启发式与11.3节中开发的一样，画出这种情况下生成的搜索树。

1	2	3
5	7	6
4		8

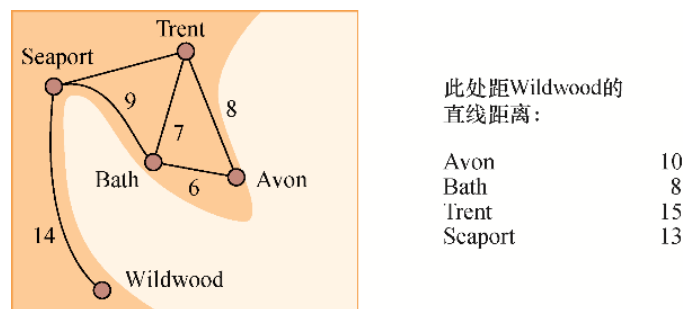
31．当解决八数码游戏时，为什么用不在正确位置的方块的数目作为启发式不如11.3节中用的那种好？

32．执行二叉树搜索（见5.5节）时决定考虑哪一半列表的技术，和执行一个启发式搜索时决定执行哪个分支的技术，二者有什么不同？

33．注意，如果一个产生式系统的状态图中有一个状态的启发值与其他状态的相比极低，并且如果从这个状态到其自身有一个产生式，那么图11-10中的算法会陷入一遍又一遍地考虑这个状态的循环。证明：如果执行该系统中任何产生式的代价至少为1，那么把启发值加上沿遍历路径到达该状态的代价，通过这样计算规划代价，就可以避免这种无限循环。

34．在一幅大的道路图上搜索两个城市间的路线时，你会用什么启发式？

35．请看在寻找从Trent到Wildwood的路线时，根据图11-10中的最佳适配算法得出的四层搜索树。这一搜索树中的每个节点表示地图上的一个城市。开始节点为Trent。当扩展一个节点时，仅添加与正被扩展的城市直接相连的城市。在每个节点中记录到Wildwood的直线距离，并将此距离用作启发值。在其处理过程中，最佳适配算法有什么不足之处吗？如果有，应该怎样纠正？



36．A*算法从两个主要方面修改了最佳适配算法。首先，A*算法记录了到达每个状态的实际成本。对于地图上的路线，实际成本便是行进的距离。其次，当选择一个要扩展的节点时，A*算法选择其实际成本与启发值之和最小的节点。请根据这两个修改，绘制第35题的搜索树。在每个节点中记录行进至这一城市的距离、到达目标城市的启发值及它们的和。找到的从Dearborn到Wildwood的路线是什么呢？

37．列出在产生式系统中有用的启发式应该具有的两个特性。

38．假定有两个桶，一个容量正好是3升，另一个容量是5升。任何时候你都可以把水从一个桶倒入另一个桶，把一个桶倒空，或把一个桶倒满。问题是要将正好4升的水倒入5升的那个桶。说明如何将这个问题设计成一个产生式系统。

39．假设你的任务是监督两辆卡车装货，每辆车最多可载14吨货。货物装在不同的板条箱里，总重28吨，但各个箱子的重量不一样。每个箱子的箱边上侧面都标示着箱子的重量。为了在两辆车上分装这些货物，你会采用什么样的启发式？

40．下列哪些是元推理的例子？

- a. 他已经走了很长时间了，所以他一定已经走远了。
- b. 因为我经常做出错误的决定，并且我做的最后两个决定是正确的，所以我将逆转我的下一个决定。
- c. 我有些疲倦了，所以我可能不会清晰地思考。
- d. 我有些疲倦了，所以我想我要打个盹儿。

41．描述人类解决框架问题的能力是如何帮助人类找到丢失的物品的。

42．a. 模仿学习与监督训练在何种意义上相似？

b. 模仿学习与监督训练在何种意义上不同？

43．调整图11-18所示的人工神经网络中的权和阈值，使其在两个输入相同（全为0或全为1）时输出1，两个输入不同（一个为0另一个为1）时输出0。

44．画一个与图11-5类似的图，表示把代数表达式 $7x+3=3x-5$ 简化为表达式 $x=-2$ 的过程。

45．扩展第44题的答案，说明尝试解题时控制系统可能遵循的其他路径。

46．画一个与图11-5类似的图，表示从初始事实“Polly is a parrot”“A parrot is a bird”和“All birds can fly”中得到结论“Polly can fly”的推理过程。

47．与第46题中的句子不同，有些鸟不会飞，如鸵鸟或折了翅膀的知更鸟。但是，要建立一个演绎推理系统，其中把对陈述“All birds can fly”的所有例外都明确列出，看来并不合理。那么，我们人类是如何确定一只鸟能飞还是不能飞的呢？

48．解释句子“I read the new tax law”在不同上下文中的不同含义。

49．说明如何把从一个城市旅行到另一个城市的问题设计成一个产生式系统。状态是什么？产生式是什么？

50．假设你必须执行3个任务A、B和C，可以以任意次序执行（但不能同时执行）。说明如何把这个问题设计成一个产生式系统，并画出其状态图。

51．如果第50题中的任务C必须在任务B之前执行，那么怎样改变状态图？

52 . a. 如果用 (i, j) 来表示“若一个列表中第 i 个位置的项大于第 j 个位置的项，则把两项交换”，其中 i 和 j 为正整数，那么下面两个序列中哪一个能更好地完成一个长度为3的列表的排序？

$(1, 3)(3, 2)$

$(1, 2)(2, 3)(1, 2)$

b. 注意，通过以这种方式表示交换序列，序列可以分成子序列，然后重新连接形成新序列。使用这种方法描述一个遗传算法，用于开发一个对长度为10的列表进行排序的程序。

53 . 假定一组机器人的每个成员都配有一对传感器，每个传感器能探测到正前方2米范围内的物体。每个机器人的形状都像一个圆形的垃圾桶，能向任何方向移动。试设计一系列实验，用来确定传感器应该装在哪里，才能制造出能成功地将一个篮球直线抛出的机器人。你的一系列实验与进化系统相比如何？

54 . 你做决定时是倾向基于反应模式还是倾向基于计划模式？你的答案是否依赖于你在决定的是中午要吃什么还是做一个职业决策？

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1. 核能、遗传工程以及人工智能领域的研究人员应该对他们的研究成果被使用的方式承担多大的责任？科学家对其研究揭示的知识是否负有责任？若因此产生了意想不到的后果呢？

2. 怎样区分智能和模拟的智能？你认为二者有区别吗？

3. 假设一个计算机化的医疗专家系统因其给出的合理建议而在医疗界享有盛誉。作为一个医生，应该在多大程度上允许这个系统代替他为病人做出治疗决定？如果医生的治疗方案与专家系统提出的治疗方案相对立，并且后来证实专家系统是正确的，那么该医生应该对其不当治疗负有责任吗？一般来说，如果一个专家系统在某个领域内很有名，那么它会在多大程度上妨碍而不是提高人类专家的判断力？

4. 许多人认为计算机的行为只不过是人类对它进行编程的结果，因此，计算机不可能有自主意志。因而，计算机也不应对它的行为负责。人脑是计算机吗？人是在出生的时候就事先被编程好的吗？人会被他们所处的环境编程吗？人要对自己的行为负责吗？

5. 是否有这样一些手段，科学即使能够去做也不应当去做？例如，如果有朝一日可以造出感知和推理技巧能与人类匹敌的机器，那么建造这样的机器合适吗？这种机器的出现会带来什么问题？今天其他科学领域的进步正在引发哪些问题？

6. 历史上有许多例子表明，科学家和艺术家的创作活动会受其所处时代的政治、宗教或其他社会因素的影响。这样的一些因素以何种方式影响着当前科研工作？具体到计算机科学领域情况如何？

7. 当今，技术的进步导致一些人的工作变得多余，许多文化至少应担负起一定的责任来帮助对这些人进行再教育。随着技术使我们越来越多的能力变得多余，社会应当或能够做些什么？

8. 假设你收到一张计算机生成的费用为0.00美元的账单，你该怎么办？假设你置之不理，30天后你又收到第二张0.00美元的催款通知，你该怎么办？假设你依然不理睬，30天后你又收到一张0.00美元的催款通知，而且还有一个提示：若不及时支付账单，将诉诸法律。谁将对此负责？

9. 你有把个性和个人计算机联系在一起的时候吗？有时计算机好像在施行报复或者固执难缠？计算机是否曾令你抓狂？对计算机恼火和对计算机所做的结果恼火有什么不同？计算机和你生过气吗？你与别的东西，如汽车、电视和圆珠笔，有过类似的关系吗？

10. 根据你对第9题的回答，人在多大程度上愿意把一个实体的行为与智能和意识的存在联系起来？人应当在多大程度上做这样的关联？对一个智能实体来说，是否可能用有别于其行为的某种方式来展现它的智能？

11. 许多人觉得，能通过图灵测试并不意味着机器有智能。一个论点是，智能行为本身并不意味着智能。但进化论的基础是适者生存，这就是一种基于行为的测试。是否进化论意味着智能行为是智能的前身？机器能通过图灵测试，是否意味着它们正变得有智能？

12. 医疗手段已经取得了很大的进步，人体的许多器官现在都能用人造器官或者捐赠人的器官来替代。可以想象，终究有一天连大脑也能换。如果这变成现实，会产生什么样的道德问题？如果一个病人的神经细胞被人造神经细胞一点点换掉，那么这个病人还是同一个人吗？这个病人会觉察到有什么不同吗？那个病人还算是人吗？

13. 一台汽车中的GPS能够提供友好的语音提示，通知驾驶员即将到来的转弯和其他操作。如果驾驶员犯错，GPS会自动进行调整，并在不带任何不当情绪的情况下指导驾驶员返回正确路线。你认为，当驾驶员要驾车去一个新目的地时，GPS可以减小驾驶员的压力吗？GPS会在哪些方面给驾驶员带来压力呢？

14. 智能手机应用现在可以提供基本的语音到语音的语言翻译。你在什么情况下使用这个功能会觉得舒服？你相信它传达的是正确意思吗？你有什么顾虑吗？

课外阅读

Banzhaf, W., P. Nordin, R. E. Deller, and F. D. Francone. *Genetic Programming: An Introduction*. San Francisco, CA: Morgan Kaufmann, 1998.

Liu, J. and J. Wu. *Multi-Agent Robotic Systems*. Boca Raton, FL: CRC Press, 2001.

Luger, G. F. *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed. Boston, MA: Addison-Wesley, 2008.

Mitchell, M. *An Introduction to Genetic Algorithms*. Cambridge, MA: MIT Press, 1998.

Negnevitsky, M. *Artificial Intelligence: A Guide to Intelligent Systems*, 3rd ed. Boston, MA: Addison-Wesley, 2011.

Nilsson, N. *Artificial Intelligence: A New Synthesis*. San Francisco, CA: Morgan Kaufmann, 1998.

Nolfi, S. and D. Floreano. *Evolutionary Robotics*. Cambridge, MA: MIT Press, 2000.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

第12章 计算理论

本章将讨论计算机科学的理论基础。从某种意义上说，正是本章所讨论的内容为计算机科学奠定了其真正的科学地位。尽管本质上有些抽象，但该知识体系已经有许多非常实际的应用。具体来说，我们将探讨它对程序设计语言能力的影响，看看它是如何形成广泛用于因特网通信的公钥加密系统的。

本章内容

12.1 函数及其计算

12.2 图灵机

12.3 通用程序设计语言

12.4 一个不可计算的函数

12.5 问题的复杂性

*12.6 公钥密码学

持久理解和学习成果

算法可以解决许多但不是所有的计算问题。AP[®]CSP

学习成果 解释能在合理时间内运行的算法与那些不能在合理时间内运行的算法之间的区别。

学习成果 解释计算机科学中可解问题与不可解问题之间的区别。

学习成果 解释计算机科学中存在的不可决定问题。

学习成果 对算法的效率、正确性和清晰度进行分析评估和经验评估。

算法可以通过复杂性来分析和分类。

学习成果 解释通用程序设计语言的意义。

学习成果 列出并对比标准复杂性类。

学习成果 解释用大O和大 Θ 符号来描述算法完成的工作量。

学习成果 解释为什么停机问题没有算法解。

学习成果 定义P和NP类。

学习成果 解释NP完全性的意义。

学习成果 解释丘奇-图灵论题及其意义。

公钥加密使用计算上复杂的数学运算来加密和解密数据。

学习成果 解释公钥密码学背后的基本概念。

本章要讨论的是有关计算机能做什么、不能做什么的问题。我们将看到，一种称为图灵机的简单机器如何被用来确定机器可解问题与机器不可解问题之间的界限。我们还将确定一个特定的问题，就是停机问题，这个问题的解决超出了算法系统的能力，因此也就超出了当今乃至未来计算机的能力。此外，我们会发现，即使在机器可解的问题中，也存在一些因解决方案太复杂而从实际角度看是不可解的问题。本章的最后会讨论如何用复杂性领域的知识来构造公钥加密系统。

12.1 函数及其计算

本章的目标是研究计算机的能力。我们想要了解机器能做什么、不能做什么，以及机器要实现其全部潜能需要具备哪些特性。我们在前面的几章中见过许多Python函数的例子，但这里，我们从更一般的计算数学函数的概念开始进行讨论。

从数学意义上讲，**函数**（function）是一组可能的输入值和一组可能的输出值之间的对应关系，它使每个可能的输入被赋予单个的输出。一个例子是将尺寸从码转换成米的函数，对于每个以码为单位的尺寸，该函数会返回以米测量相同距离时的结果值。另一个例子是`sort()`函数，该函数为每个输入的数值列表都赋予了一个输出列表，而输出列表中的项与输入列表中的一项一样，只是输出列表中对值进行了升序排列。还有一个例子是`addition()`函数，该函数的输入是一对数值，输出是这对输入值的和。

对于一个给定的输入，确定其具体的输出值的过程，称为**函数的计算**。对函数进行计算的能力非常重要，因为我们能解决问题的手段就是计算函数。要解决加法问题就必须计算`addition()`函数；要对列表进行排序就必须计算`sort()`函数。因此，计算机科学的一个基本任务就是找出要解决的问题背后的函数的技术。

递归函数理论

没有什么比被告知某些事不能做更撩拨人性的了。一旦研究人员开始确定一些不可解的问题，也就是说找不到解决问题的算法，就会有另外一些人开始研究这类问题，试图理解问题的复杂性。今天，递归函数理论学科就成了这样一个研究领域，并且很多人已经认识到了这种超难问题。事实上，正如数学家开发出数字系统来揭示超越无限的“量化”级别一样，递归函数理论学家也揭开了问题空间内的多级复杂性，这些问题已经超出了算法的能力。

例如，考虑一个系统，其中函数的输入和输出能够预先确定并记录在一个表中。每当需要函数的输出时，我们只需在表中查找给定的输入就能找到所需的输出。这样一来，这个函数的计算就简化为搜索表的过程。虽然这种系统很方便，但功能有限，因为许多函数无法完全表示成表格的形式。图12-1展示了一个例子，例中展示的函数要将以码为单位的量值转化为以米为单位的量值。因为可能的输入/输出对的列表是无限的，所以这个表注定是不完整的。

码 (输入)	米 (输出)
1	0.9144
2	1.8288
3	2.7432
4	3.6576
5	4.5720
·	·
·	·
·	·

图12-1 显示将尺寸从码转换成米的函数的尝试

计算函数的一个比较有效的方法是遵循代数式所提供的方向，而不是试图将所有可能的输入/输出组合都显示在表格中。例如，可以用代数公式

$$V = P(1+r)^n$$

来描述如何计算一个为 P 的投资额（年复利率为 r ）在 n 年后的金额。

但是，代数公式的表达能力也有它的局限性。有些函数的输入/输出关系太过复杂，以致不能用代数运算来描述。这样的例子包括三角函数，如正弦函数和余弦函数。如果要你计算 38° 的正弦值，那么你可能会画出相应的三角形，测出它的边长，然后计算所求的比率，而这样的过程就不能表示为对数值38的代数运算。用袖珍计算器来计算 38° 的正弦值也是比较费劲的。实际上，对 38° 的正弦值而言，必须

利用较复杂的数学方法来得到一个非常接近的近似值，并将其作为答案报告给你。

于是可以看出，当考虑的函数越来越复杂时，我们不得不应用更为强大的技术来计算它们。我们的问题是，不管函数的复杂性如何，我们是否总能找到一个系统来计算它们。答案是**否定的**。有一个结论令人难受，那就是存在这样的一些函数，它们过于复杂以致找不到定义明确的、一步步的过程来根据输入值确定它们的输出值。结果，这些函数的计算就超出了任何算法系统的能力范围，这样的函数就称为不可计算的，而有些函数，如果可以依据它们的输入值，通过算法来确定其输出值，就称其为**可计算的**（computable）。即使我们将讨论限制在决策问题，即输出仅为1或0（“是”或“否”）的函数，所有可能的问题也分为两个阵营：可为每个可能输入构造正确算法的**可决定**（decidable）函数和构造不出总能返回正确答案的算法的**不可决定**（undecidable）函数。

在计算机科学中，可计算函数与不可计算函数之间的区别很重要。因为机器只能执行由算法描述的任务，所以对可计算函数的研究最终是对机器能力的研究。如果我们能够确定这样的能力，即允许机器计算所有可计算函数的能力，并造出具有这些能力的机器，那么就可以确信，所造的机器就如我们所设想的一样强大。同样，如果我们发现一个问题的解决方案需要计算一个不可计算函数，那么可以得出这样的结论：该问题的求解超出了机器的能力范围。

基本知识点

- 可决定问题是可以构造算法来对所有输入回答“是”或“否”的问题（例如，“这个数字是偶数吗？”）。
- 不可决定问题是构造不出总能产生正确的是或否答案的算法的问题。

12.1节问题与练习

1. 给出一些能完全由表格形式表示的函数。
2. 给出一些函数，要求其输出可以描述为涉及其输入的一个代数表达式。
3. 给出一个不能用代数公式描述的函数。你的这个函数仍然是可计算的吗？
4. 古希腊数学家用直尺和圆规画图形。他们借此探索出一些方法，用来找出一条直线的中点，构建一个直角，以及画一个等边三角形。然而，他们的“计算系统”不能完成的“计算”是什么？

12.2 图灵机

为了理解机器的能力及其局限性，许多研究人员提出并研究了各种不同的计算设备，其中之一就是图灵机。图灵机是由艾伦·图灵于1936年提出的，今天它仍然被用作研究算法处理能力的一种工具。

12.2.1 图灵机的原理

图灵机（Turing machine）是由一个控制单元组成的，它能够通过一个读/写磁头对磁带上的符号进行读和写（见图12-2）。磁带两端可以无限延伸，并分成一个个单元，而每个单元可以包含符号的任意一个有限集合，这个集合称为机器的字母表。

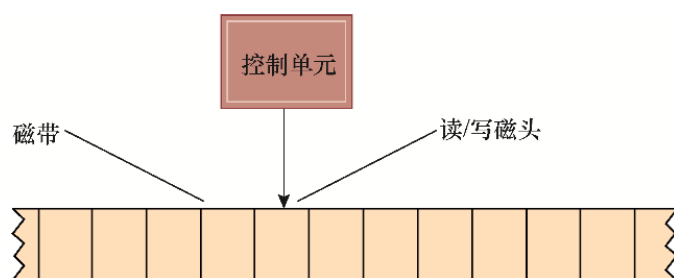


图12-2 图灵机的组成

图灵机的起源

20世纪30年代，在技术能够提供我们今天所知道的机器之前，艾伦·图灵就提出了图灵机的概念。事实上，图灵的憧憬是人用铅笔和纸来进行计算。图灵的目的是提供一个模型，并且利用这个模型来研究“计算过程”的局限性。在此前不久，1931年，哥德尔（Gödel）发表了著名的揭示计算系统局限性的论文，并且其研究的主要精力集中在理解这些局限性上。在图灵提出他的模型的同一年（1936年），埃米尔·波斯特（Emil Post）提出了另外一种模型（现在称为波斯特产生式系统），他所提出的这个模型被证明有与图灵模型一样的能力。作为这些早期研究人员洞察力的见证，他们的计算系统模型（如图灵机和波斯特产生式系统等）在计算机科学研究领域，仍然可以作为有价值的工具来使用。

在图灵机计算的任一时刻，机器一定处在有限个条件中的一个，这些条件称为状态。图灵机的计算开始于一个特定的状态，称为初始状态，而停止于另一特定的状态，称为停止状态。

图灵机的计算由机器的控制单元执行的一系列步骤组成。每一步都包括观察当前磁带单元中的符号（由读/写磁头所看到的那个），然后将符号写进这个单元，期间可能要将读/写磁头左移或右移一个单元，接下来再改变状态。要执行的确切操作是由程序决定的，程序通过机器的状态和磁带当前单元的内容来告诉控制单元做什么。

现在来考虑图灵机的一个具体的例子。为此，我们将机器的磁带表示成一条水平条带，并将条带分成一个个单元，且我们可以在单元中记录机器字母表里的符号。我们通过在当前磁带单元放置一个标签来标示机器的读/写磁头的当前位置。本例中的字母包括0、1和*。机器的磁带的样子如图12-3所示。

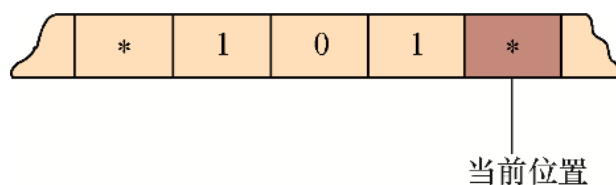


图12-3 机器的磁带的样子

通过将磁带上的符号串解释为由星号分隔的二进制数，我们可以看出，这个特定磁带包含的是值5。我们所设计的图灵机旨在将磁带上的这样一个值加1。更准确地说，假设开始位置是标在一串0和1右端的

星号，接下来要做的是改变其左边的位模式，使其可以表示下一个较大的整数。

我们机器的状态有START（开始）、ADD（相加）、CARRY（进位）、OVERFLOW（溢出）、RETURN（返回）和HALT（停止）。这些状态对应的每个动作和当前单元的内容如图12-4中的表所示。这里假设机器一直是从STRAT状态开始的。

当前状态	当前单元内容	写的值	移动方向	进入的新状态
START	*	*	左移	ADD
ADD	0	1	右移	RETURN
ADD	1	0	左移	CARRY
ADD	*	*	右移	HALT
CARRY	0	1	右移	RETURN
CARRY	1	0	左移	CARRY
CARRY	*	1	左移	OVERFLOW
OVERFLOW	(忽略)	*	右移	RETURN
RETURN	0	0	右移	RETURN
RETURN	1	1	右移	RETURN
RETURN	*	*	不移动	HALT

图12-4 实现增加值操作的图灵机

现在把这个机器应用到图12-3所示的包含值5的磁带上。可以观察到，当处在START状态时，当前单元包含*（像本例中这样），图12-4中的表指示我们要重写*，并将读/写磁头左移一个单元，这时就进入了ADD状态。做完这些后，机器的情况就如图12-5所示。

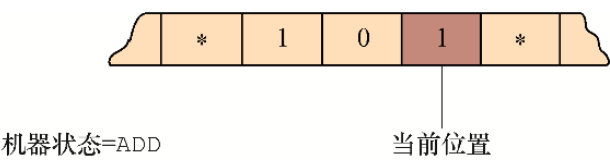


图12-5 机器状态为ADD时对应的情况

为了继续，我们查表，看看当处于ADD状态并且当前单元包含1时，机器要做些什么。图12-4所示的表告诉我们要用0代替当前单元的1，并把读/写磁头左移一个单元，这时就进入了CARRY状态。这样一来，机器的配置就变成了图12-6所示的样子。

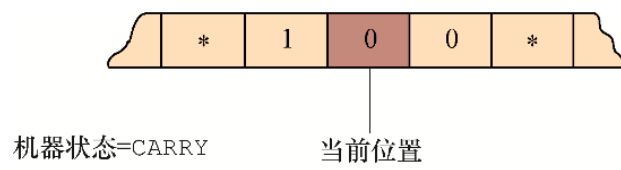


图12-6 机器状态为CARRY时对应的情况

我们再去查表，看看当机器处在CARRY状态并且当前单元包含0时要做什么。图12-4所示的表告诉我们应该用1来代替0，并把读/写磁头右移一个单元，这时就进入了RETURN状态。做完这些后，机器的配置就变成了图12-7所示的样子。

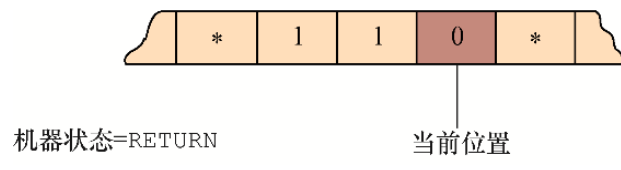


图12-7 机器状态为RETURN时对应的情况

在这种情况下，图12-4中的表指示我们继续用另一个0来代替当前单元中的0，并把读/写磁头右移一个单元，这时就保持在RETURN状态。结果，我们发现我们的机器处于图12-8所示的情况。

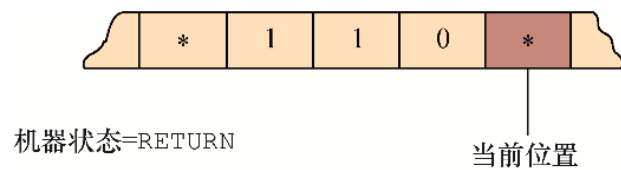


图12-8 保持RETURN状态时对应的情况

此时，我们看到，图12-4所示的表指示我们在当前单元中重写*，同时进入HALT状态。于是，机器就停止在图12-9所示的配置下（磁带上的符号就表示了所需要的值6）。

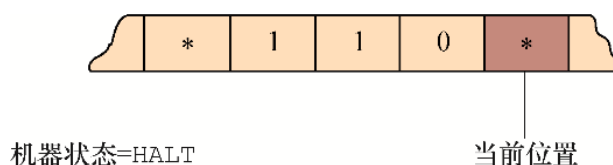


图12-9 机器状态为HALT时对应的情况

12.2.2 丘奇-图灵论题

前面例子中的图灵机可以用来计算所谓的后继函数，使用这种函数，每个非负整数输入值 n 的输出值为 $n+1$ 。我们只需要把用二进制形式表示的输入值放在机器的磁带上，运行机器，直至停止，然后就可以从磁带上读取输出值。由图灵机以这种方式计算的函数称为**图灵可计算的**（Turing computable）函数。

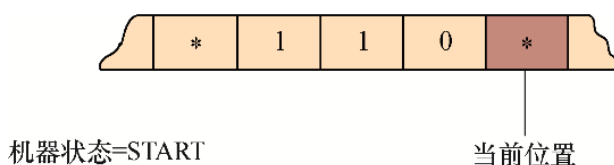
图灵猜想是指：图灵可计算函数与可计算函数是一样的。换句话说，图灵猜想，图灵机的计算能力囊括了任何算法系统的能力，或者同样也可以这么说，（与表格和代数公式这些方法形成对比）图灵机概念提供了一个环境，在此环境下，所有可计算函数的解都能够被表示。在今天，这个猜想通常被称为**丘奇-图灵论题**（Church-Turing thesis），这是为了纪念艾伦·图灵和阿隆佐·丘奇的贡献。自从图灵的

最初工作以来，已经收集了许多支持这个论题的例证，现在，丘奇-图灵论题已被广泛接受。也就是说，可计算函数和图灵可计算函数被认为是一回事。

这个猜想的意义就在于，它洞悉了计算机器的能力和局限性。更准确地说，它把图灵机的能力确立为一种标准，因而其他计算系统就能够与之进行比较。如果一个计算系统能够计算所有的图灵可计算函数，那么就可以认为它和任何计算系统一样强大。

12.2节问题与练习

1. 应用本节描述的图灵机（见图12-4），从下列初始状态开始。



2. 描述一个图灵机，要求用一个0来替换一串0和1。
3. 描述一个图灵机，要求：如果磁带上的值大于0，则该值要减1；如果磁带上的值为0，则该值保持不变。
4. 给出一个日常生活的场景，要求该场景中要有计算的活动发生。这个场景怎样与图灵机进行类比？
5. 描述一个图灵机，要求它在输入某些值时最终会停止，而在输入其他值时永不会停止。

12.3 通用程序设计语言

在第6章中，我们讨论了高级程序设计语言中的各种特性。本节中，我们要应用可计算性方面的知识来确定这些特性中哪些特性是真正必需的。我们会发现，当今的高级语言中的许多特性仅仅是增强使用的方便性，而对语言的基本能力并没有什么贡献。

我们的方法是描述一种简单的指令性程序设计语言，而这种丰富的语言足以用来表达计算所有图灵可计算函数（因此也包括所有可计算函数）的程序。因此，如果以后的程序员发现一个用这种语言解决不了的问题，那么其原因并不在于这种语言的缺陷。相反，问题出在没有用于解决这个问题的算法。具有这种特性的程序设计语言称为**通用程序设计语言**（universal programming language）。

你也许会惊奇地发现，通用程序设计语言其实并不需要很复杂。事实上，我们将要介绍的这种语言会非常简单。因为它是从通用程序设计语言中分离出来的需求的最小集合，所以我们将其称为Bare Bones。

12.3.1 Bare Bones语言

为了表述Bare Bones语言，这里就先来考虑其他程序设计语言中的变量。尽管机器本身只能处理二进制位模式，并且不知道模式所表示的内容，但是程序员可以利用这些声明语句从数据结构和数据类型（如数值列表和字母字符串）方面考虑问题。用来处理精巧的数据类

型和数据结构的高级指令在提交给机器执行之前，必须被翻译成机器指令，这些指令操纵位模式来模拟所需的操作。

为方便起见，可以将这些位模式解释成二进制计数法表示的数值。这样一来，由计算机完成的所有计算都能够表达成包括非负整数的数值计算，这是有目共睹的。而且，如果要求程序员按这种方式表示算法，那么程序设计语言就能得到简化（尽管这会大大增加程序员的负担）。

因为我们开发Bare Bones语言的目标是开发出最简单的语言，所以我们将遵循这个思路。考虑将Bare Bones语言中的所有变量都表示成位模式，为方便起见，我们将其解释为二进制计数法表示的非负整数。这样一来，一个当前被赋值为模式10的变量将包含值2，而被赋值为模式101的变量将包含值5。

利用这种约定，Bare Bones程序中的所有变量都属于同一种类型，这样一来，这种语言就不需要用于转换值类型的操作，或者描述不同变量的名字和与之相关的属性的声明语句了。在使用Bare Bones语言时，像Python一样，程序员可以在需要时只使用一个新变量名即可，这里，程序员理解的是，它是一个解释成非负整数的二进制位模式。

当然，用在Bare Bones语言中的翻译器必须能够把变量名和其他术语区分开来。要做到这一点，需要设计出Bare Bones语言的语法，以便只需通过语法就可以识别出任何术语的作用。为了达到这个目

的，我们规定：变量名必须以英文字母开始，后面可以跟字母和数字（0~9）的任意组合。这样一来，字符串XYZ、B747、abcdefghi以及X5Y都能用作变量名，而2G5、%o和x.y就不能。

现在，让我们来考虑Bare Bones语言中的过程语句。这里有3条赋值语句和一个表示循环的控制结构语句。根据Python风格的语法，我们采用的策略是：每行只写一条语句，并且使用缩进来标记循环结构体。

3条赋值语句中的每条都要求改变语句中标识的变量的内容。第一条语句可以让一个变量清零，其语法为

```
clear name
```

其中name可以是任意变量名。

另外两条赋值语句的作用本质上是相对的：

```
incr name
```

和

```
decr name
```

同样，name表示任意变量名。第一条语句使标识的变量值增加1。这样一来，如果变量Y原来的值为5，那么执行语句

```
incr Y
```

后，赋给变量Y的值就变为6。

相反，decr语句被用来将标识的变量值减1。一个例外的情况是，当变量的值已经为0时，这条语句将保持值不变。这样一来，如果变量Y的值为5，那么执行语句

```
decr Y
```

后，变量Y所赋的值就为4。然而，如果变量Y的值已经为0，那么执行这条语句后，该变量的值仍为0。

Bare Bones语言只提供了一条控制结构语句，该语句由while表示。语句序列

```
while name not 0:  
.  
.  
.
```

（其中的name表示任意变量名）在变量name不为0的情况下，会反复执行while语句下面缩进的语句或语句序列。更准确地说，在程序执行期间遇到while结构时，所标识变量的值首先和0进行比较：如果值为0，则跳过此结构，继续执行缩进循环体后面的语句；然而，如果变量的值不为0，就执行while结构中的缩进语句序列，并且将控制返回给while语句，然后再次进行比较。注意，程序员要担起循环控制的一部分责任，为了避免陷入无限循环，程序员必须在循环体中明确要求改变变量的值。例如，语句序列


```
incr X
while X not 0:
    incr Z
```

将会导致一个无限循环过程，因为一旦到达while语句，X的值永远不会为0，而语句序列

```
clear Z
while X not 0:
    incr Z
    decr X
```

最终会终止，该语句的作用是将最初与X关联的值转移给变量Z。

可以观察出，while语句可能会出现在被另一个while语句重复执行的指令中。在这种情况下，多级缩进将指示哪些指令属于内部嵌套的while语句的循环体。

最后一个例子是图12-10中的指令序列，该序列执行的结果是将与X和Y关联的值的乘积赋给Z，尽管它有一个副作用，即会破坏已经与X关联的任何非零值。（由变量w控制的while结构起到了恢复Y的原始值的作用。）

```
clear Z
while X not 0:
    clear W
    while Y not 0:
        incr Z
        incr W
        decr Y
    while W not 0:
        incr Y
        decr W
    decr X
```

图12-10 一个用于计算 $X \times Y$ 的Bare Bones程序

12.3.2 用Bare Bones语言编程

记住，我们提出Bare Bones语言的目的就是要研究什么是可能的，什么是不切实际的。事实证明，在实用的场合使用Bare Bones语言不太合适。另外，我们将很快看到，这种简单的语言达到了我们的目的，即它提供了一种基本的通用程序设计语言。在这里，我们只是要说明一下如何用Bare Bones语言来表示一些基本的操作。

我们首先注意到，通过赋值语句的组合，任何值（任何非负整数）都可以与给定的变量相关联。例如，下列语句序列能把值3赋给变量X，具体步骤是先将值0赋给X，然后对其值进行三次递增操作：

```
clear X
incr X
```

```
incr X  
incr X
```

程序中另一种常见的活动是将数据从一个地方复制到另一个地方。就Bare Bones语言而言，这就意味着我们需要能够将一个变量的值赋给另一个变量。具体可以这样来实现：先将目标变量清零，然后对其进行合适次数的递增操作。事实上，我们已经看到，语句序列

```
clear Z  
while X not 0:  
    incr Z  
    decr X
```

把与X关联的值转移给了Z。然而，这个语句序列还有一个副作用，即破坏了X的原始值。为了避免这种破坏，可以引入一个辅助变量，先将对象的值从其初始位置转移到这个辅助变量。然后，我们就可以将这个辅助变量作为数据源，并从中恢复原始变量，同时将变量的值放至所要求的目标位置上。通过这种方式，图12-11所示的语句序列实现了Today到Yesterday的转移。

我们采用语法

```
copy name1 to name2
```

（其中name1和name2都表示变量名）作为一种简略的符号，用来表示图12-11所示的语句结构。这样一来，尽管Bare Bones语言本身没有明确的copy指令，但是在写程序的时候就好像有这样的指令。而这里需要理解的是，要想将这种非正式的程序转换成实际的Bare

Bones语言程序，必须把copy语句用其等价的while结构来代替，并且所使用的辅助变量名不能与程序中其他地方已经用过的名字冲突。

```
clear Aux
clear Tomorrow
while Today not 0:
    incr Aux
    decr Today
while Aux not 0:
    incr Today
    incr Tomorrow
    decr Aux
```

图12-11 实现指令“copy today to tomorrow”的Bare Bones语句序列

12.3.3 Bare Bones的通用性

现在让我们来应用丘奇-图灵论题来证明我们的论断，即Bare Bones语言是一种通用程序设计语言。首先，可以看到，任何用Bare Bones语言编写的程序都能看成对一个函数计算的指导。函数的输入由执行程序前分配给变量的值组成，函数的输出由程序终止时变量的值组成。要计算这个函数，我们只需从变量的适当赋值开始执行这个程序，然后观察程序终止时变量的值。

在这些条件下，程序

```
incr X
```

指导由12.2节中图灵机例子计算的那个函数（后继函数）的计算。事实上，它就是将X的值增加1。同样，如果将变量X和Y解释为输入，而将变量Z解释为输出，那么程序

```
copy Y to Z
while X not 0:
    incr Z
    decr X
```

指导的就是加法函数的计算了。

研究人员已经证明，Bare Bones程序设计语言能够用于表达计算所有图灵可计算函数的算法。如果把这与丘奇-图灵论题相结合，就意味着任何可计算函数都能用Bare Bones语言编写的程序来计算。这样一来，Bare Bones语言就是一种通用程序设计语言。从这个意义上讲，如果存在一个解决问题的算法，就可以通过某个Bare Bones语言程序来解决这个问题。因此，理论上可以这么说：Bare Bones语言可以用来作为一种通用程序设计语言。

之所以说[理论上](#)可以这么说，是因为这样的语言肯定不像Python或第6章中介绍的高级语言那么方便。但是，每种高级语言实质上都包含有Bare Bones语言的特性，并将其作为核心。实际上，正是这个核心保证了每种这样的语言的通用性，而各种语言中的所有其他特性都是为了方便使用才引入的。

尽管像Bare Bones之类的语言在应用程序设计环境中并不实用，但在计算机科学的理论研究中还是能找到用武之地的。例如，在附录

E中，我们将使用Bare Bones语言作为一种工具来解决第5章提出的关于迭代结构和递归结构等价的问题。事实上，我们会发现，这种等价性的猜测被证明是正确的。

12.3节问题与练习

1. 证明语句invert X（此语句的功能是：如果X的初始值不为0，就把X的值转换为0；如果X的初始值为0，就把X的值转换为1）能够用一个Bare Bones程序段来模拟。

2. 证明即使简单的Bare Bones语言也包含一些非必要的语句，如clear语句就能利用语言中别的语句的组合来代替。

3. 证明if-else结构能够由Bare Bones语言来模拟。也就是说，用Bare Bones语言写一个程序序列来模拟下列语句的动作：

```
if X not 0:  
    S1  
else:  
    S2
```

其中S1和S2表示的是任意语句序列。

4. 证明每一条Bare Bones语句都能用Vole机器语言（见附录C）来表达。（因此，Bare Bones语言可以用来作为这种机器的程序设计语言。）

5. 怎样用Bare Bones语言来处理负数？

6. 描述由下列Bare Bones语言程序计算的函数，假设该函数的输入由X表示，输出由Z表示。

```
clear Z
while X not 0:
    incr Z
    incr Z
    decr X
```

12.4 一个不可计算的函数

现在，我们来看一个函数，该函数不是图灵可计算的，因此，依据丘奇-图灵论题，可以完全相信它在一般意义上也是不可计算的。这样一来，对这个函数的计算就超出了计算机的计算能力。

12.4.1 停机问题

我们要讨论的是与这个不可计算函数相关联的一个问题，即**停机问题**（halting problem），（简略地说）这个问题就是要预先预测当一个程序在某些条件下开始后是否能够终止（或者说停止）。例如，考虑下面这个简单的Bare Bones程序：

```
while X not 0:
    incr X
```

如果用X的初始值0来执行这个程序，则这个循环体将不会执行，并且程序的执行很快就可以终止。但是，如果用X的任意其他初始值来执行这个程序，那么这个循环将会永远执行下去，这样就导致了一个不可终止的过程。

于是，在这种情况下就很容易得出“只有当X的初始值为0时，该程序的执行才会停止”的结论。然而，随着我们考虑的例子越来越复杂，预测程序行为的任务也变得越来越复杂。事实上，在某些情况下我们将看到，这种预测任务几乎是不可能完成的。但是，我们需要先做的是规范术语，以便更准确地聚焦我们的想法。

我们的例子已经表明，一个程序最终能否终止就取决于其变量的初始值。这样一来，如果我们想预测一个程序的执行是否能终止，就必须在考虑这些初始值方面要做到比较精确。为这些值所做的选择粗看起来可能会有点儿奇怪，但是没有关系。我们的目标是利用一种称为**自引用**（self-reference）的技术，其思想是一个对象引用其自身。从“这条语句是错误的”这样的句子所表示出来的通俗的好奇心，到“所有集合的集合是否包含其自身？”这样的问题所表示的悖论，这种手法在数学上已经多次导致了令人吃惊的结果。那么，我们要做的就是建立起一组推理的步骤，而这些步骤就类似于：“如果它是，那么它就不是；但是，如果它不是，那么它就是。”

在我们的情况中，自引用是这样来实现的，即给程序中的变量赋一个初值，而这个值就表示程序本身。为此，我们注意到，每个Bare Bones程序都是利用ASCII码，以每字节一个字符的方式编码成单个长的位模式，然后将其解释成一个（相当大的）非负整数的二进制表示。我们赋给程序中变量的初始值正是这个整数值。

现在来考虑，如果在下面这个简单程序的情况下这么做会发生什么：


```
while X not 0:  
    incr X
```

我们想知道，如果我们开始这个程序的时候X被赋予了表示程序本身的整数值（见图12-12），那么执行程序后会发生什么。在这种情况下，答案非常明显。因为X可能会有一个非零值，所以程序可能会因此陷入死循环中永不终止。但是，如果我们用程序

```
clear X  
while X not 0:  
    incr X
```

做一个类似的试验，程序就会终止，因为不论变量X的初始值为多少，当程序执行到while结构时，变量X的值都会为0。

于是，我们可以给出以下定义：如果程序中所有的变量都是用程序自身的编码表示来进行初始化的，且这个程序的执行能够导致一个可终止的过程，那么这个程序就是自终止的（self-terminating）。简单来说，如果一个程序以自身作为输入开始执行且能终止，那么这个程序就是自终止的。因而，这就是我们所期望的自引用。

注意，一个程序是否是自终止的可能与编写这个程序的目的无关。它仅仅是一种属性，每个Bare Bones程序要么具有这种属性，要么不具有这种属性。也就是说，每个Bare Bones程序要么是自终止的，要么不是。

现在可以用一种更为精确的方式来描述停机问题。这个问题就是确定Bare Bones程序是不是自终止的。我们即将看到，通常来说没有

回答这个问题的算法。也就是说，当给定任何一个Bare Bones程序时，没有算法能确定这个程序是否是自终止的。因此，停机问题的解决方案超出了计算机的能力。

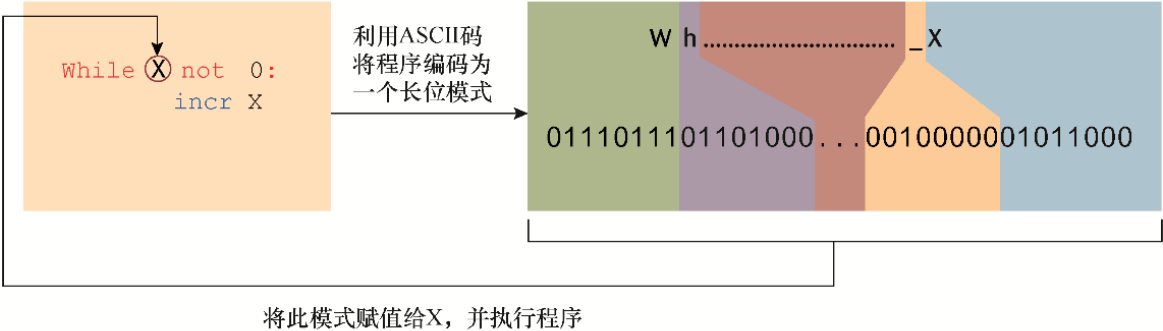


图12-12 测试一个自终止的程序

这样的一个事实，即在我们前面的例子中看上去已经解决了停机问题，而现在却声称停机问题是不可解的，听起来有些矛盾。所以，这里要暂停下来加以解释。前面例子中所用到的观察法只对那些特定的情况适用，而不能将其运用到所有的情况中去。停机问题所要求的是一种单一的通用算法，并能够用在任何Bare Bones程序中，以确定程序是否是自终止的。我们应用某些孤立的洞察力来确定某个特定的程序是否为自终止的这个能力，绝不意味着存在适用于所有情况的单一的通用方法。简而言之，我们也许能制造出可以解决某个特定停机问题的机器，但是无法制造出一个单一的机器，使之能解决出现的任何停机问题。

基本知识点

- 不可决定问题可能有解决个别问题实例的算法解，但没有能解决所有问题实例的算法解。

12.4.2 停机问题的不可解性

现在，我们要来证明求解停机问题超出了机器的能力。我们的方法是证明解决这类问题需要一个用来计算不可计算函数的算法。所涉及函数的输入是Bare Bones程序的编码版本，其输出仅限于值0和1。更准确地说，我们这样定义这个函数：表示一个自终止程序的输入就产生输出值1，而表示一个非自终止程序的输入则产生输出值0。为简明起见，我们称这个函数为**停机函数**（halting function）。

我们的任务就是要证明：停机函数是不可计算的。所用到的方法是“反证法”。简而言之就是，要证明一条语句为假，只需证明它不为真即可。于是，证明语句“停机函数是可计算的”不为真。我们的整个论证过程都概括在图12-13中。

如果停机函数是可计算的，那么（因为Bare Bones语言是一种通用程序设计语言）一定存在一个能计算该函数的Bare Bones程序。换句话说，存在一个Bare Bones程序，如果它的输入是一个自终止程序的编码版本，那么它就以输出值等于1终止；否则就以输出值等于0终止。

为了用这个程序，我们并不需要确定哪个变量是输入变量，只需要把程序的所有变量初始化为要测试的程序的编码表示。这是因为，

如果一个变量不是输入变量，那么它的初始值本质上是不会影响到最终的输出值的。因此可以得出结论：只要停机函数是可计算的，就存在一个Bare Bones程序，如果它的所有变量都被初始化成一个自终止程序的编码版本，那么它就以输出值等于1终止；否则以输出值等于0终止。

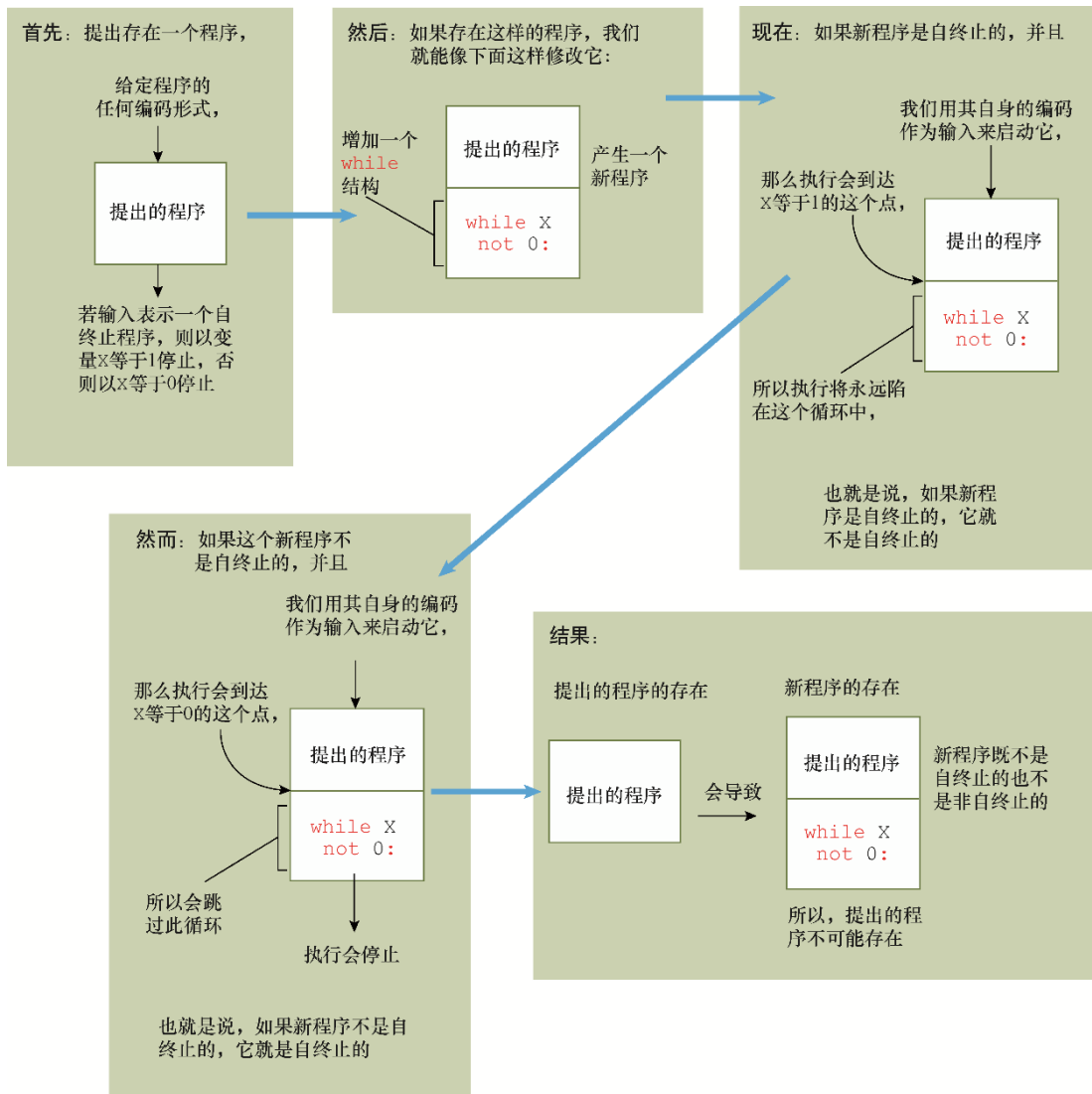


图12-13 证明停机程序的不可解性

假设该程序的输出变量名为X（如果不是，则对变量进行简单的更名即可），我们可以在程序的最后加上以下语句来修改这个程序，从而产生一个新程序：

```
while X not 0:
```

而这个新程序必须要么是自终止的，要么不是。然而，我们即将看到，它既不是自终止的，也不是非自终止的。

具体来说，如果这个新程序是自终止的，且用初始化为该程序自身的编码表示的变量来运行这个程序，那么当它执行到我们添加的while语句时，变量X就会为1。（在这一点上，这个新程序与原始程序一样，如果其输入是一个自终止程序的表示，就会产生一个1。）在这一点，程序的执行将永远陷在while结构中，因为在这个循环中没有预先提供让X值递减的措施。但是，这与新程序是自终止的假设相矛盾。因此，我们必然得出结论：新程序不是自终止的。

然而，如果这个新程序不是自终止的，且用初始化为该程序自身的编码表示的变量来运行这个程序，那么当它执行到我们添加的while语句时，变量X就会被赋值为0。（之所以会这样，是因为在该while语句之前的语句构成的原始程序，在其输入表示一个非自终止程序时，输出0。）在这种情况下，while结构中的循环不会执行，程序会停止。但是，这正是自终止程序的特性，因此，我们不得不得出结论：该新程序是自终止的。这就像早些时候我们不得不认为它不是自终止的一样。

概括来说，可以看出，我们遇到了程序中的一个不可能的情况，即一方面程序必须要么是自终止的要么不是，另一方面程序又必须既不是自终止的又不是非自终止的。其结果是，导致这种矛盾的假设必定不成立。

我们可以得出结论：停机函数是不可计算的。因为停机问题的解决依赖于这个函数的计算，所以我们必然得出结论：停机问题的解决超出了任何算法系统的能力范围。这种问题被称为**不可解问题**（unsolvable problem）。

最后，把刚才讨论过的内容与第11章中的思想联系起来。在第11章中，一个非常基本的问题是，计算机器的能力是否包含智能本身所需要的能力。回想一下，机器只能解决使用算法可解的问题，而现在我们发现有些问题使用算法无法解决。因此，问题就在于人类的大脑是否包含了比执行算法过程更多的东西？如果没有包含，那么我们在这里所确定的局限性，也就是人类思想的局限性。不必说，这是一个极具争议的问题，有时也是情绪方面的问题。例如，如果人的大脑只不过是编程过的机器的话，那么可以推断出，人类就不再拥有自由的意志。

基本知识点

- 一些问题无法用算法解决。

12.4节问题与练习

1. 下面的Bare Bones程序是自终止的吗？请解释你的答案。

```
incr X  
decr Y
```

2. 下面的Bare Bones程序是自终止的吗？请解释你的答案。

```
copy X to Y  
incr Y  
incr Y  
while X not 0:  
    decr X  
    decr X  
    decr Y  
    decr Y  
decr Y  
while Y not 0:
```

3. 下面的场景有什么问题：在某个社区里，每个人都拥有自己的房子，社区的房屋油漆工声称要粉刷所有房主自己没有刷过的房子。
(提示：谁粉刷房屋油漆工的房子？)

12.5 问题的复杂性

在12.4节中我们研究了问题的可解性。在本节中我们关注的问题是一个可解的问题是否有一个实际解。我们将会发现，有些问题在理论上是可解的，但由于过于复杂，从实际的观点来看，它们是不可解的。

基本知识点

- 许多问题可在合理时间内解决。

空间复杂性

除了从时间的角度来度量复杂性，还有一种方法就是通过度量所需的存储空间来衡量复杂性，我们将这种度量方法称为**空间复杂性**（space complexity）。也就是说，一个问题的空间复杂性由解决该问题所需的存储空间的量决定。在本书中，我们已经看到，对一个有 n 个项的列表进行排序的时间复杂性是 $O(n \lg n)$ ，而这个问题的空间复杂性不超过 $O(n+1)=O(n)$ 。要知道，利用插入排序对一个有 n 个项的列表进行排序，除了需要存放列表本身的空间，还需要1个存放临时项的空间。这样一来，如果要排序的列表越来越长，我们就会发现，每个任务所需的时间比所需的空间增长要快得多。这实际上是一个很常见的现象。因为利用空间也要花时间，所以一个问题的空间复杂性永远不会比它的时间复杂性增长得更快。

经常会在时间复杂性和空间复杂性之间做一些折中。在某些应用中，会事先进行某些计算，并将计算结果以表的形式存储起来，这样一来，在需要时就能通过表很快地检索到计算结果。这样一种“查表”技术能减少实际获取结果所需的时间，代价是表需要额外的空间。另外，经常用数据压缩来减少对存储空间的需求，代价是数据压缩和解压缩需要额外的时间。因为这两种类型的复杂性之间可能存在折中，所以算法的总体效率取决于其时间复杂性和空间复杂性。

基本知识点

- 效率既包括执行时间的使用又包括存储器的使用。

12.5.1 问题复杂性的度量

我们首先回到5.6节开始的关于算法效率的研究。在那里，我们用大写的希腊字母 Θ 来标记，并根据执行算法所需的时间对其进行分类。我们发现，插入排序算法属于 $\Theta(n^2)$ 这一类，顺序查找算法属于 $\Theta(n)$ ，而二分搜索算法属于 $\Theta(\log_2 n)$ 。现在，利用这个分类系统来帮助我们确定问题的复杂性。我们的目标是开发出一种分类系统，使其能告诉我们哪些问题比另一些问题更为复杂，并最终确定出哪些问题太过复杂，以致实际上不能解。

我们现在的研究之所以是基于算法效率的知识，其原因在于我们希望从解题的复杂性角度来衡量一个问题的复杂性。我们认为：简单问题有简单的解法；复杂问题就没有简单的解法。注意这样一个事实，一个问题有一个困难的解并不一定意味着该问题本身很复杂。毕竟，一个问题可以有許多解，而其中的某个解必然会复杂些。所以，如果要确定一个问题本身很复杂，就需要证明它的所有解都不简单。

在计算机科学领域中，让人感兴趣的问题就是那些机器可解的问题。这些问题的解都能明确地表示为算法。因此，问题的复杂性取决于解决该问题的算法的特性。更准确地说，解决一个问题的最简单算法的复杂性可以被认为是该问题本身的复杂性。

但是，如何来衡量一个算法的复杂性呢？遗憾的是，术语**复杂性**（complexity）有着不同的解释。一种解释就涉及一个算法中所包含的判定和分支数量。如果按照这种理解，那么一个复杂的算法将会有盘

根错节的判定和分支。这种解释也许和软件工程师的观点一致，软件工程师对与算法的发现和表示相关的问题感兴趣，但是这并没有获得从机器的观点所看到的复杂性的概念。机器在选择下一条要执行的指令时，其实并没有做实际的判断工作，它只是一遍一遍地遵循机器周期，每次执行的都是程序计数器所给出的指令。因此，机器能够执行一组看上去很杂乱的指令，而事实上，它就像在执行一串简单有序的指令那样轻松。事实上，有时具有较复杂表示的算法的执行比较简单的朴素算法的更高效。因此，复杂性的解释倾向于度量一个算法在表示中所遇到的难度，而不是算法本身的复杂性。

基本知识点

- 有时更高效的算法也更复杂。

从机器的角度来看，能够较准确地反映算法复杂性的一种解释是，要度量执行这个算法时必须完成的步骤数。注意，这个数与写好的程序中出现的指令数是不一样的。其循环体只有一条语句，但是其控制要求这个循环体执行100次的循环，在它被执行时，就相当于执行100条指令。所以，这样一个函数被认为比一串50条分开写的语句更复杂，尽管后者在书写形式上显得更长。重点是，**复杂性**的这个含义，最终关心的是机器在执行一个解法时所花的时间，而不是用来表示解的程序的大小。

因此，如果一个问题的所有解都需要大量的时间，就认为这个问题是复杂的。这种复杂性的定义称为**时间复杂性**（time complexity）。

在5.6节中介绍算法效率时，我们已经间接地遇到了时间复杂性这个概念。毕竟，对算法效率的研究就是对算法时间复杂性的研究，只是两者是对立的。也就是说，“较高的效率”等于“较低的复杂性”。因此，从时间复杂性的角度看，在解决一个表单搜索问题时，顺序搜索算法（属于 $\Theta(n)$ ）要比二分搜索算法（属于 $\Theta(\log_2 n)$ ）更复杂。

现在，我们运用算法复杂性的知识来获得一个确定问题复杂性的方法。在解一个问题时，如果存在一个算法，其时间复杂性为 $\Theta(f(n))$ ，并且解决该问题的其他算法没有比这更低的时间复杂性，那么我们就定义这个问题的（时间）复杂性是属于 $\Theta(f(n))$ 的，这里 $f(n)$ 是 n 的某个数学表达式。也就是说，一个问题的（时间）复杂性被定义为该问题的最优解的（时间）复杂性。遗憾的是，找到一个问题的最优解和确认该解为最优解本身往往就是一个难题。在这种情况下，大 Θ 标记的变种，称为**大O标记**（big O notation，读作“big oh notation”），被用来表示对一个问题的复杂性的了解程度。更准确地说，如果 $f(n)$ 是 n 的某个数学表达式，并且如果一个问题能够被属于 $\Theta(f(n))$ 这一类的算法解决，我们就说，这个问题是属于 $O(f(n))$ 的。这样一来，如果说一个问题是属于 $O(f(n))$ 的，也就意味着这个问题有一个复杂性属于 $\Theta(f(n))$ 的解，但是它可能还有更优解。

对搜索和排序算法的研究告诉我们，一个长度为 n 的列表（当我们只知道列表已预先排好序时）的搜索问题是属于 $O(\log_2 n)$ 的，这是因为二分搜索算法能够解决这个问题。此外，研究人员已经证明，搜索问题确实是属于 $\Theta(\log_2 n)$ 的，所以二分搜索算法就代表了这个问题的

最优解。相反，我们知道，对一个长度为 n 的列表（在我们不知道列表中值的原始分布时）的排序问题是属于 $O(n^2)$ 的，因为是用插入排序算法解决的这个问题。然而，知道排序问题是属于 $\Theta(n \log_2 n)$ 的，这就告诉我们，插入排序算法不是最优解（从时间复杂性的角度看）。

排序问题的一个更好的解决办法是归并排序算法。其方法是将列表的一些较小的、排序过的部分归并成较大的、排序好的部分，然后再进行归并，得到更大的排序过的部分。每次归并过程应用的都是介绍顺序文件时遇到的归并算法（见图9-15）。为了方便，再用图12-14来表示一下，而这次的情况是归并两个列表。完整的（递归）归并排序算法可由图12-15中所示的MergeSort()函数来表示。当要求对一个列表排序时，这个函数首先检查被排序的列表，看其是否少于两项：如果是，则该函数的任务已经完成；如果不是，则这个函数将列表分成两部分，再请求函数MergeSort的另外一个副本对这两部分进行排序，然后将这些排序好的片段归并在一起，这样就得到最后的排序过的列表。

```

def MergeLists(InputListA, InputListB, OutputList):
    if (两个输入列表为空):
        停止, OutputList为空
    if (InputListA为空):
        声明它被用完了
    else:
        声明它的第一项为当前项
    if (InputListB为空):
        声明它被用完了
    else:
        声明它的第一项为当前项
    while (两个输入表都没有用完):
        把“较小的”当前项放入OutputList
        if (该当前项是对应输入表的最后一项):
            声明该输入表被用完了
        else:
            声明该输入表的下一项为该表的当前项
    从未完的输入表中的当前项开始, 将剩下的项复制到OutputList

```

图12-14 用来归并两个列表的MergeLists函数

```

def MergeSort(List):
    if (List有多项):
        MergeSort(List的前半部分)
        MergeSort(List的后半部分)
        MergeLists(List的前半部分和后半部分) 生成一个已排序的List

```

图12-15 实现为MergeSort函数的归并排序算法

为了分析这个算法的复杂性，我们先来考虑在归并一个长度为 r 的列表和一个长度为 s 的列表时，需要在列表的项之间进行比较的次数。归并过程是这样进行的：重复地对一个列表中的项与另一个列表中的项进行比较，然后将两者中的“较小项”放入输出列表中。因此，每做一次比较，还要考虑的项数要减1。因为开始时只有 $r + s$ 个项，所以我们可以得出结论：这两个列表的归并过程涉及到的比较次数不超过 $r + s$ 次。

现在来考虑完整的归并排序算法。它是通过这样的方式来处理一个长度为 n 的列表的排序工作的，即将最初的排序问题简化为两个较小的问题，每个问题是要对一个长度约为 $n/2$ 的列表进行排序，接下来再对这两个问题进行分割，使其成为4个对长度约为 $n/4$ 的列表进行排序的问题。这种分割过程可以通过图12-16中的树结构来概括，图中树的每个节点表示的是递归过程中的一个问题，节点下面的分支表示的是从双亲衍生的更小的问题。由此，我们可以发现，将树中各个节点上发生的比较次数加起来，就能得到整个排序过程中所发生的总的比较次数。

我们先来确定树的每层上所进行的比较次数。可以看出，出现在树的任意一层的每个节点，其任务都是对原始列表的唯一一段进行排序。这个任务是由归并过程来完成的，因此正如我们已经指出过的，需要的比较次数不会多于该列表段中的数据项的数。因而，树的每一层所需的比较次数不会多于该列表段中的数据项的总数，而且，因为树中所给定的一个层的段表示的是原始列表的不相交部分，所以这个总数不会比原始列表的长度大。因此，树的每一层所涉及的比较次数都不会超过 n 。（当然，最底层涉及的对长度小于2的列表进行排序的任务根本不需要比较。）

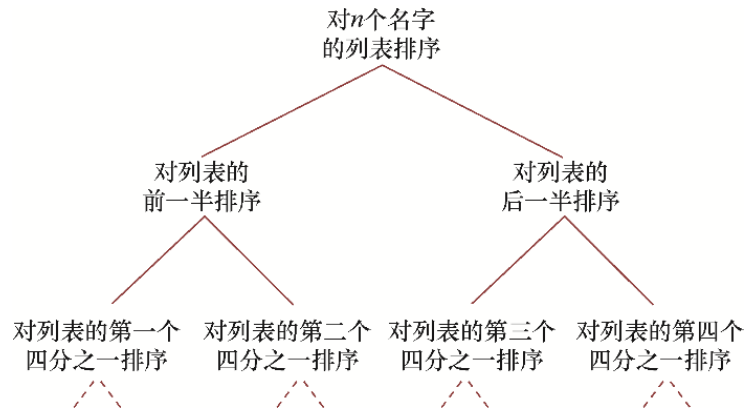


图12-16 由归并排序算法产生的问题的层次结构

现在我们来确定树中的层数。为此，可以看到，把问题分割成更小的问题这个过程一直进行到所得到的列表的长度小于2为止。这样一来，树中的层数就由分割的次数确定，从值 n 开始，反复除以2，直到其结果不大于1，那么这个次数就是 $\log_2 n$ 。更准确地说，树中涉及的比较层数不超过 $\lceil \log_2 n \rceil$ ，这里，标记 $\lceil \log_2 n \rceil$ 表示 $\log_2 n$ 的值向上取整。

最后，把树中每层所做的比较次数乘以涉及比较操作的层数，就能得到在对长度为 n 的列表进行排序时归并排序算法所做的总的比较次数。可以确定，这个次数不大于 $n \lceil \log_2 n \rceil$ 。因为 $n \lceil \log_2 n \rceil$ 对应的图形与 $n \log_2 n$ 对应的图形在形状上大致一样，我们就可以得出结论：归并排序算法是属于 $O(n \log_2 n)$ 的。把这个结论与研究人员告诉我们的排序问题的复杂性为 $\Theta(n \log_2 n)$ 这个事实相结合，这就意味着归并排序算法代表了排序问题的一个最优解。

12.5.2 多项式问题与非多项式问题

假设 $f(n)$ 和 $g(n)$ 是数学表达式。如果说 $g(n)$ 受 $f(n)$ 的约束，就意味着当把这些表达式用在越来越大的 n 值上时， $f(n)$ 的值最终会大于 $g(n)$ 的值，并且对于所有更大的 n 值， $f(n)$ 都将大于 $g(n)$ 。换句话说，如果 $g(n)$ 受 $f(n)$ 的约束，就意味着对于“较大”的 n 值， $f(n)$ 的图将会在 $g(n)$ 的图之上。例如，表达式 $\log_2 n$ 受表达式 n 的约束（见图12-17a），而 $n\log_2 n$ 受 n^2 的约束（见图12-17b）。

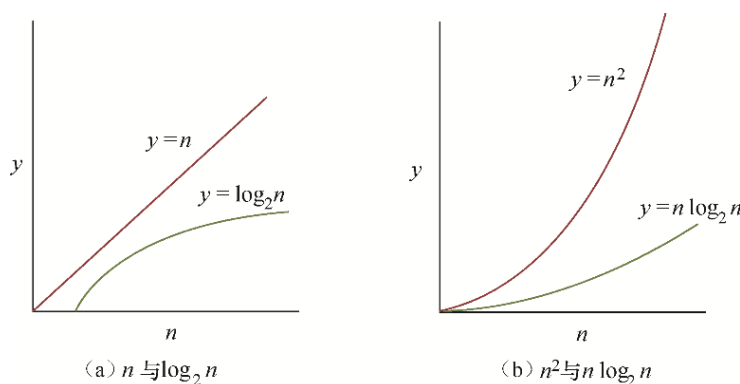


图12-17 数学表达式 n 、 $\log_2 n$ 、 $n\log_2 n$ 和 n^2 的图

如果一个问题是属于 $O(f(n))$ 的，其中表达式 $f(n)$ 要么本身是一个多项式，要么受一个多项式约束，我们就说，这个问题是一个**多项式问题**（polynomial problem）。所有多项式问题的集合用**P**表示。注意，前面的讨论告诉我们，列表的搜索和排序问题都属于P。

基本知识点

- 合理时间的意思是算法所采用的步骤数小于或等于输入大小的多项式函数（常量、线性、平方、立方等）。

说一个问题是多项式问题，是关于求解该问题所需时间的一种陈述。我们经常会说，P中的问题能够在多项式时间内解决，或者说，该问题有多项式时间解。确定属于P的问题是计算机科学中非常重要的课题，这是因为这个问题与问题是否有实际解密切相关。确实，P类之外的问题，其特征都是具有极长的执行时间，即使是对中等规模的输入也是如此。例如，考虑一个求解需要 2^n 步的问题。指数表达式 2^n 不受任何多项式约束，也就是说，如果 $f(n)$ 是一个多项式，那么，当增加 n 的值时，我们就会发现， 2^n 的值最终会大于 $f(n)$ 的值。这就意味着，复杂性为 $\Theta(2^n)$ 的算法通常比复杂性为 $\Theta(f(n))$ 的算法效率低，因而需要更多的时间。如果一个算法的复杂性是用指数表达式来确定的，就说该算法需要指数时间。

下面介绍一个具体的例子，考虑这样一个问题，即从 n 个人组成的群体中，列出所有可能的小组组合。因为可以有 2^n-1 个这样的小组（这里允许一个小组包含所有的人，但是不允许小组中没有人），所以解决此问题的任何算法都至少要有 2^n-1 步，这样一来，其复杂性也至少这么大。但是，表达式 2^n-1 作为一个指数表达式，不受任何多项式约束。因此，随着可选人群规模的增加，这个问题的任何解都会变得极为费时。

上面的小组组合问题的复杂性大只是因为它的输出规模大，与此不同的是，存在这样的一些问题，尽管它们最终的输出只是简单的是或否，但是其复杂性却非常大。一个例子就是回答涉及实数加法的一些语句的真实性问题的能力。例如，我们很容易确定问题“真有自身相

加得6的实数吗？”的答案是肯定的，而“真有自身相加得0的非零实数吗？”的答案是否定的。然而，随着涉及的这类问题越来越多，我们回答这些问题的能力开始减弱。如果我们发现自己要面对许多这样的问题，我们就可能尝试用计算机程序作为辅助。遗憾的是，回答这类问题的能力已被证明需要指数时间，于是，随着所涉及的这类问题越来越多，最终的结果是，即使是计算机也做不到以一种实时的方式给出答案。

理论上可解但不属于P的问题有着巨大的时间复杂性，这一事实让我们得出结论：从实践角度看，这些问题本质上是不可解的。计算机科学家称这类问题是**难解的**（intractable）。从而，类型P成了用来区别难解的问题和那些可能有实际解的问题的重要分界线。因此，对类型P的理解已经成了计算机科学领域的一个重要研究内容。

基本知识点

- 有一些问题即使输入规模很小也无法在合理时间内被解决。

12.5.3 NP问题

现在来考虑旅行商问题（traveling salesperson problem），该问题中涉及一个旅行商，她必须在不超出其旅行预算的情况下访问到其不同城市的每个客户。那么，她的问题就是要找出一条总长度不超过其允许里程的路径（从她家出发，遍历相关城市，然后再返回她家）。

这个问题的传统解决方案是，系统地考虑各种可能的路径，然后把每条路径的长度与里程的限制相比较，直到找到一条可接受的路径，或者是把所有可能的路径都考虑到。然而，这种方法并不能产生一个多项式时间解。随着城市数目的增加，要测试的路径数会比任何多项式增长得都快。因此，在涉及的城市数量很多的情况下，按这种方式解决旅行商问题是不切实际的。

我们可以得出结论：如果要在合理的时间范围内解决旅行商问题，必须找到一个更快的算法。观察到，如果存在令人满意的路径，并且我们碰巧一开始就选择了这条路径，我们提出的算法就能很快地终止，这激发了我们的兴趣。具体来说，下面的指令序列能够很快地执行，并且有解决这个问题的潜力：

```
取一个可能的路径，并计算其总距离。  
if (此距离不大于允许的里程数):  
    宣布找到。  
else:  
    宣布没找到。
```

然而，从技术意义上讲，这组指令不是一个算法。它的第一条指令就比较模糊，原因在于它既没有指出选择的是哪一条路径，也没有详细说明如何做出这样的决定。相反，它依赖于执行程序机制的创造性来自行做决定。我们称这样的指令为非确定性指令，并将包含这种语句的“算法”称为**非确定性算法**（nondeterministic algorithm）。

注意，随着城市数量的增加，执行上述非确定性算法所需的时间会增长得相对较慢。选择路径的过程只是产生一个城市清单，完成这

个过程所需的时间与所涉及的城市数量成正比。此外，沿着所选路径计算总距离所需的时间与所访问城市的数量也成正比，并且，将这个总数与里程的限制所需的时间与城市的数量无关。于是，执行这个非确定性算法所需的时间受一个多项式约束。因此，用非确定性算法在多项式时间内解决旅行商问题是有可能的。

当然，这个非确定性解并不能令人十分满意，因为它依赖于猜测的运气。但是，它的存在足以表明：在多项式时间内，对旅行商问题而言，存在一个确定性解。无论其真假与否，它都是一个尚未确定的问题。事实上，有许多这样的问题，即知道它们在多项式时间内执行时有非确定性解，但还没发现多项式时间内的确定性解，旅行商问题就是这些问题中的一个。对这些问题的非确定性解的可望而不可即的效率，使许多人希望在某一天能找到有效的确定性解，然而，大多数人相信这些问题太复杂，以致超出了有效的确定性算法的能力范围。

一个能够在多项式时间内用非确定性算法解决的问题，称为**非确定性多项式问题**（nondeterministic polynomial problem），或者简称**NP问题**（NP problem）。习惯上把NP类问题表示成**NP**。注意，P中的所有问题也都属于NP，这是因为任何（确定性）算法都可以加上一条非确定性指令而不影响其性能。

然而，正如旅行商问题所说明的，所有的NP问题是否也都属于P还是一个尚未确定的问题。这也许是当今计算机科学领域中最广为人知的未解问题。这个问题的解决将会带来重大的影响。例如，在12.6节中我们将讨论到，已经设计出来的加密系统的完整性依赖于解决问

题所需的大量时间，这类似于旅行商问题。如果证明这种问题存在有效解，那么这些加密系统的安全性就会受到威胁。

为了解决NP类问题事实上是否等同于P类问题这样的问题所做的努力，导致在NP类问题中发现了一类称为**NP完全问题**（NP-complete problem）的问题。这些问题都具有这样一种特征，即任何一个问题的多项式时间解也为所有其他NP类问题提供了一个多项式时间解。也就是说，如果能够在多项式时间内找到一个（确定性）算法，使其能够解决一个NP完全问题，那么这个算法就能推广到以多项式时间来解决任何其他NP类问题。于是，NP类就和P类一样了。旅行商问题是NP完全问题的一个例子。

总体来说，我们可以看出，问题可以分为可解（有算法解）和不可解（没有算法解）两类，如图12-18所示。而且，可解问题又可分为两个子类：一类是多项式问题的集合，该集合包含了有实际解的那些问题；另一类是没有确定性多项式算法的难解问题的集合，这些问题只有在输入相对较少或者仔细选取的情况下才有实际解。最后，还有比较难理解的NP问题，至今还没有很准确的分类。

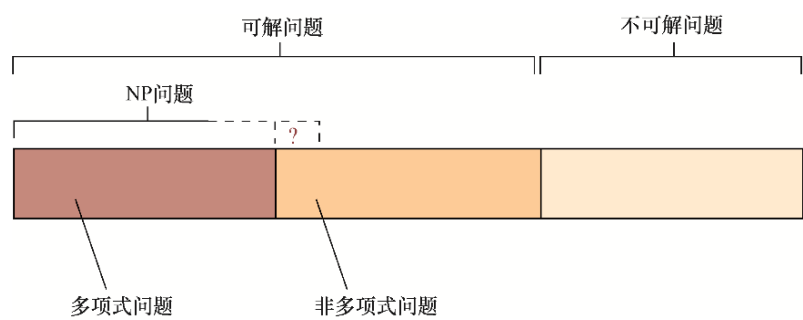


图12-18 问题分类的图形总结

遗憾的是，计算中的许多重要问题都属于难解类问题或者NP类问题，因此大多数情况下无法在合理时间内解决。现代示例包括自然语言和图像的计算机识别、电信网络中数据的最优路由选择，以及在计算机游戏中智能体的指挥。在许多情况下，如果一个算法是未知的，或者不能在有效时间内解决问题，那么可以采用统计或**启发式**技术——“经验法则”或估计几个后续步骤中哪一个最有前途的有根据的猜测。虽然不能保证找到问题的精确解或正确解，但启发式搜索经常能在合理时间内生成近似的可接受结果。

基本知识点

- 一些问题可以解，但不能在合理时间内解。在这些情况下，启发式方法可能有助于在合理时间内找到解。
- 启发式是可以让我们在典型的方法无法找到精确解的时候找到近似解的技术。
- 当精确方法太慢时，启发式方法可能有助于更快地找到近似解。

确定性的和非确定性的

在许多情况下，确定性算法和非确定性算法之间存在着明显的界限。这种差别是非常清晰和明显的。确定性算法不依赖于执行该算法的机制的创造性能力，而非确定性算法可能会依赖。例如，比较指令

走到下一个十字路口，然后向右转或向左转。

和指令

走到下一个十字路口，按照站在路口的人所告诉你的，向右转或向左转。

在这两种情况中，这个人在真正执行指令之前，所采取的转向都是不确定的。然而，第一条指令要求行人根据自己的判断来决定其要转的方向，所以这条指令是非确定性的。第二条指令对行人所要转的方向没有做这样的要求，只是告知行人每一步要做什么。如果有几个不同的人执行第一条指令，那么有些人会向右转，而有些人会向左转。如果有几个人执行第二条指令，并且收到同样的信息，那么他们会朝一个方向转。这里包含了确定性“算法”和非确定性“算法”之间的一个重要区别。如果用同样的输入数据反复执行一个确定性算法，那么每次都会完成同样的动作。然而，一个非确定性“算法”在同样的条件下反复执行，将会产生不同的动作。

12.5 节问题与练习

1. 假设一个问题能够通过属于 $\Theta(2^n)$ 的算法求解，那么对这个问题的复杂性，我们能得出什么样的结论？
2. 假设一个问题既能够通过属于 $\Theta(n^2)$ 的算法求解，又能够通过属于 $\Theta(2^n)$ 的算法求解，那么是否一个算法总是要优于另一个算法？
3. 如果一个委员会是由Alice和Bill两个成员组成的，那么请列出这个委员会的所有可能的组合。如果这个委员会是由Alice、Bill和Carol组成的，那么请列出这个委员会的所有可能的组合。如果这个委员会是由Alice、Bill、Carol和David组成的，那么小组组合情况又会怎样？

4. 给出一个多项式问题的例子。给出一个可解的非多项式问题的例子。给出一个尚未被证明是多项式问题的NP问题的例子。

5. 如果算法X的复杂性比算法Y的大，是否意味着算法X就一定比算法Y难理解？请解释你的答案。

*12.6 公钥密码学

在某些情况下，某个问题难以解决这个事实已经不再是一个缺点，而变成了一个优点。特别有趣的一个问题是，为一个给定的整数找到它的因数，如果这样的解确实存在，就一定要为这个问题找到一个有效解。例如，如果只用纸和笔，你将会发现，即使像2 173这样相对较小的数值，要找到其因数也比较花时间，而如果涉及的数大到需要用几百位数字来表示，那么即使用现在最好的因数分解技术，这个问题也还是难以解决。

对许多数学家而言，至今没能找到一种有效的方法确定大整数的因数一直是许多数学家的眼中钉，但在密码学领域中，这种情况已经被用来产生一种对报文进行加密和解密的流行方法。这种方法称为**RSA算法**（RSA algorithm），选择这个名字是为了纪念该算法的发明者Ron Rivest、Adi Shamir和Len Adleman。这种方法，利用一组称为**加密密钥**（encrypting key）的数值对报文进行加密，并利用另一组称为**解密密钥**（decrypting key）的数值对报文进行解密。知道加密密钥的人可以对报文进行加密，但是不能对报文进行解密。只有持有解密

密钥的人才能对报文进行解密。这样一来，加密密钥就可以被广泛地分发而不会破坏系统的安全性。

这样的密码系统称为**公钥加密**（public-key encryption）系统，这个术语反映了用来对报文加密的密钥可以公开而不会降低系统的安全性。事实上，加密密钥通常称为**公钥**（public key），而解密密钥则称为**私钥**（private key）（见图12-19）。

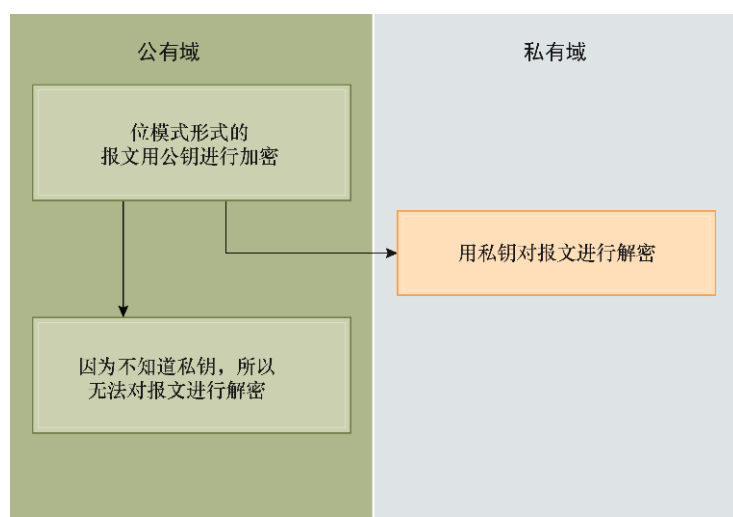


图12-19 公钥密码学

12.6.1 模表示法

为了描述RSA公钥加密系统，可以方便地采用记号 $x \% m$ 来表示数值 x 除以 m 后所得的余数，这通常读作“ x modulo m ”或“ $x \bmod m$ ”。这样一来， $9 \% 7$ 就得2，因为 $9 \div 7$ 的余数为2。类似地， $24 \% 7$ 为3，因为 $24 \div 7$ 的余数为3； $14 \% 7$ 为0，因为 $14 \div 7$ 的余数为0。注意，如果 x 是一

个0到 $m-1$ 的整数，那么 $x \% m$ 的值就为 x 本身。例如， $4 \% 9$ 的值就为4。

借助数学知识我们知道，如果 p 和 q 是素数， m 是0到 pq （表示 p 和 q 的乘积）的一个整数，那么，对于任意的正整数 k ，就有

$$1 = m^{k(p-1)(q-1)} \% pq$$

尽管在这里不证明这个命题，但考虑一个例子来解释这个命题还是有必要的。于是，我们假设 p 和 q 分别为素数3和5，并且 m 为整数4。那么，这个命题说，对于任意的正整数 k ，值 $m^{k(p-1)(q-1)}$ 除以15（3和5的乘积）将得到余数1。具体来说，如果 $k=1$ ，那么

$$m^{k(p-1)(q-1)} = 4^{1(3-1)(5-1)} = 4^8 = 65\,536$$

正如前面所讲的，该值除以15所得余数为1。而且，如果 $k=2$ ，那么

$$m^{k(p-1)(q-1)} = 4^{2(3-1)(5-1)} = 4^{16} = 4\,294\,967\,296$$

再将它除以15所得的余数为1。事实上，无论正整数 k 取什么值，都将得到余数1。

12.6.2 RSA公钥密码学

现在，我们准备在RSA算法的基础上构建和分析一个公钥加密系统。首先，选出两个不同的素数 p 和 q ，其乘积用 n 来表示。然后，另外选出两个正整数 e 和 d ，使得对于某个正整数 k ，满足 $e \times d = k(p-1)(q-1)$

+1。我们之所以将这些值称为 e 和 d 是因为它们分别是加密和解密过程的一部分。（事实上，所选取的值 e 和 d 能够满足前面的等式，这在数学上也是另一个已经证明过的事实，在这里我们不再证明。）

因此，这里选取了5个值： p 、 q 、 n 、 e 和 d 。值 e 和 n 是加密密钥，值 d 和 n 是解密密钥，值 p 和 q 只是用来构建加密系统的。

这里就考虑一个具体的例子来进行说明。假设将值 p 和 q 分别选取为7和13，那么 $n=7\times 13=91$ 。而且，将值 e 和 d 分别选取为5和29，因为 $5\times 29=145=144+1=2(7-1)(13-1)+1=2(p-1)(q-1)+1$ ，这正是所需的。这样一来，加密密钥就是 $n=91$ 和 $e=5$ ，而解密密钥则为 $n=91$ 和 $d=29$ 。我们将加密密钥分发给想要给我们发报文的人，而将解密密钥（以及 p 和 q 的值）留给自己。

我们现在来考虑如何对报文进行加密。为此，假设当前报文是按照位模式（可能用的是ASCII码或Unicode码）编码的，当将其解释为二进制表示时，这个位模式的值就小于 n 。（如果它不小于 n ，就要把报文切分成较小的段，然后分别对每段进行加密。）

假设当报文解释为二进制表示时，我们的报文表示值 m 。那么，这条报文的加密形式就是值 $c = m^e \% n$ 的二进制表示。也就是说，加密过的报文是 m^e 除以 n 后所得余数的二进制表示。

具体来说，继续就前面的例子来进行讨论，如果某人想要用加密密钥 $n=91$ 和 $e=5$ 对报文10111进行加密，他首先会看出，10111是值23的二进制表示，那么计算 $23^e = 23^5 = 6\,436\,343$ ，最后，将这个值除以 n

= 91，得到余数4。所以这条报文的加密形式就是100，即4的二进制表示。

为了解密一条用二进制记法表示的值为 c 的报文，就要计算 $c^d \% n$ 的值。也就是说，计算 c^d 的值，将结果再除以 n ，并保留所得的余数。事实上，这个余数就是初始报文的值 m ，因为

$$\begin{aligned}c^d \% n &= m^{e \times d \% n} \\&= m^{k(p-1)(q-1)+1 \% n} \\&= m \times m^{k(p-1)(q-1) \% n} \\&= m \% n \\&= m\end{aligned}$$

正如前面所述，这里用到了 $m^{k(p-1)(q-1) \% n} = m^{k(p-1)(q-1) \% pq} = 1$ ，以及 $m \% n = m$ （因为 $m < n$ ）。

继续讨论前面的例子，如果收到的报文是100，那么我们就识别出这个值为4，然后计算出 $4^d = 4^{29} = 288\,230\,376\,151\,711\,744$ ，将这个值除以 $n = 91$ ，从而得到余数23，即为用二进制记数法表示的初始报文10111。

概括来说，一个RSA公钥加密系统的产生过程是这样的：选取两个素数 p 和 q ，再从这两个数产生值 n 、 e 和 d 。值 n 和 e 被用于加密报文，即为公钥；而值 n 和 d 被用于解密报文，即为私钥（见图12-20）。这个系统的优势就在于只知道如何加密报文，而不能解密报文。这样一来，加密密钥 n 和 e 就可以被广泛地分发。就算你的对手得到这些加密

密钥，也无法对他们所截获的报文进行解密，因为只有知道解密密钥的人才能对报文进行解密。

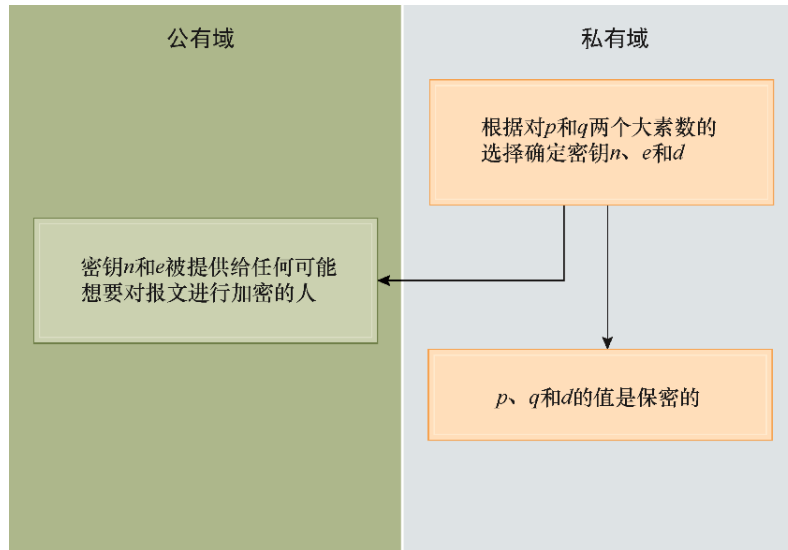


图12-20 建立一个RSA公钥加密系统

这个系统的安全性的基础在于，假定只知道加密密钥 n 和 e ，而不允许计算解密密钥 n 和 d 。然而，确实有这样的算法能做到！一种方法是对值 n 进行因式分解找到 p 和 q ，然后找一个 k 值，使 $k(p-1)(q-1)+1$ 被 e 整除（那么商就为 d ），从而确定 d 。另外，这个过程的第一步就会很耗时，特别是在所选的 p 和 q 很大的情况。事实上，如果 p 和 q 大到需要用几百个二进制位来表示，那么即使用最好的因式分解算法对 n 进行分解，也需要好几年的时间才能确定 p 和 q 。因此，一条加密过的报文的内容，即使其重要性已经过时很久，它的安全性仍然能得到保证。

时至今日，还没有人能够在不知道解密密钥的情况下找到一种对基于RSA密码学加密的报文进行解密的有效方法，因此，基于RSA算法的公钥加密技术广泛地用于因特网通信，以获取通信的秘密性。

12.6节问题与练习

1. 请找出66 043的因数。（本题可能比较耗时，不必花费太多的时间。）
2. 用公钥 $n = 91$ 和 $e = 5$ 对消息101进行加密。
3. 用私钥 $n = 91$ 和 $d = 29$ 对消息10进行解密。
4. 在一个RSA公钥密码学系统中，根据素数 $p = 7$ 、 $q = 19$ 以及加密密钥 $e = 5$ ，为解密密钥 n 和 d 找出合适的值。

复习题

1. 请说明一下，如何用Bare Bones语言模拟下列形式的结构：

```
while X equals 0:  
  .  
  .  
  .
```

2. 编写一个Bare Bones程序，使得：如果变量 X 小于等于变量 Y ，则将变量 Z 置为1；否则，将变量 Z 置为0。
3. 编写一个Bare Bones程序，将变量 Z 置为2的 X 次方。
4. 为以下每种情况编写一个Bare Bones程序序列，实现指定的活动。
 - a. 如果 X 的值为偶数，则将 Z 赋值为0；否则，将 Z 赋值为1。

b. 计算整数0 ~ X的和。

5. 编写一个Bare Bones例程，用值X除以值Y，忽略余数。也就是说，1除以2得0，5除以3得1。

6. 描述由下列Bare Bones语言程序计算的函数，假设该函数的输入由X和Y表示，输出由Z表示：

```
copy X to Z
copy Y to Aux
while Aux not 0:
    decr Z
    decr Aux
```

7. 描述由下列Bare Bones语言程序计算的函数，假设该函数的输入由X和Y表示，输出由Z表示：

```
clear Z
copy X to Aux1
copy Y to Aux2
while Aux1 not 0:
    while Aux2 not 0:
        decr Z
        decr Aux2
    decr Aux1
```

8. 编写一个Bare Bones语言程序，用于计算变量X和变量Y的异或，并将结果存放在变量Z中。你可以假设X和Y只从整数0和1开始。

9. 如果我们允许Bare Bones程序中的指令用整数值标号，并且把while循环结构用形如

```
if name not 0:
    goto label;
```

的条件转移指令代替，其中，name是任意一个变量，label是一个整数值，用来标记别处的一条指令，那么请证明：这个新语言仍然是一种通用程序设计语言。

10．在本章中，我们已经看到，语句

```
copy name1 to name2
```

是如何用Bare Bones语言来模拟的。请证明：如果Bare Bones语言中的while循环结构用一个形如

```
repeat:...  
  until (name equals 0)
```

的后测试循环结构来代替，那么上述语句仍能够被模拟。

11．证明：如果while语句用一个形如

```
repeat:  
  ...  
  until (name equals 0)
```

的后测试循环结构来代替，Bare Bones语言仍为通用程序设计语言。

12．设计一个图灵机，要求它一旦启动，将不会使用磁带上一个以上的单元，而且永不会到达停止状态。

13．设计一个图灵机，要求将当前单元左边的所有单元置0，直到遇到一个包含星号的单元为止。

14. 假设图灵机的磁带上0和1模式串的两端是用星号作为分界符的，那么请设计一个图灵机，将此模式左移一个单元，这里假设机器是从模式右端的星号处开始的。

15. 设计一个图灵机，要求把在当前单元（它包含一个星号）和其左边的第一个星号之间所发现的0、1模式串倒转。

16. 概述丘奇-图灵论题。

17. 下面的Bare Bones程序是自终止的吗？请说明理由。

```
copy X to Y
incr Y
incr Y
while X not 0:
    decr X
    decr X
    decr Y
    decr Y
decr Y
while Y not 0:
    incr X
    decr Y
while X not 0:
```

18. 下面的Bare Bones程序是自终止的吗？请说明理由。

```
while X not 0:
```

19. 下面的Bare Bones程序是自终止的吗？请说明理由。

```
while X not 0:
    decr X
```

20. 分析下面一对命题的有效性：

The next statement is true. The previous statement is false.

21．分析命题“船上的厨师为所有人做饭，但只为那些自己不做饭的人做饭。”的有效性。（提示：谁为厨师做饭？）

22．假设你在一个国家，这个国家的人要么是说真话的人，要么是说假话的人。（说真话的人总是说真话，说假话的人总是说假话。）那么，你问一个人一个什么问题就能判断出这个人是说真话的人还是说假话的人。

23．请概括说明，图灵机在理论计算机科学领域中的重要性。

24．请概括说明，停机问题在理论计算机科学领域中的重要性。

25．假设你要在一群人中找出是否有人在某一天过生日。一种方法是询问这群人中的每个成员一次。如果你采用这种方法，那么出现何种情况会让你知道存在这样的人，出现何种情况会让你知道不存在这样的人？现在，假设要找出至少一个具有某种特征的正整数，你可以应用同样的方法对整数一次一个地进行系统测试。事实上，如果某个整数具有这种特征，你会怎样把它找出来？如果没有整数具有这种特征，你是怎么知道的？为了确定一个推测是否为真，是不是必须与确定这个推测是否为假对称地进行测试？

26．在一个列表中搜索某个值的问题是多项式问题吗？请证明你的答案。

27. 设计一个算法，确定一个给定的正整数是否为素数。你的解是否高效？你的解是多项式解还是非多项式解？

28. 一个问题的多项式解是不是一直都比其指数解要好？请说明理由。

29. 一个问题有多项式解这样一个事实，是否就意味着它能一直在实际可行的时间内求解？请说明理由。

30. 程序员查理要解决这样一个问题：将一群人（人数为偶数）分成人数相同的两个小组，要使每个小组的总年龄之间的差尽可能大。他提出的解决方案是：先构建所有可能的一对小组，然后计算每对小组的总年龄之间的差，然后选差最大的那一对。但是，程序员玛丽提出的解决方案是：先将初始组按年龄排序，再分成两个小组，年龄较小的一半为一组，年龄较大的一半为另一组。这两种解决方案的复杂性分别是什么？这个问题本身是属于多项式复杂性、NP复杂性还是非多项式复杂性？

31. 为什么对列表的排序问题而言，先生成列表的所有可能排列，然后选出所需的那种排列，不是一个令人满意的对列表进行排序的方法？

32. 假设一种彩票基于的是正确选择4个整数值，每个值的范围都是1~50。同时，假设累计奖金已经大到对每种可能的组合分别买一张彩票就能获利的地步。如果买一张彩票需要花一秒的时间，那么为每种可能的组合都买一张彩票要花多长时间？如果彩票需要选取5个数，

而不是4个数，那么所需时间将如何变化？依据本章所讨论的内容，这个问题必须做些什么？

33．下面的算法是确定性的吗？请说明理由。

```
def mystery (Number):  
    if (Number > 5):  
        回答"yes"  
    else:  
        挑一个比5小的数并将这个数作为答案给出
```

34．下面的算法是确定性的吗？请说明理由。

一直向前开。
在第三个十字路口，问站在拐角处的人你应该向右转还是应该向左转。
根据那个人指的方向转弯。
再开两个街区，然后停下来。

35．请确定下列算法中非确定性的点：

```
从1~100之间选3个数。  
if (如果所选数之和大于150):  
    回答"yes"  
else:  
    选择已选出的数中的一个，将该数作为答案
```

36．下列算法是具有多项式时间复杂性还是具有非多项式时间复杂性？请说明理由。

```
def mystery (ListOfNumbers):  
    从ListOfNumbers中选一组数。  
    if (这些数加起来得125):  
        回答"yes"  
    else:  
        不给出答案
```

37．下列问题中，哪些问题是属于P类的？

a．复杂性为 n^2 的问题

b．复杂性为 $3n$ 的问题

c．复杂性为 n^2+2n 的问题

d．复杂性为 $n!$ 的问题

38．概述声明一个问题是多项式问题和声明一个问题是非确定性多项式问题之间的区别。

39．请举出一个既属于P类又属于NP类的问题的例子。

40．假设给你两个解决同一个问题的算法。一个算法的时间复杂性为 n^4 ，另一个算法的时间复杂性为 $4n$ ，那么在什么规模的输入上前者比后者更为有效？

41．假设我们要解决涉及15个城市的旅行商问题，其中任意两个城市之间只有一条路相连。有多少条不同的穿越所有城市的路径？这里假设每条路径的长度能够在1微秒内计算出来，那么计算出所有这些路径的长度要花多长时间？

42．如果将归并排序算法（见图12-15和图12-14）应用到列表Alice、Bob、Carol和David，那么要做多少次名字的比较？如果应用到列表Alice、Bob、Carol、David和Elaine，那么又将做多少次名字的比较？

43 . 请为图12-18所示的每一类问题都举一个例子。

44 . 设计一个算法，找到形如 $x^2 + y^2 = n$ 的等式的整数解，这里， n 是某个给定的正整数。确定你的算法的时间复杂性。

45 . **背包问题** (knapsack problem) 是另一个属于NP完全类的问题。该问题旨在从一个列表找出一些数，使这些数的和等于某个值。例如，列表

642	257	771	388	391	782	304
-----	-----	-----	-----	-----	-----	-----

中的数据项257、388和782，它们的和为1 427。请找出其和为1 723的数据项。你用的是什么算法？其复杂性如何？

46 . 请指出旅行商问题与背包问题的相似之处（见第45题）。

47 . 下面的列表排序算法称为**冒泡排序**。当将冒泡排序用到一个有 n 个数据项的列表中时，它需要在列表项中进行多少次比较？

```
def BubbleSort(List):
    Counter =1
    while (Counter < List中的项的个数):
        N = List中的项的个数
        while (N >1):
            if(List中的第N个项小于其前面的项):
                第N个项和前面的一项交换
            N = N-1
```

48 . 用RSA公钥加密技术对报文110进行加密，这里公钥为 $n=91$ 和 $e=5$ 。

49．用RSA公钥加密技术对报文111进行解密，这里私钥为 $n=133$ 和 $d=5$ 。

50．假设你知道一个基于RSA算法的公钥加密系统，其公钥为 $n=77$ 和 $e=7$ 。私钥是什么？怎样才能让你在一个合理的时间内解决这个问题？

51．找出107 531的因数。这个问题与本章的内容有什么关联？

52．如果正整数 n 在2到 n 的平方根范围内没有整数因子，那么我们能够得出什么结论？对于找一个正整数的因子这样的任务来说，这又告诉了你什么？

社会问题

希望下面的问题能引导读者思考一些与计算领域相关的道德、社会和法律问题。回答出这些问题还不够，还应该考虑为什么这样回答，以及你的判断是否对每个问题都标准如一。

1．假设求解一个问题的最优算法需要执行100年，那么你会认为这个问题是容易处理的吗？为什么？

2．公民有权在免受政府机构监控的情况下对报文进行加密吗？你的回答是否考虑到了“合适的”法律执行？应该谁来决定“合适的”法律执行是什么？

3. 如果人脑是一个算法设备，那么关于人性，图灵论题会有什么结果？你在多大程度上认为图灵机囊括了人脑的计算能力？

4. 我们已经看到，不同的计算模型（如有限表、代数公式、图灵机等）有不同的计算能力。不同的生物体也有不同的计算能力吗？不同的人的计算能力上有差别吗？如果有，那么具有较高能力的人应该使用那些能力来获得更好的生活方式吗？

5. 今天，有许多网站和智能手机应用提供几乎任何城市的道路图。这些网站能够帮助找到某个具体的地址，并提供放大功能来看清小范围的街区布局。这些地图网站中许多都包括具有类似缩放功能的卫星相片，能够提供单个建筑物和周围景观的详细的图像。从这个事实开始，考虑下面的一系列的假想。假设这些图像又增强到包括实时视频。假设这些实时视频图像又通过红外技术增强了。此时，其他人就能够一天24小时地监视你的家了。在这一系列技术进步中，你的隐私权是在哪个时间点开始受到侵犯的？在这一系列技术进步中，你认为我们是在哪个时间点上超越了当今间谍卫星技术的能力的？在多大的程度上这个场景就只是虚构的？

6. 假设一家公司开发了一个加密系统，并取得了专利权。该公司所在国家的政府机构是否有权以国家安全的名义来使用这个系统？该公司所在国家的政府机构是否有权以国家安全的名义限制这家公司对这个系统的商业使用？如果这家公司是跨国公司，那么情况又会怎样？

7. 假设你买了一个产品，其内部结构是加密的，那么你有权对这个产品的基本结构进行解密吗？如果有，你有权以商业方式使用这些信息吗？以非商业方式使用呢？如果它的加密是用一个秘密的加密系统实现的，并且你发现了这个秘密呢？你有权分享这个秘密吗？

8. 很多年以前，哲学家约翰·杜威（1859—1952）提出了“履责技术”（responsible technology）这个术语。请举出一些例子，说明你对“履责技术”是怎样理解的。在你给出的这些例子的基础上，阐明你自己对“履责技术”的定义。在过去的100多年里，社会实践过“履责技术”吗？应当采取措施来保证它的实施吗？如果应该，应采取什么样的措施？如果不应该，为什么？

课外阅读

Garey, M. R. and D. S. Johnson. *Computers and Intractability*. New York: W. H. Freeman, 1979.

Hofstadter, D. R. *Gödel, Escher, Bach: An Eternal Golden Braid*. St. Paul, MN: Vintage, 1980.

Hopcroft, J. E., R. Motwani, and J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA: Addison-Wesley, 2007.

Lewis, H. R. and C. H. Papadimitriou. *Elements of the Theory of Computation*, 2nd ed. Englewood Cliffs, NJ: Prentice-Hall, 1998.

Rich, E. *Automata. Computability and Complexity: Theory and Application*. Upper Saddle River, NJ: Prentice-Hall, 2008.

Richards, D. and H. Hamburger. *Logic and Language Models for Computer Science*, 3rd ed. Singapore: World Scientific, 2017.

Sipser, M. *Introduction to the Theory of Computation*, 3rd ed. Boston, MA: Cengage Learning, 2012.

Smith, C. and E. Kinber. *Theory of Computing: A Gentle Introduction*. Englewood Cliffs, NJ: Prentice-Hall, 2001.

Sudkamp, T. A. *Languages and Machines: An Introduction to the Theory of Computer Science*, 3rd ed. Boston, MA: Addison-Wesley, 2006.

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书
配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

附录A ASCII码

下面列出了ASCII码的一部分，并在每个位模式的左边都添加了一个0，以构成现在通用的8位位模式。第三列给出的是每个8位位模式对应的十六进制值。

符号	ASCII	十六进制值	符号	ASCII	十六进制值	符号	ASCII	十六进制值
换行	00001010	0A	>	00111110	3E	^	01011110	5E
回车	00001011	0D	?	00111111	3F	-	01011111	5F
空格	00100000	20	@	01000000	40	`	01100000	60
!	00100001	21	A	01000001	41	a	01100001	61
"	00100010	22	B	01000010	42	b	01100010	62
#	00100011	23	C	01000011	43	c	01100011	63
\$	00100100	24	D	01000100	44	d	01100100	64
%	00100101	25	E	01000101	45	e	01100101	65
&	00100110	26	F	01000110	46	f	01100110	66
'	00100111	27	G	01000111	47	g	01100111	67
(	00101000	28	H	01001000	48	h	01101000	68
)	00101001	29	I	01001001	49	i	01101001	69
*	00101010	2A	J	01001010	4A	j	01101010	6A
+	00101011	2B	K	01001011	4B	k	01101011	6B
,	00101100	2C	L	01001100	4C	l	01101100	6C
-	00101101	2D	M	01001101	4D	m	01101101	6D
.	00101110	2E	N	01001110	4E	n	01101110	6E
/	00101111	2F	O	01001111	4F	o	01101111	6F
0	00110000	30	P	01010000	50	p	01110000	70
1	00110001	31	Q	01010001	51	q	01110001	71
2	00110010	32	R	01010010	52	r	01110010	72
3	00110011	33	S	01010011	53	s	01110011	73
4	00110100	34	T	01010100	54	t	01110100	74
5	00110101	35	U	01010101	55	u	01110101	75
6	00110110	36	V	01010110	56	v	01110110	76
7	00110111	37	W	01010111	57	w	01110111	77
8	00111000	38	X	01011000	58	x	01111000	78
9	00111001	39	Y	01011001	59	y	01111001	79
;	00111010	3A	Z	01011010	5A	z	01111010	7A
:	00111011	3B	[	01011011	5B	{	01111011	7B
<	00111100	3C	\	01011100	5C		01111100	7C
=	00111101	3D	]	01011101	5D	}	01111101	7D

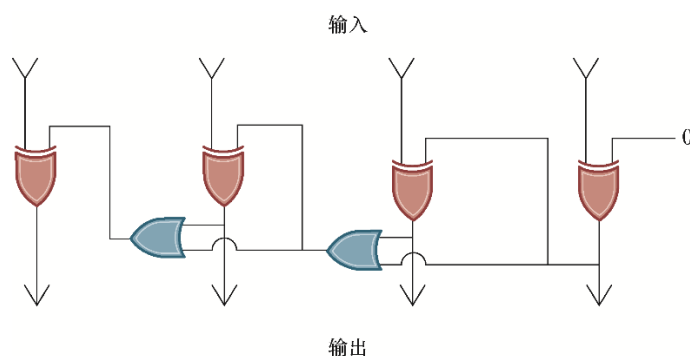
读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

附录B 用于处理二进制补码表示的电路

本附录介绍用于对二进制补码表示的值进行取反和相加的电路。我们从图B-1中的电路开始，它将一个4位的二进制补码表示转换为该值的负值的表示。例如，假设给定3的二进制补码表示，则该电路产生-3的表示。它完成这个操作所用的算法与正文中介绍的一样。也就是说，它自右向左复制位模式，直至复制到一个1，然后随着它从输入向输出移动，求剩余各位的补码。因为最右边XOR门的一个输入值固定为0，所以此门只将其另一个输入传给输出。但是，这个输出又向左传给下一个XOR门作为一个输入。如果这个输出是1，那么下一个XOR门将对其输入位取反后送给输出。而且，这个1还会通过OR门向左传送，影响下一个门。用这种方式，第一个被复制到输出的1也会向左传送，使得所有剩余的位在送到输出时都被取反。



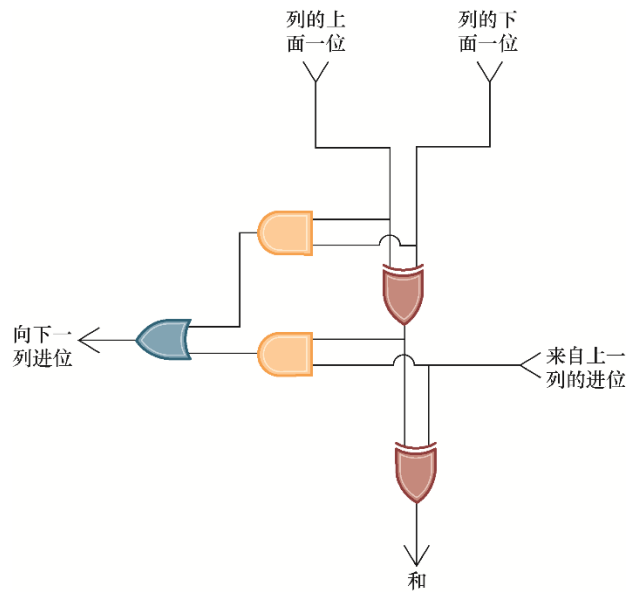
图B-1 对一个二进制补码位模式取反的电路

接下来，我们考虑一下用二进制补码表示的两个数值相加的过程。具体来说，求解

<pre>+ 0110 - - - - + 1011</pre>

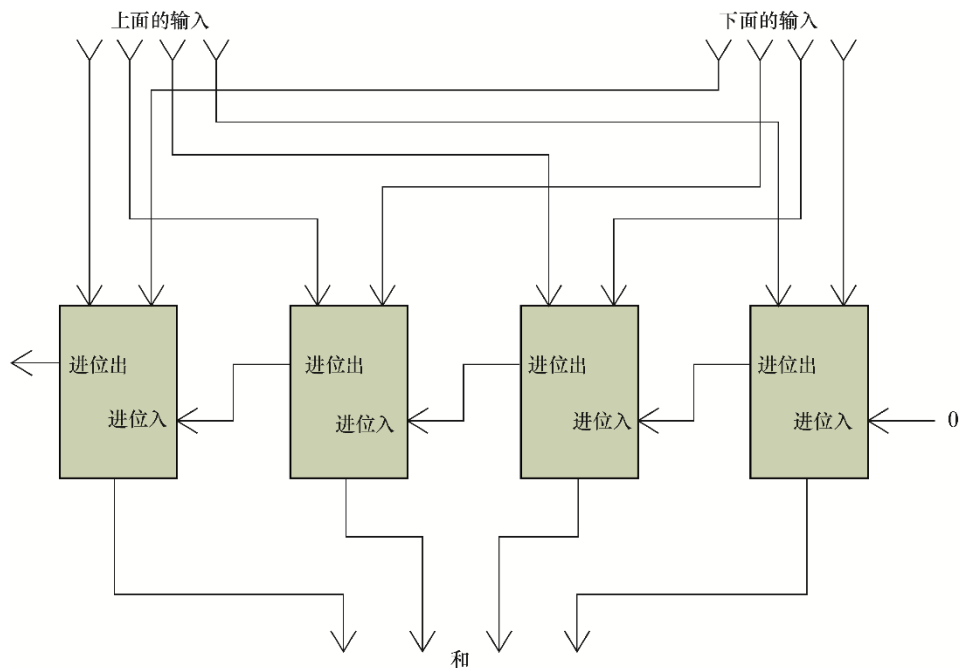
时，我们是从右到左一列一列地计算，并且每一列都执行相同的算法。因此，只要获得这类问题的一列的加法电路，就能通过重复这个单列电路来构造多列加法电路。

多列加法问题中的单列相加算法是这样的：将当前列中的两个值相加，把相加的和与上一列的进位相加，并将此和的最低有效位记为答案，然后将进位传向下一列。图B-2中的电路遵循的就是这个算法。上面的XOR门决定了两个输入位的和。下面的XOR门将这个和与上一列的进位相加。两个AND门和这个OR门一起把进位向左传送。具体来说，如果该列中最初的两个输入是1，或者这些位和进位的和都是1，就会产生一个进位1。



图B-2 在一个多列加法问题中进行单列相加的电路

图B-3展示了如何用单列电路的副本来制造用于计算4位二进制补码系统表示的两个值的和的电路。图中每个矩形表示单列加法电路的一个副本。需要注意的是：最右边的矩形的进位值总是0，因为它没有来自上一列的进位。以此类推，最左边的矩形产生的进位会被忽略。



图B-3 使用图B-2中电路的4个副本将2个二进制补码表示的数值相加的电路

因为进位信息必须从最右边的列向最左边的列传播，或者说波动，所以图B-3中的电路称为行波进位加法器（ripple adder）。尽管这类电路结构简单，但执行速度较慢，而一些更好的电路，例如，先行进位加法器，就可以将这种列到列的传播减到最小。因此，尽管图B-3中的电路足以满足我们的要求，但并不是当今机器中使用的电路。

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

附录C Vole：一种简单的机器语言

本附录介绍一种简单却有代表性的机器语言，以将要运行它的简单假想机器命名。我们首先解释一下这一机器本身的体系结构。

C.1 Vole体系结构^[1]

假设Vole计算机有16个通用寄存器，编号为0x0 ~ 0xF（十六进制表示），每个寄存器的长度为1字节（8位）。为了在指令中标识寄存器，每个寄存器被分配了唯一的4位模式，用于表示其寄存器号。于是，寄存器0x0由二进制0000标识，寄存器0x4由二进制0100标识。

[1] 田鼠（vole）是世界上大部分地区都有的粗壮短尾啮齿类动物。像本附录中描述的处理器的样子，它们很小但很有效。

Vole的主存储器中有256个单元，每个单元都被分配了一个唯一的地址，地址范围为整数0 ~ 255。因此，一个地址可以由00000000 ~ 11111111（或者是十六进制的0x00 ~ 0xFF）范围内的一个8位模式来表示。

假设浮点值以1.7节讨论过、图1-24中概括过的8位格式存储。

C.2 Vole的机器语言

每条Vole机器指令都是2字节长：前面的4位是操作码，后面的12位是操作数字段。下表列出了用十六进制记数法表示的指令及其简要说明。字母R、S和T用来在表示寄存器标识符的字段处替代十六进制数字，寄存器标识符根据指令的具体应用而变化。字母X和Y用来在不表示寄存器的变量字段处替代十六进制数字。

操作码	操作数	说明
0x1	RXY	在地址为XY的存储单元中找到的位模式加载（LOAD）寄存器R 例：0x14A3将使得地址为0xA3的存储单元的内容放入寄存器0x4
0x2	RXY	以位模式XY加载（LOAD）寄存器R 例：0x20A3将使得值0xA3放入寄存器0x0
0x3	RXY	将寄存器R中的位模式存储（STORE）在地址为XY的存储单元中 例：0x35B1将使得寄存器0x5的内容放入地址为0xB1的存储单元
0x4	0RS	将寄存器R中的位模式移入（MOVE）寄存器S 例：0x40A4将使得寄存器0xA的内容复制到寄存器0x4
0x5	RST	将寄存器S和寄存器T中的位模式相加（ADD），就像它们是二进制补码表示一样，将结果存放在寄存器R中 例：0x5726将使得寄存器0x2和寄存器0x6中的二进制数值相加，并将和存放在寄存器0x7中

操作码	操作数	说明
0x6	RST	将寄存器S和寄存器T中的位模式相加（ADD），就像它们是用浮点记数法表示的值，并将浮点结果存放在寄存器R中 例：0x634E将使得寄存器0x4和寄存器0xE中的值相加，并将结果存放在寄存器0x3中
0x7	RST	对寄存器S和寄存器T中的位模式做或（OR）运算，将结果存放在寄存器R中 例：0x7CB4将使得寄存器0xB和寄存器0x4的内容的或运算结果存放在寄存器0xC中
0x8	RST	对寄存器S和寄存器T中的位模式做与（AND）运算，将结果存放在寄存器R中 例：0x8045将使得寄存器0x4和寄存器0x5的内容的与运算结果存放在寄存器0x0中
0x9	RST	对寄存器S和寄存器T中的位模式做异或（EXCLUSIVE OR）运算，将结果存放在寄存器R中 例：0x95F3将使得寄存器0xF和寄存器0x3的内容的异或运算结果存放在寄存器0x5中
0xA	R0X	将寄存器R中的位模式向右循环移位（ROTATE）一位，进行X次。每次都把从低位端开始的那个位放入高位端 例：0xA403将使得寄存器0x4中的内容循环右移3位

操作码	操作数	说明
0xB	RXY	如果寄存器R中的位模式等于0号寄存器中的位模式，那么跳转（JUMP）到地址XY处的存储单元中的指令；否则，继续正常的执行顺序（转移是通过在执行阶段将XY复制到程序计数器来实现的） 例： 0xB43C将首先比较寄存器0x4和寄存器0x0中的内容。如果二者相等，则把模式0x3C放入程序计数器，以便要执行的下一条指令是位于这个存储地址的指令；否则，不做任何操作，程序按照正常顺序继续
0xC	000	停止（HALT）执行 例： 0xC000将使得程序停止执行

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书
配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

附录D 高级程序设计语言

本附录包含了第6章中作为例子使用的每种语言的简要背景。

D.1 Ada

Ada语言是以奥古斯塔·艾达·拜伦（Augusta Ada Byron）（1815—1851）的名字命名的，她是查尔斯·巴贝奇（Charles Babbage）的拥护者、诗人拜伦勋爵（Lord Byron）的女儿。Ada语言最初是由美国国防部开发的，开发目的是得到一种满足其所有软件开发需要的通用语言。Ada语言设计期间的重点是加入实时计算机系统程序设计的特性，这类系统常用来作为较大型机器的一部分，如导弹制导系统、建筑物内环境控制系统以及汽车和小型家用电器中的控制系统。因此，Ada语言包含用于表达并行处理环境中活动的功能，以及用于处理应用环境中可能出现的特殊情况（称为异常）的便利技术。尽管初期是作为命令式语言设计的，但Ada语言的较新版本中包含了面向对象范型。

Ada语言的设计一直强调能实现可靠软件高效开发的特性，下面这个事实例证了这个特性：波音777飞机中所有内部控制软件都是用Ada语言编写的。这也是Ada被用作SPARK语言开发起点的主要原因，如第5章所述。

D.2 C

C语言是由贝尔实验室的丹尼斯·里奇（Dennis Ritchie）于20世纪70年代初期开发的。尽管它最初是作为开发系统软件的语言设计的，但却在整个程序设计界颇为流行，并且已经被美国国家标准协会标准化。

C语言最初的设想只是跨出机器语言的一步而已。因此，与使用完整的英语单词来表达一些原语的其他高级语言相比，C语言的语法很简洁，其原语都用专门符号表示。它的这种简洁性允许高效表达复杂算法，这也是C语言广泛流行的一个主要原因。（通常说来，简洁的表示比冗长的表示更易读。）

D.3 C++

C++语言是贝尔实验室的本贾尼·斯特劳斯特鲁普（Bjarne Stroustrup）作为C语言的一个增强版本开发的。目的是开发一种可以与面向对象范型相兼容的语言。如今，C++不但凭其自身的实力成了著名的面向对象语言，而且是另外两种主要面向对象语言Java和C#的开发起点。

D.4 C

C#语言是由微软公司开发的，是.NET框架中的工具，它是为运行微软系统软件的机器开发应用软件的综合系统。C#语言与C++和Java

看起来非常相似。事实上，微软公司将C#作为一种不同的语言来推介并不是因为它是一种全新的语言，而是因为它作为一种不同的语言，微软可以根据自己的需要定制语言的一些专门特性，而不必考虑与其他语言相关的标准，也不需要考虑其他公司的专利权问题。因此，C#的创新性在于：在利用.NET框架开发软件方面，它的作用非常突出。拥有微软公司的支持，在未来的几年里，C#和.NET框架一定能够成为软件开发界的佼佼者。

D.5 FORTRAN

FORTRAN是FORmula TRANslator（公式翻译器）的简写。FORTRAN语言是最早开发的高级语言之一（发布于1957年），也是在计算界最早获得广泛认同的语言之一。多年来，它的官方描述经历了许多次的扩充，这意味着今天的FORTRAN语言与原始版本有很大的不同。事实上，通过研究FORTRAN语言的演变，能够见证研究对程序设计语言设计产生的影响。尽管最初是作为命令式语言设计的，但FORTRAN的较新版本已经包含了许多面向对象的特性。FORTRAN语言在科学界仍然是一种很流行的语言。具体来说，许多数值分析和统计软件包都是用FORTRAN语言编写的，并且可能会继续用FORTRAN语言编写。

D.6 Java

Java是Sun微系统公司在20世纪90年代早期开发的一种面向对象语言。它的设计者在很大程度上借鉴了C语言和C++语言。Java令人兴奋的不是语言本身，而是该语言的通用实现和Java编程环境中可用的大量预先设计好的模板。通用实现意味着用Java语言编写的程序能够在各种不同的机器上高效执行；模板的可用性意味着可以相对容易地开发复杂软件。例如，applet（小应用程序）及servlet（小服务程序）等模板使万维网软件的开发更加流畅。

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

附录E 迭代结构与递归结构的等价性

在本附录中，我们使用第12章中的Bare Bones语言作为工具来回答第5章中提出的关于迭代结构与递归结构谁更强大的问题。回忆一下，Bare Bones语言只包含3个赋值语句（clear、incr以及decr）和一个控制结构（由while语句构成）。这种简单的语言与图灵机具有相同的计算能力，因此，如果我们接受丘奇-图灵论题，就可以得出结论：任何具有算法解的问题都有可用Bare Bones表达的解。

迭代结构与递归结构进行比较的第一步是将Bare Bones语言的迭代结构替换成递归结构。具体方法如下：将while语句从该语言中移出，并在它的位置上提供可以把Bare Bones程序分成部分单元的能力，以及可以从程序的其他地方调用这些单元之一的能力。更准确地说，我们建议每个用修改后的语言编写的程序都由许多语法上不相交的程序单元组成。我们假设每个程序必须正好包含一个称为MAIN的单元，MAIN的句法结构如下：

```
def MAIN():  
    .  
    .  
    .
```

（其中点表示其他缩进的Bare Bones语句）可能还包含具有以下结构的其他单元（语义上从属于MAIN）：

```
def unit():  
    .  
    .  
    .
```

（其中unit__表示单元名，有和变量名一样的语法。）这种分割结构的语义是：程序总是从MAIN单元的开头开始执行，并在该单元的缩进部分执行完成时停止。除MAIN之外的程序单元都可以通过条件语句

```
if name not 0:  
    unit()
```

作为函数来调用（其中name表示任何变量名，unit表示除MAIN之外的其他任何程序单元名）。此外，我们允许MAIN之外的其他单元递归地调用自己。

有了这些附加的特性，我们就可以模拟原始Bare Bones中的while结构了。例如，一个如下形式的Bare Bones程序：

```
while X not 0:  
    S
```

（其中S表示任何Bare Bones语句序列），它可以替换为下面的单元结构：

```
def MAIN():  
    if X not 0:  
        unitA()  
def unitA():  
    S  
    if X not 0:  
        unitA()
```

因此，我们得出结论：修改后的语言具有原始Bare Bones语言的所有能力。

也可以说明，任何能用修改后的语言解决的问题都能用Bare Bones解决。做到这一点的一种方法就是，说明任何用修改后的语言表达的算法都可以用原始的Bare Bones语言编写。不过，这涉及“递归结构如何用Bare Bones语言的while结构来模拟”的一个精确描述。

针对我们的目的，比较简单的就是根据第12章中介绍的丘奇—图灵论题。具体来说，丘奇-图灵论题，加上Bare Bones与图灵机具有相同的能力这个事实，表明了没有比原始Bare Bones更强大的语言了。因此，任何能用我们修改后的语言解决的问题也能用Bare Bones解决。

我们得出的结论是，修改后的语言的能力与原始Bare Bones的能力一样。两种语言之间的唯一区别是：一种提供的是迭代控制结构，另一种提供的是递归控制结构。因此，这两种控制结构在计算能力方面实际是等价的。

读者服务：



微信扫码关注【异步社区】微信公众号，回复“e58263”获取本书配套资源以及异步社区15天VIP会员卡，近千本电子书免费畅读。

索引

符号

2D图形学, 343

3D图形学, 343

A*算法, 383

ANSI, 26

AP, 118

bit/s, 51

CAD, 29

camel样式, 167

CASE, 249

CASE工具, 249

CD, 24

CD-DA, 24

CISC, 63

CPU, 61

CRC, 53

CRC 卡, 265

CSMA/CA, 120

CSMA/CD, 119

Direct3D, 358

DNS, 128

DoS, 147

DRAM, 21

EOF, 329

FIFO, 96

Gbit/s, 51

GIF, 50

GOMS, 271

Gouraud着色, 357

GUI, 100

HTML, 134

HTTP, 133

HTTPS, 149

I/O指令, 64

ICANN, 127

IDE, 249

IMAP, 131

IP, 142

IP地址, 127

ISP, 125

JPEG, 50

Kbit/s, 51

LAN, 118

LIFO, 279

MAN, 118

Mbit/s, 51

MIMD, 88

MIME, 131

MP3, 51

MPEG, 51

NP完全问题, 421

NP问题, 421

OOP, 207

OpenGL, 358

Pascal样式, 167

POP3, 131

RAM, 21

RISC, 63

RSA算法, 423

SDHC存储卡, 25

SDRAM, 21

SDXC存储卡, 25

SD存储卡, 25

SGML, 136

shell, 100

SIMD, 88

SISD, 88

SSL, 149

TCP, 142

TIFF, 51

TLD, 127

UDP, 142

URL, 133

USB, 76

VLSI, 18

VoIP, 131

VoIP软电话, 131

WAN, 118

WiFi, 120

XML, 136

ZBR, 22

z缓冲区, 356

β 测试, 268

α 测试, 268

A

按引用传递, 221

按值传递, 221

凹凸映射, 357

B

白盒测试, 268

保留字, 225

贝塞尔曲面, 347

贝塞尔曲线, 347

背包问题, 428

边界值分析, 268

编译器, 202

变量, 209

标记, 224

标记语言, 136

标签, 134

标识符, 201

表头, 278

表尾, 278

并发处理, 236

并行处理, 236

并行处理技术, 87

并行通信, 78

病毒, 146

补码, 35

不可解问题, 415

不确定算法, 421

不相容的, 239

布尔型, 210

布尔运算, 15

C

裁剪, 354

参数, 166, 219

参数值, 82

参与者, 262

操作码, 65

操作数, 65

操作系统, 96

测试并置位, 107

插入排序, 179

差分编码, 49

产生式, 376

产生式系统, 376

常量, 212

场景, 346

场景图, 350

超级用户, 110

超取样, 362

陈述性知识, 368

程序, 2

程序化模型, 347

程序计数器, 68

程序设计, 2

程序设计范型, 204

程序设计语言, 162

程序员, 252

持久, 324

抽象, 9

抽象工具, 9

出栈, 279

触发器, 17

传输层, 139

传输速率, 23

串行通信, 78

窗口管理程序, 101

创建子进程, 108

词法分析器, 224

磁带, 23

磁道, 22

磁盘, 22

存储程序概念, 62

存储单元, 20

存储桶, 331

存储映射输入/输出, 77

存取时间, 23

错误决算问题, 326

D

打印服务器, 123

大 Θ 标记, 190

大标记, 417

代理服务器, 148

代码生成, 229

代码生成器, 224

代码优化, 229

带宽, 79

单一化, 240

登录, 110

等待, 105

等价类, 268

低位端, 20

地址, 20

地址多项式, 285

递归, 186

第二层ISP, 125

第一层ISP, 125

点分十进制记数法, 127

电子黑饵, 147

调度程序, 102

调试, 201

调用, 218

调制解调器, 78

迭代模型, 253

动力学, 362

动态存储器, 21

度量学, 248

端口, 76

端口号, 141

断言, 192

队列, 96

对等, 123

对象, 207, 231

多边形网格, 346

多道程序设计, 97, 105

多点传播, 131

多对多联系, 263

多路复用技术, 79

多路广播, 132

多任务, 98

多态, 235

多维数据集, 335

多项式问题, 420

多义文法, 226

E

二叉树, 280

二分搜索, 184

二进制补码, 28, 35

二进制记数法, 27

F

法线, 352

法向量, 357

翻译, 224

翻译器, 202

方法, 231

防病毒软件, 149

防火墙, 148

非确定性多项式问题, 421

非终结符, 225

分布式光线跟踪, 359

分布式数据库, 314

分布式系统, 124

分派程序, 102

分时, 97

分支, 281

分组, 140

封闭世界假设, 385

封闭式, 118

封装, 235

冯·诺依曼瓶颈, 78

冯·诺依曼体系结构, 78

服务器, 123

服务器端, 137

浮点, 39

浮点型, 209

符号表, 227

符号位, 35

辐射度, 360

父, 280

负载平衡, 98, 124

负载因子, 333

赋值语句, 164, 212

G

高可用性, 124

高速缓冲存储器, 62

高位端, 20

各向同性表面, 353

各向异性表面, 353

根节点, 280

根指针, 291

更新丢失问题, 326

公钥, 150, 423

公钥加密, 150, 423

功能内聚, 258

共享锁, 327

构件, 259

构件架构 (, 259

构件装配员, 259

构造器, 233

孤立点分析, 335

固定格式语言, 225

固件, 103

固件更新, 104

故事板, 362

关键帧, 362

关键字, 225

关节变量, 363

关联, 262

关联分析, 335

关系, 315

管理员, 110

光线跟踪, 359

光栅化, 355

广度优先, 380

国家代码顶级域名, 127

过程, 218

过程范型, 205

过程头, 219

过程性知识, 368

H

海量存储, 22

函数, 403

函数式范型, 205

汉明距离, 53

行程长度编码, 48

行主序, 284

黑盒测试, 268

后测试循环, 176

后继条件, 192

后面消除法, 355

互斥, 108

互联网, 121

画家算法, 356

环境光, 352

回滚, 326

回溯, 280

汇编器, 201

汇编语言, 202

霍夫曼编码, 48

J

饥饿, 115

机器人学, 392

机器无关, 202

机器语言, 63

机器指令, 63

机器周期, 68

基本路径测试, 267

基本数据类型, 210

基本条件, 186

基因, 387

基准测试, 68

级联回滚, 326

极限编程, 254

即时编译, 225

集群计算, 124

集线器, 119

记录, 211

技术文档, 269

继承, 234

寄存器, 61

寄存器单元, 61

加密密钥, 423

假脱机, 109

间谍软件, 147

间接寻址, 304

监督学习, 386

监控程序, 237

鉴别, 151

交互段, 265

交互式处理, 97

交互图, 264

交换机, 121

脚本, 208

脚本语言, 208

节点, 280, 376

结构化程序设计, 214

结构化走查, 265

结构图, 255

截断误差, 41

解密密钥, 423

解释器, 202

进程, 104

进程表, 105

进程间通信, 123

进程切换, 105

进程状态, 104

进化规划, 388

镜面反射光, 352

纠错码, 53

就绪, 105

局部变量, 219

局部照明模式, 358

聚类分析, 334

均分, 98

K

开放式, 118

开放源码开发, 254

开始状态, 376

可计算的, 404

客户端, 137

客户机, 123

客户机/服务器, 123

空间复杂性, 416

控制单元, 61

控制点, 361

控制耦合, 257

控制器, 76

控制系统, 376

快速原型开发, 254

宽带, 79

框架问题, 386

L

垃圾回收, 282

垃圾邮件过滤器, 148

类, 207, 231

类图, 262

类型描述, 334

类型提升, 229

类型转换, 229

立即寻址, 303

利益相关者, 251

粒子系统, 348

联合图像专家组, 50

链表, 287

链路层, 139

列表, 278

列模式分析, 335

列主序, 284

邻接表, 287

临界区, 108

浏览器, 133

流, 19

流程图, 175

流量控制, 142

流水线技术, 87

路由器, 122

逻辑程序设计, 205, 240

逻辑内聚, 258

逻辑移位, 74

旅行商问题, 420

M

冒泡排序, 180

门, 16

密钥, 150

面向对象范型, 207

敏捷方法, 254

命令型范型, 205

命令语句, 208

模仿, 386

模块, 255

模块化, 255

模拟电话适配器, 131

模式, 313

目标程序, 224

目标状态, 376

目录, 101

目录路径, 101

N

难解型, 420

内存管理程序, 102

内存泄露, 282

内核, 101

内聚, 258

内联网, 126

扭曲, 361

O

偶校验, 52

耦合, 257

P

帕累托法则, 267

排它锁, 327

抛弃式原型开发, 254

批处理, 96

频率模糊, 51

频率相关编码, 48

平面片, 346

平面文件, 311

平面着色, 356

评审, 267

屏蔽, 73

瀑布模型, 253

Q

欺骗, 148

奇偶校验位, 52

奇校验, 52

启动, 103

启发式, 380

千字节, 21

前测试循环, 176

前提条件, 192

嵌入式系统, 98

强化学习, 386

强人工智能, 372

强制类型转换, 228

清除, 108

丘奇—图灵论题, 407

权, 389

全局变量, 219

全局数据, 257

全局照明模式, 358

群集, 333

R

染色体, 387

人体工程学, 270

认证机构, 150

任务, 236

蠕虫, 147

入射角, 352

入栈, 279

软件, 2

软件分析员, 252

软件需求规格说明, 251

软件许可, 272

软件质量保证 (SQA)

小组, 266

弱人工智能, 372

S

散列, 331

散列表, 331

散列函数, 331

散列文件, 331

散射光, 352

扫描转换, 354

闪存, 25

闪存驱动器, 25

扇区, 22

上下文分析, 373

上下文切换, 105

哨兵, 329

舍入误差, 41

设备驱动程序, 102

设计模式, 265

深度, 280

深度优先, 380

审计软件, 110

渗色, 360

生产, 370

生命线, 264

声明语句, 208

十六进制记数法, 19

时间复杂性, 417

时间片, 105

时钟, 68

实参, 219

实例, 207, 232, 299

实例变量, 231

实型, 209

实用软件, 100

事件驱动, 224

视点, 344

视体, 354

适配器, 265

受伤等待协议, 327

属性, 315

树, 280

数据仓库, 334

数据独立性, 314

数据结构, 210

数据库, 311

数据库管理系统, 314

数据库模型, 314

数据类型, 209

数据流图, 261

数据耦合, 257

数据压缩, 48

数据字典, 261

数字化, 347

数字签名, 151

数组, 210

顺序搜索, 173

顺序文件, 328

说明性范型, 205

私钥, 150, 423

死锁, 108

搜索树, 378

搜索引擎, 6, 137

算法, 2

算术/逻辑单元, 61

算术移位, 74

索引, 210

索引文件, 330

锁定协议, 327

T

特洛伊木马, 147

特权级, 112

特权指令, 112

提交点, 326

条件转移, 64

跳数, 143

停机问题, 411

通用程序设计语言, 407

通用寄存器, 61

同构数组, 278

统一过程, 253

头指针, 287, 289

投影平面, 344

投影线, 344

投影中心, 344

透视投影, 344

图灵测试, 370

图灵可计算的, 407

图像处理, 343, 372

图像窗口, 345

图像分析, 372

图形卡, 357

图形适配器, 357

推理法则, 238, 377

退化条件, 186

吞吐量, 87

W

万维网, 6

万维网服务器, 133

网格计算, 124

网关, 122

网络, 118

网络层, 139

网桥, 121

网站邮件, 137

微处理器, 61

伪代码, 163

尾数域, 39

尾指针, 289

位, 15

位图, 73

谓词, 240

文本编辑器, 27

文本文件, 27

文法, 225

文件服务器, 123

文件管理程序, 101

文件夹, 101

纹理映射, 349

问题空间, 376

握手, 78

无损, 48

无损分解, 318

无条件转移, 64

物体, 346

X

系统软件, 99

系统文档, 269

线程, 236

相对编码, 48

像素, 28

消解, 238

消解式, 238

小数点, 33

校验和, 53

校验字节, 53

协议, 119

信号量, 107

信息检索, 374

信息提取, 374

信息隐藏, 258

形参, 219

形式语言, 203

形状因子, 360

兄弟, 280

嗅探, 147

嗅探软件, 111

虚拟内存, 102

序列图, 264

旋转延迟, 23

选择排序, 180

渲染, 344

渲染流水线, 353

寻道时间, 23

循环, 173

循环不变式, 192

循环队列, 290

循环移位, 74

训练集, 386

Y

掩码, 73

演化式原型开发, 254

叶子节点, 280

页面, 102

页面调度, 102

一对多联系, 263

一对一联系, 263

遗传算法, 387

异构数组, 211, 278

溢出, 37

因特网, 6, 124

因特网接入服务提供商, 126

阴影, 358

引导, 102

引导程序, 103

隐藏面消除, 355

隐藏终端问题, 120

应用层, 139

应用软件, 99

硬件, 2

拥塞控制, 142

用户界面, 100

用户文档, 268

用户自定义数据类型, 298

用例, 262

用例图, 261

有损, 48

有向图, 109, 376

右子指针, 291

余码记数法, 37

语法分析, 373

语法分析器, 224

语法分析树, 226

语法图, 225

语义分析, 373

语义网, 374

域, 127

域名, 127

域名服务器, 128

域名系统查找, 128

阈值, 389

元推理, 385

元组, 315

原型开发, 254

原语, 162

源, 134

源程序, 224

云计算, 124

运动捕捉, 363

运动学, 363

运算符优先级, 213

Z

暂时模糊, 51

增量模型, 253

栈, 279

栈底, 279

栈顶, 279

折射, 353

着色, 356

帧, 361

帧缓冲区, 345

真实世界的知识, 384

整型, 209

证书, 150

只读存储器, 103

知行学, 270

知识产权, 272

直接寻址, 303

指令寄存器, 68

指令指针, 283

指数域, 39

指针, 282

智能手机, 98

智能体, 368

中断, 105

中断处理程序, 105

中继器, 121

中间存在, 362

终端节点, 280

终端系统, 126

终结符, 225

终止条件, 174

重载, 213

逐步求精, 170

主板, 61

主存储器, 20

主机, 126

主元, 177

注册商, 127

注释, 208, 216

柱面, 22

专家系统, 378

专用, 118

专用寄存器, 61

转发, 142

转发表, 122

状态, 376

状态字, 78

资源, 106

子 (, 280

子句形式, 238

子模式, 313

子树, 281

子域, 128

自底向上方法, 170

自顶向下方法, 170

自然语言, 203

自适应字典编码, 49

自引用, 412

自由格式语言, 225

自终止的, 412

字处理程序, 27

字典, 49

字典编码, 49

字段, 211

字节, 20

字面量, 211

总线, 61

走样, 354

最低有效位, 20

最高有效位, 20

最佳优先, 380

左子指针, 291

作业, 96

作业队列, 96

作用域, 219

教师支持申请表



尊敬的老师：

您好！

为了确保您及时有效地申请培生整体教学资源，请您务必完整填写如下表格，加盖学院的公章后传真给我们，我们将会2~3个工作日内为您处理。

请填写所需教辅的开课信息：

采用教材			☐ 中文版 ☐ 英文版 ☐ 双语版
作 者		出版社	
版 次		ISBN	
课程时间	始于 年 月 日	学生人数	

年 月 日

资源介绍

- 教材、常规教辅（PPT、教师手册、题库等）资源，请访问
www.pearsonhighered.com/educator。（免费）
- MyLabs/Mastering系列在线平台，适合老师和学生共同使用，访问需要Access Code。（付费）

地址：100013 北京市东城区北三环东路36号环球贸易中心D座1208室100013

邮件请发送到：copub.hed@pearson.com

网站：www.pearson.com